Windows Hardware Certification Requirements

Device

December 2011

This document is provided "as-is". Information and views expressed in this document, including URL and other Internet Web site references, may change without notice.

This document does not provide you with any legal rights to any intellectual property in any Microsoft product. You may copy and use this document for your internal, reference purposes. © 2011 Microsoft. All rights reserved.

Microsoft, Windows and Windows Server are trademarks of the Microsoft group of companies. UPnP™ is a certification mark of the UPnP™ Implementers Corp. All other trademarks are property of their respective owners.

Microsoft Corporation Technical Documentation License Agreement

READ THIS! THIS IS A LEGAL AGREEMENT BETWEEN MICROSOFT CORPORATION ("MICROSOFT") AND THE RECIPIENT OF THESE MATERIALS, WHETHER AN INDIVIDUAL OR AN ENTITY ("YOU"). IF YOU HAVE ACCESSED THIS AGREEMENT IN THE PROCESS OF DOWNLOADING MATERIALS ("MATERIALS") FROM A MICROSOFT WEB SITE, BY CLICKING "I ACCEPT", DOWNLOADING, USING OR PROVIDING FEEDBACK ON THE MATERIALS, YOU AGREE TO THESE TERMS. IF THIS AGREEMENT IS ATTACHED TO MATERIALS, BY ACCESSING, USING OR PROVIDING FEEDBACK ON THE ATTACHED MATERIALS, YOU AGREE TO THESE TERMS.

For good and valuable consideration, the receipt and sufficiency of which are acknowledged, You and Microsoft agree as follows:

1. You may review these Materials only (a) as a reference to assist You in planning and designing Your product, service or technology ("Product") to interface with a Microsoft Product as described in these Materials; and (b) to provide feedback on these Materials to Microsoft. All other rights are retained by Microsoft; this agreement does not give You rights under any Microsoft patents. You may not (i) remove this agreement or any notices from these Materials, or (ii) give any part of these Materials, or assign or otherwise provide Your rights under this agreement, to anyone else.

2. These Materials may contain preliminary information or inaccuracies, and may not correctly represent any associated Microsoft Product as commercially released. All Materials are provided entirely "AS IS." To the extent permitted by law, MICROSOFT MAKES NO WARRANTY OF ANY KIND, DISCLAIMS ALL EXPRESS, IMPLIED AND STATUTORY WARRANTIES, AND ASSUMES NO LIABILITY TO YOU FOR ANY DAMAGES OF ANY TYPE IN CONNECTION WITH THESE MATERIALS OR ANY INTELLECTUAL PROPERTY IN THEM.

3. If You are an entity and (a) merge into another entity or (b) a controlling ownership interest in You changes, Your right to use these Materials automatically terminates and You must destroy them.

4. You have no obligation to give Microsoft any suggestions, comments or other feedback ("Feedback") relating to these Materials. However, any Feedback you voluntarily provide may be used in Microsoft Products and related specifications or other documentation (collectively, "Microsoft Offerings") which in turn may be relied upon by other third parties to develop their own Products. Accordingly, if You do give Microsoft Feedback on any version of these Materials or the Microsoft Offerings to which they apply, You agree: (a) Microsoft may freely use, reproduce, license, distribute, and otherwise commercialize Your Feedback in any Microsoft Offering; (b) You also grant third parties, without charge, only those patent rights necessary to enable other Products to use or interface with any specific parts of a Microsoft Product that incorporate Your Feedback; and (c) You will not give Microsoft any Feedback (i) that You have reason to believe is subject to any patent, copyright or other intellectual property claim or right of any third party; or (ii) subject to license terms which seek to require any Microsoft Offering incorporating or derived from such Feedback, or other Microsoft intellectual property, to be licensed to or otherwise shared with any third party.

5. Microsoft has no obligation to maintain confidentiality of any Microsoft Offering, but otherwise the confidentiality of Your Feedback, including Your identity as the source of such Feedback, is governed by Your NDA.

6. This agreement is governed by the laws of the State of Washington. Any dispute involving it must be brought in the federal or state superior courts located in King County, Washington, and You waive any defenses allowing the dispute to be litigated elsewhere. If there is litigation, the losing party must pay the other party’s reasonable attorneys’ fees, costs and other expenses. If any part of this agreement is unenforceable, it will be considered modified to the extent necessary to make it enforceable, and the remainder shall continue in effect. This agreement is the entire agreement between You and Microsoft concerning these Materials; it may be changed only by a written document signed by both You and Microsoft.

Introduction 33
Features 33
Device.Audio.APO 33
Device.Audio.APO.MicArrayRawData 33
Device.Audio.APO.WinPEConformance 34
Device.Audio.AudioController 35
Device.Audio.AudioController.HDAudioVersionNumber 35
Device.Audio.AudioController.HDControllerCompliance 36
Device.Audio.Base 37
Device.Audio.Base.AudioDriversSupportMute 38
Device.Audio.Base.BasicDataFormats 39
Device.Audio.Base.ChannelMasks 40
Device.Audio.Base.CopyBitPolarityClarification 41
Device.Audio.Base.DCOffset 42
Device.Audio.Base.DevicesWorkWithoutExtraSoftware 42
Device.Audio.Base.DigitalStreamsNoMixing 44
Device.Audio.Base.DockingStation 46
Device.Audio.Base.DRM 47
Device.Audio.Base.EfficientBufferManagement 47
Device.Audio.Base.ExposedAudioEndpointsAreFunctional 48
Device.Audio.Base.Fidelity 49
Device.Audio.Base.FloatSupport 52
Device.Audio.Base.FullDuplexOperation 53
Device.Audio.Base.HDAudioRemoveDevicePowerState 54
Device.Audio.Base.IMiniportWaveRTStreamNotification 54
Device.Audio.Base.IndependentInputOutputFormatSelection 56
Device.Audio.Base.InitiatorTargetBlocktransferSupport 57
Device.Audio.Base.JackConnectorStateDescription 58
Device.Audio.Base.JackDetection 59
Device.Audio.Base.KSPROPERTYAUDIOMIXLEVELTABLE 59
Device.Audio.Base.KSPROPERTYAUDIOVOLUMELEVEL 60
Device.Audio.Base.KSTopologyCompliance 61
Device.Audio.Base.NoHiddenStreamRouting 63

Device.Audio.Base.NoUncontrollableStreamRouting 64
Device.Audio.Base.NoUndiscoverableDevice 65
Device.Audio.Base.PCMNonPCMForSPDIF 66
Device.Audio.Base.PowerManagement 67
Device.Audio.Base.ProperUSBDescriptors 68
Device.Audio.Base.RealtimeDriversSupportStandardLoopedStreaming 68
Device.Audio.Base.RecordPlaybackBasicPerformance 69
Device.Audio.Base.ReportSupportedProperties 72
Device.Audio.Base.RestartWithinASpecifiedDuration 73
Device.Audio.Base.SamplePositionAccuracy 74
Device.Audio.Base.SamplingAccuracy 75
Device.Audio.Base.SPDIFSupportMinimumSamplingRate 76
Device.Audio.Base.TimeSynchronizedSampleRates 77
Device.Audio.Base.TipRing 78
Device.Audio.Base.TwoDMAEnginesAndConnections 79
Device.Audio.Base.VoiceCommunicationUAA 80
Device.Audio.Base.VolumeControlsIsLinear 83
Device.Audio.Base.VolumeGranularity 84
Device.Audio.Base.WAVEFORMATEXTENSIBLESupport 85
Device.Audio.Base.WaveRTConformance 85
Device.Audio.Base.WaveRTImplementation 86
Device.Audio.Base.ZeroGlitch 87
Device.Audio.Bluetooth 88
Device.Audio.Bluetooth.AtleastOneProfileSupport 88
Device.Audio.Bluetooth.AutomaticReconnectAttempt 89
Device.Audio.Bluetooth.ConnectDisconnectBluetooth 90
Device.Audio.Bluetooth.DriverReqs 91
Device.Audio.Bluetooth.HandsFreeCallControl 92
Device.Audio.Bluetooth.HCIDisconnect 93
Device.Audio.Bluetooth.MajorMinorClassID 93
Device.Audio.HardwareAudioProcessing 94
Device.Audio.HardwareAudioProcessing.AudioHardwareOffloading 95
Device.Audio.HardwareAudioProcessing.ETWEvent 101
Device.Audio.HardwareAudioProcessing.IMiniport 103

Device.Audio.HDAudio 103
Device.Audio.HDAudio.2AudioChannelsForHDMIorDisplayPort 104
Device.Audio.HDAudio.AnalogJackDetection 105
Device.Audio.HDAudio.DefaultAssociationNotZero 106
Device.Audio.HDAudio.DigitalJackDetection 107
Device.Audio.HDAudio.HDAudioCodecAdditionalReqs 108
Device.Audio.HDAudio.HDAudioSpecCompliance 110
Device.Audio.HDAudio.HDMIDCN 110
Device.Audio.HDAudio.HDMIKSPROPERTYJACKSINKINFO 112
Device.Audio.HDAudio.INFHasDeviceID 113
Device.Audio.HDAudio.LowPowerDCN 113
Device.Audio.HDAudio.OneCodecPortOneConnector 114
Device.Audio.HDAudio.PinConfigPortConnectivity 116
Device.Audio.HDAudio.PnPCodecDeviceID 117
Device.Audio.HDAudio.UniqueSequenceNumbers 118
Device.Audio.UAACompliance 119
Device.Audio.UAACompliance.TestUsingBluetoothClassDriver 119
Device.Audio.UAACompliance.UAA 120
Device.Audio.USB 121
Device.Audio.USB.HIDCommunications 121
Device.Audio.USB.HIDControls 122
Device.Audio.USB.MicArray 123
Device.Audio.USB.USB 124
Device.BusController.Bluetooth.Base 125
Device.BusController.Bluetooth.Base.4LeSpecification 125
Device.BusController.Bluetooth.Base.LeStateCombinations 126
Device.BusController.Bluetooth.Base.LeWhiteList 126
Device.BusController.Bluetooth.Base.MicrosoftBluetoothStack 127
Device.BusController.Bluetooth.Base.OnOffStateControllableViaSoftware 128
Device.BusController.Bluetooth.Base.Scatternet 128
Device.BusController.Bluetooth.Base.ScoDataTransportLayer 129
Device.BusController.Bluetooth.Base.SimultaneousBrEdrAndLeTraffic 130
Device.BusController.Bluetooth.Base.SpecificInformationParameters 130
Device.BusController.Bluetooth.Base.SupportsBluetooth21AndEdr 131

Device.BusController.Bluetooth.NonUSB 132
Device.BusController.Bluetooth.NonUSB.Performance 132
Device.BusController.Bluetooth.NonUSB.ScoSupport 132
Device.BusController.NearFieldProximity 133
Device.BusController.NearFieldProximity.NFCCertification 133
Device.BusController.NearFieldProximity.ProviderImplementation 134
Device.BusController.NearFieldProximity.ProximityReliability 135
Device.BusController.NearFieldProximity.RangeOfActuation 136
Device.BusController.NearFieldProximity.SessionEstablishmentPerformance 137
Device.BusController.NearFieldProximity.TaptoSetupScenario 138
Device.BusController.NearFieldProximity.TapToUseScenarios 138
Device.BusController.SdioController 139
Device.BusController.SdioController.ComplyWithIndustrySpec 139
Device.BusController.SdioController.WdfKmdfDriver 140
Device.BusController.UsbController 141
Device.BusController.UsbController.ImplementAtLeastOneXhciSpcStructForUSB2 141
Device.BusController.UsbController.MaintainDeviceStateOnResumeS1andS3 142
Device.BusController.UsbController.MustResumeWithoutForcedReset 144
Device.BusController.UsbController.PreserveDeviceStateAfterDisableEnable 144
Device.BusController.UsbController.SpecificationCompliance 145
Device.BusController.UsbController.SuperSpeedConnectorsSupportHighFullLow 146
Device.BusController.UsbController.SupportSelectiveSuspend 147
Device.BusController.UsbController.TestedUsingMicrosoftUsbStack 148
Device.BusController.UsbController.UsbifCertification 149
Device.BusController.UsbController.XhciAc64Bit 149
Device.BusController.UsbController.XhciAddInCardsMapPortsConsistently 151
Device.BusController.UsbController.XhciAddInCardsReportInternalDevices 153
Device.BusController.UsbController.XhciSupportDebuggingOnAllExposedPorts 154
Device.BusController.UsbController.XhciSupportMsiMsixInterrupts 155
Device.BusController.UsbController.XhciSupportsMinimum31Streams 156
Device.BusController.UsbController.XhciVersionCompliant 156
Device.Connectivity.BluetoothDevices 157
Device.Connectivity.BluetoothDevices.BluetoothDeviceIdProfileVer12 158
Device.Connectivity.BluetoothDevices.BluetoothDeviceIdProfileVer13 158

Device.Connectivity.BluetoothDevices.BluetoothHidLimitedDiscoverableMode 159
Device.Connectivity.BluetoothDevices.BluetoothUSBPlugandPlay 159
Device.Connectivity.BluetoothDevices.ComplementarySubsystemList 160
Device.Connectivity.BluetoothDevices.FunctionAfterSystemSuspendCycle 161
Device.Connectivity.BluetoothDevices.HidInitiatedReconnect 161
Device.Connectivity.BluetoothDevices.KeyboardsSupportPasskeyAuthentication 162
Device.Connectivity.BluetoothDevices.RespondToServiceDiscoveryRequests 163
Device.Connectivity.BluetoothDevices.SupportBluetooth21 164
Device.Connectivity.NearFieldProximity 164
Device.Connectivity.NearFieldProximity.DeviceNFCCertification 164
Device.Connectivity.NearFieldProximity.DeviceRangeOfActuation 165
Device.Connectivity.NearFieldProximity.DeviceTapToSetup 166
Device.Connectivity.NearFieldProximity.NfcForumTag 167
Device.Connectivity.NearFieldProximity.TouchMark 167
Device.Connectivity.Network.PnPX 168
Device.Connectivity.Network.PnPX.PnPX 168
Device.Connectivity.Network.VerticalPairing 170
Device.Connectivity.Network.VerticalPairing.VerticalPairing 170
Device.Connectivity.Network.VerticalPairing.WCN 171
Device.Connectivity.PciConnected 172
Device.Connectivity.PciConnected.64BitPrefetchableBar 173
Device.Connectivity.PciConnected.ConfigurationSpaceCorrectlyPopulated 174
Device.Connectivity.PciConnected.ExpressCardImplementsSerialNumber 175
Device.Connectivity.PciConnected.InterruptDisableBit 176
Device.Connectivity.PciConnected.MsiOrMsixSupport 176
Device.Connectivity.PciConnected.PciAndPcixDevicesArePciCompliant 178
Device.Connectivity.PciConnected.PCIExpress 178
Device.Connectivity.PciConnected.SubsystemIdsRequired 180
Device.Connectivity.UsbDevices 181
Device.Connectivity.UsbDevices.Addressing 182
Device.Connectivity.UsbDevices.AlternateDriver 183
Device.Connectivity.UsbDevices.CompliesWithChap9 184
Device.Connectivity.UsbDevices.DebugCompliesWithDebugSpec 185
Device.Connectivity.UsbDevices.DebugCompliesWithDebugSpecUSB3 185

Device.Connectivity.UsbDevices.DeviceAttachLessThan100ms 186
Device.Connectivity.UsbDevices.EsdRecovery 187
Device.Connectivity.UsbDevices.FunctionSuspendSelectiveSuspend 188
Device.Connectivity.UsbDevices.InstallViaUniquePnpIdentifier 189
Device.Connectivity.UsbDevices.IsochronousDeviceAndDriver 190
Device.Connectivity.UsbDevices.MsOsContainerId 191
Device.Connectivity.UsbDevices.MustBeFunctionalAfterResume 192
Device.Connectivity.UsbDevices.MustEnumerateOnEhciAndXhci 193
Device.Connectivity.UsbDevices.MustNotDisconnectDuringSuspend 194
Device.Connectivity.UsbDevices.MustResumeWithoutForcedReset 195
Device.Connectivity.UsbDevices.MustSignalAttachWithin500ms 196
Device.Connectivity.UsbDevices.MustSupportSuspend 197
Device.Connectivity.UsbDevices.PeripheralOperatesInFunctionMode 198
Device.Connectivity.UsbDevices.PortMove500ms 199
Device.Connectivity.UsbDevices.RespondAllStringRequests 200
Device.Connectivity.UsbDevices.ResponsesLimitedByWlengthField 201
Device.Connectivity.UsbDevices.SerialNumbers 202
Device.Connectivity.UsbDevices.SerialNumbersUseValidCharacters 203
Device.Connectivity.UsbDevices.SuperSpeedOnConnectViaUsb3Port 204
Device.Connectivity.UsbDevices.TestedUsingMicrosoftUsbStack 205
Device.Connectivity.UsbDevices.Usb3CompatibleWithDownLevel 206
Device.Connectivity.UsbDevices.UsbifCertification 207
Device.Connectivity.UsbDevices.UseUsbClassOnlyForControllerOrHub 208
Device.Connectivity.UsbDevices.WirelessUsbObtainsWusbLogoFromUsbif 209
Device.Connectivity.UsbDevices.WirelessUsbWiMediaAlliace 210
Device.Connectivity.UsbHub 211
Device.Connectivity.UsbHub.CompliesWithChap11 211
Device.Connectivity.UsbHub.IdentifyNumOfUserAccessiblePorts 212
Device.Connectivity.UsbHub.ImplementSuperSpeedDescriptors 213
Device.Connectivity.UsbHub.MapPortsPerUsb3Specification 215
Device.Connectivity.UsbHub.ProvideStandardInterfacesToHostPeripherals 216
Device.Connectivity.UsbHub.SuperSpeedRemainsOnAfterPortReset 217
Device.Connectivity.UsbHub.SupportSuspend 218
Device.Connectivity.UsbHub.Usb3HubCompliesWithUsb3Spec 219

Device.Connectivity.UsbHub.Usb3ReportPortStatusBitsCorrectly 220
Device.Connectivity.WSD 220
Device.Connectivity.WSD.DPWS 221
Device.Connectivity.WSD.DPWSExtensibility 222
Device.Connectivity.WSD.MetadataExchange 222
Device.Connectivity.WSD.MetadataValid 223
Device.Connectivity.WSD.Schema 224
Device.Connectivity.WSD.WSDiscovery 225
Device.DevFund.CDA 226
Device.DevFund.CDA.Application 226
Device.DevFund.Color 227
Device.DevFund.Color.DeviceColorProfilesInstall 227
Device.DevFund.DriverFramework.AllDrivers 229
Device.DevFund.DriverFramework.AllDrivers.WDFLoadGroup 229
Device.DevFund.DriverFramework.KMDF 230
Device.DevFund.DriverFramework.KMDF.HandleDDIFailures 230
Device.DevFund.DriverFramework.KMDF.Reliability 231
Device.DevFund.DriverFramework.KMDF.WDFProperINF 233
Device.DevFund.DriverFramework.KMDF.WDFRedistributables 237
Device.DevFund.DriverFramework.UMDF 238
Device.DevFund.DriverFramework.UMDF.Reliability 239
Device.DevFund.DriverFramework.UMDF.WDFProperINF 240
Device.DevFund.DriverFramework.UMDF.WDFRedistributables 244
Device.DevFund.INF 246
Device.DevFund.INF.AddReg 247
Device.DevFund.INF.AddService 247
Device.DevFund.INF.ClassInstall32 248
Device.DevFund.INF.ComplexDeviceMatching 249
Device.DevFund.INF.DDInstall.CoInstallers 250
Device.DevFund.INF.DeviceConfigOnly 252
Device.DevFund.INF.DeviceResourceConfig 253
Device.DevFund.INF.FileCopyRestriction 254
Device.DevFund.INF.FileOrRegistryModification 255
Device.DevFund.INF.InstallManagement 256

Device.DevFund.INF.LegacySyntax 257
Device.DevFund.INF.TargetOSVersion 258
Device.DevFund.Memory 259
Device.DevFund.Memory.DriverFootprint 259
Device.DevFund.Memory.NXPool 260
Device.DevFund.Reliability 260
Device.DevFund.Reliability.BasicReliabilityAndPerformance 261
Device.DevFund.Reliability.BasicSecurity 263
Device.DevFund.Reliability.BootDriverEmbeddedSignature 264
Device.DevFund.Reliability.DriverInstallUninstallReinstall 266
Device.DevFund.Reliability.DriverUninstallInstallOtherDeviceStability 267
Device.DevFund.Reliability.IOCompletionCancellation 268
Device.DevFund.Reliability.NoReplacingSysComponents 270
Device.DevFund.Reliability.NormalOpWithDEP 271
Device.DevFund.Reliability.PnPIDs 272
Device.DevFund.Reliability.PnPIRPs 274
Device.DevFund.Reliability.ProperINF 276
Device.DevFund.Reliability.RemoteDesktopServices 277
Device.DevFund.Reliability.S3S4SleepStates 278
Device.DevFund.Reliability.Signable 279
Device.DevFund.Reliability.SWDeviceInstallsUsePnPAPIs 280
Device.DevFund.Reliability.X64Support 282
Device.DevFund.Reliability.3rdParty 284
Device.DevFund.Reliability.3rdParty.FormerTests 284
Device.DevFund.Reliability.Interrupts 285
Device.DevFund.Reliability.Interrupts.BasicReliabilityAndPerformance 285
Device.DevFund.Server 286
Device.DevFund.Server.CommandLineConfigurable 286
Device.DevFund.Server.MultipleProcessorGroups 288
Device.DevFund.Server.ServerPowerManagement 289
Device.DevFund.Server.PCI 291
Device.DevFund.Server.PCI.PCIAER 291
Device.DevFund.Server.StaticTools 292
Device.DevFund.Server.StaticTools.SDVandPFD 292

Device.Digitizer.Base 293
Device.Digitizer.Base.DigitizersAppearAsHID 293
Device.Digitizer.Base.HighQualityDigitizerInput 294
Device.Digitizer.Base.HighQualityTouchDigitizerInput 295
Device.Digitizer.Pen 297
Device.Digitizer.Pen.100HzSampleRate 297
Device.Digitizer.Pen.ContactAccuracy 298
Device.Digitizer.Pen.HoverAccuracy 299
Device.Digitizer.Pen.PenRange 299
Device.Digitizer.Pen.PenResolution 300
Device.Digitizer.Touch 301
Device.Digitizer.Touch.5TouchPointMinimum 301
Device.Digitizer.Touch.Bezel 302
Device.Digitizer.Touch.DigitizerConnectsOverUSBOrI2C 303
Device.Digitizer.Touch.DigitizerJitter 304
Device.Digitizer.Touch.ExtraInputBehavior 304
Device.Digitizer.Touch.FieldFirmwareUpdatable 305
Device.Digitizer.Touch.HIDCompliantFirmware 306
Device.Digitizer.Touch.HighResolutionTimeStamp 306
Device.Digitizer.Touch.InputSeparation 307
Device.Digitizer.Touch.NoiseSuppression 308
Device.Digitizer.Touch.PhysicalDimension 309
Device.Digitizer.Touch.PhysicalInputPosition 310
Device.Digitizer.Touch.PowerStates 310
Device.Digitizer.Touch.ReportingRate 311
Device.Digitizer.Touch.ResponseLatency 312
Device.Digitizer.Touch.TouchResolution 313
Device.Digitizer.Touch.ZAxisAllowance 313
Device.Display.Monitor 314
Device.Display.Monitor.Base 314
Device.Display.Monitor.ColorimetricTolerance 315
Device.Display.Monitor.DigitalLinkProtection 317
Device.Display.Monitor.EDID 318
Device.Display.Monitor.Modes 320

Device.Display.Monitor.Stereoscopic3DModes 322
Device.Graphics.AdapterBase 323
Device.Graphics.AdapterBase.ApplicationVerifier 323
Device.Graphics.AdapterBase.DriverVersion 325
Device.Graphics.AdapterBase.PowerManagementCompliance 326
Device.Graphics.AdapterBase.RegistryEntries 327
Device.Graphics.AdapterBase.SubsystemResettable 329
Device.Graphics.AdapterRender 330
Device.Graphics.AdapterRender.MinimumDirectXLevel 330
Device.Graphics.AdapterRender.RGBFrameBuffer 331
Device.Graphics.AdapterRender.YUVSupport 332
Device.Graphics.AdapterRender.D3D101Core 333
Device.Graphics.AdapterRender.D3D101Core.D3D101CorePrimary 333
Device.Graphics.AdapterRender.D3D101WDDM11 334
Device.Graphics.AdapterRender.D3D101WDDM11.D3D101v11Primary 334
Device.Graphics.AdapterRender.D3D101WDDM12 336
Device.Graphics.AdapterRender.D3D101WDDM12.D3D101v12Primary 336
Device.Graphics.AdapterRender.D3D10ComputeShader 337
Device.Graphics.AdapterRender.D3D10ComputeShader.D3D10CoreC 337
Device.Graphics.AdapterRender.D3D10Core 338
Device.Graphics.AdapterRender.D3D10Core.D3D10CorePrimary 338
Device.Graphics.AdapterRender.D3D10D3D11LogicOps 340
Device.Graphics.AdapterRender.D3D10D3D11LogicOps.D3D10CoreD 340
Device.Graphics.AdapterRender.D3D10Multisampling4X 341
Device.Graphics.AdapterRender.D3D10Multisampling4X.D3D10CoreA 341
Device.Graphics.AdapterRender.D3D10Multisampling8X 342
Device.Graphics.AdapterRender.D3D10Multisampling8X.D3D10CoreB 342
Device.Graphics.AdapterRender.D3D10WDDM11 343
Device.Graphics.AdapterRender.D3D10WDDM11.D3D10v11Primary 343
Device.Graphics.AdapterRender.D3D10WDDM12 345
Device.Graphics.AdapterRender.D3D10WDDM12.D3D10v12Primary 345
Device.Graphics.AdapterRender.D3D111Core 347
Device.Graphics.AdapterRender.D3D111Core.D3D111CorePrimary 347
Device.Graphics.AdapterRender.D3D11Core 348

Device.Graphics.AdapterRender.D3D11Core.D3D11CorePrimary 348
Device.Graphics.AdapterRender.D3D11DoublePrecisionShader 349
Device.Graphics.AdapterRender.D3D11DoublePrecisionShader.D3D11CoreC 350
Device.Graphics.AdapterRender.D3D11DriverCommandLists 350
Device.Graphics.AdapterRender.D3D11DriverCommandLists.D3D11CoreB 351
Device.Graphics.AdapterRender.D3D11DriverConcurrentObjectCreation 352
Device.Graphics.AdapterRender.D3D11DriverConcurrentObjectCreation.D3D11CoreA 352
Device.Graphics.AdapterRender.D3D11Level9WDDM12 353
Device.Graphics.AdapterRender.D3D11Level9WDDM12.D3D9UMDDIUpdate 353
Device.Graphics.AdapterRender.D3D11PartialPrecision 354
Device.Graphics.AdapterRender.D3D11PartialPrecision.D3D11CoreE 354
Device.Graphics.AdapterRender.D3D11WDDM12 355
Device.Graphics.AdapterRender.D3D11WDDM12.D3D11v12Primary 355
Device.Graphics.AdapterRender.D3D11WDDM12DoublePrecisionShader 356
Device.Graphics.AdapterRender.D3D11WDDM12DoublePrecisionShader.D3D11v12C 356
Device.Graphics.WDDM 357
Device.Graphics.WDDM.Base 358
Device.Graphics.WDDM.Checklist 359
Device.Graphics.WDDM.GPUFenceCommands 361
Device.Graphics.WDDM.Display 362
Device.Graphics.WDDM.Display.Base 362
Device.Graphics.WDDM.Display.GammaCorrection 363
Device.Graphics.WDDM.Display.HotPlugDetection 364
Device.Graphics.WDDM.Display.I2CSupport 366
Device.Graphics.WDDM.Display.MediaCenterResolutionTiming 367
Device.Graphics.WDDM.Display.Multimon 368
Device.Graphics.WDDM.Display.ResetToVGA 370
Device.Graphics.WDDM.Display.HDMIorDPDCNs 371
Device.Graphics.WDDM.Display.HDMIorDPDCNs.DCNCompliance 371
Device.Graphics.WDDM.Display.TVOut 373
Device.Graphics.WDDM.Display.TVOut.Base 374
Device.Graphics.WDDM.Display.TVOut.DAC 375
Device.Graphics.WDDM.Display.TVOut.Encoder 375
Device.Graphics.WDDM.DisplayRender 376

Device.Graphics.WDDM.DisplayRender.Base 376
Device.Graphics.WDDM.DisplayRender.OutputProtection 377
Device.Graphics.WDDM.DisplayRender.Stability 379
Device.Graphics.WDDM.DisplayRender.OutputProtection 380
Device.Graphics.WDDM.DisplayRender.OutputProtection.Windows7 380
Device.Graphics.WDDM.Render 382
Device.Graphics.WDDM.Render.Base 382
Device.Graphics.WDDM.Render.VideoDecoding 383
Device.Graphics.WDDM.Render.VideoProcessing 385
Device.Graphics.WDDM.Render.Windows7.VideoDecoding 387
Device.Graphics.WDDM11 388
Device.Graphics.WDDM11.Base 388
Device.Graphics.WDDM11.Display 389
Device.Graphics.WDDM11.Display.Base 390
Device.Graphics.WDDM11.DisplayRender 390
Device.Graphics.WDDM11.DisplayRender.Base 391
Device.Graphics.WDDM11.DisplayRender.D3D9Overlay 391
Device.Graphics.WDDM11.DisplayRender.D3D9Overlay.D3D9Overlay 392
Device.Graphics.WDDM11.Render 393
Device.Graphics.WDDM11.Render.Base 393
Device.Graphics.WDDM11.Render.ContentProtection 393
Device.Graphics.WDDM11.Render.ContentProtection.ContentProtection 394
Device.Graphics.WDDM11.Render.DXVAHD 395
Device.Graphics.WDDM11.Render.DXVAHD.DXVAHD 395
Device.Graphics.WDDM12 396
Device.Graphics.WDDM12.Base 396
Device.Graphics.WDDM12.Display 400
Device.Graphics.WDDM12.Display.Base 400
Device.Graphics.WDDM12.Display.ContainerIDSupport 403
Device.Graphics.WDDM12.Display.DisplayOutputControl 404
Device.Graphics.WDDM12.Display.ModeEnumeration 406
Device.Graphics.WDDM12.Display.PnpStopStartSupport 409
Device.Graphics.WDDM12.Display.ProvideLinearFrameBuffer 410
Device.Graphics.WDDM12.DisplayOnly 411

Device.Graphics.WDDM12.DisplayOnly.Base 412
Device.Graphics.WDDM12.DisplayRender 415
Device.Graphics.WDDM12.DisplayRender.Base 415
Device.Graphics.WDDM12.DisplayRender.ProcessingStereoscopicVideoContent 419
Device.Graphics.WDDM12.DisplayRender.ProcessingStereoscopicVideoContent.ProcessingStereoscopicVideoContent 420
Device.Graphics.WDDM12.DisplayRender.RuntimePowerMgmt 421
Device.Graphics.WDDM12.DisplayRender.RuntimePowerMgmt.RuntimePowerMgmt 421
Device.Graphics.WDDM12.Render 422
Device.Graphics.WDDM12.Render.Base 422
Device.Graphics.WDDM12.Render.D3D11VideoDecoding 425
Device.Graphics.WDDM12.Render.D3D11VideoProcessing 427
Device.Graphics.WDDM12.Render.DirectFlip 430
Device.Graphics.WDDM12.Render.FlipOnVSyncMmIo 431
Device.Graphics.WDDM12.Render.OfferReclaim 432
Device.Graphics.WDDM12.Render.PreemptionGranularity 434
Device.Graphics.WDDM12.Render.Stereoscopic3DArraySupport 435
Device.Graphics.WDDM12.Render.TDRResiliency 437
Device.Graphics.WDDM12.Render.UMDLogging 439
Device.Graphics.WDDM12.Render.XPSRasterizationConformance 441
Device.Graphics.WDDM12.RenderOnly 441
Device.Graphics.WDDM12.RenderOnly.Base 442
Device.Graphics.WDDM12.StandbyHibernateFlags 445
Device.Graphics.WDDM12.StandbyHibernateFlags.StandbyHibernateFlags 445
Device.Graphics.XDDM 446
Device.Graphics.XDDM.Stability 446
Device.Imaging.Printer.Base 447
Device.Imaging.Printer.Base.applicationVerifier 448
Device.Imaging.Printer.Base.autoConfiguration 449
Device.Imaging.Printer.Base.configurationFiles 450
Device.Imaging.Printer.Base.connectionRecovery 451
Device.Imaging.Printer.Base.connectivityRobustness 452
Device.Imaging.Printer.Base.deviceCapabilities 453
Device.Imaging.Printer.Base.DocumentProperties 454

Device.Imaging.Printer.Base.driverCategory 455
Device.Imaging.Printer.Base.DriverEventFiles 456
Device.Imaging.Printer.Base.driverIsolation 456
Device.Imaging.Printer.Base.driverPackage 457
Device.Imaging.Printer.Base.driverStability 458
Device.Imaging.Printer.Base.faxModem 459
Device.Imaging.Printer.Base.faxTIA592 459
Device.Imaging.Printer.Base.faxV34 460
Device.Imaging.Printer.Base.GDLFile 461
Device.Imaging.Printer.Base.infFile 461
Device.Imaging.Printer.Base.jobCancellation 463
Device.Imaging.Printer.Base.JSBidiExtender 464
Device.Imaging.Printer.Base.metadata 464
Device.Imaging.Printer.Base.portMonitors 465
Device.Imaging.Printer.Base.PrinterExtension 466
Device.Imaging.Printer.Base.printerInterfaces 467
Device.Imaging.Printer.Base.PrinterUIApp 467
Device.Imaging.Printer.Base.printProcessor 468
Device.Imaging.Printer.Base.printRegions 469
Device.Imaging.Printer.Base.printTicket 470
Device.Imaging.Printer.Base.rendering 472
Device.Imaging.Printer.Base.TCPMon 473
Device.Imaging.Printer.Base.XPSDrv 474
Device.Imaging.Printer.Cluster 475
Device.Imaging.Printer.Cluster.cluster 475
Device.Imaging.Printer.OXPS 477
Device.Imaging.Printer.OXPS.OXPS 477
Device.Imaging.Printer.TCPIP 478
Device.Imaging.Printer.TCPIP.CompatID 478
Device.Imaging.Printer.USB 479
Device.Imaging.Printer.USB.CompatID 479
Device.Imaging.Printer.USB.JSBidiExtender 480
Device.Imaging.Printer.WSD 481
Device.Imaging.Printer.WSD.CompatID 481

Device.Imaging.Printer.WSD.Rally 482
Device.Imaging.Printer.WSD.WSPrint 483
Device.Imaging.Printer.XPS 484
Device.Imaging.Printer.XPS.XPS 485
Device.Imaging.Scanner.Base 487
Device.Imaging.Scanner.Base.dataTransfer 487
Device.Imaging.Scanner.Base.driverInstallation 490
Device.Imaging.Scanner.Base.errorHandling 490
Device.Imaging.Scanner.Base.metadata 493
Device.Imaging.Scanner.Base.MFPmultiplePorts 493
Device.Imaging.Scanner.Base.RawFileFormat 494
Device.Imaging.Scanner.Base.scannerInterfaces 495
Device.Imaging.Scanner.Base.statusMessages 496
Device.Imaging.Scanner.Base.wia20 497
Device.Imaging.Scanner.Base.WIAArchitecture 498
Device.Imaging.Scanner.Base.WIAProperties 499
Device.Imaging.Scanner.DistributedScanManagement 499
Device.Imaging.Scanner.DistributedScanManagement.DistributedScanManagement 500
Device.Imaging.Scanner.WSD 500
Device.Imaging.Scanner.WSD.Rally 500
Device.Imaging.Scanner.WSD.WSScan 501
Device.Input.FingerPrintReader 502
Device.Input.FingerPrintReader.Base 502
Device.Input.FingerPrintReader.Extensions 504
Device.Input.FingerPrintReader.ManagementApps 505
Device.Input.FingerPrintReader.SensorEngineDB 506
Device.Input.FingerPrintReader.WBDI 508
Device.Input.GameController.CommonController 510
Device.Input.GameController.CommonController.XInput 511
Device.Input.GameController.GenericController 512
Device.Input.GameController.GenericController.DirectInput 512
Device.Input.HID 512
Device.Input.HID.I2CDeviceUniqueHWID 513
Device.Input.HID.I2CProtocolSpecCompliant 513

Device.Input.HID.MCEClassDriver 514
Device.Input.HID.MCEClassDriverWindows7 516
Device.Input.HID.MCERemoteControlCompliance 517
Device.Input.HID.UsbSpecificationCompliant 519
Device.Input.Keyboard 520
Device.Input.Keyboard.BrowserMultimediaKeysUseMSApis 521
Device.Input.Keyboard.DynamicKeyboards 521
Device.Input.Keyboard.HotKeyFunctionAPI 523
Device.Input.Keyboard.KernelModeDriversUseWdfKmdf 524
Device.Input.Keyboard.LogoFlagKey 525
Device.Input.Keyboard.MultipleKeyboard 526
Device.Input.Keyboard.PS2UniqueHWID 527
Device.Input.Keyboard.ScanCode 528
Device.Input.PointDraw 529
Device.Input.PointDraw.KernelModeDriversUseWdfKmdf 529
Device.Input.PointDraw.PS2UniqueHWID 530
Device.Input.Sensor.Accelerometer 531
Device.Input.Sensor.Accelerometer.SensorDataType 531
Device.Input.Sensor.Accelerometer.SensorReportInterval 532
Device.Input.Sensor.Accelerometer.ShakeEvent 533
Device.Input.Sensor.ALS 534
Device.Input.Sensor.ALS.SupportRequiredData 534
Device.Input.Sensor.Base 535
Device.Input.Sensor.Base.SupportDataTypesAndProperties 536
Device.Input.Sensor.Compass 539
Device.Input.Sensor.Compass.InclinometerDataType 539
Device.Input.Sensor.Compass.SensorDataType 540
Device.Input.Sensor.Compass.SensorReportInterval 542
Device.Input.Sensor.DeviceOrientation 543
Device.Input.Sensor.DeviceOrientation.SensorDataType 543
Device.Input.Sensor.Gyroscope 544
Device.Input.Sensor.Gyroscope.SensorDataType 544
Device.Input.Sensor.Gyroscope.SensorReportInterval 546
Device.Input.Sensor.Location 547

Device.Input.Sensor.Location.SupportRequiredDataFieldsForReport 547
Device.Input.Sensor.Presence 550
Device.Input.Sensor.Presence.SensorDataType 550
Device.Input.SmartCardMiniDriver 552
Device.Input.SmartCardMiniDriver.DoNotStopWhenResourcesAreUnavailable 552
Device.Input.SmartCardMiniDriver.SpecsAndCertifications 553
Device.Input.SmartCardMiniDriver.SupportMultipleInstancesOnASystem 555
Device.Input.SmartCardReader 556
Device.Input.SmartCardReader.PinDataEntryKeyboardCompliesWithIso 556
Device.Input.SmartCardReader.SmartCardService 557
Device.Input.SmartCardReader.Supports258And259BytePackets 557
Device.Input.SmartCardReader.SupportsDirectAndInverseConvention 558
Device.Input.SmartCardReader.SupportsInsertionAndRemovalMonitor 559
Device.Input.SmartCardReader.SupportsMinClockFrequency 560
Device.Input.SmartCardReader.SupportsMinDataRateOf9600bps 560
Device.Input.SmartCardReader.SupportsNegotiableAndSpecificModes 561
Device.Input.SmartCardReader.SupportsResetCommand 562
Device.Input.SmartCardReader.UsbCcidCompliesWithUsbDeviceClassSpec 563
Device.Input.SmartCardReader.UsbCcidIssuesNak 564
Device.Media.DMR.AV 564
Device.Media.DMR.AV.AVC 565
Device.Media.DMR.AV.WMV 567
Device.Media.DMR.Base 568
Device.Media.DMR.Base.ChangingFriendlyName 569
Device.Media.DMR.Base.ContentPlaybackWithoutUserInput 570
Device.Media.DMR.Base.DeviceInformation 572
Device.Media.DMR.Base.DisplayedMetadata 573
Device.Media.DMR.Base.DLNA15CertificationCompliance 574
Device.Media.DMR.Base.DMPPlayback 576
Device.Media.DMR.Base.DMPSelectionOfAdvertisedResources 577
Device.Media.DMR.Base.DMRStateAfterFindingAnError 579
Device.Media.DMR.Base.EndOfStream 580
Device.Media.DMR.Base.Events 581
Device.Media.DMR.Base.MetaDataPackage 582

Device.Media.DMR.Base.MetadataSize 583
Device.Media.DMR.Base.OptionalFieldsEntries 584
Device.Media.DMR.Base.PausingAStream 585
Device.Media.DMR.Base.PlaybackPerformance 587
Device.Media.DMR.Base.PlayBackPerformanceConstrainedBandwidth 589
Device.Media.DMR.Base.PlaysMediaContinuously 590
Device.Media.DMR.Base.ProtocolInfoField 591
Device.Media.DMR.Base.ResponseToSetAVTransportURIActions 592
Device.Media.DMR.Base.SeekOperations 593
Device.Media.DMR.Base.SendsSSDPByeByeMessage 595
Device.Media.DMR.Base.SetNextAVTransportURI 597
Device.Media.DMR.Base.VolumeControl 599
Device.Media.DMR.Base.WakeOnLAN 600
Device.Media.DMR.Base.WiFiDirectSupport 602
Device.Media.DMR.Base.WMDRMNDLinkProtectionSupport 602
Device.Media.DMR.Image 604
Device.Media.DMR.Image.JPEG 604
Device.Media.DMS.Audio 605
Device.Media.DMS.Audio.AACAudioStreaming 605
Device.Media.DMS.Audio.AlbumArt 607
Device.Media.DMS.Audio.MP3AudioStreaming 608
Device.Media.DMS.Audio.WMAStreaming 609
Device.Media.DMS.AV 612
Device.Media.DMS.AV.AVCVideoStreaming 612
Device.Media.DMS.AV.ThumbnailImagesForTheAVMediaClass 613
Device.Media.DMS.AV.WMVStreaming 615
Device.Media.DMS.Base 617
Device.Media.DMS.Base.ChangingFriendlyName 617
Device.Media.DMS.Base.DeviceInformation 618
Device.Media.DMS.Base.DLNA15CertificationCompliance 619
Device.Media.DMS.Base.LowPowerModeSupport 620
Device.Media.DMS.Base.MetaDataPackage 622
Device.Media.DMS.Base.MultipleHTTPConnections 622
Device.Media.DMS.Base.OptionalFieldsEntries 623

Device.Media.DMS.Base.Performance 624
Device.Media.DMS.Base.RangeRequests 626
Device.Media.DMS.Base.SearchOperations 627
Device.Media.DMS.Base.StreamsContinuously 629
Device.Media.DMS.Base.WakeOnLAN 629
Device.Media.DMS.Image 631
Device.Media.DMS.Image.JPEGImageTransfer 631
Device.Media.DMS.Image.ThumbnailImagesForTheImageMediaClass 632
Device.Network.DevFund 633
Device.Network.DevFund.NdisVersion 634
Device.Network.DevFund.NdisVersionLegacy 634
Device.Network.DevFund.NPOS 635
Device.Network.DevFund.SelectiveSuspend 636
Device.Network.LAN.Base 636
Device.Network.LAN.Base.100MbOrGreater 637
Device.Network.LAN.Base.32MulticastAddresses 638
Device.Network.LAN.Base.AdvProperties 639
Device.Network.LAN.Base.AnyBoundary 639
Device.Network.LAN.Base.IPv4AndIPv6OffloadParity 640
Device.Network.LAN.Base.NDISCalls 641
Device.Network.LAN.Base.NDISRequirements 642
Device.Network.LAN.Base.PacketFiltering 642
Device.Network.LAN.Base.PreserveOSServices 643
Device.Network.LAN.Base.PriorityVLAN 644
Device.Network.LAN.Base.ShortPacketPadding 645
Device.Network.LAN.Base.SupportIEEEE8023 646
Device.Network.LAN.ChecksumOffload 647
Device.Network.LAN.ChecksumOffload.ChecksumOffload 647
Device.Network.LAN.DCB 648
Device.Network.LAN.DCB.DCB 648
Device.Network.LAN.GRE 649
Device.Network.LAN.GRE.GREPacketTaskOffloads 649
Device.Network.LAN.IPsec 650
Device.Network.LAN.IPsec.IPsec 650

Device.Network.LAN.KRDMA 651
Device.Network.LAN.KRDMA.KRDMA 652
Device.Network.LAN.LargeSendOffload 653
Device.Network.LAN.LargeSendOffload.LargeSendOffload 653
Device.Network.LAN.NetworkDirect 654
Device.Network.LAN.NetworkDirect.KeyMPIUsagePatterns 654
Device.Network.LAN.NetworkDirect.LowDataErrors 655
Device.Network.LAN.NetworkDirect.MeetMinRates 655
Device.Network.LAN.NetworkDirect.NDISInterface 657
Device.Network.LAN.NetworkDirect.NDSPISpec 657
Device.Network.LAN.NetworkDirect.RegisterMemoryRates 658
Device.Network.LAN.NetworkDirect.RemoteMemoryViaByteBoundaries 659
Device.Network.LAN.NetworkDirect.WIMDriverInjection 660
Device.Network.LAN.PM 661
Device.Network.LAN.PM.PowMgmtNDIS 661
Device.Network.LAN.PM.WakeOnLANPatterns 662
Device.Network.LAN.PM.WakePacket 664
Device.Network.LAN.RSC 665
Device.Network.LAN.RSC.RSC 665
Device.Network.LAN.RSS 665
Device.Network.LAN.RSS.RSS 666
Device.Network.LAN.RSS.SetHashFunctionTypeAndValue 667
Device.Network.LAN.RSS.SupportHashTypeNDISRSSAPSHASHTYPETCPIPV4 668
Device.Network.LAN.RSS.SupportIndirectionTablesSizes 669
Device.Network.LAN.RSS.SupportToeplitzHashFunction 670
Device.Network.LAN.RSS.SupportUpdatesToRSSInfo 670
Device.Network.LAN.SRIOV 671
Device.Network.LAN.SRIOV.SRIOV 672
Device.Network.LAN.TCPChimney 673
Device.Network.LAN.TCPChimney.ComplyWithNDIS 673
Device.Network.LAN.TCPChimney.ComplyWithTCPIPProtocol 674
Device.Network.LAN.TCPChimney.HandlesOutOfOrderData 679
Device.Network.LAN.TCPChimney.ImplementSufficientlyGranularTimers 680
Device.Network.LAN.TCPChimney.NeighborStateObjTimestampsComplyWithWDK 681

Device.Network.LAN.TCPChimney.Support1024Connections 682
Device.Network.LAN.TCPChimney.Support64bitAddresses 683
Device.Network.LAN.VMQ 683
Device.Network.LAN.VMQ.VirtualMachineQueues 684
Device.Network.MobileBroadband.CDMA 685
Device.Network.MobileBroadband.CDMA.ComplyWithBaseReq 686
Device.Network.MobileBroadband.CDMA.FWComplyWithMBSpec 687
Device.Network.MobileBroadband.CDMA.ImplementSMS 688
Device.Network.MobileBroadband.CDMA.MultiCarrierFunctionality 688
Device.Network.MobileBroadband.CDMA.SupportUSBSelectiveSuspend 689
Device.Network.MobileBroadband.CDMA.SupportWakeOnMB 690
Device.Network.MobileBroadband.GSM 691
Device.Network.MobileBroadband.GSM.ComplyWithBaseReq 691
Device.Network.MobileBroadband.GSM.EAPSIM 693
Device.Network.MobileBroadband.GSM.FWComplyWithMBSpec 693
Device.Network.MobileBroadband.GSM.ImplementSMS 694
Device.Network.MobileBroadband.GSM.MultiCarrierFunctionality 695
Device.Network.MobileBroadband.GSM.SupportFastDormancy 696
Device.Network.MobileBroadband.GSM.SupportUSBSelectiveSuspend 696
Device.Network.MobileBroadband.GSM.SupportWakeOnMB 697
Device.Network.MobileBroadband.GSM.USSD 698
Device.Network.MobileBroadband.WiMax 698
Device.Network.MobileBroadband.WiMax.ComplyWithBaseReq 699
Device.Network.MobileBroadband.WiMax.ImplementIPPacket 700
Device.Network.MobileBroadband.WiMax.MultiCarrierFunctionality 701
Device.Network.MobileBroadband.WiMax.SupportUSBSelectiveSuspend 702
Device.Network.MobileBroadband.WiMax.SupportWakeOnMB 702
Device.Network.Router 703
Device.Network.Router.BasicCompatibility 704
Device.Network.Router.BasicPerf 705
Device.Network.Router.ConeOrRestrictedNAT 706
Device.Network.Router.GetTotalBytesPerf 706
Device.Network.Router.GetTotalBytesSupport 707
Device.Network.Router.NATLoopback 708

Device.Network.Router.PnPXUPnPSupport 709
Device.Network.Router.PresentationURLPnPProperty 710
Device.Network.Router.UPnPIGD 711
Device.Network.Router.UPnPPortMappings 711
Device.Network.Router.WCNDynamicPIN 712
Device.Network.Router.WFACertified 712
Device.Network.Router.WPSVer2 713
Device.Network.Router.WPSVer2PushButton 714
Device.Network.WLAN.Base 714
Device.Network.WLAN.Base.ConformToNDIS 715
Device.Network.WLAN.Base.ImplementD0PacketCoalescing 716
Device.Network.WLAN.Base.ImplementVoicePersonalWMMPowerSave 716
Device.Network.WLAN.Base.MeetPerformanceReq 717
Device.Network.WLAN.Base.MeetScanAndConnReq 718
Device.Network.WLAN.Base.MinimizeCPUUtilization 720
Device.Network.WLAN.Base.OnlyWDFOrNDIS630Calls 721
Device.Network.WLAN.Base.PassWiFiAllianceCertification 722
Device.Network.WLAN.Base.PermitIEToRequestAndResponseAF 722
Device.Network.WLAN.Base.SupportFiltering32MulticastAddresses 723
Device.Network.WLAN.Base.SupportIEEE80211w 724
Device.Network.WLAN.Base.SupportMultiDeviceInstances 724
Device.Network.WLAN.Base.SupportPromiscuousAndMulticastPacketFiltering 725
Device.Network.WLAN.Base.SupportSeparateBeaconAndProbeIE 726
Device.Network.WLAN.Base.SupportVirtualWiFi 726
Device.Network.WLAN.Base.SupportWiFiAutoSaveMode 727
Device.Network.WLAN.Base.TransmitPacketsOnAnyBoundary 728
Device.Network.WLAN.CSBBase 728
Device.Network.WLAN.CSBBase.ConformToNDIS 729
Device.Network.WLAN.CSBBase.ImplementVoicePersonalWMMPowerSave 729
Device.Network.WLAN.CSBBase.MeetPerformanceReq 730
Device.Network.WLAN.CSBBase.MeetScanAndConnReq 731
Device.Network.WLAN.CSBBase.MinimizeCPUUtilization 733
Device.Network.WLAN.CSBBase.MustSupportD0PacketCoalescing 734
Device.Network.WLAN.CSBBase.OnlyWDFOrNDIS630Calls 735

Device.Network.WLAN.CSBBase.PassWiFiAllianceCertification 735
Device.Network.WLAN.CSBBase.PermitIEToRequestAndResponseAF 736
Device.Network.WLAN.CSBBase.SupportFiltering32MulticastAddresses 737
Device.Network.WLAN.CSBBase.SupportIEEE80211w 737
Device.Network.WLAN.CSBBase.SupportPromiscuousAndMulticastPacketFiltering 738
Device.Network.WLAN.CSBBase.SupportSeparateBeaconAndProbeIE 739
Device.Network.WLAN.CSBBase.SupportVirtualWiFi 739
Device.Network.WLAN.CSBBase.SupportWiFiAutoSaveMode 740
Device.Network.WLAN.CSBBase.TransmitPacketsOnAnyBoundary 741
Device.Network.WLAN.CSBNLO 741
Device.Network.WLAN.CSBNLO.SupportNetworkListOffload 741
Device.Network.WLAN.CSBSoftAP 742
Device.Network.WLAN.CSBSoftAP.SupportSoftAP 742
Device.Network.WLAN.CSBWiFiDirect 743
Device.Network.WLAN.CSBWiFiDirect.SupportAtLeast2WiFiDirectPortsConcurrently 743
Device.Network.WLAN.CSBWiFiDirect.SupportAtLeast4Clients 744
Device.Network.WLAN.CSBWoWLAN 745
Device.Network.WLAN.CSBWoWLAN.MustSupportWakeOnWLAN 745
Device.Network.WLAN.NLO 746
Device.Network.WLAN.NLO.SupportNetworkListOffload 746
Device.Network.WLAN.SoftAP 747
Device.Network.WLAN.SoftAP.SupportSoftAP 747
Device.Network.WLAN.WiFiDirect 748
Device.Network.WLAN.WiFiDirect.SupportAtLeast2WiFiDirectPortsConcurrently 748
Device.Network.WLAN.WiFiDirect.SupportAtLeast4Clients 749
Device.Network.WLAN.WoWLAN 749
Device.Network.WLAN.WoWLAN.ImplementWakeOnWLAN 749
Device.Portable.Core 750
Device.Portable.Core.AudioCodec 751
Device.Portable.Core.CustomDeviceServices 753
Device.Portable.Core.DeviceServices 755
Device.Portable.Core.MediaSync 757
Device.Portable.Core.ModelID 758
Device.Portable.Core.MTP 759

Device.Portable.Core.MTPFunctionality 766
Device.Portable.Core.MTPMultiSession 767
Device.Portable.Core.MTPObjectProperties 769
Device.Portable.Core.MTPStreams 771
Device.Portable.Core.TransportBluetooth 773
Device.Portable.Core.TransportIP 774
Device.Portable.Core.TransportIPDLNA 776
Device.Portable.Core.TransportUSB 777
Device.Portable.Core.VideoCodec 778
Device.Portable.DigitalCamera 782
Device.Portable.DigitalCamera.MTP 783
Device.Portable.DigitalVideoCamera 787
Device.Portable.DigitalVideoCamera.MTP 787
Device.Portable.MediaPlayer 792
Device.Portable.MediaPlayer.MTP 792
Device.Portable.MobilePhone 797
Device.Portable.MobilePhone.MTP 797
Device.Storage.Controller 802
Device.Storage.Controller.BasicFunction 802
Device.Storage.Controller.BitLocker 803
Device.Storage.Controller.ClassCode 804
Device.Storage.Controller.InfFile 805
Device.Storage.Controller.MiniportDriverModel 806
Device.Storage.Controller.Ata 807
Device.Storage.Controller.Ata.Interface 807
Device.Storage.Controller.Boot 808
Device.Storage.Controller.Boot.BasicFunction 809
Device.Storage.Controller.Boot.BitLocker 810
Device.Storage.Controller.BootDeviceGreaterThan 810
Device.Storage.Controller.BootDeviceGreaterThan.BasicFunction 810
Device.Storage.Controller.Fc 812
Device.Storage.Controller.Fc.Interface 812
Device.Storage.Controller.Fcoe 813
Device.Storage.Controller.Fcoe.Interface 813

Device.Storage.Controller.Fcoe.Interoperability 814
Device.Storage.Controller.Flush 815
Device.Storage.Controller.Flush.BasicFunction 815
Device.Storage.Controller.Iscsi 816
Device.Storage.Controller.Iscsi.Interface 816
Device.Storage.Controller.Iscsi.Boot 818
Device.Storage.Controller.Iscsi.Boot.FwTable 818
Device.Storage.Controller.Optical 820
Device.Storage.Controller.Optical.BasicFunction 820
Device.Storage.Controller.Raid 821
Device.Storage.Controller.Raid.BasicFunction 821
Device.Storage.Controller.Raid.PassThroughDiskSupport 822
Device.Storage.Controller.Sas 823
Device.Storage.Controller.Sas.Interface 823
Device.Storage.Controller.Sata 824
Device.Storage.Controller.Sata.Interface 824
Device.Storage.Hd 825
Device.Storage.Hd.BasicFunction 826
Device.Storage.Hd.PhysicalSectorSizeReportsAccurately 827
Device.Storage.Hd.1394 828
Device.Storage.Hd.1394.Compliance 828
Device.Storage.Hd.Alua 829
Device.Storage.Hd.Alua.Compliance 829
Device.Storage.Hd.Ata 830
Device.Storage.Hd.Ata.BasicFunction 830
Device.Storage.Hd.Ata.Dma 831
Device.Storage.Hd.AtaProtocol 832
Device.Storage.Hd.AtaProtocol.Performance 832
Device.Storage.Hd.AtaProtocol.Protocol 833
Device.Storage.Hd.DataVerification 834
Device.Storage.Hd.DataVerification.BasicFunction 834
Device.Storage.Hd.Ehdd 835
Device.Storage.Hd.Ehdd.Compliance 835
Device.Storage.Hd.EnhancedStorage 838

Device.Storage.Hd.EnhancedStorage.1667Compliance 838
Device.Storage.Hd.FibreChannel 839
Device.Storage.Hd.FibreChannel.Compliance 839
Device.Storage.Hd.Flush 840
Device.Storage.Hd.Flush.BasicFunction 840
Device.Storage.Hd.Hybrid 841
Device.Storage.Hd.Hybrid.Piton 841
Device.Storage.Hd.Iscsi 843
Device.Storage.Hd.Iscsi.BasicFunction 843
Device.Storage.Hd.Mpio 845
Device.Storage.Hd.Mpio.BasicFunction 845
Device.Storage.Hd.OffloadedDataTransfer 846
Device.Storage.Hd.OffloadedDataTransfer.CopyOffload 846
Device.Storage.Hd.PersistentReservation 849
Device.Storage.Hd.PersistentReservation.ClusterFailover 849
Device.Storage.Hd.RaidArray 850
Device.Storage.Hd.RaidArray.BasicFunction 850
Device.Storage.Hd.RaidArray.BitLocker 851
Device.Storage.Hd.ReadZeroOnTrimUnmap 852
Device.Storage.Hd.ReadZeroOnTrimUnmap.BasicFunction 852
Device.Storage.Hd.Sas 853
Device.Storage.Hd.Sas.ComplyWithIndustrySpec 853
Device.Storage.Hd.Sata 854
Device.Storage.Hd.Sata.BasicFunction 854
Device.Storage.Hd.Scsi 855
Device.Storage.Hd.Scsi.Connectors 855
Device.Storage.Hd.Scsi.ParallelInterface 856
Device.Storage.Hd.ScsiEnclosureService 857
Device.Storage.Hd.ScsiEnclosureService.Compliance 857
Device.Storage.Hd.ScsiProtocol 859
Device.Storage.Hd.ScsiProtocol.ReferenceSpec 859
Device.Storage.Hd.ScsiProtocol.SamCompliance 860
Device.Storage.Hd.ScsiProtocol.SpcCompliance 861
Device.Storage.Hd.ThinProvisioning 865

Device.Storage.Hd.ThinProvisioning.BasicFunction 865
Device.Storage.Hd.Trim 868
Device.Storage.Hd.Trim.BasicFunction 868
Device.Storage.Hd.Uas 869
Device.Storage.Hd.Uas.Compliance 869
Device.Storage.Hd.UasOnEHCI 870
Device.Storage.Hd.UasOnEHCI.BasicFunction 870
Device.Storage.Hd.Usb 871
Device.Storage.Hd.Usb.Compatibility 871
Device.Storage.Hd.Usb3 873
Device.Storage.Hd.Usb3.Compliance 873
Device.Storage.Hd.WindowsToGoCapableUSBDrive 874
Device.Storage.Hd.WindowsToGoCapableUSBDrive.WindowsToGoCapableUSBDrive 874
Device.Storage.Optical 876
Device.Storage.Optical.CdRawRecording 876
Device.Storage.Optical.CommandPerformance 877
Device.Storage.Optical.DriveDefinition 880
Device.Storage.Optical.Features 881
Device.Storage.Optical.MmcVersion 882
Device.Storage.Optical.Profiles 883
Device.Storage.Optical.RealTimeStreaming 884
Device.Storage.Optical.BluRayReader 885
Device.Storage.Optical.BluRayReader.Profiles 885
Device.Storage.Optical.BluRayWriter 886
Device.Storage.Optical.BluRayWriter.Profiles 886
Device.Storage.Optical.Sata 887
Device.Storage.Optical.Sata.AsynchronousNotification 887
Device.Streaming.HMFT 887
Device.Streaming.HMFT.Decoding 888
Device.Streaming.HMFT.Encoding 891
Device.Streaming.Tuner 901
Device.Streaming.Tuner.AnalogRequirements 901
Device.Streaming.Tuner.AnalogSupportsAudioAndVideo 903
Device.Streaming.Tuner.AutomatedCaptureGraph 904

Device.Streaming.Tuner.AVStreamWDMAndInterfaceRequirements 905
Device.Streaming.Tuner.ConditionalAccessSystems 907
Device.Streaming.Tuner.ContinuousChannelChanges 908
Device.Streaming.Tuner.ContinuousReboots 909
Device.Streaming.Tuner.ContinuousStreaming 910
Device.Streaming.Tuner.DetectsSignalsFromAllSources 911
Device.Streaming.Tuner.DigitalTunerUsesBDA 912
Device.Streaming.Tuner.FirstRun 913
Device.Streaming.Tuner.HybridComboImplementation 913
Device.Streaming.Tuner.IPFunctionality 914
Device.Streaming.Tuner.ISDBTSupportBML 915
Device.Streaming.Tuner.iSerialNumber 916
Device.Streaming.Tuner.MPEGEncoding 917
Device.Streaming.Tuner.MSVidCtl 917
Device.Streaming.Tuner.MultipleClientAppSupport 918
Device.Streaming.Tuner.MultipleStreams 919
Device.Streaming.Tuner.PBDA 919
Device.Streaming.Tuner.PBDACustomizedProductRequirements 920
Device.Streaming.Tuner.PBDAExtensibilityPages 921
Device.Streaming.Tuner.ProperlySignalsChannelChanges 922
Device.Streaming.Tuner.ResourcesAcquiredAtDeviceAcquire 923
Device.Streaming.Tuner.ScanForServices 923
Device.Streaming.Tuner.ScheduledRecording 924
Device.Streaming.Tuner.SignalQualityAndLock 925
Device.Streaming.Tuner.SleepStates 926
Device.Streaming.Tuner.SupportsAutomaticRendering 927
Device.Streaming.Tuner.TimeShiftedPlayback 928
Device.Streaming.Tuner.UVC 928
Device.Streaming.Tuner.UVCDriver 929
Device.Streaming.Tuner.WMCEncoding 930
Device.Streaming.Webcam.Base 931
Device.Streaming.Webcam.Base.AVStreamWDMAndInterfaceRequirements 931
Device.Streaming.Webcam.Base.BasicPerf 933
Device.Streaming.Webcam.Base.DirectShowAndMediaFoundation 934

Device.Streaming.Webcam.Base.IPFunctionality 934
Device.Streaming.Webcam.Base.KSCategoryVideoCameraRegistration 935
Device.Streaming.Webcam.Base.MultipleClientAppSupport 936
Device.Streaming.Webcam.Base.SurpriseRemoval 937
Device.Streaming.Webcam.Base.UsageIndicator 937
Device.Streaming.Webcam.H264 938
Device.Streaming.Webcam.H264.H264Support 938
Device.Streaming.Webcam.NonMSDriver 940
Device.Streaming.Webcam.NonMSDriver.VideoInfoHeader2 940
Device.Streaming.Webcam.USBClassDriver 941
Device.Streaming.Webcam.USBClassDriver.UVC 941
Device.Streaming.Webcam.USBClassDriver.UVCDriver 942

Introduction

This release to web (RTW) document contains the Windows Hardware Certification requirements for Windows 8 devices. These requirements are Microsoft's guidelines for designing Windows internal or external devices. Successfully following this guidance will allow a partner to receive certification for their device and signing for their drivers.

The requirements are organized by feature using a Camel Case naming convention, which facilitates grouping related requirements and communicating their relationship to the Windows feature they are intended to support. Tests assessing compliance with the features are exposed during testing with the Hardware Certification Kit and can be related directly back to these requirements.

Some requirements have passed forward from Logo requirements for earlier Windows versions which used a category based structure. We have included the older LogoPoint ID in the comments section for your convenience.

If Implemented Requirements

The Windows Hardware Certification program declares requirements which must be met by any device under the feature area of device fundamentals, requirements for each connectivity used by the device, and requirements for specific device type features or technologies in the PC ecosystem. Additional functionality is built into Windows which is optional, and can offer a competitive edge for manufacturers should they chose to implement these features. In these cases, there are requirements which must be met only if the additional functionality is implemented. Because these additional requirements apply only if the relevant functionality is implemented, they are referred to as "if-implemented" requirements in this document.

The Hardware Certification Kit detects features exposed by a product automatically. The detected features the will be tested for compliance whether they were mandatory to successfully be certified as a defined product type or if the feature is optional. The title of the requirement or the exception field will indicate when a requirement applies.

Features

Device.Audio.APO

Description:

This APO must match all APO tests

Related Requirements:

- Device.Audio.APO.MicArrayRawData
- Device.Audio.APO.WinPEConformance

Device.Audio.APO.MicArrayRawData

Target Feature: Device.Audio.APO

Title: System effect in capture path provides RAW data from microphone array when requested by the client

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM
- Windows Server 2008 Release 2 x64
- Windows Vista Client x86
- Windows Vista Client x64
- Windows 8 Server x64

Description:

If a microphone array processing algorithm is provided in a Windows system effect audio processing object (APO) instantiated in system effect local effect (LFX) insert point in capture path, it must provide all the individual audio streams from the array when a client asks for a format greater than one stream/channel. This allows the APO to provide hardware compensation processing and microphone array processing to the client that takes advantage of the entire APO but allows clients that rely on the microphone array processing that resides higher up in the audio subsystem to take advantage of hardware compensation in the APO but not the array processing in it.

Exceptions: Not Specified

Business Justification:

This helps Windows ensure mic array function correctly.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 9/17/2008

Comments:

AUDIO-0045

Device.Audio.APO.WinPEConformance

Target Feature: Device.Audio.APO

Title: System effect APOs comply with Windows Protected Environment conformance and compliance criteria

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

Audio Processing Objects (APOs) that are loaded into the system effect infrastructure by the audio device driver INF must be signed with a certificate indicating conformance and compliance with the robustness rules of the Windows Protected Environment. This signature attribute is granted automatically as part of the submission process dependent on signing of the Test Agreements.

See Code Signing for Protected Media Components in Windows at http://go.microsoft.com/fwlink/?LinkId=70751.

Exceptions: Not Specified

Business Justification:

This helps ensure compatibility with protected media.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 7/17/2008

Comments:

AUDIO-0048

Device.Audio.AudioController

Description:

This Audio Controller must match all Audio Controller tests

Related Requirements:

- Device.Audio.AudioController.HDAudioVersionNumber
- Device.Audio.AudioController.HDControllerCompliance

Device.Audio.AudioController.HDAudioVersionNumber

Target Feature: Device.Audio.AudioController

Title: HD Audio 1.0 compliant hardware sets appropriate registers to specify the version number

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

HD Audio hardware that complies with HD Audio specification version 1.0 must set the correct version number in the appropriate registers. The VMAJ and VMIN registers must specify a major version number of 01h and a minor version number of 00h.

Design Notes:

In future HD Audio specification revisions the register values may be updated. It is assumed that any future requirements that reference an updated revision of the specification will also require using the VMAJ and VMIN registers defined by that revision of the specification.

Exceptions: Not Specified

Business Justification:

This helps Windows understand the HD Audio version number and improves compatibility.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 7/17/2008

Comments:

AUDIO-0013

Device.Audio.AudioController.HDControllerCompliance

Target Feature: Device.Audio.AudioController

Title: HD Audio controllers comply with the Intel HD Audio specification

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

An audio or modem controller must be implemented as an HD Audio controller (except where noted otherwise within this document). The controller must:

Be implemented according to Intel High Definition Audio Controller specification, Revision 1.0.

Be updated to comply with future specification revisions.

Comply with future HD Audio specification ECRs in accordance with policies around new hardware requirements.

Exceptions: Not Specified

Business Justification:

This ensures compatibility with HD audio specification and enables Windows works correctly with the device.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 7/17/2008

Comments:

AUDIO-0012

Device.Audio.Base

Description:

This Device must match all base tests.

Related Requirements:

- Device.Audio.Base.AudioDriversSupportMute
- Device.Audio.Base.BasicDataFormats
- Device.Audio.Base.ChannelMasks
- Device.Audio.Base.CopyBitPolarityClarification
- Device.Audio.Base.DCOffset
- Device.Audio.Base.DevicesWorkWithoutExtraSoftware
- Device.Audio.Base.DigitalStreamsNoMixing
- Device.Audio.Base.DockingStation
- Device.Audio.Base.DRM
- Device.Audio.Base.EfficientBufferManagement
- Device.Audio.Base.ExposedAudioEndpointsAreFunctional
- Device.Audio.Base.Fidelity
- Device.Audio.Base.FloatSupport
- Device.Audio.Base.FullDuplexOperation
- Device.Audio.Base.HDAudioRemoveDevicePowerState

- Device.Audio.Base.IMiniportWaveRTStreamNotification
- Device.Audio.Base.IndependentInputOutputFormatSelection
- Device.Audio.Base.InitiatorTargetBlocktransferSupport
- Device.Audio.Base.JackConnectorStateDescription
- Device.Audio.Base.JackDetection
- Device.Audio.Base.KSPROPERTYAUDIOMIXLEVELTABLE
- Device.Audio.Base.KSPROPERTYAUDIOVOLUMELEVEL
- Device.Audio.Base.KSTopologyCompliance
- Device.Audio.Base.NoHiddenStreamRouting
- Device.Audio.Base.NoUncontrollableStreamRouting
- Device.Audio.Base.NoUndiscoverableDevice
- Device.Audio.Base.PCMNonPCMForSPDIF
- Device.Audio.Base.PowerManagement
- Device.Audio.Base.ProperUSBDescriptors
- Device.Audio.Base.RealtimeDriversSupportStandardLoopedStreaming
- Device.Audio.Base.RecordPlaybackBasicPerformance
- Device.Audio.Base.ReportSupportedProperties
- Device.Audio.Base.RestartWithinASpecifiedDuration
- Device.Audio.Base.SamplePositionAccuracy
- Device.Audio.Base.SamplingAccuracy
- Device.Audio.Base.SPDIFSupportMinimumSamplingRate
- Device.Audio.Base.TimeSynchronizedSampleRates
- Device.Audio.Base.TipRing
- Device.Audio.Base.TwoDMAEnginesAndConnections
- Device.Audio.Base.VoiceCommunicationUAA
- Device.Audio.Base.VolumeControlsIsLinear
- Device.Audio.Base.VolumeGranularity
- Device.Audio.Base.WAVEFORMATEXTENSIBLESupport
- Device.Audio.Base.WaveRTConformance
- Device.Audio.Base.WaveRTImplementation
- Device.Audio.Base.ZeroGlitch

Device.Audio.Base.AudioDriversSupportMute

Target Feature: Device.Audio.Base

Title: Audio drivers support mute

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64

- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

Any audio driver capable of supporting compressed format need to expose KSPROPERTY_AUDIO_MUTE

Exceptions: Not Specified

Business Justification:

For information on how to twiddle the Pin Widget Control enable bit, see the Intel High Definition Audio Specification, Revision 1.0or later.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 7/17/2008

Comments:

AUDIO-0001

Device.Audio.Base.BasicDataFormats

Target Feature: Device.Audio.Base

Title: Audio subsystem supports basic data formats

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

When the Microsoft software sample rate conversion (SRC) is used, hardware SRC is not required. Windows provides software mixing and SRC, which eliminates the requirement for hardware to support multiple sample rates.

Device Type	Required Sample Rate

Communications-only audio device	*At least one* of 16 kHz, 44.1 kHz, or 48 kHz*
General-purpose or media-capable audio device	*At least one* of 44.1 kHz or 48 kHz**

* Bluetooth HFP drivers that do not implement wideband speech are bound by the standard to support only 8 kHz. This is an accepted exception.

** Unless further specified by external specifications. For example, HDMI audio requires both 44.1 kHz and 48 kHz, and Bluetooth A2DP requires both 44.1 kHz and 48 kHz for a Bluetooth sink. Windows HCK tests may enforce these stricter specifications.

Support for other rates (8, 11.025, 16, 22.05, 32, 96, 192, and 384 kHz) in hardware is optional.

Design Notes:

This requirement is valid for both input and output devices.

Exceptions: Not Specified

Business Justification:

Full duplex audio is essential to support emerging communications applications such as Internet Protocol (IP) telephony, conferencing, and network gaming. These applications require the audio system to play back and record simultaneously. The following requirements ensure that full duplex operation is available and performance is consistent across implementations.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 7/17/2008

Comments:

AUDIO-0023

Device.Audio.Base.ChannelMasks

Target Feature: Device.Audio.Base

Title: Audio device that supports multichannel audio formats properly handles channel masks

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

If the audio device supports multichannel audio formats, the audio device driver must deal with channel masks consistent with the content and the current selected speaker configuration.

If supported, the device properly handles 5.1 and 7.1 PCM formats. The channels are routed to the proper analog lines, and these requirements apply for all the channels except LFE.

See the Audio Driver Support for Home Theater Speaker Configurations whitepaper at http://go.microsoft.com/fwlink/?LinkId=65430.

Exceptions: Not Specified

Business Justification:

Multi-channel audio needs to behave in a predictable manner with Windows.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 7/17/2008

Comments:

AUDIO-0035

Device.Audio.Base.CopyBitPolarityClarification

Target Feature: Device.Audio.Base

Title: HD Audio devices that expose a digital output must meet HD audio design change notification "Copy Bit Clarification"

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64

Description:

HD Audio devices that expose a digital output must meet HD audio design change notification "Copy Bit Clarification"

Refer to http://download.intel.com/design/chipsets/hdaudio/HDA041-A_DCN_-_Copy_Bit_Polarity_Clarification_rev1p0.pdf

Exceptions: Not Specified

Business Justification:

This is needed for class driver compatibility and to correctly play copy protected content.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 9/17/2008

Comments:

NEW

Device.Audio.Base.DCOffset

Target Feature: Device.Audio.Base

Title: Audio capture device DC offset is limited within range of + or - 0.15 on a scale from -1.0 to +1.0

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

Audio capture device DC offset is limited within range of + or - 0.15 on a scale from -1.0 to +1.0

Exceptions: Not Specified

Business Justification:

To ensure good audio capture quality

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 9/17/2008

Comments:

AUDIO-0055

Device.Audio.Base.DevicesWorkWithoutExtraSoftware

Target Feature: Device.Audio.Base

Title: Devices that work with UAA class drivers must support basic functionality without installing any additional software or drivers.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

UAA audio devices must meet basic functionality requirements following a new Windows installation and prior to the installation of any additional software (drivers, system services or applications). When running exclusively with the built-in Windows support, audio devices as a minimum shall:

- Correctly render stereo sound from built-in speakers

- Correctly render mono sound from mono render devices

- Correctly transmit a stereo sound signal through line output connectors

- Correctly capture a stereo line-in signal

- Correctly capture a stereo mic-in signal

- Correctly capture a mono sound from mono capture devices

- Correctly support, Mute, Volume Control

Identical functionality must also be supported with any custom driver provided with the device.

If the physical design implements line-in and/or mic-in, then line-in and/or mic-in should work with class drivers in Windows.

Design Notes:

This requirement checks the interaction of the UAA compatible devices with the class driver and insures it is equivalent on basic functionality to that provided by third party drivers.

UAA class drivers support:

1. HD Audio

- High Definition Audio Specification Revision 1.0.

2. USB Audio 1.0

- Universal Serial Bus Device Class Definition for Audio Devices Release 1.0

- Universal Serial Bus Device Class Definition for Audio Data Formats Release 1.0

- Universal Serial Bus Device Class Definition for Terminal Types Release 1.0

- Universal Serial Bus Device Class Definition for MIDI Devices Release 1.0

Hardware mute and volume controls if implemented must be compatible with Windows class drivers. In particular:

- HD Audio hardware volume controls if implemented must be designed as amplifier widgets, and not as HD Audio volume knob widgets. See HD Audio spec sections 7.3.3.7 "Amplifier Gain/Mute", 7.3.4.10 "Amplifier Capabilities", and not 7.3.3.29 "Volume Knob".

- HD Audio mute is implemented many ways in the class driver. If an amplifier widget has the "mute capable" bit set, sending a mute control down must mute the signal path through that amplifier widget. See HD Audio spec sections 7.3.3.7 "Amplifier Gain/Mute" and 7.3.4.10 "Amplifier Capabilities". If a pin widget has "input capable" or "output capable" set, setting "input enable" or "output enable" to 0 must mute the signal path through that pin widget. See HD Audio spec sections 7.3.3.13 "Pin Widget Control" and 7.3.4.9 "Pin Capabilities." Setting "digital enable" bit to 0 on a digital converter must mute the signal path through that digital converter. See HD Audio spec sections 7.3.3.9 "Digital Converter Control" and 7.3.4.6 "Audio Widget Capabilities."

- USB hardware volume controls if implemented must be designed as the proper set of descriptors and associated command responses. Refer to USB Audio Specification 1.0, Mute Control and Volume Control as defined in sections 5.2.2.4.3.1 and 5.2.2.4.3.2.

Exceptions: Not Specified

Business Justification:

Only UAA class drivers ship in Windows and will be installed on any UAA device on a new Windows installation. Third party drivers posted on Windows Update require additional user action to be downloaded and installed. Devices must provide basic audio functionality with class drivers to protect the end user experience.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2011

Comments:

AUDIO-0088

Device.Audio.Base.DigitalStreamsNoMixing

Target Feature: Device.Audio.Base

Title: Audio sources are available as digital streams without analog mixing to the audio subsystem

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

Audio sources must be available as digital audio streams that are accessible to the system-wide kernel; that is, they must not rely exclusively on any analog mixing stage between the DAC converter and the speaker jack as the only means for output. Sources that continue to offer an analog mixing output configuration must also provide the user a configurable digital option.

One model for providing the user such an option is Windows support for CD music.

This requirement covers the following audio sources, which must be available digitally to USB speakers if attached:

CD-ROM or DVD

TV tuner

FM radio Voice modem

PC beep notifications are exempt from this requirement.

Design Notes: Analog microphone and line in with available analog-to-digital converters (ADCs) are digital-ready by definition. Devices for which the operating system supports emulation equivalents, such as hardware-accelerated 3-D and MIDI synthesis (Microsoft DirectSound 3D emulation and the Windows GS Wavetable SW Synth) are acceptable.

Exceptions: Not Specified

Business Justification:

This requirement assures that all audio content can be made available at both the analog jack and USB port. Elimination of the dependency on analog mixing for output is key to making computer audio easier to configure and use, and it removes a major obstacle for USB audio rendering devices.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 7/17/2008

Comments:

AUDIO-0030

Device.Audio.Base.DockingStation

Target Feature: Device.Audio.Base

Title: Audio jacks on docking stations

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM
- Windows 7 Client x86
- Windows 7 Client x64

Description:

Audio jacks on docking stations need to support the below three states:

1) The system is not plugged in to the dock:

The dock audio device should not appear in the Sound control panel at all.

2) The system is plugged in to the dock, but nothing is plugged into the jack:

The dock audio device should appear in the Sound control panel as "unplugged" (DEVICE_STATE_DISCONNECTED.)

3) The system is plugged in to the dock, and something is plugged into the jack:

The dock audio device should appear in the Sound control panel as "working" (DEVICE_STATE_ACTIVE.)

When the system is undocked and is using the HD Audio class driver, it is acceptable for devices on dock to show as unplugged.

Exceptions: Not Specified

Business Justification:

Ensures good audio experience for the end user when using docking station with audio jacks.

Scenarios:

Ensures compatibility with Windows.

Success Metric: Not Specified

Enforcement Date: 9/16/2011

Comments:

NEW

Device.Audio.Base.DRM

Target Feature: Device.Audio.Base

Title: Audio device implements DRM support as defined in the Windows Driver Kit

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM
- Windows Server 2008 Release 2 x64
- Windows 8 Server x64

Description:

Audio devices must comply with Windows trusted audio paths for digital rights management (DRM). Hardware that complies with Windows DRM supports DRM level 1300. The audio drivers must not call the DrmForwardContentToFileObject function.

The DRM requirement does not apply if the underlying device is a Bluetooth audio device.

Design Notes: See the"Digital Rights Management" topic in the Windows Driver Kit.

Exceptions: Not Specified

Business Justification:

This requirement is necessary for DRM compliance with Windows.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 7/17/2008

Comments:

AUDIO-0014

Device.Audio.Base.EfficientBufferManagement

Target Feature: Device.Audio.Base

Title: PCI-based audio device supports efficient audio buffer management

Applicable OS Versions:

- Windows 8 Client x86

- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

The audio device must be able to fully function when the system can provide only single 4K pages of contiguous memory. In other words, the audio device can require many 4K pages of memory, but it must not require the largest block of contiguous memory to exceed one 4K page.

The audio device and associated device-specific driver must not introduce unnecessary latency. If the audio driver adds more than 2 ms of computational latency between buffer transfer and queuing for rendering, the driver must provide a programmatic method for a latency-sensitive application to temporarily disable the computation.

Exceptions: Not Specified

Business Justification:

This requirement ensures audio support in docking and dynamic loading scenarios where memory may be fragmented with respect to pages. This requirement also helps to ensure that telephony applications closely resemble the performance of a conventional phone and minimizes the possibility that audio and video streams will appear out of synch.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 7/17/2008

Comments:

AUDIO-0029

Device.Audio.Base.ExposedAudioEndpointsAreFunctional

Target Feature: Device.Audio.Base

Title: Audio device must be functional (capable of capture/render) all the time while the system is powered on.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86

- Windows 7 Client x64
- Windows 8 Client ARM

Description:

Exposed Audio Endpoints in a system must be functional (capable of capture/render) all the time while the system is powered on. Built-in speakers and microphones must work while the system is operational.

Exposed audio end points continue to function even during system state changes such as:

* while the power source changes from external to battery or vice versa

* while the GPU switches from a to b

Exceptions: Not Specified

Business Justification:

Audio devices needs to remain functional

Scenarios:

Audio works as expected

Success Metric: Not Specified

Enforcement Date: 9/17/2008

Comments:

NEW

Device.Audio.Base.Fidelity

Target Feature: Device.Audio.Base

Title: Audio solution delivers a minimum fidelity audio experience

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

Description: Audio solution, either integrated or add-on discrete streaming audio device, must meet the followingminimum fidelity audio requirements. Measurements are performed electronically via externally accessible jacks (not in-air) for all the below defined device types when they exist on the device/system. Frequency sweep and/or multi-tone test methods may be used when appropriate.

The Device Types in the tables reflect those defined by the Intel HD Audio Spec and the Microsoft HD Audio Pin Configuration Programming Guidelines.

Device Type	Requirement	Value	Frequency range at 48KHz and above
Analog Line Output Jack	THD+N	>= 65 dB	[20 Hz, 20 KHz]
	Dynamic range with signal present	>= 80 dB A-weight	[20 Hz, 20 KHz]
	Magnitude Response	<= +-.25 dB ripple (.5dB peak to peak delta), 1 dB at upper band edge, 3dB at lower band edge	[20 Hz, 20 KHz]
	Sampling frequency accuracy	0.02%.	
	Cross-talk attenuation	>= 50 dB	[20 Hz, 15 KHz]
	Full scale output voltage	>= .707 Vrms	
	Dynamic range with signal present during system activity	>= 80 dBA-weight	[20Hz, 20kHz]
	Interchannel phase delay	30 degrees or 12.5 microseconds, whichever is greater	[20 Hz, 20 KHz]
Analog Speaker Output Jack (Example: 125mW into 8 Ohm load)	THD+N	>= 65 dB	[20 Hz, 20 KHz]
	Dynamic range with signal present	>= 80 dB A-weight	[20 Hz, 20 KHz]
	Magnitude Response	<= +-.25 dB ripple (.5dB peak to peak delta), 1dB band edges	[20 Hz, 20 KHz]
	Sampling frequency accuracy	0.02%.	
	Cross-talk attenuation	>= 50 dB	[20 Hz, 15 KHz]
	Full scale output voltage	>= .707 Vrms	
	Dynamic range with signal present during system activity	>= 80 dB A-weight	[20Hz, 20kHz]
	Interchannel phase delay	30 degrees or 12.5 microseconds, whichever is greater	[20 Hz, 20 KHz]
Analog Headphone Out Jack	THD+N	>= 65dB >= 45 dB at 32 Ohm Load	[100 Hz, 20 KHz]
	Dynamic range with signal present	>= 80 dB A-weight >= 60 dB at 32 Ohm Load	[100 Hz, 20 KHz]
	Magnitude Response	<= +-.25 dB ripple (.5dB peak to peak delta), 1 dB at upper band edge, 3dB at lower band edge	[100 Hz, 20 KHz]
	Sampling frequency accuracy	0.02%.	
	Cross-talk attenuation	>= 50 dB	[100 Hz, 15 KHz]

	Full scale output voltage	>=120 mVrms at 320 Ohm load	(3)
	Dynamic range with signal present during system activity	>= 80 dB A-weight >= 60 dB A-weight at 32 Ohm Load	[100Hz, 20kHz]
	Interchannel phase delay	30 degrees or 12.5 microseconds, whichever is greater	[100 Hz, 20 KHz]
Analog Line In Jack	THD+N	>= 65 dB	[20 Hz, 20 KHz]
	Dynamic range with signal present	>= 80 dB A-weight	[20 Hz, 20 KHz]
	Magnitude Response	<= +-.25 dB ripple (.5dB peak to peak delta), 1 dB at upper band edge, 3dB at lower band edge	[20 Hz, 20 KHz]
	Sampling frequency accuracy	0.02%.	
	Full scale input voltage	>= .707 Vrms	
	Device DC offset	within+/-0.15 on a scale from -1.0 to +1.0	
Analog Microphone In Jack	THD+N	>= 65 dB	[100 Hz, 20 KHz]
	Dynamic range with signal present	>= 70 dB A-weight	[100 Hz, 20 KHz]
	Magnitude Response	<= +-.25 dB ripple (.5dB peak to peak delta), 1 dB at upper band edge, 3dB at lower band edge	[100 Hz, 20 KHz]
	Sampling frequency accuracy	0.02%.	
	Full scale input voltage	>= 0.0707 Vrms	
	Device DC offset	within+/-0.15 on a scale from -1.0 to +1.0	

1. For codecs on systems regardless of voltage the full scale input and output full scale voltage requirement changes to >= 0.707 Vrms. Speaker output expected to be half into 8 Ohm for 3.3V codecs.

2. Reference level: 1Vrms. Full Scale is defined as a signal that contains samples at the maximum digital value. .

3. These are two examples. Testing will occur at these two endpoints and anywhere within this envelope. Smaller output voltages (>=120mVrms) are permitted when required by regulatory and safety standards.

Design Notes:

For more information on the Audio Fidelity Testing Policy see http://go.microsoft.com/fwlink/?LinkId=72638.

Exceptions: Not Specified

Business Justification:

Audio fidelity reaches parity with mainstream consumer electronics.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: September 16, 2008

Comments:

AUDIO-0006

Device.Audio.Base.FloatSupport

Target Feature: Device.Audio.Base

Title: Audio subsystem that implements native support for float32 and float64 supports it correctly

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

Microsoft recommends native support for 32-bit and 64-bit floating-point data types but does not require it at this time. However, if native support is implemented, it must be supported in the following way: float audio data must be full scale between -1.0 and 1.0 (with silence at 0.0). This range allows the full use of 23 bits for the mantissa, while achieving the best precision. The Windows audio engine processes float audio data as normalized into this range.

Design Notes: The audio system uses float for processing; converting between integer and float complicates the design and consumes unnecessary CPU cycles.

Exceptions: Not Specified

Business Justification:

Better integration between audio hardware and the OS audio system is important to ensure the best user experience

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 7/17/2008

Comments:

AUDIO-0032

Device.Audio.Base.FullDuplexOperation

Target Feature: Device.Audio.Base

Title: Audio subsystem supports full duplex operation

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM
- Windows Server 2008 Release 2 x64
- Windows 8 Server x64

Description:

Full duplex audio is essential to support emerging communications applications such as IP telephony, conferencing, and network gaming. These applications require the audio system to play back and record simultaneously.

At least one audio subsystem in a PC system must support full-duplex operation.Secondary audio subsystems may be added to the system that support only half-duplex operation.

Exceptions: Not Specified

Business Justification:

: Full duplex audio is essential to support emerging communications applications such as Internet Protocol (IP) telephony, conferencing, and network gaming. These applications require the audio system to play back and record simultaneously. The following requirements ensure that full duplex operation is available and performance is consistent across implementations.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 9/17/2008

Comments:

AUDIO-0024

Device.Audio.Base.HDAudioRemoveDevicePowerState

Target Feature: Device.Audio.Base

Title: HD Audio Codec Driver Must Not Leave Function Group in D3Cold State Upon Unload

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

By the exit of the IRP handler for IRP_MJ_PNP/IRP_MN_REMOVE_DEVICE, an HD Audio Codec driver must have

(a) remembered or discovered the current power state of the function group and

(b) if that current function group power state was D3 Cold, the driver must have changed it to a different power state. The function group power state upon exit is required to be D3.

Exceptions: Not Specified

Business Justification:

HD Audio devices need to honor specified power states.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 9/17/2008

Comments:

NEW

Device.Audio.Base.IMiniportWaveRTStreamNotification

Target Feature: Device.Audio.Base

Title: WaveRT drivers support pull mode audio streaming technology by implementing the IMiniportWaveRTStreamNotification interface.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64

- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM
- Windows Server 2008 Release 2 x64
- Windows 8 Server x64

Description:

By supporting this functionality in the driver, future Microsoft Windows operating systems will be able to employ more efficient techniques for supplying and retrieving data buffers from the audio device. In turn, this will reduce the overall audio latency on the system.

Design Notes:

IMiniportWaveRTStreamNotification

The IMiniportWaveRTStreamNotification interface augments the IMiniportWaveRTStream interface, providing additional methods to facilitate DMA driver event notifications. The port driver accesses this interface by querying the IMiniportWaveRTStream interface (via QueryInterface) that it received by calling the IMiniportWaveRT::NewStream method. IMiniportWaveRTStreamNotification inherits from the IUnknown interface.

IMiniportWaveRTStreamNotification is not supported in operating systems earlier than Windows Vista.

In addition to the methods that IMiniportWaveRTStreamNotification inherits from the IUnknown interface, IMiniportWaveRTStreamNotification supports the following methods:

- IMiniportWaveRTStreamNotification::AllocateBufferWithNotification
- IMiniportWaveRTStreamNotification::FreeBufferWithNotification
- IMiniportWaveRTStreamNotification::RegisterNotificationEvent
- IMiniportWaveRTStreamNotification::UnregisterNotificationEvent

In addition to the above, the driver INF will also need to be modified with the following changes to support event notifications:

1. Use AddReg directive to reference a new *add-registry-section* to add endpoint property keys. This step can be skipped if such a section already exists. The following example adds a new *add-registry-section* (HDAudio.EPProperties.AddReg) under HdAudModel.PrimarySpeakerTopo:

[HdAudModel.PrimarySpeakerTopo]

AddReg=HDAudio.EPProperties.AddReg

2. Next, create an *add-registry-section*, if it does not already exist, to add endpoint property keys to the registry. Example below adds the appropriate property key for the first endpoint declared in this INF:

[HDAudio.EPProperties.AddReg]

HKR,"EP\\0",%PKEY_AudioEndpoint_Association%,,%KSNODETYPE_ANY%

HKR,"EP\\0",%PKEY_AudioEndpoint_Supports_EventDriven_Mode%,0x00010001,0x1

3. In the strings section, add the following section for the value of the property keys used in Step 2 above:

PKEY_AudioEndpoint_Association="{1DA5D803-D492-4EDD-8C23-E0C0FFEE7F0E},2"

PKEY_AudioEndpoint_Supports_EventDriven_Mode="{1DA5D803-D492-4EDD-8C23-E0C0FFEE7F0E},7"

Please reference Windows Driver Kit documentation for details on writing an INF file.

More detailed information on Wave Port Driver for Real-Time Audio Streaming can be found in Windows Hardware Developer Central.

Exceptions: Not Specified

Business Justification:

This is needed to help restore parity with Windows in regards to audio latency. Applications that are timing sensitive in regards to audio such as communication applications may find that the latency increase in later Windows Operating Systems prevent their communication applications from functioning properly.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2009

Comments:

AUDIO-0054

Device.Audio.Base.IndependentInputOutputFormatSelection

Target Feature: Device.Audio.Base

Title: Audio subsystem supports independent selection of input and output sample formats

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

If the built-in or external audio device includes both input and output capabilities, the audio device must support independent selection of input and output sample rates and support concurrent streaming at arbitrarily selected sample rates.

Exceptions: Not Specified

Business Justification:

Full duplex audio is essential to support emerging communications applications such as Internet Protocol (IP) telephony, conferencing, and network gaming. These applications require the audio system to play back and record simultaneously. The following requirements ensure that full duplex operation is available and performance is consistent across implementations.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 7/17/2008

Comments:

AUDIO-0042

Device.Audio.Base.InitiatorTargetBlocktransferSupport

Target Feature: Device.Audio.Base

Title: PCI-based audio device supports initiator, target, and block transfer

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

Full-duplex audio sample transport must be supported by using separate PCI bus-mastering hardware for playback and capture sample streams.

Design Notes: See PCI Local Bus Specification, Revision 2.3 (PCI2.3) or later, "Bus Master."

Exceptions: Not Specified

Business Justification:

Helps ensure compatibility with PCI local bus

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 7/17/2008

Comments:

AUDIO-0028

Device.Audio.Base.JackConnectorStateDescription

Target Feature: Device.Audio.Base

Title: Audio drivers support specific properties to describe state of jack/connector

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

Audio drivers must support the following properties: KSPROPERTY_JACK_DESCRIPTION and KSPROPERTY_JACK_DESCRIPTION2.

* For every endpoint exposed by an HD Audio driver, the driver must respond to a KSPROPERTY_JACK_DESCRIPTION request with a KSJACK_DESCRIPTION structure.

*For every endpoint exposed by an HD Audio driver, the driver must respond to a KSPROPERTY_JACK_DESCRIPTION2 request with a KSJACK_DESCRIPTION2 structure.

*The structures must be populated to accurately reflect the hardware state.

* Third-party jack-presence-detecting drivers use KSJACK_DESCRIPTION.IsConnected for KSPROPERTY_JACK_DESCRIPTION and jackdesc2_presence_detect_capability for KSPROPERTY_JACK_DESCRIPTION2.

Refer to http://msdn.microsoft.com/en-us/library/ff537484(v=VS.85).aspx

Exceptions:

USB audio 1.0

Business Justification:

Jack connector state description helps Windows in correctly routing media to devices.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2009

Comments:

AUDIO-0077

Device.Audio.Base.JackDetection

Target Feature: Device.Audio.Base

Title: Audio devices need to support jack detection

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

Analog and digital jacks need to support jack presence detection, except for USB Audio 1.0 devices and S/PDIF.Third party drivers must advertise this to the OS via KSJACK_DESCRIPTION2.JackCapabilities and relay jack connection state to the OS via KSJACK_DESCRIPTION.IsConnected

Exceptions: Not Specified

Business Justification:

This requirement helps Windows conform to default device heuristics in routing audio streams to appropriate end points.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

AUDIO-0016

Device.Audio.Base.KSPROPERTYAUDIOMIXLEVELTABLE

Target Feature: Device.Audio.Base

Title: Audio driver that implements KSNODETYPE_SUPERMIX correctly implements the KSPROPERTY_AUDIO_MIX_LEVEL_TABLE property

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM
- Windows Server 2008 Release 2 x64
- Windows 8 Server x64

Description:

If a driver implements support forKSNODETYPE_SUPERMIX then that node must correctly support the KSPROPERTY_AUDIO_MIX_LEVEL_TABLE property whose value is a multiple of decibels.

Design Notes: See the Windows Driver kit, "KSNODETYPE_SUPERMIX." The decibel values are documented on MSDN.

Exceptions: Not Specified

Business Justification:

To move the OS and the devices we support into the entertainment space it is important that the systems running our entertainment OS SKUs behave in a manner familiar to the users. Volume control on the PC today differs wildly from system to system and this requirement will help create a more consistent, predictable volume control.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 9/17/2008

Comments:

AUDIO-0039

Device.Audio.Base.KSPROPERTYAUDIOVOLUMELEVEL

Target Feature: Device.Audio.Base

Title: Audio driver that implements KSNODETYPE_VOLUME correctly supports the KSPROPERTY_AUDIO_VOLUMELEVEL property

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM
- Windows Server 2008 Release 2 x64
- Windows 8 Server x64

Description:

If a driver implements support for KSNODETYPE_VOLUME, that node must correctly support the KSPROPERTY_AUDIO_VOLUMELEVEL property whose value is a multiple of decibels.

Design Notes: See the Windows Driver kit, "KSNODETYPE_VOLUME." The decibel values are documented on MSDN.

Exceptions: Not Specified

Business Justification:

To move the OS and the devices we support into the entertainment space it is important that the systems running our entertainment OS SKUs behave in a manner familiar to the users. Volume control on the PC today differs wildly from system to system and this requirement will help create a more consistent, predictable volume control.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 9/17/2008

Comments:

AUDIO-0038

Device.Audio.Base.KSTopologyCompliance

Target Feature: Device.Audio.Base

Title: Audio Device Driver provides kernel streaming topology according to the documentation in the Microsoft Windows Driver Kit

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

Some important examples (not inclusive of all the driver has to adhere to, see the WDK for full disclosure):

Check Pin KsDataRange

If a datarange structure (KSDATARANGE structure) has a "Specifier" value KSDATAFORMAT_SPECIFIER_WAVEFORMATEX, then:

FormatSize must be sizeof(KSDATARANGE_AUDIO).

KSDATARANGE_AUDIO values must have:

SampleFrequency is between 1 Hz and 2,000,000 Hz

BitsPerSample are between 8 and 32 bits.

Check Orphaned Pins

All pins must have at least one internal connection and none can be orphaned.

Node Verifications:

All Nodes Pin I/O Count

For all node types specified in the MSDN, the following is a list of the required number of input and output connections.

KS Node	Number of Inputs	Number of Outputs
KSNODETYPE_MUX	>= 1	1
KSNODETYPE_SUM	> =1	1
KSNODETYPE_DEMUX	1	> 1
KSNODETYPE_ACOUSTIC_ECHO_CANCEL	2	2
KSNODETYPE_DEV_SPECIFIC	Not Specified	Not Specified
All other nodes	1	1

Check Orphaned Nodes

Checks to make sure that all nodes have connections to other nodes and no orphaned nodes are left.

All channel properties, including channels returning Boolean values, such as mute, must include one MembersHeader, and the MembersCount field must accurately describe the number of channels. This requirement also applies to mono controls.

Test your device driver with the KS Topology test to ensure compliance with this requirement.

Exceptions: Not Specified

Business Justification:

This helps ensure driver compatibility with Windows.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 9/17/2008

Comments:

AUDIO-0052

Device.Audio.Base.NoHiddenStreamRouting

Target Feature: Device.Audio.Base

Title: Audio device relies on Windows to support various throughput scenarios

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

Audio device must not rely on analog circuitry designed to mix audio signals between the various device inputs and outputs or signals routing from one DAC to multiple output connectors or from multiple input connectors to one ADC for playback and capture operations in other ways than defined in the UAA HD Audio Pin Config Guidelines.

The device must be able to rely on the operating system to support various throughput and monitoring scenarios and provide independent or otherwise pre-defined by Microsoft audio device implementation guidelines audio connectivity on the PC.

This requirement does not prohibit a codec from having a mixer, it implies that the codec must not rely on the mixer for I/O.

This requirement does not prohibit hardware from supporting offloading, provided the HW audio capabilities are exposed in a manner consistent with Windows.

DACs and ADCs must have direct I/O from jacks to the operating system.

Design Notes: The PC streaming audio device should behave like a transparent entity without any processing, including mixing paths in the analog domain that the operating system is unaware of. This enables predictability and a uniform audio experience for all Windows users.

Exceptions: Not Specified

Business Justification:

This helps Windows make appropriate decisions on media stream routing to ensure a predictable user experience.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: July 13, 2008

Comments:

AUDIO-0003

Device.Audio.Base.NoUncontrollableStreamRouting

Target Feature: Device.Audio.Base

Title: Audio driver does not perform undiscoverable stream redirection or perform other hidden stream handling that is unknown and/or uncontrollable by user or the Windows Audio System.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

Audio driver does not perform undiscoverable stream redirection or perform other hidden stream handling that is unknown and/or uncontrollable by user or the Windows Audio System.

Audio driver does not perform hidden stream redirection, routing, switching, splitting, mixing, muxing to other exposed or hidden logical audio devices, applications or other entities but ensures the audio stream from the audio system endpoint for a particular logical device is only directed to that particular logical device that the application is streaming to, as set by the Windows user in the Windows Sound control panel.

The handling of streams is an application layer feature and must not be performed by audio drivers in fashions not discoverable to Windows.

This requirement does not apply to hardware processing as described by KSNODETYPE_AUDIO_ENGINE.

Design Notes: A Windows friendly audio driver exposes the capabilities and peculiarities of the independent logical audio endpoints the audio device or system audio implementation supports. The audio driver provides other hardware specific support enabling use of the device on Windows but the audio driver does not enact policies on the streams coming from the Windows application layer destined for a selected logical audio endpoint on the device or other devices.

Exceptions: Not Specified

Business Justification:

The move towards a discoverable audio subsystem empowers WIndows applications and Windows itself to provide flexible and powerful media streaming policies for user/OEm control. This initiative abstracts the choices of where streams go from the hardware and kernel mode driver code where it is hidden and uncontrollable by the Windows user in most cases. This requirement will enable a more powerful, feature rich Windows audio system in the future and also allow more control to the applications run

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2009

Comments:

AUDIO-0053

Device.Audio.Base.NoUndiscoverableDevice

Target Feature: Device.Audio.Base

Title: Audio device does not use undiscoverable and/or uncontrollable non-linear audio processing that is on by default.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x64
- Windows 7 Client x86
- Windows 8 Client ARM

Description:

Some applications depend on accessing unaltered audio data to provide the intended user experience. Products must be capable of providing audio data from Windows applications to listeners ear, or from users mouth to Windows application that is not modified in any way by the Audio Device in hardware, firmware or 3rd party software.This means that there shall be no undiscoverable or uncontrollable hardware, firmware or 3rd party software-based AGC, AEC, Beam Forming, Noise suppression or anything else that significantly alters the audio samples (e.g. non-linear processing) from/to the device.

Processing of this type is allowable on audio recording or playback streams if a means is provided for users to disable the processing on their systems, either by exposing the effects as APOs or through another solution. Once disabled by a user, processing must remain off on that product until a user turns it back on.

There is an exception for processing that protects system reliability. This may be on by default and not provide a disable mechanism. Companies that implement reliability effects must ensure the processing elements are reliable and do not pose compatibility issues with the wide range of Windows application needs.

Digital effects may default to enabled or disabled on first use.

Exceptions: Not Specified

Business Justification:

Ensures compatibility with Windows

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2009

Comments:

AUDIO-0083

Device.Audio.Base.PCMNonPCMForSPDIF

Target Feature: Device.Audio.Base

Title: Audio device supports S/PDIF output for PCM and non-PCM data streams

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

If S/PDIF output is implemented, such Audio device needs to support both PCM and non-PCM data streams.

Design Notes: See recommended requirements in the Universal Audio Architecture UAA Hardware Design Guidelines at http://go.microsoft.com/fwlink/?LinkId=50734.

Exceptions: Not Specified

Business Justification:

The ability to provide connectivity to AVRs with the prevalent SPDIF protocol support enables key entertainment scenarios like playing back DVD with compressed multi-channel audio content.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 7/17/2008

Comments:

AUDIO-0041

Device.Audio.Base.PowerManagement

Target Feature: Device.Audio.Base

Title: Audio device complies with related power management specifications

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

Audio devices must comply with Audio Device Class Power Management Reference Specification, Version 1.0, which provides definitions of the OnNow device power states (D0D3) for these devices. The specification also covers the device functionality expected in each power state and the possible wake-up event definitions for the class. The device and driver must implement support for power state D3. Support for other device power management states is optional.

For implementation details, refer to Audio Device Power Management Reference Specification, Version 1.0, at http://go.microsoft.com/fwlink/?LinkId=58377.

Exceptions: Not Specified

Business Justification:

This is needed for compatibility with Windows.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 9/17/2008

Comments:

AUDIO-0026

Device.Audio.Base.ProperUSBDescriptors

Target Feature: Device.Audio.Base

Title: USB audio device to properly set descriptor to indicate the purpose of device

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

USB audio device must properly set descriptor to indicate the purpose of device according to the USB spec http://www.usb.org/developers/devclass_docs/termt10.pdf

Exceptions: Not Specified

Business Justification:

Industry spec conformance

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 9/17/2008

Comments:

NEW

Device.Audio.Base.RealtimeDriversSupportStandardLoopedStreaming

Target Feature: Device.Audio.Base

Title: KS category realtime drivers need to support at least standard looped streaming. Other KS category audio drivers need to support standard streaming.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86

- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM
- Windows Server 2008 Release 2 x64
- Windows 8 Server x64
- Windows Vista Client x86
- Windows Vista Client x64

Description:

To enable user to take advantage of the most efficient lowest latency path and to play audio correctly on the windows platform, the windows audio engine requires:

* KS category realtime drivers to support at least standard looped streaming.

* Other KS category audio drivers to support at least standard streaming.

Exceptions: Not Specified

Business Justification:

Enable low latency audio

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 9/17/2008

Comments:

NEW

Device.Audio.Base.RecordPlaybackBasicPerformance

Target Feature: Device.Audio.Base

Title: Digital audio record and playback meet basic performance requirements for audio device

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x64
- Windows Server 2008 x86
- Windows 8 Client ARM
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Server x64

- Windows Vista Client x86
- Windows Vista Client x64

Description:

The audio device must be able to meet the minimum performance requirements as identified in the following table. Measurements are performed electronically (not in-air) for all the below defined device types when they exist on the device/system. Frequency sweep and/or multi-tone test methods may be used when appropriate. The Device Types in the tables reflect those defined by the Intel HD Audio Spec and the Microsoft HD Audio Pin Configuration Programming Guidelines.

Note that device submissions must pass the Audio Fidelity tests.

ALL PC SKUs:

Device Type	Requirement	Value	Frequency range
Analog Line Output Jack	THD+N	>= 65 dB	[20 Hz,20 KHz]
	Dynamic range with signal present	>= 80 dB A-weight	[20 Hz,20 KHz]
	Magnitude Response	<=+-.25dB Ripple (.5dB peak to peak delta); 1 dB at upper band edge, 3dB at lower band edge	[20 Hz,20 KHz]
	Sampling frequency accuracy	0.02%.	
	Line output cross-talk	>= 50 dB	[20 Hz,15 KHz]
	Full scale output voltage	>= 1 Vrms	
	Noise level during system activity	>= 80 dB FSA-weight	
	Interchannel phase delay	30 degrees or 12.5 microseconds, whichever is greater	[20 Hz,20 KHz]
Analog Speaker Output Jack (Example: 125mW into 8 Ohm load)	THD+N	>= 65 dB	[20 Hz,20 KHz]
	Dynamic range with signal present	>= 80 dB A-weight	[20 Hz,20 KHz]
	Magnitude Response	<=+- .25dB ripple (.5dB peak to peak delta), 1 dB at upper band edge, 3dB at lower band edge	[20 Hz,20 KHz]
	Sampling frequency accuracy	0.02%.	
	Line output cross-talk	>= 50 dB	[20 Hz,15 KHz]
	Full scale output	>= 1 Vrms	

	voltage		
	Noise level during system activity	>= 80 dB FS A-weight	
	Interchannel phase delay	30 degrees or 12.5 microseconds, whichever is greater	[20 Hz,20 KHz]
Analog Headphone Out Jack	THD+N	>= 65 dB >= 45 dB at 32 Ohm Load	[100 Hz,20 KHz]
	Dynamic range with signal present	>= 80 dB A-weight >= 60dB A-weight at 32 Ohm Load	[100 Hz,20 KHz]
	Magnitude Response	<=+- .25dB ripple (.5dB peak to peak delta), 1 dB at upper band edge, 3dB at lower band edge	[100 Hz,20 KHz]
	Sampling frequency accuracy	0.02%.	
	Headphone output cross-talk	>= 50 dB	[20 Hz,15 KHz]
	Full scale output voltage	>= 1 Vrmsat 300 Ohm load >= 300 mVrms at 32 Ohm load	
	Noise level during system activity	>= 80 dB FS A-weight	
	Interchannel phase delay	30 degrees or 12.5 microseconds, whichever is greater	[20 Hz,20 KHz]
Analog Line In Jack	THD+N	>= 65 dB FS	[20 Hz,20 KHz]
	Dynamic range with signal present	>= 80 dB A-weight	[20 Hz,20 KHz]
	Magnitude Response	<=+- .25 dB ripple (.5dB peak to peak delta), 1 dB at upper band edge, 3dB at lower band edge	[20 Hz,20 KHz]
	Sampling frequency accuracy	0.02%.	
	Full scale input voltage	>= 1 Vrms	
Analog Microphone In Jack	THD+N	>= 65 dB FS	[100 Hz,20 KHz]
	Dynamic range with signal present	>= 70 dB A-weight	[100 Hz,20 KHz]
	Magnitude Response	<= +-.25 dB ripple (.5dB peak to peak delta), 1 dB at upper band edge, 3dB at lower band edge	[100 Hz,20 KHz]
	Sampling frequency accuracy	0.02%.	
	Full scale input voltage	>= 0.1 Vrms	

1. For codecs on mobile systems regardless of voltage the full scale input and output full scale voltage requirement changes to >= 0.707 Vrms. Speaker output expected to be half into 8 Ohm for 3.3V codecs. More audio fidelity friendly analog power supply voltages in the range of 4V-5V are recommended.

2. Reference level: 1Vrms. Full Scale is defined as a signal that contains samples at the maximum digital value.

3. These are two examples. Testing will occur at these two endpoints and anywhere within this envelope. Smaller output voltages (>=120mVrms) are permitted when required by regulatory and safety standards.

Exceptions: Not Specified

Business Justification:

This requirement enables Microsoft to drive the PC towards par with CE device audio fidelity for entertainment purposes and to usable levels of audio fidelity for high quality voice communication, speech recognition and gaming.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 7/30/2008

Comments:

AUDIO-0025

Device.Audio.Base.ReportSupportedProperties

Target Feature: Device.Audio.Base

Title: The audio driver correctly reports all supported properties

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

If the audio device and driver support additional properties, the audio driver must report all supported properties correctly to optimize speaker configuration.

Design Notes: If the driver has analog output, the driver exposes a DAC node in its topology.The driver must then support KSPROPSETID_Audio and KSPROPERTY_AUDIO_CHANNEL_CONFIG on that node through a filter handle. The driver then correctly reports support for this property (that is, BASIC_SUPPORT call with KSP_NODE.[node ID of DAC] must succeed) and reports the _GET and _SET capabilities. See the Windows Driver Kit, "Streaming Devices."

Exceptions: Not Specified

Business Justification:

The PC in the living room as the Media Center hub remains an important goal to Microsoft. This requirement will improve the user experience setting up a PC in the living room. Moving the OS into the home includes attaching it to various speaker setups and the OS needs information to allow the correct speaker setting to be found.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 9/17/2008

Comments:

AUDIO-0033

Device.Audio.Base.RestartWithinASpecifiedDuration

Target Feature: Device.Audio.Base

Title: Audio device restarts working within a delay of 10 seconds for S1-S3 and 15 seconds for S4.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM
- Windows Server 2008 Release 2 x64
- Windows 8 Server x64

Description:

Audio device restarts working within a delay of 10 seconds for S1-S3 and 15 seconds for S4.

Refer to: http://download.microsoft.com/download/1/6/1/161ba512-40e2-4cc9-843a-923143f3456c/AudPMSpc.rtf

Exceptions: Not Specified

Business Justification:

User experience in power state changes.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 9/17/2008

Comments:

NEW

Device.Audio.Base.SamplePositionAccuracy

Target Feature: Device.Audio.Base

Title: Audio driver reports render sample position with defined accuracy for stream synchronization

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64

Description:

HD Audio drivers must be able to report the current position of the buffer being rendered with an accuracy of 1/20000th of a second, or with frame accuracy (as defined in the HD Audio specification) the current position of the buffer being rendered, in relation to the samples given to the codec. This applies to both the compressed and uncompressed data.

This requirement does not imply that the compressed and uncompressed streams are synchronized. The requirement covers both types of streams but that is the extent of the interaction between the stream types.

For USB audio devices, the required accuracy is 1ms for USB Audio 1.0 implementations and 0.125ms for USB Audio 2.0 implementations.

For all other audio devices, the required accuracy is 1ms.

Exceptions: Not Specified

Business Justification:

Justification: To be able to ensure the most accurate synchronization between audio streams and between audio and video streams the position information provided by the audio driver must be accurate. This accuracy will help improve the PC media consumption user experience to the level of Consumer Electronics which is important as the PC moves into the home entertainment space. This requirement also helps RTC features like Automatic Echo Cancellation work better and provide an improved user experience in the business PC market where VOIP

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 7/17/2008

Comments:

AUDIO-0027

Device.Audio.Base.SamplingAccuracy

Target Feature: Device.Audio.Base

Title: Audio sampling position error needs to be less than 0.02 %

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

Audio sampling position error needs to be less than 0.02 % in order to help accurate echo cancellation when using the device for communication purposes. Sampling rates for capture or render shall be one of the following: 48kHz, 44.1kHz or 16kHz.

Exceptions: Not Specified

Business Justification:

Ensuring accuracy of sampling position

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 9/17/2008

Comments:

NEW

Device.Audio.Base.SPDIFSupportMinimumSamplingRate

Target Feature: Device.Audio.Base

Title: Audio subsystem that includes an S/PDIF port supports minimum sampling rates

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

The audio subsystemmust support renderingat the followingsampling rates over the S/PDIF port:

44.1 kHz

48 kHz

88.2 kHz (optional)

96 kHz

However, ifnative hardware support of sampling rates in the audiosubsystemis less than 96kHz or only up to 48kHz, rendering over S/PDIF port must be supported only for 44.1 and 48kHz.

Exceptions: Not Specified

Business Justification:

Supporting a larger range of sampling rates over S/PDIF ports would ensure compatibility with content encoded using newer codecs like WMA Pro which support a larger range of sampling rates that provide higher fidelity audio. Multichannel content is carried over S/PDIF link in its compressed form & if the audio driver only supports rendering a small range of sampling rates over S/PDIF port, newer content at higher sampling rates would need to be decoded, resampled & re-encoded resulting in a lower quality experience & potentially introducing latency issues.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 7/17/2008

Comments:

AUDIO-0034

Device.Audio.Base.TimeSynchronizedSampleRates

Target Feature: Device.Audio.Base

Title: Audio subsystem supports time-synchronized sample rates if both input and output capabilities are present

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

If the built-in or external audio device includes input and output capabilities, the timing relationship between input and output sample rates must remain constant (that is, no drift). For example, if 8 kHz is selected for both input and output sampling rate, audio hardware must ensure that the sampling rate for input and output is precisely matched.

Further, when input and output sample rates are set to integer ratios, the actual sample rate ratios must match (that is, no drift). For example, if an 8-kHz input sampling rate and a 32-kHz output sampling rate are selected, the ratio of actual sampling rates must be precisely 8:32. This requirement can be accomplished by ensuring that both input and output sampling rates are derived from the same clock and that sample rate divisors are set correctly.

Design Notes: This requirement helps ensure that AEC and NS algorithms maintain performance and convergence. This requirement does not apply to inputs and outputs where the input source sets a clock such as a digital S/PDIF input.

Exceptions: Not Specified

Business Justification:

Full duplex audio is essential to support emerging communications applications such as Internet Protocol (IP) telephony, conferencing, and network gaming. These applications require the audio system to play back and record simultaneously. The following requirements ensure that full duplex operation is available and performance is consistent across implementations.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 9/17/2008

Comments:

AUDIO-0043

Device.Audio.Base.TipRing

Target Feature: Device.Audio.Base

Title: Audio jacks must use defined tip/ring connections to ensure proper audio channel path.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

If the system exposes a multi-channel analog logical device, then each analog output must have independent DAC resource.

The audio jacks must use defined tip/ring connections to ensure proper audio channel path.

Audio line in	**Left line in**	**Tip of connector**	
	Right line in	Ring of connector	
Audio line out (front left and right)	Left front out	Tip of connector	
	Right front out	Ring of connector	
Microphone in (mono)	Microphone in	Tip of connector	
	4V Bias	Ring of connector	
Microphone in (stereo)	Left Mic in	Tip of connector	
	Right Mic in	Ring of connector	
Side surround left and right out	Left surround	Tip of connector	
	Right surround	Ring of connector	
Rear surround left and right out	Left back	Tip of connector	
	Right back	Ring of connector	
Center speaker & LFE (subwoofer) out	Front center out	Tip of connector	
	LFE (subwoofer) out	Ring of connector	

Design Notes: See the Intel HD Audio Specification, Revision 1.0, and Microsoft HD Audio Pin Configuration Programming Guidelines.

See recommended requirements in the Universal Audio Architecture UAA Hardware Design Guidelines at http://go.microsoft.com/fwlink/?LinkId=50734.

Exceptions: Not Specified

Business Justification:

Enables uniform and predictable audio connectivity experience based on standardized audio connectors.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: July 17, 2008

Comments:

AUDIO-0002

Device.Audio.Base.TwoDMAEnginesAndConnections

Target Feature: Device.Audio.Base

Title: Audio device that supports digital output has at least two independent DMA engines and a separate physical connection for digital output using one of the available DMA engines

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM
- Windows Server 2008 Release 2 x64
- Windows 8 Server x64

Description:

The audio controller that supports digital output must have two independent DMA engines, one that can be used for wave output and the other to make it possible to support AC-3 over S/PDIF at the same time. The digital audio output capability is supported through a separate physical connector identified for digital audio output and used only for digital audio output.

Given physical limitations, PC system may have limited input and/or output streams. Secondary HD audio controller in such systems may implement fewer DMA engines.

Design Notes: With support for two independent DMA engines, a different signal can be streamed to each connector simultaneously. For example, sending a DVD player application's Dolby Digital stream to the S/PDIF connector while simultaneously sending a voice conversation to the analog connectors. The S/PDIF port needs to be represented as its own audio "device" separate from the analog outputs. Therefore, it will have its own policy configuration, including the preferred data format for a specific signal.

Exceptions: Not Specified

Business Justification:

Justification: Ease of use, better user experience for eHome Media Center type SKUs that deal with digital output (typically AC-3 data from DVDs to home receiver systems).

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 7/17/2008

Comments:

AUDIO-0031

Device.Audio.Base.VoiceCommunicationUAA

Target Feature: Device.Audio.Base

Title: Voice Communication devices must be UAA compliant audio devices with an appropriate communication centric form factor exposed to the operating system through available mechanisms

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64

Description:

Voice communication devices must expose themselves to the operating system as Universal Audio Architecture (UAA)-compliant audio devices. The devices must include an appropriate communication-centric form factor, such as a headset or handset, that Windows can recognize and support.

Webcams that have a microphone expose a microphone form factor device by using USB descriptors according to the USB Audio 1.0 specification.

Aggregate USB audio devices (that is, audio devices that have input and output on the same device) expose themselves to Windows as handset or headset device types by using USB descriptors according to the USB Audio 1.0 specification.

For integrated and PCI audio devices that have analog jacks, the jacks are exposed accurately by using the pin configuration registers and driver topology. The driver topology uses relevant KSNODETYPE descriptors.

There are two options to identify devices as communication-class devices for audio testing with Windows.

- Audio endpoint devices of certain KSNODETYPE descriptors are automatically treated as communication devices for testing purposes. Custom drivers should map to one of these KSNODETYPEs for the device to be recognized as a communication-class device. The following list is a full list of communication-class KSNODETYPE descriptors:

KSNODETYPE_MICROPHONE

KSNODETYPE_DESKTOP_MICROPHONE

KSNODETYPE_PERSONAL_MICROPHONE

KSNODETYPE_OMNI_DIRECTIONAL_MICROPHONE

KSNODETYPE_MICROPHONE_ARRAY

KSNODETYPE_PROCESSING_MICROPHONE_ARRAY

KSNODETYPE_COMMUNICATION_SPEAKER

KSNODETYPE_HANDSET

KSNODETYPE_HEADSET

KSNODETYPE_SPEAKERPHONE_NO_ECHO_REDUCTION

KSNODETYPE_ECHO_SUPPRESSING_SPEAKERPHONE

KSNODETYPE_ECHO_CANCELLING_SPEAKERPHONE

KSNODETYPE_PHONE_LINE

KSNODETYPE_TELEPHONE

KSNODETYPE_DOWN_LINE_PHONE

Microsoft class drivers map device types to these KSODETYPE descriptors based on information from device hardware or firmware. For example, the Microsoft USB Audio class driver directly maps the USB Audio terminal types that are defined in tables 2-2, 2-4, and 2-5 of the Universal Serial Bus Device Class Definition for Terminal Types, Revision 1.0, March, 1998 to the corresponding KSNODETYPE descriptors above, with three exceptions that do not clearly imply a communication function. These exceptions are the following:

Terminal Type	Code	I/O	Description
Input Undefined	0x0200	I	Input terminal, undefined type
Bi-directional Undefined	0x0400	I/O	Bi-directional terminal, undefined type
Telephony Undefined	0x0500	I/O	Telephony terminal, undefined type

To see the Universal Serial Bus Device Class Definition for Terminal Types, Revision 1.0, March, 1998, visit the following website:

http://www.usb.org/developers/devclass_docs/termt10.pdf

Communication-class USB audio device manufacturers must use one of the mapped terminal types to be tested against communication class requirements and to be used correctly in Windows.

Other drivers may choose different mapping criteria. As long as they map to a KSNODETYPE descriptor that is listed above, the drivers will be considered communication-class during testing. For information on expressing KSNODETYPE descriptors in a WDM driver, see the following website:

http://msdn.microsoft.com/en-us/library/ms790325.aspx

- KSNODETYPE mappings are the preferred solution. However, if these mappings are not sufficient, it is possible to declare a given device as communication-class by adding the .inf driver.To use the .inf method, follow these steps:

1. Use the **AddReg** directive to reference a new add-registry section to add endpoint property keys. This step can be skipped if such a section already exists. The following example adds a new Endpoint.AddReg add-registry sectionunder the USBAudio.MyDevice.Interfacesection/method/other element:

[USBAudio.MyDevice.Interface]

AddReg=Endpoint.AddReg

2. Next, create an add-registry section, if it does not already exist, to add endpoint property keys to the registry. The following example adds the appropriate property key for the first capture endpoint that is declared to be a communication device. The capture endpoint is declared in this INF method for the KSNODETYPE_ANY node:

[Endpoint.AddReg]

HKR,"EP\\0",%PKEY_AudioEndpoint_Association%,,%KSNODETYPE_ANY%

HKR,"EP\\0",%PKEY_Endpoint_EndpointRoleAffinity%,0x00010001,0x00000204

Note that there are three possible valid values for this key, based on whether the device is a render device, a capture device, or both:

- 0x00000104 A render device of the associated KSNODETYPE descriptor would be tested as a communication device.
- 0x00000204 A capture device of the associated KSNODETYPE descriptor would be tested as a communication device.
- 0x00000304 A both render and capture device of the associated KSNODETYPE descriptor would be tested as a communication device.

3. In the strings section, add the following section for the value of the property keys that are used in step 2:

PKEY_AudioEndpoint_Association="{1DA5D803-D492-4EDD-8C23-E0C0FFEE7F0E},2"

PKEY_Endpoint_EndpointRoleAffinity = "{b3f8fa53-0004-438e-9003-51a46e139bfc},13"

For more information about writing an INF file, see the Windows Driver Kit documentation.

Exceptions: Not Specified

Business Justification:

Ensures compatibility with Windows

Scenarios:

Scenario It is very easy for me to use my computer for Internet voice communication with my family and colleagues. My computer's audio devices tell Windows about their role, and my VOIP application can automatically find the right device for the job without any input from me. When I use my computer for voice communication with my family or colleagues, I can easily understand them and they can easily understand me. I can take and place my calls regardless of other audio activity on my computer. With my computer's audio solution, I have a quality calling experience comparable to today's wired telephones or speaker phones.

Success Metric: Not Specified

Enforcement Date: 6/1/2009

Comments:

AUDIO-0081

Device.Audio.Base.VolumeControlsIsLinear

Target Feature: Device.Audio.Base

Title: Audio driver volume controls are linear

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

Signal response (as measured by electrical ordigital signal level) changes in linearity with the volume control within 3% tolerance.Forexample:, a volume slider change of 10dB should result in an measured volume change within 10dB + or - 0.3dB

Exceptions: Not Specified

Business Justification:

This helps audio applications manage echo cancellation and also help provide a consistent audio user experience.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 9/17/2008

Comments:

NEW

Device.Audio.Base.VolumeGranularity

Target Feature: Device.Audio.Base

Title: Audio solution that implements topology volume nodes uses a resolution equal to or better than 1.5 dB

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

Topology volume nodes must have a resolution equal to or better than 1.5dB and implement driver support for volume level as defined in the Windows Driver Kit.

Design Notes: See the Windows Driver Kit, "KSPROPERTY_AUDIO_VOLUMELEVEL."

Exceptions: Not Specified

Business Justification:

Volume behavior on the PC should behave like Consumer Electronics devices to enable an easier transition into the entertainment space. Innovative OS features like AGC (Automatic Gain Control) is supported easier with this requirement in place.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 9/17/2008

Comments:

AUDIO-0037

Device.Audio.Base.WAVEFORMATEXTENSIBLESupport

Target Feature: Device.Audio.Base

Title: Audio device driver supports WAVEFORMATEXTENSIBLE

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM
- Windows Server 2008 Release 2 x64
- Windows 8 Server x64

Description:

The audio device driver must support the WAVEFORMATEXTENSIBLE format.

Design Notes: Due to the audio system design in Windows and the availability of a Local Effect (LFX) insert in the audio engine we need to ensure consistency and simplification for audio drivers and the format container that they should expect.

Exceptions: Not Specified

Business Justification:

Ensures consistency and simplification for audio drivers

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 9/17/2008

Comments:

AUDIO-0047

Device.Audio.Base.WaveRTConformance

Target Feature: Device.Audio.Base

Title: Audio device is designed to be WaveRT-port-friendly

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x86

- Windows 7 Client x64
- Windows 8 Client ARM
- Windows Server 2008 Release 2 x64
- Windows 8 Server x64

Description:

A UAA HD Audio-compatible implementation meets this requirement automatically. To be considered WaveRT port friendly, the audio subsystem must support the following:

*Cyclic DMA engine with a scatter-gather list.

*Position register that is separate from other hardware registers (can be a copy).

*Ability to split samples between pages.

*Ability to loop on buffers without software intervention.

This requirement does not apply to external or internal USB audio, bluetooth and 1394 audio devices.

Design Notes: See "A Wave Port Driver for Real-Time Audio Streaming" at http://go.microsoft.com/fwlink/?LinkId=40502.

Exceptions: Not Specified

Business Justification:

This improves reliability of audio drivers by avoiding kernel mode audio processing.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: July 17, 2008

Comments:

AUDIO-0010

Device.Audio.Base.WaveRTImplementation

Target Feature: Device.Audio.Base

Title: Audio device driver is based on the Windows WaveRT miniport WDM driver model

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x64
- Windows 7 Client x86
- Windows 8 Client ARM

- Windows Server 2008 Release 2 x64
- Windows 8 Server x64

Description:

Integrated or discrete audio device driver must be based on the Microsoft Windows WaveRT miniport WDM driver model.Requirement details are defined in the white paper titled "A Wave Port Driver for Real-Time Audio Streaming."

The legacy portsWaveCyclic or WavePCI are not used to support the audio device on Windows.

For device technologies such as USB Audio 1.0 based devices where the audio driver model is not specifically called out, any WDM audio driver model is allowed.

If the audio processing is offloaded to hardware, it is acceptable to use the WaveCyclic driver model.

See the white paper titled "A Wave Port Driver for Real-Time Audio Streaming."

Exceptions: Not Specified

Business Justification:

Glitch Resilient OS features combined with simple and effective WaveRT drivers running on UAA compliant hardware creates the best possible media playback and recording experience possible on Windows.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: June 01, 2009

Comments:

AUDIO-0001

Device.Audio.Base.ZeroGlitch

Target Feature: Device.Audio.Base

Title: Audio devices do not glitch during multiple simultaneous streaming

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM
- Windows 8 Server x64

Description:

Audio device can support 12 simultaneous streams that all make it to the jack with zero glitches both in audio software and offload hardware during such simultaneous streaming. Systems must pass this requirement with audio effects enabled, if that is the default configuration for the product.

Exceptions: Not Specified

Business Justification:

Ensures good audio experience for the end user when using multiple audio streams

Scenarios:

Ensures compatibility with Windows.

Success Metric: Not Specified

Enforcement Date: 9/16/2011

Comments:

NEW

Device.Audio.Bluetooth

Description:

This audio device uses the Bluetooth Audio Driver.

Related Requirements:

- Device.Audio.Bluetooth.AtleastOneProfileSupport
- Device.Audio.Bluetooth.AutomaticReconnectAttempt
- Device.Audio.Bluetooth.ConnectDisconnectBluetooth
- Device.Audio.Bluetooth.DriverReqs
- Device.Audio.Bluetooth.HandsFreeCallControl
- Device.Audio.Bluetooth.HCIDisconnect
- Device.Audio.Bluetooth.MajorMinorClassID

Device.Audio.Bluetooth.AtleastOneProfileSupport

Target Feature: Device.Audio.Bluetooth

Title: Bluetooth Audio Device needs to support at least one of the below profiles.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

The Bluetooth Audio Device must be Bluetooth SIG compliant in at least one of the following profiles:

- A2DP profile as defined in the Advanced Audio Distribution Profile version 1.2 (A2DP_SPEC_V12) Bluetooth SIG specification.
- Hands-Free as defined in the Hands-Free Version 1.5 (HFP 1.5_SPEC_V10) Bluetooth SIG specification.
- AVRCP as defined in the Audio/Video Remote Control Profile Version 1.3 (AVRCP_SPEC_V13) Bluetooth SIG specification.

Exceptions: Not Specified

Business Justification:

Scenarios: Bluetooth SIG compliance allows for the display and usage of the Bluetooth icon and name. This is important for consumer device branding recognition. Assured compatibility with OS supplied components that rely onBluetooth SIGdefined functionality

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2009

Comments:

AUDIO-0060

Device.Audio.Bluetooth.AutomaticReconnectAttempt

Target Feature: Device.Audio.Bluetooth

Title: Bluetooth audio devices paired with a PC will automatically attempt to reconnect to the PC after they are powered up or come back into range.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

The reconnection method must follow the appropriate connection procedures for connection to an Audio Gateway as outlined in the Bluetooth Sig specifications for:

1. Hands-Free Profile

2. A2DP Profile

3. AVRCP Profile

Exceptions: Not Specified

Business Justification:

Scenarios: After powering up a BT Audio device that is within range of the PC it is currently paired with, the BT Audio device will become available for use without any additional button presses on the device or the PC. After coming back into range of the PC, audio starts streaming back to the BT Audio device without any additional button presses being made on the device or the PC.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2009

Comments:

AUDIO-0058

Device.Audio.Bluetooth.ConnectDisconnectBluetooth

Target Feature: Device.Audio.Bluetooth

Title: Bluetooth audio devices support the properties required for Sound Control Panel to control the connection and disconnection of Bluetooth audio devices.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

Bluetooth audio devices support the properties required for Sound Control Panel to control their connection and disconnection.

KSPROPSETID_BtAudio needs to support both:

* KSPROPERTY_ONESHOT_DISCONNECT

* KSPROPERTY_ONESHOT_RECONNECT

More info at http://msdn.microsoft.com/en-us/library/ff537446(VS.85).aspx

Exceptions: Not Specified

Business Justification:

Enable good user experience with bluetooth audio devices.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 9/17/2008

Comments:

NEW

Device.Audio.Bluetooth.DriverReqs

Target Feature: Device.Audio.Bluetooth

Title: Bluetooth Audio Device Driver Requirements

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM
- Windows 8 Server x64

Description:

1. Bluetooth SIG Qualification
The Bluetooth Audio driver must pass the necessary Bluetooth SIG Qualification tests required by the Bluetooth SIG for the profiles listed in the Bluetooth Profiles section above.

2. HID Call Control
The Bluetooth Audio driver must map the Hands-free call control information to the appropriate HID Call Control messages as defined in the Windows 7 HID Call Control specification.

3. Volume Change Notification
If volume change notifications are received from the hands-free profile that indicate the volume level of the Bluetooth audio device has changed, the driver must inform the OS of the volume change, consistent with Section 4.28 of the Bluetooth audio Hands-Free specification, HFP 1.5_SPEC_V10.

Design Notes:

Bluetooth Profiles and SIG Qualification
The Bluetooth Audio Profile documentation is maintained by the Bluetooth SIG. The Bluetooth SIG Qualification is owned and defined by the Bluetooth SIG. Qualification is a necessary pre-condition of the necessary intellectual property license for the Bluetooth wireless technology. Detailed information can be found on the Bluetooth SIGs website at http://www.bluetooth.org.

Volume Change Notification
The KSEVENTSETID_AudioControlChange is used by the Bluetooth audio driver to notify the OS that the Bluetooth audio device volume level has changed. Not all Bluetooth Hands-free devices will notify the driver of volume changes made by the user. For those that do, the Bluetooth audio driver must use this event to notify the OS of the change. Note that this is not to be used for AVRCP volume notifications as those volume change notifications are handled by the OS. Additional information on KSEVENTSETID_AudioControlChange can be found at http://msdn.microsoft.com

Exceptions: Not Specified

Business Justification:

The requirements listed here are necessary to provide support for the basic audio functionality included in Windows for Bluetooth audio devices.This functionality has been implemented in the Windows Bluetooth audio class driver and it is important that 3rd party Bluetooth audio drivers comply with required functionality such that the basic Windows audio features function properly.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2009

Comments:

AUDIO-0087

Device.Audio.Bluetooth.HandsFreeCallControl

Target Feature: Device.Audio.Bluetooth

Title: Bluetooth audio devices that utilize the Hands Free profile call control platform will comply with this requirement.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

The documententitled, Call Control Platform API/DDI details the usage and implementation of this functionality. This document is available on http://msdn.microsoft.com.

Design Notes: This requirement applies to 3rd parties who create Hands-Free device drivers and opt to use the Bluetooth Call Control platform API/DDI.

Exceptions: Not Specified

Business Justification:

Current HID design for call control is difficult to implement and non-standardized. With the new call control platform in Windows 8, it becomes significantly easier to allow an application to communicate with the call control buttons on Bluetooth devices, and vice-versa. Developers who use this new API/DDI will save time.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Audio.Bluetooth.HCIDisconnect

Target Feature: Device.Audio.Bluetooth

Title: Bluetooth Audio Devices must complete an HCIDisconnect before powering down

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

This step is required to allow for timely notification to the system that the device is no longer available. This is needed in order toreroute audio to analternate audio sink seamlessly when the Bluetooth audio device is turned off providing for a good end user experience.

Exceptions: Not Specified

Business Justification:

This is needed in order for our stream routing mechanism to function in a timely manner. ScenarioUser is listening to music on desktop speakers. They turn on their BT Headphones. Music is automatically routed to the BT headphones. They then turn off their BT headphones. Music is automatically routed 'in a timer manner' back to the desktop speakers.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2009

Comments:

AUDIO-0061

Device.Audio.Bluetooth.MajorMinorClassID

Target Feature: Device.Audio.Bluetooth

Title: Bluetooth audio devices to expose Major/Minor Class of Device identifier and accurately reflect form factor/primary usage.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

Bluetooth audio devices must expose the proper Major and Minor Class of Device identifier. The values chosen must accurately reflect the form factor and primary usage of the device.

Please refer to the following Bluetooth specifications from the Bluetooth Special Interest Group website:

For A2DP devices, class of device ID is defined in Section 5.3 "SDP Interoperability Requirements" of the Advanced Audio Distribution Profile Specification A2DP_SPEC_V12" document.

For Hands-Free devices, class of device ID is defined in Section 5.5.1 "Class of Device" of the Hands-Free Profile Specification version 1.5 in the "HFP 1.5_SPEC_V10" document.

Exceptions: Not Specified

Business Justification:

This is needed for compatibility with Windows.

Scenarios:

Scenarios: · Windows Communication applications are easily able to find my device by querying for Communication devices. · Windows is properly able to rank my device so that stream routing can send audio to it in a predictable and logical manner. · Windows displays the proper Icons for my BT Audio Device allowing for easy identification in the Audio Control Panel.

Success Metric: Not Specified

Enforcement Date: 6/1/2009

Comments:

AUDIO-0057

Device.Audio.HardwareAudioProcessing

Description:

HardwareAudioProcessing

Related Requirements:

- Device.Audio.HardwareAudioProcessing.AudioHardwareOffloading

- Device.Audio.HardwareAudioProcessing.ETWEvent
- Device.Audio.HardwareAudioProcessing.IMiniport

Device.Audio.HardwareAudioProcessing.AudioHardwareOffloading

Target Feature: Device.Audio.HardwareAudioProcessing

Title: Hardware that supports offloaded audio render processing meets this requirement

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM
- Windows 8 Server x64

Description:

If a hardware solution supports offloaded audio render processing, the driver must expose a KS filter and a single KSNODETYPE_AUDIO_ENGINE node with appropriate pin factories connected.

If a hardware solution supports the offloading of audio render processing, mixing, or decoding, the driver must expose a KS filter. For each rendering path through that filter that supports hardware offloading the driver must expose a single KSNODETYPE_AUDIO_ENGINE node, connecting directly to only the following pin factories:

- two KS sink pin factories
- a single KS source pin factory for reference stream support

If a driver exposes a KSNODETYPE_AUDIO_ENGINE node, the driver and hardware must support base-level functionality.

If a driver exposes a KSNODETYPE_AUDIO_ENGINE node, the driver and hardware must support the following capabilities:

- Audio mixer with at least 3 simultaneous inputs (2 offload and 1 host process)
- Volume and mute capabilities both pre- and post-mixing
- Metering reporting (support for querying per-stream peak values, both pre & post-mix)
    - For stream metering (pre-mixing), metering levels should be reported after the LFX and before volume control
    - For endpoint metering (post-mixing), metering levels should be reported:
        - Before volume control and GFX, *when the GFX is an encoder*

        - After the GFX and before volume control, *when the GFX is not an encoder*

- Reference stream (support for sending the audio stream post-mix back to the Windows audio stack)

- The reference stream provided should be the final output to the audio device, or, if encoding is taking place, just prior to encoding.

If a driver exposes a KSNODETYPE_AUDIO_ENGINE node, the driver must support KSPROPSETID_AudioEngine with certain properties.

If a driver exposes a KSNODETYPE_AUDIO_ENGINE node, the driver must support KSPROPSETID_AudioEngine with the following properties:

- KSPROPERTY_AUDIOENGINE_LFXENABLE
- KSPROPERTY_AUDIOENGINE_GFXENABLE
- KSPROPERTY_AUDIOENGINE_MIXFORMAT
- KSPROPERTY_AUDIOENGINE_PROCESSINGPERIOD
- KSPROPERTY_AUDIOENGINE_DEVICEFORMAT
- KSPROPERTY_AUDIOENGINE_SUPPORTEDDEVICEFORMATS
- KSPROPERTY_AUDIOENGINE_DESCRIPTOR
- KSPROPERTY_AUDIOENGINE_WAVERT_CURRENT_WRITE_POSITION
- KSPROPERTY_AUDIOENGINE_BUFFER_SIZE_RANGE
- KSPROPERTY_AUDIOENGINE_LOOPBACK_PROTECTION
- KSPROPERTY_AUDIOENGINE_VOLUMELEVEL
- KSPROPERTY_AUDIO_VOLUMELEVEL
- KSPROPERTY_AUDIO_MUTE
- KSPROPERTY_AUDIO_PEAKMETER2
- KSPROPERTY_AUDIO_PRESENTATION_POSITION

If a driver exposes a KSNODETYPE_AUDIO_ENGINE node, the driver must expose certain pin factories.

If a driver exposes a KSNODETYPE_AUDIO_ENGINE node, the driver must expose the following pin factories:

- Host process pin factory

        - Must support only a single instance
- Offload pin factory
    - Must support at least two instances
- Loopback pin factory
    - Must support at least a single instance

In addition, the following must be met:

- Loopback pins must:
    - Have a Possible Global Instances of at least 1
    - Support at least 1 instance regardless of what else is going on in the system
- To enable scenarios like cross-fade, offload-capable endpoints must support 1 loopback pin instance + 1 host pin instance + each of the following in isolation, assuming no other offload endpoints are being used at the time:
    - Any required PCM format + Any required PCM format (the same, or different)
    - Any required PCM format + Any required MP3 format
    - Any required MP3 format + Any required MP3 format
    - Any required PCM format + Any required mono or stereo AAC format
    - Any required MP3 format + Any required mono or stereo AAC format
    - Any required mono or stereo AAC format + Any required mono or stereo AAC format
- The loopback pin must support
    - The HW mix format
    - The device format (which can be publically queried from the endpoint property store)

If a hardware solution supports offloaded audio render processing, the same functionality provided in hardware (e.g., processing, effects, etc.) must be available in software as well.

In order to provide a consistent user experience and prevent confusion when a user enables or configures functionality that exists in only hardware or only software, the capabilities provided must be equal in both hardware and software.

If a driver exposes a KSNODETYPE_AUDIO_ENGINE node, the driver must support streaming specific PCM formats.

Description:

If a driver exposes a KSNODETYPE_AUDIO_ENGINE node, the driver must support streaming the following PCM formats on the offload pins:

- Sampling rates:
- 8 kHz
- 11.025 kHz
- 16 kHz
- 22.050 kHz
- 32 kHz
- 44.1 kHz
- 48 kHz
- 88.2 kHz
- 96 kHz
- 176.4 kHz
- 192 kHz
- Bit Depths:
- 8-bit
- 16-bit
- 24-bit
- Channel configurations of 1.0, 2.0,2.1, 3.1, 4.0, 5.0 and 5.1

If hardware supports offloaded audio render processing, the steady-state latency for real-time PCM audio must be less than 20 ms.

If hardware supports offloaded audio render processing, the steady-state latency (as measured between the render KS endpoint and the capture KS endpoint on the same device with all processing disabled and the smallest supported buffer sizes being used) must be less than 20 ms.

If a hardware solution supports offloaded audio render processing, the startup latency must be less than 80 ms for compressed formats and 15ms for PCM.

If hardware supports offloaded audio render processing, the startup latency (as defined as the time between just before KSCreatePin and the audio play position beginning to advance) must be less than 80 ms for compressed formats and 15ms for PCM.

If a hardware solution supports offloaded audio render processing, the hardware must support certain buffer sizes.

If a hardware solution supports offloaded audio render processing, the hardware must support buffer sizes:

- As large as (or larger than) 1 second
- As small as (or smaller than) 10 milliseconds

Times above assume an audio format of 2 channels, 48 kHz, 16-bit PCM.

If a hardware solution supports offloaded audio render processing, the pause or stop latency must be less than 10ms.

If a hardware solution supports offloaded audio render processing, the pause or stop latency (as measured between a pause/stop command and the audio pausing/stopping) must be less than 10ms.

The CPU consumption by an audio driver must be less than 5% and memory usage must be less than 100MB.

An audio driver must not consume more than 5% of the total CPU processing available and must not consume more than 100MB of system RAM.

Systems that support connected standby must support certain codecs and processing capabilities.

In order to ensure a high-quality user experience onsystems that support connected standby where the in-box Windows codecs may be unavailable, the follow codecs and processing functionality are required:

- MP3 decoder
    - all MPEG formats except VBR
    - Decode requirement is limited to mono and stereo formats; decoder must be able to accept multichannel MP3 content, but is only required to decode the stereo content
- AAC decoder

Channel configurations:

Mono

Stereo

3.0

4.0 (L, R, C, Rear)

5.0

5.1

Sample rates:

All AAC sample rates

Bit rates:

All bit rates

Profile levels:

0x28 AAC L1

0x29 AAC L2

0x2A AAC L4

0x2B AAC L5

0x2C High Efficiency AAC Profile L2

0x30 High Efficiency AAC v2 Profile L2

Payload types:

0 Raw

1 ADTS

Audio Object types:

2 AAC LC

5 SBR

29 PS

- Sample rate converter

All decoding and processing functionality provided must be exposed via support for the corresponding KS properties via the Portcls WaveRT driver.

Exceptions: Not Specified

Business Justification:

Help preserve compatibility with windows while conserving battery life on systems that support hardware offload

Scenarios:

This helps compatibility with Windows.

Success Metric: Not Specified

Enforcement Date: 9/17/2011

Comments:

NEW

Device.Audio.HardwareAudioProcessing.ETWEvent

Target Feature: Device.Audio.HardwareAudioProcessing

Title: If a hardware solution supports offloaded audio render processing, then the driver must raise an ETW event to report glitches detected by the hardware audio engine.

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client ARM
- Windows 8 Client x86
- Windows 8 Client x64

Description:

The audio driver needs to raise an Event Tracing for Windows (ETW) event to report glitches detected by the HW Audio Engine. This event should at least include the reason of the glitching and the DMA buffer information.

The following events are defined for the miniport to reports events through Portcls interface callbacks.

typedef enum

{

eMINIPORT_IHV_DEFINED = 0,

eMINIPORT_BUFFER_COMPLETE,

eMINIPORT_PIN_STATE,

eMINIPORT_GET_STREAM_POS,

eMINIPORT_SET_WAVERT_BUFFER_WRITE_POS,

eMINIPORT_GET_PRESENTATION_POS,

eMINIPORT_PROGRAM_DMA,

eMINIPORT_GLITCH_REPORT

} ePcMiniportEngineEvent;

Event type	Parameter 1	Parameter 2	Parameter 3	Parameter 4

IHV specific event types (defined and used by IHVs)	Defined and used by IHVs	Defined and used by IHVs	Defined and used by IHVs	Defined and used by IHVs
Buffer complete	Current linear buffer position	Current WaveRtBufferWritePosition	Data length completed	0
Pin State	Current linear buffer position	Current WaveRtBufferWritePosition	0->KS_STOP 1->KS_ACQUIRE 2->KS_PAUSE 3->KS_RUN	0
Get stream position	Current linear buffer position	Current WaveRtBufferWritePosition	0	0
Set WaveRt buffer write position	Current linear buffer position	Current WaveRtBufferWritePosition received from portcls	Target WaveRtBufferWritePosition received from portcls	0
Get Presentation Position	Current linear buffer position	Current WaveRtBufferWritePosition	Presentation position	0
Program DMA	Current linear buffer position	Current WaveRtBufferWritePosition	Starting WaveRt buffer offset	Data length
Glitch detection	Current linear buffer position	Current WaveRtBufferWritePosition	1:WaveRT buffer under run 2:decoder errors 3: receive the same wavert buffer write pos two in a row	When it's 3, the offending write position

Exceptions: Not Specified

Business Justification:

This helps deliver a good user experience with Windows.

Scenarios:

Ensures compatibility with Windows.

Success Metric: Not Specified

Enforcement Date: 8/8/2011

Comments: Not Specified

Device.Audio.HardwareAudioProcessing.IMiniport

Target Feature: Device.Audio.HardwareAudioProcessing

Title: If a hardware solution supports offloaded audio render processing, the driver must implement IMiniportAudioEngineNode and IMiniportStreamAudioEngineNode

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client ARM
- Windows 8 Client x64
- Windows 8 Client x86

Description:

IMiniportAudioEngineNode contains a list of methods related to the offload KS properties targeting the audio engine node via KS filter handle. A miniport driver's WaveRT miniport class needs to inherit not only from IMiniportWaveRT interface, it also needs to inherit IMiniportAudioEngineNode interface and implement all the defined methods.

IMiniportStreamAudioEngineNode contains a list of methods related to the offload KS properties targeting the audio engine node via pin instance handle. A miniport driver's WaveRT miniport class needs to inherit not only from IMiniportWaveRTStreamNotification interface, it also needs to inherit IMiniportStreamAudioEngineNode interface and implement all the defined methods.

Exceptions: Not Specified

Business Justification:

This helps ensure a good user experience with Windows.

Scenarios:

Ensure compatibility with Windows.

Success Metric: Not Specified

Enforcement Date: 8/8/2011

Comments: Not Specified

Device.Audio.HDAudio

Description:

This audio device uses the HD Audio Driver.

Related Requirements:

- Device.Audio.HDAudio.2AudioChannelsForHDMIorDisplayPort
- Device.Audio.HDAudio.AnalogJackDetection
- Device.Audio.HDAudio.DefaultAssociationNotZero

- Device.Audio.HDAudio.DigitalJackDetection
- Device.Audio.HDAudio.HDAudioCodecAdditionalReqs
- Device.Audio.HDAudio.HDAudioSpecCompliance
- Device.Audio.HDAudio.HDMIDCN
- Device.Audio.HDAudio.HDMIKSPROPERTYJACKSINKINFO
- Device.Audio.HDAudio.INFHasDeviceID
- Device.Audio.HDAudio.LowPowerDCN
- Device.Audio.HDAudio.OneCodecPortOneConnector
- Device.Audio.HDAudio.PinConfigPortConnectivity
- Device.Audio.HDAudio.PnPCodecDeviceID
- Device.Audio.HDAudio.UniqueSequenceNumbers

Device.Audio.HDAudio.2AudioChannelsForHDMIorDisplayPort

Target Feature: Device.Audio.HDAudio

Title: Display adapter or chipset with HDMI audio or DisplayPort audio capabilities must implement two channel audio support that is compliant with the HD Audio specification

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

A display subsystem on aplatform that does not support connected standby but supports HDMI audio or DisplayPort audio capabilities must implement a minimum of two channel audio support compliant with HD Audio specification.

The minimum requirement is to use an HD Audio compliant solution exposing an SPDIF Output with a static format support of Stereo PCM, 16bit depth with sampling rate of 44.1kHz and 48kHz.

HDMI endpoints do not have to be based on HD Audio if they are a part of Systems that support connected standby.

Additional information on how to expose an SPDIF Output in an HD Audio compliant controller/codec configuration can be found in the Intel HD Audio specification.

Expose the logical independent HDMI Audio or DisplayPort audiodevice as outlined below to be exposed as an HDMI Audio device in Windows:

Intel HMDI Audio HD Audio Spec ECR:
Pin complex.PinCaps.HdmiCapable == 1,
AND PinConfig.DefDevice = DigitalOtherOut (0x5)
General/Geometric location irrelevant here

or

Intel HD Audio 1.0 method:
Pin config general location is internal (1)
AND geometric location is Special1 (0x8)
AND PinConfig.DefDevice = DigitalOtherOut (0x5)
Pincaps.HdmiCapable is irrelevant here.
Also, see the UAA Hardware Design Guidelines available at
http://go.microsoft.com/fwlink/?linkid=50734.

Exceptions: Not Specified

Business Justification:

Enables HDMI Audio playback functionality on Windows using Windows Audio class drivers.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2009

Comments:

AUDIO-0049

Device.Audio.HDAudio.AnalogJackDetection

Target Feature: Device.Audio.HDAudio

Title: HD Audio solution supports jack-presence detection for analog jacks

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

The HD Audio codec and the system board must implement HD Audio-compliant jack-presence detection for analog jacks. Presence detection implies that the codec with required system

components (jack connector and jack detection circuit) must be able to detect the presence of jack insertion into and jack removal from the input/output connectors that the codec is using. When this occurs an unsolicited response is sent so that software can be notified without constantly polling the device. Implementation of unsolicited response support for jack detection events is required for Windowsalthough it may be worded as optional in the HD Audio specification.

This requirement is unrelated to the feature of automatic sensing of what the peripheral might be. Sensing by using impedance matching is not required.

This requirement specifically means that the codec implements presence detection on each exposed pin that is connected to a system connecter (jack) and that the system board implements an audio jack detection circuit (HD Audio Specification section 7.4.2) external to the codec for each jack on the system. This requirement does not apply to device types defined in the HD Audio codec's pin configuration register defaults as a Line Connector device using an RCA type physical connector.

See the Microsoft UAA HD Audio Pin Configuration Programming Guidelines white paper for additional clarifications on the specified jack connectors that require jack detection.
http://www.microsoft.com/whdc/device/audio/PinConfig.mspx

Exceptions: Not Specified

Business Justification:

To enable the ease of use value of HD Audio we need to be able to detect when the user connects an audio peripheral.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 7/17/2008

Comments:

AUDIO-0017

Device.Audio.HDAudio.DefaultAssociationNotZero

Target Feature: Device.Audio.HDAudio

Title: Values in the Default Association field of the HD Audio codec pin configuration register are not set to zero (reserved value)

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64

- Windows 8 Client ARM

Description:

The value zero is reserved and must never be used as a default association value.

Exceptions: Not Specified

Business Justification:

This ensures compatibility with HD audio specification for Windows.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 7/17/2008

Comments:

AUDIO-0021

Device.Audio.HDAudio.DigitalJackDetection

Target Feature: Device.Audio.HDAudio

Title: HD Audio solution supports jack-presence detection for digital jacks

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

The HD Audio codec and the system board must implement HD Audio-compliant jack-presence detection for digital jacks. Presence detection implies that the codec with required system components (jack connector and jack detection circuit) must be able to detect the presence of jack insertion into and jack removal from the input/output connectors that the codec is using except for S/PDIF jacks. When this occurs an unsolicited response is sent so that software can be notified without constantly polling the device. Implementation of unsolicited response support for jack detection events is required for Windows although it may be worded as optional in the HD Audio specification.

This requirement is unrelated to the feature of automatic sensing of what the peripheral might be.

Exceptions: Not Specified

Business Justification:

To enable the ease of use value of HD Audio we need to be able to detect when the user connects an audio peripheral.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 7/17/2008

Comments:

NEW

Device.Audio.HDAudio.HDAudioCodecAdditionalReqs

Target Feature: Device.Audio.HDAudio

Title: HD Audio codec supports additional requirements not specified in the Intel High Definition Audio specification

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64

Description:

To be UAA compliant, an HD Audio codec must implement the following features, which are not necessarily required by Intel High Definition Audio Specification:

- Speaker compensation is the only valid scenario for audio signal processing of an audio stream by a codec, and then it is valid only if the speakers are hardwired to the pin complex that contains the processing node (such as integrated laptop speakers). This requirement does not apply to decryption of protected audio streams.
- When all of an HDAudio codec's widgets are configured in the benign processing state, the codec performs no nonlinear or time-variant processing on the audio streams that pass through it.
- Software must be able to set all processing nodes to the benign processing state, and the codec must function according to UAA baseline requirements while in this state.

- An HDAudio codec must be accessible only through the HDAudio bus controller. The codec must not expose registers or other hardware mechanisms that are accessible through either memory or I/O address space. This requirement does not encompass HDMI or DisplayPort. For HDMI or DisplayPort, please refer to the the HD audio HDMI DCN.
- Modem and audio functionality must not be combined. Although the same piece of silicon can house both modem and audio devices, the functions must be separate devices and must not share any software or hardware resources (such as ADCs or DACs).
- When the HD Audio link is in a running state (HD Audio controller is in D0), UAA-compliant HD Audio codecs must respond to commands even when powered down in all required device power-management states. In effect, the digital section of the codec must remain powered.
- Codecs must respond to a verb even if addressed at a nonexistent widget or if the verb itself is invalid.
- Function group nodes must have node IDs in the range 0 to 127. This restriction does not apply to node IDs for widget nodes.
- In a system with one or more HDAudio codecs, the system BIOS must initialize the Configuration Default Register for each codec pin widget based on the system configuration/implementation of the HD Audio codec while considering the Microsoft Pin Configuration Programming Guidelines so that the UAA HDAudio function class driver's topology parser can create a functional device topology for the codec. The default data in the HD Audio codec pin configuration registers must not misrepresent the hardware capabilities, and the Configuration Default Registers must not be null (all zeros).
- A function group in an HDAudio codec must expose a nonzero subsystem ID. The BIOS overwrites the subsystem ID if necessary. If the BIOS cannot program the subsystem ID or if it does so incorrectly, the hardware must supply a default, vendor-specific subsystem ID.

Exceptions: Not Specified

Business Justification:

This requirement is part of several others that aim to ensure HW compliance with the Windows UAA HD Audio class driver. This is important for partners to take full advantage of the class driver.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 9/16/2008

Comments:

AUDIO-0016

Device.Audio.HDAudio.HDAudioSpecCompliance

Target Feature: Device.Audio.HDAudio

Title: HD Audio codec for audio complies with the Intel High Definition Audio specification

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM
- Windows Server 2008 Release 2 x64
- Windows 8 Server x64

Description:

If the codec is for an audio implementation, it must be implemented according to Intel High Definition Audio Specification, Revision 1.0, and updated when commercially possible to comply with HD Audio specification DCRs.

Exceptions: Not Specified

Business Justification:

This requirement is part of several others that aim to ensure HW compliance with the Windows UAA HD Audio class driver. This is important for partners to take full advantage of the class driver.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 7/17/2008

Comments:

AUDIO-0015

Device.Audio.HDAudio.HDMIDCN

Target Feature: Device.Audio.HDAudio

Title: If hardware supports multi-channel HDMI or DisplayPort audio consistent with the method defined by HD Audio, then the hardware must comply with the HD Audio HDMI Design Change Notifications (DCN).

Applicable OS Versions:

- Windows 8 Client x86

- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

If hardware supports multi-channel HDMI or DisplayPort audio consistent with the method defined by HD Audio, then the hardware must comply with the HD Audio HDMI Design Change Notifications (DCN).

Design Notes: If hardware supports HDMI or DisplayPort audio and implements any of the verbs listed below, the hardware must comply with the following HD Audio DCNs:

HDA034-A2: HDMI Content Protection and Multi-Channel Support
HDA039-A: HDMI/ELD Memory Structure
HDA036-A: DisplayPort Support and HDMI Miscellaneous Corrections

If hardware supports HDMI or DisplayPort audio, implements any of the verbs listed below and supports the transmission of high bit-rate (HBR) formats, the hardware must comply with the HD Audio DCN HDA035-A: HDMI High Bit Rate Support.

Verbs:

F2Fh (Get HDMI ELD Data)

F2Dh (Get Converter Channel Count)

72Dh (Set Converter Channel Count)

F2Eh (Get HDMI Data Island Packet Size Info)

72Eh (Set HDMI Data Island Packet Size Info)

F30h (Get HDMI Data Island Packet Index)

730h (Set HDMI Data Island Packet Index)

F31h (Get HDMI Data Island Packet Data)

731h (Set HDMI Data Island Packet Data)

F32h (Get HDMI Data Island Packet Transmit-Control)

732h (Set HDMI Data Island Packet Transmit-3Control)

F33h (Get Content Protection Control)

733h (Set Content Protection Control)

F34h (Get Converter Channel to HDMI Slot Mapping)

734h (Set Converter Channel to HDMI Slot Mapping)

Exceptions: Not Specified

Business Justification:

This ensures compatibility of multi-channel audio devices with Windows.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2009

Comments:

AUDIO-0078

Device.Audio.HDAudio.HDMIKSPROPERTYJACKSINKINFO

Target Feature: Device.Audio.HDAudio

Title: Audio drivers using HD audio specification and exposing HDMI or DisplayPort endpoint support the KSPROPERTY_JACK_SINK_INFO property

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM
- Windows Server 2008 Release 2 x64
- Windows 8 Server x64

Description:

Audio drivers that expose HDMI or DisplayPort endpoints must support the KSPROPERTY_JACK_SINK_INFO property.

Design Notes: For every endpoint exposed by an audio driver that supports HDMI or DisplayPort audio, the driver must respond to a KSPROPERTY_JACK_SINK_INFO request with a KSJACK_SINK_INFORMATION structure. The structure shall be correctly populated.

Exceptions: Not Specified

Business Justification:

Exposing KSPROPERTY_JACK_SINK_INFO property helps Windows provide more meaningful info to end users.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2009

Comments:

AUDIO-0079

Device.Audio.HDAudio.INFHasDeviceID

Target Feature: Device.Audio.HDAudio

Title: INF file for HD Audio codec includes properly formatted device ID string for each supported codec device

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

Vendors that supply custom HD Audio function drivers must include an INF file that follows guidelines for device identification strings that are defined in Plug and Play Guidelines for High Definition Audio Devices, "INF Files for HD Audio Codecs."

Design Notes: See Guidelines at http://www.microsoft.com/whdc/device/audio/HD-aud_PnP.mspx.

Exceptions: Not Specified

Business Justification:

This helps ensure device compatibility with Windows.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 7/17/2008

Comments:

AUDIO-0019

Device.Audio.HDAudio.LowPowerDCN

Target Feature: Device.Audio.HDAudio

Title: If HD Audio codec supports HD Audio Low-Power DCN then it needs to implements the Low-Power specification correctly in hardware, firmware and 3rd party software (driver)

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

An HD audio Low-Power Design Change Notification (DCN)compatible HD Audio 1.0 based codec implements Low-Power capabilities according to the Low-Power specification and Microsoft UAA Hardware Design Guidelines.

Design Notes: See the RAND licensed Intel HD Audio specification, the HD Audio Low-Power DCN amendment to that specification and the Microsoft UAA Hardware Design Guidelines

Exceptions: Not Specified

Business Justification:

This requirement translates into battery life improvements on laptops

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2009

Comments:

AUDIO-0062

Device.Audio.HDAudio.OneCodecPortOneConnector

Target Feature: Device.Audio.HDAudio

Title: A single HD Audio codec port is used for a single connector

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x64
- Windows 7 Client x86
- Windows Server 2008 x64
- Windows Server 2008 x86
- Windows 8 Client ARM

- Windows Server 2008 Release 2 x64
- Windows 8 Server x64
- Windows Vista Client x86
- Windows Vista Client x64

Description:

Each HD Audio codec port connects to one and only one audio source, destination, or jack. For compatibility with the UAA class driver do not double-up on input or output ports in ways that cannot be exposed to the class driver through the information in the pin configuration registers.

Designs that use GPIOs under control of third-party function drivers must default to an appropriate hardware configuration when the UAA class driver is loaded.

Exceptions:

- Combination jacks (headphone/S/PDIF) are allowed if the digital output is exposed as a separate, independent always on device using the HD Audio pin configuration register values and the analog section of the jack supports jack presence detection.
- Combination jacks that have both a speaker and a microphone are also acceptable provided the connector supports microphone and speaker jack presence detection.
- There is another exception to this requirement with regards to an audio device pin that feeds two different connectors intended for SPDIF protocol content. In the case where a system or device exposes an RCA jack (Co-ax) and an optical output for the SPDIF protocol stream from one codec pin this is permitted only if the audio driver exposes the pin as outlined in the design note section below
- It is also acceptable to build a system that has two headphone jacks fed by the same HD Audio pin.

Design Notes:

An array of jack descriptions can also be used to show that a pair of jacks is equivalent. The following example would indicate to the user that the yellow RCA jack and the black digital optical jack will carry the same signal:

```
KSJACK_DESCRIPTION ar_SPDIF_Jacks[] =

{

// jack 1

{

(SPEAKER_FRONT_LEFT | SPEAKER_FRONT_RIGHT), // ChannelMapping (L,R)

RGB(255,255,0), // Color (yellow)

eConnTypeRCA, // ConnectionType (RCA)
```

eGeoLocRear, // GeoLocation

eGenLocPrimaryBox, // PortConnection

TRUE // IsConnected

},

// jack 2

{

(SPEAKER_FRONT_LEFT | SPEAKER_FRONT_RIGHT), // (L,R)

RGB(0,0,0), // (black)

eConnTypeOptical, // (optical)

eGeoLocRear,

eGenLocPrimaryBox,

TRUE

}

};

Clarification: This exception has nothing to do with HDMI Audio, it only covers SPDIF on two physically different SPDIF connectors and this exception does NOT allow HDMI Audio outputs to share a codec pin with SPDIF. That is still prohibited.

Exceptions: Not Specified

Business Justification:

To enable a base level experience with the Microsoft UAA HD Audio class driver the system design and use of output connections must be predictably tied to one codec port. This enables the ideal user experience through separate independent devices assigned to device roles.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: July 17, 2008

Comments:

AUDIO-0011

Device.Audio.HDAudio.PinConfigPortConnectivity

Target Feature: Device.Audio.HDAudio

Title: Pin configuration register for an HD Audio codec has specific settings for port connectivity field depending on implementation

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

A pin widget's port connectivity value of 0x01 (No Connection) is valid only when a system in which the HD Audio codec resides has no jack or integrated device attached to the pin widget.A port connectivity setting of 0x02 (10b) should be used only in those cases where a trace on a circuit board directly connects the codec and an integrated device such as a speaker amplifier or microphoneA port connectivity setting of 0x03 (011b) is specifically disallowed. Each pin widget must connect to a single audio endpoint.

Exceptions: Not Specified

Business Justification:

This requirement ensures the Microsoft UAA HD Audio class driver can create logical devices for Windows users based on the information gleaned from HW and firmware only.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 7/17/2008

Comments:

AUDIO-0020

Device.Audio.HDAudio.PnPCodecDeviceID

Target Feature: Device.Audio.HDAudio

Title: HD Audio codec follows Plug and Play requirements for codec device identification

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64

- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64

Description:

HD Audio codecs must comply with the Plug and Play requirements for proper identification that are described in Plug and Play Guidelines for High Definition Audio Devices, "HD Audio Codec."

Design Notes: See Guidelines at http://www.microsoft.com/whdc/device/audio/HD-aud_PnP.mspx.

Exceptions: Not Specified

Business Justification:

ensure plug and play

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 7/17/2008

Comments:

audio-0018

Device.Audio.HDAudio.UniqueSequenceNumbers

Target Feature: Device.Audio.HDAudio

Title: HD Audio Codec Pin configuration register defaults: Sequence numbers within the same default association are unique

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

In the data configured by the BIOS or in the default values from the codec manufacturer, the sequence numbers within the pin configuration registers default association must be unique within the same association except for association 0xf, in which all instances should support Sequence ==0.

Design Notes: See Pin Configuration Guidelines for High Definition Audio Devices, at http://go.microsoft.com/fwlink/?LinkId=58572.

Exceptions: Not Specified

Business Justification:

This is an essential requirement enabling the Microsoft UAA HD Audio class driver to identify the capabilities of the HD Audio codec and expose logical devices based on that information. Without proper pin config data the class driver is not able to expose any usable devices to the Windows user.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 7/17/2008

Comments:

AUDIO-0022

Device.Audio.UAACompliance

Description:

This audio device uses HD Audio, Bluetooth, or USB Audio Drivers

Related Requirements:

- Device.Audio.UAACompliance.TestUsingBluetoothClassDriver
- Device.Audio.UAACompliance.UAA

Device.Audio.UAACompliance.TestUsingBluetoothClassDriver

Target Feature: Device.Audio.UAACompliance

Title: Bluetooth devices tested using Bluetooth class driver

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM
- Windows 8 Server x64

Description:

Bluetooth devices (e.g. headset, speakers) need to use Windows Bluetooth Audio Class drivers for certification.

Exceptions: Not Specified

Business Justification:

Help preserve compatibility with windows

Scenarios:

This helps compatibility with Windows.

Success Metric: Not Specified

Enforcement Date: 9/17/2011

Comments:

NEW

Device.Audio.UAACompliance.UAA

Target Feature: Device.Audio.UAACompliance

Title: Audio device is compliant with one of the appropriate technology specifications supported by the UAA initiative

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x64
- Windows 7 Client x86
- Windows Server 2008 x64
- Windows Server 2008 x86
- Windows 8 Client ARM

Description:

An audio device must comply with the appropriate standard as supported by the Microsoft Universal Audio Architecture Initiative. The supported standards are USB Audio, IEEE 1394 Audio, Bluetooth Audio and High Definition Audio. The relevant Windows audio class driver loads, runs, and passes functionality tests on the implementation. This includes meeting minimum performance requirements as defined in the Microsoft UAA Hardware Design Guidelines.

This requirement does not apply to systems that support connected standby or devices that support USB Audio 2.0.

Design Notes: See "Universal Audio Architecture" at http://go.microsoft.com/fwlink/?LinkId=40631.

See "USB Audio Devices and Windows" at http://go.microsoft.com/fwlink/?LinkId=40632.

Exceptions: Not Specified

Business Justification:

This helps ensure compatibility with Windows universal audio architecture.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: September 17, 2008

Comments:

AUDIO-0009

Device.Audio.USB

Description:

This audio device uses the USB Audio Driver.

Related Requirements:

- Device.Audio.USB.HIDCommunications
- Device.Audio.USB.HIDControls
- Device.Audio.USB.MicArray
- Device.Audio.USB.USB

Device.Audio.USB.HIDCommunications

Target Feature: Device.Audio.USB

Title: Audio capable and video capable and audio/video capable USB communication devices implement HID controls according to USB HID Specifications

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM
- Windows Server 2008 Release 2 x64
- Windows 8 Server x64

Description:

Communication devices that implement a USB HID interface must be compliant with the USB Device Class Definition for Human Interface Devices (HID), Version 1.1, and USB Usage Tables for HID Power Devices, Version 1.12.

Devices may not use Reserved usages from any Standard Usage Page.

Scenario

It is very easy for me to use my PC for internet voice communication with my family and colleagues because my PCs audio devices tell Windows about their role and my VOIP application can automatically find the right device for the job without any input from me.

When I use my PC for voice communication with my family or colleagues I can easily understand them and they can easily understand me. I can take and place my calls regardless of other audio activity on my PC. With my PCs audio solution I have a quality calling experience comparable to todays wired telephones and/or speakerphones.

Design Notes: Design and Implementation Note See the Windows Driver Kit, "HID and Windows."

Exceptions: Not Specified

Business Justification:

The business benefits of this new device category include strengthening the Windows Logo through increased visibility of the logo itself (on more retail packages in more stores) and by adding another set of validated experiences within the Windows Logo ecosystem the logo will mean Great Experiences to more people in more situations.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2009

Comments:

AUDIO-0082

Device.Audio.USB.HIDControls

Target Feature: Device.Audio.USB

Title: USB audio device uses USB HID audio controls to keep the operating system informed of user interactions with the device

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x64
- Windows 7 Client x86
- Windows Server 2008 x64
- Windows Server 2008 x86
- Windows 8 Client ARM
- Windows Server 2008 Release 2 x64
- Windows 8 Server x64

Description:

USB audio devices must use USB HID specification-compliant HID to control basic functions. If volume adjustment controls are implemented on the USB audio device, it must declare itself as a consumer control device (usage 0x01), as defined in Consumer Page (page 0x0C) in the USB Usage Tables for HID Power Devices, Release 1.1, and in Windows support for HID-based audio controls.

Design Notes: See "HID Audio Controls and Windows" at http://go.microsoft.com/fwlink/?LinkId=40491.

Exceptions: Not Specified

Business Justification:

Without knowledge of volume/mute settings on the device the OS cannot make intelligent policy decisions that enable more predictable and better fidelity user experiences with Windows

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 9/17/2008

Comments:

AUDIO-0044

Device.Audio.USB.MicArray

Target Feature: Device.Audio.USB

Title: Standalone USB Audio based microphone array device complies with the Microsoft USB Audio 1.0 design guidelines and Microsoft Microphone Array Design Guidelines

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

An externally connected USB based microphone array device must comply with the UAA-supported technology standard, must comply with the USB Device Class Definition for Audio Devices 1.0, and must be implemented according to the guidelines in "Microphone Array Support in Windows Vista."

The device must report itself and its capabilities according to the design guidelines in the Microsoft USB Audio Microphone Array Design Guidelines.

Exceptions: Not Specified

Business Justification:

This requirement aims to capture the significant investments we have made in Microphone Array processing technology (beam forming, noise reduction, AEC, etc.) in Windows.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: September 17, 2008

Comments:

AUDIO-0008

Device.Audio.USB.USB

Target Feature: Device.Audio.USB

Title: USB Audio Device follows UAA USB Audio Design Guidelines

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x64
- Windows 7 Client x86
- Windows Server 2008 x64
- Windows Server 2008 x86
- Windows 8 Client ARM

Description:

A USB audio-based audio device in a stand-alone external form factor or in an AVR or in other permutations follows the Microsoft UAA USB Audio Design Guidelines.

Design Notes: See Microsoft UAA USB Audio Design Guidelines at http://go.microsoft.com/fwlink/?LinkId=50734.

Exceptions: Not Specified

Business Justification:

To maintain the choice within the UAA initiative it is necessary to have 1394 audio devices implemented according to the Microsoft UAA 1394 audio design guidelines.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: July 17, 2008

Comments:

AUDIO-0005

Device.BusController.Bluetooth.Base

Description:

Bluetooth Controller - Generic radio tests

Related Requirements:

- Device.BusController.Bluetooth.Base.4LeSpecification
- Device.BusController.Bluetooth.Base.LeStateCombinations
- Device.BusController.Bluetooth.Base.LeWhiteList
- Device.BusController.Bluetooth.Base.MicrosoftBluetoothStack
- Device.BusController.Bluetooth.Base.OnOffStateControllableViaSoftware
- Device.BusController.Bluetooth.Base.Scatternet
- Device.BusController.Bluetooth.Base.ScoDataTransportLayer
- Device.BusController.Bluetooth.Base.SimultaneousBrEdrAndLeTraffic
- Device.BusController.Bluetooth.Base.SpecificInformationParameters
- Device.BusController.Bluetooth.Base.SupportsBluetooth21AndEdr

Device.BusController.Bluetooth.Base.4LeSpecification

Target Feature: Device.BusController.Bluetooth.Base

Title: Bluetooth controllers must support the Bluetooth 4.0+LE specification requirements

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 8 Client ARM

Description:

The Bluetooth controller must comply with the Basic Rate (BR) and Low Energy (LE) Combined Core Configuration Controller Parts and Host/Controller Interface (HCI) Core Configuration requirements outlined in the Compliance Bluetooth Version 4.0 specifications.

Exceptions: Not Specified

Business Justification:

To support standards.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: Windows 8 RC

Comments:

BUSPORT-8002

Device.BusController.Bluetooth.Base.LeStateCombinations

Target Feature: Device.BusController.Bluetooth.Base

Title: Bluetooth controllers must support a minimum set of LE state combinations.

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 8 Client ARM

Description:

The Bluetooth controller must allow the spec LE state combinations (as allowed in section [Vol 6] Part B, Section 1.1.1 of the Bluetooth version 4.0 spec):

Only the following states are not required to be supported:

0x0000000000800000 Active Scanning State and Initiating State combination supported.

0x0000000004000000 Passive Scanning state and Slave Role combination supported.

0x0000000008000000 Active Scanning state and Slave Role combination supported.

Exceptions: Not Specified

Business Justification:

This ensures Bluetooth controllers provide a good user experience across BR/EDR and LE.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: Windows 8 RC

Comments:

New

Device.BusController.Bluetooth.Base.LeWhiteList

Target Feature: Device.BusController.Bluetooth.Base

Title: Bluetooth controllers must support a minimum LE white list size of 25 entries.

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 8 Client ARM

Description:

The Bluetooth controller must support a minimum of 25 entries in its white list for remote Low Energy (LE) devices.

Exceptions: Not Specified

Business Justification:

This ensures that Windows machines are able to save power by only processing incoming data from known devices.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: Windows 8 RC

Comments:

BUSPORT-8001

Device.BusController.Bluetooth.Base.MicrosoftBluetoothStack

Target Feature: Device.BusController.Bluetooth.Base

Title: Must test using Microsoft's Bluetooth stack

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 8 Client ARM

Description:

The Bluetooth controllers must be tested with Microsoft's Bluetooth stack when submitting for hardware certification.

Exceptions: Not Specified

Business Justification:

This ensures Bluetooth controllers can work well with the inbox Microsoft Bluetooth stack.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: Windows 8 RC

Comments:

New

Device.BusController.Bluetooth.Base.OnOffStateControllableViaSoftware

Target Feature: Device.BusController.Bluetooth.Base

Title: Bluetooth controllers' On/Off state must be controllable via software

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 8 Client ARM

Description:

Bluetooth controllers On/Off state shall be controllable via software as described in Bluetooth Software Radio Switch. The Off state is defined, at a minimum, as disabling the antenna component of the Bluetooth module so there can be no transmission/reception. There must not be any hardware-only switches to control power to the Bluetooth radio.

The radio must maintain on/off state across sleep and reboot.

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: Windows 8 RC

Comments:

BUSPORT-8005

Device.BusController.Bluetooth.Base.Scatternet

Target Feature: Device.BusController.Bluetooth.Base

Title: Bluetooth host controller supports Bluetooth scatternet

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 7 Client x64
- Windows 7 Client x86
- Windows 8 Client ARM

Description:

The Bluetooth host controller must support at least two concurrent piconets (also known as a scatternet). The host controller must also be able to allow the host to join a device that is requesting

a connection to the existing piconet when the local radio is the master of that piconet. This requirement is described in the Specification of the Bluetooth System, Version 2.1 + Enhanced Data Rate (EDR) (Baseband Specification), Section 8.6.6.

Design Notes:

The scatternet support should follow the enhanced scatternet support errata that are defined by the Bluetooth Special Interest Group (SIG).

Exceptions: Not Specified

Business Justification:

To ensure a good Windows experience when using multiple Bluetooth devices.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

BUSPORT-0021

Device.BusController.Bluetooth.Base.ScoDataTransportLayer

Target Feature: Device.BusController.Bluetooth.Base

Title: Bluetooth host controllers support the SCO data transport layer as specified in the Bluetooth 2.1+EDR specifications

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 7 Client x64
- Windows 7 Client x86
- Windows 8 Client ARM

Description:

The Bluetooth host controller must comply with the Synchronous Connection Oriented (SCO)-USB requirements that are outlined inthe Specification of the Bluetooth System, Version 2.1 + Enhanced Data Rate (EDR), Part A, Section 3.5.

Exceptions: Not Specified

Business Justification:

This ensures voice audio scenarios work with Windows.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

BUSPORT-0023

Device.BusController.Bluetooth.Base.SimultaneousBrEdrAndLeTraffic

Target Feature: Device.BusController.Bluetooth.Base

Title: Bluetooth controllers must support simultaneous BR/EDR and LE traffic.

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 8 Client ARM

Description:

Bluetooth controllers must allow the simultaneous use of both Basic Rate (BR)/Enhanced Data Rate (EDR) and Low Energy (LE) radios.

Exceptions: Not Specified

Business Justification:

This ensures a good Windows user experience across BR/EDR and LE device scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: Windows 8 RC

Comments:

BUSPORT-8004

Device.BusController.Bluetooth.Base.SpecificInformationParameters

Target Feature: Device.BusController.Bluetooth.Base

Title: Bluetooth host controller implements specific Informational parameters to provide accurate information about the host controller's capabilities

Applicable OS Versions:

- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows 8 Client x64

- Windows 8 Client ARM

Description:

The manufacturer fixes the informational parameters, which provide valuable information about the Bluetooth device and the capabilities of the host controller. Bluetooth host controllers must implement the **HCI_Read_Local_Version_Information** command and **HCI_Read_Local_Supported_Features** command as described in the Specification of the Bluetooth System, Version2.1 + Enhanced Data Rate (EDR), Part E, Section 7.4. Required support includes the mechanism for reporting the supported version and features.

Exceptions: Not Specified

Business Justification:

To ensure reliability.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

BUSPORT-0018

Device.BusController.Bluetooth.Base.SupportsBluetooth21AndEdr

Target Feature: Device.BusController.Bluetooth.Base

Title: Bluetooth controllers must support the Bluetooth 2.1+EDR specification requirements

Applicable OS Versions:

- Windows 7 Client x64
- Windows 7 Client x86

Description:

The Bluetooth host controller must comply with the requirements that are outlined in the Specification of the Bluetooth System Version 2.1 + Enhanced Data Rate (EDR).

Exceptions: Not Specified

Business Justification:

To ensure standards compliance.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2009

Comments:

BUSPORT-0026

Device.BusController.Bluetooth.NonUSB

Description:

Bluetooth Controller - NonUSB connected radios

Related Requirements:

- Device.BusController.Bluetooth.NonUSB.Performance
- Device.BusController.Bluetooth.NonUSB.ScoSupport

Device.BusController.Bluetooth.NonUSB.Performance

Target Feature: Device.BusController.Bluetooth.NonUSB

Title: Non-USB Bluetooth controllers must achieve at least a throughput of 700kbps

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 8 Client ARM

Description:

Non-USB Bluetooth controllers must achieve at least a throughput of 700 kbps throughput at the RFCOMM layer.

Exceptions: Not Specified

Business Justification:

This ensures a good Windows user experience with non-USB Bluetooth controllers.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: Windows 8 RC

Comments:

BUSPORT-8002

Device.BusController.Bluetooth.NonUSB.ScoSupport

Target Feature: Device.BusController.Bluetooth.NonUSB

Title: Non-USB connected Bluetooth controllers use sideband channel for SCO

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 8 Client ARM

Description:

In order to ensure a high quality audio experience All non-USB connected Bluetooth controllers must use a sideband channel for SCO (i.e. SCO over an I^2S/PCM interface)

Exceptions: Not Specified

Business Justification:

This ensures a good Windows audio user experience with non-USB Bluetooth controllers.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: Windows 8 RC

Comments:

New

Device.BusController.NearFieldProximity

Description:

Near Field Proximity is a set of short range wireless technologies to enable communication between a personal computer and a device.

Related Requirements:

Device.BusController.NearFieldProximity.NFCCertification
Device.BusController.NearFieldProximity.ProviderImplementation
Device.BusController.NearFieldProximity.ProximityReliability
Device.BusController.NearFieldProximity.RangeOfActuation
Device.BusController.NearFieldProximity.SessionEstablishmentPerformance
Device.BusController.NearFieldProximity.TaptoSetupScenario
Device.BusController.NearFieldProximity.TapToUseScenarios

Device.BusController.NearFieldProximity.NFCCertification

Target Feature: Device.BusController.NearFieldProximity

Title: NFC Implementations must receive NFC certification

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Client ARM

Description:

An NFP provider that implements air interface specifications incorporated by NFC Forum by reference as approved specifications must receive Certification from the NFC Forum, inclusive of:

Wave 1 compliance

SNEP compliance

Exceptions:

None

Business Justification:

Obtaining certification from the NFC forum ensures proper NFC behavior.

Scenarios:

This ensures proper NFC behavior.

Success Metric: Pass/Fail

Enforcement Date: 6/1/2010

Comments:

Connect-0091 Reworded

Device.BusController.NearFieldProximity.ProviderImplementation

Target Feature: Device.BusController.NearFieldProximity

Title: Device proximity adheres to the Proximity Provider model

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Client ARM

Description:

Any technology that implements the GUID_DEVINTERFACE_PROXIMITYPROVIDER device driver interface specified in the 'Windows 8 Near Field Proximity Implementation Specification' document is defined as an NFP provider and must meet all the design and implementation requirements laid

out within the spec as well as all other applicable NFP certification requirements. The spec can be found at:

http://go.microsoft.com/fwlink/?LinkId=237135

Exceptions:

None

Business Justification:

Ensures well behaved providers.

Scenarios:

Enables all device proximity scenarios.

Success Metric: Pass/Fail

Enforcement Date: Windows RC

Comments: Not Specified

Device.BusController.NearFieldProximity.ProximityReliability

Target Feature: Device.BusController.NearFieldProximity

Title: Proximity provider meets session reliability requirements

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Client ARM

Description:

An NFP provider must support performant completion of Windows scenarios.

Within a period of 2.5 minutes the devices must be able to establish 95 successful sessions out of 100 attempts.

The test scenario consists of:

Provide two machines to carry out the test with a test app.

Tap the two machines together.

Validate that a session is established between the two devices within one second.

Repeat from step 2.

Attempt this 100 times

Exceptions:

None

Business Justification:

Allows users to have confidence with proximity feature and ensures reliability.

Scenarios:

Two devices are within proximity of each other 100 consecutive times and sessions can be established at least 95 times.

Success Metric: Pass/Fail

Enforcement Date: Windows RC

Comments: Not Specified

Device.BusController.NearFieldProximity.RangeOfActuation

Target Feature: Device.BusController.NearFieldProximity

Title: Proximity technology meets range of actuation

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Client ARM

Description:

An NFP provider must support an effective operating volume to enable users to successfully use NFP technology with Windows in 95 times out of 100 attempts for all tap and do scenarios. Refer to the most current version of the 'Windows 8 Near Field Proximity Implementation Specification' document for detailed placement guidance, as well as acceptable, minimum, and maximum values for the required effective operating volume. The spec can be found at:

http://go.microsoft.com/fwlink/?LinkId=237135

Exceptions:

None

Business Justification:

Ensures connections will only be established in a small range rather than larger ones (such as Bluetooth or wireless). Confines the range so that the appropriate technology is tested._

Scenarios:

When tapping two devices together, a connection cannot be established if the touch point of the host device is greater than 10cm away from the other touch point.

Success Metric: Pass/Fail

Enforcement Date: Windows RC

Comments: Not Specified

Device.BusController.NearFieldProximity.SessionEstablishmentPerformance

Target Feature: Device.BusController.NearFieldProximity

Title: Proximity technology establishes a session in 0.5 second

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Client ARM

Description:

An NFP provider must support the creation of sessions within 0.5 seconds from the point of being detected within the effective operating volume. As a point of information, the baseline expected performance in this test for NFC is as follows:

- Two apps attempt to establish sessions.
- During the session establishment, the total data transferred at the air interface inclusive ofNDEF framing should be no more than 2 x 16kb, so a total of 32kbits transferredbidirectionally.The expected worst-case transfer rate for NFC is 106kbps, so the total time required at the air interface should be 32kb/106kbps, so 0.3 seconds of time.

The remaining 0.2 seconds of time is for buffering and latency in the busses and stacks.

Exceptions:

None

Business Justification:

Ensures a timely connection for proximity aware applications.

Scenarios:

When tapping two devices together with 12 proximity aware applications using sessions, sessions must be established within 0.5 second.

Success Metric: Pass/Fail

Enforcement Date: Windows RC

Comments: Not Specified

Device.BusController.NearFieldProximity.TaptoSetupScenario

Target Feature: Device.BusController.NearFieldProximity

Title: Provider must support the handling of the tap and setup scenario

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

An NFP provider must unwrap the out-of-band pairing information from the transmission format so that only the out-of-band pairing information is presented to Windows. Detailed guidance and requirements are further defined in the most current version of the 'Windows 8 Near Field Proximity Implementation Specification' document. The spec can be found at:

http://go.microsoft.com/fwlink/?LinkId=237135

Exceptions:

None.

Business Justification:

Allows users to have confidence with proximity feature and ensures reliability.

Scenarios:

Enables support for tap to setup.

Success Metric: Pass/Fail

Enforcement Date: Windows RC

Comments: Not Specified

Device.BusController.NearFieldProximity.TapToUseScenarios

Target Feature: Device.BusController.NearFieldProximity

Title: Provider must support the handling of the tap and use scenarios

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Client ARM

Description:

An NFP provider must support the end-to-end NFP use cases of tap and use, tap and launch, tap and share, tap and acquire, and tap and receive. Details are further defined in the most current version of the 'Windows 8 Near Field Proximity Implementation Specification' document. The spec can be found at:

http://go.microsoft.com/fwlink/?LinkId=237135

Exceptions:

None.

Business Justification:

Allows users to have confidence with proximity feature and ensures reliability.

Scenarios:

Support tap to use, launch, share and acquire.

Success Metric: Pass/Fail

Enforcement Date: Windows RC

Comments: Not Specified

Device.BusController.SdioController

Description: Not Specified

Related Requirements:

- Device.BusController.SdioController.ComplyWithIndustrySpec
- Device.BusController.SdioController.WdfKmdfDriver

Device.BusController.SdioController.ComplyWithIndustrySpec

Target Feature: Device.BusController.SdioController

Title: SDIO Controller Complies with industry Standard

Applicable OS Versions:

- Windows Server 2008 x86
- Windows 8 Server x64

- Windows 8 Client x64
- Windows 8 Client x86
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86
- Windows Server 2008 x64

Description:

Secure Digital I/O (SDIO) host controllers must comply with PCI 2.3 or later requirements for that interface. For PCI configuration registers and interface information, see the SD Host Controller Specification, Version 1.0, Appendix A.

Exceptions: Not Specified

Business Justification:

To support SDIO scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2007

Comments:

Taken from Busport-0011

Device.BusController.SdioController.WdfKmdfDriver

Target Feature: Device.BusController.SdioController

Title: SDIO Controller driver must be a WDF KMDF Implementation

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

The SDIO Controller driver must be written using the Windows Driver Framework (WDF) Kernel Mode Driver Framework for the drivers implementation.

Exceptions: Not Specified

Business Justification:

To support SDIO scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2007

Comments:

Taken from Busport-0011

Device.BusController.UsbController

Description:

Requirements that apply to USB Controllers

Related Requirements:

- Device.BusController.UsbController.ImplementAtLeastOneXhciSpcStructForUSB2
- Device.BusController.UsbController.MaintainDeviceStateOnResumeS1andS3
- Device.BusController.UsbController.MustResumeWithoutForcedReset
- Device.BusController.UsbController.PreserveDeviceStateAfterDisableEnable
- Device.BusController.UsbController.SpecificationCompliance
- Device.BusController.UsbController.SuperSpeedConnectorsSupportHighFullLow
- Device.BusController.UsbController.SupportSelectiveSuspend
- Device.BusController.UsbController.TestedUsingMicrosoftUsbStack
- Device.BusController.UsbController.UsbifCertification
- Device.BusController.UsbController.XhciAc64Bit
- Device.BusController.UsbController.XhciAddInCardsMapPortsConsistently
- Device.BusController.UsbController.XhciAddInCardsReportInternalDevices
- Device.BusController.UsbController.XhciSupportDebuggingOnAllExposedPorts
- Device.BusController.UsbController.XhciSupportMsiMsixInterrupts
- Device.BusController.UsbController.XhciSupportsMinimum31Streams
- Device.BusController.UsbController.XhciVersionCompliant

Device.BusController.UsbController.ImplementAtLeastOneXhciSpcStructForUSB2

Target Feature: Device.BusController.UsbController

Title: xHCI Controllers implement at least one xHCI Supported Protocol Capability structure for USB 2.0

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

Extensible Host Controller Interface (xHCI) controllers implement at least one xHCI Supported Protocol Capability structure for USB 2.0 as described in section 7.2 of the xHCI Specification.

This affects backward compatibility with USB 2.0.

Exceptions: Not Specified

Business Justification:

To support USB Controller functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 12/1/2010

Comments:

Busport-0047

Device.BusController.UsbController.MaintainDeviceStateOnResumeS1andS3

Target Feature: Device.BusController.UsbController

Title: USB host controller maintains device state on resume from S1 or S3

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

For the host controller to maintain the device state, the USB host controller must not issue a USB bus reset on a system sleep state transition from S3 to S0 or from S1 to S0.

USB host controllers that are integrated or embedded into the south bridge chipset must decouple the USB bus reset from the PCI bus reset to reduce resume time. Resume operations from S1 or S3 must not generate USB bus resets. A USB bus reset is a reset signal that is sent over the USB bus to USB devices that are connected to the USB host controller.

Systems that have a USB keyboard attached are allowed to perform USB bus resets to unlock the system by using a password when the system resumes from S3.

For security purposes, the BIOS in a mobile system is allowed to issue a USB bus reset if the system is attached to a docking station that has a hard disk drive (HDD) that is password-locked on first resume.

A reset of the HDD password is allowed whether or not the mobile system is docked. The following scenarios are allowed:

- Undocked systems with a password-enabled HDD
- Docked systems with a password-enabled HDD
- Addition or removal of an HDD

If the docking station does not have a native HDD or the docking station does not have a password, the BIOS must not issue a USB bus reset.

It is acceptable to allow the controller to lose power in S3 when the system is on battery power.

Design Notes: See the Enhanced Host Controller Interface Specification for Universal Serial Bus, Revision 1.0, Appendix A.

This requirement does not apply to Systems that Support Connected Standby.

Exceptions: Not Specified

Business Justification:

This requirement reduces resume time and maintains the device state on resume from S3.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2007

Comments:

Busport-0017

Device.BusController.UsbController.MustResumeWithoutForcedReset

Target Feature: Device.BusController.UsbController

Title: All USB host controllers work properly upon resume from sleep, hibernation or restart without a forced reset.

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

All USB host controllers work properly upon resume from sleep, hibernation or restart without a forced reset of the USB host controller

Design Notes:

A reset of the entire USB Host Controller results in significantly increased time that it takes for all USB devices to become available after system resume since there could be only one device at address 0 at a time, this enumeration has to be serialized for all USB devices on the bus. We have also seen that resetting the host controller can lead to an illegal SE1 signal state on some host controllers, which in turn can cause some USB devices to hang or drop off the bus. Moreover, devices cannot maintain any private state across sleep resume as that state will be lost on reset.

Exceptions: Not Specified

Business Justification:

To support USB Controller functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2010

Comments:

Taken from Connect-0126. that requirement referenced devices and controllers

Device.BusController.UsbController.PreserveDeviceStateAfterDisableEnable

Target Feature: Device.BusController.UsbController

Title: USB controller preserves device states after a disable and re-enable

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

If a USB controller is disabled and then re-enabled, all devices that were attached to the controller before the USB controller was disabled are required to be present after the USB controller is re-enabled.

Exceptions: Not Specified

Business Justification:

To support USB Controller functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2009

Comments:

Bussport-0032

Device.BusController.UsbController.SpecificationCompliance

Target Feature: Device.BusController.UsbController

Title: USB host controller that supports USB functionality complies with the appropriate specification

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86

- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

Host controllers that provide USB 1.1 functionality but dont provide USB 2.0 or USB 3.0 functionality must comply with either the Open Host Controller Interface (OHCI) Specification for USB or the Universal Host Controller Interface (UHCI) Design Guide, Revision 1.1. Host controllers that provide USB 2.0 functionality but dont provide USB 3.0 functionality must comply with the Enhanced Host Controller Interface Specification (EHCI) for Universal Serial Bus 2.0. EHCI host controllers must comply with the Enhanced Host Controller Interface Specification for Universal Serial Bus, Revision 1.0 or later.

Exceptions: Not Specified

Business Justification:

To support USB functionality.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

Busport-0013

Device.BusController.UsbController.SuperSpeedConnectorsSupportHighFullLow

Target Feature: Device.BusController.UsbController

Title: Exposed Super Speed capable connectors support high, full and low speed USB devices

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

Extensible Host Controller Interface (xHCI) controllers are backward-compatible with high-speed, full-speed, and low-speed USB devices. Backward compatible means that all USB devices enumerate and function at their intended speeds.

This requirement ensures that low-, full-, and high-speed devices continue to work when xHCI controllers become more prevalent in systems.

Design Notes: Integrated rate-matching (TT) hubs can be used to achieve this backward compatibility.

Exceptions: Not Specified

Business Justification:

To support USB Controller functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 12/1/2010

Comments:

Busport-0043

Device.BusController.UsbController.SupportSelectiveSuspend

Target Feature: Device.BusController.UsbController

Title: USB Host Controller supports selective suspend

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

A USB host controller can selectively suspend devices that support the selective suspend feature.

Exceptions: Not Specified

Business Justification:

To support USB Controller functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2009

Comments:

Busport-0031

Device.BusController.UsbController.TestedUsingMicrosoftUsbStack

Target Feature: Device.BusController.UsbController

Title: xHCI Controllers must be tested with Microsoft's xHCI Stack installed

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64

Description:

Extensible Host Controller Interface (xHCI) Controllers must be tested with Microsofts xHCI stack installed and enabled on a Windows system.

Note: During USB-IF self testing a specific USB Test Stack is installed for testing purposes, this is expected and acceptable.

Exceptions: Not Specified

Business Justification:

To support USB Controller functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: Windows 8 RC

Comments:

New

Device.BusController.UsbController.UsbifCertification

Target Feature: Device.BusController.UsbController

Title: USB Host Controller is USB IF certified

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

Starting June 1, 2011, USB host controllers must pass USB Implementers Forum (IF) testing.

For details see the following link:

http://msdn.microsoft.com/en-us/windows/hardware/gg463175.aspx

Note: Since USB-IF is currently not certifying controllers for Windows on ARM systems, the Windows on ARM controllers are exempt from needing to get full USB-IF certification. Instead the WoA controllers are expected to pass all Windows Hardware Certification tests which include eventing, loop back and registers tests that get run as part of USB-IF certification.

Exceptions:

Systems that Support Connected Standby may pass a subset of USB-IF tests instead of being USB-IF certified, since USB-IF does not certify Systems that Support Connected Standby.

Business Justification:

To support USB Controller functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2011

Comments:

Busport-0034: Added exception for SOC implementations

Device.BusController.UsbController.XhciAc64Bit

Target Feature: Device.BusController.UsbController

Title: xHCI Controllers set the AC64 bit in the HCCPARAMS register to 1

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64

Description:

xHCI Controllers set the AC64 bit in the HCCPARAMS register to 1 as described in Section 5.3.6 of the xHCI specification.

Therefore the controller must support:

- 64-bit addressing, described in section 5.3.6, and
- 64-bit register access, described in section 5.1.

Design notes:

Checking for AC64 to be set is a simple register check in the compliance driver.

To test 64-bit addressing, we will need to require the WLK users client system to have at least 6GB of RAM. The test will use MmAllocateContiguousMemorySpecifyCache to get physical memory above 4GB. It will validate in some way that the controller can access this memory area.

The test will try writing one or more registers using a 64-bit register access and reading back using 64-bit register access to confirm that registers are updated correctly. An example of a reasonable register to test is: **Event Ring Segment Table Base Address Register (ERSTBA)** (section 5.3.2.3.2).

If AC64 is not set, there is nothing to test.

Exceptions:

This Requirement does NOT apply to Systems that Support Connected Standby.

Business Justification:

To support USB Controller functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 12/1/2010

Comments:

Busport-0045: Does not apply to SOC Implementations

Device.BusController.UsbController.XhciAddInCardsMapPortsConsistently

Target Feature: Device.BusController.UsbController

Title: xHCI add in cards must map USB 3.0 and USB 2.0 ports consistently

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64

Description:

Consistent USB 2.0 and USB 3.0 port mapping is required to help the operating system to effectively manage the ports.

Note: This requirement only applies to add-in cards because port mapping for integrated xHCI controllers should be performed via Advanced Configuration and Power Interface (ACPI). For more information, see the SYSFUND 226 requirement.

For Extensible Host Controller Interface (xHCI) add-in cards (where add-in card is defined as a card that is not integrated onto the motherboard), the complexity of this requirement varies significantly depending on whether the add-in card contains any internal (integrated or embedded) hubs.

If there are no internal hubs, then the portnumbering must correlate as given in xHCI v1.x Specification. That is, the firstUSB 3.0port must be connected to the same connector as the first 2.0 port, the second with the second, and so on. For example, if the USB 2.0 ports are numbered 1 and 2, and the USB 3.0 ports are numbered 3 and 4, ports 1 and 3 must map to the same connector, and ports 2 and 4 must map to the same connector. For more information, see the xHCI v1.x Specification, sections 4.24.2.1 and 4.24.2.2. If the host does not have any internal hubs, then the remaining text of this requirement can be ignored.

However, if there are internal hubs (either integrated or embedded), then the requirement is more involved. Note that strictly speaking, XHCI specification does not allow such hubs for add-in cards because the port mapping information cannot be communicated to the software via ACPI. But through this requirement, we are allowing such hubs and defining the required port mapping. However, this mechanism has some limitations and it does not allow arbitrary configurations that are allowed for integrated controllers when described by ACPI.

For add-in cards, xHCI host controllers may implement integrated hubs and/or embedded hubs as defined in xHCI specification sections 4.24.2.1 and 4.24.2.2. Embedded hubs need not be limited to being on the system board. However, the following limitations apply:

- Embedded hubs of add-in cards must be USB 3.0 hubs (this limitation is unique to the scenario of this requirement and not part of the xHCI specification).
- An add-in card may have at most 1 integrated hub.
- If an add-in card has an integrated hub, it must have only 1 USB2 protocol port on the root hub. This port is the port connected to the integrated hub.
- An add-in xHCI card that implements an integrated hub must set the Integrated Hub Implemented (IHI) bit in the USB 2.0 xHCI Supported Protocol Capability structure to 1 for the root hub port connected to an integrated hub (refer to section 7.2.2.1.3.2 of the xHCI specification).
- All integrated or embedded hubs must be marked non-removable in their parent ports.

The implementation of integrated hubs determines the External Ports of the controller. External Ports are a concept defined in section 4.24.2 of the xHCI specification to order ports so that they can be mapped to connectors. In all cases, let there be *n* USB2 protocol External Ports numbered 1 to *n*, and *m* USB3 protocol External Ports numbered *n*+1 to *n*+*m*.

External Port numbers are assigned to meet the following properties (not defined in the xHCI specification). Note that integrated hubs must be USB 2.0 hubs.

- If the xHCI implements an integrated hub, then *n*, the number of USB2 protocol External Ports, equals the number of downstream facing ports on the integrated hub.
- Otherwise, *n* equals the number of downstream facing USB2 protocol ports on the root hub.
- *m*, the number of USB3 protocol External Ports, equals the number of downstream facing USB3 protocol ports on the root hub.
- Assign External Port numbers such that External Ports 1 through *n* are USB2 protocol ports and External Ports *n*+1 through *n*+*m* are USB3 protocol external ports, and the ordering ports within each protocol is preserved.

If embedded hub(s) are not present: The USB2 protocol External Ports and USB3 protocol External Ports must be mapped to connectors using the default mapping described in section 4.24.2.2 of the xHCI specification under the heading When an Embedded hub is not implemented.

If embedded hub(s) are present: The embedded hubs must be USB 3.0 hubs. First determine the connector mapping as it would be without any embedded hubs, using the default mapping from section 4.24.2.2 of the xHCI specification. For each embedded hub, both upstream ports must be connected to the same connector. The embedded hubs downstream ports map to new connectors in the same way as the ports of a non-embedded USB 3.0 hub.

Non-exposed connectors: Devices embedded in the host controller must be marked non-removable in their parent ports. If, according to the connector mapping above, a non-removable peripheral devices connector is shared with a second port, the second port must not be connected or connectable to any device. On the other hand, any connector whose port(s) are all marked as removable is considered to be an exposed connector, i.e. it must be physically connectable.

Note that if there is no ACPI information, a root hub cannot have both an embedded USB2 device and an integrated USB2 hub; instead, the embedded device must be attached to the integrated hub.

Exceptions:

This Requirement does NOT apply to Systems that Support Connected Standby

Business Justification:

To support USB Controller functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 12/1/2010

Comments:

Busport-0048: Extensive Re-Write to provide more details and clarification; does NOT apply to SOC Implementations

Device.BusController.UsbController.XhciAddInCardsReportInternalDevices

Target Feature: Device.BusController.UsbController

Title: xHCI controller add in cards correctly report internally attached devices

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64

Description:

Extensible Host Controller Interface (xHCI) controllers must indicate internally attached devices by setting the device removable (DR) bit in the PORTSC register to 1 for every port that has an internally attached device. This applies to controllers that do not have ACPI information. For more information, see section 5.4.8 of the xHCI Specification.

- This requirement will prevent the operating system from flagging non-removable devices as removable.
- Add-in cards are defined as host controllers that are not integrated onto the motherboard.

Design Notes:

Note: This requirement only applies to add-in cards because port mapping for integrated xHCI controllers should be performed via Advanced Configuration and Power Interface (ACPI).

Exceptions: Not Specified

Business Justification:

To support USB Controller functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 12/1/2010

Comments:

Busport-0046: Added clarification that this applies to controllers that don't have ACPI Info and added design notes.

Device.BusController.UsbController.XhciSupportDebuggingOnAllExposedPorts

Target Feature: Device.BusController.UsbController

Title: xHCI Controllers support USB debugging on all exposed ports

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64

Description:

Extensible Host Controller Interface (xHCI) host controllers are debug-capable on all ports. Ports that have embedded non-removable devices attached do not need to report debug capability.

- USB debugging is defined in section 7.6 of the xHCI specification.
- This requirement does not apply to add-in card host controllers.

This requirement is effective as of June 2012.

Exceptions: Not Specified

Business Justification:

To support USB Controller functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2012

Comments:

Busport-0042

Device.BusController.UsbController.XhciSupportMsiMsixInterrupts

Target Feature: Device.BusController.UsbController

Title: xHCI Controllers support MSI and/or MSI-X interrupts

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64

Description:

Extensible Host Controller Interface (xHCI) controllers support Message Signaled Interrupts (MSI) and MSI-X interrupts as defined in section 6.8 of the PCI Local Bus Specification, revision 3.0 and section 5.2.6 of the xHCI Specification.

Exceptions:

This Requirement does NOT apply to Systems that Support Connected Standby

Business Justification:

To support USB Controller functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 12/1/2010

Comments:

Busport-0044: Does NOT apply to SOC Implementations

Device.BusController.UsbController.XhciSupportsMinimum31Streams

Target Feature: Device.BusController.UsbController

Title: xHCI controller must support at least 31 primary streams per endpoint

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

Refer to the eXtensible Host Controller Interface specification, section 4.12.2.

This requirement is for the MaxPSASize in the HCCPARAMS to be set to 4 at the minimum to enable ultimate data transfer rate with UAS devices.

Storage devices based on the USB Attached SCSI Protocol (UASP) will utilize streams to achieve faster data transfer rates. To enable the best experience with these devices, every xHCI controller will need to support at least 31 primary streams.

Exceptions: Not Specified

Business Justification:

To support USB Controller functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: Windows 8 RC

Comments:

New

Device.BusController.UsbController.XhciVersionCompliant

Target Feature: Device.BusController.UsbController

Title: USB 3.0 controllers are XHCI version compliant

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

Effective June 1, 2012, USB 3.0 controllers must comply with the Extensible Host Controller Interface (xHCI) Specification version 1.0. and any USB-IF Errata that are released by the USB-IF.

Exceptions: Not Specified

Business Justification:

To support USB Controller functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2012

Comments:

Busport-0041: Added "and any USB-IF Errata that are released by the USB-IF." to the end of the requirement.

Device.Connectivity.BluetoothDevices

Description:

Devices which connect to the PC via Bluetooth

Related Requirements:

- Device.Connectivity.BluetoothDevices.BluetoothDeviceIdProfileVer12
- Device.Connectivity.BluetoothDevices.BluetoothDeviceIdProfileVer13
- Device.Connectivity.BluetoothDevices.BluetoothHidLimitedDiscoverableMode
- Device.Connectivity.BluetoothDevices.BluetoothUSBPlugandPlay
- Device.Connectivity.BluetoothDevices.ComplementarySubsystemList
- Device.Connectivity.BluetoothDevices.FunctionAfterSystemSuspendCycle
- Device.Connectivity.BluetoothDevices.HidInitiatedReconnect
- Device.Connectivity.BluetoothDevices.KeyboardsSupportPasskeyAuthentication
- Device.Connectivity.BluetoothDevices.RespondToServiceDiscoveryRequests
- Device.Connectivity.BluetoothDevices.SupportBluetooth21

Device.Connectivity.BluetoothDevices.BluetoothDeviceIdProfileVer12

Target Feature: Device.Connectivity.BluetoothDevices

Title: Devices which support Bluetooth must implement the DeviceID profile, version 1.2

Applicable OS Versions:

- Windows 7 Client x64
- Windows 7 Client x86

Description:

The Bluetooth system defines a Service Discovery Protocol (SDP). Bluetooth PC peripherals must support SDP as described in Specification of the Bluetooth System, Version2.1+EDR, Part B. The Plug and Play information is a single record that gives the devices VID/PID.

Exceptions: Not Specified

Business Justification:

This allows for unique identification of the remote device type.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

CONNECT-0006

Device.Connectivity.BluetoothDevices.BluetoothDeviceIdProfileVer13

Target Feature: Device.Connectivity.BluetoothDevices

Title: Devices which support Bluetooth must implement the Device ID (DI) profile version 1.3 or Device Information Service (DIS), as applicable

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

Bluetooth PC peripherals must include the Device ID record as specified in the Device ID profile, version 1.3, for BR/EDR Bluetooth or the Device Information Service (DIS), version 1.1, Bluetooth LE.

Exceptions: Not Specified

Business Justification:

This allows for unique identification of the remote device type.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: Windows 8 RC

Comments:

CONNECT-8002

Device.Connectivity.BluetoothDevices.BluetoothHidLimitedDiscoverableMode

Target Feature: Device.Connectivity.BluetoothDevices

Title: Bluetooth HID devices must be discoverable only in Limited Discoverable Mode.

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 8 Client ARM

Description:

Bluetooth HID devices must be discoverable only in Limited Discoverable Mode.

Exceptions: Not Specified

Business Justification:

This allows Bluetooth HID devices to be prioritized when discovering devices.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: Not Specified

Comments:

CONNECT-8001

Device.Connectivity.BluetoothDevices.BluetoothUSBPlugandPlay

Target Feature: Device.Connectivity.BluetoothDevices

Title: Bluetooth wireless technology device supports Plug and Play on the applicable bus

Applicable OS Versions:

- Windows 7 Client x86

- Windows 7 Client x64
- Windows Vista Client x86
- Windows Vista Client x64
- Windows 8 Client x86
- Windows 8 Client x64

Description:

USB must comply with Specification of the Bluetooth System, Version 1.1 or later, "Part H:2, USB Transport Layer."

Exceptions: Not Specified

Business Justification:

To comply with standards.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: June 1, 2006

Comments:

CONNECT-0001

Device.Connectivity.BluetoothDevices.ComplementarySubsystemList

Target Feature: Device.Connectivity.BluetoothDevices

Title: Bluetooth wireless technology subsystem end product lists Windows operating system in its complementary subsystem list

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 7 Client x64
- Windows 7 Client x86
- Windows 8 Client ARM

Description:

The Bluetooth subsystem end product must list the Windows operating system in the complementary subsystem list as described in Bluetooth Qualification Program Reference Document, Version1.0, Section6.1, "Bluetooth Subsystems."

Exceptions: Not Specified

Business Justification:

This fulfills a Bluetooth Qualification requirement.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

CONNECT-0008

Device.Connectivity.BluetoothDevices.FunctionAfterSystemSuspendCycle

Target Feature: Device.Connectivity.BluetoothDevices

Title: Bluetooth device appears and continues to function properly after a system suspend resume cycle

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 7 Client x64
- Windows 7 Client x86
- Windows 8 Client ARM

Description:

Bluetooth devices which are present before any system sleep state are present upon wake from that sleep state.

Exceptions: Not Specified

Business Justification:

To verify that Bluetooth devices work properly after suspend/resume cycles.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: Not Specified

Comments:

CONNECT-0129

Device.Connectivity.BluetoothDevices.HidInitiatedReconnect

Target Feature: Device.Connectivity.BluetoothDevices

Title: HID Devices which support Bluetooth support HID-initiated re-connect

Applicable OS Versions:

- Windows 8 Client x86
- Windows 7 Client x64
- Windows 7 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

The HIDReconnectInitiate attribute (defined in Bluetooth HID Profile, 1.0, Section7.11.5, HIDReconnectInitiate) must be enabled. To automatically reconnect to the host if the connection is dropped, the device must enter page mode.

Exceptions: Not Specified

Business Justification:

This ensures the HID device will attempt to connect back to the Windows system if there is an unexpected disconnect.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 9/5/2008

Comments:

CONNECT-0011

Device.Connectivity.BluetoothDevices.KeyboardsSupportPasskeyAuthentication

Target Feature: Device.Connectivity.BluetoothDevices

Title: Bluetooth Keyboards which implement Secure Simplified Pairing must support the Passkey authentication method

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 7 Client x64
- Windows 7 Client x86
- Windows 8 Client ARM

Description:

Keyboards which implement Secure Simplified Pairing must support the Passkey authentication method.

Exceptions: Not Specified

Business Justification:

This requirement ensures the security of user data being passed to the Windows system.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2009

Comments:

CONNECT-0097

Device.Connectivity.BluetoothDevices.RespondToServiceDiscoveryRequests

Target Feature: Device.Connectivity.BluetoothDevices

Title: Bluetooth Devices respond to Service Discovery requests before requiring authentication and while in inquiry scan state.

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 7 Client x64
- Windows 7 Client x86
- Windows 8 Client ARM

Description:

Service discovery must be completed before requiring authentication.

Bluetooth PC peripherals must support Security Specification as described in Specification of the Bluetooth System, Version2.1+EDR, Part H.

Exceptions: Not Specified

Business Justification:

This is a Bluetooth requirement.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

CONNECT-0007

Device.Connectivity.BluetoothDevices.SupportBluetooth21

Target Feature: Device.Connectivity.BluetoothDevices

Title: Devices which support Bluetooth must implement the Bluetooth 2.1 requirements

Applicable OS Versions:

- Windows 7 Client x86
- Windows 7 Client x64

Description:

The Bluetooth devices must comply with the Bluetooth 2.1 + EDR requirements outlined in Bluetooth Version 2.1 + EDR specifications.

Exceptions: Not Specified

Business Justification:

This ensures Windows users can take advantage of Bluetooth 2.1 features.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2012

Comments:

CONNECT-0096

Device.Connectivity.NearFieldProximity

Description:

Near Field Proximity is a set of short range wireless technologies to enable communication between a personal computer and a device.

Related Requirements:

Device.Connectivity.NearFieldProximity.DeviceNFCCertification
Device.Connectivity.NearFieldProximity.DeviceRangeOfActuation
Device.Connectivity.NearFieldProximity.DeviceTapToSetup
Device.Connectivity.NearFieldProximity.NfcForumTag
Device.Connectivity.NearFieldProximity.TouchMark

Device.Connectivity.NearFieldProximity.DeviceNFCCertification

Target Feature: Device.Connectivity.NearFieldProximity

Title: NFC Implementations must receive NFC certification

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Client ARM

Description:

A device that implements air interface specifications incorporated by the NFC Forum by reference as approved specifications must receive NFC Certification from the NFC Forum, inclusive of:

Wave 1 compliance

SNEP compliance

Exceptions:

None.

Business Justification:

Obtaining certification from the NFC forum ensures proper NFC behavior.

Scenarios:

This ensures proper NFC behavior.

Success Metric: Pass/Fail

Enforcement Date: Windows RC

Comments:

None

Device.Connectivity.NearFieldProximity.DeviceRangeOfActuation

Target Feature: Device.Connectivity.NearFieldProximity

Title: Proximity technology meets range of actuation

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Client ARM

Description:

A device that implements NFP technology must support an effective operating volume to enable users to successfully use NFP technology with Windows in 95 times out of 100 attempts for all tap and do scenarios. Refer to the most current version of the 'Windows 8 Near Field Proximity Implementation Specification' document for detailed placement guidance, as well as acceptable, minimum, and maximum values for the required effective operating volume. The spec can be found at:

http://go.microsoft.com/fwlink/?LinkId=237135

Exceptions:

None

Business Justification:

Ensures connections will only be established in a small range rather than larger ones (such as Bluetooth or wireless). Confines the range so that the appropriate technology is tested.

Scenarios:

When tapping two devices together, a connection must be established at a range at least to 4 cm and cannot be established at a range greater than 10 cm away from the local device with respect to the remote device.

Success Metric: Pass/Fail

Enforcement Date: Windows RC

Comments: Not Specified

Device.Connectivity.NearFieldProximity.DeviceTapToSetup

Target Feature: Device.Connectivity.NearFieldProximity

Title: Proximity device supports tap and setup

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

A device that supports pairing with Windows using NFP technology must implement the tap and setup scenario defined in more detail in the most current version of the 'Windows 8 Near Field Proximity Implementation Specification' document. The spec can be found at:

http://go.microsoft.com/fwlink/?LinkId=237135

Exceptions:

None

Business Justification:

Allows users to have confidence with proximity feature and ensures reliability.

Scenarios:

Enables common scenarios for proximity devices.

Success Metric: Pass/Fail above

Enforcement Date: Windows RC

Comments: Not Specified

Device.Connectivity.NearFieldProximity.NfcForumTag

Target Feature: Device.Connectivity.NearFieldProximity

Title: Tap and setup with passive devices

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Client ARM

Description:

A device that uses only an NFC Forum tag as the NFP technology must adhere to the NFC Forum NDEF specification.

Exceptions: Not Specified

Business Justification:

Ensures proper NFC behavior.

Scenarios:

Provider implementing NFC

Success Metric: Pass/Fail

Enforcement Date: Windows RC

Comments: Not Specified

Device.Connectivity.NearFieldProximity.TouchMark

Target Feature: Device.Connectivity.NearFieldProximity

Title: If using NFC as a proximity technology, then there must be a Microsoft Windows defined touch mark on the device.

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Client ARM

Description:

To help users locate and use the proximity technology, the use of a visual mark is required by, and is only permitted for, NFC NFP providers (those NFP providers that implement air interface specifications incorporated by the NFC Forum by reference as approved specifications). Refer to the most current version of the 'Windows 8 Near Field Proximity Implementation Specification' document for placement guidance and mark usage requirements. The spec can be found at:

http://go.microsoft.com/fwlink/?LinkId=237135

Exceptions: Not Specified

Business Justification:

Users will know where to tap devices together

Scenarios:

Users want to have a proximity experience and need to know where to tap devices together.

Success Metric: Pass/Fail

Enforcement Date: Windows RC

Comments: Not Specified

Device.Connectivity.Network.PnPX

Description:

Root for former "Rally" technologies

Related Requirements:

- Device.Connectivity.Network.PnPX.PnPX

Device.Connectivity.Network.PnPX.PnPX

Target Feature: Device.Connectivity.Network.PnPX

Title: A network-enabled device must implement PnP-X and comply with the PnP-X specification

Applicable OS Versions:

- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows 8 Client x64

Description:

A network-enabled device must comply with the Plug and Play Extensions (PnP-X) specification. Examples of network-enabled devices are network media players (digital media receiver [DMR]/digital media player [DMP]/networked media device [NMD]), network-attached storage (NAS)/digital media server (DMS), and 802.11 digital picture frames (DPFs).

To take advantage of PnP-X, the device manufacturer must support one of the inbox function discovery providers (Web Service Dynamic Discovery [WS-Discovery] and Simple Service Discovery Protocol [SSDP]) and provide the PnP-X metadata extensions in the Devices Profile for Web Services (DPWS) or a UPnP device.

If a device implements both UPnP and web services (such as DPWS), it must either use the same universally unique identifier (UUID) for the UPnP root device and the Web Services Endpoint Reference Address or use the ContainerID PnP-X tag in both transports with the same ContainerID value in both transports.

This requirement is focused on network-attached (IP-connected) devices. As such, computer-attached network cards (for example, USB-to-Ethernet dongles) are exempt from this requirement.

Design Notes:

For implementation details, see the PnP-X: Plug and Play Extensions for Windows Specification at http://go.microsoft.com/fwlink/?LinkID=123172.

For more information, see the "Vertical Pairing" white paper at http://www.microsoft.com/whdc/connect/rally/WiFiVerticalPair.mspx. This white paper also describes updates to PnP-X, such as the new namespace introduced for Windows 7.

Additional information and sample scan be found in the Windows Rally Development Kit at http://go.microsoft.com/fwlink/?LinkId=109368.

Discovery describes how Windows determines that a device is present. For physically connected devices, discovery occurs through Peripheral Component Interconnect (PCI), USB, and other physical bus enumerators. For network-connected devices (NCDs), Windows uses network communication protocols to discover the presence of a device. Under Windows, PnP-X allows network-connected devices to be discovered and installed on a client computer as if they were physically connected.

Although this requirement is marked "I" (If-Implemented), refer to the appropriate device class requirements to determine whether it is required for a specific device category.

Exceptions:

This requirement is focused on network-attached (IP-connected) devices. As such, computer-attached network cards (for example, USB-to-Ethernet dongles) are exempt from this requirement.

Business Justification:

NCDs must be as easy to use as bus-connected devices. PnP-X enables this ease of use. And, it identifies the PC as the best place to manage the increasing number of NCDs in the home. In addition, when an NCD uses PnP-X, it can take full advantage of the Windows Update infrastructure.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

CONNECT-0100

Device.Connectivity.Network.VerticalPairing

Description:

Root for former "Rally" technologies

Related Requirements:

- Device.Connectivity.Network.VerticalPairing.VerticalPairing
- Device.Connectivity.Network.VerticalPairing.WCN

Device.Connectivity.Network.VerticalPairing.VerticalPairing

Target Feature: Device.Connectivity.Network.VerticalPairing

Title: Network-attached devices must implement vertical pairing

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86

Description:

Devices must implement the Wi-Fi based Plug and Play Extensions (PnP-X) devices that adhere to vertical-pairing principles as outlined by the "Installing Wi-Fi Devices Using Rally Vertical Pairing" white paper at http://www.microsoft.com/whdc/connect/rally/WiFiVerticalPair.mspx.

Exceptions: Not Specified

Business Justification:

The intention is to ensure that such as device properly identifies the PnP-X device's universally unique identifier (UUID) and protocol during the Wireless Provisioning Services (WPS)/Wi-Fi

provisioning stage through the Microsoft vertical-pairing WPS vendor extensions, as outlined by the white paper.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Connectivity.Network.VerticalPairing.WCN

Target Feature: Device.Connectivity.Network.VerticalPairing

Title: An 802.11 network-enabled device that operates as a station (STA) must implement WCN-NET and meet basic 802.11 requirements

Applicable OS Versions:

- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows 8 Client x64

Description:

A 802.11 network-enabled device that operates as a station (STA) must meet the following requirements:

- The device must implement WCN-NET and comply with the specification.
- The device must implement WCN-NET vertical-pairing extensions and indicate whether it supports a PnP-X transport protocol. If the device supports a PnP-X transport protocol, it must ensure correct universally unique identifier (UUID) alignment.
- If WCN-UFD is implemented, it must comply with the specification.
- If the device has a display that capable of showing a four-digit or eight-digit number, it must support displaying a dynamic Windows Connect Now (WCN) PIN without user intervention. The PIN must be displayed for a minimum of two minutes after the device receives a Wireless Provisioning Services (WPS) M2D message with the value of "Windows" in the WPS Model Name attribute.
- If the device does not have a display that is capable of showing a four-digit or eight-digit number, it must provide a physical label affixed to the device that includes the eight-digit PIN and clearly labels the PIN value as a PIN (for example, PIN: 12345670).
- The device must be certified by the Wi-Fi Alliance, including Wi-Fi certification, Wi-Fi Protected Access 2 (WPA2) certification, and Wi-Fi Protected Setup certification.

Design Notes:

For implementation details, see the WCN-NET specification at http://go.microsoft.com/fwlink/?LinkId=109371 and the WCN-UFD specification at http://go.microsoft.com/fwlink/?LinkId=109372.

For more information, see the "Installing Wi-Fi Devices Using Rally Vertical Pairing" white paper at http://www.microsoft.com/whdc/connect/rally/WiFiVerticalPair.mspx.

Additional information can be found at http://go.microsoft.com/fwlink/?LinkId=109373 and http://go.microsoft.com/fwlink/?LinkId=109368.

WCN-NET is required. WCN-UFD is optional and is supported in Windows for backward compatibility with devices that are designed to support WCN functionality for Windows XP with Service Pack 2.

A device uses WCN-NET vertical-pairing extensions to indicate that its supports PnP-X. The device must provide a single UUID that is provided in both the WCN-NET exchange and the UPnP/Web Services for Devices (WSD) device file or provide the UPnP/WSD device UUID in the TransportUUID attribute of the WCN-NET vertical-pairing extension. The UUID that is provided in UPnP or WSD must be in lowercase (decimal digits can also be used).

For WSD implementations, the WSD UUID is provided as the endpoint reference address and must be of the form urn:uuid:. For UPnP implementations, the UPnP UUID is provided as the root device UUID.

Exceptions: Not Specified

Business Justification:

Devices that employ the previously mentioned specification will be easy for an end user to add to a wireless network.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

CONNECT-0099

Device.Connectivity.PciConnected

Description: Not Specified

Related Requirements:

- Device.Connectivity.PciConnected.64BitPrefetchableBar
- Device.Connectivity.PciConnected.ConfigurationSpaceCorrectlyPopulated
- Device.Connectivity.PciConnected.ExpressCardImplementsSerialNumber
- Device.Connectivity.PciConnected.InterruptDisableBit
- Device.Connectivity.PciConnected.MsiOrMsixSupport

- Device.Connectivity.PciConnected.PciAndPcixDevicesArePciCompliant
- Device.Connectivity.PciConnected.PCIExpress
- Device.Connectivity.PciConnected.SubsystemIdsRequired

Device.Connectivity.PciConnected.64BitPrefetchableBar

Target Feature: Device.Connectivity.PciConnected

Title: PCI-X and PCI Express devices that use prefetchable memory BARs, implement 64-bit prefetchable memory base address registers (BARs)

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Server x64
- Windows Vista Client x86
- Windows Vista Client x64

Description:

Devices that sit on the PCI-X or PCI Express bus and use prefetchable memory BARs must implement 64-bit prefetchable memory BARs on x64-based systems.

Design Notes:

See Firmware Allocation of PCI Device Resources in Windows

http://msdn.microsoft.com/en-us/windows/hardware/gg462986

If the device supports 64-bit prefetchable memory BARs, Windowsattempts to assign a region above 4 GB. In a PCI bridge, Windows ignores boot configuration for an entire device path emanating from the bridge in whose scope this method is defined. For the bridge and devices below it to be assigned a region above 4 GB, all devices in the path must support 64-bit prefetchable BARs. If this is not true, the rebalance code runs and moves all resource assignments below 4 GB, because the goal is to start as many devices as possible.

Refer to PCI local bus specifications revision 3.0, section 6.2.5 for information on Base Address Registers (BAR), and section 3.1.2 for information on the characteristics of prefetchable memory.

Exceptions: Not Specified

Business Justification:

To relieve memory resource constraints below 4GB on 64-bit systems, devices should support 64-bit prefetchable memory capabilities.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2007

Comments:

CONNECT-0038

Device.Connectivity.PciConnected.ConfigurationSpaceCorrectlyPopulated

Target Feature: Device.Connectivity.PciConnected

Title: Configuration space for PCI device is correctly populated

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64
- Windows Vista Client x86
- Windows Vista Client x64

Description:

PCI2.3 describes the configuration space that the system uses to identify and configure each device that is attached to the bus. The configuration space is made up of a header region and a device-dependent region. Each configuration space must have a 64-byte header at offset0. All the device registers that the device circuit uses for initialization, configuration, and catastrophic error handling must fit within the space between byte 64 and byte 255.

All other registers that the device uses during normal operation must be located in normal I/O or memory space. Unimplemented registers or reads to reserved registers must finish normally and return zero. Writes to reserved registers must finish normally, and the data must be discarded.

All registers that the device requires at interrupt time must be in I/O or memory space.

Exceptions: Not Specified

Business Justification:

To support PCI scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2006

Comments:

CONNECT-0033

Device.Connectivity.PciConnected.ExpressCardImplementsSerialNumber

Target Feature: Device.Connectivity.PciConnected

Title: A single ExpressCard module that supports both USB and PCI Express interfaces implements a common serial number

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64
- Windows Vista Client x86
- Windows Vista Client x64

Description:

An ExpressCard module that supports both USB and PCI Express interfaces on a single module must implement the common serial number as described in the PCI ExpressCard Electromechanical Specification.

This is the only method that Windows will use to determine the relationship of USB and PCI Express on one module.

Exceptions: Not Specified

Business Justification:

This is the ONLY method the OS will use to determine the relationship of the two buses on one module.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2006

Comments:

CONNECT-0017

Device.Connectivity.PciConnected.InterruptDisableBit

Target Feature: Device.Connectivity.PciConnected

Title: PCI and PCI-X devices, that are PCI 2.3 compliant, support the interrupt-disable bit

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64
- Windows Vista Client x86
- Windows Vista Client x64

Description:

All PCI and PCI-X devices that claim support for PCI Local Bus Specification Revision 2.3 or later, must support the interrupt-disable bit.

Design Notes: See PCI Local Bus Specification, Revision2.3, Section6.2.2.

Exceptions: Not Specified

Business Justification:

To support standards for the support of interrupt disable bit.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2006

Comments:

INPUT-0037

Device.Connectivity.PciConnected.MsiOrMsixSupport

Target Feature: Device.Connectivity.PciConnected

Title: PCI device that reports PCI-X capability in the PCI configuration space and that generates interrupts may support MSI or MSI-X but is not required to do so

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Server x64
- Windows Vista Client x86
- Windows Vista Client x64

Description:

As part of the PCI Conventional Specification 2.2 or later, PCI-X Addendum, Section 3.2, all PCI-X devices that generate interrupts must support message-signaled interrupts.

For MSI implementation, MSI capabilities must be implemented in the configuration space.

For MSI-X implementation, MSI-X capabilities must be implemented in the configuration space.

However, because PCI-X is being replaced by PCI Express and many existing implementations do not support MSI or MSI-X, this requirement will not be applicable if either the MSI or MSI-X implementations are not part of the design.

Any device that claims to support MSI or MSI-X must support message-signaled interrupts or will fail the relevant WDK tests.

Design Notes:

Message Signaled Interrupt for PCI-X device is required by industry standard specification. However, see above.

Exceptions: Not Specified

Business Justification:

To support PCI scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2006

Comments:

CONNECT-0042

Device.Connectivity.PciConnected.PciAndPcixDevicesArePciCompliant

Target Feature: Device.Connectivity.PciConnected

Title: PCI and PCI-X devices, at a minimum, are PCI 2.1 compliant

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows Vista Client x86
- Windows Vista Client x64
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64

Description:

All PCI and PCI-X devices must comply with the PCI Local Bus Specification, Revision 2.1 or later. This requirement applies to devices on X86, IA64 and x64 systems.

Design Notes:

See PCI Local Bus Specification, Revision 2.1 or later.

Exceptions: Not Specified

Business Justification:

To support PCI scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2007

Comments:

Connect-0069

Device.Connectivity.PciConnected.PCIExpress

Target Feature: Device.Connectivity.PciConnected

Title: PCI Express requirement

Applicable OS Versions:

- Windows 8 Client x86

- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Server x64
- Windows Vista Client x86
- Windows Vista Client x64

Description:

1. Device driver for PCI Express device does not modify VC Enable settings:

The device driver must not modify the "VC Enable" bit (PCI Express Base Specification, Version1.0a, Section7.11.7, VC Resource Control Register: bit 31) for any of the devices extended (non-VC0) virtual channel or channels.

2. **PCI Express link active state L1 exit latency does not exceed 64 S:**

A PCI Express link, between root complex and device or bridge, cannot have an active state L1 exit latency of more than 64 microseconds on systems unless the link is associated with a PCI Express cable; that is, a value of 111b cannot be reported in the link capabilities register field 17:15. See PCIe Express Base Specification, Revision 1.0a, Section 7.8.6.

3. **PCI Express hot-plug port that supports firmware-controlled hot plug uses the _OSC control method for enable and disable**:

All PCI Express hot-plug ports that support firmware-controlled hot-plugs must support the_OSC control method for enabling and disabling firmware-controlled hot-plug as described in the PCI Firmware Specification Version 3.0. See PCI Express Base Specification, Revision 1.1, Section 6.7.4.

4. **PCI Express component implements a single device number on its primary interface:**

Every PCI Express component (that is, physical device) is restricted to implementing a single device number on its primary interface (upstream port), but it may implement up to eight independent functions within that device number. See PCI Express Base Specification, Revision1.1 (or later), Section 7.3.1.

5. PCI Express device implements support for MSI or MSI-X:

MSI support, which is optional for PCI 2.1, PCI 2.2, and PCI 2.3 devices, is required for PCI Express devices. This capability can be either implemented as MSI or MSI-X. See PCI Express Base Specification, Revision1.0a, Section 6.1.

6. **PCI Express root complex supports the enhanced (memory-mapped) configuration space access mechanism:**

The root complex must support the enhanced configuration space access mechanism as defined in PCI Express Base Specification, Revision 1.1 (or later), Section 7.9.

7. **PCI Express device that can interrupt supports the interrupt disable bit:**

If an interrupt is implemented, PCI Express devices must support the interrupt disable functionality described in PCI Local Bus Specification, Revision2.3. This bit disables the device or function from asserting INTx. A value of 0 enables the assertion of its INTx signal. A value of 1 disables the assertion of its INTx signal. This bits state upon reset is 0

Exceptions: Not Specified

Business Justification:

Windows doesn't support non-VC0 virtual channels.

Standardized HW equals stable code. If we have multiple HW implementations then there are more code paths to consider. Some requirements are for functionality that will help the system become more reliable. This includes Interrupt Disable Functionality, Devices that map their internal registers into I/O space, Infinite Flow Control (FC) units etc.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2006

Comments:

CONNECT-0034

Device.Connectivity.PciConnected.SubsystemIdsRequired

Target Feature: Device.Connectivity.PciConnected

Title: Device IDs include PCI subsystem IDs

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Server x64
- Windows Vista Client x86
- Windows Vista Client x64

Description:

The SID and SVID fields must comply with the SID requirement in PCI Local Bus Specification 2.3 and the implementation details in "PCI Device Subsystem IDs and Windows."

AMR devices and MR devices on the system board are not exempt from the requirement for SID and SVID.

SVID is not required for PCIe to PCI/PCI-X bridges.

Design Notes:

See "PCI Device Subsystem IDs and Windows" at http://msdn.microsoft.com/en-us/windows/hardware/gg463287.aspx.

Exceptions: Not Specified

Business Justification:

To support PCI scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2006

Comments:

CONNECT-0032

Device.Connectivity.UsbDevices

Description:

Applies to all devices connected via USB including USB Hubs. Does not apply to USB Controllers

Related Requirements:

- Device.Connectivity.UsbDevices.Addressing
- Device.Connectivity.UsbDevices.AlternateDriver
- Device.Connectivity.UsbDevices.CompliesWithChap9
- Device.Connectivity.UsbDevices.DebugCompliesWithDebugSpec
- Device.Connectivity.UsbDevices.DebugCompliesWithDebugSpecUSB3
- Device.Connectivity.UsbDevices.DeviceAttachLessThan100ms
- Device.Connectivity.UsbDevices.EsdRecovery
- Device.Connectivity.UsbDevices.FunctionSuspendSelectiveSuspend
- Device.Connectivity.UsbDevices.InstallViaUniquePnpIdentifier
- Device.Connectivity.UsbDevices.IsochronousDeviceAndDriver
- Device.Connectivity.UsbDevices.MsOsContainerId
- Device.Connectivity.UsbDevices.MustBeFunctionalAfterResume
- Device.Connectivity.UsbDevices.MustEnumerateOnEhciAndXhci
- Device.Connectivity.UsbDevices.MustNotDisconnectDuringSuspend

- Device.Connectivity.UsbDevices.MustResumeWithoutForcedReset
- Device.Connectivity.UsbDevices.MustSignalAttachWithin500ms
- Device.Connectivity.UsbDevices.MustSupportSuspend
- Device.Connectivity.UsbDevices.PeripheralOperatesInFunctionMode
- Device.Connectivity.UsbDevices.PortMove500ms
- Device.Connectivity.UsbDevices.RespondAllStringRequests
- Device.Connectivity.UsbDevices.ResponsesLimitedByWlengthField
- Device.Connectivity.UsbDevices.SerialNumbers
- Device.Connectivity.UsbDevices.SerialNumbersUseValidCharacters
- Device.Connectivity.UsbDevices.SuperSpeedOnConnectViaUsb3Port
- Device.Connectivity.UsbDevices.TestedUsingMicrosoftUsbStack
- Device.Connectivity.UsbDevices.Usb3CompatibleWithDownLevel
- Device.Connectivity.UsbDevices.UsbifCertification
- Device.Connectivity.UsbDevices.UseUsbClassOnlyForControllerOrHub
- Device.Connectivity.UsbDevices.WirelessUsbObtainsWusbLogoFromUsbif
- Device.Connectivity.UsbDevices.WirelessUsbWiMediaAlliace

Device.Connectivity.UsbDevices.Addressing

Target Feature: Device.Connectivity.UsbDevices

Title: USB device can be configured to any USB address

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

To ensure that the devices function at all device address ranges, USB devices must be able to be configured to any USB address that the host provides. The device must enumerate and function correctly at any address ranging from 1 to 127. See USB Specification, Revision 2.0 or later, Sections9.1.1.4 and 9.4.6.

Justification:

Reduces resume time and maintains device state on resume from S3.

Exceptions: Not Specified

Business Justification:

To support USB connectivity functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

Connect-0052

Device.Connectivity.UsbDevices.AlternateDriver

Target Feature: Device.Connectivity.UsbDevices

Title: USB devices should allow the Operating System to load an alternate driver on device

Applicable OS Versions:

- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows Server 2008 Release 2 x64

Description:

Devicesmust retain basic USB functionality after a driver re-enumeration occurs. For example, a driver that redirects I/O to alternate paths requires re-enumeration. A common way to perform USB redirection is to issue an IOCTL_USB_CYCLE_PORT report on the target driver to trigger re-enumeration of the physical device.

Justification:

When a virtual PC needs to connect to a physical USB device, the virtual PC loads a driver that is called an "Alternate Driver". This event triggers re-enumeration. After re-enumeration occurs many times, the physical device can no longer function. This requirement attempts to address this problem.

Exceptions:

This Requirement does NOT apply to Systems that Support Connected Standby

Business Justification:

To support USB connectivity functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: Not Specified

Comments:

Connect-0123: Does NOT apply to SOC Implementations

Device.Connectivity.UsbDevices.CompliesWithChap9

Target Feature: Device.Connectivity.UsbDevices

Title: USB device complies with implementation details from the USB specification

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

Any devices that are connected externally or internally to a USB port must be tested as USB devices that is, the devices provide the capabilities of one or more functions, a hub to the host, or both, as described in USB Specification, Revision 2.0 or later, Chapters 9 and 11. Therefore, these requirements apply to any device that is connected to a USB port: the USB specification and any related USB device class specification, and the Windows Hardware Certification program requirements for USB and the related device class.

Exceptions: Not Specified

Business Justification:

To support USB connectivity functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

Connect-0044

Device.Connectivity.UsbDevices.DebugCompliesWithDebugSpec

Target Feature: Device.Connectivity.UsbDevices

Title: USB debug device complies with USB2 Debug Device Specification

Applicable OS Versions:

- Windows Server 2008 x86
- Windows 8 Server x64
- Windows 8 Client x64
- Windows 8 Client x86
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86
- Windows Server 2008 x64
- Windows 8 Client ARM

Description:

USB devices designed for debug purposes over USB 2.0 must comply with USB2 Debug Device Functional Specification, which includes details on the device framework, commands, and additional operational requirements.

Exceptions: Not Specified

Business Justification:

To support USB connectivity functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

CONNECT-0063

Device.Connectivity.UsbDevices.DebugCompliesWithDebugSpecUSB3

Target Feature: Device.Connectivity.UsbDevices

Title: USB3 debug device complies with USB3 Debug Device Specification

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client x64
- Windows 8 Client x86

- Windows 8 Client ARM
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64

Description:

USB devices designed for debug purposes over USB 3.0 must comply with USB3 Debug Device Functional Specification, which includes details on the device framework, commands, and additional operational requirements.

Exceptions: Not Specified

Business Justification:

To support USB connectivity functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: June 30, 2012

Comments:

CONNECT-8063

Device.Connectivity.UsbDevices.DeviceAttachLessThan100ms

Target Feature: Device.Connectivity.UsbDevices

Title: USB device that signals device-attach responds after at least 100 ms

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

When the USB device has signaled device-attach, the operating system provides a debounce interval of 100ms. The device must respond at the end of that interval. This is described in USB Specification, Revision2.0, Section 7.1.7.3. This requirement ensures that the electrical and mechanical

connections are stable before the attached device is reset.

Justification:

Some devices after a few iterations of going into s3 fail to show up on the device tree

Exceptions: Not Specified

Business Justification:

To support USB connectivity functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

Connect-0104

Device.Connectivity.UsbDevices.EsdRecovery

Target Feature: Device.Connectivity.UsbDevices

Title: USB device does not trigger ESD recovery on USB hubs

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

Devices must not trigger ESD recovery when connected to a USB hub. Triggering ESD causes fail-safe code to be executed in the hub driver and negatively affects the functionality of other devices attached to the hub. This is described in USB Specification, Revision 2.0 or later, Sections 7.2.3.

Justification:

Triggering ESD causes fail safe code to be executed in the hub driver and will impact the functionality of other devices attached to the hub.

Exceptions: Not Specified

Business Justification:

To support USB connectivity functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

Connect-0053

Device.Connectivity.UsbDevices.FunctionSuspendSelectiveSuspend

Target Feature: Device.Connectivity.UsbDevices

Title: USB 3.0 devices correctly implement Function Suspend and Selective Suspend

Applicable OS Versions:

- Windows Server 2008 x86
- Windows 8 Server x64
- Windows 8 Client x64
- Windows 8 Client x86
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86
- Windows Server 2008 x64
- Windows 8 Client ARM

Description:

Any function that is in a suspend state before a device is selectively suspended remains in the function suspend state when the device is resumed from the selective suspend state. Devices must not place all device functions in the function suspend state. Devices must report the selective suspend state.

SuperSpeed devices ignore the DEVICE_REMOTE_WAKEUP feature selector.

When all functions of a SuperSpeed device are in the function suspend state and the PORT_U2_TIMEOUT field is programmed to 0xFF, the device initiates U2 after 10 milliseconds (ms) of link inactivity. For more information, see section 9.2 of the USB 3.0 Specification.

Devices that are resumed from the selective suspend state retain a minimum set of device state information as specified in section 9.2.5.2 of the USB 3.0 Specification.

Exceptions: Not Specified

Business Justification:

To support USB connectivity functionality and scenarios. Function suspend is a new feature in USB 3.0. This requirement helps to clear up some of the confusion around how this feature will function. Because this new feature is very important for power savings, we plan more thorough testing in the Windows Logo Kit (WLK).

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 12/1/2010

Comments:

CONNECT-0131

Device.Connectivity.UsbDevices.InstallViaUniquePnpIdentifier

Target Feature: Device.Connectivity.UsbDevices

Title: Third-party drivers for USB devices install through a unique PnP identifier match or a compatible ID match when allowed

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

All Universal Serial Bus (USB) devices must have VID and PID sections in the PnP Device ID string. Third-party USB function drivers must not install through a compatible ID match, unless specifically permitted for a particular technology. The list of devices drivers that can be installed using a compatible ID match (Class, SubClass, Prot) is the following:

WirelessUSB Cable Association Driver (CLASS_EF&SUBCLASS_03&PROT_01)

Exceptions: Not Specified

Business Justification:

To support USB connectivity functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2008

Comments:

CONNECT-0082

Device.Connectivity.UsbDevices.IsochronousDeviceAndDriver

Target Feature: Device.Connectivity.UsbDevices

Title: Isochronous USB device and driver requirement

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

1. ISO USB device and driver provide multiple alternate settings to support maximum flexibility of hardware interface options:

If any alternate setting consumes isochronous bandwidth, devices and drivers must provide multiple alternate settings for each interface.

2. USB device and driver do not use isochronous bandwidth for alternate setting 0:

Devices and drivers must not use isochronous bandwidth for alternate setting 0. Devices must consume bandwidth only when they are in use.

3.USB isochronous full-speed or high-speed device that uses more than 50 percent of USB bus bandwidth provides alternate settings:

If a USB isochronous full-speed or high-speed device uses more than 50 percent of USB bus bandwidth, it must provide alternative settings that allow the device to switch to a setting that uses less than 50 percent of the bus bandwidth and operate as a device of that particular class (ex. if it is a camera then it must continue to stream video), even though it may be in a lower quality mode.

Devices with attached host controllers are exempt from this requirement.

4. USB device with interfaces containing isochronous endpoints has at least one alternative interface for low bandwidth scenarios:

USB device must provide at least one alternative interface for low bandwidth as described in USB Specification, Revision 2.0 or later, Section 9.6.5.

Design Notes:

See section 9.6.5 in the Universal Serial Bus Specification, Revision 2.0.

If two or more devices are connected that use more than 50 percent of the bus bandwidth and do not provide alternate settings, only one of the devices works at a time.

See USB Specification, Revision 2.0or later, Sections 5.6 and 5.7.

Justification: Alternate settings are at the interface descriptor level, specifying different amounts of bandwidth a device requests the host controller to reserve for it. Imagine a conference camera with settings like 50% ,30%, 20%. It starts by asking the host if 50% is available and whether the host can reserve it. If not, then the host chooses the next alternate setting till they have a mutual agreement.

Exceptions: Not Specified

Business Justification:

To support USB connectivity functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

CONNECT-0048

Device.Connectivity.UsbDevices.MsOsContainerId

Target Feature: Device.Connectivity.UsbDevices

Title: USB devices which implement the MS OS Container ID descriptor implement it correctly

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

If a multifunction USB device implements the Microsoft operating system **ContainerID** descriptor, the device does this in the Microsoft operating system feature descriptor.

The Microsoft operating system **ContainerID** descriptor allows Windows to correctly detect multifunction devices. The descriptor provides a way for all the device nodes to appear as one physical object in the **Devices and Printers** user interface (UI).

Exceptions: Not Specified

Business Justification:

To support USB connectivity functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2010

Comments:

CONNECT-0120

Device.Connectivity.UsbDevices.MustBeFunctionalAfterResume

Target Feature: Device.Connectivity.UsbDevices

Title: Attached USB devices must be functional after resuming from system power states.

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

Devices not entering a timely ready state will be marked code 10 or other by the system. Certain classes of devices do not properly respond to system events, such as resume, and require upper driver or expect precise boot timings in order to function properly. Device also must be able to function without a port reset upon resume but also remain functional if a reset does occur.

A device must be in the attached state (USB Specification 2.0, section 9.1) to be configured and device must be in the configured state before its functions maybe used (aka, the device is useable). This is per the USB spec 2.0 as in section 9.1 and 9.2.6.2 After a port is reset or resumed, the USB

System Software is expected to provide a recovery interval of 10 ms before the device attached to the port is expected to respond to data transfers. The device may ignore any data transfers during the recovery interval."

Clarification:

Devices must be functional after resuming from system power states whether a port reset is issued or not.

Justification:

Devices not entering a timely ready state will be marked code 10 or other by system. Certain classes of devices do not have sufficient resources to handle system events, such as resume, and require upper driver or expect precise boot timings in order to function properly.

Exceptions: Not Specified

Business Justification:

To support USB connectivity functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2009

Comments:

Connect-0109

Device.Connectivity.UsbDevices.MustEnumerateOnEhciAndXhci

Target Feature: Device.Connectivity.UsbDevices

Title: All USB devices must enumerate and operate on EHCI and xHCI controllers as well as downstream of full speed, high speed, and SuperSpeed USB hubs

Applicable OS Versions:

- Windows Server 2008 x86
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM
- Windows Server 2008 Release 2 x64
- Windows 8 Server x64
- Windows Server 2008 x64

Description:

All USB devices must enumerate and operate as a user would expect on Enhanced Host Controller Interface (EHCI) controllers and Extensible Host Controller Interface (xHCI) controllers. All USB devices must also operate as a user would expect downstream of a full-speed, high-speed or SuperSpeed hub.

The following scenarios will be covered in this requirement:

- EHCI + device under test (DUT)
- EHCI + high-speed hub + full-speed hub + DUT
- EHCI + high-speed hub + DUT
- xHCI + DUT
- xHCI + SuperSpeed hub + DUT
- xHCI + High speed hub + DUT

When selecting a system with xHCI controllers for testing this requirement, note that some of these hubs may already be integrated into the controller chipset or embedded into the system that is under testing. External physical ports must be checked for their exact routing to the xHCI controller to test the correct scenario.

Exceptions: Not Specified

Business Justification:

To support USB connectivity functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2011

Comments:

Connect-0139

Device.Connectivity.UsbDevices.MustNotDisconnectDuringSuspend

Target Feature: Device.Connectivity.UsbDevices

Title: USB devices must not disconnect from the upstream port while going to or resuming from selective suspend.

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64

- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

USB devices must not disconnect from the upstream port during the selective suspend process.

To test this requirement, we will cause the device to go into the selective suspend state and then resume the device. During this process, we will observe the port status bits of the upstream port and verify that the device does not disconnect during this time. We will repeat this process several times.

Exceptions: Not Specified

Business Justification:

To support USB connectivity functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2011

Comments:

CONNECT-0142

Device.Connectivity.UsbDevices.MustResumeWithoutForcedReset

Target Feature: Device.Connectivity.UsbDevices

Title: All USB devices work properly upon resume from sleep, hibernation or restart without a forced reset of the USB host controller

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM
- Windows Server 2008 Release 2 x64

Description:

All USB devices work properly upon resume from sleep, hibernation or restart without a forced reset of the USB host controller

Design Notes:

Registry key ForceHCResetOnResume documented at the KB below is not needed for devices to function properly upon resume in Windows 7.

http://support.microsoft.com/kb/928631

Note that a known set of currently existing devices do require a forced reset upon resume, these devices should be covered in a list kept by the OS which will reset these devices upon resume. The goal of this requirement is to ensure that this list of devices which need a reset to appear after resume does not grow and that devices can properly handle sleep state transitions without being reset.
A reset of the entire USB Host Controller results in significantly increased time that it takes for all USB devices to become available after system resume since there could be only one device at address 0 at a time, this enumeration has to be serialized for all USB devices on the bus. We have also seen that resetting the host controller can lead to an illegal SE1 signal state on some host controllers, which in turn can cause some USB devices to hang or drop off the bus. Moreover, devices cannot maintain any private state across sleep resume as that state will be lost on reset.

Exceptions: Not Specified

Business Justification:

To support USB connectivity functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2010

Comments:

Connect-0126: Removed Host controller from requirement title and body. created a new requirement specifically for the Host Controller.

Device.Connectivity.UsbDevices.MustSignalAttachWithin500ms

Target Feature: Device.Connectivity.UsbDevices

Title: Devices must signal attach within 500 ms after the system resumes.

Applicable OS Versions:

- Windows Server 2008 x86
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64

- Windows Server 2008 x64
- Windows 8 Client ARM

Description:

After the system resumes from sleep, the hub driver will fetch the status of the port to which the device is connected. If the device does not show as connected, the hub driver will wait 500milliseconds (ms) before the hub driver queries the port status again. If the device appears as connected within that time, the hub will not need to re-enumerate the device for Plug and Play, which will result in a better user experience. This requirement already exists for bus-powered devices in the USB specification. The requirement will now also apply to self-powered devices.

Justification:

Some devices take a long time to be discoverable by the operating system. This requirement does not try to enforce the device being back from sleep from a functional point of view. A certain amount of time is necessary to know if the device remains on the bus.

Exceptions: Not Specified

Business Justification:

To support USB connectivity functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2011

Comments:

Connect-0140

Device.Connectivity.UsbDevices.MustSupportSuspend

Target Feature: Device.Connectivity.UsbDevices

Title: All Bus powered USB devices support USB Suspend after periods of inactivity

Applicable OS Versions:

- Windows Server 2008 x86
- Windows 8 Server x64
- Windows 8 Client x64
- Windows 8 Client x86
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86
- Windows Server 2008 x64
- Windows 8 Client ARM

Description:

Bus powered devices can go into the Suspend state from any powered state. They begin the transition to the Suspend state after they see a constant Idle state on their upstream facing bus lines for more than 3.0 ms. The device must actually be suspended, drawing only suspend current from the bus after no more than 10 ms of bus inactivity on all its ports, as described in the USB Specification, Revision 2.0, Sections 7.1.7.6, 6 9.1.1.6 and 9.2.6.2

Justification:

Existing requirement - required based on industry foundation, standards

Exceptions: Not Specified

Business Justification:

To support USB connectivity functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

CONNECT-0094

Device.Connectivity.UsbDevices.PeripheralOperatesInFunctionMode

Target Feature: Device.Connectivity.UsbDevices

Title: USB peripheral operates in function mode when connected to a computer host controller

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

When connected to a systems host controller, a USB peripheral must operate as a USB function only. A function is a USB device that provides additional capabilities to the host controller. This is described in USB Specification, Revision 2.0 or later, Section 4

Exceptions: Not Specified

Business Justification:

To support USB connectivity functionality and scenarios according to the USB Specification.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

Connect-0056

Device.Connectivity.UsbDevices.PortMove500ms

Target Feature: Device.Connectivity.UsbDevices

Title: If the software enables the SuperSpeed and then resets the 2.0 port, device should correctly move over

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

In a USB 3.0 hub, two downstream ports, a USB 2.0 port and a SuperSpeed port, share the same connector. The USB 3.0 Specification says that if a device is currently connected on the USB 2.0 port, the device should try to reconnect at the SuperSpeed port when software resets the port.

This situation has two relevant time intervals: The time that is necessary for the device to disconnect from the USB 2.0 port, and the total time that is necessary for the device to reconnect on the SuperSpeed port.

The time between the time that the reset starts and the time that the device starts RxDetect on USB 3.0 can be up to 1 millisecond (ms). RxDetect can take up to 16 ms in the worst case.The time between the time that RxDetect succeeds and the time that RxDetect signals a disconnect on USB 2.0 can be up to 1 ms. Therefore, the total time from the start of the reset to disconnect signaling can be up to 18 ms.Accounting for the polling interval of the hub, the total comes to 50 ms.

Next, the time between the time that RxDetect succeeds and the time that the device enters U0 can be up to 300 ms.With an extra margin for hub control transfers and other delays, the total comes to 500 ms.

To test this requirement, we will first disable the SuperSpeed termination, then re-enable SuperSpeed termination after some time, and then reset the USB 2.0 port. After these actions occur, the software will wait for a port status change notification on the USB 2.0 port and the SuperSpeed port. We will verify that the USB 2.0 port shows a status change that indicates that the device disconnected from the USB 2.0 port and that the SuperSpeed port then shows a status change that indicates that the device connected on the SuperSpeed port.

For more information, see Figure 9-1 in section 9.1.1 of the USB 3.0 Specification.

Exceptions:

This Requirement does NOT apply to Systems that Support Connected Standby

Business Justification:

To support USB connectivity functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2011

Comments:

Connect-0135: does NOT apply to SOC Implementations

Device.Connectivity.UsbDevices.RespondAllStringRequests

Target Feature: Device.Connectivity.UsbDevices

Title: USB device responds to all string requests that the host sends to indexes

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

USB devices must respond accordingly to string requests that the host sends. Devices must stall if no string is stored at the index being queried or if a request error exists. Devices must not reset themselves or stop functioning. This is described in USB Specification, Revision 2.0 or later, Section 9.6.

Exceptions: Not Specified

Business Justification:

To support USB connectivity functionality and scenarios according to the USB Specification and prevent abnormal behavior due discrepancies between the indexes being queried and the presence of information stored.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

Connect-0054

Device.Connectivity.UsbDevices.ResponsesLimitedByWlengthField

Target Feature: Device.Connectivity.UsbDevices

Title: USB device responses to host requests are limited in size by the wLength field

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

All USB device requests contain a wLength field. Responses by the USB device to host requests must be of size <= wLength field of the device request as defined in the USB Specification, Revision1.1 or later, Section 9.3.5.

Exceptions: Not Specified

Business Justification:

To support USB connectivity functionality and scenarios according to the USB Spec and prevent potential buffer over runs and security issues if the devices respond with more data than expected orillegal data.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

Connect-0059

Device.Connectivity.UsbDevices.SerialNumbers

Target Feature: Device.Connectivity.UsbDevices

Title: USB serial numbers are implemented for specific device classes and are unique across specific device models

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

USB serial numbers must be implemented for the following device classes:

Bluetooth (Class Code 0xE0, SubClass 0x01, Protocol 0x01)

Communication device class (Class Code 0x02)

Mass storage (Class Code 0x08)

Scanning/imaging (Class Code 0x06)

Printing (Class Code 0x07)

Host Wire Adapters and Device Wire Adapters (Class Code 0xE0, subclass 02)

USB serial numbers are optional for all other device classes. Additionally, if serial numbers are implemented on the devices model, all devices of the same model must have unique serial numbers.

Design Notes:

For more information on USB device class details, see "Defined 1.0 Class Codes" at http://go.microsoft.com/fwlink/?LinkId=40497.

For more information on implementation of serial numbers, see USB Specification, Revision 2.0 or later, Section 9.6.

Exceptions: Not Specified

Business Justification:

To support USB connectivity functionality and scenarios. Having duplicate serial numbers detriments the end user experience serial numbers should identify devices uniquely. If two devices with the same serial number are connected to a system, Windows will treat one of them as if it didntdidn't have a serial number. Properly-implemented serial numbers improve the Windows user experience as described below. Justification for requiring serial numbers for certain device classes: We dontdon't mandate serial numbers everywhere because of cost. Serial numbers improve the user experience with any USB device in general with Windows, because they allow Windows to keep track of devices no matter what USB port they are plugged into. When the user plugs in a device without a serial number to a new USB port, the device is setup as if it has never been seen before (a process that takes several seconds and notifies the user). Devices with serial numbers are enumerated faster, if the same device has been connected to the system on a different port before. Another benefit of being able to keep track of devices via serial number is that per-device data, such as user-visible settings and internal driver data, can stick to the device even when the device is plugged into a new port. The classes called out in this requirement have device parameters that shouldnt change when moved from port to port. Hence the devices must have a serial number. Without serial numbers the user would see the following behavior when plugging a device into a new port: Bluetooth radios would lose all LINKS with their devices and all devices downstream of Bluetooth would need to be re-detected and re-bonded. Modems would lose their connectoids. Mass storage devices would lose their drive letter and performance settings. Printers would lose their print queue characteristics and connections, leaving a dangling orphaned device in printer control panel. Devices that dontdon't generally move around (like keyboards and mice) dontdon't need the extra expense of serial numbers, so they arentaren't on the list.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

Connect-0045

Device.Connectivity.UsbDevices.SerialNumbersUseValidCharacters

Target Feature: Device.Connectivity.UsbDevices

Title: USB device that implements manufacturer-defined serial numbers contains valid characters

Applicable OS Versions:

- Windows Server 2008 x86
- Windows 8 Server x64
- Windows 8 Client x64
- Windows 8 Client x86
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86
- Windows Server 2008 x64
- Windows 8 Client ARM

Description:

A USB serial number must be a string that contains a manufacturer-determined ID composed of valid characters. Valid characters are defined in the Windows Driver Kit, "USB_DEVICE_DESCRIPTOR."

Justification:

Devices with invalid serial numbers force drivers to remove these invalid characters when reporting the info to PNP. Doing so may cause two devices with different serial numbers (the numbers different via an invalid char) to be reported to PNP as having the same serial number.

Exceptions: Not Specified

Business Justification:

To support USB connectivity functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

CONNECT-0062

Device.Connectivity.UsbDevices.SuperSpeedOnConnectViaUsb3Port

Target Feature: Device.Connectivity.UsbDevices

Title: If upstream SuperSpeed termination is on, devices must always connect on the USB 3.0 port and never connect on the USB 2.0 port

Applicable OS Versions:

- Windows Server 2008 x86
- Windows 8 Server x64
- Windows 8 Client x64

- Windows 8 Client x86
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86
- Windows Server 2008 x64
- Windows 8 Client ARM

Description:

In a USB 3.0 hub, two downstream ports, a USB 2.0 port and a Superspeed port, share the same connector. When a SuperSpeed (that is, non-hub) device is plugged into such a connector, the device must always connect on the SuperSpeed port. The device must never connect on the USB 2.0 port.

To test this requirement, the software will verify that the USB 3.0 port status bits show a connected device and that the USB 2.0 port status bits do not show a connected device.

For a definition of connect, see section 2 of the USB 3.0 Specification under connected.

Exceptions:

This requirement does NOT apply to Systems that Support Connected Standby

Business Justification:

To support USB connectivity functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2011

Comments:

CONNECT-0141: does NOT apply to SOC Implementations

Device.Connectivity.UsbDevices.TestedUsingMicrosoftUsbStack

Target Feature: Device.Connectivity.UsbDevices

Title: USB Devices must be tested with Microsoft's xHCI Stack installed

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64

- Windows 8 Server x64
- Windows 8 Client ARM

Description:

All USB Devices (Low, Full, High and Super Speed devices) must be tested with Microsofts Extensible Host Controller Interface (xHCI) Stack installed and enabled on a Windows system.

Note: During USB-IF self testing a specific USB Test Stack is installed for testing purposes, this is expected and acceptable.

Exceptions: Not Specified

Business Justification:

To support USB connectivity functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: Windows 8 RC

Comments:

New

Device.Connectivity.UsbDevices.Usb3CompatibleWithDownLevel

Target Feature: Device.Connectivity.UsbDevices

Title: USB 3.0 devices are backwards compatible with down level controllers and hubs

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

USB 3.0 devices both enumerate and function on USB 2.0 controllers. USB 3.0 devices also need to enumerate and function when USB 3.0 devices are connected to a full-speed hub or to a high-speed hub, according to section 5.3.1.1 of the USB 2.0 specification. Function means to operate as a user would expect a device of that class to operate.

Design Notes:

USB 3.0 devices must be able to do the following:

- Reset successfully at high and full speeds.
- Respond successfully to standard requests at high and full speeds for device and configuration descriptors. Standard requests are set_address, set_configuration, and get_descriptor.
- Return appropriate information.

USB 3.0 devices must also be able to function at a minimum level, although USB 3.0 devices will not operate at super speed. For example, a USB 3.0 mass storage device must be able to transfer files at full speed mode as well as at super speed mode. Although data transfer is much slower at the lower speed, data transfer is still possible.

Exceptions: Not Specified

Business Justification:

To support USB connectivity functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2010

Comments:

CONNECT-0130

Device.Connectivity.UsbDevices.UsbifCertification

Target Feature: Device.Connectivity.UsbDevices

Title: USB IF Tests are passing or device is USB IF certified

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

Effective June 1, 2011, USB devices must pass USB Implementers Forum (IF) tests or be USB-IF certified.

See white paper on Windows Logo Kit USB-IF Testing

at http://www.microsoft.com/whdc/connect/usb/wlk-usb-if-testing.mspx

Justification:

This is an existing requirement that is based on industry specifications.

Exceptions: Not Specified

Business Justification:

To support USB connectivity functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2011

Comments:

Connect-0093

Device.Connectivity.UsbDevices.UseUsbClassOnlyForControllerOrHub

Target Feature: Device.Connectivity.UsbDevices

Title: Third-party INF files include the class “USB” only if the device is a USB host controller, a root, or an external hub

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

Class USB is often used incorrectly for devices that do not have a predefined class. For example, a USB mouse uses class HID, whereas a USB smartcard uses class smartcard reader. Class USB is reserved for host controllers and root or external USB hubs. If the vendor has a device that has no Windows-defined class but uses USB as the bus, it must define its own class or class GUID. The setup

class associated with the type of USB device, not with the bus type, must be used. The setup class "USB" (ClassGuid = {36fc9e60-c465-11cf-8056-444553540000}) is reserved for USB host controllers and root or external USB hubs. It must not be used for other device categories.

Design Notes:

Microsoft provides system-defined setup classes for most device types. System-defined setup class GUIDs are defined in the Windows Driver Kit, "Devguid.h."

If you choose the wrong class, the device appears in an incorrect location in Device Manager and in the Windows Vista UI. Using this class incorrectly may cause the device driver to fail hardware compatibility testing.

For a list of Windows class GUIDs, see the Windows Driver Kit, "System-Supplied Device Setup Classes" at http://msdn.microsoft.com/en-us/library/ff553419(VS.85).aspx.

Exceptions: Not Specified

Business Justification:

To support USB connectivity functionality and scenarios. Better end user experience in UI and device manager. Incorrect co-installers aren't accidentally run on this device.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

CONNECT-0058

Device.Connectivity.UsbDevices.WirelessUsbObtainsWusbLogoFromUsbif

Target Feature: Device.Connectivity.UsbDevices

Title: Wireless USB device or host obtains Certified Wireless USB logo from the USB-IF

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

Description:

All Wireless USB devices must get a Certified Wireless USB Logo from the USB-IF.

Exceptions: Not Specified

Business Justification:

To support USB connectivity functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2009

Comments:

CONNECT-0105

Device.Connectivity.UsbDevices.WirelessUsbWiMediaAlliace

Target Feature: Device.Connectivity.UsbDevices

Title: Certified Wireless USB device or host passes all required WiMedia Alliance compliance tests.

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

Wireless USB device must pass WiMedia Alliance radio compliance tests.

Exceptions: Not Specified

Business Justification:

To support USB connectivity functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2009

Comments:

CONNECT-0081

Device.Connectivity.UsbHub

Description:

Requirements that apply only to USB Hubs

Related Requirements:

- Device.Connectivity.UsbHub.CompliesWithChap11
- Device.Connectivity.UsbHub.IdentifyNumOfUserAccessiblePorts
- Device.Connectivity.UsbHub.ImplementSuperSpeedDescriptors
- Device.Connectivity.UsbHub.MapPortsPerUsb3Specification
- Device.Connectivity.UsbHub.ProvideStandardInterfacesToHostPeripherals
- Device.Connectivity.UsbHub.SuperSpeedRemainsOnAfterPortReset
- Device.Connectivity.UsbHub.SupportSuspend
- Device.Connectivity.UsbHub.Usb3HubCompliesWithUsb3Spec
- Device.Connectivity.UsbHub.Usb3ReportPortStatusBitsCorrectly

Device.Connectivity.UsbHub.CompliesWithChap11

Target Feature: Device.Connectivity.UsbHub

Title: USB hub that implements USB functionality complies with the related specification

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

USB hubs that fit into one of the USB device class definitions must comply with the appropriate USB specification as follows:

USB 1.1, Revision1.1

USB 2.0, Revision2.0

Exceptions: Not Specified

Business Justification:

To support USB connectivity functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

Connect-0049

Device.Connectivity.UsbHub.IdentifyNumOfUserAccessiblePorts

Target Feature: Device.Connectivity.UsbHub

Title: USB hub correctly identifies and reports the number of ports that the user can access

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

The USB hub must include details in its hub descriptor that provide the operating system with an accurate count of the number of downstream-facing ports that the hub supports and that are exposed to the user. See USB Specification, Revision 2.0, Section11.23,and USB 3.0 Specification, Section 10.14. Root hubs are exempt from this requirement.

Exceptions:

Does not apply to Root Hubs

Business Justification:

To support USB connectivity functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

Connect-0046: Updated references to USB specifications

Device.Connectivity.UsbHub.ImplementSuperSpeedDescriptors

Target Feature: Device.Connectivity.UsbHub

Title: USB 3.0 hubs must properly implement the SuperSpeed hub descriptor, the standard SuperSpeed descriptors, the USB 2.0 ports, and USB 3.0 hubs report version 2.1.

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

As described in section 10.0 of the USB 3.0 Specification, a USB 3.0 hub incorporates a USB 2.0 hub and a SuperSpeed hub. All exposed downstream ports on a USB 3.0 hub must support both SuperSpeed and USB 2.0 connections.

For more information on USB 3.0 devices that operate at speeds other than SuperSpeed, see section 9.2.6.6 of the USB 3.0 Specification.

When a USB 3.0 hub is operating in SuperSpeed mode, the hub must correctly implement the descriptors that are described in Section 10.13 of the USB 3.0 Specification in addition to the standard USB descriptors.

As described in section 10.13.1 of the USB 3.0 Specification, the hub must support the following SuperSpeed descriptors in SuperSpeed mode:

- Device descriptor
- Binary Device Object Store (BOS) descriptor
- USB 2.0 Extension
- SuperSpeed USB Device Capability descriptor
- Container ID
- Configuration descriptor (SuperSpeed information)

- Interface descriptor
- Endpoint descriptor (for status change endpoint)
- Endpoint Companion descriptor (for status change endpoint)

The class-specific descriptors for SuperSpeed hubs that must be supported, as defined in section 10.13.2.1 in the USB 3.0 Specification, are as follows:

SuperSpeed Hub descriptor

Implementation Details

Standard USB SuperSpeed descriptors will be tested via the USB Command Verifier chapter 9 test for USB 3.0 devices. In the Driver Test Manager (DTM), this test is the USB Device Framework Test.

Table 1. SuperSpeed Hub Descriptor Asserts

Assert	Description
1	The **bDescLength** field of the SuperSpeed Hub descriptor has a value of 0xCh.
2	The **bDescriptorType** field of the SuperSpeed Hub descriptor has a value of 0x2Ah.
3a	Bits D1..D0 of the **wHubCharacteristics** field of the SuperSpeed Hub descriptor must be 00 if the hub supports ganged power switching.
3b	Bits D1..D0 of the **wHubCharacteristics** field of the SuperSpeed Hub descriptor must be 01 if the hub supports individual port power switching.
4a	Bit D2 of the **wHubCharacteristics** field of the SuperSpeed Hub descriptor must be 0 for a hub that is not part of a compound device.
4b	Bit D2 of the **wHubCharacteristics** field of the SuperSpeed Hub descriptor must be 1 for a hub that is part of a compound device.
5	Bits D4:D3 of the **wHubCharacteristics** field of the SuperSpeed Hub descriptor must not be 11 or 10 for a self-powered hub.
6	Bits D15D5 of the **wHubCharacteristics** field of the SuperSpeed Hub descriptor are reserved and must be 0.
7	The **bPwrOn2PwrGood** field of the SuperSpeed Hub descriptor must be 0 if the hub does not support power switching.
8	The **bHubHdrDecLat** field of the SuperSpeed Hub descriptor must be in the range of 0x00h to 0x04h.
9	The **DeviceRemovable** field of the SuperSpeed Hub descriptor reports the correct non-removable ports as non-removable.

10 The **bNbrPorts** field of the SuperSpeed Hub descriptor reports the correct number of ports on the hub.

Exceptions: Not Specified

Business Justification:

To support USB connectivity functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2011

Comments:

Connect-0132

Device.Connectivity.UsbHub.MapPortsPerUsb3Specification

Target Feature: Device.Connectivity.UsbHub

Title: USB 3.0 hubs must map their ports as defined in section 10.1 of the USB 3.0 Specification

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

External USB 3.0 hubs must follow the USB3 specifications connector mapping, except when the USB 3.0 hub may have an unequal number of USB 2.0 and USB 3.0 protocol ports.

For example, the SuperSpeed part of the 3.0 Hub has *n* ports and 2.0 part of the 3.0 Hub has *m* ports and minimum(*m,n*) = *p*. The first *p* ports of the SuperSpeed part and the 2.0 part should be mapped in order, so that port 1 of SuperSpeed hub maps to port 1 of the 2.0 hub, port 2 of the SuperSpeed hub maps to port 2 of the 2.0 hub and so on until port number *p*. Each of these p port mappings must either have an external connector for it **OR** both of the ports in the mapping should be marked as non-removable in their respective parent hub descriptors. Note, that if the ports in a mapping are marked as non-removable, only one of the ports in that mapping can have a device attached to it.

The other port in that mapping should neither have any device attached to it nor a device can be attached to it i.e. the port should remain unused.

If there are excess ports on the 2.0 part (i.e. *m-p* is not 0), each of those *m-p* ports may either be exposed to the user **OR** can be marked as non-removable. If there are excess ports on the SuperSpeed part (i.e. *n-p* is not 0), none of those *n-p* ports should be exposed to the user, all of them should be marked as non-removable.

Exceptions: Not Specified

Business Justification:

To support USB connectivity functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2011

Comments:

Connect-0133

Device.Connectivity.UsbHub.ProvideStandardInterfacesToHostPeripherals

Target Feature: Device.Connectivity.UsbHub

Title: USB hub provides standard connection interfaces to the host and USB peripherals

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

All USB hubs must have one upstream-facing port and at least one downstream-facing port. The upstream-facing port must be a single Std-A receptacle for downstream traffic. The downstream-facing port or ports must be one of the following:

Std-B receptacle

Mini-B receptacle

Captive Std-A cable, for upstream traffic

Micro - B receptacle

This is described in USB Specification, Revision1.1 or later, Sections 6.2 and 11.1.2. and Micro-USB cables and connectors Revision 1.01

Exceptions: Not Specified

Business Justification:

To support USB connectivity functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

Connect-0060

Device.Connectivity.UsbHub.SuperSpeedRemainsOnAfterPortReset

Target Feature: Device.Connectivity.UsbHub

Title: USB 3.0 hubs must not turn off SuperSpeed termination on downstream ports upon over-current and port reset

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

DSPORT.Powered-off must be entered only when Rx termination is not detected or downstream Vbus is off. ClearPortFeature(PORT_POWER), Overcurrent, Upstream port reset, and SetConfig(0)must not cause SuperSpeed termination to be removed unless Vbus is also removed. If Vbus is removed, SuperSpeed termination must be re-enabled when Vbus is back on. If Upstream Vbus is removed, but the hub still provides Downstream Vbus (self-powered hub), then itmust turn downstream SuperSpeed terminations back on.

For more information, see figure 10-9 in the USB 3.0 Specification.

Exceptions: Not Specified

Business Justification:

To support USB connectivity functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2011

Comments:

Connect-0136

Device.Connectivity.UsbHub.SupportSuspend

Target Feature: Device.Connectivity.UsbHub

Title: USB hubs must support suspend, and downstream devices must not drop off the bus when the hub resumes from selective suspend

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

USB hubs must support the selective suspend state, as stated in both the USB Specification and other logo program requirements. After a hub is resumed from the selective suspend state, all devices that were attached downstream of the hub, and that were not removed while the hub was suspended, must be present.

Exceptions: Not Specified

Business Justification:

To support USB connectivity functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2011

Comments:

Connect-0137

Device.Connectivity.UsbHub.Usb3HubCompliesWithUsb3Spec

Target Feature: Device.Connectivity.UsbHub

Title: USB 3.0 Hubs are compliant with USB 3.0 specification.

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

USB 3.0 Hubs must be compliant with Universal Serial Bus (USB) 3.0 Specification

USB 3.0 Hubs must :

- Pass the USB-IF interoperability tests
- Pass the USB 3.0 Hub compliance test suite
- Pass the USB 3.0 CV test

Exceptions: Not Specified

Business Justification:

To support USB connectivity functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2011

Comments:

Connect-8003

Device.Connectivity.UsbHub.Usb3ReportPortStatusBitsCorrectly

Target Feature: Device.Connectivity.UsbHub

Title: USB 3.0 hubs must always report the port status bits correctly as per the USB 3.0 Specification

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

In the current stack, a number of invalid port status bit combinations that the hub reports are ignored. Any invalid combination of port status bits will be treated as an error. In particular, checks will follow these actions:

- Resetting a port
- Suspending and resuming a port
- System resume

A hub should not report spurious change interrupts. A hub should complete the port status interrupt transfer without reporting changes.

Exceptions: Not Specified

Business Justification:

To support USB connectivity functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2011

Comments:

Connect-0134

Device.Connectivity.WSD

Description: Not Specified

Related Requirements:

- Device.Connectivity.WSD.DPWS
- Device.Connectivity.WSD.DPWSExtensibility
- Device.Connectivity.WSD.MetadataExchange
- Device.Connectivity.WSD.MetadataValid
- Device.Connectivity.WSD.Schema
- Device.Connectivity.WSD.WSDiscovery

Device.Connectivity.WSD.DPWS

Target Feature: Device.Connectivity.WSD

Title: Devices which use or interact with the Web Services on Devices API (WSDAPI) comply with Device Profiles for Web Services (DPWS) specification

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86
- Windows Server 2008 x64
- Windows Server 2008 x86
- Windows Vista Client x64
- Windows Vista Client x86

Description:

Devices which plan to use or interact with Microsoft Windows' implementation of DPWS, the Web Services on Devices API (WSDAPI), must implement the DPWS specification themselves. (WSDAPI)

Design Notes:

DPWS Specification available at

http://go.microsoft.com/fwlink/?LinkId=109231

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

CONNECT-0114

Device.Connectivity.WSD.DPWSExtensibility

Target Feature: Device.Connectivity.WSD

Title: Devices Profile for Web Services Devices must accept messages that contain extensibility sections, and process the messages as appropriate.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86
- Windows Server 2008 x64
- Windows Server 2008 x86
- Windows Vista Client x64
- Windows Vista Client x86

Description:

Devices Profile for Web Services (DPWS) devices must accept messages where the XML has been extended. If the device understands the content in the extensible section, it may process it.

Design Notes:

DPWS Specification available at

http://go.microsoft.com/fwlink/?LinkId=109231

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

CONNECT-0119

Device.Connectivity.WSD.MetadataExchange

Target Feature: Device.Connectivity.WSD

Title: Devices Profile for Web Services (DPWS) Devices support metadata exchange

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86
- Windows Server 2008 x64
- Windows Server 2008 x86
- Windows Vista Client x64
- Windows Vista Client x86

Description:

DPWS Devices which interact with the Web Services on Devices API (WSDAPI) must support metadata exchange as defined in the metadata exchange specification.

Design Notes:

Metadata Exchange specification can be obtained at http://go.microsoft.com/fwlink/?LinkId=109248

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

CONNECT-0117

Device.Connectivity.WSD.MetadataValid

Target Feature: Device.Connectivity.WSD

Title: Devices which interact with the Web Services on Devices (WSDAPI) produce metadata that conforms to the Devices Profile for Web Services

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64

- Windows 7 Client x64
- Windows 7 Client x86
- Windows Server 2008 x64
- Windows Server 2008 x86
- Windows Vista Client x64
- Windows Vista Client x86

Description:

Devices which interact with WSDAPI must populate the Metadata as defined in the Device Profile for Web Services Specification of February 2006.

Design Notes:

The Device Profile for Web Services Specification of February 2006 is available at http://go.microsoft.com/fwlink/?LinkId=109231

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

CONNECT-0118

Device.Connectivity.WSD.Schema

Target Feature: Device.Connectivity.WSD

Title: A network-enabled device that implements Devices Profile for Web Services (DPWS) must adhere to the protocol and schema.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86
- Windows Server 2008 x64
- Windows Server 2008 x86
- Windows Vista Client x64
- Windows Vista Client x86

Description:

A network-enabled device that implements Devices Profile for Web Services (DPWS) must adhere to the Devices Profile for Web Services as described by the schema.

The device must also reference the namespace URI as described in The Devices Profile for Web Service specification.

A device the implements DPWS must adhere to the Web Services Description Language (WSDL) associated with the logo device class. The WSDL defines services as collections of network endpoints, or ports. WSDL specification provides an XML format for documents for this purpose. Devices must implement the WSDL version 1.1.

Design Notes:

See the Web Services Description Language (WSDL) Version 1.1 at http://www.w3.org/TR/2001/NOTE-wsdl-20010315

See the Devices Profile for Web Services schema at http://schemas.xmlsoap.org/ws/2006/02/devprof/devicesprofile.xsd.

See the Devices Profile for Web Service specification at http://specs.xmlsoap.org/ws/2006/02/devprof/devicesprofile.pdf.

Additional information can be found in the Windows Rally Development Kit at http://go.microsoft.com/fwlink/?LinkId=109368.

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

CONNECT-0101

Device.Connectivity.WSD.WSDiscovery

Target Feature: Device.Connectivity.WSD

Title: Devices Profile for Web Services (DPWS) Devices interacting with the Web Services on Devices API (WSDAPI) implement WS-Discovery

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64

- Windows 8 Server x64
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86
- Windows Server 2008 x64
- Windows Server 2008 x86
- Windows Vista Client x64
- Windows Vista Client x86

Description:

DPWS Devices must implement WS-Discovery to work with WSDAPI.

Design Notes:

WS-Discovery specification can be obtained at http://go.microsoft.com/fwlink/?LinkId=109247

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

CONNECT-0116

Device.DevFund.CDA

Description:

Custom Driver Access for privileged application usage.

Related Requirements:

- Device.DevFund.CDA.Application

Device.DevFund.CDA.Application

Target Feature: Device.DevFund.CDA

Title: Custom Driver Access

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64

- Windows 8 Server x64
- Windows 8 Client ARM

Description:

If a device driver supports a privileged app performing Custom Driver Access, it must declare a restricted interface.

By declaring a restricted interface, the following requirements must be met:

1. Assert that the device io control interfaces provided by this device driver are intended to be accessed by a privileged app running in app container accessing hardware functionality using CreateDeviceAccessInstance() and IDeviceIoControl() on Windows 8.

2. The restricted interface cannot be opened directly from an app container.

A device driver declares an interface is restricted by setting the DEVPKEY_DeviceInterface_Restricted property to true on that interface.

Exceptions: Not Specified

Business Justification:

To support new model of Custom Driver Access.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.DevFund.Color

Description: Not Specified

Related Requirements:

- Device.DevFund.Color.DeviceColorProfilesInstall

Device.DevFund.Color.DeviceColorProfilesInstall

Target Feature: Device.DevFund.Color

Title: Device that uses an ICC or WCS XML profile properly installs the profile and is compliant with the appropriate profile specification

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64

- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

Devices that use a vendor-supplied International Color Consortium (ICC) or Windows Color System (WCS) Extensible Markup Language (XML) profile or profiles must associate the profile or profiles with the device.

Devices that create sRGB output can associate the sRGB Color Space Profile.icm profile (that is, the Windows default ICC profile) or the sRGB.cdmp profile (that is, the Windows default WCS XML profile) with the device. Devices that are sRGB-compliant are not required to provide an ICC or WCS XML profile.

Note: For display monitors that are integrated into a system and are not required to support Plug and Play for their installed LCD display, the ICC or WCS XML profile must be installed manually by using an appropriate monitor information (INF) file. OEMs should install the correct configuration as part of the operating system pre-installation process. If necessary, the INF file will be available to the user for manual reinstallation. Mobile computers that have dual-scan supertwist nematic (DSTN) or reflective LCD displays do not require ICC or WCS XML profiles.

Design Notes:

By default, Windows supports devices that create sRGB output. Devices that use an output other than sRGB can use an INF file to install an ICC or WCS XML profile that is appropriate to the preferred display resolution, as identified in the extended display identification data (EDID), at 32 bits per pixel (bpp). For an LCD or other non-CRT display device, the profile should be based on the native display mode, or resolution and color depth, for which the display is designed.

The ICM APIs and functionality are defined in the Microsoft Platform SDK and the Windows Driver Kit.

WCS XML profiles are specified together with examples in the Microsoft Platform SDK and the Windows Driver Kit.

This requirement will be tested by the "WCSPlugInRobustness" test in the Windows Hardware Certification Kit (WHCK).

Exceptions: Not Specified

Business Justification:

This requirement ensures a consistent color and resolution experience across the devices a system may interact with.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2006

Comments:

DEVFUND-0018

Device.DevFund.DriverFramework.AllDrivers

Description:

Driver framework requirements applicable to all drivers

Related Requirements:

- Device.DevFund.DriverFramework.AllDrivers.WDFLoadGroup

Device.DevFund.DriverFramework.AllDrivers.WDFLoadGroup

Target Feature: Device.DevFund.DriverFramework.AllDrivers

Title: Only Windows Driver Framework (WDF) can use the WdfLoadGroup driver load order group

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64

Description:

Only WDF (Wdf01000.sys) must use the WdfLoadGroup driver load order group.

User-Mode Driver Framework (UMDF) and Kernel-Mode Driver Framework (KMDF) drivers must not use this driver load order group, because that prevents installation of WDF boot start drivers.

Exceptions: Not Specified

Business Justification:

If WdfLoadGroup load order group value is used by drivers other than the Framework, it prevents WDF boot start drivers from loading.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 12/1/2010

Comments:

DEVFUND-0048

Device.DevFund.DriverFramework.KMDF

Description:

Driver framework requirements for KMDF

Related Requirements:

- Device.DevFund.DriverFramework.KMDF.HandleDDIFailures
- Device.DevFund.DriverFramework.KMDF.Reliability
- Device.DevFund.DriverFramework.KMDF.WDFProperINF
- Device.DevFund.DriverFramework.KMDF.WDFRedistributables

Device.DevFund.DriverFramework.KMDF.HandleDDIFailures

Target Feature: Device.DevFund.DriverFramework.KMDF

Title: Kernel Mode Driver Framework (KMDF) drivers are designed to handle DDI failures gracefully

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64

Description:

Kernel Mode Driver Framework (KMDF) drivers must handle failure gracefully. When a device driver interface (DDI) returns an NTSTATUS code, the driver must check the return value before the driver proceeds.

If a failure occurs, DDI out parameters must not be used. Use of these parameters will cause drivers to crash or stop responding. Use of these parameters will also lead to bug checks and other unreliable behavior.

Design Notes:

The WDFTester tool from the Windows Driver Kit (WDK) is used for fault injection on the DDIs during Windows Hardware Certification testing. This tool resides in the following location in the WDK:

%wdk%\WDKVersionNumber\tools\wdf\wdftester

For more information about the WDFTester tool, visit the following website:

http://msdn.microsoft.com/en-us/library/ff556110.aspx

The Device Disable Enable test, the Sleep test, and the Plug and Play test from WDK will be used according to the WDFTester tool. The DDIs that are called from the driver during these tests will be fault injected to return unsuccessful status codes. When a DDI returns an unsuccessful status code, out parameters of that DDI may also be assigned to NULL values. If a DDI failure occurs, out parameters should be used according to the documented behavior.

Exceptions: Not Specified

Business Justification:

This ensures that KMDF drivers recover from failure gracefully to improve end user experience and system reliability.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2009

Comments:

DEVFUND-0038

Device.DevFund.DriverFramework.KMDF.Reliability

Target Feature: Device.DevFund.DriverFramework.KMDF

Title: Kernel Mode Driver Framework (KMDF) drivers are architected to maximize reliability and stability and do not "leak" resources such as memory and KMDF objects

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x64
- Windows 8 Client x86
- Windows 8 Server x64

Description:

Kernel-Mode Driver Framework (KMDF) drivers must use pool memory responsibly. Handles that the drivers pass to device driver interfaces (DDIs) must conform to the pattern of the parameter. The state of the drivers must be consistent with the use of **WDFREQUEST** objects and **WDFQUEUE** objects.

Event callback functions that the driver registers must adhere to interrupt request level (IRQL) restrictions.

Design Notes:

For more information about developing drivers that meet this requirement, visit the following websites:

http://msdn.microsoft.com/en-us/library/aa973499.aspx

http://www.microsoft.com/whdc/driver/wdf/KMDF.mspx

The following tools can be enabled to validate this requirement for all KMDF drivers:

- Windows Driver Foundation (WDF) Verifier.
- Handle tracking. Handle tracking will be enabled on all KMDF objects.
- Enhanced Verifier for Framework 1.9 KMDF drivers. Enhanced Verifier is new for Framework 1.9. This tool can be enabled by using the EnhancedVerifierOptions registry value. To enable Enhanced Verifier, set the following registry values for the drivers Parameters\Wdf key:

HKLM\System\CurrentControlSet\Services\\Parameters\Wdf

EnhancedVerifierOptions REG_DWORD 1

VerifierOn REG_DWORD 1

TrackHandles MULTI_SZ *

- Driver Verifier. To enable Driver Verifier, use the following command:

    Verifier /flags 0xfb /driver

This command will run the KMDF driver under Driver Verifier with all flags set except the Low Resource Simulation flag. For more information about Driver Verifier, visit the following website:

http://msdn.microsoft.com/en-us/library/ff545448.aspx

In the Windows Windows Hardware Certification Kit, the WDF Test can be run to validate this requirement.

Exceptions: Not Specified

Business Justification:

These requirements and testing will help ensure high-quality KMDF drivers because the drivers must conform to the Framework requirements.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2009

Comments:

DEVFUND-0037

Device.DevFund.DriverFramework.KMDF.WDFProperINF

Target Feature: Device.DevFund.DriverFramework.KMDF

Title: Windows Driver Framework (WDF) driver INF files are properly structured

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64

Description:

All information (INF) files in Windows Driver Foundation (WDF) driver packages must call WDF-specific sections and directives properly. Correctly structured INF sections help ensure that the driver will be installed properly. However, even when a driver is installed, a poorly or wrongly configured section can cause unpredictable problems during the operation of the driver or device. These problems can be prevented by following the guidelines for WDF INF settings.

To meet this requirement, all WDF INF files must have the following:

- INF Coinstaller section, as follows:

[DDInstall.Coinstallers]

CopyFiles=

AddReg=

If the MSU update package for WDF version 1.11 is used to install the Framework then WDF v1.11 drivers don't use the WDF coinstaller. If the MSU update package is used then KMDF drivers don't reference KMDF update coinstaller wdfcoinstaller0100X.dll, and UMDF drivers dont reference the

UMDF update coinstaller wudfupdate_0100X.dll. But UMDF drivers still need to reference the config coinstaller wudfcoinstaller.dll, which is an inbox coinstaller and isn't part of the driver package. UMDF drivers must reference the inbox coinstaller as in the below example;

```
[Echo_Install.NT.CoInstallers]
AddReg=CoInstallers_AddReg

[CoInstaller.AddReg]

HKR,CoInstallers32,0x00010000,WudfCoinstaller.dll
```

- INF WDF Install section must be referenced as follows:

For Kernel-Mode Driver Framework (KMDF) drivers:

```
[DDInstall.Wdf]

KmdfService= <ServiceName>, <Kmdf_Install>

[Kmdf_Install]

KmdfLibraryVersion=
```

KMDF v1.11 drivers don't use the WDF install section if the MSU package is used.

For User-Mode Driver Framework (UMDF) drivers:

```
[DDInstall.Wdf]

UmdfService=<ServiceName>,<Umdf_Install>

UmdfServiceOrder=

UmdfDispatcher [Only for USB Drivers and Drivers with file handle I/O targets]=

UmdfImpersonationLevel[optional]=

[Umdf_Install]

UmdfLibraryVersion=

DriverCLSID=

ServiceBinary=
```

UMDF v1.11 drivers must still use the WDF install section if the MSU package is used; this section is needed by the inbox UMDF coinstaller.

- All UMDF driver INF files must have a WUDFRD service installation section for OSes downlevel to Windows 8, as follows:

```
[WUDFRD_ServiceInstall]
```

DisplayName = "Windows Driver Foundation - User-mode Driver Framework Reflector"

ServiceType = 1

StartType = 3

ErrorControl = 1

ServiceBinary = %12%\WUDFRd.sys

LoadOrderGroup = Base

For Win8 UMDF drivers can use a unique service instead of WudfRd. In order to accomplish this drivers need to abide by the following rules;

1. Change the AddService directive to a unique value.
2. Change the services DisplayName to a unique value.
3. Add StartName=\Driver\WudfRd directive to the service entry.
4. Add ServiceBinary= %12%\WUDFRd.sys directive to the service entry.

Below is an example UMDF driver using the unique service;

[Echo_Install.NT.Services]

AddService=WudfEchoDriver,0x00000002 WUDFEchoDriver_ServiceInstall,

[WUDFEchoDriver_ServiceInstall]

DisplayName = %WudfEchoDriverDisplayName%

ServiceType = 1

StartType = 3

ErrorControl = 1

ServiceBinary = %12%\WUDFRd.sys

LoadOrderGroup = Base

StartName = \Driver\WudfRd

UMDF drivers can’t use a unique service for OSes downlevel to Win8Windows 8. If a UMDF driver wants to use a unique service for Win8 but still needs to work for downlevel OSes then this driver should use OS specific sections in their INF. The below example demonstrates an INF using OS specific install sections;

[Manufacturer]

```
%MSFT% = Microsoft,NTx86.6.0,NTx86.6.2

[Microsoft.NTx86.6.0]

%Sensors.DeviceDesc% = Sensors_Install_VistaWin7,HID_DEVICE_UP:0020_U:0001

[Microsoft.NTx86.6.2]

%Sensors.DeviceDesc% = Sensors_Install_Win8,HID_DEVICE_UP:0020_U:0001

[Sensors_Install_VistaWin7]

--- Install the device this way on Vista/Win7 ---

[Sensors_Install_Win8]

--- Install the device in a different way on Win8 ---
```

- All WDF drivers that use a WinUSB driver must have the following service installation settings:

```
[WinUsb_ServiceInstall]

DisplayName = "WinUSB Driver"

ServiceType = 1

StartType = 3

ErrorControl = 1

ServiceBinary = %12%\WinUSB.sys
```

- Service names, hardware IDs (HWIDs), display names, and UMDF class identifiers (CLSIDs) cannot be copy-pasted from WDF samples.

Design Notes:

For more information about WDF-specific INF settings, visit the following websites:

http://www.microsoft.com/whdc/driver/wdf/wdfbook.mspx

http://msdn.microsoft.com/en-us/library/ff560526.aspx

http://msdn.microsoft.com/en-us/library/ff560526.aspx

Exceptions: Not Specified

Business Justification:

This requirement helps ensure good quality of driver setup files for WDF drivers and improves the compatibility and reliability of certified drivers.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2009

Comments:

DEVFUND-0040

Device.DevFund.DriverFramework.KMDF.WDFRedistributables

Target Feature: Device.DevFund.DriverFramework.KMDF

Title: Windows Driver Framework (WDF) drivers are packaged to contain the RTM free versions of redistributables

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64

Description:

All Windows Driver Framework (WDF) drivers that are submitted for certification through the Windows Hardware Certification Program for Hardware must meet the following guidelines for packaging redistributables, or co-installers:

- All WDF drivers, for all versions of the WDF, for all operating systems can be submitted for certification
- Redistributables that are included in the driver package must be the release to manufacturing (RTM) fre version of the redistributables. Some WDF version 1.7 coinstallers that were available on the Microsoft Connect website caused serious installation problems. Microsoft has removed these coinstallers from Connect and replaced them with fixed coinstallers. The problematic WDF coinstallers were version 1.7.6001.0. The fixed coinstallers are version 1.7.6001.18000. For logo certification submissions, drivers should use the version 1.7.6001.18000 co-installers.

Kernel-Mode Driver Framework (KMDF) has coinstallers for versions 1.0, 1.1, 1.5, 1.7, 1.9 and 1.11. You can use all of these coinstallers for certification submissions, provided that you use the RTM fre versions.

User-Mode Driver Framework (UMDF) has coinstallers for versions 1.5, 1.7, 1.9 and 1.11. You can use all of these coinstallers in your driver package, provided that you use the RTM fre versions.

WDF version 1.11 coinstallers that are released via Microsoft Connect have the most enhanced version of the framework. We advise partners to use the latest version of the framework to develop and distribute WDF drivers.

Design Notes:

WDF version 1.11 drivers can install the frameworks using an MSU update package instead of using the 1.11 coinstallers. If WDF version 1.11 drivers use the MSU install package then KMDF drivers don't use a co-installer, but WDF version 1.11 UMDF drivers still reference the inbox co-installer named WudfCoinstaller.dll. WudfCoinstaller.dll is inbox to Windows 8, and UMDF drivers aren't packaged with it, only the INF file should reference this co-installer.

The WDF 1.11 coinstallers and the MSU package are distributed via Microsoft Connect.

Partners who want to have driver packages signed for down-level operating systems prior to WDF 1.11 RTM should use the WDF 1.9 RTM co-installers.

For more information about WDF driver installation, visit the following website:

http://msdn.microsoft.com/en-us/library/ff544213.aspx

Exceptions: Not Specified

Business Justification:

WDF drivers must be packaged with the correct versions of redistributables in order to function on user's systems.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2009

Comments:

DEVFUND-0039

Device.DevFund.DriverFramework.UMDF

Description:

Driver framework requirements for UMDF

Related Requirements:

- Device.DevFund.DriverFramework.UMDF.Reliability
- Device.DevFund.DriverFramework.UMDF.WDFProperINF
- Device.DevFund.DriverFramework.UMDF.WDFRedistributables

Device.DevFund.DriverFramework.UMDF.Reliability

Target Feature: Device.DevFund.DriverFramework.UMDF

Title: User Mode Driver Framework (UMDF) drivers must be secure, stable, reliable and not have application compatibility issues.

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64

Description:

To help ensure that all User-Mode Driver Framework (UMDF) drivers meet security standards, are stable and reliable, and do not have application compatibility issues, drivers must not have any object leaks.

Object leaks can be diagnosed by enabling object tracking. If a memory leak occurs, set the Reference Count Tracking setting to On. This setting logs the history for add of reference and release of reference counts.

These features can be set to On by using the following registry values:

HKLM\Software\Microsoft\WindowsNT\CurrentVersion\WUDF\Services\{193a1820-d9ac-4997-8c55-be817523f6aa}

TrackObjects REG_DWORD 1

TrackRefCounts REG_DWORD 1

UMDF drivers must also meet the following requirements:

- The drivers must not attempt to use invalid handles.
- The drivers must use critical sections and file locks properly. The primary purpose of the locks test is to help ensure that the application properly uses critical sections.
- The drivers must not cause heap memory corruption.
- The drivers must correctly use virtual address space manipulation functions, such as **VirtualAlloc**, **VirtualFree**, and **MapViewOfFile**.
- The drivers must not hide access violations by using structured exception handling.

- The drivers must correctly use thread local storage functions.
- The drivers must use COM correctly.

Partners can verify that the drivers meet these requirement by enabling Microsoft Application Verifier's handles, locks, heaps, memory, exceptions, and Transport Layer Security (TLS) settings for the UMDF host process (that is, WUDFHost.exe) during driver development and testing.

For more information, see the *Developing Drivers with the Windows Driver Foundation* book at the following website:

http://www.microsoft.com/whdc/driver/wdf/wdfbook.mspx

Design Notes:

Application Verifier will help check UMDF drivers extensively to help ensure stable and reliable drivers.

For all UMDF drivers, Application Verifier will be enabled when driver reliability tests are executed.

Application Verifier can be downloaded from the following Microsoft website:

http://www.microsoft.com/downloads/en/details.aspx?FamilyID=c4a25ab9-649d-4a1b-b4a7-c9d8b095df18

To enable Application Verifier for WUDFHost.exe, run the following command:

appverif -enable handles locks heaps memory COM exceptions TLS -for WUDFHost.exe

For more information about UMDF, visit the following website:

http://www.microsoft.com/whdc/driver/wdf/UMDF.mspx

Exceptions: Not Specified

Business Justification:

This requirement and the associated testing will enable better reliability checks on UMDF drivers.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2009

Comments:

DEVFUND-0036

Device.DevFund.DriverFramework.UMDF.WDFProperINF

Target Feature: Device.DevFund.DriverFramework.UMDF

Title: Windows Driver Framework (WDF) driver INF files are properly structured

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64

Description:

All information (INF) files in Windows Driver Foundation (WDF) driver packages must call WDF-specific sections and directives properly. Correctly structured INF sections help ensure that the driver will be installed properly. However, even when a driver is installed, a poorly or wrongly configured section can cause unpredictable problems during the operation of the driver or device. These problems can be prevented by following the guidelines for WDF INF settings.

To meet this requirement, all WDF INF files must have the following:

- INF Coinstaller section, as follows:

[DDInstall.Coinstallers]

CopyFiles=

AddReg=

If the MSU update package for WDF version 1.11 is used to install the Framework then WDF v1.11 drivers don't use the WDF coinstaller. If the MSU update package is used then KMDF drivers don't reference KMDF update coinstaller wdfcoinstaller0100X.dll, and UMDF drivers don't reference the UMDF update coinstaller wudfupdate_0100X.dll. But UMDF drivers still need to reference the config coinstaller wudfcoinstaller.dll, which is an inbox coinstaller and isn't part of the driver package.

UMDF drivers must reference the inbox coinstaller as in the below example;

[Echo_Install.NT.CoInstallers]
AddReg=CoInstallers_AddReg

[CoInstaller.AddReg]

HKR,CoInstallers32,0x00010000,WudfCoinstaller.dll

- INF WDF Install section must be referenced as follows:

For Kernel-Mode Driver Framework (KMDF) drivers:

[DDInstall.Wdf]

KmdfService= <ServiceName>, <Kmdf_Install>

[Kmdf_Install]

KmdfLibraryVersion=

KMDF v1.11 drivers don’t use the WDF install section if the MSU package is used.

For User-Mode Driver Framework (UMDF) drivers:

[DDInstall.Wdf]

UmdfService=<ServiceName>,<Umdf_Install>

UmdfServiceOrder=

UmdfDispatcher [Only for USB Drivers and Drivers with file handle I/O targets]=

UmdfImpersonationLevel[optional]=

[Umdf_Install]

UmdfLibraryVersion=

DriverCLSID=

ServiceBinary=

UMDF v1.11 drivers must still use the WDF install section if the MSU package is used; this section is needed by the inbox UMDF coinstaller.

- All UMDF driver INF files must have a WUDFRD service installation section for OSes downlevel to Windows 8, as follows:

[WUDFRD_ServiceInstall]

DisplayName = "Windows Driver Foundation - User-mode Driver Framework

Reflector"

ServiceType = 1

StartType = 3

ErrorControl = 1

ServiceBinary = %12%\WUDFRd.sys

LoadOrderGroup = Base

For Win8 Windows 8 UMDF drivers can use a unique service instead of WudfRd. In order to

accomplish this drivers need to abide by the following rules;

1. Change the AddService directive to a unique value.
2. Change the services DisplayName to a unique value.
3. Add StartName=\Driver\WudfRd directive to the service entry.
4. Add ServiceBinary= %12%\WUDFRd.sys directive to the service entry.

Below is an example UMDF driver using the unique service;

[Echo_Install.NT.Services]

AddService=WudfEchoDriver,0x00000002 WUDFEchoDriver_ServiceInstall,

[WUDFEchoDriver_ServiceInstall]

DisplayName = %WudfEchoDriverDisplayName%

ServiceType = 1

StartType = 3

ErrorControl = 1

ServiceBinary = %12%\WUDFRd.sys

LoadOrderGroup = Base

StartName = \Driver\WudfRd

UMDF drivers cant cannot use a unique service for OSes downleveldown-level to Win8. If a UMDF driver wants to use a unique service for Win8 but still needs to work for downleveldown-level OSes then this driver should use OS specific sections in their INF. The below example demonstrates an INF using OS specific install sections;

[Manufacturer]

%MSFT% = Microsoft,NTx86.6.0,NTx86.6.2

[Microsoft.NTx86.6.0]

%Sensors.DeviceDesc% = Sensors_Install_VistaWin7,HID_DEVICE_UP:0020_U:0001

[Microsoft.NTx86.6.2]

%Sensors.DeviceDesc% = Sensors_Install_Win8,HID_DEVICE_UP:0020_U:0001

[Sensors_Install_VistaWin7]

--- Install the device this way on Vista/Win7 ---

[Sensors_Install_Win8]

--- Install the device in a different way on Win8 ---

- All WDF drivers that use a WinUSB driver must have the following service installation settings:

[WinUsb_ServiceInstall]

DisplayName = "WinUSB Driver"

ServiceType = 1

StartType = 3

ErrorControl = 1

ServiceBinary = %12%\WinUSB.sys

- Service names, hardware IDs (HWIDs), display names, and UMDF class identifiers (CLSIDs) cannot be copy-pasted from WDF samples.

Design Notes:

For more information about WDF-specific INF settings, visit the following websites:

http://www.microsoft.com/whdc/driver/wdf/wdfbook.mspx

http://msdn.microsoft.com/en-us/library/ff560526.aspx

http://msdn.microsoft.com/en-us/library/ff560526.aspx

Exceptions: Not Specified

Business Justification:

This requirement helps ensure good quality of driver setup files for WDF drivers and improves the compatibility and reliability of certified drivers.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2009

Comments:

DEVFUND-0040

Device.DevFund.DriverFramework.UMDF.WDFRedistributables

Target Feature: Device.DevFund.DriverFramework.UMDF

Title: Windows Driver Framework (WDF) drivers are packaged to contain the RTM free versions of redistributables

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64

Description:

All Windows Driver Framework (WDF) drivers that are submitted for a logo or signaturecertification through the Windows Logo Program for Hardware Certification Program must meet the following guidelines for packaging redistributables, or co-installers:

- All WDF drivers, for all versions of the WDF, for all operating systems can be submitted for a logo or signaturecertification.

- Redistributables that are included in the driver package must be the release to manufacturing (RTM) fre version of the redistributables. Some WDF version 1.7 coinstallers that were available on the Microsoft Connect website caused serious installation problems. Microsoft has removed these coinstallers from Connect and replaced them with fixed coinstallers. The problematic WDF coinstallers were version 1.7.6001.0. The fixed coinstallers are version 1.7.6001.18000. For logo certification submissions, drivers should use the version 1.7.6001.18000 co-installers.

Kernel-Mode Driver Framework (KMDF) has coinstallers for versions 1.0, 1.1, 1.5, 1.7, 1.9 and 1.11. You can use all of these coinstallers for logo certification submissions, provided that you use the RTM fre versions.

User-Mode Driver Framework (UMDF) has coinstallers for versions 1.5, 1.7, 1.9 and 1.11. You can use all of these coinstallers in your driver package, provided that you use the RTM fre versions.

WDF version 1.11 coinstallers that are released via Microsoft Connect have the most enhanced version of the framework. We advise partners to use the latest version of the framework to develop and distribute WDF drivers.

Design Notes:

WDF version 1.11 drivers can install the frameworks using an MSU update package instead of using the 1.11 coinstallers. If WDF version 1.11 drivers use the MSU install package then KMDF drivers don't use a co-installer, but WDF version 1.11 UMDF drivers still reference the inbox co-installer

named WudfCoinstaller.dll. WudfCoinstaller.dll is inbox to Windows 8, and UMDF drivers aren't packaged with it, only the INF file should reference this co-installer.

The WDF 1.11 coinstallers and the MSU package are distributed via Microsoft Connect.

Partners who want to have driver packages signed for down-level operating systems prior to WDF 1.11 RTM should use the WDF 1.9 RTM co-installers.

For more information about WDF driver installation, visit the following website:

http://msdn.microsoft.com/en-us/library/ff544213.aspx

Exceptions: Not Specified

Business Justification:

WDF drivers must be packaged with the correct versions of redistributables in order to function on user's systems.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2009

Comments:

DEVFUND-0039

Device.DevFund.INF

Description:

INF restictions

Related Requirements:

- Device.DevFund.INF.AddReg
- Device.DevFund.INF.AddService
- Device.DevFund.INF.ClassInstall32
- Device.DevFund.INF.ComplexDeviceMatching
- Device.DevFund.INF.DDInstall.CoInstallers
- Device.DevFund.INF.DeviceConfigOnly
- Device.DevFund.INF.DeviceResourceConfig
- Device.DevFund.INF.FileCopyRestriction
- Device.DevFund.INF.FileOrRegistryModification
- Device.DevFund.INF.InstallManagement
- Device.DevFund.INF.LegacySyntax
- Device.DevFund.INF.TargetOSVersion

Device.DevFund.INF.AddReg

Target Feature: Device.DevFund.INF

Title: When using an AddReg directive, each AddReg entry must specify HKR as the registry root

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

HKR (meaning relative root) is the only registry root identifier that can be referenced in an AddReg section in an INF file. Other root identifiers, including HKCR, HKCU, HKLM and HKU, are restricted from use in an AddReg section. The AddReg directive is intended to be used for device installation and configuration purposes only.

Design Notes:

All registry keys declared in an AddReg section of an INF file must use the relative root identifier (HKR) as the registry root value.

Exceptions:

*This is a requirement for Windows 8 Client ARM, but is recommended for Windows 8 Client x86 and Windows 8 Client x64. It will be required in the future for those architectures.

Business Justification:

This requirement improves driver and device installation performance and reliability by allowing Windows to optimize related driver staging and device configuration operations on the system.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.DevFund.INF.AddService

Target Feature: Device.DevFund.INF

Title: INF files can only install driver-related services

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64

- Windows 8 Server x64
- Windows 8 Client ARM

Description:

An INF AddService directive can only reference services that are driver related. Services that are not driver related, such as a Microsoft Win32 service, cannot be referenced or installed using an INF file.

Design Notes:

An INF AddService directive service-install-section may only specify a ServiceType type-code of the following:

- SERVICE_DRIVER
- SERVICE_KERNEL_DRIVER
- SERVICE_FILE_SYSTEM_DRIVER

Exceptions:

*This is a requirement for Windows 8 Client ARM, but is recommended for Windows 8 Client x86 and Windows 8 Client x64. It will be required in the future for those architectures.

Business Justification:

This requirement improves driver and device installation performance and reliability by allowing Windows to optimize related driver staging and device configuration operations on the system.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.DevFund.INF.ClassInstall32

Target Feature: Device.DevFund.INF

Title: INF files must not define a custom class installer within a ClassInstall32 section

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

An INF file may not specify a custom class installer within a ClassInstall32 section. Therefore, a driver package cannot execute a custom class installer during device installation.

Design Notes:

Developers should use one of the existing inbox device setup classes for their device. If it is necessary to define a new device setup class, the new setup class cannot employ a class installer as part of the device installation process. The following example shows an INF ClassInstall32 section which defines a custom class installer and therefore fails this requirement.

[ClassInstall32.ntx86] ; Declare a ClassInstall32 section for the x86

; architecture

AddReg=SetupClassAddReg ; Reference to the ClassInstall32 AddReg section

; Place additional class specific directives here

[SetupClassAddReg] ; Declare a class specific AddReg section

; Device class specific AddReg entries appear here

; The next line defines the class installer that will be executed when

; installing devices of this device-class type. Defining a registry entry

; of this type is no longer supported and the driver package fails to meet

; this device fundamental requirement.

[HKR,,Installer32,,"class-installer.dll,class-entry-point"]

Exceptions:

*This is a requirement for Windows 8 Client ARM, but is recommended for Windows 8 Client x86 and Windows 8 Client x64. It will be required in the future for those architectures.

Business Justification:

This requirement improves driver and device installation performance and reliability by allowing Windows to optimize related driver staging and device configuration operations on the system.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.DevFund.INF.ComplexDeviceMatching

Target Feature: Device.DevFund.INF

Title: INF directives related to complex device matching logic are not supported

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

INF files will not support complex device matching logic. Specifically, the capability to specify a DeviceID for a device which should not be installed, when a matching HardwareID or CompatibleID exists in the DDInstall section, will not be supported.

Design Notes:

The following INF directive may not be referenced in an INF file:

1. ExcludeID

Exceptions:

*This is a requirement for Windows 8 Client ARM, but is recommended for Windows 8 Client x86 and Windows 8 Client x64. It will be required in the future for those architectures.

Business Justification:

This requirement improves driver and device installation performance and reliability by allowing Windows to optimize related driver staging and device configuration operations on the system.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.DevFund.INF.DDInstall.CoInstallers

Target Feature: Device.DevFund.INF

Title: INF files must not reference any co-installers within a DDInstall.CoInstallers section

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

An INF file may not reference any co-installers within a DDInstall.CoInstallers section. Therefore, a driver package cannot execute any co-installers during device installation.

Design Notes:

Execution of co-installers is prohibited during device installation. The following examples show the registration of a device-specific co-installer and a device-class co-installer. Both types of co-installers are not permitted in an INF file and inclusion will result in failure to meet the requirement.

Device-specific co-installer example:

;Registeringoneormoredevice-specificco-installersrequiresadding

;addingaREG_MULTI_SZvalueusinganAddRegdirective.Thefollowing

;showsthegeneralformforregisteringadevice-specificco-installer.

;:

;:

[DestinationDirs];Destinationdirfortheco-installerdll

XxxCopyFilesSection=11;DIRID_for%WINDIR%\System32dir

;Xxx=driverordeviceprefix

;:

;:

[XxxInstall.OS-platform.CoInstallers];Defineco-installerssection

CopyFiles=XxxCopyFilesSection;Copyfilesdirective

AddReg=Xxx.OS-platform.CoInstallers_AddReg;Addregistrydirective

[XxxCopyFilesSection];Definetheco-installercopyfiles

XxxCoInstall.dll;section

[Xxx.OS-platform.CoInstallers_AddReg];Definetheco-installerAddReg

;section

; The next line defines the co-installer that will be executed when

; installing this device. Defining a registry entry of this type is no

; longer supported and the driver package fails to meet this device

; fundamental requirement.

HKR,,CoInstallers32,0x00010000,"XxxCoInstall.dll,

XxxCoInstallEntryPoint"

Device-class co-installer example:

[Xxx.OS-platform.CoInstallers_AddReg];Definetheco-installerAddReg

;section

;Similarformattothedevice-specificco-installerexample,exceptthe

;registrylocationisunderHKLM.Thenextlinedefinestheco-installer

;executedafteranyinstallationoperationscompleteforthegivendevice

; setup class GUID. Defining a registry entry of this type is no

; longer supported and the driver package fails to meet this device

;fundamentalrequirement.

HKLM,System\CurrentControlSet\Control\CoDeviceInstallers,

{SetupClassGUID},0x00010008,"DevClssCoInst.dll[,DevClssEntryPoint]"

Exceptions:

*This is a requirement for Windows 8 Client ARM, but is recommended for Windows 8 Client x86 and Windows 8 Client x64. It will be required in the future for those architectures.

Business Justification:

This requirement improves driver and device installation performance and reliability by allowing Windows to optimize related driver staging and device configuration operations on the system.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.DevFund.INF.DeviceConfigOnly

Target Feature: Device.DevFund.INF

Title: INF files cannot reference INF directives that are not directly related to the configuration of a device

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64

- Windows 8 Client ARM

Description:

INF directives that provide configuration functionality beyond what is necessary to configure device hardware are no longer supported. The INF file and all supporting files in the driver package must be used only for device installation and configuration.

Design Notes:

The following INF directives may not be referenced in an INF file:

1. RegisterDlls
2. UnregisterDlls
3. ProfileItems
4. UpdateInis
5. UpdateIniFields
6. Ini2Reg

Exceptions:

*This is a requirement for Windows 8 Client ARM, but is recommended for Windows 8 Client x86 and Windows 8 Client x64. It will be required in the future for those architectures.

Business Justification:

This requirement improves driver and device installation performance and reliability by allowing Windows to optimize related driver staging and device configuration operations on the system.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.DevFund.INF.DeviceResourceConfig

Target Feature: Device.DevFund.INF

Title: INF based device resource configuration and non-PnP related configuration cannot be performed within an INF file

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64

- Windows 8 Server x64
- Windows 8 Client ARM

Description:

INF files cannot be used to perform device resource configuration or non-PnP related configuration tasks. Several INF directives and sections are no longer supported.

Design Notes:

The following INF sections and directives cannot be referenced in an INF file:

1. [DDInstall.LogConfigOverride] section
2. LogConfig
3. [DDInstall.FactDef] section

Exceptions:

*This is a requirement for Windows 8 Client ARM, but is recommended for Windows 8 Client x86 and Windows 8 Client x64. It will be required in the future for those architectures.

Business Justification:

This requirement improves driver and device installation performance and reliability by allowing Windows to optimize related driver staging and device configuration operations on the system.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.DevFund.INF.FileCopyRestriction

Target Feature: Device.DevFund.INF

Title: INF based file copy restrictions

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

File copy destination locations are limited to prevent driver packages from installing drivers in inappropriate locations on the system.

Design Notes:

When using the CopyFiles directive, the destination directory specified for a file must be one of the following DIRID values:

- 11 (corresponds to the %WINDIR%\System32 directory)
- 12 (corresponds to the %WINDIR%\System32\Drivers directory)

Only these destination directories expressed as the appropriate DIRID will be a valid copy file location.

Exceptions:

*This is a requirement for Windows 8 Client ARM, but is recommended for Windows 8 Client x86 and Windows 8 Client x64. It will be required in the future for those architectures.

Business Justification:

This requirement improves driver and device installation performance and reliability by allowing Windows to optimize related driver staging and device configuration operations on the system.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.DevFund.INF.FileOrRegistryModification

Target Feature: Device.DevFund.INF

Title: Deleting or modifying existing files, registry entries and/or services is not allowed from within an INF file

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

INF file directives which delete or modify registry entries, services and files are no longer supported.

Design Notes:

The following INF directives may not be referenced in an INF file:

1. DelReg

2. DelService
3. DelPropertry
4. BitReg
5. DelFiles
6. RenFiles

Exceptions:

*This is a requirement for Windows 8 Client ARM, but is recommended for Windows 8 Client x86 and Windows 8 Client x64. It will be required in the future for those architectures.

Business Justification:

This requirement improves driver and device installation performance and reliability by allowing Windows to optimize related driver staging and device configuration operations on the system.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.DevFund.INF.InstallManagement

Target Feature: Device.DevFund.INF

Title: Management of files installed using an INF file is restricted to the system

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

Any files that are installed onto the system using an INF file are managed exclusively by Windows. Plug and Play (PnP) prevents applications from directly modifying the files that are referenced in the INF.

Design Notes:

An INF file must include the PnpLockDown directive set to value 1 in the [Version] section. This would appear as follows in the INF file:

[Version]

; Other Version section directives here

PnpLockDown=1

Exceptions:

*This is a requirement for Windows 8 Client ARM, but is recommended for Windows 8 Client x86 and Windows 8 Client x64. It will be required in the future for those architectures.

Business Justification:

This requirement improves driver and device installation performance and reliability by allowing Windows to optimize related driver staging and device configuration operations on the system.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.DevFund.INF.LegacySyntax

Target Feature: Device.DevFund.INF

Title: Legacy service configuration cannot be performed within an INF file

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

Service configuration using legacy INF syntax is no longer supported.

Design Notes:

The following INF service install section directive may not be referenced in an INF file:

1. LoadOrderGroup

Exceptions:

*This is a requirement for Windows 8 Client ARM, but is recommended for Windows 8 Client x86 and Windows 8 Client x64. It will be required in the future for those architectures.

Business Justification:

This requirement improves driver and device installation performance and reliability by allowing Windows to optimize related driver staging and device configuration operations on the system.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.DevFund.INF.TargetOSVersion

Target Feature: Device.DevFund.INF

Title: The TargetOSVersion decoration in an INF file cannot contain a ProductType flag or SuiteMask flag

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

Within the [Manufacturer] section of an INF file, a TargetOSVersion decoration is used to identify the target OS of the driver package. The TargetOSVersion decoration cannot contain a ProductType flag or SuiteMask flag.

Design Notes:

In Windows 7 and earlier OS versions, the TargetOSVersion decoration is formatted as follows:

nt[Architecture].[OSMajorVersion][.[OSMinorVersion][.[ProductType][\

.[SuiteMask]]]]

Beginning in Windows 8, the ProductType field and SuiteMask field are no longer valid fields in the TargetOSVersion decoration.

Exceptions:

*This is a requirement for Windows 8 Client ARM, but is recommended for Windows 8 Client x86 and Windows 8 Client x64. It will be required in the future for those architectures. Please submit feedback if your driver/accompanying software cannot meet this requirement.

Business Justification:

This requirement improves driver and device installation performance and reliability by allowing Windows to optimize related driver staging and device configuration operations on the system.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.DevFund.Memory

Description:

Requirements related to memory profile

Related Requirements:

- Device.DevFund.Memory.DriverFootprint
- Device.DevFund.Memory.NXPool

Device.DevFund.Memory.DriverFootprint

Target Feature: Device.DevFund.Memory

Title: Drivers must occupy a limited memory footprint

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64

Description:

Drivers must occupy less than or equal to the following size of non-paged code pages in memory:

Non Paged Code Pages

Driver type	x86	x64
Graphics drivers	<= 10MB	<= 10MB
All other driver types	<= 1.66MB	<= 5.45 MB

Driver locked allocations (including MDL allocations and contiguous memory allocations)

- 12MB for all driver types for both architectures

Design Notes: The corresponding test will check the size of the drivers non paged code pages in MB.

Exceptions: Not Specified

Business Justification:

Driver non-paged memory usage constitutes a fixed cost in terms of memory utilization for the overall lifetime of a system. These contribute substantially toward the total OS memory footprint, and most drivers are present in memory at all times. Optimizing driver memory will provide an improved user experience and better overall system responsiveness due to greater availability of memory for user applications.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.DevFund.Memory.NXPool

Target Feature: Device.DevFund.Memory

Title: All driver pool allocations must be in NX pool

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64

Description:

Driver pool allocations must be made in the non-executable (NX) pool.

Design Notes:

A new type of non-paged pool has been introduced which is non-executable (NX) Pool. Since it is non-executable, it is inherently more secure as compared to executable non-paged (NP) Pool, and provides better protection against overflow attacks.

Exceptions: Not Specified

Business Justification:

Moving allocations to the non-executable pool, the surface area of attack for a rogue binary's executable code is minimized.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.DevFund.Reliability

Description:

Reliability tests containing content of the former DEVFUND tests

Related Requirements:

- Device.DevFund.Reliability.BasicReliabilityAndPerformance
- Device.DevFund.Reliability.BasicSecurity
- Device.DevFund.Reliability.BootDriverEmbeddedSignature
- Device.DevFund.Reliability.DriverInstallUninstallReinstall
- Device.DevFund.Reliability.DriverUninstallInstallOtherDeviceStability
- Device.DevFund.Reliability.IOCompletionCancellation
- Device.DevFund.Reliability.NoReplacingSysComponents
- Device.DevFund.Reliability.NormalOpWithDEP
- Device.DevFund.Reliability.PnPIDs
- Device.DevFund.Reliability.PnPIRPs
- Device.DevFund.Reliability.ProperINF
- Device.DevFund.Reliability.RemoteDesktopServices
- Device.DevFund.Reliability.S3S4SleepStates
- Device.DevFund.Reliability.Signable
- Device.DevFund.Reliability.SWDeviceInstallsUsePnPAPIs
- Device.DevFund.Reliability.X64Support

Device.DevFund.Reliability.BasicReliabilityAndPerformance

Target Feature: Device.DevFund.Reliability

Title: Drivers are architected to maximize reliability and stability and do not "leak" resources such as memory (DEVFUND-0016)

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM
- Windows 8 Server x64
- Windows Vista Client x64
- Windows Vista Client x86
- Windows XP x86

Description:

Driver components must not cause the system to crash or leak resources. These resources include but are not limited to the following:

- Memory
- Graphics Device Interface (GDI) or user objects
- Kernel objects such as files, mutex, semaphore, and device handles
- Critical sections
- Disk space
- Printer handles

Design Notes:

To improve the reliability and stability of Windows drivers, all drivers will be subjected to a series of generic driver quality tests. These tests include:

Embedded Signature Verification Test this test verifies that boot start drivers are embedded signed.

Device Install Check for File System Consistency this test verifies that no system resources have been overwritten during the process of a device/driver install

Device Install Check for Other Device Stability this test verifies that no device or driver, except the device under test, has been affected by the device(s)/driver(s) install or co-install process

PCI Root Port Surprise Remove Test This surprise removes PCI root port for the device (if applicable)

PNP (disable and enable) with IO Before and After This test performs basic I/O and basic PNP disable/enable on the test device(s)

Reinstall with IO Before and After this test uninstalls and reinstalls the drivers for test device(s) and runs I/O on these device(s)

Sleep with PNP (disable and enable) with IO Before and After This test cycles the system through various sleep states and performs I/O and basic PNP (disable/enable) on test device(s) before and after each sleep state cycle

Sleep with IO Before and After This test cycles the system through various sleep states and performs I/O on device(s) before and after each sleep state cycle

Plug and Play Driver Test This test exercises PnP-related code paths in the driver under test

Device Path Exerciser Test This consists of a set of tests, each of which concentrates on a different entry point or I/O interface. These tests are designed to assess the robustness of a driver, not its functionality.

All of these tests will be run with Driver Verifier enabled with standard settings.

In addition Driver Verifier will be enabled on all applicable kit tests.

Exceptions: Not Specified

Business Justification:

System crashes, memory leaks, driver installation/uninstallation failures, poor power management/usage, PNP errors as well as IO related errors contribute to a poor end user experience. These tests help isolate some common driver problems.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2006

Comments:

DEVFUND-0016

Device.DevFund.Reliability.BasicSecurity

Target Feature: Device.DevFund.Reliability

Title: Device driver must properly handle various user-mode as well as kernel to kernel I/O requests (DEVFUND-0004)

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM
- Windows XP x86
- Windows Vista Client x86
- Windows Vista Client x64
- Windows Server 2003 x64
- Windows Server 2003 x86
- Windows XP x64

Description:

Driver reliability and security are connected. Reliable drivers help protect the system from malicious attacks.

Compliance will be validated by running the Device Path Exerciser test against the device driver. Device Path Exerciser consists of a set of tests, each of which concentrates on a different entry point or I/O interface. These tests are designed to assess the robustness of a driver, not its functionality.

During a test, Device Path Exerciser typically sends hundreds of thousands of similar calls to the driver in rapid succession. The calls include varying data access methods, valid and invalid buffer lengths and addresses and permutation of the function parameters, including spaces, strings, and characters that might be misinterpreted by a flawed parsing or error-handling routine.

The device driver must comply with the reliability guidelines that are defined in the Windows Driver Kit. All user mode I/O requests and kernel-to-kernel I/O requests must be handled properly to help ensure secure devices and drivers.

Design Notes:

Potential security vulnerabilities include the failure to check for a buffer overflow, the inability to handle bad data from user mode, and the mishandling of unexpected entry points into the driver. If such vulnerabilities are left unidentified and uncorrected, malicious programs could potentially issue denial-of-service attacks or otherwise bypass system security.

For additional information, see the "Creating Reliable and Secure Drivers" and "Creating Reliable Kernel-Mode Drivers" topics in the Windows Driver Kit.

Exceptions: Not Specified

Business Justification:

This requirement exercises the driver and helps ensure that the driver will be reliable and secure when the driver is used in a system.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2006

Comments:

DEVFUND-0004

Device.DevFund.Reliability.BootDriverEmbeddedSignature

Target Feature: Device.DevFund.Reliability

Title: Boot drivers must be self-signed with an embedded signature (DEVFUND-0029)

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64

- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM
- Windows Vista Client x86
- Windows Vista Client x64

Description:

All boot start drivers must be embedded-signed using a Software Publisher Certificate (SPC) from a commercial certificate authority. The SPC must be valid for kernel modules. Drivers must be embedded-signed through self-signing before the driver submission.

Design Notes:

For more information about how to embedded-sign a boot start driver, see Step 6: Release-Sign a Driver Image File by Using an Embedded Signature" at the following website: http://go.microsoft.com/fwlink/?LinkId=237093

After the file is embedded-signed, use SignTool to verify the signature. Check the results to verify that the root of the SPC's certificate chain for kernel policy is "Microsoft Code Verification Root." The following command line verifies the signature on the toaster.sys file:

Signtool verify /kp /v amd64\toaster.sys

Verifying: toaster.sys

SHA1 hash of file: 2C830C20CF15FCF0AC0A4A04337736987C8ACBE3

Signing Certificate Chain:

Issued to: Microsoft Code Verification Root

Issued by: Microsoft Code Verification Root

Expires: 11/1/2025 5:54:03 AM

SHA1 hash: 8FBE4D070EF8AB1BCCAF2A9D5CCAE7282A2C66B3

Successfully verified: toaster.sys

Number of files successfully Verified: 1

Number of warnings: 0

Number of errors: 0

In the Windows Logo Hardware Certification Kit, this requirement will be tested by using the Embedded Signature Verification test.

Exceptions: Not Specified

Business Justification:

Boot drivers must be embedded-signed in order to work properly with the boot process.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2007

Comments:

DEVFUND-0029

Device.DevFund.Reliability.DriverInstallUninstallReinstall

Target Feature: Device.DevFund.Reliability

Title: Device and Driver Installation/un-installation/re-installation must be completed without any error, including function drivers for a multi-function device (DEVFUND-0030)

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM
- Windows XP x86
- Windows XP x64
- Windows Server 2003 x86
- Windows Server 2003 x64
- Windows Vista Client x86
- Windows Vista Client x64

Description:

Device and driver installation, uninstallation, or reinstallation must not fail in any case.

Driver installation must not cause the system to stop running or to restart without user interaction, unless the operating system requires a restart.

For multi-function devices that have separate drivers that enable separate functions, each driver must be capable of installing and uninstalling independently with no start order or hidden dependencies. A multi-function device is a single device that supports multiple functionalities.

Devices that use inbox drivers for operation must also meet this requirement. This requirement does not apply to Internet Small Computer System Interface (iSCSI) devices.

Design Notes:

In the case of multi-function devices, a supervisory driver that loads different drivers for individual functions does not work well with Windows. In particular, driver support is likely to be lost after an operating system reinstallation or upgrade, or when new drivers are distributed through Windows Update. Therefore, such supervisory drivers should be avoided.

This requirement will be tested by using the "Reinstall with IO" test in the Windows Hardware Certification Kit (WHCK).

Exceptions:

This requirement does not apply to Internet Small Computer System Interface (iSCSI) devices. PEP and HAL extensions will be considered for exemption from this requirement.

Business Justification:

This requirement will help ensure a good user experience when the user installs or uninstalls device drivers. A good user experience is critical when the operating system is being upgraded. Testing driver installation, uninstallation, and reinstallation will reduce unpleasant user experiences. The test tool will help us to make sure that all devices and drivers that have a logoare certified never prevent Windows from upgrading smoothly.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2009

Comments:

DEVFUND-0030

Device.DevFund.Reliability.DriverUninstallInstallOtherDeviceStability

Target Feature: Device.DevFund.Reliability

Title: Installing or uninstalling the driver must not reduce or eliminate functionality of other devices or other functional parts of the same device installed on the system (DEVFUND-0006)

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x64

- Windows 7 Client x86
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

Installing or uninstalling a device driver must not reduce or eliminate functionality of other devices that are installed on the system.

This requirement also applies to functional units of a multi-function device, whether that functional unit is on the multi-function device or on the system as a whole.

Design Notes:

The steps for testing this requirement are outlined in the Device install check for other device stability test in the Windows Hardware Certification Kit (WHCK): http://msdn.microsoft.com/en-us/library/ff561407(VS.85).aspx .

Exceptions: Not Specified

Business Justification:

This requirement helps ensure that installing or uninstalling a device or a functional part of a device does not impair the functionality of other parts of the same device or other devices that are on the system. This will help ensure high reliability and availability of the devices that are installed on the system and of the system as a whole.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2006

Comments:

DEVFUND-0006

Device.DevFund.Reliability.IOCompletionCancellation

Target Feature: Device.DevFund.Reliability

Title: Device driver follows design details in the I/O Completion/Cancellation Guidelines (DEVFUND-0013)

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86

- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

I/O completion and cancellation guidelines for drivers provide prescriptive requirements to help ensure that device drivers do not stop responding and can be fully cancelled. These requirements also suggest best practices that drivers can follow to improve performance.

Based on the guideline, all device drivers must meet the following requirements:

- All I/O request packets (IRPs) that are cancelled must be completed within five seconds in an environment that is not resource constrained. No cancellation should be missed even though the cancellation requests may arrive at any instant, even before driver's dispatch routine sees the IRP.

- All resources that a cancelled IRP allocates must be released at IRP cancellation time to prevent hindering system performance under a high cancellation load. The cancellation of the IRP should shut down any I/O intensive process.

In addition, we strongly recommend that drivers do not depend on an additional allocation of resources during cancellation.

Design Notes:

The Windows I/O Manager includes enhancements to support cancellation of the MJ_IRP_CREATE process. The Win32 application programming interfaces (APIs) include enhancements to support cancellation of synchronous operations, such as CreateFile. These enhancements allow I/O cancellation in more environments than in earlier operating systems. For more information, see the "I/O Completion/Cancellation Guidelines" whitepaper at the following website:

http://www.microsoft.com/whdc/driver/kernel/default.mspx.

For more information about designing completion and cancellation logic for drivers, see the following topics in the Windows Development Kit (WDK):

Completing IRPs

Canceling IRPs

Cancel-Safe IRP Queues

Using the System's Cancel Spin Lock

Exceptions: Not Specified

Business Justification:

The primary justification for this requirement is to prevent drivers from causing applications to stop responding. Additionally, users cannot terminate or restart the applications. Drivers that cause applications to stop responding are a significant cause of customer dissatisfaction. A secondary justification for this requirement is to improve the customer experience by permitting I/O cancellation to occur on demand by a user, through the use of user interface (UI) elements such as a universal stop button or cancel buttons. This requirement was introduced in Windows Vista. The requirement adds demands to existing driver cancellation logic and adds the requirement that drivers support cancelling creation requests. Drivers that stop responding can lead to sometimes random application and operating system failures that result in lost customer productivity, lost customer data, and the need to reboot the computer. Beyond these very serious problems, nonexistent or poor I/O cancellation support prevents applications from successfully stopping operations without restarting the application.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2006

Comments:

DEVFUND-0013

Device.DevFund.Reliability.NoReplacingSysComponents

Target Feature: Device.DevFund.Reliability

Title: Vendor-supplied drivers or software must not replace system components (DEVFUND-0005)

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

Driver or software installation must not overwrite any Microsoft-authored operating system components.

If a manufacturers information file (INF) copies any files that the operating system supplies, the INF must copy those files from the Windows product disk or preinstalled source files, unless the component is a licensed, redistributable component.

Drivers that are not authored by Microsoft are not allowed to be named after a Microsoft-supplied driver.

Design Notes:

Each Windows product has a unique set of files that use system file protection. For information about protected operating system files, see the **SfcGetNextProtectedFile** and **SfcIsFileProtected** application programming interfaces (APIs) in the Microsoft platform software development kit (SDK).

Exceptions: Not Specified

Business Justification:

This requirement helps ensure that third-party drivers do not overwrite or use the same names as the operating system components. This requirement addresses Windows operating system reliability when these third-party drivers are loaded.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2006

Comments:

DEVFUND-0005

Device.DevFund.Reliability.NormalOpWithDEP

Target Feature: Device.DevFund.Reliability

Title: All drivers must operate normally and execute without errors with Data Execution Prevention (DEP) enabled (DEVFUND-0041)

Applicable OS Versions:

- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM
- Windows 8 Server x64

Description:

To help ensure proper device and driver behavior and to enhance the security of Windows systems against viruses and other security threats, all drivers must operate normally when Data Execution Prevention (DEP) is enabled. DEP monitors programs to help make sure that the programs use

system memory safely. DEP also protects the system by closing any program that is trying to execute code from memory in an incorrect way.

To meet this requirement, drivers must not execute code from data pages such as default heap pages, various stack pages, and memory pool pages.

DEP is a set of hardware and software technologies that perform additional checks on memory to help prevent malicious code from running on a system. The primary benefit of DEP is to help prevent code execution from data pages. Typically, code is not executed from the default heap and the stack. Hardware-enforced DEP detects code that is running from these locations and raises an exception when execution occurs. Software-enforced DEP can help prevent malicious code from taking advantage of exception handling mechanisms in Windows.

Design Notes:

For more information about DEP, including how to enable DEP, visit the following website:

http://support.microsoft.com/kb/875352

The test for DEP is currently part of the systems test category in the Windows Hardware Certification Kit (WHCK). A device version of this test will be introduced before this requirement is enforced fo certification.

Exceptions: Not Specified

Business Justification:

DEP can help enhance the security of systems that are running Windows operating systems. DEP also helps protect against malicious code, viruses, and other security threats. Making this requirement fundamental for devices will help ensure that all drivers that are signed through the Hardware Certification Program are protected, and that the drivers prevent malware from being signed.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2009

Comments:

DEVFUND-0041

Device.DevFund.Reliability.PnPIDs

Target Feature: Device.DevFund.Reliability

Title: Plug and Play IDs embedded in hardware devices, including each functional unit of a multi-function device, must have device IDs to support Plug and Play (DEVFUND-0003)

Applicable OS Versions:

- Windows Server 2008 x86

- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

Each device that is connected to an expansion bus must be able to supply its own device ID. Each function or device on a multi-function add-on device that is individually enumerated must also provide a device ID for the bus that the function or device uses.

The following are the specific requirements for Plug and Play device IDs:

- Each separate function or device on the system board must be separately enumerated. Therefore, each function or device must provide a device ID for the bus that it uses, as required in the current Plug and Play specification.

- If a device on an expansion card is enumerated, the device must have a unique ID and its own resources according to the current device ID requirements for the bus to which the card is connected. This requirement includes devices that are separately enumerated on multi-function cards or multi-function chips.

- A Plug and Play ID can be shared with other devices that have the same model name, as defined in the device-specific Plug and Play specification.

- Each logical function of the device must have a distinct Plug and Play ID.

- The install (INF) section must key off only the most specific ID according to the Plug and Play guidelines in the Windows Driver Kit.

- For legacy devices such as serial, parallel, and infrared ports, the legacy Plug and Play guidelines define requirements and clarifications for automatic device configuration, resource allocation, and dynamic disable capabilities .

Note: Devices that are completely invisible to the operating system, such as out-of-band systems management devices or Intelligent Input/Output (I2O) hidden devices, still must use Advanced Configuration and Power Interface (ACPI) methods to properly reserve resources and avoid potential conflicts.

The following are the exceptions to the individual device ID requirement for multi-function devices:

- Multiple devices of the same device class, such as multiline serial devices, do not need individual device IDs.

- Devices that are generated by an accelerator or auxiliary processor and that do not have independent hardware I/O do not need individual device IDs. That processor must have an ID, and the MF.sys file must be used to enumerate the dependent devices.

If an OEM uses a proprietary mechanism to assign asset or serial numbers to hardware, this information must be available to the operating system through Windows hardware instrumentation technology.

Design Notes:

See Windows Hardware Instrumentation Implementation Guidelines (WHIIG), Version1.0, at the following website:

http://go.microsoft.com/fwlink/?LinkId=237095

Exceptions: Not Specified

Business Justification:

A unique Plug and Play ID provides a good end user experience for devices. Because a unique device installs each device driver, this requirement helps prevent the issues that occur in Windows Update. This requirement now also includes all aspects of Plug and Play that are relevant for multi-function devices to enable a good Plug and Play experience when the device is used with Windows. This requirement enhances compatibility and reliability when Windows is used with certified devices.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2006

Comments:

DEVFUND-0003

Device.DevFund.Reliability.PnPIRPs

Target Feature: Device.DevFund.Reliability

Title: Drivers must support all PnP IRPs

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64

- Windows 8 Server x64
- Windows 8 Client ARM
- Windows XP x86
- Windows Vista Client x86
- Windows Vista Client x64
- Windows XP x64
- Windows Server 2003 x86
- Windows Server 2003 x64

Description:

Drivers must support all Plug and Play I/O request packets (IRPs) according to the requirements on the following website:

http://msdn.microsoft.com/en-us/library/ff558807(v=VS.85).aspx

The following IRPs are often the cause of driver issues. Special attention should be given to their implementation:

- Removal
    - IRP_MN_QUERY_REMOVE_DEVICE
    - IRP_MN_CANCEL_REMOVE_DEVICE
    - IRP_MN_REMOVE_DEVICE
- Rebalancing
    - IRP_MN_QUERY_STOP_DEVICE
    - IRP_MN_QUERY_RESOURCE_REQUIREMENTS
    - IRP_MN_FILTER_RESOURCE_REQUIREMENTS
    - IRP_MN_CANCEL_STOP_DEVICE
    - IRP_MN_STOP_DEVICE
    - IRP_MN_START_DEVICE
    - IRP_MN_REMOVE
- Surprise Removal
    - IRP_MN_SURPRISE_REMOVAL

Exceptions: Not Specified

Business Justification:

Compliance with the IRPs contributes to providing a good Plug and Play experience.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 12/1/2010

Comments:

DEVFUND-0047

Device.DevFund.Reliability.ProperINF

Target Feature: Device.DevFund.Reliability

Title: Device driver must have a properly formatted INF for its device class (DEVFUND-0001)

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

Driver installation and removal must use Windows-based methods. Therefore, only information file (INF)-based installation routines are allowed. A device driver must have a properly formatted INF for its device as described in the Windows Driver Kit (WDK) "Creating an INF File" topic.

Design Notes:

The INFTest against a single INF" test in the Windows Hardware Certification Kit (WHCK) validates this requirement. For more information about this test, see the Help documentation of the test kit.

Note: If the device does not provide an INF file (that is, the device uses the inbox driver and the INF file only), this requirement does not apply.

Exceptions:

If the device does not provide an INF file (that is, the device uses the inbox driver and the INF file only), this requirement does not apply.

Business Justification:

Using Windows-based methods for driver installation and removal provide end users with a consistent experience and improve the ability to manage the drivers.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2006

Comments:

DEVFUND-0001

Device.DevFund.Reliability.RemoteDesktopServices

Target Feature: Device.DevFund.Reliability

Title: Client and server devices must function properly before, during, and after fast user switching or a Microsoft Remote Desktop Services session (DEVFUND-0009)

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

Devices must support Fast User Switching (FUS) and Remote Desktop Services without losing data before, during, or after sessions. Any user interface (UI) for the device must be shown in the session to which the UI applies. Device usage must not be indefinitely blocked in alternate user sessions.

Exceptions: Not Specified

Business Justification:

FUS and Remote Desktop Services are Windows features. To provide a good and consistent user experience, each device needs to work properly with these services.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2006

Comments:

DEVFUND-0009

Device.DevFund.Reliability.S3S4SleepStates

Target Feature: Device.DevFund.Reliability

Title: All devices and drivers must support S3 and S4 sleep states of the system they are integrated on or connected to (DEVFUND-0043)

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows XP x86
- Windows Vista Client x86
- Windows Vista Client x64
- Windows XP x64
- Windows Server 2003 x86
- Windows Server 2003 x64
- Windows 8 Client ARM

Description:

All devices and drivers must meet the following requirements for systems that are entering S3 and S4 sleep states:

- All devices and drivers must correctly support the request of a system that is going into S3 or S4 states.
- Devices and drivers must not veto the request from the system.
- The devices must support both the S3 and S4 states.
- All devices must be capable of resuming from S3 and S4 sleep states and be fully functional after waking up.
- The device and all its functional units (in the case of multi-function devices) must be enumerated appropriately after the device resumes.
- All devices and drivers must respond properly to Plug and Play events, IOCtl calls, and I/O requests that are issued concurrently with sleep state transitions.

This requirement helps ensure that all certified and signed devices will support the S3 and S4 sleep states when the devices are used as part of a system or are connected externally to a system. This requirement will help the systems conserve power when the system is not being used.

Power management is an important aspect of a good user experience. The system should be able to control which devices are put into a sleep state when the devices are not being used. All devices must comply with the request from the system to go into a sleep state and not veto the request, thereby putting an additional drain on the power source.

Design Notes:

This requirement will be tested by using the "Sleep Stress with IO" test and the "PnPDTest with Concurrent IO in parallel with DevPathExer" test in the Windows Hardware Certification Kit (WHCK).

The system that is used for testing must support S3 and S4.

Note that systems that support Connected Standby will not support S3. Devices in such systems must support S4 requests of that system.

Exceptions: Not Specified

Business Justification:

This requirement will help ensure that the certified devices cooperate with a client system that is trying to go into an S3 or an S4 sleep state to conserve power when the system is not being used.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2009

Comments:

DEVFUND-0043

Device.DevFund.Reliability.Signable

Target Feature: Device.DevFund.Reliability

Title: Device drivers must be able to be signed by Microsoft (DEVFUND-0002)

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64

- Windows 8 Client ARM

Description:

All devices must have code signed drivers. Drivers that are submitted for the Windowscertification must be able to be code signed and must meet the Microsoft guidelines that are defined in the Windows Driver Kit "WHQL Digital Signatures" topic. All boot start drivers must be embedded-signed by using a certificate that is valid for kernel modules. Devices must be embedded-signed via self-signing before the devices are submitted to Microsoft for certification.It isrecommend that all drivers are embedded-signed via self-signing before the drivers are submitted to Microsoft, although this is only required for boot start drivers.

Design Notes:

For requirements for digital signatures, see the "Driver Signing/File Protection" topic at the following website:

http://go.microsoft.com/fwlink/?LinkId=36678

The INF2CAT signability verification tool installs automatically the first time that you create a submission on Microsoft's website. For more information about the INF2CAT tool, visit the following website:

http://go.microsoft.com/fwlink/?LinkId=109929

Exceptions:

This requirement does not apply to devices that use the inbox drivers of the operating system.

Business Justification:

This requirement helps ensure that the driver that is signed through the certification program will be compatible with the Windows operating system. This requirement applies only to submissions that include third-party drivers that are to be signed as part of the Windows Hardware Certification Programprogram. This requirement does not apply to devices that use the inbox drivers of the operating system.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2006

Comments:

DEVFUND-0002

Device.DevFund.Reliability.SWDeviceInstallsUsePnPAPIs

Target Feature: Device.DevFund.Reliability

Title: Software-initiated device-installs use Plug and Play APIs to install device drivers (DEVFUND-0015)

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

Device installers that directly manipulate Plug and Play resources contribute to system instability. Therefore, direct manipulation of Plug and Play resources will be blocked on later releases of Windows. To help ensure compatibility with Windows releases, standard Plug and Play application programming interfaces (APIs) must be used to install device drivers.

Design Notes:

In Windows Vista and later operating systems, standard Plug and Play calls such as the **SetupCopyOEMInf** call pre-stage all required files for device installation on the system automatically. Pre-staging of driver packages will facilitate driver package migration during a system upgrade to Windows Vista or later Windows operating systems. We strongly encourage the use of the Driver Install Framework tools to meet this logo certification requirement. Use of DIFxAPI, DIFxAPP, or DPInst DIFx tools fulfills this requirement.

Exceptions: Not Specified

Business Justification:

Enforcement of this requirement at Windows Vista has enabled Direct Media Interface (DMI) to implement formal lockdown in later Windows versions, with greatly reduced risk to device compatibility.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2006

Comments:

DEVFUND-0015

Device.DevFund.Reliability.X64Support

Target Feature: Device.DevFund.Reliability

Title: Device drivers support x64 versions of Windows (DEVFUND-0014)

Applicable OS Versions:

- Windows Vista Client x86
- Windows Vista Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client x86

Description:

All kernel mode or user mode products and drivers that are submitted for a Microsoft signature or for Windows Hardware Certification must support the x64 version of that specific Windows operating system, with certain exceptions that are described below. All x64 device drivers must adhere to the Microsoft x64 software calling convention, as defined in the Windows Driver Kit.

This requirement applies to Windows Vista and later operating systems. The requirement applies to all drivers submitted for certification.

An x86 driver submission is optional in all cases. However, if a vendor submits an x86 driver or device, the vendor must also submit an x64 driver. Update submissions for x86 drivers do not need to include x64 drivers again unless the updates also apply to x64 drivers.

When a vendor submits any device or driver for x86 architecture, and the device or driver is expected to work with x64 operating system inbox drivers, the Windows Hardware Certification Kit (WHCK) test results of that device with the x64 inbox drivers must also be included in the submission.

This requirement does not apply to IA-64 devices and drivers. IA-64 devices and drivers are not required to support the x64 architecture.

All products refer to all hardware IDs that are enumerated in the operating system for that device or driver. Virtual hardware IDs, or hardware IDs that do not correspond to a physical device but are created in a driver only, must be enumerated identically for x86, IA64 and x64 architectures. No additional prefix or suffix may be appended to differentiate between x86, IA64, and x64 architecture virtual hardware IDs.

This requirement does not apply to devices that are physically embedded on a system that is capable of supporting only an x86 operating system. Vendors that submit devices under this exemption must

provide justification in the Readme file that is included with the submission package. Justification must include relevant information, such as the system name on which the device is embedded and the processor that is used on the system.

For Windows Server 2008: Legacy devices that are submitted for a Windows Server 2008 signature are exempt from this requirement and can make an x86-only submission. Vendors that submit devices under this exemption must provide justification in the Readme file that is included with the submission package. Legacy devices are devices that were at the end-of-life stage when Windows Server 2008 was released, but that are included in systems that will be tested for the supported designation and that need to provide Windows Server 2008 signed drivers for system submissions.

Design Notes:

To increase the safety and stability of the Microsoft Windows platform, unsigned kernel-mode code will not load and will not run on Windows Vista 64-bit platforms. For additional details, see the document that is titled "Digital Signatures for Kernel Modules on Systems Running Windows Vista" at the following website:

http://go.microsoft.com/fwlink/?LinkId=108140

For more information about creating drivers to run on 64-bit editions of Windows operating systems, see the checklist for 64-bit Microsoft drivers at the following website:

http://go.microsoft.com/fwlink/?LinkId=108141

For more information about the Microsoft x64 calling convention, see the "Calling Convention for x64 64-bit Environments" topic in the Windows Driver Kit.

Notes about testing:

This requirement will be tested during the submission approval process after the submission is uploaded to Microsoft. Validation is performed at the hardware ID level.

The test enumerates all hardware IDs that are included in the x86 driver. The test also verifies the following:

- The same hardware IDs are also included in the x64 driver package.
- The hardware IDs have the right descriptors for the operating system's x64 platform.
- The x64 driver has been tested on the x64 platform and results have been submitted.

If a match is not found in the current submission package, the test will search through past approved x64 submissions to find the hardware IDs.

For valid exemption cases, the details of the exempt hardware IDs must be included in the Readme file that is included with the submission.

Exceptions: Not Specified

Business Justification:

Due to the prevalence of x64 architectures, Windows needs to be supported by a viable population of drivers on these for all supported architectures.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2006

Comments:

DEVFUND-0014

Device.DevFund.Reliability.3rdParty

Description:

Reliability tests containing content of the former DEVFUND tests

Related Requirements:

- Device.DevFund.Reliability.3rdParty.FormerTests

Device.DevFund.Reliability.3rdParty.FormerTests

Target Feature: Device.DevFund.Reliability.3rdParty

Title: Former Tests Mapping Requirement

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

The feature Device.DevFund.Reliability.3rdParty and this requirement are a placeholder for mapping of former DevFund tests not found in other requirement.

Exceptions:

This requirement does not apply to devices that use the inbox drivers of the operating system.

Business Justification:

To map former DevFund tests

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2006

Comments: Not Specified

Device.DevFund.Reliability.Interrupts

Description:

Reliability with respect to device interrupts

Related Requirements:

- Device.DevFund.Reliability.Interrupts.BasicReliabilityAndPerformance

Device.DevFund.Reliability.Interrupts.BasicReliabilityAndPerformance

Target Feature: Device.DevFund.Reliability.Interrupts

Title: Drivers must not exceed the maximum number of interrupts, and must support resource arbitration down to a minimum level as defined by the operating system

Applicable OS Versions:

- Windows 8 Server x64

Description:

The driver must be able to tolerate system re-balancing of interrupt resources with any alternative chosen by the OS without failures, including the theoretical minimum of one line based interrupt.

Interrupt arbitration may require multiple iterations. Drivers must be prepared to tolerate cases where their initial interrupt request is rejected. In order to support optimal and timely interrupt arbitration, drivers should provide multiple alternatives at successively reduced interrupt count. Drivers should avoid requesting more than one interrupt per core when possible. Any request for greater than 2048 interrupts per device function will be rejected per the PCIe 3.0 defined MSI-X table entry limit of 2048 per device.

Design Notes:

This requirement is currently untested.

Exceptions: Not Specified

Business Justification:

Requesting more than one interrupt per core can lead to IDT exhaustion in settings where many devices are present. Requesting a total number of interrupts based on the number of cores often leads to memory allocation issues.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: RC

Comments:

NEW

Device.DevFund.Server

Description: Not Specified

Related Requirements:

- Device.DevFund.Server.CommandLineConfigurable
- Device.DevFund.Server.MultipleProcessorGroups
- Device.DevFund.Server.ServerPowerManagement

Device.DevFund.Server.CommandLineConfigurable

Target Feature: Device.DevFund.Server

Title: Windows Server device drivers which have configurable settings provide command line utility or function for device and driver management

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows Server 2008 Release 2 x64
- Windows 8 Server x64

Description:

Windows Server device drivers are required to supply a command-line, scriptable, or answer-file-capable utility or functionality for device and driver management. The following device categories are included:

- Network, including teaming and Infiniband
- Storage, including multipath I/O (MPIO)
- Bus
- Other drivers that may have configurable settings

The specific requirements are the following:

- The utility or tool functionality may use Visual Basic Scripting Edition (VBS), PowerShell, Windows Management Instrumentation (WMI), other functionality that the Windows Server Core option supports, or a proprietary utility or other menu system that functions in the Server Core option.
- The utility or functionality must operate from the command line or be a WMI object and provider that is compatible with the Windows Management Instrumentation Command-line (WMIC) tool.
- The utility must be provided as part of the driver package and be installed by default on the system with the driver.
- The utility must be able to correctly query, display, and change any values that can be changed for the driver.
- The utility must not incorrectly create or set options that do not exist for a specific device or driver. The utility must be capable of changing any setting if the operating system does not provide the ability to change that setting from the command line.
- Changed values or ranges that the user inputs must be automatically checked to ensure that the user input is valid.
- Changes that the utility makes must not require any network or storage stack restarts or system reboots, unless a boot, system, or paging behavior or location is modified.
- Changes that the utility makes are persistent across reboots.
- Help about the utility usage and options is available locally on the system. For example, the utility must provide a "configutil /?" command-line option, or the WMI options for the product must be exposed through standard WMIC commands.
- The utility should not be installed by the information file (INF).
- The utility should be installed by default. This can be accomplished by using a co-installer.

This requirement does not apply to storage arrays, storage fabrics, switches, or other devices that are external to the server system and that can be managed by any system that is attached to the Ethernet, Fibre Channel, or other network.

This requirement does not apply to printers or any device that does not have configurable settings. For example, in a system that uses the graphical interface, there are no "Advanced", "Power Management", or other additional tabs in the Device Manager interface, nor are any utilities available from the vendor that achieve the same effect.

If vendors provide access to functionality only through graphical tools with no command-line or WMI access, vendors must provide information to Microsoft about any functionality that is available only by using graphical tools and is not accessible from the command line or WMI provider. Microsoft may determine, in its sole and final discretion, whether the exception(s) will be permitted.

This requirement applies to any physical device, or device that has a PCI ID, that has a driver; to any driver that is in the network, storage, file system or other stacks; and to any device or driver that otherwise operates at kernel or user mode in the operating system instance on the server.

Exceptions: Not Specified

Business Justification:

This allows for easy command-line scripting to configure underlying hardware.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2010

Comments:

DEVFUND-0046

Device.DevFund.Server.MultipleProcessorGroups

Target Feature: Device.DevFund.Server

Title: Drivers must operate correctly on servers that have processors configured into multiple processor groups

Applicable OS Versions:

- Windows Server 2008 Release 2 x64
- Windows 8 Server x64

Description:

Windows Server uses the concept of KGROUPs to extend the number of processors that a single instance of the operating system can support to more than 64. Both enlightened and unenlightened device drivers must operate correctly in multiple KGROUP configurations.

Design Notes:

For more information, see the Supporting Systems That Have More Than 64 Processors: Guidelines for Developers white paper at the following website:

http://www.microsoft.com/whdc/system/Sysinternals/MoreThan64proc.mspx

This requirement is tested for all server device categories. The test uses BCDEdit functionality to change the boot configuration database (BCD) of the operating system, thus changing the size of the processor groups (the **groupsize** setting) so that multiple processor groups are created. The test also uses BCDEdit functionality to add the **groupaware** setting to the BCD. This changes the behavior of several now-legacy application programming interfaces (APIs) so that the test finds more code errors.

The operating system will not ship with any of these settings. These settings are for testing only and will not be supported in production. To reconfigure the system for normal operations, these settings must be removed from the BCD and the system must be rebooted. The system that is used for testing must include at least four processor cores.

Vendors may configure the system so that it is similar to the Windows Logo Hardware Certification Kit (WLKWHCK) and Device Test Manager (DTM) systems. Vendors can perform their own tests in a multiple processor group configuration, as follows.

The command lines to add the group settings and reboot the computer are the following:

bcdedit.exe /set groupsize 2

bcdedit.exe /set groupaware on

shutdown.exe -r -t 0 -f

The command lines to remove the group settings and reboot the computer are the following:

bcdedit.exe /deletevalue groupsize

bcdedit.exe /deletevalue groupaware

shutdown.exe -r -t 0 -f

Exceptions: Not Specified

Business Justification:

This allows the Windows Server to extend the number of supported processors for a single instance beyond 64.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2009

Comments:

DEVFUND-0035

Device.DevFund.Server.ServerPowerManagement

Target Feature: Device.DevFund.Server

Title: Windows Server device drivers must support Query Power and Set Power Management requests

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64

- Windows Server 2008 Release 2 x64
- Windows 8 Server x64

Description:

During transition into the hibernation (S4) system sleep state, device drivers receive power requests from the operating system. In Windows Server 2008, a server is placed into a pseudo-S4 state during a processor or memory hot replace operation. Device drivers must honor the Query Power and Set Power requests that are dispatched as part of this operation. Device drivers must queue all I/O requests during this pseudo-S4 operation.

A device driver must support this pseudo-S4 sleep state by correctly handling the Query Power and Set Power power management requests. A device driver should never reject a Query Power request. When a device driver receives an S4 Set Power power management request, the driver must transition its devices into a D3 device power state and stop all I/O operations. This includes any direct memory access (DMA) transfers that are currently in progress. When the driver's devices are in a low power state, interrupts are disabled, and all in-progress I/O operations are halted, the replace operation can continue without affecting the device driver.

While a driver's devices are in the D3 power state, the device driver must queue any new I/O requests that the driver receives. A device driver must use an I/O time-out period for the I/O requests that the driver processes. This time-out period must be long enough so that the I/O requests will not time out if they are stopped or queued during the replacement of a partition unit. When the operating system resumes from the pseudo-S4 sleep state, the device driver can resume processing any stopped or queued I/O requests.

The D3 state may be either D3 Hot, to support devices that must respond to external stimuli, or D3 Cold.

A device driver must not bind itself to a uniquely identifiable instance of system hardware, such as a specific processor. If this occurs, the driver may fail if the partition unit that contains that hardware is replaced in the hardware partition.

Design Notes:

For more information, see the Driver Compatibility for Dynamic Hardware Partitioning white paper at the following website:

http://go.microsoft.com/fwlink/?LinkId=237096
http://www.microsoft.com/whdc/system/platform/server/dhp.mspx

Exceptions: Not Specified

Business Justification:

This allows for better power management in Server systems.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 9/1/2008

Comments:

DEVFUND-0042

Device.DevFund.Server.PCI

Description:

PCI

Related Requirements:

- Device.DevFund.Server.PCI.PCIAER

Device.DevFund.Server.PCI.PCIAER

Target Feature: Device.DevFund.Server.PCI

Title: Windows Server PCI Express devices are required to support Advanced Error Reporting [AER] as defined in PCI Express Base Specification version 2.1.

Applicable OS Versions:

- Windows 8 Server x64

Description:

See Tables 6-2, 6-3, 6-4 and 6-5 of the PCI Specification on how errors are detected in hardware, the default severity of the error, and the expected action taken by the agent which detects the error with regards to error reporting and logging.

All three methods of error reporting; completion status, error messages, error forwarding\data poisoning.

Completion status enables the Requester to associate an error with a specific Request.

Error messages indicate if the problem is correctable or not, and fatal or not

Error forwarding\data poisoning can help determine if a particular Switch along the path poisoned the TLP

The following table lists which errors in section 6.2 are required to be reported:

Type of ErrorsRequired? Action

ERR_COR (correctable) No No action, not logged in Event View, system takes no action

ERR_FATAL (fatal, non-correctable) Yes WHEA handles, logged

ERR_NONFATAL (non-fatal) Yes None

Exceptions: Not Specified

Business Justification:

This requirement supports Windows Server RAS, which is a key tenet of Windows Server platforms.

Scenarios:

Windows Server RAS

Success Metric: Pass/Fail

Enforcement Date: 1/1/2012

Comments: Not Specified

Device.DevFund.Server.StaticTools

Description: Not Specified

Related Requirements:

- Device.DevFund.Server.StaticTools.SDVandPFD

Device.DevFund.Server.StaticTools.SDVandPFD

Target Feature: Device.DevFund.Server.StaticTools

Title: Driver Development includes static analysis to improve reliability using Static Driver Verifier (SDV) and Prefast for Drivers (PFD)

Applicable OS Versions:

- Windows 8 Server x64

Description:

Server driver development must include log files for Static Driver Verifier (SDV) and Prefast for Drivers (PFD). Because these tools may produce false errors, you need only submit the logs rather than provide logs which are fully passing.

This requirement is limited to storage, networking, and unclassified drivers only.

Design Notes:

Microsoft's Static Analysis Tools, namely, Prefast for Drivers (PFD) and Static Driver Verifier (SDV) have been found to be highly effective in improving driver reliability by identifying coding issues that would be otherwise difficult to find. Because PFD may produce false errors or warnings, you must fix only those errors and warnings that you deem to be true problems with your driver code.

The results file will be captured by the Windows Logo Hardware Certification Kit for inclusion in the submission package.

Note that there is no requirement that the submitted and subsequently distributed binary be compiled using Microsoft Windows Device Kit or other tools.

It is highly recommended that you fix errors and warnings that might be determined to indicate true problems with your driver code.

Exceptions:

Drivers that are not storage, networking, or unclassified are not required to meet this requirement.

Business Justification:

SDV and PFD provide driver analysis that ensures driver reliability,reliability and a stable system for end users.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Digitizer.Base

Description:

Base for Digitizers

Related Requirements:

- Device.Digitizer.Base.DigitizersAppearAsHID
- Device.Digitizer.Base.HighQualityDigitizerInput
- Device.Digitizer.Base.HighQualityTouchDigitizerInput

Device.Digitizer.Base.DigitizersAppearAsHID

Target Feature: Device.Digitizer.Base

Title: Digitizers appear to the Windows operating system as human interface device (HID) devices

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

Digitizers must not report themselves as a mouse or other proprietary device. In the USB human interface device (HID) usage tables specification, this identification consists of the digitizer page and the usage ID to specify the collection application for pen and touch screens.

Exceptions: Not Specified

Business Justification:

Windows pen and touch features are not available to devices that are not identified as HID digitizers.

Scenarios:

Windows Pen and Touch

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

Touch 2

Device.Digitizer.Base.HighQualityDigitizerInput

Target Feature: Device.Digitizer.Base

Title: Digitizers must provide a high-quality input experience

Applicable OS Versions:

- Windows 7 Client x86
- Windows 7 Client x64

Description:

For devices that support pen or touch input, a pen or touch device must appear to Windows as a human interface device (HID) pen or touch device, respectively. If the device appears a mouse, pen and touch features will not be enabled, and the mouse input requirements will apply.

Pen digitizer requirements are as follows:

- Pen digitizers must appear to the operating system as HID pen digitizers and not as mouse or other proprietary devices.
- Sample rate must be at least 100 Hertz.
- Resolution must be at least 600 pixels per inch and at least five times the display resolution.
- While hovering within 5 millimeters, the pen's position and the position that the device reports must be within 2 millimeters of each other. This accuracy requirement applies whether the input is stationary or in motion.
- The physical contact with the device and the contact position that the device reports must be within 2 millimeters of each other. This accuracy requirement applies whether the input is stationary or in motion.

Touch digitizer requirements are as follows:

- Touch digitizers must appear to the operating system as HID touch digitizers and not as mouse or other proprietary devices.
- Sample rate must be at least 100 Hertz.
- Resolution must be at least 200 pixels per inch and at least matching the display resolution.
- In terms of jitter, if a contact is stationary, the reported position data must not change.
- In terms of contact accuracy, tracing a line, circle, or other predetermined pattern should produce data that is within 0.5 millimeters of the expected data pattern. The pattern may be offset as a whole in accordance with the following contact-offset requirement.
- The physical contact with the device and the contact position that the device reports must be within 2 millimeters of each other. This requirement applies whether the input is stationary or in motion.

Note that we encourage performing linearity calibration before running the pen and touch tests. For more information, see the section about linearity calibration in the OEM Preinstallation Kit (Windows OPK) documentation. For resistive touch digitizers, we recommend optimizing for the touch experience: 80 grams-force spacers provide a good experience.

Exceptions: Not Specified

Business Justification:

Maintains a high quality digitizer input experience.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

INPUT-0001v5

Device.Digitizer.Base.HighQualityTouchDigitizerInput

Target Feature: Device.Digitizer.Base

Title: Windows Touch digitizers must provide a high-quality input experience

Applicable OS Versions:

- Windows 7 Client x86
- Windows 7 Client x64

Description:

Input requirements apply to all touchable areas, including edges and corners, and will be tested over a regular distribution of points and patterns that cover the entire surface. For battery-operated

devices, the requirements must be met whether the device is running on AC or battery power. The Windows Touch logo program is available for multi-touch digitizers that have a screen size of 30 inches or smaller. Multi-touch digitizers that are greater than 30 inches can obtain a signed driver through the unclassified category.

- Multi-touch digitizers must appear to the operating system as human interface device (HID) digitizers, and not as mouse or other proprietary devices.
- Sample rate must be at least 50 Hertz per finger.
- Resolution must be at least 25 pixels per inch and at least matching the display resolution.
- In terms of jitter, for all fingers, if a contact is stationary, the reported position data must not change.
- No data must be reported for locations where contact is not made.
- In terms of contact accuracy, the following requirements must be met on all corners of the screen and at least 95 percent of points and patterns tested.

For a single touch on a stationary contact point, the contact position reported must be within 2.5 millimeters of the target point. For a single touch that traces a line, circle, or other predetermined pattern, the contact data reported must be within 2.5 millimeters of the target pattern, with an offset from the pattern that varies no more than 1 millimeter for every 10 millimeters of travel, and without interruption to the pattern.

For additional touches on a stationary contact point, the contact position reported must be within 5 millimeters of the target point. For additional touches that trace a line, circle, or other predetermined pattern, the contact data reported must be within 5 millimeters of the target pattern, with an offset from the pattern that varies no more than 2 millimeters for every 10 millimeters of travel, and without interruption to the pattern.

Definitions are as follows:

- *Pixels per inch*. Calculation of sqrt(*x*^2 + *y*^2)/diagonal screen size in inches, where *x* is the number of pixels on the horizontal axis and *y* is the number of pixels on the vertical axis.
- *Target point*. The location targeted on the screen. For a target point that is smaller than the area of contact, the digitizer should determine which part of the contact area should be reported, such as the geometric center of the area or the point of greatest pressure. Microsoft tests will be conducted via the geometric center of the contact area of a typical finger (or rounded stylus) that is at least 12.5 millimeters in diameter. Calibration should occur before logo testing for certification.
- *Single touch*. A touch made when no other contact is present on the screen.
- *Additional touches*. One or more touches made when a contact is already present on the screen, or multiple touches placed simultaneously on the screen.

The Windows Touch logo program is a full test. Independent Hardware Vendors (IHVs) must submit hardware to the Windows Touch Test Lab for manual verification. For more information about the Windows Touch Test Lab, go to http://www.microsoft.com/whdc/device/input/WindowsTouch_Test-Lab.mspx.

Manufacturers should contact their account manager for copies of the OEM Preinstallation Kit (Windows OPK) guidelines. A white paper that provides guidance on HID pen and touch digitizer drivers can be found at http://www.microsoft.com/whdc/device/input/PEN_touch.mspx.

Exceptions: Not Specified

Business Justification:

Ensures a high-quality touch input experience on Win7.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

INPUT-0046v8

Device.Digitizer.Pen

Description:

Feature for Pen based Digitizers

Related Requirements:

- Device.Digitizer.Pen.100HzSampleRate
- Device.Digitizer.Pen.ContactAccuracy
- Device.Digitizer.Pen.HoverAccuracy
- Device.Digitizer.Pen.PenRange
- Device.Digitizer.Pen.PenResolution

Device.Digitizer.Pen.100HzSampleRate

Target Feature: Device.Digitizer.Pen

Title: 100Hz Sample Rate

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

The pen digitizer will have a sample rate of at least 100Hz.

Exceptions: Not Specified

Business Justification:

A high packet rate promotes high performance, perceived responsiveness of the system, and data integrity for contacts in fast motion.

Scenarios:

Pen

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

Port of Win7 pen requirement

Device.Digitizer.Pen.ContactAccuracy

Target Feature: Device.Digitizer.Pen

Title: Pen contact accuracy

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

The physical contact with the device and the contact position the device reports must be within 2 millimeters of each other. This applies whether the input is stationary or in motion.

Exceptions: Not Specified

Business Justification:

Supports expected location of pen input in relation to physical input position.

Scenarios:

Pen

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

Port of Win7 pen requirement

Device.Digitizer.Pen.HoverAccuracy

Target Feature: Device.Digitizer.Pen

Title: Pen hover accuracy

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

While hovering within 5 millimeters, the pen's position and the position the device reports must be within 2 millimeters of each other. This applies whether the input is stationary or in motion.

Exceptions: Not Specified

Business Justification:

Supports precise and user predictable experience of cursor movement.

Scenarios:

Pen

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

Port of Win7 pen requirement

Device.Digitizer.Pen.PenRange

Target Feature: Device.Digitizer.Pen

Title: The pen digitizer must prevent false recognition of touch gestures from the non-interactive hand

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

The pen digitizer must report that the pen is within range when it is 10 millimeters away from the screen. X and Y coordinates are not required to be reported at 10 millimeters.

Exceptions: Not Specified

Business Justification:

Many user scenarios for touch machines include the use of pen, especially for text input while resting their writing hand on the screen.

Scenarios:

Windows Pen

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

Touch 15

Device.Digitizer.Pen.PenResolution

Target Feature: Device.Digitizer.Pen

Title: Pen digitizer resolution

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

The pen digitizer resolution must be at least 150 pixels per inch and equal to the native display resolution or greater.

Exceptions: Not Specified

Business Justification:

To enable optimal pen usage experience and accuracy.

Scenarios:

Pen

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

Port of Win7 pen requirement

Device.Digitizer.Touch

Description:

Windows Touch interface for digitizer devices.

Related Requirements:

- Device.Digitizer.Touch.5TouchPointMinimum
- Device.Digitizer.Touch.Bezel
- Device.Digitizer.Touch.DigitizerConnectsOverUSBOrI2C
- Device.Digitizer.Touch.DigitizerJitter
- Device.Digitizer.Touch.ExtraInputBehavior
- Device.Digitizer.Touch.FieldFirmwareUpdatable
- Device.Digitizer.Touch.HIDCompliantFirmware
- Device.Digitizer.Touch.HighResolutionTimeStamp
- Device.Digitizer.Touch.InputSeparation
- Device.Digitizer.Touch.NoiseSuppression
- Device.Digitizer.Touch.PhysicalDimension
- Device.Digitizer.Touch.PhysicalInputPosition
- Device.Digitizer.Touch.PowerStates
- Device.Digitizer.Touch.ReportingRate
- Device.Digitizer.Touch.ResponseLatency
- Device.Digitizer.Touch.TouchResolution
- Device.Digitizer.Touch.ZAxisAllowance

Device.Digitizer.Touch.5TouchPointMinimum

Target Feature: Device.Digitizer.Touch

Title: Touch digitizer supports a minimum of five simultaneous touch inputs

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

The touch digitizer must support a minimum of five simultaneous touch inputs. This applies to all touchable areas, including edges and corners.

Exceptions: Not Specified

Business Justification:

This requirement will lead to a good touch experience.

Scenarios:

Windows Touch

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

Touch 1

Device.Digitizer.Touch.Bezel

Target Feature: Device.Digitizer.Touch

Title: Touch digitizer edges must be easily touched

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

Any bezel that surrounds the display area must not interfere with the user's ability to interact with the edges of the display area.

Tablet device bezels must be flush with the display and border areas to allow for edge to edge consistency across the device's surface. Raised or inconsistent bezels will interfere with user interactions and prevent the user from successfully touching user interfaces, located along the display area edges.

Non-tablet devices may include a minimal z-height in the bezel to allow for touch sensors. To compensate for the bezel, a consistent border area surrounding the display area must be allocated to allow for uninterrupted edge interactions. The boundary between border and display areas must be consistent in z-height and have no physical edge. The boundary should be smooth and not interfere with user interactions.

The bezel, border and display areas are defined as:

Bezel area: Outer most area sounding the border and display areas. The bezel area is a non-active area for touch input. Please refer to **System.Client.Tablet.BezelWidth** for specific Tablet form factor only requirements.

Border area: Area between the bezel and display areas. The border area is an indirectly active area where a user's finger will interact along the display areas edge. The border area must not be less than 20 millimeters to allow for uninterrupted user interactions.

Display area: Inner most area surrounded by the border area. The display area is the active area for display output and touch input.

Exceptions: Not Specified

Business Justification:

The end user must be able to access all touchable parts of the user interface or an application.

Scenarios:

Ensures the end user's ability to interact with the edges of the screen.

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments: Not Specified

Device.Digitizer.Touch.DigitizerConnectsOverUSBOrI2C

Target Feature: Device.Digitizer.Touch

Title: Digitizer connects over USB or I^2C

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

The digitizer must connect to the system over a USB or I^2C bus. These buses support the descriptor for the human interface device (HID) or digitizer according to the Human Interface Design Protocol for USB.

Exceptions: Not Specified

Business Justification:

To ensure consistent and tested platform.

Scenarios:

Windows Touch

Success Metric: Pass/Fail

Enforcement Date: Windows RC

Comments:

Touch 3; Update

Device.Digitizer.Touch.DigitizerJitter

Target Feature: Device.Digitizer.Touch

Title: Digitizer's jitter is a maximum of 1 millimeter over 10 millimeters of travel

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

Non-stationary touch inputs must not:

- Exhibit input jitter which exceeds a maximum of 1 millimeter of jitter over 10 millimeters of travel
- Report duplicate packets for inputs
- Report jitter in the direction opposite of the direction of travel

Stationary touch inputs must produce 0 millimeters of jitter while they are held.

For both non-stationary and stationary inputs there must not be any loss, introduction, or swapping of the reported inputs.

Exceptions: Not Specified

Business Justification:

Windows can incorrectly recognize the interaction as a drag or other movement. This problem causes users to feel frustrated and to perceive the system as untrustworthy. Correctly reporting the integrity of contact during motion is important.

Scenarios:

Windows Touch

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

Touch 7

Device.Digitizer.Touch.ExtraInputBehavior

Target Feature: Device.Digitizer.Touch

Title: Digitizers do not report inputs greater than maximum

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

If the digitizer supports n simultaneous touch inputs (Where 'n' is the maximum number of supported touch inputs reported through HID), the first n inputs remain valid while additional inputs up to 5 must be ignored. If more than n+5 inputs are placed on the screen and accurate tracking of the original n inputs cannot be guaranteed, then it is strongly recommended to stop tracking all inputs including n+5.

Exceptions: Not Specified

Business Justification:

This requirement promotes a good end user experience.

Scenarios:

Windows Touch

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

Touch 4; Updated

Device.Digitizer.Touch.FieldFirmwareUpdatable

Target Feature: Device.Digitizer.Touch

Title: Touch Digitizer firmware must be field updatable by customer

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client ARM
- Windows 8 Client x86

Description:

Touch digitizer firmware binaries must be updateable by the customer in the field.

Exceptions: Not Specified

Business Justification:

Allows the customer to be able to update the touch digitizer firmware when updates are made available.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

New

Device.Digitizer.Touch.HIDCompliantFirmware

Target Feature: Device.Digitizer.Touch

Title: Touch digitizer firmware is human interface device (HID) compliant and does not require additional driver installation.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

Proper human interface device (HID) compliant firmware will not require any additional driver installation.

For more information on implementation, see http://go.microsoft.com/fwlink/p/?LinkId=226808

Exceptions: Not Specified

Business Justification:

Compatibility with Windows.

Scenarios:

Windows Touch

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

Touch 13; Update

Device.Digitizer.Touch.HighResolutionTimeStamp

Target Feature: Device.Digitizer.Touch

Title: High resolution time stamp

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

One per frame, snapped in association to the frame sample time and not any other time in another stage of the pipeline for example, taking the time the scan started rather than when the packet is produced or transmitted.

The time stamp can be taken at the beginning or end of the frame, but the setting should remain consistent. There is no need to synchronize it to any definition of absolute time. Use rollover for the time stamp, so there is no need to reset to zero.

Exceptions: Not Specified

Business Justification:

If a higher-resolution time stamp is available to determine exactly when the data was sampled (which equals the exact time when the finger was touching the reported coordinate of the screen), for example, the gesture recognition can calculate to better determine the intended gesture

Scenarios:

Windows Touch

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

New

Device.Digitizer.Touch.InputSeparation

Target Feature: Device.Digitizer.Touch

Title: Input Separation

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

Distance is measured to center of input and with inputs of 9 millimeters in size.
Converging/Diverging zoom interactions must meet at 12 millimeters or less separation for horizontal and vertical, and 15 millimeters or less for diagonal.

Interactions:

- Zoom diverging and converging 2-finger:
    - Starting contact position is vertical or horizontal: start at 12mm or less
    - Starting contact position is diagonal: start at 15mm or less
- Tap (2-finger, 3-finger, 4-finger):
    - Contact position is horizontal or vertical: tap with contacts 12mm apart or less
    - Contact position is diagonal: tap with contacts 15mm apart or less
- Swipe - parallel movement (2-finger, 3-finger, 4-finger; up, down, left, right):
    - Contact position is horizontal or vertical: swipe with contacts 12mm apart or less
    - Contact position is diagonal: swipe with contacts 15mm apart or less

Exceptions: Not Specified

Business Justification:

Offers a better end user experience.

Scenarios:

Windows Touch

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

Touch 10; Updated

Device.Digitizer.Touch.NoiseSuppression

Target Feature: Device.Digitizer.Touch

Title: The touch digitizer does not report data for locations where touch input is not made

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

The touch digitizer will not report data (Phantom/Ghost contacts) for locations where touch input is not made. This applies for both when the system is actively receiving user input and when it is not receiving user input.

Exceptions: Not Specified

Business Justification:

This guideline promotes an optimal end user experience.

Scenarios:

Windows Touch

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

Touch 8

Device.Digitizer.Touch.PhysicalDimension

Target Feature: Device.Digitizer.Touch

Title: Touch digitizer reports physical dimensions

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

The touch digitizer must report dimensions to the OS which match the visible active size of the display. Reported dimensions can be less than the physical dimensions in cases where the touch digitizer extends beyond the display and bezel into non-visible space. Touch digitizer dimensions will be reported via the Physical Dimensions property.

Exceptions: Not Specified

Business Justification:

Inaccurate information about the physical dimensions can affect the ability of Windows Touch to accurately recognize gestures.

Scenarios:

Windows Touch

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

Touch 16; Updated

Device.Digitizer.Touch.PhysicalInputPosition

Target Feature: Device.Digitizer.Touch

Title: Digitizer reports physical contact with the device and the contact position

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

The touch digitizer reports all inputs within plus or minus 1 millimeter of the center of the physical input for all touchable areas.

All pixels on the screen must be touchable, including edges and corners.

For additional details on verification and testing of this requirement please see http://go.microsoft.com/fwlink/?LinkID=234575

Exceptions: Not Specified

Business Justification:

Minimal offset between the actual and reported points of contact is a primary factor in the real and perceived accuracy of the system.

Scenarios:

Windows Touch

Success Metric: Pass/Fail

Enforcement Date: Windows RC

Comments:

Touch 14; Update

Device.Digitizer.Touch.PowerStates

Target Feature: Device.Digitizer.Touch

Title: The touch controller is required to implement three different power states: Active, Idle, and Sleep

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

The touch controller is required to implement three different power states:

Active - The state where the touch controller is fully powered and functioning per device requirements.

Idle - The state transitioned to from 'Active' when the touch controller has not received input for a specified period of time.Idle timeout is vendor specified (between 5-30 seconds)

Off - The state where the touch controller is powered down

For recommendations of power consumption please refer to
http://go.microsoft.com/fwlink/?LinkID=234153

Exceptions: Not Specified

Business Justification:

This guideline promotes better battery life and power consumption regardless of form factor.

Scenarios:

Windows Touch

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

Touch 12; Update

Device.Digitizer.Touch.ReportingRate

Target Feature: Device.Digitizer.Touch

Title: 100 Hertz minimum reporting rate for all touch inputs

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

When in an active power state, the touch digitizer reporting rate must be maintained at a minimum of 100 Hertz for all touch inputs reported through HID, for both stationary and non-stationary inputs. All reports must be uniquely sampled.

Exceptions: Not Specified

Business Justification:

A high packet rate promotes high performance, perceived responsiveness of the system, and data integrity for contacts in fast motion.

Scenarios:

Windows Touch

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

Touch 9

Device.Digitizer.Touch.ResponseLatency

Target Feature: Device.Digitizer.Touch

Title: Digitizer response latency for idle and active states

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

The touch digitizer must have response latency from an active state that is not greater than 25 milliseconds for the initial input. The touch digitizer must have response latency from an idle state that is not greater than 50 milliseconds for the initial input. Both Active and Idle are internal power state of the touch controller. The response latency for subsequent contacts in an active state should not be greater than 15 milliseconds. Response latency will be measured as the time when the input touches the screen to the time when the Windows operating system receives the data from the hardware.

Exceptions: Not Specified

Business Justification:

This guideline promotes a better end user experience.

Scenarios:

Windows Touch

Success Metric: Pass/Fail

Enforcement Date: Windows RC

Comments:

Touch 11; Updated

Device.Digitizer.Touch.TouchResolution

Target Feature: Device.Digitizer.Touch

Title: Touch digitizer resolution equals native display resolution or greater

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

At a minimum, the touch digitizer resolution will be equal the native display resolution or greater. Every pixel of the display in an integrated touch digitizer is required to be accessible to touch input. Every pixel includes pixels on the edges and in the corners of the display.

For additional details on verification and testing of this requirement please see http://go.microsoft.com/fwlink/?LinkID=234575

Exceptions: Not Specified

Business Justification:

Pixel level resolution is important for graphical applications.

Scenarios:

Windows Touch

Success Metric: Pass\Fail

Enforcement Date: Windows RC

Comments:

Touch 6

Device.Digitizer.Touch.ZAxisAllowance

Target Feature: Device.Digitizer.Touch

Title: Maximum z-axis allowance for touch detection

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

The maximum allowed z-axis for touch detection is 0.5 millimeters. Where possible, a user's input should make physical contact with the screen before a touch input is registered.

Exceptions: Not Specified

Business Justification:

Ensures no accidental contacts of the screen are made while the user is navigating.

Scenarios:

Windows Touch

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

New

Device.Display.Monitor

Description: Not Specified

Related Requirements:

- Device.Display.Monitor.Base
- Device.Display.Monitor.ColorimetricTolerance
- Device.Display.Monitor.DigitalLinkProtection
- Device.Display.Monitor.EDID
- Device.Display.Monitor.Modes
- Device.Display.Monitor.Stereoscopic3DModes

Device.Display.Monitor.Base

Target Feature: Device.Display.Monitor

Title: Base requirements for displays to ensure good end user experience

Applicable OS Versions:

- Windows 8 Server x64

- Windows 8 Client ARM
- Windows 8 Client x64
- Windows 8 Client x86
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64

Description:

All connectors on the monitor must be set to a mode which will not apply CE style overscan or underscan by default. It is ok for the monitor to provide an option to allow the user to configure overscan/underscan using a On screen display.

All video displays that provide a HDMI connector, must support the ITC flag as defined in the HDMI specification.

All digital displays are required to have a single HPD signal transition from low to high on device connection and power up. Periodic toggling of HPD signal after connection or power up is not allowed.

Multiple transition lead source to notify the OS of multiple device arrival and removal event; causing undesirable mode set flashing.

Exceptions: Not Specified

Business Justification:

When users connect a PC to a display, sometimes the start menu or a portion of the desktop might not be visible or the desktop looks shrunk with black borders around it. Therefore the display must not perform overscan/underscan unless the user specifically requests it. If the ITC flag (as defined in the HDMI specification) is set over HDMI, then the display knows that it is connected to a PC and must not apply overscan/underscan compensation. Hence displays that provide a HDMI connector must support the ITC flag to ensure the entire image fits on the screen. Displays must not do scaling since it impacts the readability of text.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Display.Monitor.ColorimetricTolerance

Target Feature: Device.Display.Monitor

Title: Computer display devices can accurately render colors after being calibrated, to within certain colorimetric error tolerances.

Applicable OS Versions:

- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64

Description:

All computer displays must meet the following colorimetric measurement requirements:

1. Maximum luminance level greater than or equal to 75cd/m^2
2. Average 1994 Delta E less than or equal to 20 for the set of 32 colors specified by IEC 61966-4 (Section 11 for Inter-Channel Dependency)

In addition to the requirements for all displays, standalone or desktop displays must meet the following measurement requirements:

1. Average 1994 Delta E less than or equal to 10 for the desktop set of common colors specified by the Windows Color Quality Test Kit
2. Maximum 1994 Delta E less than or equal to 15 for the desktop set of common colors specified by the Windows Color Quality Test Kit

In addition to the requirements for all displays, integrated or notebook displays must meet the following measurement requirements:

1. Average 1994 Delta E less than or equal to 10 for the notebook set of common colors specified by the Windows Color Quality Test Kit.
2. Maximum 1994 Delta E less than or equal to 15 for the notebook set of common colors specified by the Windows Color Quality Test Kit

Design Notes:

1. Before performing measurements, the display can be calibrated or characterized (or both)using an ICC or WCS device color profile, or the default sRGB color profile may be used. This means that the devices default state (i.e. the out of the box settings for contrast, brightness, color temperature, etc.) may not meet the Windows color fidelity requirements.
2. Notebook LCDs are expected to meet these values when the system is running on AC power, not DC (battery) power.

Reference Specifications:

- IEC 61966-4 Multimedia Systems and Equipment - Colour Measurement and Management - Part 4: Equipment using liquid crystal display panels. http://webstore.iec.ch/webstore/webstore.nsf/artnum/025978
- Windows Color Quality Test Kit: http://www.microsoft.com/whdc/archive/colortest.mspx

Note: The test content with in the logo kit replaces the Windows Color Quality test kit.

Exceptions:

This requirement does NOT apply to projectors. A separate Color Fidelity requirement targeted specifically to projectors may be introduced in the future but currently there is no such Logo requirement.

Business Justification:

Consumers expect computer displays to produce accurate, consistent colors: the red in an image will not appear orange or pink or suffer from poor brightness or saturation. Faithfully reproducing colors in digital media content is necessary to enable important computing experiences such as digital photography, video, printing and online commerce. Compliance with the Windows color fidelity requirements will ensure a reasonable level of color accuracy and consistency to meet user's expectations. These requirements were developed in collaboration with leading industry LCD vendors in 2001 and take into account limitations in LCD technology.

Scenarios:

Users viewing digital images see colors that are reasonably consistent between different displays and computers, and also are representative of the original materials. They can assume that people viewing the images on different computers will have experiences that are similar to their own. Users are able to view the contents of their computer displays in common indoor lighting conditions.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

This was the Win7 Display-0069 requirement

Device.Display.Monitor.DigitalLinkProtection

Target Feature: Device.Display.Monitor

Title: Display monitors that supports digital inputs must support digital link protection on all digital inputs

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

- Windows 8 Server x64
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86

Description:

Displays with digital inputs, such as Digital Visual Interface (DVI), High-Definition Multimedia Interface, (HDMI), DisplayPort , etc.. must support a digital monitor link protection mechanism such as High-bandwidth Digital Content Protection (HDCP).

Exceptions: Not Specified

Business Justification:

Digital link protection mechanisms such as High-Bandwidth Digital Content Protection (HDCP) are utilized to protect premium digital content sent over digital connectors from a source to a display monitor. Hence media playback applications will attempt to turn on HDCP if it is not already on. If HDCP fails, then the application may choose to not play the content, or constrict the content. As of Jan 2009, even DVD playback requires HDCP when playing on digital connectors.

Scenarios:

Playback of premium Blu-Ray content using 3rd party media playback applications; Playback of DRM protected content; Protected DVD playback using media playback applications like Windows Media Player.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

This was the Win7 Display-0099 requirement.

Device.Display.Monitor.EDID

Target Feature: Device.Display.Monitor

Title: Display device implements the EDID data structure

Applicable OS Versions:

- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

The monitor must transmit an EDID structure that contains all required fields, as defined in VESA Enhanced Extended Display Identification Data Standard (E-EDID), Release A, Section 3. This EDID must also contain a unique Manufacturer Name, Product code ID and Serial Number. (Serial number is not required for an integrated panel on a mobile or all in one system.)

For analog CRTs, EDID content must indicate at least one VESA mode at 75 Hz or higher for each supported resolution.

All monitors must support E-EDID by implementing an EDID 1.3 or later data structure that:

- Includes timing data for the preferred display mode in Timing #1.
    - For an LCD or other fixed-format display, this display mode is the native, progressively scanned mode of the panel.
    - For other display types, this is the optimal, progressively scanned display mode, which is based on the size and capabilities of the device and must meet the requirements for refresh rates defined above.
- Implements the screen size or aspect ratio fields, bytes 0x15 and 0x16 per the supported EDID version with accurate dimensions.
- Sets byte 0x18, Bit 1 to indicate that the preferred mode meaning per the supported EDID version.
- Includes a unique serial number in at least one of the ID Serial Number field or a Display Product Serial Number string in one of the base block 18 byte descriptors.
- Implements a Display Product Name string in one of the base block 18 byte descriptors, optional for an integrated panel. This string must be suitable for user interface usage.
- Implements a Display Range Limits in one of the base block 18 byte descriptors, unless the device is a Non-Continuous Frequency (multi-mode) display.

Mobile and other all-in-one systems must transmit an EDID structure in one of three ways:

- LCD panel provides one, which is similar to an externally attached monitor.
- If the LCD panel does not provide one, then the WDDM miniport is responsible for defining and providing it to the operating system.
- The WDDM driver may execute the ACPI _DDC method on the child device associated with the internal panel to retrieve an EDID from the system BIOS.

Display devices which implement features such as more than 8 bits per primary color must use EDID 1.4 in order to ensure these capabilities can be expressed to the OS and applications.

Design Notes:

The ACPI specification defines the method to acquire the EDID from the BIOS to achieve equivalent functionality as specified in ACPI 2.0b, Appendix B, or later.

Exceptions: Not Specified

Business Justification:

LCD panels only support one native resolution. All other resolutions need to be scaled to be displayed on the panel. Therefore LCD panels provide the user with the best experience (text and video playback) when the display is set to its native resolution. Especially for clarity of text, Windows has implemented specific features like ClearType and High DPI.

Scenarios:

Windows installation:

A User powers up the system for the first time;

Windows automatically detects all the display devices connected to the system and sets them to their native resolution.

Monitor Plug:

A User connects a display device to the system. Windows automatically detects the new display device and sets the native resolution

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

This was the Win7 Display-0111 requirement that became effective June 1, 2010 and replaces Display-0065

Device.Display.Monitor.Modes

Target Feature: Device.Display.Monitor

Title: Requirement for resolution support for Display Devices

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM
- Windows 8 Server x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64

Description:

A display device can have multiple connectors. The following are the required modes that a display must support on each connector and indicate the support via the EDID (a display is free to support additional modes and call them out in the EDID as well)

For an integrated panel:

The native resolution of the panel must be greater than or equal to 1024 x 768. The native resolution must be supported at 60 Hz progressive or greater or the closest frequency appropriate for the region.

For HD15, DVI, HDMI and DisplayPort connector:

The native resolution of the panel must be greater than or equal to 1024 x 768. The native resolution must be supported at 60 Hz progressive or greater or the closest frequency appropriate for the region.

The following modes must be supported by the display and included in the Established timings in the EDID

- 640 x 480 at 60 Hz progressive (Byte 23h, bit 5 in the Established timing)
- 800 x 600 at 60 Hz progressive (Byte 23h, bit 0 in the Established timing)
- 1024 x 768 at 60 Hz progressive (Byte 24h, bit 3 in the Established timing)

These modes can be supported as full screen or centered

For all other connectors like S-Video, Component, and Composite:

The connector must support the maximum allowable mode as defined in the specification of the standard.

Exceptions: Not Specified

Business Justification:

- Windows UI looks the best on a display that is running at its native resolution. The reason for this is that the pixels are displayed on the screen with no scaling. Also advanced technologies like ClearType are able to operate at a sub pixel level to optimize the clarity of the text. Therefore it is critical for a display to support its native mode.
- On Windows 8, on an integrated panel, Windows will always use the native mode for all scenarios. Therefore it is not necessary for the integrated panel to support other modes.
- On Windows 8, running on a WDDM 1.0 or WDDM 1.1 driver, Windows uses the 640 x 480 mode for displaying the bug check screen.
- On Windows 8, the default mode used by the Basic Display driver on an external monitor is 1024 x 768. This is because many legacy BIOS can most reliably set this mode.

- Windows UI is optimized to run on modes that are 1024 x 768 and greater. Therefore it is critical that each display device supports this mode because it would give the user the opportunity to select it at a later time

Scenarios:

- Integrated Display:
    - User powers up a Windows 8 machine.
    - The firmware (BIOS/UEFI) sets the native mode according to the System.Fundamentals.Firmware.UEFI.Display requirement before Windows is running
    - Windows starts and continues running at the native mode of the panel
- External Display:
    - User powers up a Windows 8 machine with an external monitor connected
    - Windows will use the WDDM 1.2 driver to set the display to its native mode
- Running the Microsoft Basic Display Driver
    - User is running a Windows 8 machine that is running the Microsoft Basic Display driver
    - By default, Windows will set a 1024 x 768 mode
    - However, the user may select a higher mode by using the Display Control Panel

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New;

Device.Display.Monitor.Stereoscopic3DModes

Target Feature: Device.Display.Monitor

Title: A Stereo 3D External Display or Internal Mobile Panel must support a Stereo mode equivalent to its native or preferred resolution.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM
- Windows 8 Server x64

Description:

The native or preferred resolution of the Stereo 3D Display must have an equivalent Stereo mode. The native or preferred resolution of the Display is exposed through its EDID.

Example: If the native resolution of the Stereo 3D Display is 1920 x 1200 in Mono, then it must also support the same native resolution in the Stereo mode.

Exceptions: Not Specified

Business Justification:

This requirement is necessary for a good quality of experience for running Stereo applications in Windows. Windows will switch to Stereo mode when the user launches a Stereo application. Having a stereo resolution equivalent to the native resolution in mono mode will ensure that there is no mode change failure when the user launches a windowed mode or full-screen Stereo application. If the Stereo 3D Display does not support a Stereo equivalent of the native resolution, then the user will be unable to run the stereo application unless a different resolution is selected manually from the Screen Resolution display applet.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New

Device.Graphics.AdapterBase

Description:

The Base feature set required by all graphic devices.

Related Requirements:

- Device.Graphics.AdapterBase.ApplicationVerifier
- Device.Graphics.AdapterBase.DriverVersion
- Device.Graphics.AdapterBase.PowerManagementCompliance
- Device.Graphics.AdapterBase.RegistryEntries
- Device.Graphics.AdapterBase.SubsystemResettable

Device.Graphics.AdapterBase.ApplicationVerifier

Target Feature: Device.Graphics.AdapterBase

Title: Graphics driver components comply with Application Verifier test criteria

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86

Description:

All user-mode modules (.dll or .exe files) that are part of a graphics driver must satisfy the test criteria for Application Verifier tests. During testing of the driver Application Verifier must be turned on for processes where the driver modules are executing. Application Verifier will cause a break when an error is detected.

For graphics drivers, AppVerifier is required for critical executables (i.e. DWM.exe as an example) used to test stability or robustness of the graphics driver.

In addition, Application Verifier must be enabled on IHV's control panel executable as part of this requirement

These Application Verifier tests must be turned on for the processes during testing:

- com
- exceptions
- handles
- heaps
- inputoutput
- leak
- locks
- memory
- rpc
- threadpool
- tls
- hangs

Exceptions: Not Specified

Business Justification:

WDDM display drivers have user mode components. App Verifier increases robustness of the user mode components by looking for memory corruption, leaks, and general API misuse. This improves overall driver stability and therefore stability of the platform.

Scenarios:

Scenarios involve long-haul machine use, where even an infrequent random hang or crash would be very disruptive.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

This was Win7 Graphics-0075 requirement

Device.Graphics.AdapterBase.DriverVersion

Target Feature: Device.Graphics.AdapterBase

Title: Driver DLL for graphic adapter or chipset has properly formatted file version

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86

Description:

The file version of the graphic driver DLLs (UMD, KMD) and .SYS files must match each other and must be of the form:

A.BB.CC.DDDD

- The A field must be set to 9 for WDDM 1.2 drivers on Windows 8.
- The A field must be set to 8 for WDDM 1.1 drivers on Windows 7.
- The A field must be set to 7 for WDDM 1.0 drivers on Windows Vista.
- The A field must be set to 6 for XDDM drivers on Windows Vista.

For Windows 7 and earlier (WDDM 1.1 and earlier) drivers the BB field must be set to the DDI version that the driver supports:

- DirectX 9 drivers (which expose any of the D3DDEVCAPS2_* caps) must set BB equal to 14.

- DirectX 10 drivers must set BB equal to 15.
- D3D11-DDI driver on D3D**10** hardware must set BB equal to 16.
- D3D11-DDI driver on D3D**11** hardware must set BB equal to 17.

For Windows 8 (WDDM 1.2) drivers the BB field must be set to the highest DirectX Feature Level supported by the driver on the graphics hardware covered by the driver:

- A Feature Level 9 driver must set BB equal to 14
- A Feature Level 10 driver must set BB equal to 15
- A Feature Level 11 driver must set BB equal to 17
- A Feature Level 11_1 driver must set BB equal to 18

The CC field can be equal to any value between 01 and 99.

The DDDD field can be set to any numerical value between 0 and 9999.

For example:

Windows Vista DirectX 9.0-compatible WDDM drivers can use the range 7.14.01.0000 to 7.14.99.9999

Windows 7 DirectX 10.0-compatible WDDM 1.1 drivers can use the range 8.15.01.0000 to 8.15.99.9999

Windows 8 WDDM 1.2 driver on DX10 hardware would be 9.15.99.9999

Exceptions: Not Specified

Business Justification:

–

Scenarios:

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New;

Device.Graphics.AdapterBase.PowerManagementCompliance

Target Feature: Device.Graphics.AdapterBase

Title: Graphics adapter complies with VESA and ACPI power management specifications to enable system sleep

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 8 Server x64
- Windows 8 Client ARM
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86

Description:

To ensure correct implementation of operating system-controlled power management, graphic adapters and their drivers must respond to:

- Required device (D0 and D3)power states as highlighted in the WDK
- Operating system power management for ACPI power states
- *VESA BIOS Power Management Functions, which defines extensions to VGA ROM BIOS services for power management. (*This line does not apply to UEFIGOPbasedplatforms)

Exceptions: Not Specified

Business Justification:

The graphics adapter must comply with the above specification to enable system sleep transitions such as Standby and Hibernate. This results in power savings when the system is not in use.

Scenarios:

System sleep and resume

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New; Win7 Graphics-0006

Device.Graphics.AdapterBase.RegistryEntries

Target Feature: Device.Graphics.AdapterBase

Title: Device driver install applications for a graphics device create required registry entries

Applicable OS Versions:

- Windows 8 Server x64

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 8 Client ARM
- Windows Server 2008 Release 2 x64
- Windows 7 Client x86
- Windows 7 Client x64

Description:

The following registry entries must be created during video driver installation:

- REG_BINARY: HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\Video\PART_GUID\ID\InstalledDisplayDrivers: This key should contain the User-mode driver name.

- REG_BINARY: HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\Video\PART_GUID\ID\HardwareInformation.MemorySize

The below Hardware Information values are written by WDDM driver:

- REG_BINARY: HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\Video\PART_GUID\ID\HardwareInformation.ChipType

- REG_BINARY: HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\Video\PART_GUID\ID\HardwareInformation.DACType

- REG_BINARY: HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\Video\PART_GUID\ID\HardwareInformation.AdapterString

- REG_BINARY: HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\Video\PART_GUID\ID\HardwareInformation.BiosString

- REG_BINARY: HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\Video\PART_GUID\ID\HardwareInformation.MemorySize

The following INF directives must be included in the INF:

- Feature Score This is a General Installation setting that must be supported by all WDDM drivers.

- Copy Flags to Support PNP Stop directive

- Driver\Services Start Type directive

- Capability Override settings to disable OpenGL
- [Version] section directives
- [SourceDiskNames] section directives
- General x64 directives
- General [Install] section directives

The Driver DLL must have a properly formatted file version as defined in the requirement Device.Graphics.AdapterBase.DriverVersion

Exceptions: Not Specified

Business Justification:

Several Windows components and 3rd party applications rely on the information being surfaced through the above registry keys which are written by the installation application or by the WDDM driver. Missing or incorrect information could result in application compatibility problems or wrong OS behavior.Examples:The HardwareInformation values are used by the Advanced Settings tab in the Display Control Panel The FeatureScore INF key is used for driver ranking and installation The InstalledDisplayDrivers registry key is used by WHQL testsFor additional details on the above registry keys and their description, please refer to the Installing Display Miniport and User-Mode Display Drivers in the Windows Driver Kit (WDK) on MSDN.

Scenarios:

Please refer to the Business Justification above which discusses the scenarios as well.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New; Win7 Graphics-0034

Device.Graphics.AdapterBase.SubsystemResettable

Target Feature: Device.Graphics.AdapterBase

Title: A graphics subsystem must be resettable to a normal operational state

Applicable OS Versions:

- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64

Description:

If the GPU is hung for any reason, independent of what the hardware is processing at the time, it must be resettable to a normal operational state. This basically implies that TDR must be supported by any GPU.

Hybrid system should be able to handle TDR just like non-hybrid system and have mechanism to reset either (or both) the GPU to bring the system back to a functioning state when the operating system detects a hang.

Design Notes:

The ability to reset the GPU is independent of the current working state of the system. See the Windows Driver Kit, TBD topic.

Exceptions: Not Specified

Business Justification:

This feature will allow us to recover from faults in the display hardware, resulting is a better user experience. If we do not implement this feature, more blue screens will result.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

Graphics-0046

Device.Graphics.AdapterRender

Description:

The Render feature of a Graphic Device.

Related Requirements:

- Device.Graphics.AdapterRender.MinimumDirectXLevel
- Device.Graphics.AdapterRender.RGBFrameBuffer
- Device.Graphics.AdapterRender.YUVSupport

Device.Graphics.AdapterRender.MinimumDirectXLevel

Target Feature: Device.Graphics.AdapterRender

Title: Graphics Adapter implements minimum hardware acceleration capabilities

Applicable OS Versions:

- Windows 8 Client x86

- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86

Description:

The Display subsystem is required to implement the DirectX 9 hardware specification and the driver is required to expose through the D3D9 UMD DDI the capabilities of Direct3D 10 Feature Level 9_3 as described in MSDN here:

- http://msdn.microsoft.com/en-us/library/ff476876(v=VS.85).aspx
- http://msdn.microsoft.com/EN-US/library/ff476150.aspx
- http://msdn.microsoft.com/EN-US/library/ff476149.aspx
- http://msdn.microsoft.com/en-us/library/ff471324(v=VS.85).aspx

Exceptions: Not Specified

Business Justification:

Windows 8 will leverage a modern graphics stack that supports HLSL programmability. Shader Model 3.0 was introduced at the tail end of the Windows XP lifecycle. It has garnered broad adoption and is used ubiquitously across the PC, XBOX console and Windows Phone. To support the modern graphics stack and the benefits it provides to key applications like Internet Explorer, Windows Live and Microsoft Office, all Windows hardware on any form factor is required to support D3D9 with capabilities corresponding to Direct3D 10 Feature Level 9_3.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New; Win7 Graphics-0015

Device.Graphics.AdapterRender.RGBFrameBuffer

Target Feature: Device.Graphics.AdapterRender

Title: Display chipset implements specified component order and endian representation for 8-bpp or greater integer RGB frame buffer formats

Applicable OS Versions:

- Windows 8 Client x86

- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86

Description:

For an N-bit-per-component RGB frame buffer format, the lowest N bits must contain the blue component, the next N bits must contain the green component, the next N bits must contain the red component, and the remaining 32-(3 x N) bits may contain alpha. The resulting 32-bit value must be stored in memory in little-endian format.

Exceptions: Not Specified

Business Justification:

For basic failure modes, the OS will address the graphics device as a linear framebuffer. To correctly display colors in this mode, and agreed upon ordering of bytes is required.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New; Win7 Graphics-0010

Device.Graphics.AdapterRender.YUVSupport

Target Feature: Device.Graphics.AdapterRender

Title: Display driver that supports YUV textures can process textures and functions correctly

Applicable OS Versions:

- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64

Description:

If the hardware supports YUV texture surfaces and the capability is reported, then the driver must be able to process these surfaces without any intermediate transforms and function correctly.

Design Notes:

It should be assumed that the YUV data is in the BT.601 color space unless the YUV texture is greater than 576 height in which case it is in the BT.709 color space, and the appropriate color space conversion matrices should be used when reading the texture.

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

Graphics-0042

Device.Graphics.AdapterRender.D3D101Core

Description:

D3D 10.1 core feature

Related Requirements:

- Device.Graphics.AdapterRender.D3D101Core.D3D101CorePrimary

Device.Graphics.AdapterRender.D3D101Core.D3D101CorePrimary

Target Feature: Device.Graphics.AdapterRender.D3D101Core

Title: If Graphics Device supports Direct3D 10.1, it must comply with the Direct3D 10.1 and DXGI Specifications

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM
- Windows 7 Client x64
- Windows 7 Client x86
- Windows Server 2008 Release 2 x64

Description:

If the graphics devices implements Direct3D 10.1 it must meet all requirements defined in the Direct3D 10.1 specification and must provide sufficient performance for Direct3D 10.1 features.

Since Direct3D 10.1 is a superset of Direct3D 10, implementation of Direct3D 10.1 also requires support of Direct3D 10 feature set.

All features required by this specification must be exposed by the device driver including those features defined for the DXGI DDI header.

Design Notes:

Sufficient performance is described in requirement System.Fundamentals.Graphics.DisplayRender.Performance

Exceptions: Not Specified

Business Justification:

D3D10.1 was an update to the D3D10 specification initially supported in Windows Vista SP1. It provides BGRA format support to allow applications better performance and interoperability with mixed mode GDI and D3D applications. It further enhances the Anti-aliasing capabilities of the D3D10 specifications.D3D10 and by extension D3D10.1is a tightly defined platform which Windows continues to leverage for consistent, performant, hardware accelerated graphical experiences.

Scenarios:

D3D10 provides a well-defined consistent platform for applications and application developers to provide graphically rich experiences that are consistent across a wide variety of hardware and price points.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New

Device.Graphics.AdapterRender.D3D101WDDM11

Description:

D3D 10.1 core feature with WDDM 1.1 additions

Related Requirements:

- Device.Graphics.AdapterRender.D3D101WDDM11.D3D101v11Primary

Device.Graphics.AdapterRender.D3D101WDDM11.D3D101v11Primary

Target Feature: Device.Graphics.AdapterRender.D3D101WDDM11

Title: If WDDM 1.1 Graphics Device supports Direct3D 10.1, it must comply with the Direct3D 10.1 and DXGI Specifications and support BGRA

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86

Description:

If the graphics hardware implements Direct3D 10.1 it must meet all requirements defined in the Direct3D 10.1 specification and must provide sufficient performance for Direct3D 10.1 features.

Since Direct3D 10.1 is a superset of Direct3D 10, implementation of Direct 3D 10.1 also requires support of Direct3D 10 feature set. Additionally the following features originally defined in the D3D9 Hardware specification are now required:

- BGRA

All features required by this specification must be exposed by the device driver including those features defined for the DXGI DDI header.

Design Notes:

Sufficient performance is described in requirement System.Fundamentals.Graphics.DisplayRender.Performance

Exceptions: Not Specified

Business Justification:

D3D10.1 was an update to the D3D10 specification initially supported in Windows Vista SP1. It provides BGRA format support to allow applications better performance and interoperability with mixed mode GDI and D3D applications. It further enhances the Anti-aliasing capabilities of the D3D10 specifications.D3D10 and by extension D3D10.1is a tightly defined platform which Windows continues to leverage for consistent, performant, hardware accelerated graphical experiences.

Scenarios:

D3D10 provides a well defined consistent platform for applications and application developers to provide graphically rich experiences that are consistent across a wide variety of hardware and price points.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New

Device.Graphics.AdapterRender.D3D101WDDM12

Description:

D3D 10.1 core feature with WDDM 1.2 additions

Related Requirements:

- Device.Graphics.AdapterRender.D3D101WDDM12.D3D101v12Primary

Device.Graphics.AdapterRender.D3D101WDDM12.D3D101v12Primary

Target Feature: Device.Graphics.AdapterRender.D3D101WDDM12

Title: If WDDM 1.2 Graphics Device supports Direct3D 10.1, it must comply with the Direct3D 10.1, DXGI and D3D10 portion of the Direct3D 11.1 Feature Specifications

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

If the graphics hardware implements Direct3D 10.1 it must meet all requirements defined in the Direct3D 10.1 specification and must provide sufficient performance for Direct3D 10.1 features.

Since Direct3D 10.1 is a superset of Direct3D 10, implementation of Direct3D 10.1 also requires support of Direct3D 10 feature set. Additionally the following features originally defined in the D3D9 Hardware specification are now required:

- BGRA
- Half Precision pixel formats (5551, 565, 4444)
- Same Surface Blts

Please see the Direct3D 11.1 Features Spec for complete details of all new features required to be exposed with a WDDM 1.2 driver for D3D10+ hardware.

All features required by this specification must be exposed by the device driver including those features defined for the DXGI DDI header.

Design Notes:

Sufficient performance is described in requirement System.Fundamentals.Graphics.DisplayRender.Performance

Exceptions: Not Specified

Business Justification:

D3D10.1 was an update to the D3D10 specification initially supported in Windows Vista SP1. It provides BGRA format support to allow applications better performance and interoperability with mixed mode GDI and D3D applications. It further enhances the Anti-aliasing capabilities of the D3D10 specifications.D3D10 and by extension D3D10.1is a tightly defined platform which Windows continues to leverage for consistent, performant, hardware accelerated graphical experiences.

Scenarios:

D3D10 provides a well defined consistent platform for applications and application developers to provide graphically rich experiences that are consistent across a wide variety of hardware and price points.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New

Device.Graphics.AdapterRender.D3D10ComputeShader

Description:

D3D10* Shader Model 4_* Compute Shader Functionality

Related Requirements:

- Device.Graphics.AdapterRender.D3D10ComputeShader.D3D10CoreC

Device.Graphics.AdapterRender.D3D10ComputeShader.D3D10CoreC

Target Feature: Device.Graphics.AdapterRender.D3D10ComputeShader

Title: If graphic hardware implements D3D10* Shader Model 4_* Compute Shader Functionality it must conform to the D3D11 hardware specifications

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client ARM
- Windows 8 Client x64
- Windows 8 Client x86
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86

Description:

The D3D11 Specification allows for optionally implementing Shader Model 4_* Compute Shader functionality on D3D10* hardware. If the hardware includes support for and the driver exposes this functionality it must conform to the specifications for this feature as defined in the D3D11 Hardware Specification.

Exceptions: Not Specified

Business Justification:

This requirement is necessary to ensure consistency of this feature across multiple implementations, and provide Application developers a robust platform for application development and deployment on the Windows platform.

Scenarios:

D3D11 provides a well defined consistent platform for applications and application developers to provide graphically rich experiences that are consistent across a wide variety of hardware and price points. This feature provides the vendor with the opportunity to differentiate their hardware and provide capabilities for Direct Compute. For additional information on Compute Shaders see the MSDN article: http://msdn.microsoft.com/en-us/library/ff476331(v=VS.85).aspx

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New;

Device.Graphics.AdapterRender.D3D10Core

Description:

D3D 10 core feature

Related Requirements:

- Device.Graphics.AdapterRender.D3D10Core.D3D10CorePrimary

Device.Graphics.AdapterRender.D3D10Core.D3D10CorePrimary

Target Feature: Device.Graphics.AdapterRender.D3D10Core

Title: If Graphics Devices supports Direct3D 10, it must comply with the Direct3D 10 and DXGI specifications

Applicable OS Versions:

- Windows 8 Client x86

- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86

Description:

If the graphic hardware implements Direct3D 10 it must meet all requirements defined in the Direct3D 10 specification and must provide sufficient performance for Direct3D 10 features.

All features required by this specification must be exposed by the device driver including those features defined for the DXGI DDI header. The following list includes some of the required features called out in the Direct3D 10 specification:

- Geometry shader
- Stream output
- Integer instruction set
- New compressed formats
- Render to vertex buffer
- Render to cube map
- Render to volume

Design Notes:

Sufficient performance is described in requirement System.Fundamentals.Graphics.DisplayRender.Performance

Exceptions: Not Specified

Business Justification:

D3D10 was designed to be the baseline graphics hardware requirement for Windows Vista. The primary motivation for this baseline was the desire to present developers with a cleaner API that delivers several key values. Innovation: New features like Geometry shader and Stream Output satisfy the large appetite for graphics technology innovation in the PC ecosystem. Since graphics is a significant area of differentiation for platforms, form factors and applications, Windows must continue to be a leader in this area. Cleaner API: Improved syntax and generalized structure so developing graphics applications is easier Improved performance: Direct3D 10 includes several innovations that deliver better performance Improved Quality: Due to a more strict specification (e.g. rasterization, floating point), testing and certification is more reliable and required less investment in special cases.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New; Win7 Graphics-0020

Device.Graphics.AdapterRender.D3D10D3D11LogicOps

Description:

D3D10-D3D11 Logic Ops

Related Requirements:

- Device.Graphics.AdapterRender.D3D10D3D11LogicOps.D3D10CoreD

Device.Graphics.AdapterRender.D3D10D3D11LogicOps.D3D10CoreD

Target Feature: Device.Graphics.AdapterRender.D3D10D3D11LogicOps

Title: If graphic hardware implements Logic Ops functionality it must conform to the D3D11 hardware specifications

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client ARM
- Windows 8 Client x64
- Windows 8 Client x86
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86

Description:

The D3D11.1 Specification allows for optionally implementing Logic Ops functionality on D3D10, D3D10.1 and D3D11hardware. If the hardware supports and exposes support for this functionality it must conform to the specifications for this feature as defined in the D3D11.1 Hardware Specification.

Exceptions: Not Specified

Business Justification:

This requirement is necessary to ensure consistency of this feature across multiple implementations, and provide Application developers a robust platform for application development and deployment on the Windows platform.

Scenarios:

D3D11 provides a well defined consistent platform for applications and application developers to provide graphically rich experiences that are consistent across a wide variety of hardware and price points. This feature provides the vendor with the opportunity to differentiate their hardware and provide additional capabilities.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New;

Device.Graphics.AdapterRender.D3D10Multisampling4X

Description:

D3D10 Multisampling (4X)

Related Requirements:

- Device.Graphics.AdapterRender.D3D10Multisampling4X.D3D10CoreA

Device.Graphics.AdapterRender.D3D10Multisampling4X.D3D10CoreA

Target Feature: Device.Graphics.AdapterRender.D3D10Multisampling4X

Title: If graphic hardware implements D3D10 4x Multisampling it must conform to the D3D10 hardware specifications

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client ARM
- Windows 8 Client x64
- Windows 8 Client x86
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86

Description:

The D3D10 Specification allows for optionally implementing 4X Multisampling. If the hardware includes support for and the driver exposes this functionality it must conform to the specifications for this feature as defined in the D3D10 Hardware Specification.

Exceptions: Not Specified

Business Justification:

This requirement is necessary to ensure consistency of this feature across multiple implementations, and provide Application developers a robust platform for application development and deployment on the Windows platform.

Scenarios:

D3D10 provides a well defined consistent platform for applications and application developers to provide graphically rich experiences that are consistent across a wide variety of hardware and price points. This feature provides the vendor with the opportunity to differentiate their hardware and provide capabilities for improved image quality.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New;

Device.Graphics.AdapterRender.D3D10Multisampling8X

Description:

D3D10* Multisampling (8X)

Related Requirements:

- Device.Graphics.AdapterRender.D3D10Multisampling8X.D3D10CoreB

Device.Graphics.AdapterRender.D3D10Multisampling8X.D3D10CoreB

Target Feature: Device.Graphics.AdapterRender.D3D10Multisampling8X

Title: If graphic hardware implements D3D10* 8X Multisampling then it must conform to the D3D10 hardware specifications.

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client ARM
- Windows 8 Client x64
- Windows 8 Client x86
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86

Description:

The D3D10 Specification allows for optionally implementing 8X Multisampling. If the hardware includes support for and the driver exposes this functionality it must conform to the specifications for this feature as defined in the D3D10 Hardware Specification.

Exceptions: Not Specified

Business Justification:

This requirement is necessary to ensure consistency of this feature across multiple implementations, and provide Application developers a robust platform for application development and deployment on the Windows platform.

Scenarios:

D3D10 provides a well defined consistent platform for applications and application developers to provide graphically rich experiences that are consistent across a wide variety of hardware and price points. This feature provides the vendor with the opportunity to differentiate their hardware and provide capabilities for improved image quality.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New;

Device.Graphics.AdapterRender.D3D10WDDM11

Description:

D3D 10 core feature with WDDM 1.1 additions

Related Requirements:

- Device.Graphics.AdapterRender.D3D10WDDM11.D3D10v11Primary

Device.Graphics.AdapterRender.D3D10WDDM11.D3D10v11Primary

Target Feature: Device.Graphics.AdapterRender.D3D10WDDM11

Title: If the graphic hardware implements Direct3D 10 and the driver is WDDM1.1, it must comply with the Direct3D 10 and DXGI Specifications and support BGRA

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM
- Windows Server 2008 Release 2 x64

- Windows 7 Client x64
- Windows 7 Client x86

Description:

If the graphic hardware implements Direct3D 10 and the driver is WDDM1.1, it must meet all requirements defined in the Direct3D 10 specification and must provide sufficient performance for Direct3D 10 features. Additionally the following features originally defined in the D3D9 Hardware specification are now required:

- BGRA

All features required by this specification must be exposed by the device driver including those features defined for the DXGI DDI header. The following list includes some of the required features called out in the Direct3D 10 specification:

- Geometry shader
- Stream output
- Integer instruction set
- New compressed formats
- Render to vertex buffer
- Render to cube map
- Render to volume

Design Notes:

Sufficient performance is described in requirement System.Fundamentals.Graphics.DisplayRender.Performance

Exceptions: Not Specified

Business Justification:

D3D10 was designed to be the baseline graphics hardware requirement for Windows Vista. The primary motivation for this baseline was the desire to present developers with a cleaner API that delivers several key values. Innovation: New features like Geometry shader and Stream Output satisfy the large appetite for graphics technology innovation in the PC ecosystem. Since graphics is a significant area of differentiation for platforms, form factors and applications, Windows must continue to be a leader in this area. Cleaner API: Improved syntax and generalized structure so developing graphics applications is easier Improved performance: Direct3D 10 includes several innovations that deliver better performance Improved Quality: Due to a more strict specification (e.g. rasterization, floating point), testing and certification is more reliable and required less investment in special cases.

Scenarios:

D3D10 provides a well defined consistent platform for applications and application developers to provide graphically rich experiences that are consistent across a wide variety of hardware and price points.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New; Win7 Graphics-0020

Device.Graphics.AdapterRender.D3D10WDDM12

Description:

D3D 10 core feature with WDDM 1.2 additions

Related Requirements:

- Device.Graphics.AdapterRender.D3D10WDDM12.D3D10v12Primary

Device.Graphics.AdapterRender.D3D10WDDM12.D3D10v12Primary

Target Feature: Device.Graphics.AdapterRender.D3D10WDDM12

Title: If the graphics hardware implements Direct3D 10 and the Driver is WDDM1.2, it must comply with the Direct3D 10, DXGI and the D3D10 additions in the Direct3D 11.1 Feature Specifications

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

If the graphic hardware implements Direct3D 10 and the driver is WDDM1.2 it must meet all requirements defined in the Direct3D 10 specification and must provide sufficient performance for Direct3D 10 features. Additionally the following features originally defined in the D3D9 Hardware specification are now required:

- BGRA
- Half Precision pixel formats (5551, 565, 4444)
- Same Surface Blts

Please see the Direct3D 11.1 Features Spec for complete details of all new features required to be exposed with a WDDM 1.2 driver for D3D10+ hardware.

All features required by this specification must be exposed by the device driver including those features defined for the DXGI DDI header. The following list includes some of the required features called out in the Direct3D 10 specification:

- Geometry shader
- Stream output
- Integer instruction set
- New compressed formats
- Render to vertex buffer
- Render to cube map
- Render to volume

Design Notes:

Sufficient performance is described in requirement System.Fundamentals.Graphics.DisplayRender.Performance

Exceptions: Not Specified

Business Justification:

D3D10 was designed to be the baseline graphics hardware requirement for Windows Vista. The primary motivation for this baseline was the desire to present developers with a cleaner API that delivers several key values. Innovation: New features like Geometry shader and Stream Output satisfy the large appetite for graphics technology innovation in the PC ecosystem. Since graphics is a significant area of differentiation for platforms, form factors and applications, Windows must continue to be a leader in this area. Cleaner API: Improved syntax and generalized structure so developing graphics applications is easier Improved performance: Direct3D 10 includes several innovations that deliver better performance Improved Quality: Due to a more strict specification (e.g. rasterization, floating point), testing and certification is more reliable and required less investment in special cases.

Scenarios:

D3D10 provides a well defined consistent platform for applications and application developers to provide graphically rich experiences that are consistent across a wide variety of hardware and price points.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New; Win7 Graphics-0020

Device.Graphics.AdapterRender.D3D111Core

Description:

D3D 11.1 core feature

Related Requirements:

- Device.Graphics.AdapterRender.D3D111Core.D3D111CorePrimary

Device.Graphics.AdapterRender.D3D111Core.D3D111CorePrimary

Target Feature: Device.Graphics.AdapterRender.D3D111Core

Title: If Graphics Device supports Direct3D 11.1, it must comply with the Direct3D 11.1 and DXGI Specifications

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86

Description:

A graphics device must meet all requirements defined in the Direct3D 11.1 specification and must provide sufficient performance for Direct3D 11.1 features.

Direct3D 11.1 is a superset of Direct3D 11, (which is a strict superset of Direct3D 10.1 and Direct3D 10) therefore implementation of Direct3D 11.1 also requires full support for the features defined by theDirect3D 10, Direct3D 10.1 and Direct3D 11 specifications respectively.

All features required by this specification must be exposed by the device driver including those features defined for the DXGI DDI header.

Design Notes:

Sufficient performance is described in requirement System.Fundamentals.Graphics.DisplayRender.Performance

Exceptions: Not Specified

Business Justification:

D3D11.1 is a update to D3D11 delivering a few key features like:Logic OpsHalf Precision pixel formats (5551, 565, 4444)Same Surface BltsUAVs at every stageUAV-Multi-sample Antialiasing Target Independent Rasterization (TIR)New shader instructions The features are focused primarily around enabling a modern hardware accelerated graphics stack for Windows 8 and enabling performance across a wider range variety of hardware architectures and price points.D3D11.1 is a tightly defined platform from which Windows continues to leverage for consistent, performant, hardware accelerated graphical experiences.

Scenarios:

D3D11.1 provides a well defined consistent platform for applications and application developers to provide graphically rich experiences that are consistent across a wide variety of hardware and price points.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New

Device.Graphics.AdapterRender.D3D11Core

Description:

D3D 11 core feature

Related Requirements:

- Device.Graphics.AdapterRender.D3D11Core.D3D11CorePrimary

Device.Graphics.AdapterRender.D3D11Core.D3D11CorePrimary

Target Feature: Device.Graphics.AdapterRender.D3D11Core

Title: If Graphics Device implements Direct3D 11, it must comply with the Direct3D 11 and DXGI Specifications

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

If a graphics device implements Direct3D 11, it must meet all requirements defined in the Direct3D 11 specification and must provide sufficient performance for Direct3D 11 features.

Since Direct3D 11 is a superset of Direct3D 10, implementation of Direct 3D 11 also requires support of Direct3D 10.1 feature set and by extension Direct3D 10.

All features required by this specification must be exposed by the device driver including those features defined for the DXGI DDI header.

Design Notes:

Sufficient performance is described in requirement System.Fundamentals.Graphics.DisplayRender.Performance

Exceptions: Not Specified

Business Justification:

D3D11 was introduced with the release of Windows7. It was developed to push the envelope of graphics hardware capabilities that strive to achieve near photo realism in a real time rendering system. At the time of its release the graphics industry entered a new era where graphics hardware was utilized for general purpose computation in massively parallel scenarios. PCs that wish to be considered top of the line devices could derive significant value by integrating D3D11 graphics hardware. D3D11 hardware would be expected in high-end gaming and media devices. The key features of D3D11 include:Innovation: Direct3D pushes the quality and performance bar of the graphics ecosystem with key technologies like tessellation, multithreading, dynamic shader linking, and improved texture compression formats. General Purpose Computing on GPUs: After years of innovation, programmability and parallel performance advancements in the graphics pipeline lend themselves well to DirectCompute a means of utilizing the massive computational power of GPUs for general purposes (e.g. technical computing) Platform Continuity: Direct3D11 is a strict superset of D3D10 and D3D10.1 so investments made on those platforms translate well to D3D11 as well

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New;

Device.Graphics.AdapterRender.D3D11DoublePrecisionShader

Description:

D3D11* Double Precision Shader Functionality

Related Requirements:

- Device.Graphics.AdapterRender.D3D11DoublePrecisionShader.D3D11CoreC

Device.Graphics.AdapterRender.D3D11DoublePrecisionShader.D3D11CoreC

Target Feature: Device.Graphics.AdapterRender.D3D11DoublePrecisionShader

Title: If graphic hardware implements D3D11* Double Precision it must conform to the feature specification as outlined in the D3D11 hardware specification

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client ARM
- Windows 8 Client x64
- Windows 8 Client x86
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86

Description:

The D3D11 Specification allows for optionally implementing Double Precision Shader functionality on D3D11* hardware. If the hardware includes support for and the driver exposes this functionality it must conform to the specifications for this feature as defined in the D3D11 Hardware Specification.

Exceptions: Not Specified

Business Justification:

This requirement is necessary to ensure consistency of this feature across multiple implementations, and provide Application developers a robust platform for application development and deployment on the Windows platform.

Scenarios:

D3D11 provides a well defined consistent platform for applications and application developers to provide graphically rich experiences that are consistent across a wide variety of hardware and price points. This feature provides the vendor with the opportunity to differentiate their hardware and provide additional capabilities for improved computational accuracy.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New;

Device.Graphics.AdapterRender.D3D11DriverCommandLists

Description:

D3D11* Driver Command Lists

Related Requirements:

- Device.Graphics.AdapterRender.D3D11DriverCommandLists.D3D11CoreB

Device.Graphics.AdapterRender.D3D11DriverCommandLists.D3D11CoreB

Target Feature: Device.Graphics.AdapterRender.D3D11DriverCommandLists

Title: If graphics hardware implements the D3D11* driver command list it must conform to the feature specification as defined in the D3D11 hardware specification

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client ARM
- Windows 8 Client x64
- Windows 8 Client x86
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86

Description:

The D3D11 Specification allows for optionally implementing DriverCommandListfunctionality on D3D11* hardware. If the hardware includes support for and the driver exposes this functionality it must conform to the specifications for this feature as defined in the D3D11 Hardware Specification.

Exceptions: Not Specified

Business Justification:

This requirement is necessary to ensure consistency of this feature across multiple implementations, and provide Application developers a robust platform for application development and deployment on the Windows platform.

Scenarios:

D3D11 provides a well defined consistent platform for applications and application developers to provide graphically rich experiences that are consistent across a wide variety of hardware and price points. This feature provides the vendor with the opportunity to differentiate their hardware and provide additional capabilities for improved performance.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New;

Device.Graphics.AdapterRender.D3D11DriverConcurrentObjectCreation

Description:

D3D11* Driver Concurrent Object Creation

Related Requirements:

- Device.Graphics.AdapterRender.D3D11DriverConcurrentObjectCreation.D3D11CoreA

Device.Graphics.AdapterRender.D3D11DriverConcurrentObjectCreation.D3D11CoreA

Target Feature: Device.Graphics.AdapterRender.D3D11DriverConcurrentObjectCreation

Title: If graphics hardware implements the D3D11* Driver Concurrent Object Creation it must conform to the feature specification as defined in the D3D11 hardware specification

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client ARM
- Windows 8 Client x64
- Windows 8 Client x86
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86

Description:

The D3D11 Specification allows for optionally implementing Driver Concurrent Object creation functionality on D3D11* hardware. If the hardware includes support for and the driver exposes this functionality it must conform to the specifications for this feature as defined in the D3D11 Hardware Specification.

Exceptions: Not Specified

Business Justification:

This requirement is necessary to ensure consistency of this feature across multiple implementations, and provide Application developers a robust platform for application development and deployment on the Windows platform.

Scenarios:

D3D11 provides a well defined consistent platform for applications and application developers to provide graphically rich experiences that are consistent across a wide variety of hardware and price points. This feature provides the vendor with the opportunity to differentiate their hardware and provide additional capabilities for improved performance.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New;

Device.Graphics.AdapterRender.D3D11Level9WDDM12

Description:

The Windows 8 WDDM 1.2 Updates to the D3D9 UM DDI

Related Requirements:

- Device.Graphics.AdapterRender.D3D11Level9WDDM12.D3D9UMDDIUpdate

Device.Graphics.AdapterRender.D3D11Level9WDDM12.D3D9UMDDIUpdate

Target Feature: Device.Graphics.AdapterRender.D3D11Level9WDDM12

Title: If the graphics hardware driver implements the WDDM1.2 specification it must include the D3D9 User Mode DDI additions as defined by the D3D11.1 API/DDI Specification

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

A WDDM 1.2 graphics driver is required to implement the D3D9 Adapter DDI and D3D9 DDI additions as defined in the D3D11.1 API/DDI specification in addition to the D3D9 DDI as defined by the DirectX 9 hardware and driver specifications.

Exceptions: Not Specified

Business Justification:

Windows 8 will leverage a modern graphics stack more extensively than ever before. The WDDM 1.2 driver model enables D3D9 graphics hardware to expose additional capabilities for more efficient rendering on Tile based renderers. The D3D11 runtime exposes these new capabilities when running on D3D9 hardware through Feature Level 9_3.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New; Win7 Graphics-0015

Device.Graphics.AdapterRender.D3D11PartialPrecision

Description:

D3D11 Partial Precision shader support

Related Requirements:

- Device.Graphics.AdapterRender.D3D11PartialPrecision.D3D11CoreE

Device.Graphics.AdapterRender.D3D11PartialPrecision.D3D11CoreE

Target Feature: Device.Graphics.AdapterRender.D3D11PartialPrecision

Title: If graphic hardware implements the D3D11.1 Partial Precision Shader Functionality it must conform to the feature specification as defined in the D3D11.1 hardware specification

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client ARM
- Windows 8 Client x64
- Windows 8 Client x86

Description:

The D3D11.1 Specification allows for optionally implementing Partial Precision Shader functionality on D3D9, D3D10* and D3D11* hardware with a WDDM 1.2 driver. If the hardware includes support forand the driver exposes this functionality it must conform to the specifications for this feature as defined in the D3D11.1 Hardware Specification.

Exceptions: Not Specified

Business Justification:

This requirement is necessary to ensure consistency of this feature across multiple implementations, and provide Application developers a robust platform for application development and deployment on the Windows platform.

Scenarios:

D3D11 provides a well defined consistent platform for applications and application developers to provide graphically rich experiences that are consistent across a wide variety of hardware and price points. This feature provides the vendor with the opportunity to differentiate their hardware and provide additional capabilities for improved computational accuracy.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New;

Device.Graphics.AdapterRender.D3D11WDDM12

Description:

D3D 11 core feature with WDDM 1.2 additions

Related Requirements:

- Device.Graphics.AdapterRender.D3D11WDDM12.D3D11v12Primary

Device.Graphics.AdapterRender.D3D11WDDM12.D3D11v12Primary

Target Feature: Device.Graphics.AdapterRender.D3D11WDDM12

Title: If WDDM 1.2 Graphics Device implements Direct3D 11, it must comply with the Direct3D 11, DXGI and the D3D10 portion of the Direct3D 11.1 Features Specification

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

If a graphics device implements Direct3D 11, it must meet all requirements defined in the Direct3D 11 specification and must provide sufficient performance for Direct3D 11 features.

Since Direct3D 11 is a superset of Direct3D 10, implementation of Direct 3D 11 also requires support of Direct3D 10.1 feature set and by extension Direct3D 10.

In Windows 8 the following features originally defined in the D3D9 Hardware specification are now also required:

- Half Precision pixel formats (5551, 565, 4444)
- Same Surface Blts

Please see the Direct3D 11.1 Features Spec for complete details of all new features required to be exposed with a WDDM 1.2 driver for D3D10+ hardware.

All features required by this specification must be exposed by the device driver including those features defined for the DXGI DDI header.

Design Notes:

Sufficient performance is described in requirement System.Fundamentals.Graphics.DisplayRender.Performance

Exceptions: Not Specified

Business Justification:

D3D11 was introduced with the release of Windows7. It was developed to push the envelope of graphics hardware capabilities that strive to achieve near photo realism in a real time rendering system. At the time of its release the graphics industry entered a new era where graphics hardware was utilized for general purpose computation in massively parallel scenarios. PCs that wish to be considered top of the line devices could derive significant value by integrating D3D11 graphics hardware. D3D11 hardware would be expected in high-end gaming and media devices. The key features of D3D11 include:Innovation: Direct3D pushes the quality and performance bar of the graphics ecosystem with key technologies like tessellation, multithreading, dynamic shader linking, and improved texture compression formats. General Purpose Computing on GPUs: After years of innovation, programmability and parallel performance advancements in the graphics pipeline lend themselves well to DirectCompute a means of utilizing the massive computational power of GPUs for general purposes (e.g. technical computing) Platform Continuity: Direct3D11 is a strict superset of D3D10 and D3D10.1 so investments made on those platforms translate well to D3D11 as well

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New;

Device.Graphics.AdapterRender.D3D11WDDM12DoublePrecisionShader

Description:

D3D11* Double Precision Shader Functionality with additional ops codes introduced with WDDM 1.2

Related Requirements:

- Device.Graphics.AdapterRender.D3D11WDDM12DoublePrecisionShader.D3D11v12C

Device.Graphics.AdapterRender.D3D11WDDM12DoublePrecisionShader.D3D11v12C

Target Feature: Device.Graphics.AdapterRender.D3D11WDDM12DoublePrecisionShader

Title: If graphic hardware implements the D3D11* Double Precision Shader Functionality with WDDM 1.2 driver additions it must conform to the feature specifications as defined in the D3D11 hardware specification

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client ARM
- Windows 8 Client x64
- Windows 8 Client x86

Description:

The D3D11 Specification allows for optionally implementing Double Precision Shader functionality on D3D11* hardware. If the hardware includes support for and the driver exposes this functionality it must conform to the specifications for this feature as defined in the D3D11 Hardware Specification. For Windows 8 a WDDM 1.2 Graphics device driver if it supports Double Precision Shader functionality is required to support the extended double precision math as described in the Shader Model Improvements Specification.

Exceptions: Not Specified

Business Justification:

This requirement is necessary to ensure consistency of this feature across multiple implementations, and provide Application developers a robust platform for application development and deployment on the Windows platform.

Scenarios:

D3D11 provides a well defined consistent platform for applications and application developers to provide graphically rich experiences that are consistent across a wide variety of hardware and price points. This feature provides the vendor with the opportunity to differentiate their hardware and provide additional capabilities for improved computational accuracy.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New;

Device.Graphics.WDDM

Description:

The base feature set implemented by drivers supporting all versions of the WDDM

Related Requirements:

- Device.Graphics.WDDM.Base
- Device.Graphics.WDDM.Checklist
- Device.Graphics.WDDM.GPUFenceCommands

Device.Graphics.WDDM.Base

Target Feature: Device.Graphics.WDDM

Title: Graphics drivers must be implemented per WDDM 1.0 spec

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client x64
- Windows 8 Client x86
- Windows 8 Client ARM
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64

Description:

As implied by the WDDM 1.0 Specification, Display Drivers must minimally support D3D9 and Pixel Shader 2.0.

WDDM 1.0 introduces the following key requirements:

- User-mode Display Driver
- Video Memory Manager
    - (p1) Linear Memory Manager
    - (p1) Rectangular Memory Manager
- GPU Scheduler
- Mode-Switch Architecture Cleanup
- Merged Miniport and DLL
- Recovery from Hardware Hangs
- Simplified Kernel-mode Objects
- Legacy DDI Consolidation
- Hot-plug of Display Cards, Hot Docking, and Support for Clone View

MSDN documentation is updated based on the WDDM 1.0 Specification. Please verify MSDN documentation for WDDM 1.0 requirements.

Exceptions: Not Specified

Business Justification:

Key benefits to the end-user include: Improved Stability by moving parts of the display driver into user mode memory, and adding support for hang recovery (Timeout Driver Recovery - TDR). Video Memory Virtualization Prevents a single application from allocating all video memory for its exclusive use. Scheduling more effective sharing (multitasking) of GPU. Security GPU sharing and video memory virtualization prevent a single app from taking over these resources. Simplifies driver development by obsoleting legacy Direct3D features. OS promotes legacy fixed function features to D3D9 and Shader 2.0. WDDM removes the notion of Device Lost and supports newer Direct3D interfaces such as D3D9Ex, D3D10, 10.1.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

Win7 Graphics-0007

Device.Graphics.WDDM.Checklist

Target Feature: Device.Graphics.WDDM

Title: All Graphics devices comply with base requirements checklist for graphics cards, chipsets and drivers.

Applicable OS Versions:

- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64

Description:

[Version 2: Noted that digital connectors highly recommended but not required until Jun 2010.]

[Version 3: Removingdigital connectors requirement point]

All Graphics Cards need to adhere to the following checklist:

Memory Allocations and Access (applicable for all form factors)

- The GPU must not access an allocation after the last DMA buffer, that was submitted to the GPU referencing that allocation, is reported as complete.
- The GPU must not access any allocations which were not specified in the allocation list associated with the DMA buffer being executed.

Card/Chipset Requirements (not applicable for server devices)

- For a multi-headed display adapter, it is recommended that the display adapter expose all display resources through a single function and not as a multifunction device. If a sound

controller or tuner is part of the device, the device can then be exposed as a multifunction device.

- If a second display class function is exposed for legacy compatibility the adapter must be fully functional without using the second head. The second(or additional) functional heads must:
  - Have sub-class 80 (other) to avoid a generic driver being used.
  - Not have an expansion ROM.
  - Not describe more than 1 MB of total memory space resources.
  - Not duplicate the frame buffer resources.
  - Not describe any interrupt resources.
  - Not describe any I/O space resources.
  - Non display base class functions on the same device, such as a multimedia device class, sub class video device, or sub class audio device, are not subject to these restrictions

WDDM Driver Checklist (not applicable for non-WDDM drivers)

- WDDM drivers must not report a submission fence as completed until the operation for the associated submission is truly completed. This is required by the scheduling model in WDDM. The WDDM must not use the DDI in such a way that the stability of Windows is compromised.
- WDDM drivers must insert a fence/interrupt pair for each fence requested and must not hold off reporting the completion of that fence. The fence must be reported as soon as the associated required interrupt is generated by the GPU. For example, it must not wait until the timer interrupt or until the next VSync. This is required by the Scheduling model in DDM.
- WDDM drivers must not wait or block during DdiSubmitCommand which is necessary from the perspective of the Video Scheduler in WDDM. The driver must not wait or block during a DdiBuildPagingBuffer call as well.
- The WDDM driver must not expose more than one node per physical engine. The driver and GPU cannot schedule a single physical engine between multiple nodes.
- The WDDM driver must not map a virtual address to video memory which is directly exposed to an application. This is fundamental to the implementation of video memory management support in the WDDM driver. The WDDM driver must not hide or expose video memory in a way that the video memory manager is unaware of.

Exceptions to this are allowed specifically for the implementation of GPU Developer Tools (Debuggers, Profilers,). Such exception must apply only in a scenario where the GPU developer

tools, in order to perform, need to map video memory to virtual address space, for the duration of the session and only for the process of the application being operated on.

- The WDDM driver must not expose any memory segments which are used for the sole purpose of reporting additional video memory than is actually present for its appropriate use. The correct amount of video memory must be reported for use by various applications through Windows. The WDDM driver can use up to 5% of the system memory for internal use such as cursor bitmaps, ring buffer, etc; any amount above this must not be used to hide or expose video memory in a fashion such that the video memory manager is unaware of.
- A WDDM driver must use ACPI for all interactions with the system BIOS. SMI is currently allowed but highly discouraged by Microsoft. See WinHEC 2005 presentation, *TWAR05007*

Design Notes:

For more information on any of the items in the Details section, refer to the Windows Driver Kit and search for the relevant keywords.

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

Graphics-0074

Device.Graphics.WDDM.GPUFenceCommands

Target Feature: Device.Graphics.WDDM

Title: GPU that is capable of processing fence commands in the command queue triggers an interrupt

Applicable OS Versions:

- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64

Description:

When the hardware consumes a fence command, it must notify the operating system by triggering an interrupt to the CPU, with the fence ID communicated to the ISR.

Design Notes:

See the Windows Driver Kit

Exceptions: Not Specified

Business Justification:

This feature is required to make GPU scheduling work. Same as interruptible HW.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

Graphics-0044

Device.Graphics.WDDM.Display

Description:

The base feature set implemented by drivers supporting all versions of the WDDM Display DDIs

Related Requirements:

- Device.Graphics.WDDM.Display.Base
- Device.Graphics.WDDM.Display.GammaCorrection
- Device.Graphics.WDDM.Display.HotPlugDetection
- Device.Graphics.WDDM.Display.I2CSupport
- Device.Graphics.WDDM.Display.MediaCenterResolutionTiming
- Device.Graphics.WDDM.Display.Multimon
- Device.Graphics.WDDM.Display.ResetToVGA

Device.Graphics.WDDM.Display.Base

Target Feature: Device.Graphics.WDDM.Display

Title: Graphics drivers must be implemented per WDDM 1.0 spec

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client x64
- Windows 8 Client x86
- Windows 8 Client ARM
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86

Description:

See requirement **Device.Graphics.WDDM.Base**

Exceptions: Not Specified

Business Justification:

In order for testing to be conducted correctly within the Windows Hardware Certification kit, this requirement was created to ensure the feature it is associated with is exposed correctly for testing.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New

Device.Graphics.WDDM.Display.GammaCorrection

Target Feature: Device.Graphics.WDDM.Display

Title: Graphics Adapter must support gamma correction in hardware without using any additional graphics memory bandwidth

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client ARM
- Windows 8 Client x64
- Windows 8 Client x86
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86

Description:

ICM uses the ability to perform gamma correction for the attached monitor and to allow game applications to switch palettes. This capability also supports transition effects in applications. To support ICM, the display adapter or chipset gamma must be programmatically adjustable.

To perform gamma correction in hardware, downloadable RAM DAC entries (LUT) must be included.

The LUT must implement at least 256 entries per component input for 8-bit color channel components. Hardware may implement the LUT for larger component resolutions by using interpolation if at least 128 sample points are used.

This ability must be supported without requiring the use of graphics memory bandwidth.

Exceptions: Not Specified

Business Justification:

Not having this will break the logon/logoff, sleep/wake, hibernate/resume fade effect, and Win7 Display Color Calibration (DCC) tool and its display calibration loader. This would also break every third-party display calibration tool, including those by X-Rite and DataColor. This would make Windows useless for serious digital color photography work. This is required for servers also since DCC is included in the Experience Pack for the Server SKUs.

Scenarios:

Display Color Calibration User launches the “Display Color Calibration Tool” provided by Windows As part of the available options, the user can configure the gamma Full screen game User launches a full screen DirectX game User access the “Options” and typically has the ability to adjust the “gamma”

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New; Win7 Graphics-0028

Device.Graphics.WDDM.Display.HotPlugDetection

Target Feature: Device.Graphics.WDDM.Display

Title: Graphics adapter must reliably detect the connect and disconnect event of display devices.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86

Description:

Windows supports two levels of display detection interruptible and poll-able.

- Interruptible is defined as the case where the graphics adapter is able to automatically detect a change in display connectivity and report it to Windows. The ability of the hardware to automatically generate an interrupt for display connectivity change is called Hot Plug Detect (HPD). The HPD must not cause any visual corruption on any display already connected to the system.

- Poll-able is defined as the case where Windows has to explicitly query the graphics adapter to query for a change in the display connectivity. In some cases visual corruption might be displayed for a very brief time.

All standard digital connectors (DisplayPort, HDMI, DVI) support HPD and the graphics adapter must report these connectors as interruptible. Once the hardware generates the interrupt, the graphics adapter must notify Windows via the DxgkCbIndicateChildStatus DDI found here: http://msdn.microsoft.com/en-us/library/ff559522(VS.85).aspx.

Analog connectors (HD-15, S-Video, Component, Composite) are not required to support interruptible display detection. However, it is possible that HPD can be implemented in the hardware (e.g. load sensing, I2C) for such connectors. In such a case the graphics adapter must report this connector as interruptible as above. Software polling cannot be used to achieve HPD functionality for analog connectors.

For those analog connectors, where HPD is not implemented in the hardware, the graphics adapter must report the connector as polled. In such a case the graphics adapter must only perform detection on the connector when explicitly requested by Windows via the D3DKMTPollDisplayChildren DDI, found here: http://msdn.microsoft.com/en-us/library/ff547077(v=VS.85).aspx.

Design Notes:

See the Windows Driver Kit: http://msdn.microsoft.com/en-us/library/ff559522(VS.85).aspx for" DxgkCbIndicateChildStatus."

The following HPD methods are VESA standards: DVI HPD is covered in the VESA Plug and Play (PnP) Standard for the Display/Graphics Subsystem, Release A. DisplayPort HPD is covered in all versions of the DisplayPort standard.

Exceptions: Not Specified

Business Justification:

HPD is required on HDMI, DVI, and DisplayPort and there are existing industry specifications on HPD for both HW and SW, however this requirement is primarily to ensure a robust system level solution for detecting and configuring monitors. This will help provide the best experience to end consumers when they plug and play a monitor to the PC over digital interfaces.

Scenarios:

Plug in a monitor over a digital interface: The OS will recognize the newly connected monitor via a hot plug event notification from the graphics adapter and will add this monitor to the list of available monitors. Disconnect in a monitor over a digital interface: When a monitor is disconnected, the monitor removal hot plug event is detected by the graphics subsystem and the OS is notified. The OS will accordingly remove this monitor from the list of available monitors.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New; Win7 Graphics-0022 & Graphics-0031 merge

Device.Graphics.WDDM.Display.I2CSupport

Target Feature: Device.Graphics.WDDM.Display

Title: Graphics device driver must have I2C support in WDDM

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client ARM
- Windows 8 Client x64
- Windows 8 Client x86
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86

Description:

The following is the set of interfaces that every WDDM driver is required to implement for I2C support:

DxgkDdiI2CReceiveDataFromDisplay

DxgkDdiI2CTransmitDataToDisplay

These interfaces are documented in the WDK and can be found here.

Exceptions: Not Specified

Business Justification:

Windows uses the I2C interfaces to obtain the EDID from each of the displays connected to the system. Obtaining the EDID is critical for setting the native resolution of the panel. In cases where the display connector does not inherently support hot plug detect, the graphics driver might use the I2C line to detect the presence/absence of the display device. Additionally, Windows provides a rich set of APIs for Monitor configuration. These APIs are useful for advanced configuration of Display devices. Such advanced configuration is for being able to programmatically control display settings like Brightness, Contrast, Color calibration, image position, image size etc. The APIs are documented here on MSDN. The above mentioned interfaces are needed to support this set of APIs.

Scenarios:

Connecting a Display Device User connects a new display device to the system Graphics Adapter detects the presence of the display Windows will use the I2C line to query the display for the EDID Windows will use the EDID to set the native resolution of the display Detecting a Display User

connects a new display device to the system However, hot plug is not supported User opens the display control panel and presses the “Detect” button The graphics driver uses I2C to reliably detect the presence of the display and reports it to Windows. Advanced Display Control Application If a software developer wants to develop an application that has the ability to programmatically control the display settings, then he would use these APIs.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New; Old ID Gfx8077

Device.Graphics.WDDM.Display.MediaCenterResolutionTiming

Target Feature: Device.Graphics.WDDM.Display

Title: Display subsystem that supports Media Center functionality must support digital television (EIA/CEA-861B) resolutions and timings over digital interfaces

Applicable OS Versions:

- Windows 7 Client x86
- Windows 7 Client x64

Description:

VESA specifies display modes and timing information that standard VGA graphics and display monitors use. VESA does not specify display modes and timing information for consumer displays (TV). To support Media Center functionality requirements for display modes and timing, the graphics subsystem must also support all display modes and timing information over VGA, DVI, HDMI, or other digital interconnect, as required by EIA/CEA-861B. These display modes must be exposed by the video miniport so that they appear in the default timings list.

The required modes are those listed as mandatory in the EIA/CEA-861B specification as well as both the two high definition modes:

- 1280x720p (60Hz, 59Hz and 50Hz)
- 1920x1080i or 1920x1080p(60Hz, 59Hz and 50Hz)
- 720x480p (59Hz)
- 720x576p (50Hz)

All other variants of resolutions and refresh rates defined in EIA/CEA-861B are optional, but recommended to support the widest range of consumer displays.

Design Notes:

For EIA/CEA requirement details, see EIA/CEA-861-B, Sections 3.1 and 4, "A DTV Profile for Uncompressed High-Speed Digital Interfaces."

59Hz is defined in the Windows Display Driver Model (WDDM)to be equivalent to 59.94Hz.

Exceptions: Not Specified

Business Justification:

The TV output from the PC must at least be equal to CE set top boxes available today. Video playback in Media Center will degrade.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

Graphics-0055

Device.Graphics.WDDM.Display.Multimon

Target Feature: Device.Graphics.WDDM.Display

Title: If a Graphics adapter supports more than 1 source and 1 target, it must support all multiple-monitor configurations in Windows

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client ARM
- Windows 8 Client x64
- Windows 8 Client x86
- Windows 7 Client x64
- Windows 7 Client x86
- Windows Server 2008 Release 2 x64

Description:

A graphics driver can enumerate sources and targets based on its capabilities. Windows queries the driver for the number of sources and targets by calling the DxgkDdiEnumVidPnCofuncModality DDI, found here http://msdn.microsoft.com/en-us/library/ff559649.aspx. Windows supports the following monitor configurations:

1. Single monitor configuration only one physical monitor is active and the entire desktop is displayed on it

2. Extended monitor configuration multiple monitors are active in and different parts of the desktop are displayed on it. GDI must be made aware of the monitor boundaries such that Windows features like maximize, aero snap etc. work according to spec

3. Duplicate monitor configuration the exact same desktop contents are displayed on multiple monitors

The number of targets must always be greater than or equal to the number of sources.

If the driver enumerates exactly 1 source and 1 target, then no multiple monitor configurations are supported.

If the driver enumerates exactly 1 source and multiple targets, then the driver must support single monitor and duplicate monitor configurations.

If the driver enumerates multiple sources and multiple targets, then the driver must support all the supported configurations. Additionally:

- the capability of each source when enabled by its self, with respect to resolution, Direct 3D, protected content playback, should be the same. The capabilities of the target will vary based on the target type.
- the operating system must be able to drive any target from any source, although the driver can constrain which targets can be driven in combination.

Multiple-monitor support is built into Windows; therefore, graphics drivers must not include any special code to provide support already available in the OS.

It must be possible for a user to set all the configurations supported using the Windows Display Control Panel and by pressing the Win+P key combination.

Exceptions: Not Specified

Business Justification:

Windows enables different user interfaces based on the number of sources and targets enumerated by the driver. Once the driver has enumerated the sources and targets it is important for the driver to support all the multiple monitor configurations that are expected by Windows so that the user interface can accurately represent the current state and allow the user to make changes to the monitor configurations. It is important that all drivers support these configurations so that the user has confidence in the Windows platform and its eco system.

Scenarios:

Projection User connects a Windows laptop to a projector User hits the Win + P key and is shown 4 options User can select any option with confidence that it will work Desktop User connects multiple monitors to his desktop User opens the display control panel and configures the monitors in extended mode

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Graphics.WDDM.Display.ResetToVGA

Target Feature: Device.Graphics.WDDM.Display

Title: Display adapter or chipset supports standard VGA modes and can be reset to standard VGA modes

Applicable OS Versions:

- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64

Description:

The display adapter or chipset must support standard VGA. If necessary, the device must be able to reset to standard VGA from high-resolutions. To reset the device sufficiently implies that the device is fully functional as a VGA device. It must be fully functional when programmed via the video BIOS and when programmed directly through standard VGA registers.

The hardware and BIOS must be able to support original VGA mode 12h and VESA modes to allow at least the minimum desktop resolution for Windows, the native resolution of any integrated display and, on external monitors, resolutions of 640 x 480, 800 x 600 and 1024 x 768. All required modes should be supported with at least 16 bits per pixel, though 32 bits per pixel is highly recommended.

Since the 16-bit BIOS is executed within an emulated environment, the BIOS implementation cannot assume that the code is being run natively and therefore must not trigger SMIs in order to perform the requested action. Likewise, since the BIOS is executed while the OS is running, it must not touch hardware outside of the display device.

It is recommended that the BIOS validate the compatibility of all display resolutions it enumerates against the capabilities of the active monitor(s). If a monitor does not support a display timing compatible with a timing available to the BIOS, the resolution must be reported as not supported by hardware configuration by setting D0 of the Mode Attributes when the BIOS is invoked via int 10h function 4f01h for this mode.

If no monitor capabilities are available (missing EDID) for an active on an analog connecter, the BIOS should report at least the above required resolutions as supported by hardware configuration. When the OS attempts to set one of these modes, the BIOS should set the mode using a 60Hz progressive timing for a monitor connector (example HD15) or using the BIOSes default timing for an analog TV connector.

Hybrid Graphics: Not all devices in a Hybrid Graphics system are required to support standard VGA. At least the boot device must support standard VGA and must be able to reset to standard VGA. These requirements exist to ensure basic functionality of the display adapter/chipset in all circumstances.

Design Notes:

The list of VESA modes can be obtained from VESA (www.vesa.org). VGA is defined by IBM spec.

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

Graphics-0013

Device.Graphics.WDDM.Display.HDMIorDPDCNs

Description:

The optional feature implemented by WDDM drivers supporting the audio DCNs over HDMI or DisplayPort.

Related Requirements:

- Device.Graphics.WDDM.Display.HDMIorDPDCNs.DCNCompliance

Device.Graphics.WDDM.Display.HDMIorDPDCNs.DCNCompliance

Target Feature: Device.Graphics.WDDM.Display.HDMIorDPDCNs

Title: Display driver that contains either an HD Audio interface supporting multi-channel HDMI or a DisplayPort audio consistent with HD Audio must comply with HD Audio HDMI & DisplayPort DCNs

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client x64
- Windows 8 Client x86
- Windows 8 Client ARM
- Windows 7 Client x86
- Windows 7 Client x64

Description:

If a display driver is designed for an HDMI or DisplayPort adapter or chipset that contains an HD Audio interface that implements any of the **verbs listed below**, the graphics driver must:

- Comply with the following HD Audio DCNs:
    - HDA034-A2: HDMI Content Protection and Multi-Channel Support
    - HDA039-A: HDMI/ELD Memory Structure

    - HDA036-A: DisplayPort Support and HDMI Miscellaneous Corrections

- Read and parse the EDID from any attached HDMI or DisplayPort sink device and populate the ELD buffer to accurately reflect the hardware and sink device capabilities as required in the HD Audio DCN HDA039-A: HDMI/ELD Memory Structure.
- Program all possible Short Audio Descriptors (SADs) from the EDID into the ELD
- Correctly program the Presence Detect (PD) and ELD Valid (ELDV) bits on the HDMI or DP transmitter hardware for consumption by the audio driver in response to the following events as outlined in the HD Audio DCN HDA034-A2: HDMI Content Protection and Multi-Channel Support.
    - Hot plug events (HDMI/DisplayPort Sink Connected/Disconnected)
        - According to DCN referenced above
    - Video mode changes
        - According to DCN referenced above
    - Graphics power state transitions
        - When the graphics subsystem exits power state D0:
            - If sink is attached, PD = 1, ELDV = 0
            - Otherwise, PD = 0, ELDV = 0
        - When graphics subsystem enters power state D0:
            - If sink is attached, PD = 1, ELDV = 1
            - Otherwise, PD = 0, ELDV = 0
    - Driver load
        - According to DCN referenced above
    - Driver unload
        - PD = 1, ELDV = 1
        - ELD_Version = 1Fh; indicative of basic audio
- Respond to HD Audio-initiated requests for HDCP as outlined in the HD Audio DCN HDA034-A2: HDMI Content Protection and Multi-Channel Support within 10 seconds.
- Ensure that the READY and CES (current encryption state) values in the CP_CONTROL verb accurately reflect the state of the display subsystem, as outlined in the HD Audio DCN HDA034-A2: HDMI Content Protection and Multi-Channel Support.

The Verbs are:

- F2Fh (Get HDMI ELD Data)
- F2Dh (Get Converter Channel Count)
- 72Dh (Set Converter Channel Count)
- F2Eh (Get HDMI Data Island Packet Size Info)
- 72Eh (Set HDMI Data Island Packet Size Info)
- F30h (Get HDMI Data Island Packet Index)
- 730h (Set HDMI Data Island Packet Index)
- F31h (Get HDMI Data Island Packet Data)
- 731h (Set HDMI Data Island Packet Data)
- F32h (Get HDMI Data Island Packet Transmit-Control)
- 732h (Set HDMI Data Island Packet Transmit-3Control)
- F33h (Get Content Protection Control)
- 733h (Set Content Protection Control)
- F34h (Get Converter Channel to HDMI Slot Mapping)
- 734h (Set Converter Channel to HDMI Slot Mapping)

Exceptions: Not Specified

Business Justification:

HDMI audio is heavily dependent on the video driver properly handling the communication of data, parameters and supported formats to the audio driver. This requirement ensures that video drivers perform these functions and enable audio over HDMI.

Scenarios:

Stream audio or video from your PC to your HDMI television or receiver.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New; Win7 Graphics-0073

Device.Graphics.WDDM.Display.TVOut

Description:

If the feature set implemented by drivers which devices contain an SVideo or Composite output

Related Requirements:

- Device.Graphics.WDDM.Display.TVOut.Base
- Device.Graphics.WDDM.Display.TVOut.DAC
- Device.Graphics.WDDM.Display.TVOut.Encoder

Device.Graphics.WDDM.Display.TVOut.Base

Target Feature: Device.Graphics.WDDM.Display.TVOut

Title: TV-out capable display subsystem supports specific timing standards for NTSC and PAL/SECAM regions

Applicable OS Versions:

- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64

Description:

If the display subsystem supports composite or S-video TV-out, it must support NTSC and PAL/SECAM timings appropriate for the region of sale on its other display output interfaces(s) such as DVI or HDMI, as follows:

- NTSC Regions: 720x480, 59-Hz
- PAL/SECAM Regions: 720x576, 50-Hz

The refresh rate of 59.94 Hz is an abbreviated reference used for video. The full refresh rate is 60000/1001 Hz and must be supported as such. Systems equipped with analog three-wire component video output connectors must also implement support for standard definition video timings defined in EIA/CEA-770.2-C and the May 2005 errata to this standard. Support for high-definition analog timings is optional.

Design Notes:

The refresh rate of 59-Hz is an abbreviated reference used for video.

Exceptions: Not Specified

Business Justification:

The TV output from the PC must at least be equal to CE set top boxes available today. Video playback in Media Center will degrade.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

Graphics-0045

Device.Graphics.WDDM.Display.TVOut.DAC

Target Feature: Device.Graphics.WDDM.Display.TVOut

Title: TV out-capable display subsystem connects to a dedicated default DAC and timing generator

Applicable OS Versions:

- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64

Description:

For Video quality purposes, TV output must run at a resolution that is different from the VGA output. A dedicated TV Video subsystem with an independent timing generator must display content at independent resolutions from other video output connectors.

If TV output is supported simultaneously with another output (DVI, CRT, or LVDS), the TV timing must not be affected by the timing requirements of the other outputs and vice-versa, so that a user may run TV-out in a multi-monitor configuration.

Exceptions: Not Specified

Business Justification:

Video playback in Media Center will degrade.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

Graphics-0050

Device.Graphics.WDDM.Display.TVOut.Encoder

Target Feature: Device.Graphics.WDDM.Display.TVOut

Title: TV out-capable display subsystem that supports a TV out encoder as a second head meets general multiple-monitor requirements

Applicable OS Versions:

- Windows 7 Client x86

- Windows 7 Client x64
- Windows Server 2008 Release 2 x64

Description:

The TV-out encoder on the display adapter must be treated as a second or even third independent head.

Exceptions: Not Specified

Business Justification:

TV out must act as a secondary or third head if implemented, and must meet a general multi-mon requirements.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

Graphics-0049

Device.Graphics.WDDM.DisplayRender

Description:

The base feature set implemented by drivers supporting all versions of the WDDM for both Display and Render DDIs

Related Requirements:

- Device.Graphics.WDDM.DisplayRender.Base
- Device.Graphics.WDDM.DisplayRender.OutputProtection
- Device.Graphics.WDDM.DisplayRender.Stability

Device.Graphics.WDDM.DisplayRender.Base

Target Feature: Device.Graphics.WDDM.DisplayRender

Title: Graphics drivers must be implemented per WDDM 1.0 spec

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client x64
- Windows 8 Client x86
- Windows 8 Client ARM
- Windows Server 2008 Release 2 x64

- Windows 7 Client x64
- Windows 7 Client x86

Description:

See requirement **Device.Graphics.WDDM.Base**

Exceptions: Not Specified

Business Justification:

In order for testing to be conducted correctly within the Windows Hardware Certification Kit, this requirement was created to ensure the feature it is associated with is exposed correctly for testing.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New

Device.Graphics.WDDM.DisplayRender.OutputProtection

Target Feature: Device.Graphics.WDDM.DisplayRender

Title: Display adapter supports output connectors with content protection features and provides control via Protected Media Path-Output Protection Manager (PVP-OPM) and Certified Output Protection Protocol (COPP).

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM
- Windows 8 Server x64

Description:

To enable playback of protected DVD content, digital video outputs must support a digital monitor link protection mechanism such as HDCP to obtain Windows8 Client Logo. This requirement is applicable for all Full Graphics devices.

The OPM API and Media Foundation PMP require display drivers to properly implement the OPM DDIs to handle premium content. High grade premium content will not be passed to the display driver unless the display driver has a PVP-OPM certificate. Drivers that support the OPM DDIs must have a driver certificate to testify that the compliance rules, robustness rules, and terms of the PVP-OPM legal agreement, have been met.

The display driver must support both the COPP and PVP-OPM driver interfaces for controlling and signaling video output protection state.

Output protection behaviors are specified by the PVP-OPM compliance rules. The document is available to graphics vendors by request to wmla@microsoft.com.

The WDDM driver must implement the necessary Display Mode Management DDIs and structures as documented in the Windows Driver Kit and the WDDM documentation to enable TV playback. and Analog Content Protection (ACP) support for Media Center.

D3DKMDT_VIDPN_PRESENT_PATH_CONTENT: Media center will use this information to change the TV mode (TV_PLAYBACK vs. WIN_GRAPHICS)

D3DKMDT_VIDPN_PRESENT_PATH_COPYPROTECTION_TYPE and D3DKMDT_VIDPN_PRESENT_PATH_COPYPROTECTION_SUPPORT: These structures are necessary to provide ACP support through the LDDM driver.

S-Video and composite video output interfaces must support the following:

CGMS-A on Line 20 as specified by IEC 61880

CGMS-A on Line 21 as specified by EIA-608-B

Component (YPbPr) outputs must support the following:

CGMS-A with redistribution control as specified by EIA-805

When the TV-out interface is enabled, the display driver must expose a 720x480 60-Hz display mode and/or a 720x576 50-Hz display mode. These display modes must be exposed by the video miniport and added to the default timings list for Windows applications to set when connected to a standard definition TV.

Supports SDTV Modes:

The WDDM miniport driver must set this parameter to TRUE for the video output to expose SDTV modes like NTSC, PAL or SECAM.

Cable Ready systems with CableCARD support with digital video outputs (for example HDMI, or DP) must support a CableLabs approved digital monitor link protection mechanism such as HDCP.

Design Notes:

S-Video and composite video output interfaces may be implemented through the same connector. If a proprietary interface is used, an adapter must be made available either included with the system or available for purchase at point of sale. System or device that uses 7-pin S-Video connectors is not required to provide an adapter so long as the first four pins on the 7-pin connector are electrically compatible with a standard 4-pin S-Video connector. Microsoft recommends including SCART output when appropriate for the region of sale.

Exceptions: Not Specified

Business Justification:

Content providers will not allow use of protected content without engaging video output link protections per the compliance rules in force. For example, protected DVD or Blu-Ray playback will usually require activating HDCP on HDMI, or DP connectors.

Scenarios:

User acquires protected content from the Internet, and displays on a monitor or TV User records protected broadcast TV content, and displays on a monitor or TV User plays protected content from a DVD or Blu-Ray disc to a monitor or TV

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New; Win7 Graphics-0001

Device.Graphics.WDDM.DisplayRender.Stability

Target Feature: Device.Graphics.WDDM.DisplayRender

Title: All WDDM graphics drivers must not generate any hangs or faults under prolonged stress conditions.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86

Description:

Graphics drivers must function properly, and not generate any hangs or faults throughout the duration of stress testing as specified in the

- "4-hour WDDM Profile"

To "stress" a graphics driver, Comparative Reliability Analyzer for Software and Hardware (CRASH) launches and terminates other test applications to simulate real-world scenarios.

Exceptions: Not Specified

Business Justification:

The CRASH tests which are exercised through this logo requirement ensure reliability and stability of the graphics driver on the Windows platform. These tests which simulate various productivity, graphics and gaming scenarios exercise various code paths in the graphics stack. They also stress the system to ensure that the stability is not affected with an increased workload.

Scenarios:

The CRASH tests exercise the following scenarios in a stress environment. It ensures that these common end-user scenarios work under stress conditions without affecting the end-user experience:Productivity Desktop Graphics Gaming

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New; Win7 Graphics-0023

Device.Graphics.WDDM.DisplayRender.OutputProtection

Description:

The base optional WDDM feature set for Windows 7 containing requirements for OPM

Related Requirements:

- Device.Graphics.WDDM.DisplayRender.OutputProtection.Windows7

Device.Graphics.WDDM.DisplayRender.OutputProtection.Windows7

Target Feature: Device.Graphics.WDDM.DisplayRender.OutputProtection

Title: Display adapter supports output connectors with content protection features and provides control via Protected Media Path-Output Protection Manager (PVP-OPM) and Certified Output Protection Protocol (COPP).

Applicable OS Versions:

- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64

Description:

To enable playback of protected DVD content, digital video outputs must support a digital monitor link protection mechanism such as HDCP to obtain Windows 7Logo. This is an if implemented feature.

The OPM API and Media Foundation PMP require display drivers to properly implement the OPM DDIs to handle premium content. High grade premium content will not be passed to the display

driver unless the display driver has a PVP-OPM certificate. Drivers that support the OPM DDIs must have a driver certificate to testify that the compliance rules, robustness rules, and terms of the PVP-OPM legal agreement, have been met.

The display driver must support both the COPP and PVP-OPM driver interfaces for controlling and signaling video output protection state.

Output protection behaviors are specified by the PVP-OPM compliance rules. The document is available to graphics vendors by request to wmla@microsoft.com.

The WDDM driver must implement the necessary Display Mode Management DDIs and structures as documented in the Windows Driver Kit and the WDDM documentation to enable TV playback. and Analog Content Protection (ACP) support for Media Center.

The WDDM driver must implement the necessary Display Mode Management DDIs and structures as documented in the Windows Driver Kit and the WDDM documentation to enable TV playback and Analog Content Protection (ACP) support for Media Center.

D3DKMDT_VIDPN_PRESENT_PATH_CONTENT: Media center will use this information to change the TV mode (TV_PLAYBACK vs. WIN_GRAPHICS)

D3DKMDT_VIDPN_PRESENT_PATH_COPYPROTECTION_TYPE and D3DKMDT_VIDPN_PRESENT_PATH_COPYPROTECTION_SUPPORT: These structures are necessary to provide ACP support through the LDDM driver.

S-Video and composite video output interfaces must support the following:

CGMS-A on Line 20 as specified by IEC 61880

CGMS-A on Line 21 as specified by EIA-608-B

Component (YPbPr) outputs must support the following:

CGMS-A with redistribution control as specified by EIA-805

When the TV-out interface is enabled, the display driver must expose a 720x480 60-Hz display mode and/or a 720x576 50-Hz display mode. These display modes must be exposed by the video miniport and added to the default timings list for Windows applications to set when connected to a standard definition TV.

Supports SDTV Modes:

The WDDM miniport driver must set this parameter to TRUE for the video output to expose SDTV modes like NTSC, PAL or SECAM.

Cable Ready systems with CableCARD support with digital video outputs (for example HDMI, or DP) must support a CableLabs approved digital monitor link protection mechanism such as HDCP.

Design Notes:

S-Video and composite video output interfaces may be implemented through the same connector. If a proprietary interface is used, an adapter must be made available either included with the system or available for purchase at point of sale. System or device that uses 7-pin S-Video connectors is not required to provide an adapter so long as the first four pins on the 7-pin connector are electrically compatible with a standard 4-pin S-Video connector. Microsoft recommends including SCART output when appropriate for the region of sale.

Exceptions: Not Specified

Business Justification:

Content providers will not allow use of protected content without engaging video output link protections per the compliance rules in force. For example, protected DVD or Blu-Ray playback will usually require activating HDCP on HDMI, or DP connectors.

Scenarios:

User acquires protected content from the Internet, and displays on a monitor or TV User records protected broadcast TV content, and displays on a monitor or TV User plays protected content from a DVD or Blu-Ray disc to a monitor or TV

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New; Win7 Graphics-0001

Device.Graphics.WDDM.Render

Description:

The base feature set implemented by drivers supporting all versions of the WDDM Render DDIs

Related Requirements:

- Device.Graphics.WDDM.Render.Base
- Device.Graphics.WDDM.Render.VideoDecoding
- Device.Graphics.WDDM.Render.VideoProcessing
- Device.Graphics.WDDM.Render.Windows7.VideoDecoding

Device.Graphics.WDDM.Render.Base

Target Feature: Device.Graphics.WDDM.Render

Title: Graphics drivers must be implemented per WDDM 1.0 spec

Applicable OS Versions:

- Windows 8 Server x64

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 8 Client ARM
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86

Description:

See requirement **Device.Graphics.WDDM.Base**

Exceptions: Not Specified

Business Justification:

In order for testing to be conducted correctly within theWindows Hardware Certificationkit, this requirement was created to ensure the feature it is associated with is exposed correctly for testing.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New

Device.Graphics.WDDM.Render.VideoDecoding

Target Feature: Device.Graphics.WDDM.Render

Title: Display driver supports the DirectX VA 2.0 Video Decoder DDI

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86

Description:

Display WDDM drivers must support the DXVA 2.0 Video Decoder DDI.

WDDM drivers must support at least one of the following sets of MPEG2 GUIDs:

- DXVA2_ModeMPEG2_VLD and DXVA2_ModeMPEG2_iDCT

- DXVA2_ModeMPEG2_VLD and DXVA2_ModeMPEG2_MoComp
- DXVA2_ModeMPEG2_iDCT
- DXVA2_ModeMPEG2and1_VLD

And at least one of the H.264 GUIDs:

- DXVA_ModeH264_MoComp
- DXVA_ModeH264_VLD

In addition, WDDM drivers must support at least one of the following VC1 modes:

- DXVA2_ModeVC1_B (Mo-Comp + Post-Proc**)
- DXVA2_ModeVC1_C (iDCT + Mo-Comp+ Post-Proc**)
- DXVA2_ModeVC1_D (VLD + IDCT + Mo-Comp + Post-Proc**)

If the display adapter supports hardware-accelerated decode of H.264, it must support either the DXVA_ModeH264_MoComp GUID or the DXVA_ModeH264_VLD GUID.

Finally, WDDM drivers must support Standardized AES 128 for both MPEG2 and H.264.

** Note that it is not a requirement to implement the "Post-Proc" stage on DXVA2_ModeVC1 GUIDs.

Design Notes

MPEG-2 support is required on x86 and x64 architectures and operating systems only.

Exceptions: Not Specified

Business Justification:

Requiring video decode support on all Windows system ensures that consumers can rely upon Windows as a platform for viewing high-quality video content.

Scenarios:

The end-user wishes to watch video content in the most common video encoding formats: MPEG2, H.264, and VC1 (Blu-ray). Media Foundation as well as a number of 3rd party applications utilize DXVA2 for playback of these formats.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New; Win7 Graphics-0025

Device.Graphics.WDDM.Render.VideoProcessing

Target Feature: Device.Graphics.WDDM.Render

Title: Display WDDM driver supports the DirectX VA 2.0 Video Processor DDI

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64

Description:

Display WDDM drivers must support the DXVA 2.0 Video Processor DDI and implement support for the following Video Processor Device GUIDs:

- DXVADDI_VideoProcProgressiveDevice
- DXVADDI_VideoProcBobDevice

The following DXVA2 video processor caps must be supported for each of the video processor device GUIDs:

- DXVADDI_VIDEOPROCESS_YUV2RGB
- DXVADDI_VIDEOPROCESS_YUV2RGBEXTENDED
- DXVADDI_VIDEOPROCESS_STRETCHX
- DXVADDI_VIDEOPROCESS_STRETCHY
- DXVADDI_VIDEOPROCESS_SUBRECTS
- DXVADDI_VIDEOPROCESS_SUBSTREAMS
- DXVA2_VideoProcess_SubStreamsExtended
- DXVA2_VideoProcess_Constriction

In addition, to support DXVADDI_VIDEOPROCESS_YUV2RGBEXTENDED caps, the following color parameters must be supported when color-space converting from a YUV surface to a RGB surface:

- D3DDDIARG_VIDEOPROCESSBLT.DestFormat.NominalRange
    - DXVADDI_NominalRange_Unknown (should be interpreted as 0_255 if a sophisticated algorithm is not implemented)
    - DXVADDI_NominalRange_0_255

    - DXVADDI_NominalRange_16_235

- D3DDDIARG_VIDEOPROCESSBLT.pSrcSurfaces[].SampleFormat.VideoTransferMatrix
    - DXVADDI_VideoTransferMatrix_Unknown (should be interpreted as BT601 unless the source surface is greater than 576 height in which case should be interpreted as BT709)
    - DXVADDI_VideoTransferMatrix_BT709
    - DXVADDI_VideoTransferMatrix_BT601

The following YUV formats must be supported as the video stream:

- YUY2 - 8-bit packed 4:2:2
- NV12 - 8-bit planar 4:2:0

If the video processor supports 10-bit YUV formats, the following formats must be supported:

- Y210 - 10-bit packed 4:2:2
- Y410 - 10-bit packed 4:4:4
- P210 - 10-bit planar 4:2:2
- P010 - 10-bit planar 4:2:0

If the video processor supports 16-bit YUV formats, the following formats must be supported:

- Y216 - 16-bit packed 4:2:2
- Y416 - 16-bit packed 4:4:4
- P216 - 16-bit planar 4:2:2
- P016 - 16-bit planar 4:2:0

The following YUV format must be supported as the video sub-streams:

- AYUV - 8-bit packed 4:4:4

For these formats, color converting YUV-to-RGB blits executed through the VideoProcessBlt function must at least be able to utilize BT. 601 and BT. 709 conversion matrices. This process allows the graphics to switch between the YUV-to-RGB matrix transforms for different color formats, to ensure proper handling of video that originates from different standard color spaces such as those defined in ITU-R Recommendations BT. 601 and BT.709, is required.

Design Notes

MPEG-2 support is required on x86 and x64 architectures and operating systems only.

Exceptions: Not Specified

Business Justification:

Basic functionality of defined in the most common video specifications (e.g. H.264, Blu-ray) require conversion of all of the specified YUV formats to RGB data for display of video. In order to enable full support for these encoding formats, all WDDM hardware must support these conversion capabilities. High quality color conversion ensures comparable video quality can be obtained for video to competitive platforms.

Scenarios:

The end-user wishes to watch video content in the most common video encoding formats: MPEG2, H.264, and VC1 (Blu-ray). Media Foundation as well as a number of 3rd party application utilize DXVA2 for playback of these formats.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New; Win7 Graphics-0032

Device.Graphics.WDDM.Render.Windows7.VideoDecoding

Target Feature: Device.Graphics.WDDM.Render

Title: Display driver supports the DirectX VA 2.0 Video Decoder DDI

Applicable OS Versions:

- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64

Description:

For both Windows Vista Premium Logo and Windows 7 Logo, display WDDM drivers must support the DXVA 2.0 Video Decoder DDI.

WDDM drivers must support at least the following GUIDs:

((DXVA2_ModeMPEG2_VLD and DXVA2_ModeMPEG2_iDCT) or

(DXVA2_ModeMPEG2_VLD and DXVA2_ModeMPEG2_MoComp) or

(DXVA2_ModeMPEG2_iDCT)) and

(DXVA2_ModeVC1_B (Mo-Comp + Post-Proc**) or

DXVA2_ModeVC1_C (iDCT + Mo-Comp+ Post-Proc**)or

DXVA2_ModeVC1_D (VLD + IDCT + Mo-Comp + Post-Proc**)) and

If the display adapter supports hardware-accellerated decode of H.264, it must support the following GUID

(DXVA_ModeH264_MoComp or DXVA_ModeH264_VLD)

** Note that it is not a requirement to implement the "Post-Proc" stage on DXVA2_ModeVC1 GUIDs.

Exceptions:

This requirement is if-implemented for Windows Server 2008 Release 2 x64

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Graphics.WDDM11

Description:

The base feature set implemented by drivers supporting WDDM 1.1

Related Requirements:

- Device.Graphics.WDDM11.Base

Device.Graphics.WDDM11.Base

Target Feature: Device.Graphics.WDDM11

Title: Graphic drivers written for a discrete graphic adapters or an integrated graphics adapters devices must meet all requirements defined in the WDDM 1.1 specifications.

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client x64
- Windows 8 Client x86
- Windows 8 Client ARM
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64

Description:

WDDM 1.1 introduces the following key requirements over WDDM 1.0:

- Hardware acceleration of select GDI features.
- Implementation of DMM DDIs for Connecting and Configuring Displays
- Support for 32-bit BGRA pixel format compatible with GDI. (Through DirectX10 and later DDIs.)
- Provide additional information to aid in debugging VSync TDRs.
- Kernel mode drivers must be compiled with Frame Pointer Optimizations (FPO) disabled.
- Optional features, if implemented, must be incorporated according to the specifications and WDK documentation, including Standardized AES128, DXVA-HD, overlays, and Direct3D11.

MSDN documentation is updated based on the WDDM 1.1 Specification. Pleaseconsult MSDN documentation for WDDM 1.1 requirements.

Exceptions: Not Specified

Business Justification:

Key Benefits from the newly introduced WDDM 1.1 Features include:Reduced memory usage compared with WDDM1.0 drivers. GDI Hardware acceleration allows resources to be consolidated into video memory. The need for a separate system memory copy for GDI use is reduced. Improved user experience in connecting Monitors, TVs, LCD displays and Projectors Support for 32-bit BGRA pixel formats through Direct3D10 (and later) allows for interop with GDI. Disabling FPO in kernel mode driver and new VSYNC TDR information improves debug gability and ultimately improves driver quality. Optional features introduced with WDDM1.1 provide the following:AES128 Standardized mechanism for encrypting video as it passes from CPU to GPU over the PCI (or other) bus. DXVA-HD Offloads certain high definition video processing features to the GPU. Overlays Exposes overlay hardware in a consistent way. Improves security and performance for video player applications compared with tradition presentation BLT methods. DirectX11 Added support for compute devices and multi-threaded app support.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New

Device.Graphics.WDDM11.Display

Description:

The base feature set implemented by drivers supporting all versions of the WDDM Display DDIs

Related Requirements:

- Device.Graphics.WDDM11.Display.Base

Device.Graphics.WDDM11.Display.Base

Target Feature: Device.Graphics.WDDM11.Display

Title: Graphic drivers written for a discrete graphic adapters or an integrated graphics adapters devices must meet all requirements defined in the WDDM 1.1 specifications.

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client x64
- Windows 8 Client x86
- Windows 8 Client ARM
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64

Description:

See requirement **Device.Graphics.WDDM11.Base**

Exceptions: Not Specified

Business Justification:

In order for testing to be conducted correctly within the WLK2.0 kit, this requirement was created to ensure the feature it is associated with is exposed correctly for testing.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New

Device.Graphics.WDDM11.DisplayRender

Description:

The base feature set implemented by drivers supporting all versions of the WDDM for both Display and Render DDIs

Related Requirements:

- Device.Graphics.WDDM11.DisplayRender.Base

Device.Graphics.WDDM11.DisplayRender.Base

Target Feature: Device.Graphics.WDDM11.DisplayRender

Title: Graphic drivers written for a discrete graphic adapters or an integrated graphics adapters devices must meet all requirements defined in the WDDM 1.1 specifications.

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client x64
- Windows 8 Client x86
- Windows 8 Client ARM
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86

Description:

See requirement **Device.Graphics.WDDM11.Base**

Exceptions: Not Specified

Business Justification:

In order for testing to be conducted correctly within the Windows Hardware Certification kit, this requirement was created to ensure the feature it is associated with is exposed correctly for testing.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New

Device.Graphics.WDDM11.DisplayRender.D3D9Overlay

Description:

The optional feature implemented by WDDM 1.1 drivers and greater allowing for surfaces to present in a hardware overlay.

Related Requirements:

- Device.Graphics.WDDM11.DisplayRender.D3D9Overlay.D3D9Overlay

Device.Graphics.WDDM11.DisplayRender.D3D9Overlay.D3D9Overlay

Target Feature: Device.Graphics.WDDM11.DisplayRender.D3D9Overlay

Title: WDDM1.1 driver supports Direct3D 9 Overlays

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86

Description:

If the WDDM1.1 driver supports Direct3D 9 Overlays the video overlay presentation requirements are as follows:

1. A WDDM v1.1 driver must set the D3DCAPS_OVERLAY bit in the D3DCaps9.Caps field.
2. A WDDM v1.1 driver must support query type D3DDDICAPS_CHECKOVERLAYSUPPORT to the user mode pfnGetCaps DDI call.
3. A WDDM v1.1 driver must support overlays in at least one valid configuration (Displaymode, OverlayFormat, Width, and Height) when called to DDICHECKOVERLAYSUPPORTDATA for supported overlay and the Max width and height of supported overlay must be greater than zero.

Exceptions: Not Specified

Business Justification:

The WDDM v1.1 overlay DDI enables Direct3D9 based applications to use video overlays while running Aero Glass (DWM On). The feature improves upon the security model for video overlay presentation as well as performance by eliminating per frame composition.

Scenarios:

A user wishes to pay video using an existing Windows 7 or Windows Vista era video application. The user can continue to enjoy protected media in the application as the overlay feature used to ensure copy protection is available in Windows 8.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Graphics.WDDM11.Render

Description:

The base feature set implemented by drivers supporting all versions of the WDDM Render DDIs

Related Requirements:

- Device.Graphics.WDDM11.Render.Base

Device.Graphics.WDDM11.Render.Base

Target Feature: Device.Graphics.WDDM11.Render

Title: Graphic drivers written for a discrete graphic adapter or an integrated graphics adapter device must meet all requirements defined in the WDDM 1.1 specifications.

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client x64
- Windows 8 Client x86
- Windows 8 Client ARM
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86

Description:

See requirement **Device.Graphics.WDDM11.Base**

Exceptions: Not Specified

Business Justification:

In order for testing to be conducted correctly within the WLK2.0 kit, this requirement was created to ensure the feature it is associated with is exposed correctly for testing.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New

Device.Graphics.WDDM11.Render.ContentProtection

Description:

The optional feature implemented by WDDM 1.1 drivers allowing for D3D9 surfaces to be protected.

Related Requirements:

- Device.Graphics.WDDM11.Render.ContentProtection.ContentProtection

Device.Graphics.WDDM11.Render.ContentProtection.ContentProtection

Target Feature: Device.Graphics.WDDM11.Render.ContentProtection

Title: WDDM1.1 Driver supports GPU-based Content Protection

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86

Description:

If the WDDM1.1 driver supports GPU based content protection it must support the following features:

- Encryption BLT -- Video to System BLT with encryption
- Driver Protections of Shared Surfaces
- Encryption on Eviction Encryption of video memory resources when evicted from video memory.
- Decryption BLTSystem to Video BLT with decryption

For more details on each of these features, please refer to the Windows WDK documentation.

Exceptions: Not Specified

Business Justification:

This WDDM v1.1 content protection DDI enables Windows Media Center as well as 3rd party video players in windowed mode with DWM on. The feature support scales from driver/software to driver/hardware protections.

Scenarios:

A user wishes to play video using an existing Windows 7 or Windows Vista era video application. The user can continue to enjoy protected media in the application as the content protection DDI used to ensure copy protection is available in Windows 8.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Graphics.WDDM11.Render.DXVAHD

Description:

The optional feature implemented by WDDM 1.1 drivers supporting the new state based video processing DDIs.

Related Requirements:

- Device.Graphics.WDDM11.Render.DXVAHD.DXVAHD

Device.Graphics.WDDM11.Render.DXVAHD.DXVAHD

Target Feature: Device.Graphics.WDDM11.Render.DXVAHD

Title: WDDM1.1 driver supports DXVA-HD

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86

Description:

If the WDDM1.1 driver supports DXVA-HD then the following input formats (D3DDDICAPS_DXVAHD_GETVPINPUTFORMATS) must be supported:

1. YUY2
2. AYUV
3. NV12
4. YV12
5. X8R8G8B8

6. A8R8G8B8

Also the driver must support the following output formats (D3DDDICAPS_DXVAHD_GETVPOUTPUTFORMATS)

1. X8R8G8B8
2. A8R8G8B8

Exceptions: Not Specified

Business Justification:

The WDDM v1.1 DXVA-HD requirements ensure that processing of the common video formats is enabled for all existing applications that rely on DXVA and DXVA-HD.

Scenarios:

A user wishes to pay video using an existing Windows 7 or Windows Vista era video application. The user can continue to enjoy media in the application as DXVA-HD is available for the most common viewing formats.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New; Win7 Graphics-0032

Device.Graphics.WDDM12

Description:

The base feature set implemented by drivers supporting WDDM 1.2.

Related Requirements:

- Device.Graphics.WDDM12.Base

Device.Graphics.WDDM12.Base

Target Feature: Device.Graphics.WDDM12

Title: Graphics drivers must be implemented per WDDM 1.2 spec

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client x64
- Windows 8 Client x86
- Windows 8 Client ARM

Description:

Windows 8 also introduces features and capabilities that require graphics driver changes. These incremental changes range from small changes such as smooth rotation, to large changes such as 3D stereo, and D3D11 video support. The WDDM driver model that provides these Windows 8 features is referred to as "WDDM v1.2" WDDM v1.2 is a superset of WDDM 1.1, and WDDM 1.0.

WDDM v1.2 is required by all systems shipped with Windows 8. WDDM 1.0 and WDDM 1.1 will only be supported with legacy devices on legacy systems. The best experience, and Windows 8 specific features are only enabled by a WDDM 1.2 driver. A WDDM driver that implements some WDDM 1.2 required features, but not all required features will fail to load on Windows 8.

For Windows 8 XDDM is officially retired and XDDM drivers will no longer load on Windows 8 Client or Server.

Below is a summary these WDDM versions:

Operating System	Driver Models Supported	D3D versions supported	Features enabled
Windows Vista	WDDM 1.0 XDDM on Server and limited UMPC	D3D9, D3D10	Scheduling, Memory Management, Fault tolerance, D3D9 & 10
Windows Vista SP1 / Windows 7 client pack	WDDM 1.05 XDDM on Server 2008	D3D9, D3D10, D3D10.1	+ BGRA support in D3D10, D3D 10.1
Windows 7	WDDM 1.1 XDDM on Server 2008 R2	D3D9, D3D10, D3D10.1, D3D11	GDI Hardware acceleration, Connecting and configuring Displays, DXVA HD, D3D11
Windows 8	WDDM 1.2	D3D9, D3D10, D3D10.1, D3D11, D3D11.1	Smooth Rotation, 3D Stereo, D3D11 Video, GPU Preemption, TDR Improvements Diagnostic Improvements, Performance and Memory usage Optimizations, Power Management, etc.

WDDM v1.2 also introduces new types of graphics drivers, targeting specific scenarios and is described below:

a. WDDM Full Graphics Driver: This is the full version of the WDDM graphics driver that supports hardware accelerated 2D & 3D operations. This driver is fully capable of handling all the render, display and video functions. WDDM 1.0 and WDDM 1.1 are full graphics drivers. All Windows 8 client systems must have a full graphics WDDM 1.2 device as the primary boot device.

b. WDDM Display Only Driver: This driver is only supported as a WDDM 1.2 driver and enables IHVs to write a WDDM based kernel mode driver that is capable of driving display only devices. The OS handles the 2D or 3D rendering using a software simulated GPU.

c. WDDM Render Only Driver: This driver is only supported as a WDDM 1.2 driver and enables IHVs to write a WDDM driver that supports only rendering functionality. Render only devices are not allowed as the primary graphics device on client systems.

Table below explains the scenario usage for the new driver types:

	Client	**Server**	**Client running in a Virtual Environment**	**Server Virtual**
Full Graphics	Required as boot device	Optional	Optional	Optional
Display Only	Not allowed	Optional	Optional	Optional
Render Only	Optional as non primary adapter	Optional	Optional	Optional
Headless	Not allowed	Optional	N/A	N/A

WDDM v1.2 Feature caps

Table below lists the requirements for a WDDM v1.2 driver to specify to Windows the WDDM Driver Type, version and the feature caps (visible to dxgkrnl) that WDDM v1.2 drivers are required to set. If a driver has wrongfully claimed itself as WDDM v1.2 or has implemented partial features (only some of the mandatory features), then it will fail to create an adapter and the system will fall back to the Microsoft Basic Display Driver.

WDDM Driver Requirements

WDDM driver type	**DDI requirements**
Full Graphics	Implement all the Render-specific and the Display-specific required DDIs
Display-Only	Implement all the Display-specific DDIs and return a null pointer for all the Render-specific DDIs

Render-Only	Implement all the Render-specific DDIs and return a null pointer for all the Display-specific DDIs **OR** Implement all the DDIs for a full WDDM driver but report DISPLAY_ADAPTER_INFO::NumVidPnSources = 0 and DISPLAY_ADAPTER_INFO::NumVidPnTargets = 0

WDDM v1.2 Feature Caps

Feature	WDDM Driver Type			Feature Caps
	Full Graphics	**Render Only**	**Display Only**	
WDDM version	**M**	**M**	**M**	DXGK_DRIVERCAPS::WDDMVersion
Bugcheck and PnP Stop support for Non VGA	**M**	**NA**	**M**	DXGK_DRIVERCAPS::SupportNonVGA
Optimized screen rotation Support	**M**	**NA**	**M**	DXGK_DRIVERCAPS::SupportSmoothRotation
GPU Preemption	**M**	**M**	**NA**	DXGK_DRIVERCAPS::PreemptionCaps
FlipOnVSyncMmIo	**M**	**M**	**NA**	DXGK_FLIPCAPS::FlipOnVSyncMmIo FlipOnVSyncMmIo is NOT a new feature; this is already documented and has been available since Windows Vista; the requirement here is to set FlipOnVSyncMmIo cap
TDR Improvements	**M**	**M**	**NA**	DXGK_DRIVERCAPS::SupportPerEngineTDR
Optimizing the graphics stack to improve performance on sleep & resume	**O**	**O**	**NA**	DXGK_SEGMENTDESCRIPTOR3::Flags
Stereoscopic 3D: New infrastructure to process and present stereoscopic content	**O**	**NA**	**NA**	D3DKMDT_VIDPN_SOURCE_MODE_TYPE
DirectFlip	**M**	**NA**	**NA**	DXGK_DRIVERCAPS::SupportDirectFlip
GDI Hardware acceleration (This is a required WDDM v1.1 feature)	**M**	**M**	**NA**	DXGK_PRESENTATIONCAPS::SupportKernelModeCommandBuffer
GPU power management of idle and active	**O**	**O**	**O**	If this feature is supported the DDI functions must be supported (SetPowerComponentFState and

power	PowerRuntimeControlRequest)

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New; old id Gfx8078

Device.Graphics.WDDM12.Display

Description:

Display feature requirements for all WDDM12 drivers which support the display specific DDIs.

Related Requirements:

- Device.Graphics.WDDM12.Display.Base
- Device.Graphics.WDDM12.Display.ContainerIDSupport
- Device.Graphics.WDDM12.Display.DisplayOutputControl
- Device.Graphics.WDDM12.Display.ModeEnumeration
- Device.Graphics.WDDM12.Display.PnpStopStartSupport
- Device.Graphics.WDDM12.Display.ProvideLinearFrameBuffer

Device.Graphics.WDDM12.Display.Base

Target Feature: Device.Graphics.WDDM12.Display

Title: Requirements for a WDDM graphics adapter to support display functionality

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client ARM
- Windows 8 Client x86
- Windows 8 Client x64

Description:

In Windows 8 and beyond WDDM has been extended to support a WDDM driver that is only responsible for Display Scan out capabilities. Such a driver is not allowed to support any Rendering capabilities.

A driver is considered a WDDM Display Only driver if it implements the following DDIs

Common WDDM DDIs

These DDIs are for the common device functionalities such as PnP support and Power support. These functions are required by all WDDM drivers and if not implemented the driver will not be started by Windows. These DDIs are already documented in the WDK.

Required:

DxgkDdiAddDevice

DxgkDdiStartDevice

DxgkDdiStopDevice

DxgkDdiRemoveDevice

DxgkDdiDispatchIoRequest

DxgkDdiSetPowerState

DxgkDdiUnload

Optional:

DxgkDdiInterruptRoutine*

DxgkDdiDpcRoutine

DxgkDdiNotifyAcpiEvent

DxgkDdiQueryInterface

DxgkDdiControlEtwLogging

DxgkDdiEscape

DxgkDdiCollectDbgInfo

*DxgkDdiInterruptRoutine function is required if current hardware device reports hardware interrupt. In this case , if driver did not supply this DDI function, OS would fail the initialization. If current hardware does not have interrupt and driver supplies this DDI function, OS will still allow this driver to be loaded and DxgkDdiInterruptRoutine will never be called.

* DdiNotifyAcpiEvent DDI function is used to notify graphics drivers on some ACPI events. It is optional for rendering device. On normal WDDM graphics devices, this DDI function will return a flag to indicate dxgkrnl.sys to take some further actions such as reset display mode or poll the connected monitors. For the rendering only device, these flags are not used and must be set to zero

Display Only DDIs

Following DDI functions are required to be implemented by a Display Only driver. If the driver does not supply all of these DDIs, Windows will fail to initialize this driver.

DxgkDdiQueryChildRelations

DxgkDdiQueryChildStatus

DxgkDdiQueryDeviceDescriptor

DxgkDdiResetDevice

DxgkDdiQueryAdapterInfo

DxgkDdiSetPalette *

DxgkDdiSetPointerPosition **

DxgkDdiSetPointerShape **

DxgkDdiIsSupportedVidPn

DxgkDdiRecommendFunctionalVidPn ***

DxgkDdiEnumVidPnCofuncModality

DxgkDdiSetVidPnSourceVisibility

DxgkDdiCommitVidPn

DxgkDdiUpdateActiveVidPnPresentPath

DxgkDdiRecommendMonitorModes

DxgkDdiGetScanLine

DxgkDdiQueryVidPnHWCapability

DxgkDdiPresentDisplayOnly

DxgkDdiReleasePostDisplayOwnership

DxgkDdiSystemDisplayEnable

DxgkDdiSystemDisplayWrite

DxgkDdiI2CReceiveDataFromDisplay

DxgkDdiI2CTransmitDataFromDisplay

*DxgkDdiSetPalette is a required DDI. If driver does not support any palette mode, it should still supply this DDI.

**DxgkDdiSetPointerPosition and DxgkDdiSetPointerShape are required DDIs. If driver does not support any hardware cursor, it should still supply these two DDIs.

***DxgkDdiRecommendFunctionalVidPn is a required DDI. If driver does not support any ACPI event, it should still supply this DDI.

Exceptions: Not Specified

Business Justification:

Microsoft would like Windows to provide a Modern User Experience on all its SKUs. To achieve this, various parts of the Windows UI and APIs are dependent on the WDDM. Therefore it is important that WDDM is always available. If WDDM and its APIs are always available, then the ISVs can focus on one API set. Currently many server system ship with XDDM hardware. Such hardware does not support the WDDM. The WDDM Display Only driver was designed especially to enable partners to write WDDM drivers for such hardware to provide the benefits of WDDM to the user.

Scenarios:

User installs Windows 8 Server on a system that has an XDDM graphics hardware Windows does not have any XDDM graphics driver in the box, so the Microsoft Basic Display Only Driver (aka Standard VGA driver) is automatically installed. This is a WDDM driver and the user gets a Modern Windows UX along with ability to use the WDDM API set The user installs a WDDM Display Only Driver provided by the IHVAt this time key WDDM features are made available on this system Full D3D and D2D support Support for Aero Glass Multi monitor support Hot plug detect

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New; Old ID Gfx8087

Device.Graphics.WDDM12.Display.ContainerIDSupport

Target Feature: Device.Graphics.WDDM12.Display

Title: Graphics Adapter must support the DDI for Container ID

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client ARM
- Windows 8 Client x64
- Windows 8 Client x86

Description:

A graphics adapter must implement the **DxgkDdiGetDeviceContainer** DDI. Windows will use this DDI to ask the driver for the container ID. The driver must try and obtain the Container ID from the display hardware.

In case the Display device does not provide a Container ID, then Windows will automatically manufacture a Container ID for the display. The uniqueness of the Container ID is achieved by using the Manufacture ID, Product ID and Serial number of the display obtained from the EDID. Using this information, a driver for another device can generate the same container ID as the display device by using the RtlGenerateClass5Guid function included in wdm.h.

Exceptions: Not Specified

Business Justification:

During Windows 7 the concept of Container ID was introduced for devices. Per MSDN, A container ID is a system-supplied device identification string that uniquely groups the functional devices associated with a single-function or multifunction device installed in the computer. Starting in Windows7, the container ID is used to visually group and represent different devices that are physically related to each other. By supporting Container ID for display devices, it would be possible to link other functions of a Display like Audio, USB etc with the Display device. The container ID of any device can be obtained using the IoGetDevicePropertyData API.

Scenarios:

Input devices User has a USB hub in his monitor User connects a keyboard and mouse to the monitor User opens the Device Hub and sees his monitor If the user double clicks on the monitor it will show him the device stage page which will indicate the presence of other devices like mouse and keyboard connected to it Audio devices User has 2 monitors connected with HDMI cable User has speakers connected to one of the monitors User opens the Device Hub and sees his monitors User double clicks on the monitor and it will show him the device stage page which will indicate the presence of the speakers along with tasks he can perform on that device Digitizers User has 2 touch capable monitors connected to the PC User opens the Device Hub and sees his monitors User double clicks on the monitor and it will show him the device stage page which will indicate the presence of touch capabilities for the monitor like Calibration etc.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New; Old ID Gfx8092

Device.Graphics.WDDM12.Display.DisplayOutputControl

Target Feature: Device.Graphics.WDDM12.Display

Title: Support for WDDM taking control of Display output while WDDM driver is running

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client ARM
- Windows 8 Client x64

- Windows 8 Client x86

Description:

In Windows 8 and beyond, there is a need for a more seamless hand off of the display control between Windows and the WDDM graphics driver. This means that in some cases Windows needs to take over the control of the Display scan out without having to PnP Stop the WDDM Driver.

One such scenario is when Windows needs to bug check the system and display the blue screen. This set of DDIs enables that cleaner hand off.

The following DDIs are required to be implemented by a Full and Display Only WDDM 1.2 driver

DxgkDdiSystemDisplayEnable

DxgkDdiSystemDisplayWrite

These DDIs are documented here on WDK.

The following are the requirements for WDDM driver while implementing these DDIs:

1. When Windows calls DxgkDdiSystemDisplayEnable, driver must cancel all the GPU operations or reset GPU to an idle state
2. Windows will provide a Target ID as part of the DDI call. The driver must continue to drive the display associated with the target ID
    a. The driver must check the connectivity of the display on the provided Target ID. If specified target does not have a display connected, driver should fail this call with STATUS_NOT_SUPPORTED.
    b. The driver must disable the signal to all other displays connected to the adapter except the target ID provided.
        i. If this is not possible, driver should try and put a blank image on all other displays
        ii. If this is not possible, driver must leave the last image on the screen unchanged
3. For the selected Target ID, the driver must keep the current display mode and provide this mode back to Windows as part of the DDI call
    a. In case the driver is not able to maintain the current mode OR the target ID is not part of the active topology, the driver should try and set the native resolution of the display OR a high res mode. At the very least the driver must reset to a mode which is larger than or equals to 640x480 24bpp color format on the specified target.
    b. It is NOT required that driver should use linear frame buffer mode. But driver should support the write operation from a D3DDDIFMT_A8R8G8B8 source to this frame buffer.

4. This function might be called at high IRQL level or after system has been bugcheck. Driver should put this function in non-paged code section and only use non-paged memory.
    a. Windows kernel mode functions might be also NOT available when this function is called.
5. Once the driver has handed over Display Control to Windows, Windows will use the DxgkDdiSystemDisplayWrite DDI to update the screen image. Windows will use the DDI to write a block of image from specified source image to the screen which is reset via DxgkDdiSystemDisplayEnable function. This function will pass the driver the start address of source image as well as the stride, width, and height. The color format of source image is always D3DDDIFMT_X8R8G8B8. Windows guarantees that the source image is in non-paged memory. Driver must write this source image to current screen at (PostionX, PositionY).
6. This function might be called at high IRQL level or after system has been bugcheck. Driver should put this function in non-paged code section and only use non-paged memory.
    a. Windows kernel mode functions might be also NOT available when this function is called.
7. It is recommended to use the CPU to write the image from source to the frame buffer since the bugcheck might be caused by repeated TDR and the GPU might be in an unknown condition.

Exceptions: Not Specified

Business Justification:

There are 3 main advantages of implementing these DDIs32 bit Windows adds support for UEFI systems Even if Windows needs to bug check the system, the user is not returned to low color, low resolution mode User gets a less jarring experience when Windows bug checks

Scenarios:

BugcheckWindows detects an error and needs to bug check the system Windows takes over the display control from the WDDM driver Windows maintains the same high resolution on the main monitor and displays the error message User gets a better experience

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New; Old ID Gfx8088

Device.Graphics.WDDM12.Display.ModeEnumeration

Target Feature: Device.Graphics.WDDM12.Display

Title: Mode enumeration requirements

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client x64
- Windows 8 Client x86
- Windows 8 Client ARM

Description:

Target modes

- Graphics Adapter must be able to scan out any resolution that is greater than or equal to 1024 x 768 progressive at 32 bpp and a maximum timing which is up to 148.5 Mhz pixel clock per the VESA and Industry Standards and Guidelines for Computer Display Monitor Timing (DMT) Nov 2007 Update or CEA-861-E specs. The graphics driver may support modes that are greater than or less then these constraint's but the driver is not required to do so.
- The graphics drier is not required to support any interlaced timings, but may choose to do so
- During DxgkDdiEnumVidPnCofuncModality DDI call, the supported target modes must be greater than or equal to the pinned source modes except for analog TV connector type or if the target cannot support any timing with a resolution of 1024 x 768 or greater. This means that for all other conditions the driver is only allowed to scale up. Driver can support downscaling if the user requests it specifically for overscan compensation on TVs.

Source modes

- Graphics Adapter must support the native resolution of the integrated panel.
- If the Graphics Adapter has enough bandwidth, it must support the native resolution of any connected display
- Graphics driver must enumerate at least one source mode for every achievable Detailed timing in the EDID of the display
- If the only mode supported by the display device is less than or equal to 640 x 480, then the driver can downscale in this case. However, the source mode must be at least 800 x 600.
- Graphics driver must only enumerate sources modes of 32 bpp or higher.

For Duplicate (Clone) mode

- If there are 2 displays configured in duplicate mode, the Graphics Adapter must support setting the following configuration
    - If there are any common detailed timings between the 2 displays that are less than or equal to 1920 x 1080 progressive at 32 bpp, the Graphics Adapter must support driving the displays at this timing in duplicate mode
    - At a minimum 1024 x 768 must be supported in duplicate mode

- For more than 2 displays configured in duplicate mode, the Graphics Adapter may chose an appropriate mode to support
- At a minimum, irrespective of the number of displays in duplicate mode, the Graphics Adapter must be able to support a Source Mode of 1024 x 768 progressive at 32 bpp in duplicate mode

For Extend mode

- If there are 2 displays configured in extended mode, the Graphics Adapter must support setting the following configuration
    - On systems with integrated displays: Native resolution on integrated display and simultaneously up to 1920 x 1080 progressive at 32 bpp on the non-integrated display
    - Up to 1920 x 1080 progressive at 32 bpp on each non-integrated display
- For more than 2 displays configured in extended mode, the Graphics Adapter may chose appropriate set of modes to support based on available bandwidth

Exceptions: Not Specified

Business Justification:

There is a large and diverse graphic device ecosystem. So the user has many choices when it comes to buying a graphic device to suit their needs. However, it is important to ensure that the most common scenarios work on all configurations running Windows. It is becoming more and more common for users to connect their TV to their PC for streaming videos. Therefore it is very important that this scenarios works reliably. For TVs 1080p is the most common setting and therefore it is important to support that on Windows PCs.

Scenarios:

Projection User connects a projector to a laptop Windows enumerates the list of supported modes in duplicate mode and enables the projector in duplicate mode At the very least, a mode of 1024 x 768 is set Watching a movie on TV User connects a TV to his laptop Windows enumerates the list of supported modes in duplicate mode and enables the projector in duplicate mode User uses Win+P to go to extend mode Windows sets the integrated panel to native resolution and the TV to 1080p Workstation User has 2 monitors connected to his desktop Both monitors are configured to their native resolution

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New; Old ID Gfx8079

Device.Graphics.WDDM12.Display.PnpStopStartSupport

Target Feature: Device.Graphics.WDDM12.Display

Title: Support for PnP Stop in WDDM

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client ARM
- Windows 8 Client x64
- Windows 8 Client x86

Description:

Since the introduction of WDDM, every driver is required to support PnP Start and Stop. This requirement enhances the support for PnP start and stop in WDDM.

Once a WDDM driver is stopped, Windows needs to take over the display control and when the driver is started, Windows needs to hand back display control to the driver.

In Windows 8, WDDM 1.2 introduces a new DDI to add support for UEFI based systems and also to improve the user experience in such a scenario.

The following DDIs are required to be implemented by a Full and Display Only WDDM 1.2 driver

DxgkDdiReleasePostDisplayOwnership

The following are the requirements for the driver when implementing this DDI

1. Windows will provide a Target ID as part of the DDI call. The driver must continue to drive the display associated with the target ID
    a. The driver must check the connectivity of the display on the provided Target ID. If specified target does not have a display connected, driver should fail this call with STATUS_NOT_SUPPORTED.
    b. The driver must disable the signal to all other displays connected to the adapter except the target ID provided.
        i. If this is not possible, driver should try and put a blank image on all other displays
        ii. If this is not possible, driver must leave the last image on the screen unchanged
2. If there is an ACPI ID associated with the target ID, then that must be provided back to Windows as part of the return data structure PDXGK_DISPLAY_INFORMATION
3. For the selected Target ID, the driver must keep the current display mode and provide this mode back to Windows as part of the DDI call

a. In case the driver is not able to maintain the current mode OR the target ID is not part of the active topology, the driver should select an alternate active target and try and maintain the current resolution of that target. If that is not possible the driver should try and set the native resolution of the display OR a high res mode. At the very least the driver must reset to a mode which is larger than or equals to 800x600 24bpp (D3DDDIFMT_R8G8B8) or 32bpp (D3DDDIFMT_X8R8G8B8).

b. If no target is active, then the driver should attempt to enable a target. Preferably the internal panel (if available)

4. Driver must clean the current frame buffer, disable hardware cursor, disable all overlays and set to default GAMMA ramp.

5. Driver must make the current frame buffer linear (either by using swizzle range or disabling swizzle mode).

6. Driver must make the current frame buffer accessible by CPU

7. Driver must ensure that visibility is set to enabled on the specified target

Exceptions: Not Specified

Business Justification:

There are 3 main advantages of implementing these DDIs User gets a better experience during driver upgrade because window size and position is not changed 32 bit Windows adds support for UEFI systems User remains at a native resolution even in cases where an IHV provided WDDM graphics driver is not available

Scenarios:

Boot/Resume User boots a system that runs UEFI in the pre-OS environment UEFI sets the native resolution of the connected display device Windows starts and continues using the same resolution as set by UEFI The user always remains in a high resolution mode throughout his boot/resume experience Driver Upgrade User is running a full WDDM driver User chooses to upgrade the driver with a new one made available on Windows Update. The driver get un installed and Windows falls back to Microsoft Basic Display Driver. At this time there is no change in screen resolution or window position

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New; Old ID Gfx8076

Device.Graphics.WDDM12.Display.ProvideLinearFrameBuffer

Target Feature: Device.Graphics.WDDM12.Display

Title: Graphics Device can provide a linear frame buffer usable by Windows

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

There are numerous scenarios where components in the Windows OS need to be able to update the display when an IHV driver is not loaded. Some of these scenarios are: Boot, Bugcheck, safemode, driver upgrades, etc. To ensure a graphics device is compatible with these Windows scenarios it must:

1) Ensure updates to the frame buffer must be scanned out without requiring addition IHV/OEM software/drivers.

2) Provide a linear frame buffer that is CPU accessible on demand from the IHV driver.

3) On UEFI systems at boot using UEFI 2.3 GOP

4) On BIOS systems using VBE 3.0 standards.

This requirement is intended to provide the necessary support of SYSFUND-8081(SystemUEFIDisplay) and SYSFUND-8082(DisplayReqVideoBIOS)

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New; OLD ID Gfx8068

Device.Graphics.WDDM12.DisplayOnly

Description:

The optional feature set implemented by WDDM 1.2 drivers which support only the display specific DDIs.

Related Requirements:

- Device.Graphics.WDDM12.DisplayOnly.Base

Device.Graphics.WDDM12.DisplayOnly.Base

Target Feature: Device.Graphics.WDDM12.DisplayOnly

Title: Requirements for a WDDM graphics adapter to support display functionality

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client ARM
- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 Release 2 x64

Description:

In Windows 8 and beyond WDDM has been extended to support a WDDM driver that is only responsible for Display Scan out capabilities. Such a driver is not allowed to support any Rendering capabilities.

A driver is considered a WDDM Display Only driver if it only implements the following DDIs and does not implement any of the render DDIs

Common WDDM DDIs

These DDIs are for the common device functionalities such as PnP support and Power support. These functions are required by all WDDM drivers and if not implemented the driver will not be started by Windows. These DDIs are already documented in the WDK.

Required:

DxgkDdiAddDevice

DxgkDdiStartDevice

DxgkDdiStopDevice

DxgkDdiRemoveDevice

DxgkDdiDispatchIoRequest

DxgkDdiSetPowerState

DxgkDdiUnload

Optional:

DxgkDdiInterruptRoutine*

DxgkDdiDpcRoutine

DxgkDdiNotifyAcpiEvent

DxgkDdiQueryInterface

DxgkDdiControlEtwLogging

DxgkDdiEscape

DxgkDdiCollectDbgInfo

*DxgkDdiInterruptRoutine function is required if current hardware device reports hardware interrupt. In this case , if driver did not supply this DDI function, OS would fail the initialization. If current hardware does not have interrupt and driver supplies this DDI function, OS will still allow this driver to be loaded and DxgkDdiInterruptRoutine will never be called.

* DdiNotifyAcpiEvent DDI function is used to notify graphics drivers on some ACPI events. It is optional for rendering device. On normal WDDM graphics devices, this DDI function will return a flag to indicate dxgkrnl.sys to take some further actions such as reset display mode or poll the connected monitors. For the rendering only device, these flags are not used and must be set to zero

Display Only DDIs

Following DDI functions are required to be implemented by a Display Only driver. If the driver does not supply all of these DDIs, Windows will fail to initialize this driver.

DxgkDdiQueryChildRelations

DxgkDdiQueryChildStatus

DxgkDdiQueryDeviceDescriptor

DxgkDdiResetDevice

DxgkDdiQueryAdapterInfo

DxgkDdiSetPalette *

DxgkDdiSetPointerPosition **

DxgkDdiSetPointerShape **

DxgkDdiIsSupportedVidPn

DxgkDdiRecommendFunctionalVidPn ***

DxgkDdiEnumVidPnCofuncModality

DxgkDdiSetVidPnSourceVisibility

DxgkDdiCommitVidPn

DxgkDdiUpdateActiveVidPnPresentPath

DxgkDdiRecommendMonitorModes

DxgkDdiGetScanLine

DxgkDdiQueryVidPnHWCapability

DxgkDdiPresentDisplayOnly

DxgkDdiReleasePostDisplayOwnership

DxgkDdiSystemDisplayEnable

DxgkDdiSystemDisplayWrite

*DxgkDdiSetPalette is a required DDI. If driver does not support any palette mode, it should still supply this DDI.

**DxgkDdiSetPointerPosition and DxgkDdiSetPointerShape are required DDIs. If driver does not support any hardware cursor, it should still supply these two DDIs.

***DxgkDdiRecommendFunctionalVidPn is a required DDI. If driver does not support any ACPI event, it should still supply this DDI.

Design Note:

The only color format supported for display only drivers is D3DDDIFMT_A8R8G8B8 therefore these drivers should only enumerate source modes of this format.

Exceptions: Not Specified

Business Justification:

Microsoft would like Windows to provide a Modern User Experience on all its SKUs. To achieve this, various parts of the Windows UI and APIs are dependent on the WDDM. Therefore it is important that WDDM is always available. If WDDM and its APIs are always available, then the ISVs can focus on one API set. Currently many server systems ship with XDDM hardware. Such hardware does not support the WDDM. The WDDM Display Only driver was designed especially to enable partners to write WDDM drivers for such hardware to provide the benefits of WDDM to the user.

Scenarios:

User installs Windows 8 Server on a system that has an XDDM graphics hardware Windows does not have any XDDM graphics driver in the box, so the Microsoft Basic Display Only Driver (aka Standard VGA driver) is automatically installed. This is a WDDM driver and the user gets a Modern Windows UX along with ability to use the WDDM API set The user installs a WDDM Display Only Driver provided by the IHV At this time key WDDM features are made available on this system Full D3D and D2D support Support for Aero Glass Multi monitor support Hot plug detect

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New; Old ID Gfx8087

Device.Graphics.WDDM12.DisplayRender

Description:

The optional feature set implemented by WDDM 1.2 drivers supporting both display and render DDIs.

Related Requirements:

- Device.Graphics.WDDM12.DisplayRender.Base

Device.Graphics.WDDM12.DisplayRender.Base

Target Feature: Device.Graphics.WDDM12.DisplayRender

Title: Requirements for a WDDM graphics adapter implementing both Render and Display DDIs

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client ARM
- Windows 8 Client x86
- Windows 8 Client x64

Description:

In Windows 8 and beyond WDDM has been extended to support a WDDM driver that is only responsible for Display Scan out capabilities. Such a driver is not allowed to support any Rendering capabilities.

In Windows 8 and beyond WDDM has also been extended to support a WDDM driver that is only responsible for Rendering and Compute DDIs. Such a driver is not allowed to support any Display Scan out capabilities.

Driver's implementing both sets of DDI's (Display and Render) are considered full graphics adapters.

Common WDDM DDIs

These DDIs are for the common device functionalities such as PnP support and Power support. These functions are required by all WDDM drivers and if not implemented the driver will not be started by Windows. These DDIs are already documented in the WDK.

Required:

DxgkDdiAddDevice

DxgkDdiStartDevice

DxgkDdiStopDevice

DxgkDdiRemoveDevice

DxgkDdiDispatchIoRequest

DxgkDdiSetPowerState

DxgkDdiUnload

Optional:

DxgkDdiInterruptRoutine*

DxgkDdiDpcRoutine

DxgkDdiNotifyAcpiEvent

DxgkDdiQueryInterface

DxgkDdiControlEtwLogging

DxgkDdiEscape

DxgkDdiCollectDbgInfo

*DxgkDdiInterruptRoutine function is required if current hardware device reports hardware interrupt. In this case , if driver did not supply this DDI function, OS would fail the initialization. If current hardware does not have interrupt and driver supplies this DDI function, OS will still allow this driver to be loaded and DxgkDdiInterruptRoutine will never be called.

Display Only DDIs

Following DDI functions are required to be implemented by a Display Only driver. If the driver does not supply all of these DDIs, Windows will fail to initialize this driver.

DxgkDdiQueryChildRelations

DxgkDdiQueryChildStatus

DxgkDdiQueryDeviceDescriptor

DxgkDdiResetDevice

DxgkDdiQueryAdapterInfo

DxgkDdiSetPalette *

DxgkDdiSetPointerPosition **

DxgkDdiSetPointerShape **

DxgkDdiIsSupportedVidPn

DxgkDdiRecommendFunctionalVidPn ***

DxgkDdiEnumVidPnCofuncModality

DxgkDdiSetVidPnSourceVisibility

DxgkDdiCommitVidPn

DxgkDdiUpdateActiveVidPnPresentPath

DxgkDdiRecommendMonitorModes

DxgkDdiGetScanLine

DxgkDdiQueryVidPnHWCapability

DxgkDdiPresentDisplayOnly

DxgkDdiReleasePostDisplayOwnership

DxgkDdiSystemDisplayEnable

DxgkDdiSystemDisplayWrite

*DxgkDdiSetPalette is a required DDI. If driver does not support any palette mode, it should still supply this DDI.

**DxgkDdiSetPointerPosition and DxgkDdiSetPointerShape are required DDIs. If driver does not support any hardware cursor, it should still supply these two DDIs.

***DxgkDdiRecommendFunctionalVidPn is a required DDI. If driver does not support any ACPI event, it should still supply this DDI.

* DdiNotifyAcpiEvent DDI function is used to notify graphics drivers on some ACPI events. It is optional for rendering device. On normal WDDM graphics devices, this DDI function will return a flag to indicate dxgkrnl.sys to take some further actions such as reset display mode or poll the connected monitors. For the rendering only device, these flags are not used and must be set to zero

Rendering only DDIs

These DDI functions are for the rendering functionalities.

Required:

DxgkDdiInterruptRoutine

DxgkDdiDpcRoutine

DxgkDdiResetDevice

DxgkDdiCreateDevice

DxgkDdiDestroyAllocation

DxgkDdiDescribeAllocation

DxgkDdiOpenAllocation

DxgkDdiCloseAllocation

DxgkDdiGetStandardAllocationDriverData

DxgkDdiSubmitCommand

DxgkDdiPreemptCommand

DxgkDdiBuildPagingBuffer

DxgkDdiResetFromTimeout

DxgkDdiRestartFromTimeout

DxgkDdiQueryCurrentFence

DxgkDdiControlInterrupt

DxgkDdiDestroyDevice

DxgkDdiPresent

DxgkDdiCreateAllocation

DxgkDdiPatch

DxgkDdiRender

DxgkDdiRenderKm

Optional:

If rendering hardware support swizzling ranger, driver should also supply following two functions

DxgkDdiAcquireSwizzlingRange

DxgkDdiReleaseSwizzlingRange

Exceptions: Not Specified

Business Justification:

Microsoft would like Windows to provide a Modern User Experience on all its SKUs. To achieve this, various parts of the Windows UI and APIs are dependent on the WDDM. Therefore it is important that WDDM is always available. If WDDM and its APIs are always available, then the ISVs can focus on one API set. Currently many server systems ship with XDDM hardware. Such hardware does not support the WDDM. The WDDM Display Only driver was designed especially to enable partners to write WDDM drivers for such hardware to provide the benefits of WDDM to the user. Over the years there has been an increasing focus on GPGPU scenarios for doing vast amount of complex mathematical or graphical operations. Some of the common examples of this are graphics rendering for animation, database processing, financial & astronomical calculations and oil and gas

exploration. The use of GPGPU has proved benefits in performance, power consumption and cost. This feature makes it easier for an OEM to ship a system specifically targeted for such a use without having the added complexity of supporting a display device

Scenarios:

Display User installs Windows 8 Server on a system that has an XDDM graphics hardware Windows does not have any XDDM graphics driver in the box, so the Microsoft Basic Display Only Driver (aka Standard VGA driver) is automatically installed. This is a WDDM driver and the user gets a Modern Windows UX along with ability to use the WDDM API set The user installs a WDDM Display Only Driver provided by the IHV At this time key WDDM features are made available on this system Rendering Server Render Farm User configures a Server system that only contains one GPU installed as a Render only device and does not have any display devices connected to the system. The user accesses this Server by utilizing Remote Desktop Sessions or alternate means. Now the user launches a DirectX application that needs to use the advanced computing power of the GPU for some processing (like graphics render farm)Note: GDI based applications will not work in this case unless, a Full WDDM driver or a Display Only driver is installed along with this OR the user connects via a Remote Desktop session. The application uses the DirectX APIs to enumerate WDDM GPUs and finds the Render only device and utilizes it for its computational needs Additional GPU for compute User configures a system with 2 GPUs. One a full WDDM driver which support Render and Display functions. The second a WDDM render only driver that only support render functions The user has a monitor connected to the full WDDM graphics hardware and uses it to log into the system Now the user launches an application that needs to use the advanced computing power of the GPU for some processing (like graphics render farm) The application uses the DirectX APIs to enumerate WDDM GPUs and finds both the GPUs. The application selects the Render only device and utilizes it for its computational needs but uses the Full WDDM driver for its UI rendering needs.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New full graphics requirement

Device.Graphics.WDDM12.DisplayRender.ProcessingStereoscopicVideoContent

Description:

The optional feature set implemented by WDDM 1.2 drivers which support stereoscopic video processing.

Related Requirements:

- Device.Graphics.WDDM12.DisplayRender.ProcessingStereoscopicVideoContent.ProcessingStereoscopicVideoContent

Device.Graphics.WDDM12.DisplayRender.ProcessingStereoscopicVideoContent.ProcessingStereoscopicVideoContent

Target Feature: Device.Graphics.WDDM12.DisplayRender.ProcessingStereoscopicVideoContent

Title: If Display adapter supports presentation and processing of stereoscopic video content it must conform to the DXGI specification for Stereo 3D

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

WDDM 1.2 drivers may optionally process video data encoded for stereo. If the driver publishes that it can support stereo, it must support processing of content in horizontally and vertically arranged left and right eye views, as well as separate streams for the left and right eye views.

In addition, if the driver publishes that it can support stereo, it may optionally also process the following stereo content layouts:

- Row interleaved
- Column interleaved
- Checkerboard
- Flipped Configurations

Finally, the driver may optionally support new stereo processing features:

- Mono offset
- Stereo Eye Adjustment

For all optional content layouts and processing features, the driver must publish the appropriate cap to indicate if it supports the feature.

Exceptions: Not Specified

Business Justification:

Enabling physical and streaming 3DS media content on Win8 will allow consumers to watch their favorite media content on their PCs and utilize their stereo video hardware (TVs, monitors, shutter glasses, etc) with Windows. This should also spur the sales of high-end PCs, Monitors and Laptops. Mono offset provides the ability to generate menus and overlays for stereo displays from non-stereo content. This is used for stereo blu-ray menus.

Scenarios:

On capable hardware, stereo video can be processed or displayed if provided in the various configurations defined in the H.264 MVC specification and other stereo video format specifications. On capable hardware, an image can be converted into a stereo pair of images in order to provide the appearance of depth in menus and controls for blu-ray discs and other interactive elements. On capable hardware, stereo video can be adjusted by the viewer to have a greater or lesser eye disparity in order to reduce eye strain for the viewer.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New; Old ID Gfx8069

Device.Graphics.WDDM12.DisplayRender.RuntimePowerMgmt

Description:

The optional feature set implemented by WDDM 1.2 drivers which support the runtime power management DDIs.

Related Requirements:

- Device.Graphics.WDDM12.DisplayRender.RuntimePowerMgmt.RuntimePowerMgmt

Device.Graphics.WDDM12.DisplayRender.RuntimePowerMgmt.RuntimePowerMgmt

Target Feature: Device.Graphics.WDDM12.DisplayRender.RuntimePowerMgmt

Title: If implemented - WDDM 1.2 drivers must use the new WDDM 1.2 GPU Power Management DDI for F-state and p-state power management.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM
- Windows 8 Server x64

Description:

WDDM 1.2 drivers can optionally support F-state and p-state Power Management.

For more details on the use of the SoC GPU Power Management WDDM v1.2 DDI, please refer to the Windows 8 WDK documentation.

Exceptions: Not Specified

Business Justification:

Energy Efficiency has become a key distinguishing feature for OEMs today. Given the proliferation of small form factors, the concept of All day Battery Life is being strived for. The scenario here is that you carry your mobile device without the power brick and only at the end of the day; you plug it into the power source for recharging the battery. GPUs and Displays are two of the largest power consumers in desktops and mobile devices; hence the importance of managing the Idle and Active power cannot be understated.

Scenarios:

"All Day Battery Life": Mobile form factor device is able to go into Idle and save power due to individual system components shutting down if not in use. Systems that support Connected Standby: New Windows 8 systems that are capable of smartphone-like power model and have sufficiently low idle power to remain on when the user has completed interacting with the system.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New; Old id Gfx8083

Device.Graphics.WDDM12.Render

Description:

The base feature set implemented by WDDM 1.2 drivers which support the render specific DDIs.

Related Requirements:

- Device.Graphics.WDDM12.Render.Base
- Device.Graphics.WDDM12.Render.D3D11VideoDecoding
- Device.Graphics.WDDM12.Render.D3D11VideoProcessing
- Device.Graphics.WDDM12.Render.DirectFlip
- Device.Graphics.WDDM12.Render.FlipOnVSyncMmIo
- Device.Graphics.WDDM12.Render.OfferReclaim
- Device.Graphics.WDDM12.Render.PreemptionGranularity
- Device.Graphics.WDDM12.Render.Stereoscopic3DArraySupport
- Device.Graphics.WDDM12.Render.TDRResiliency
- Device.Graphics.WDDM12.Render.UMDLogging
- Device.Graphics.WDDM12.Render.XPSRasterizationConformance

Device.Graphics.WDDM12.Render.Base

Target Feature: Device.Graphics.WDDM12.Render

Title: Requirements for a WDDM graphics adapter to support render functionality

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client ARM
- Windows 8 Client x64
- Windows 8 Client x86

Description:

In Windows 8 and beyond WDDM has been extended to support a WDDM driver that is only responsible for Rendering and Compute DDIs. Such a driver is not allowed to support any Display Scan out capabilities.

A driver is considered a WDDM Render Only driver if one of the following two conditions is met:

1. The driver implements all the DDIs required for a full WDDM driver but reports 0 sources and 0 targets

2. The driver implements all the Render specific DDIs and returns a null pointer for all the Display specific DDIs. The DDIs required for this are called out below

Common WDDM DDIs

These DDI functions are for the common device functionalities such as PnP support and Power support. These functions are required by all WDDM drivers and if not implemented the driver will not be started by Windows. These DDIs are already documented in the WDK: http://msdn.microsoft.com/en-us/library/ff554066(v=VS.85).aspx.

Required:

DxgkDdiAddDevice

DxgkDdiStartDevice

DxgkDdiStopDevice

DxgkDdiRemoveDevice

DxgkDdiDispatchIoRequest

DxgkDdiSetPowerState

DxgkDdiUnload

Optional:

DdiNotifyAcpiEvent *

* DdiNotifyAcpiEvent DDI function is used to notify graphics drivers on some ACPI events. It is optional for rendering device. On normal WDDM graphics devices, this DDI function will return a flag to indicate dxgkrnl.sys to take some further actions such as reset display mode or poll the connected monitors. For the rendering only device, these flags are not used and must be set to zero

Rendering only DDIs

These DDI functions are for the rendering functionalities.

Required:

DxgkDdiInterruptRoutine

DxgkDdiDpcRoutine

DxgkDdiResetDevice

DxgkDdiCreateDevice

DxgkDdiDestroyAllocation

DxgkDdiDescribeAllocation

DxgkDdiOpenAllocation

DxgkDdiCloseAllocation

DxgkDdiGetStandardAllocationDriverData

DxgkDdiSubmitCommand

DxgkDdiPreemptCommand

DxgkDdiBuildPagingBuffer

DxgkDdiResetFromTimeout

DxgkDdiRestartFromTimeout

DxgkDdiQueryCurrentFence

DxgkDdiControlInterrupt

DxgkDdiDestroyDevice

DxgkDdiPresent

DxgkDdiCreateAllocation

DxgkDdiPatch

DxgkDdiRender

DxgkDdiRenderKm

Optional:

If rendering hardware support swizzling ranger, driver should also supply following two functions

DxgkDdiAcquireSwizzlingRange

DxgkDdiReleaseSwizzlingRange

Exceptions: Not Specified

Business Justification:

Over the years there has been an increasing focus on GPGPU scenarios for doing vast amount of complex mathematical or graphical operations. Some of the common examples of this are graphics rendering for animation, database processing, financial & astronomical calculations and oil and gas exploration. The use of GPGPU has proved benefits in performance, power consumption and cost. This feature makes it easier for an OEM to ship a system specifically targeted for such a use without having the added complexity of supporting a display device

Scenarios:

Server Render Farm User configures a Server system that only contains one GPU installed as a Render only device and does not have any display devices connected to the system. The user accesses this Server by utilizing Remote Desktop Sessions or alternate means. Now the user launches a DirectX application that needs to use the advanced computing power of the GPU for some processing (like graphics render farm) Note: GDI based applications will not work in this case unless, a Full WDDM driver or a Display Only driver is installed along with this OR the user connects via a Remote Desktop session. The application uses the DirectX APIs to enumerate WDDM GPUs and finds the Render only device and utilizes it for its computational needs Additional GPU for compute User configures a system with 2 GPUs. One a full WDDM driver which support Render and Display functions. The second a WDDM render only driver that only support render functions The user has a monitor connected to the full WDDM graphics hardware and uses it to log into the system Now the user launches an application that needs to use the advanced computing power of the GPU for some processing (like graphics render farm) The application uses the DirectX APIs to enumerate WDDM GPUs and finds both the GPUs. The application selects the Render only device and utilizes it for its computational needs but uses the Full WDDM driver for its UI rendering needs.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New; Old ID Gfx8089

Device.Graphics.WDDM12.Render.D3D11VideoDecoding

Target Feature: Device.Graphics.WDDM12.Render

Title: Display driver supports the DirectX 11 Video Decoder DDI

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86

- Windows 8 Server x64
- Windows 8 Client ARM

Description:

Display WDDM 1.2drivers must support the DirectX 11 Video Decoder DDI.

WDDM drivers must support at least one of the following sets of MPEG2 GUIDs:

- D3D11_DECODER_PROFILE_MPEG2_VLD and D3D11_DECODER_PROFILE_MPEG2_IDCT
- D3D11_DECODER_PROFILE_MPEG2_VLD and D3D11_DECODER_PROFILE_MPEG2_MOCOMP
- D3D11_DECODER_PROFILE_MPEG2_IDCT
- D3D11_DECODER_PROFILE_MPEG2AND1_VLD

And at least one of the H.264 GUIDs:

- D3D11_DECODER_PROFILE_H264_MOCOMP_NOFGT
- D3D11_DECODER_PROFILE_H264_MOCOMP_FGT
- D3D11_DECODER_PROFILE_H264_VLD_NOFGT
- D3D11_DECODER_PROFILE_H264_VLD_FGT

In addition, WDDM drivers must support at least one of the following VC1 modes:

- D3D11_DECODER_PROFILE_VC1_MOCOMP
- D3D11_DECODER_PROFILE_VC1_IDCT
- D3D11_DECODER_PROFILE_VC1_VLD

WDDM drivers must support Standardized AES 128 (defined in the WDDM v1.1 specifications) for both MPEG2 and H.264.

Finally, WDDM drivers must support DXGI_FORMAT_420_OPAQUE and DXGI_FORMAT_NV_12 as decoder output formats.

Design Notes

- MPEG-2 support is required on x86 and x64 architectures and operating systems only.
- ** Note that it is not a requirement to implement the "Post-Proc" stage on DXVA2_ModeVC1 GUIDs.

Exceptions: Not Specified

Business Justification:

Requiring video decode support on all Windows system ensure us that consumers can rely upon Windows as a platform for viewing high-quality video content.

Scenarios:

The end-user wishes to watch video content in the most common video encoding formats: MPEG2, H.264, and VC1 (Blu-ray). Media Foundation as well as a number of 3rd party applications utilize DXVA2 for playback of these formats.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New; Old ID Gfx8072

Device.Graphics.WDDM12.Render.D3D11VideoProcessing

Target Feature: Device.Graphics.WDDM12.Render

Title: Display driver supports appropriate DDIs for DirectX 11 Video Processing

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

The following video processor device capability DDIs must be supported:

- BOB Deinterlacing
    - DXVAHDDDI_PROCESSOR_CAPS_DEINTERLACE_BOB
    - D3D11_1DDI_VIDEO_PROCESSOR_PROCESSOR_CAPS_DEINTERLACE_BOB
- Constriction
    - DXVAHDDDI_FEATURE_CAPS_CONSTRICTION
    - D3D11_1DDI_VIDEO_PROCESSOR_FEATURE_CAPS_CONSTRICTION
- Rotation
    - DXVAHDDDI_FEATURE_CAPS_ROTATION
    - D3D11_1DDI_VIDEO_PROCESSOR_FEATURE_CAPS_ROTATION
- Arbitrary stretching/shrinking

- Both full and nominal RGB ranges must be supported on the input and the output YCbCr data, specifically:
    - 0 to 255
        - DXVAHDDDI_STREAM_STATE_INPUT_COLOR_SPACE_DATA

RGB_Range field is set to 0

    - DXVAHDDDI_BLT_STATE_OUTPUT_COLOR_SPACE_DATA RGB_Range field is set to 0
        - D3D11_1DDI_VIDEO_PROCESSOR_COLOR_SPACE RGB_Range field set to 0
    - 16 to 235
        - DXVAHDDDI_STREAM_STATE_INPUT_COLOR_SPACE_DATA

RGB_Range field is set to 1

    - DXVAHDDDI_BLT_STATE_OUTPUT_COLOR_SPACE_DATA RGB_Range field is set to 1
        - D3D11_1DDI_VIDEO_PROCESSOR_COLOR_SPACE RGB_Range field set to 1
- BT. 601 and BT. 709 conversion matrices must be supported on the input and the output YCbCr data, specifically:
- DXVAHDDDI_STREAM_STATE_INPUT_COLOR_SPACE_DATA.YCbCr_Matrix
        - Both setting the field to 0 (representing BT.601) and 1 (representing BT.709) must be supported.
- DXVAHDDDI_BLT_STATE_OUTPUT_COLOR_SPACE_DATA.YCbCr_Matrix
        - Both setting the field to 0 (representing BT.601) and 1 (representing BT.709) must be supported.
    - D3D11_1DDI_VIDEO_PROCESSOR_COLOR_SPACE.YCbCr_Matrix
        - Both setting the field to 0 (representing BT.601) and 1 (representing BT.709) must be supported.
- Arbitrary background color
- Per-stream source rectangle
- Per-stream destination rectangle
- Overall destination rectangle
- At least one stream

The following formats must be supported for input:

- D3DFMT_420_OPAQUE
- D3DFMT_NV12
- D3DFMT_YUY2
- Any formats supported as output from DXVA 2.0 decode or DirectX 11 Video decode

Depending on the feature level made available by the driver for Direct3D (e.g. 9.1 9.3 for Direct3D 9 DDIs, 10.0 for Direct3D 10 DDIs, etc.), the following formats must be supported for output:

- 9.1 9.3 Feature Levels
    - D3DFMT_A8R8G8B8
    - D3DFMT_NV12
- 10.0 10.1 Feature Levels
    - DXGI_FORMAT_R16G16B16A16_FLOAT
    - DXGI_FORMAT_R8G8B8A8_UNORM
    - DXGI_FORMAT_R10G10B10A2_UNORM
    - DXGI_FORMAT_R8G8B8A8_UNORM_SRGB
    - DXGI_FORMAT_NV12
    - The following formats are also required if the driver supports these as a swapchain format:
        - DXGI_FORMAT_B8G8R8A8_UNORM
        - DXGI_FORMAT_B8G8R8A8_UNORM_SRGB
        - DXGI_FORMAT_R10G10B10_XR_BIAS_A2_UNORM
- 11.0 Feature Level and beyond
    - DXGI_FORMAT_R16G16B16A16_FLOAT
    - DXGI_FORMAT_R8G8B8A8_UNORM
    - DXGI_FORMAT_R10G10B10A2_UNORM
    - DXGI_FORMAT_R8G8B8A8_UNORM_SRGB
    - DXGI_FORMAT_NV12, DXGI_FORMAT_B8G8R8A8_UNORM
    - DXGI_FORMAT_B8G8R8A8_UNORM_SRGB
    - DXGI_FORMAT_R10G10B10_XR_BIAS_A2_UNORM

Design Notes

- MPEG-2 support is required on x86 and x64 architectures and operating systems only.

Exceptions: Not Specified

Business Justification:

Basic functionality of defined in the most common video specifications (e.g. H.264, Blu-ray) require conversion of all of the specified YUV formats to RGB data for display of video. In order to enable full support for these encoding formats, all WDDM hardware must support these conversion capabilities.High quality color conversion ensures comparable video quality can be obtained for video to competitive platforms.

Scenarios:

The end-user wishes to watch video content in the most common video encoding formats: MPEG2, H.264, and VC1 (Blu-ray). Media Foundation as well as a number of 3rd party applications utilize D3D11 for playback of these formats.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New; Old ID Gfx8071

Device.Graphics.WDDM12.Render.DirectFlip

Target Feature: Device.Graphics.WDDM12.Render

Title: Driver must Support the DirectFlip DDI

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM
- Windows 8 Server x64

Description:

The driver must publish the SupportDirectFlip field as TRUE in the DXGK_DRIVERCAPS structure.

WDDM 1.2 drivers for the Direct3D 11 DDI must implement the D3D11_1DDI_CHECKDIRECTFLIPSUPPORT DDI and return TRUE from the DDI when it is called with at least the following parameters:

- The app resource matches the DWM resource in format and pixel dimensions
- The D3DDDI_CHECKDIRECTFLIP_IMMEDIATE flag not set

WDDM 1.2 drivers for the Direct3D 9 DDI must implement the D3DDDIARG_CHECKDIRECTFLIPSUPPORT DDI and return TRUE from the DDI when it is called with at least the following parameters:

- The app resource matches the DWM resource in format and pixel dimensions
- The D3DDDI_CHECKDIRECTFLIP_IMMEDIATE flag not set

The driver must support the creation of shared primary swapchains, specifically identified as resources created with:

- pPrimaryDesc as non-NULL.
- D3D10_DDI_RESOURCE_MISC_SHARED set in MiscFlags.
- D3D10_DDI_BIND_SHADER_RESOURCE, D3D10_DDI_BIND_PRESENT, and D3D10_DDI_BIND_RENDER_TARGET all set in BindFlags.

When the SharedPrimaryTransition bit is set in the DXGK_SETVIDVIDPNSOURCEADDRESS DDI, the driver must:

- Validate that the hardware can seamlessly switch between the two allocations.
- Perform a seamless switch to the new primary indicated by the DDI.

Exceptions: Not Specified

Business Justification:

To ensure optimal power consumption for video playback and other full-screen scenarios, providing DirectFlip enables a minimum amount of memory bandwidth for displaying full-screen content while ensuring smooth transitions between full-screen applications, other applications, and the desktop environment.

Scenarios:

The user wishes to view a video or run an application that covers the entire screen. When the user enters or exits the application or notifications appear over the application, no mode change is required and the experience is smooth. Furthermore, the user enjoys extended battery life on mobile devices as memory bandwidth requirements are reduced for full screen applications like video.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New;

Device.Graphics.WDDM12.Render.FlipOnVSyncMmIo

Target Feature: Device.Graphics.WDDM12.Render

Title: WDDM1.2 drivers supports a memory mapped I/O (MMIO)-based flip that takes effect on the next vertical sync

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client ARM
- Windows 8 Client x64
- Windows 8 Client x86

Description:

All WDDM1.2 drivers must publish the FlipOnVSyncMmIo field as TRUE in the DXGK_FLIPCAPS structure, and implement behavioroutlined under FlipOnVSyncMmIo here:

http://msdn.microsoft.com/en-us/library/ff561069(v=VS.85).aspx

Exceptions: Not Specified

Business Justification:

In Windows 8, the GPU scheduler will attempt to preempt hardware even in the presence of pending flips. To prevent tearing and rendering artifacts the driver is required to support the MmIoFlip based model. Explicitly requiring this support upfront will reduce the chance of rendering issues appearing on customer machines.In addition, support for the hardware flip queue code path is eliminated. In past operating systems, this led to some reliability problems which were difficult to diagnose.

Scenarios:

The user wishes to play a game on a powerful desktop with multiple GPUs in a Linked adapter (LDA) configuration, examples of which are AMD PowerExpress, and NVIDIA Hybrid power. The user is pleased with performance of task switching and absence of rendering artifacts when this happens.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New;

Device.Graphics.WDDM12.Render.OfferReclaim

Target Feature: Device.Graphics.WDDM12.Render

Title: WDDM 1.2 drivers must use the Offer and Reclaim DDI to reduce memory footprint

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client ARM
- Windows 8 Client x64

- Windows 8 Client x86

Description:

WDDM 1.2 drivers must use the new UMD Offer and Reclaim DDI to reduce the overhead of memory resources created for temporary surfaces in local video and system memory. An example is the use of staging buffers to accelerate the process of uploading data to GPU local memory. By offering these resources, the operating system can reuse the memory without risking the expense of destroying and recreating them at high frequency.

WDDM Drivers also often keep allocations that an application is no longer using so that if the application creates another allocation with the same properties, it can give back the old one. This reduces the performance impact of rapidly destroying and re-creating allocations, a common bad behavior for applications, but it also can have a very high memory cost. By offering those allocations, the WDDM 1.2 driver can keep most of its performance without wasting memory.

Below are detailed sub-requirements:

- WDDM 1.2 drivers must always Offer internal staging buffers once they have been used. These temporary buffers generally hold data being moved or copied from a source to a destination. In Windows 7 and previous operating systems, these temporary surfaces are left in memory even though they were no longer in use.
- WDDM 1.2 drivers must always Offer the deferred deletions of surfaces which are recycled: If a driver decides to defer the destruction of a surface, it must at least offer it to the video memory manager.
- WDDM 1.2 driver should only Offer packed allocations when all of the individual resources contained in the allocation have been Offer-ed. The allocation as a whole should be reclaimed when any individual resources have been reclaimed.
- WDDM 1.2 drivers must always Offer Discarded allocations if it implements its own Renaming mechanism.
- WDDM 1.2 drivers should never pack allocations larger than 64KB with other allocations, guaranteeing for application that offer will give them the expected benefit.

For more details on the use of the Offer and Reclaim WDDM v1.2 DDI, please refer to the Windows 8 WDK documentation.

Exceptions: Not Specified

Business Justification:

Use of Offer and Reclaim API by WDDM 1.2 drivers will provide the following benefits: WDDM 1.2 drivers using this API will use less memory resources under memory pressure. More memory will be available for other applications running on the system and are in need of system memory. Releasing memory allocations used by temporary staging surfaces once the work is done will result in a lower memory footprint for WDDM drivers and also applications which use the Offer API. Reduced time to

enter and come out of Hibernate memory resources Offer-ed by WDDM 1.2 drivers will be deleted when the system enters Hibernate. This will reduce the time to enter Hibernation.

Scenarios:

Use of Offer and Reclaim API by WDDM 1.2 drivers will result in efficient use of video and system memory. On systems with less system memory, more memory will be available to other applications running on the system as the WDDM v1.2 drivers will Offer back memory resources when not needed. On systems with sufficient system memory with no memory pressure, the applications will be able to use generous amounts of memory through extra caches and temporary surfaces to speed up performance. Under memory pressure, the use of the Offer and Reclaim API will result in prudent use of memory, making it available to other applications when needed. Temporary memory resources which are Offered by the WDDM 1.2 driver will be deleted by the Video Memory Manager when the system goes into Hibernation. This will result in time savings in the event of system hibernation as these resources do not have to be written to the Hiberfile. Time for system resume will also be correspondingly less as the Hiberfile is smaller.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New; Old ID Gfx8086

Device.Graphics.WDDM12.Render.PreemptionGranularity

Target Feature: Device.Graphics.WDDM12.Render

Title: WDDM 1.2 drivers must report the granularity level of GPU Preemption.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM
- Windows 8 Server x64

Description:

A WDDM 1.2 driver must report the GPU Preemption granularity for running Graphics and Compute scenarios.

The WDDM 1.2 must do the following:

- The WDDM 1.2 driver must first set the PreemptionAware flag and the MultiEngineAware flag in the _DXGK_VIDSCHCAPS structure through the DxgkDdiQueryAdapterInfo DDI.
- The WDDM 1.2 driver must set the appropriate GPU Preemption granularity through D3DKMDT_PREEMPTION_CAPS PreemptionCaps.

There is no mandatory requirement on the actual level of preemption for Graphics or Compute. For example, if a GPU does not support any level of Mid-DMA Buffer Graphics Preemption such as that on the triangle boundary or others, then it can report this cap as D3DKMDT_GRAPHICS_PREEMPTION_DMA_BUFFER_BOUNDARY. Similarly, if it does not support any level of finer grained preemption for Compute, then it must report the cap as D3DKMDT_COMPUTE_PREEMPTION_DMA_BUFFER_BOUNDARY.

However, if the GPU does support Mid-DMA Buffer or Packet Preemption, then the WDDM 1.2 driver must choose the appropriate cap in order to take advantage of the benefits offered by finer grained GPU Preemption for Graphics and/or Compute.

For more details on the use of the GPU Preemption WDDM v1.2 DDI, please refer to the Windows 8 WDK documentation.

Exceptions: Not Specified

Business Justification:

The goal of GPU Preemption is to improve desktop and touch responsiveness by allowing desktop compositions and foreground applications low latency access to the GPU even on a busy desktop with the GPU being used for background compute task. GPU Compute has become more pervasive over the past decade. GPUs are increasingly used for distributed computing and data-parallel tasks such as image processing, video encoding and decoding, scientific simulation and several others. Depending on the exact nature of the workload, some of these tasks take significant amounts of time to complete execution. When needed, these long-running tasks need to be interrupted as quickly as possible so that the GPU can be used to render the desktop or response from user touch input.A WDDM 1.2 driver which supports GPU Preemption as highlighted in the Details section above will also benefit from better system fault-tolerance through the TDR Improvements feature.

Scenarios:

On capable hardware, the GPU can be preempted in the middle of execution of a DMA buffer if there is another task waiting on the GPU.Long-running Compute tasks do not interfere with the responsiveness of the UI of other graphics applications running at the same time.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New; Old id Gfx8091

Device.Graphics.WDDM12.Render.Stereoscopic3DArraySupport

Target Feature: Device.Graphics.WDDM12.Render

Title: WDDM1.2 drivers must support Stereoscopic 3D in D3D by adding expanded array support.

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client ARM
- Windows 8 Client x64
- Windows 8 Client x86

Description:

New support required by WDDM 1.2

DDI or feature	WDDM 1.1	WDDM 1.2
Creation of backbuffers (and primaries) using D3D10DDIARG_CREATERESOURCE & D3D11DDIARG_CREATERESOURCE	Restricted to ArraySize = 1	Generalized to ArraySize = 2
DXGI UM backbuffer DDIs (DXGI_DDI_ARG_BLT, DXGI_DDI_ARG_ROTATE_RESOURCE_IDENTITIES, DXGI_DDI_ARG_PRESENT, DXGI_DDI_ARG_SETDISPLAYMODE)	Restricted to backbuffers	Same, but including stereo back buffers
Presentation in general (kernel and user)	Restricted to backbuffers	Same, but including stereo back buffers
Sharing	Restricted to ArraySize = 1	Generalized to *any* ArraySize (including > 2)
GDI interop	Restricted to ArraySize = 1	Generalized to *any* ArraySize (including > 2)
HLSL non-arrayed sampler declarations	Restricted to ArraySize = 1	Generalized to *any* ArraySize (including > 2)

HLSL non-arrayed sampler declarations indicates allowing certain shaders that sample from single-subresource-resources currently to also sample from single-subresource-views of resources with multiple subresources.

Note that drivers must support extended array for Stereo 3D APIs but for fullscreen exclusive Stereo 3D apps to display, the driver should also support Stereo scanout on the output. For example in a multi-adapter system with two WDDM 1.2 graphics adapters either adapter may be used to create a stereo *windowed* swapchain, even if only one of the adapters actually supports stereo output.

Background info

The following table shows how in **Windows 7 / WDDM 1.1** there was a simple sequence of nested classes of resources and some of the DDIs that only applied to specific subsets.

Type of resource (general to specific)	DDIs that only apply to specific resources
D3D Texture2D resources	
D3D Texture2D resources with ArraySize <= 2	
D3D Texture2D resources with ArraySize == 1	Sharing, GDI interop
Backbuffers	DXGI_DDI_ARG_BLT, etc.
Primaries	Specific presentation operations

WDDM 1.2 generalizes backbuffers (and primaries) to include stereo pairs (ArraySize = 2; while WDDM1.2 backbuffers and primaries are resources with ArraySize =1) All DDIs that are specific to

backbuffers are implicitly and explicitly generalized to support these new stereo backbuffers (e.g. DXGI_DDI_ARG_BLT).

Some features that are implicitly used by swapchains (e.g. sharing) and that had been limited to only ArraySize=1 textures are generalized to support resources with any ArraySize.

GDI interop is also extended to support resources with any ArraySize. GDI interop isnt strictly necessary for stereo support but is included to meet the goal of making it easy to port mono applications to support stereo. (The decision to support general ArraySize and not just 1 or 2 for shared resources and GDI interop was based on the judgment that further special casing here would not be best for the API.)

Exceptions: Not Specified

Business Justification:

This requirement is necessary for a good quality of experience for running Stereo applications in Windows 8. Windows will switch to Stereo mode when the user launches a Stereo application. Having a stereo resolution equivalent to the native resolution in mono mode will ensure that there is no mode change failure when the user launches a windowed mode or full-screen Stereo application. If the Stereo 3D Display does not support a Stereo equivalent of the native resolution, then the user will be unable to run the stereo application unless a different resolution is selected manually from the Screen Resolution display applet.

Scenarios:

The Stereo 3D Display on a Stereo-capable system is presently in the native mono resolution as the user is working with non-stereo or mono applications. The user then launches a Stereo application. This results in the Stereo 3D Display switching to the stereo mode in the same resolution. The user is able to successfully visualize the application in Stereo as Windows will be running in a Stereo mode. In the case this requirement is not met, the above scenario will fail and the user experience will be as follows: The Stereo 3D Display on a Stereo-capable system is presently in the native mono resolution as the user is working with non-stereo or mono applications. The user then launches a Stereo application. Windows will try to switch to Stereo mode with the same display resolution. However, as the Stereo 3D Display lacks a Stereo mode in the previously running native mono mode, a mode set failure will be encountered. This may result in an application failure or simply mono visualization of the stereo application. The user will have to try a different resolution from the display applet.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New; Old ID Gfx8081

Device.Graphics.WDDM12.Render.TDRResiliency

Target Feature: Device.Graphics.WDDM12.Render

Title: WDDM 1.2 drivers for GPUs which support Per-Engine Reset must implement the Windows 8 DDI for TDR Resiliency.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM
- Windows 8 Server x64

Description:

A WDDM 1.2 driver for a GPU supporting Per-Engine Reset must implement the TDR DDI for improved reliability and resiliency.

The WDDM 1.2 must do the following:

- The WDDM 1.2 driver must satisfy the WDDM 1.2 GPU Preemption requirement.
- The WDDM 1.2 driver must implement the following GPU Engine DDIs:
    - QueryDependentEngineGroup
    - QueryEngineStatus
    - ResetEngine
- WDDM 1.2 drivers must continue supporting the pre-Windows 8 TDR behavior of Adapter-wide Reset and Restart. Semantics of these existing DDIs remain the same as before Windows 8.

For more details on the use of the GPU Preemption and TDR Improvements WDDM v1.2 DDI, please refer to the Windows 8 WDK documentation.

Exceptions: Not Specified

Business Justification:

The goal of an improved mechanism for GPU Timeout Detection & Recovery (or TDR) is improved resiliency to GPU hangs which are triggered by long-running graphics workloads taking more than the allowed time for work completion. On previous Windows operating systems, repeated operations like these result in repeated TDRs which eventually crash the system. Applications which do compute on the GPU that can submit work to the GPU can also take much longer to complete than the allowed timeout. With the Windows 8 feature support for GPU Preemption, these tasks can execute without interfering with other applications, including the desktop window manager (DWM). The TDR improvements consist of the following changes: Detection change: Allow applications to opt out of TDR if they want to (long-running compute scenarios). This class of applications won't hit a TDR for running for extended periods of time as long as the application remains preemptible and allows other tasks to run. Recovery change: Only the hung GPU engine is reset and only the device having submitted the work is put in error; no one else is impacted by the TDR. Repeated TDRs: There

will be no more bugchecks on repeated TDRs in most cases. Applications having caused multiple successive faults won't be allowed to touch the GPU for some amount of time. Non-interactive applications such as Compute applications will be allowed to use as much of the GPU as they need as long as they dont interfere with other applications which need to share the GPU. In order to keep the desktop responsive, applications that negatively interfere with DWM will be penalized by preventing them from accessing the GPU.

Scenarios:

Per-Process TDR: Windows 8 will reset only the offending process which caused the TDR. Other running processes will not be impacted. Compute applications: Compute applications can utilize the GPU and keep running for extended periods of time as long as there are no other applications waiting on GPU resources.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New; Old ID Gfx8084

Device.Graphics.WDDM12.Render.UMDLogging

Target Feature: Device.Graphics.WDDM12.Render

Title: WDDM 1.2 drivers must implement WDDM User-Mode Driver or UMD Logging to aid diagnosability.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM
- Windows 8 Server x64

Description:

A WDDM 1.2 driver must expose the relationship between the Direct3D resources and Video Memory allocations by implementing the UMD logging ETW interfaces. Below are the specific requirements for the drivers:

- UMD must report allocation size specified in CreateResource call, even if size of memory allocated is greater
- UMD must log MapAllocation event for each of its internal allocation and specify purpose of that allocation
- UMD must log MapAllocation event for each renamed surface
- UMD must log MapAllocation for every surface that it packs into an existing surface

- UMD must log MapAllocation for every surface that currently exists in the event of a rundown
- UMD must log MapAllocation in response to CreateResource call, even if no new memory is allocated
- UMD must log UnmapAllocation each time its internal allocation is destroyed
- UMD must log UnmapAllocation each time application destroys and allocation, even if actual memory allocation is not destroyed
- UMD must log UnmapAllocation each time it closes one of the renamed allocations
- UMD must log UnmapAllocation for each surface that is packed into a larger allocation

In addition to logging MapAllocation and UnmapAllocation events as they happen, the Graphics Driver must be able to report all existing mappings between resources and allocations at any point in time.

For more details on the use of the UMD Logging WDDM v1.2 DDI, please refer to the Windows 8 WDK documentation.

Exceptions: Not Specified

Business Justification:

With an increasing number of graphics applications utilizing GPU resources, the diagnosability of graphics performance and video memory related issues has become critical. To get a more actionable breakdown of video memory, the driver needs to expose the relationship between D3D resources and VidMm allocations. With this information added to ETW traces it will be possible to see the video memory allocations from the APIs perspective. For developers, it will clarify memory costs that are currently very hard to see, like internal fragmentation or the impact of rapidly discarding surfaces. It will also enable Microsoft to work better with customers and partners who provide traces for analysis of performance problems. In particular it will help overcome a common blocking point in investigating memory-related performance issues: the application is using too large a working set, but cannot tell which API resources or calls are causing the problem.

Scenarios:

Developer Experience: Graphics application developers and system manufacturers will be able to get insight into video memory usage patterns using tools leveraging ETW tracing. These tools will help the developers optimize the performance and particularly memory footprint of their applications on Windows 8 systems.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New; old ID Gfx8085

Device.Graphics.WDDM12.Render.XPSRasterizationConformance

Target Feature: Device.Graphics.WDDM12.Render

Title: WDDM 1.2 drivers must support XPS Rasterization

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client ARM
- Windows 8 Client x64
- Windows 8 Client x86

Description:

To ensure a quality printing experience on Windows 8 the WDDM1.2 Graphic drivers must support XPS Rasterization

Exceptions: Not Specified

Business Justification:

In Windows 8, the XPS Rasterization component uses Direct2D in hardware rendering mode to rasterize XPS pages into WIC bitmaps whenever the component is invoked on a machine with a WDDM 1.2 driver. The XPS Rasterization component is used by many print device drivers to produce device specific PDLs. The XPSRAS Display Conformance Requirement tests whether a WDDM 1.2 GPU driver produces correct rasterization results when used by Direct2D in the context of the XPS Rasterizer. The XPS Rasterizer is a system component used heavily by Windows print drivers to rasterize an XML Paper Specification (XPS) Print Descriptor Language (PDL). To determine the correctness of rasterization results, a comparison is performed between the results obtained from the XPS Rasterizer when executed on a system with the subject WDDM 1.2 GPU driver, and results obtain from baseline use of the XPS Rasterizer.

Scenarios:

Content printed on Windows 8 to a print device that uses the XPS Rasterization component looks the same as content printed to the same device on Windows 7.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New; Old ID Gfx8090

Device.Graphics.WDDM12.RenderOnly

Description:

The optional feature set implemented by WDDM 1.2 drivers which support only the render specific DDIs.

Related Requirements:

- Device.Graphics.WDDM12.RenderOnly.Base

Device.Graphics.WDDM12.RenderOnly.Base

Target Feature: Device.Graphics.WDDM12.RenderOnly

Title: Requirements for a WDDM graphics adapter to support render functionality

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client ARM
- Windows 8 Client x64
- Windows 8 Client x86

Description:

In Windows 8 and beyond WDDM has been extended to support a WDDM driver that is only responsible for Rendering and Compute DDIs. Such a driver is not allowed to support any Display Scan out capabilities.

A driver is considered a WDDM Render Only driver if one of the following two conditions is met:

1. The driver implements all the DDIs required for a full WDDM driver but reports 0 sources and 0 targets
2. The driver implements all the Render specific DDIs and returns a null pointer for all the Display specific DDIs. The DDIs required for this are called out below

Common WDDM DDIs

These DDI functions are for the common device functionalities such as PnP support and Power support. These functions are required by all WDDM drivers and if not implemented the driver will not be started by Windows. These DDIs are already documented in the WDK.

Required:

DxgkDdiAddDevice

DxgkDdiStartDevice

DxgkDdiStopDevice

DxgkDdiRemoveDevice

DxgkDdiDispatchIoRequest

DxgkDdiSetPowerState

DxgkDdiUnload

Optional:

DdiNotifyAcpiEvent *

* DdiNotifyAcpiEvent DDI function is used to notify graphics drivers on some ACPI events. It is optional for rendering device. On normal WDDM graphics devices, this DDI function will return a flag to indicate dxgkrnl.sys to take some further actions such as reset display mode or poll the connected monitors. For the rendering only device, these flags are not used and must be set to zero

Rendering only DDIs

These DDI functions are for the rendering functionalities.

Required:

DxgkDdiInterruptRoutine

DxgkDdiDpcRoutine

DxgkDdiResetDevice

DxgkDdiCreateDevice

DxgkDdiDestroyAllocation

DxgkDdiDescribeAllocation

DxgkDdiOpenAllocation

DxgkDdiCloseAllocation

DxgkDdiGetStandardAllocationDriverData

DxgkDdiSubmitCommand

DxgkDdiPreemptCommand

DxgkDdiBuildPagingBuffer

DxgkDdiResetFromTimeout

DxgkDdiRestartFromTimeout

DxgkDdiQueryCurrentFence

DxgkDdiControlInterrupt

DxgkDdiDestroyDevice

DxgkDdiPresent

DxgkDdiCreateAllocation

DxgkDdiPatch

DxgkDdiRender

DxgkDdiRenderKm

Optional:

If rendering hardware support swizzling ranger, driver should also supply following two functions

DxgkDdiAcquireSwizzlingRange

DxgkDdiReleaseSwizzlingRange

Exceptions: Not Specified

Business Justification:

Over the years there has been an increasing focus on GPGPU scenarios for doing vast amount of complex mathematical or graphical operations. Some of the common examples of this are graphics rendering for animation, database processing, financial & astronomical calculations and oil and gas exploration. The use of GPGPU has proved benefits in performance, power consumption and cost. This feature makes it easier for an OEM to ship a system specifically targeted for such a use without having the added complexity of supporting a display device

Scenarios:

Server Render Farm User configures a Server system that only contains one GPU installed as a Render only device and does not have any display devices connected to the system. The user accesses this Server by utilizing Remote Desktop Sessions or alternate means. Now the user launches a DirectX application that needs to use the advanced computing power of the GPU for some processing (like graphics render farm) Note: GDI based applications will not work in this case unless, a Full WDDM driver or a Display Only driver is installed along with this OR the user connects via a Remote Desktop session. The application uses the DirectX APIs to enumerate WDDM GPUs and finds the Render only device and utilizes it for its computational needsAdditional GPU for compute User configures a system with 2 GPUs. One a full WDDM driver which support Render and Display functions. The second a WDDM render only driver that only support render functionsThe user has a monitor connected to the full WDDM graphics hardware and uses it to log into the system Now the user launches an application that needs to use the advanced computing power of the GPU for some processing (like graphics render farm) The application uses the DirectX APIs to enumerate WDDM GPUs and finds both the GPUs. The application selects the Render only device and utilizes it for its computational needs but uses the Full WDDM driver for its UI rendering needs.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New; Old ID Gfx8089

Device.Graphics.WDDM12.StandbyHibernateFlags

Description:

The optional feature implemented by WDDM 1.2 drivers supporting the new stand by hibernation flags.

Related Requirements:

- Device.Graphics.WDDM12.StandbyHibernateFlags.StandbyHibernateFlags

Device.Graphics.WDDM12.StandbyHibernateFlags.StandbyHibernateFlags

Target Feature: Device.Graphics.WDDM12.StandbyHibernateFlags

Title: WDDM v1.2 graphics drivers can optionally specify if they support the Windows 8 optimized standby or hibernate flags

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client ARM
- Windows 8 Client x64
- Windows 8 Client x86

Description:

To improve performance on sleep & resume for standby and hibernate, a WDDM 1.2 drivers can utilize the new StandbyHibernateFlags (PreservedDuringStandby, PreservedDuringHibernate, and PartiallyPreservedDuringHibernate). To utilize this feature drivers must set one or more of the StandbyHibernateFlags when enumerating segment capabilities. Doing so indicates that eviction should be skipped during power state transition for the corresponding segments. Verification of memory retention during power transition will be verified for all WDDM 1.2 driver setting a StandbyHibernateFlag.

Exceptions: Not Specified

Business Justification:

Smarter Resource Utilization: Reduction in sleep and resume times by avoiding unnecessary memory eviction and duplication in Integrated Graphics. End user experience: By reducing the response time for resume from standby state, PC users will be less frustrated and can become productive faster with quicker access to their productivity tools.

Scenarios:

Sleep: Whenever a user closes the laptop lid for switching between conference rooms, the user will notice Windows 8 responds quickly to power down and sleep in comparison with the Windows 7

experience. Resume: PCs are quickly accessible to users when resuming from sleep, while remaining silent and consuming the least possible power when not actively working.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New; Old ID Gfx8093

Device.Graphics.XDDM

Description:

The base feature set implemented by drivers developed using the XDDM

Related Requirements:

- Device.Graphics.XDDM.Stability

Device.Graphics.XDDM.Stability

Target Feature: Device.Graphics.XDDM

Title: All XDDM graphics drivers must not generate any hangs or faults under prolonged stress conditions.

Applicable OS Versions:

- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86

Description:

Graphics drivers must function properly, and not generate any hangs or faults throughout the duration of stress testing as specified in the

- "Legacy StressProfile"

To "stress" a graphics driver, Comparative Reliability Analyzer for Software and Hardware (CRASH) launches and terminates other test applications to simulate real-world scenarios.

Exceptions: Not Specified

Business Justification:

The CRASH tests which are exercised through this logo requirement ensure reliability and stability of the graphics driver on the Windows platform. These tests which simulate various productivity, graphics and gaming scenarios exercise various code paths in the graphics stack. They also stress the system to ensure that the stability is not affected with an increased workload.

Scenarios:

The CRASH tests exercise the following scenarios in a stress environment. It ensures that these common end-user scenarios work under stress conditions without affecting the end-user experience:Productivity Desktop Graphics Gaming

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

New; Win7 Graphics-0023

Device.Imaging.Printer.Base

Description: Not Specified

Related Requirements:

- Device.Imaging.Printer.Base.applicationVerifier
- Device.Imaging.Printer.Base.autoConfiguration
- Device.Imaging.Printer.Base.configurationFiles
- Device.Imaging.Printer.Base.connectionRecovery
- Device.Imaging.Printer.Base.connectivityRobustness
- Device.Imaging.Printer.Base.deviceCapabilities
- Device.Imaging.Printer.Base.DocumentProperties
- Device.Imaging.Printer.Base.driverCategory
- Device.Imaging.Printer.Base.DriverEventFiles
- Device.Imaging.Printer.Base.driverIsolation
- Device.Imaging.Printer.Base.driverPackage
- Device.Imaging.Printer.Base.driverStability
- Device.Imaging.Printer.Base.faxModem
- Device.Imaging.Printer.Base.faxTIA592
- Device.Imaging.Printer.Base.faxV34
- Device.Imaging.Printer.Base.GDLFile
- Device.Imaging.Printer.Base.infFile
- Device.Imaging.Printer.Base.jobCancellation
- Device.Imaging.Printer.Base.JSBidiExtender
- Device.Imaging.Printer.Base.metadata
- Device.Imaging.Printer.Base.portMonitors
- Device.Imaging.Printer.Base.PrinterExtension
- Device.Imaging.Printer.Base.printerInterfaces
- Device.Imaging.Printer.Base.PrinterUIApp
- Device.Imaging.Printer.Base.printProcessor
- Device.Imaging.Printer.Base.printRegions
- Device.Imaging.Printer.Base.printTicket

- Device.Imaging.Printer.Base.rendering
- Device.Imaging.Printer.Base.TCPMon
- Device.Imaging.Printer.Base.XPSDrv

Device.Imaging.Printer.Base.applicationVerifier

Target Feature: Device.Imaging.Printer.Base

Title: Printer driver components must comply with Application Verifier test criteria

Applicable OS Versions:

- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows Server 2008 x64
- Windows Server 2008 x86
- Windows Vista Client x86
- Windows Vista Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Server x64

Description:

All user-mode modules (.dll or .exe files) that are part of a printer driver must satisfy the criteria for Application Verifier tests. During driver testing, Application Verifier must be turned on for processes in which the driver modules execute. Application Verifier causes a break when Application Verifier detects an error.

For printer drivers, common applications that must have Application Verifier turned on during testing are the following:

- Splwow64.exe.
- Spoolsv.exe. The process loads the rendering and UI portions of the driver during print testing.
- Printfilterpipelinesvc.exe. The process loads the XPS rendering filters.
- Any vendor-supplied applications that are part of the driver package, such as a custom status monitor. All portions of the driver package that is being signed must be robust.
- Any tests that load driver modules for rendering or UI purposes. The tests commonly load UI and rendering modules

The following Application Verifier tests must be turned on for these processes during testing:

- Heap

- Locks
- Handles
- FilePaths
- HighVersionLie
- DFWChecksNonSetup
- SecurityChecks
- Printing

Design Notes:

For information about Application Verifier, see http://go.microsoft.com/fwlink/?LinkId=58371.

Exceptions: Not Specified

Business Justification:

Application Verifier tests catch common hard-to-diagnose programming errors during runtime and make the problems repeatable and explicit. Enabling Application Verifier is very important to making robustness testing effective.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: July 11, 2008

Comments:

IMAGING-0044

Device.Imaging.Printer.Base.autoConfiguration

Target Feature: Device.Imaging.Printer.Base

Title: Printers must support autoconfiguration

Applicable OS Versions:

- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows Server 2008 x64
- Windows Server 2008 x86
- Windows Vista Client x64
- Windows Vista Client x86
- Windows Server 2008 Release 2 x64

- Windows 8 Server x64

Description:

If the print device supports installable hardware options, such as memory, duplex units, or finisher units, the print driver must support the automatic update of the device configuration that is registered in the system. Device configuration includes print UIs that are associated with the driver and the result of platform APIs that report device configuration. Automatic updates must not require end-user intervention. A device that does not support installable hardware options automatically satisfies this requirement.

Design Notes:

The next version of Windows supports print driver autoconfiguration as defined in the Windows Driver Kit documentation. Although this requirement does not require autoconfiguration as defined in the Windows Driver Kit documentation, it is recommended.

Exceptions: Not Specified

Business Justification:

Autoconfiguration of printers is a platform advancement feature that is provided for independent hardware vendors (IHVs).

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: June 01, 2006

Comments: Not Specified

Device.Imaging.Printer.Base.configurationFiles

Target Feature: Device.Imaging.Printer.Base

Title: Version 4 print drivers provide valid configuration files.

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client x86
- Windows 8 Client x64

Description:

Version 4 print drivers must provide validconfigurationfiles.

- These files must be syntactically valid according to the WDK.
- When supported by the hardware, the configuration files must support the following features:

        - Orientation
        - Duplexing
        - Collate
        - N-Up
        - Paper Size
        - Paper Type
        - Tray
        - Quality
        - Color
        - Stapling
        - Hole-punches
        - Binding
    - GPD or PPD files should define the majority of the features and constraints represented in the driver's PrintCapabilities. JavaScript implementations should supplement these capabilities as needed.
    - JavaScript files must be syntactically valid according to the WDK.
        - They must be included in the driver package and cannot be in a subdirectory in the package.
        - They may be included only with version 4 print drivers
        - They should be designed securely and validate any untrusted data that will be parsed; this includes PrintCapabilities, PrintTicket, XPS Documents and Property Bags.

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2009

Comments: Not Specified

Device.Imaging.Printer.Base.connectionRecovery

Target Feature: Device.Imaging.Printer.Base

Title: A printer must continue to operate normally if a computer becomes unavailable during a print job

Applicable OS Versions:

- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 x64
- Windows Server 2008 x86
- Windows 8 Client x86
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64

Description:

If a computer becomes unavailable during a print job (for example, if the computer enters a sleep state or a network failure or other event interrupts the connection between the computer and printer), the printer must recover so that normal print operations can continue without user interaction.

Design Notes:

Specifically, the printer must not fail into a state in which the end user must manually power cycle the printer or clear a paper jam.

The printer does not have to complete or continue the failed print job when the computer becomes available again. However, when computer-to-printer communication is restored, new print jobs must be able to spool and complete normally.

Exceptions: Not Specified

Business Justification:

Because some users do not have immediate access to or sole control of a printer, it is important that the printer fail gracefully and continue to process print jobs without problems even when a single job is interrupted. This is especially important in corporate scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2006

Comments: Not Specified

Device.Imaging.Printer.Base.connectivityRobustness

Target Feature: Device.Imaging.Printer.Base

Title: A printer must recover from a surprise disconnect or reconnect

Applicable OS Versions:

- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows Server 2008 x64
- Windows Server 2008 x86
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64

Description:

Printers must be able to recover from a surprise disconnect or reconnect from the host or the network, regardless of what the printer is doing at the time. The disconnect or reconnect must leave the system in a consistent state. The host must be able to submit the next print job. It is not acceptable to require a reboot of the host or the printer to re-establish communications.

The existing job does not have to complete after the reconnect occurs.

Design Notes:

The printer does not have to finish or resume the current print job or any print jobs already in the printers memory. The printer must behave normally after the computer reconnects. All print jobs must print normally after communication is restored.

Exceptions: Not Specified

Business Justification:

This requirement improves device and driver robustness to reduce the need for user or administrator intervention.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: June 01, 2006

Comments: Not Specified

Device.Imaging.Printer.Base.deviceCapabilities

Target Feature: Device.Imaging.Printer.Base

Title: A printer must correctly support the DeviceCapabilities and GetDeviceCaps application programming interfaces (APIs) based on the guidelines in the Windows Driver Kit

Applicable OS Versions:

- Windows 8 Client x64

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64

Description:

This requirement clarifies the use of existing WLK tests. The Print Driver Device Capabilities test determines whether the printer driver returns the correct information for the **DeviceCapabilities** and **GetDeviceCaps** API calls.

For more information, see http://msdn.microsoft.com/en-us/library/dd183552(v=vs.85).aspx

and

http://msdn.microsoft.com/en-us/library/ff566075(v=VS.85).aspx.

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: June 01, 2006

Comments: Not Specified

Device.Imaging.Printer.Base.DocumentProperties

Target Feature: Device.Imaging.Printer.Base

Title: A driver must implement the DocumentProperty API according to the guidelines in the Windows Driver Kit

Applicable OS Versions:

- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64

Description:

This test clarifies the use of existing WLK tests.

The Document Properties test exercises a printer driver's user interface (UI). The test calls the **DocumentProperties** API by using various well-formed and malformed parameters. The test then validates the results.

For more information, see http://msdn.microsoft.com/en-us/library/ff562200(v=vs.85).aspx.

Exceptions: Not Specified

Business Justification:

This requirement helps ensure that drivers are implemented correctly.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: June 01, 2006

Comments: Not Specified

Device.Imaging.Printer.Base.driverCategory

Target Feature: Device.Imaging.Printer.Base

Title: Version 4 printer drivers must declare a valid printer category

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 8 Server x64

Description:

All V4 printer drivers must declare a validDriverCategory in their driver manifest.

V3 drivers are not required to declare a category. If a V3 driver does declare a DriverCategory, it must be valid value.

The DriverCategory must be one of the following values:

- PrintFax.Printer
- PrintFax.Fax
- PrintFax.Printer.File
- PrintFax.Printer.Virtual
- PrintFax.Printer.Service

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: July 10, 2008

Comments: Not Specified

Device.Imaging.Printer.Base.DriverEventFiles

Target Feature: Device.Imaging.Printer.Base

Title: Driver Event files are implemented according to the guidance in the WDK

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client x86
- Windows 8 Client x64

Description:

Driver Event files are implemented according to the guidance in the WDK

- Driver Event files are syntactically valid

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2009

Comments: Not Specified

Device.Imaging.Printer.Base.driverIsolation

Target Feature: Device.Imaging.Printer.Base

Title: A printer driver that supports driver isolation must do so as defined in Windows Driver Kit

Applicable OS Versions:

- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64

- Windows Server 2008 x64
- Windows Server 2008 x86

Description:

Print drivers must support driver isolation as defined in the Windows Driver Kit.

With driver isolation, the printer executes print-related plug-ins such as drivers and print processors out of the spooler process. This increases print system stability.

Design Notes:

The Print Driver Sandboxing technical whitepaper is planned for a future date. Please send requests to prninfo@microsoft.com.

Exceptions: Not Specified

Business Justification:

Driver isolation improves system robustness.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2009

Comments: Not Specified

Device.Imaging.Printer.Base.driverPackage

Target Feature: Device.Imaging.Printer.Base

Title: Print device drivers must support package installation infrastructure

Applicable OS Versions:

- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64
- Windows Vista Client x86
- Windows Vista Client x64

Description:

A driver that depends on other driver packages must declare that it is package-aware and use the new dependency definition keywords.

Design Notes:

Vendors must use the package installation infrastructure as defined in the Windows Driver Kit.

Exceptions: Not Specified

Business Justification:

The new package infrastructure enables several scenarios, such as Secure Point and Print, printer migration, and system backup.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: June 01, 2007

Comments:

imaging-0009

Device.Imaging.Printer.Base.driverStability

Target Feature: Device.Imaging.Printer.Base

Title: Printer driver components do not cause a break in any process in which they are loaded

Applicable OS Versions:

- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows Server 2008 x64
- Windows Server 2008 x86
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64
- Windows Vista Client x64
- Windows Vista Client x86

Description:

A driver must not cause a break in the spooler service (spoolsv.exe) or in any other process in which the driver's components are loaded. Breaks must not occur in the driver modules themselves or indirectly through leaving the process in an inconsistent state, such as by corrupting process memory.

Exceptions: Not Specified

Business Justification:

Improving device and device driver robustness reduces the need for user or administrator intervention.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: June 01, 2007

Comments: Not Specified

Device.Imaging.Printer.Base.faxModem

Target Feature: Device.Imaging.Printer.Base

Title: Fax modem successfully sets up a connection and transmits pages

Applicable OS Versions:

- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86

Description:

A fax modem must meet the following requirements:

The modem must be able to send and receive five faxes of 10 pages in various document formats.

When a service shutdown or hibernate is requested, the modem must abort all ongoing calls (both send and receive).

Exceptions: Not Specified

Business Justification:

This requirement helps ensure that faxes are implemented correctly.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: June 01, 2006

Comments:

Imaging 0003

Device.Imaging.Printer.Base.faxTIA592

Target Feature: Device.Imaging.Printer.Base

Title: Fax Class 2.0 implementations must comply with the TIA-592 command set

Applicable OS Versions:

- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86

Description:

If the device supports Fax Class 2.0, the fax must comply with the TIA-592 standard.

Exceptions: Not Specified

Business Justification:

This requirement helps ensure compliance with industry standards.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: June 01, 2006

Comments: Not Specified

Device.Imaging.Printer.Base.faxV34

Target Feature: Device.Imaging.Printer.Base

Title: Any Fax V.34 implementation must comply with ITU-T recommendation V.34

Applicable OS Versions:

- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86

Description:

A fax that supports V.34 must adhere to International Telecommunications Union Telecommunication Standardization Sector (ITU-T) recommendation V.34: "A modem operating at data signaling rates of up to 33,600 bit/s for use on the general switched telephone network and on leased point-to-point 2-wire telephone-type circuits".

Exceptions: Not Specified

Business Justification:

This requirement helps ensure compliance with industry standards.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: June 01, 2006

Comments: Not Specified

Device.Imaging.Printer.Base.GDLFile

Target Feature: Device.Imaging.Printer.Base

Title: GDL files must use correct syntax according to the guidelines in the Windows Driver Kit

Applicable OS Versions:

- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64

Description:

This requirement clarifies the use of existing WLK tests. The Generic Description Language (GDL) Correctness Test determines whether GDL files use correct syntax according to the WDK.

For more information, see http://msdn.microsoft.com/en-us/library/ff549787(v=VS.85).aspx and http://msdn.microsoft.com/en-us/library/ff563355(v=VS.85).aspx.

Exceptions: Not Specified

Business Justification:

GDL files must be syntactically correct

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: June 01, 2006

Comments: Not Specified

Device.Imaging.Printer.Base.infFile

Target Feature: Device.Imaging.Printer.Base

Title: INF files must use the correct syntax according to the guidelines in the Windows Driver Kit

Applicable OS Versions:

- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64

Description:

This requirement clarifies the use of existing Windows Hardware Certification Kit tests.

INFGate determines whether INF files andv4 manifest use correct syntax according to the WDK.

Version 4 print drivers use version 4 INFs and manifest files.

V4 driver INF must:

- Define a PrinterDriverID for each driver in the INF to indicate when drivers are compatible for the sake of printer sharing
- PrinterDriverID must be GUID
- Set a DataFile as a GPD or PPD file
- Use only the following DestinationDirs: DriverStore, 66000; or Color directory, 66003
- Must specify a filter xml pipeline config file named *-pipelineconfig.xml, OR specify RequiredClass in the v4 driver manifest

V4 driver manifest files must:

- Define a PrinterDriverID for each driver in the manifest to indicate when drivers are compatible for the sake of printer sharing.
- PrinterDriverID must be GUID
- Set a Data File as a GPD or PPD file

V4 drivers must not:

- Utilize any of the following INF directives ClassInstall32, ClassInstall32.Services, DDInstall.Services, DDInstall.HW, DDInstall.CoInstallers, DDInstall.FactDef, DDInstall.LogConfigOverride, DDInstall.Interfaces, InterfaceInstall32, DefaultInstall, DefaultInstall.Services, AddReg, DelReg, DelFiles, RenFiles, AddService, DelService, AddInterface, BitReg, LogConfig, UpdateInis, UpdateIniFields, or Ini2Reg.

Version 3 INF files must use correct syntax according to the WDK.

For more information on v3 INF files, see http://msdn.microsoft.com/en-us/library/ff560902(v=VS.85).aspx and http://msdn.microsoft.com/en-us/library/ff563414(v=VS.85).aspx.

Exceptions: Not Specified

Business Justification:

This requirement helps ensure that INF files are written correctly.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: June 01, 2006

Comments: Not Specified

Device.Imaging.Printer.Base.jobCancellation

Target Feature: Device.Imaging.Printer.Base

Title: A printer must handle software problems or print job cancellations gracefully

Applicable OS Versions:

- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows Server 2008 x64
- Windows Server 2008 x86
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64

Description:

If a driver encounters a software-related problem when the driver spools or de-spools a print job, the driver must ensure that jobs can successfully print in the future.

Printers also must be able to handle a job being canceled at any time without wasting consumables, such as print media, or entering a state that requires human intervention to print.

Exceptions: Not Specified

Business Justification:

Poorly written drivers can stop print queues on print servers that are under heavy load, preventing any forward progress. This requirement improves device and device driver robustness to reduce the need for user or administrator intervention.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: July 10, 2008

Comments: Not Specified

Device.Imaging.Printer.Base.JSBidiExtender

Target Feature: Device.Imaging.Printer.Base

Title: JavaScript Bidi Extenders are implemented according to the guidance in the WDK

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

JavaScript Bidi Extenders are implemented according to the guidance in the WDK

- They must be included in the driver package and cannot be in a subdirectory in the package.
- They may only be included with version 4 print drivers.
- They should be designed securely and validate any untrusted data that will be parsed.
- They must be referenced in the v4 manifest files.

They must use syntactically valid JavaScript, implemented according to the WDK.

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2009

Comments: Not Specified

Device.Imaging.Printer.Base.metadata

Target Feature: Device.Imaging.Printer.Base

Title: Printer and MFP driver components must include an authored device metadata document

Applicable OS Versions:

- Windows 8 Client x64
- Windows 7 Client x86

- Windows 7 Client x64
- Windows 8 Client x86
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64

Description:

Devices that provide DeviceStage metadata that includes a photorealistic image of the device, company logo, and basic task information must do so correctly.

Tasks must only appear when the tasks are available. For example, scanner tasks must not appear on a printer-only connection, and Internet tasks must not appear if the Internet is unavailable.

Design Notes:

More information available in the Windows SDK and in the WDK. Please send questions to prninfo@microsoft.com.

Exceptions: Not Specified

Business Justification:

This requirement helps ensure that devices work with Windows.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2009

Comments: Not Specified

Device.Imaging.Printer.Base.portMonitors

Target Feature: Device.Imaging.Printer.Base

Title: A v4 print driver must use an inbox provided port monitor.

Applicable OS Versions:

- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows Server 2008 x64
- Windows Server 2008 x86
- Windows 8 Server x64
- Windows Vista Client x64
- Windows Vista Client x86

Description:

- Custom port monitors are not allowed for v4 printer drivers. They must use an inbox provided port monitor.
- V3 printer driver, print processor, or language monitor may use a custom port monitor, but must be able to de-spool print jobs when the device is configured in a queue that uses any port monitor

Exceptions: Not Specified

Business Justification:

Improving device and device driver robustness reduces the need for user or administrator intervention.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: June 01, 2007

Comments: Not Specified

Device.Imaging.Printer.Base.PrinterExtension

Target Feature: Device.Imaging.Printer.Base

Title: Printer extensions are implemented according to the guidance in the WDK

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client x86
- Windows 8 Client x64

Description:

Printer extensions are implemented according to the guidance in the WDK

- They must run in the appropriate integrity level
- They should be designed securely and validate any untrusted data that will be parsed; this includes PrintCapabilities, PrintTicket, XPS Documents and Property Bags.
- They must not register system services on installation
- They must register with the print system for any events they should be invoked for
- They must communicate with the print system using the prescribed interfaces

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2009

Comments: Not Specified

Device.Imaging.Printer.Base.printerInterfaces

Target Feature: Device.Imaging.Printer.Base

Title: A printer device must support at least one non-legacy interface

Applicable OS Versions:

- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows Server 2008 Release 2 x64
- Windows 8 Server x64
- Windows Vista Client x64
- Windows Vista Client x86

Description:

Printers and multi-function print (MFP) devices must be able to connect to the computer through a non-legacy interface such as Ethernet, USB, IEEE 1394, or Bluetooth. Printing devices may offer IEEE 1284 (parallel) ports in addition to a non-legacy connector.

Exceptions: Not Specified

Business Justification:

Non-legacy ports provide a much better user experience than legacy ports.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: June 01, 2006

Comments: Not Specified

Device.Imaging.Printer.Base.PrinterUIApp

Target Feature: Device.Imaging.Printer.Base

Title: Printer UI apps are implemented according to the guidance in the WDK

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

Printer UI apps are implemented according to the guidance in the WDK

- They must run in the appropriate integrity level
- They should be designed securely and validate any untrusted data that will be parsed; this includes PrintCapabilities, PrintTicket, XPS Documents and Property Bags.
- They must not register system services on installation
- They must register with the print system for any events they should be invoked for
- They must communicate with the print system using the prescribed interfaces

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2009

Comments: Not Specified

Device.Imaging.Printer.Base.printProcessor

Target Feature: Device.Imaging.Printer.Base

Title: Print processors must be implemented based on the guidelines in the Windows Driver Kit

Applicable OS Versions:

- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64

Description:

This requirement clarifies the use of existing WLK tests. The Print Processor API test helps ensure that print processors are implemented based on WDK guidelines.

All print processors must support the following endpoints:

- **OpenPrintProcessor**
- **ClosePrintProcessor**
- **ControlPrintProcessor**
- **EnumPrintProcessorDatatypesW**
- **PrintDocumentOnPrintProcessor**
- **GetPrintProcessorCapabilities**

For more information, seehttp://msdn.microsoft.com/en-us/library/ff566084(v=VS.85).aspx.

Exceptions: Not Specified

Business Justification:

This requirement helps ensure that drivers are implemented correctly.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: June 01, 2006

Comments: Not Specified

Device.Imaging.Printer.Base.printRegions

Target Feature: Device.Imaging.Printer.Base

Title: A printer must support printable regions accurately

Applicable OS Versions:

- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows 8 Client ARM
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64
- Windows Vista Client x86
- Windows Vista Client x64

Description:

The printer must be able to print output for the entire area that appears in the printable region that the user can select in the Windows UI.

Design Notes:

Note that if the printer supports a "borderless" printing feature, this restriction may be relaxed to allow for devices whose printable region extends beyond the dimensions of the physical media sheet. In these cases, the printer must be able to print output to the extent of the page dimension.

This test applies to all paper sizes that the printer physically supports. If the printer supports auto-scaling and the UI exposes additional paper sizes to the user that cannot be physically loaded into the printer, the printer must maintain correct aspect ratios during resizing. In this context, "auto-scaling" is any feature in the hardware or driver that changes the print job to fit on an available paper size without user interaction or warning.

If the printer does not support printing on a physical medium that is at least 4" x 4" in size, the printer is exempt from this requirement.

Exceptions: Not Specified

Business Justification:

These are the basic requirements for writing a printer driver which works well with windows.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: June 01, 2007

Comments: Not Specified

Device.Imaging.Printer.Base.printTicket

Target Feature: Device.Imaging.Printer.Base

Title: Printer driver supports PrintTicket/PrintCapabilities

Applicable OS Versions:

- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows Server 2008 x64
- Windows Server 2008 x86
- Windows Vista Client x86
- Windows Vista Client x64
- Windows Server 2008 Release 2 x64

- Windows 8 Server x64

Description:

Print devices must fully support the **PrintTicket** and **PrintCapabilities** objects. Depending on your print driver implementation, this may or may not require implementing certain **PrintTicket** or **PrintCapabilities** interfaces. For more information, see the WDK documentation.

Printer drivers must support the public Print Schema element types for each keyword if the printer driver or print device includes the specified functionality. Printer drivers must also support the public Print Schema element types for each keyword if the appropriate functionality is present but non-configurable. The element types that the printer driver must support for each keyword include all such Features, Options, ParameterDef, ParameterRef, Property, and ScoredProperty that the public Print Schema definition contains. Printer drivers do not have to support public Print Schema keywords if the printer driver or print device does not include the specified functionality.

Printer drivers must support the following basic Print Schema element types:

- **DocumentCollate**
- **JobCopiesAllDocuments**
- **JobDuplexAllDocumentsContiguously**
- **PageColorManagement**
- **PageImageableSize**
- **PageMediaSize**
- **PageMediaType**
- **PageOrientation**
- **PageOutputColor**
- **PageResolution**
- **PageICMRenderingIntent**
- One of the following: **JobInputBin**, **DocumentInputBin**, or **PageInputBin**

Exceptions: Not Specified

Business Justification:

Print Ticket is a key feature that will advance the print platform.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: July 12, 2008

Comments: Not Specified

Device.Imaging.Printer.Base.rendering

Target Feature: Device.Imaging.Printer.Base

Title: A printer must correctly render print jobs

Applicable OS Versions:

- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64

Description:

This requirement clarifies the use of the following existing WLK tests to ensure that printers correctly render print jobs:

- Pgremlin/Pgremlin2

This test produces numerous pages of output that include shapes, gradient fills, alpha blends and some complex fonts. The test checks Device Driver Interface (DDI) calls that a driver can make.

- WinFX Markup Content Rendering Test

The WinFX Markup Content Rendering test loads eight WinFX XAML files and produces output on the specified printer.

- Photo Print Test

The Photo Print Test prints landscape- and portrait-oriented photos to exercise printer functionality.

For more information about Pgremlin, seehttp://msdn.microsoft.com/en-us/library/ff566081(v=VS.85).aspx.

For more information about Pgremlin2, seehttp://msdn.microsoft.com/en-us/library/ff566079(v=VS.85).aspx.

For more information about the WinFX Markup Content Rendering Test,seehttp://msdn.microsoft.com/en-us/library/ff569275(v=VS.85).aspx.

For more information about the Photo Print Test, see http://msdn.microsoft.com/en-us/library/ff565234(v=VS.85).aspx.

Exceptions: Not Specified

Business Justification:

This requirement helps ensure that drivers are implemented correctly.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: June 01, 2006

Comments: Not Specified

Device.Imaging.Printer.Base.TCPMon

Target Feature: Device.Imaging.Printer.Base

Title: Network-connected printers that support Port 9100 printing with the Microsoft Standard Port Monitor (TCPMon) must provide rich status for the device

Applicable OS Versions:

- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows Server 2008 Release 2 x64
- Windows 8 Server x64
- Windows Vista Client x64
- Windows Vista Client x86

Description:

If the printer uses a network interface port connection and supports Port 9100 printing (raw printing) with the Microsoft Standard Port Monitor (TCPmon), it must also support Simple Network Management Protocol (SNMP), Host Resource Management Information Base (MIB), and Common Printer MIB, and Printer Port Monitor MIB 1.0 (IEEE-ISTO5107.1-2005) so that the operating system can provide rich status for the device.

Exceptions: Not Specified

Business Justification:

Network printers must follow a standard protocol to work correctly with Windows.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: June 27, 2008

Comments: Not Specified

Device.Imaging.Printer.Base.XPSDrv

Target Feature: Device.Imaging.Printer.Base

Title: A printer must have a driver that correctly implements XPSDrv printer driver architecture

Applicable OS Versions:

- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows 8 Server x64
- Windows 8 Client ARM
- Windows Server 2008 Release 2 x64

Description:

XPSDrv is a new extension to the printer driver architecture. A driver that implements the XPSDrv printer driver architecture must do the following:

- Implement a Version 3 driver architecture configuration module. The configuration module must support **PrintTicket** and **PrintCapabilities** objects for all functionality.
- Include a valid filter pipeline configuration file.

A driver that implements the XPSDrv printer driver architecture must not do the following:

- Implement a Version 3 driver architecture rendering module.
- Implement a print processor.

If the XPSDrv driver supports a print device that can consume the XPS Document format as a printer description language (PDL), no filters are required. Otherwise, or if the driver supplies filters, the driver must satisfy the following requirements:

- The first filter in the XPSDrv driver filter pipeline must consume the XPS Document format.
- The XPSDrv driver filter pipeline must produce a PDL that the target print device can interpret.
- Filters in the XPSDrv driver filter pipeline must do the following:
    - Conform to the rendering rules in the XML Paper Specification.
    - Conform to the PrintTicket processing rules in the XML Paper Specification.
- Filters in the XPSDrv driver filter pipeline must not do the following:

- Call into the Common Language Runtime (CLR) or the WinFX run-time components for any functionality.
- Display user interface (UI) content.

XPSDrv must fully support the **PrintTicket** and **PrintCapabilities** objects. XPSDrv drivers must also support the following additional keywords in the described under the the printTicket requirement.

- **PageScaling**
- **JobDigitalSignatureProcessing**
- **PagePhotoPrintingIntent**
- **PageOutputQuality**

For Windows 7, Windows Server 2008 R2 and later,a printer must have a driver that correctly implements XPSDrv printer driver architecture. An XPSDrv driver is not required for Windows Vista Basic, and Windows Server 2008 and earlier.

Exceptions: Not Specified

Business Justification:

This is needed for compatibility with Windows.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: July 12, 2008

Comments:

Imaging-0010

Device.Imaging.Printer.Cluster

Description: Not Specified

Related Requirements:

- Device.Imaging.Printer.Cluster.cluster

Device.Imaging.Printer.Cluster.cluster

Target Feature: Device.Imaging.Printer.Cluster

Title: Print driver implements cluster requirements

Applicable OS Versions:

- Windows 7 Client x86

- Windows 7 Client x64
- Windows Server 2008 x64
- Windows Server 2008 x86
- Windows Server 2008 Release 2 x64

Description:

Many printers may be hosted on clustered print servers. To work properly on a cluster, print drivers must:

- Use only one print processor binary, defined in the INF
- Implement InitializePrintMonitor2 on any custom port monitors used and access the registry only through the provided interface.

Design Notes:

Print Processor

Print processor binaries must be a single file and be defined in the driver INF using the PrintProcessor directive. Other print processor binaries may not migrate or work properly in cluster failover. Print processors may call into other DLLs if they are:

- A system DLL
- In the print processor's directory
- A print driver file in the driver's directory
- AND it gracefully handles cases where a print driver file no longer exists.

Port Monitors

The InitializePrintMonitor2 interface provides the port monitor with rich information about the system environment (local-only, cluster-only, or both). It helps isolate the port monitor from potential compatibility or migration issues. Port monitors should not attempt to access the registry outside this interface.

Exceptions: Not Specified

Business Justification:

TBA

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2006

Comments: Not Specified

Device.Imaging.Printer.OXPS

Description: Not Specified

Related Requirements:

- Device.Imaging.Printer.OXPS.OXPS

Device.Imaging.Printer.OXPS.OXPS

Target Feature: Device.Imaging.Printer.OXPS

Title: V4 drivers that support OpenXPS must be implemented according to the rules specified in the WDK.

Applicable OS Versions:

- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64

Description:

For Windows 8, a correctly implemented version 4 print driver will satisfy the XPS requirements. The v4 driver can support either MSXPS or Open XPS.

V4 print drivers that support OpenXPS, either exclusively or in dual-format support mode with XPS, must satisfy the following requirements:

- Driver manifest must specify either XpsFormat=OpenXPS, XpsFormat=OpenXPS, XPS or XpsFormat=XPS, OpenXPS in the DriverRender section
- The first filter must be able to consume OpenXPS document format when provided such by the print filter pipeline manager
- All filters must conform to the rendering rules in the ECMA Open XML Paper Specification v. 1.0 (Ecma 388)
- All filters must conform to the PrintTicket processing rules in the PrintSchema Specification 1.0.
- The v4 driver filter pipeline must produce a PDL that the target print device can interpret.
- Filters in the v4 driver pipeline supporting OpenXPS must NOT do the following:

    - Call into the Common Language Runtime (CLR) or the WinFX run-time components for any functionality.
    - Display user interface (UI) content.
- OpenXPS supporting drivers must adhere to all other v4 rules. Dual-format drivers must adhere to both OpenXPS and MSXPS requirements and successfully handle either format.

Exceptions: Not Specified

Business Justification:

This is needed for compatibility with Windows.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: July 12, 2008

Comments:

Imaging-0010

Device.Imaging.Printer.TCPIP

Description: Not Specified

Related Requirements:

- Device.Imaging.Printer.TCPIP.CompatID

Device.Imaging.Printer.TCPIP.CompatID

Target Feature: Device.Imaging.Printer.TCPIP

Title: Printers must implement a Compatible ID in their IEEE1284 string according to the rules specified in the WDK.

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64

Description:

Printers must implement a Compatible ID in their IEEE1284 string for devices that connect over USB, WSD and TCP/IP using the Port Monitor MIB.

The Compatible ID must indicate:

- Manufacturer Name or Code
- Device Class Description
    - Recommended:
        - Include PDL
        - any other relevant device class information
        - Example: Fabrikam_XPS_A3_laser

Devices that receive the Windows 7 logo before June 1,2012 are exempt from this requirement.

Link to Compatible ID Whitepaper: http://msdn.microsoft.com/en-us/windows/hardware/gg463313.aspx

Exceptions: Not Specified

Business Justification:

Using Compatible ID in Printers will enhance device coverage. Users will be more likely to find print drivers for devices that implement a Compatible ID.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: June 01, 2012

Comments: Not Specified

Device.Imaging.Printer.USB

Description: Not Specified

Related Requirements:

- Device.Imaging.Printer.USB.CompatID
- Device.Imaging.Printer.USB.JSBidiExtender

Device.Imaging.Printer.USB.CompatID

Target Feature: Device.Imaging.Printer.USB

Title: Printers must implement a Compatible ID in their IEEE1284 string according to the rules specified in the WDK.

Applicable OS Versions:

- Windows 8 Client x64
- Windows 7 Client x86

- Windows 7 Client x64
- Windows 8 Client x86
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64

Description:

Printers must implement a Compatible ID in their IEEE1284 string for devices that connect over USB, WSD and TCP/IP using the Port Monitor MIB.

The Compatible ID must indicate:

- Manufacturer Name or Code
- Device Class Description
    - Recommended:
        - Include PDL
        - any other relevant device class information
        - Example: Fabrikam_XPS_A3_laser

Devices that receive the Windows 7 logo before June 1, 2012 are exempt from this requirement.

Link to Compatible ID Whitepaper: http://msdn.microsoft.com/en-us/windows/hardware/gg463313.aspx

Exceptions: Not Specified

Business Justification:

Using Compatible ID in Printers will enhance device coverage. Users will be more likely to find print drivers for devices that implement a Compatible ID.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: June 01, 2012

Comments: Not Specified

Device.Imaging.Printer.USB.JSBidiExtender

Target Feature: Device.Imaging.Printer.USB

Title: USB JavaScript Bidi Extenders are implemented according to the guidance in the WDK

Applicable OS Versions:

- Windows 8 Server x64

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

USB JavaScript Bidi Extenders are implemented according to the guidance in the WDK

- They must be included in the driver package and cannot be in a subdirectory in the package.
- They may only be included with version 4 print drivers.
- They should be designed securely and validate any untrusted data that will be parsed.
- They must be referenced in the v4 manifest files.

They must use syntactically valid JavaScript, implemented according to the WDK.

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2009

Comments: Not Specified

Device.Imaging.Printer.WSD

Description: Not Specified

Related Requirements:

- Device.Imaging.Printer.WSD.CompatID
- Device.Imaging.Printer.WSD.Rally
- Device.Imaging.Printer.WSD.WSPrint

Device.Imaging.Printer.WSD.CompatID

Target Feature: Device.Imaging.Printer.WSD

Title: Printers must implement a Compatible ID in their IEEE1284 string according to the rules specified in the WDK.

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 8 Server x64

Description:

Printers must implement a Compatible ID in their IEEE1284 string for devices that connect overUSB, WSD and TCP/IP using the Port Monitor MIB.

The Compatible ID must indicate:

- Manufacturer Name or Code
- Device Class Description
    - Recommended:
        - Include PDL
        - any other relevant device class information
        - Example: Fabrikam_XPS_A3_laser

Devices that receive the Windows 7 logo before June 1, 2012 are exempt from this requirement.

Link to Compatible ID Whitepaper: http://msdn.microsoft.com/en-us/windows/hardware/gg463313.aspx

Exceptions: Not Specified

Business Justification:

Using Compatible ID in Printers will enhance device coverage. Users will be more likely to find print drivers for devices that implement a Compatible ID.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: June 01, 2012

Comments: Not Specified

Device.Imaging.Printer.WSD.Rally

Target Feature: Device.Imaging.Printer.WSD

Title: Network-attached printers and MFPs must implement the Windows Rally Technologies

Applicable OS Versions:

- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows Server 2008 Release 2 x64
- Windows 8 Server x64

Description:

Network-connected printers, scanners, and MFPs that implement any of the following Windows Rally requirements must comply completely with the implemented requirement:

- Connect-0098 (optional): Devices that implement 802.3 or 802.11 may implement the Link Layer Topology Discovery (LLTD) protocol. This requirement applies only if the device implements LLTD. LLTD implementation is not required.
- Connect-0099 (required): A 802.11 network-enabled device that operates as a station must implement WCN-NET and meet basic 802.11 requirements.
- Connect-0100 (required): A network-enabled device that implements Plug and Play Extensions (PnP-X) must comply with the specification.
- IMAGING-0004 (required): A network-connected device must implement Windows network-connected Web Services for Devices (WSD) correctly for the device type.

Design Notes:

For more information, see the PnP-X: Plug and Play Extensions for Windows specification at http://go.microsoft.com/fwlink/?LinkID=123172,

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2008

Comments: Not Specified

Device.Imaging.Printer.WSD.WSPrint

Target Feature: Device.Imaging.Printer.WSD

Title: Network-connected printers must implement Windows network-connected Web Services for Devices

Applicable OS Versions:

- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64

Description:

Printers or MFP devices must support the Device Profile for Web Services and the Print Web Development Partnership (WDP) by using the Microsoft WSD Port Monitor (WSDMon).

The printer or MFP device must support the following events that the Print Service Definition includes:

- **PrinterElementsChangeEvent**
- **PrinterStatusSummaryEvent**
- **PrinterStatusConditionEvent**
- **PrinterStatusConditionClearedEvent**
- **JobStatusEvent**
- **JobEndStateEvent**

In addition, the printer or MFP device must support all required operations that Section 5, Table 2 of the Print Service Definition outlines.

Design Notes:

For more information, see the Print Service Definition 1.0 at http://go.microsoft.com/fwlink/?LinkId=109841.

Exceptions: Not Specified

Business Justification:

Web Services for Devices provides an it just works scenario for network printers.

Scenarios:

IMAGING 0004

Success Metric: Not Specified

Enforcement Date: July 12, 2008

Comments: Not Specified

Device.Imaging.Printer.XPS

Description: Not Specified

Related Requirements:

- Device.Imaging.Printer.XPS.XPS

Device.Imaging.Printer.XPS.XPS

Target Feature: Device.Imaging.Printer.XPS

Title: A printer must have a driver that correctly implements XPSDrv printer driver architecture

Applicable OS Versions:

- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64

Description:

For Windows 8, a correctly implemented version 4 print driver will satisfy this requirement. The v4 driver can support either MSXPS or Open XPS.

V4 print drivers that support MSXPS, either exclusively or in dual-format support mode with OpenXPS, must satisfy the following requirements:

- Driver manifest must specify either XpsFormat=OpenXPS, XpsFormat=OpenXPS, XPS or XpsFormat=XPS, OpenXPS in the DriverRender section
- Thefirst filter must be able to consume XPS document format when provided such by the print filter pipeline manager
- All filters must conform to the rendering rules in the XML Paper Specification
- All filters must conform to the PrintTicket processing rules in the PrintSchema Specification 1.0?
- The v4 driver filter pipeline must produce a PDL that the target print device can interpret.
- Filters in the v4 driver pipeline supporting XPS must NOT do the following:
    - Call into the Common Language Runtime (CLR) or the WinFX run-time components for any functionality.
    - Display user interface (UI) content.
- XPS supporting drivers must adhere to all other v4 rules. Dual-format drivers must adhere to both OpenXPS and MSXPS requirements and successfully handle either format.

For Windows 7, Windows Server 2008 R2 and later, a printer must have a driver that correctly implements XPSDrv printer driver architecture. An XPSDrv driver is not required for Windows Vista Basic, and Windows Server 2008 and earlier.

A v3 driver that implements the XPSDrv printer driver architecture must do the following:

- Implement a Version 3 driver architecture configuration module. The configuration module must support PrintTicket and PrintCapabilities objects for all functionality.
- Include a valid filter pipeline configuration file.

A driver that implements the XPSDrv printer driver architecture must not do the following:

- Implement a GDI rendering module. Is this what we used to say?
- Implement a print processor.

If the XPSDrv driver supports a print device that can consume the XPS Document format as a printer description language (PDL), no filters are required. Otherwise, or if the driver supplies filters, the driver must satisfy the following requirements:

- The first filter in the XPSDrv driver filter pipeline must consume the XPS Document format.
- The XPSDrv driver filter pipeline must produce a PDL that the target print device can interpret.
- Filters in the XPSDrv driver filter pipeline must do the following:
    - Conform to the rendering rules in the XML Paper Specification.
    - Conform to the PrintTicket processing rules in the XML Paper Specification.
- Filters in the XPSDrv driver filter pipeline must not do the following:
    - Call into the Common Language Runtime (CLR) or the WinFX run-time components for any functionality.
    - Display user interface (UI) content.

XPSDrv must fully support the PrintTicket and PrintCapabilities objects. XPSDrv drivers must also support the following additional keywords in the described under the printTicket requirement.

- PageScaling
- JobDigitalSignatureProcessing
- PagePhotoPrintingIntent
- PageOutputQuality

Exceptions: Not Specified

Business Justification:

This is needed for compatibility with Windows.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: July 12, 2008

Comments:

Imaging-0010

Device.Imaging.Scanner.Base

Description: Not Specified

Related Requirements:

- Device.Imaging.Scanner.Base.dataTransfer
- Device.Imaging.Scanner.Base.driverInstallation
- Device.Imaging.Scanner.Base.errorHandling
- Device.Imaging.Scanner.Base.metadata
- Device.Imaging.Scanner.Base.MFPmultiplePorts
- Device.Imaging.Scanner.Base.RawFileFormat
- Device.Imaging.Scanner.Base.scannerInterfaces
- Device.Imaging.Scanner.Base.statusMessages
- Device.Imaging.Scanner.Base.wia20
- Device.Imaging.Scanner.Base.WIAArchitecture
- Device.Imaging.Scanner.Base.WIAProperties

Device.Imaging.Scanner.Base.dataTransfer

Target Feature: Device.Imaging.Scanner.Base

Title: WIA drivers must support specific data transfer implementations

Applicable OS Versions:

- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 x64
- Windows Server 2008 x86
- Windows 8 Client x86
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64

Description:

Windows Image Acquisition (WIA) drivers must do the following:

- Use only the **Write**, **Seek**, and **SetSizeIStream** methods during downloads.

- Use only the **Read**, **Seek**, and **StatIStream** methods during uploads.
- Send valid **lPercentComplete** values during calls to the **IWiaMiniDrvTransferCallback::SendMessage**<element>. **lPercentComplete** must be between 0 and 100 inclusive.
- Send correct **ulTransferredBytes** values during calls to **IWiaMiniDrvTransferCallback::SendMessage**.
- Append new data to the end of streams that the **IWiaMiniDrvTransferCallback::GetNextStream**<element> receives if the streams are not empty.
- Return success values from the I**Wia**(SYLVIA PEARCE: Should this be IWia (as it is in other instances)?) **MiniDrv::drvAcquireItemData** method (SYLVIA PEARCE: If this isn't correct, please replace it with the correct element.) when the driver calls I**WiaMiniDrv::drvAcquireItemData** by using good parameters in all supported formats.
- Release their references to the application's **IStream** object before their **IWiaMiniDrv::drvAcquireItemData** methods return or call **IWiaMiniDrvTransferCallback::GetNextStream**.
- Send one stream that contains a multi-page file during downloads by using all supported TYMED_MULTIPAGE_FILE formats.
- Send one stream for each item during downloads by using all supported TYMED_FILE formats.
- Return S_FALSE from **IWiaMiniDrv::drvAcquireItemData** if **IWiaMiniDrvTransferCallback::SendMessage** returns S_FALSE.
- Continue to transfer any subsequent items during multi-item downloads after **IWiaMiniDrvTransferCallback::GetNextStream** returns WIA_STATUS_SKIP_ITEM.
- Return S_OK from **IWiaMiniDrv::drvAcquireItemData** during single-item and multi-item downloads after the **IWiaMiniDrvTransferCallback::GetNextStream** returns WIA_STATUS_SKIP_ITEM for any item.
- Abort transfer of all items after **IWiaMiniDrvTransferCallback::GetNextStream** returns S_FALSE.
- Return from **IWiaMiniDrv::drvAcquireItemData** calls during canceled transfers in less time than during completed transfers.
- Return a failure code from **IWiaMiniDrv::drvAcquireItemData** if **IWiaMiniDrvTransferCallback::GetNextStream** fails.
- Return a standard COM failure code from **IWiaMiniDrv::drvAcquireItemData** if **IWiaMiniDrvTransferCallback::GetNextStream** returns a NULL stream pointer.

- Return a failure code from **IWiaMiniDrv::drvAcquireItemData** if **IWiaMiniDrvTransferCallback::SendMessage** fails.
- Successfully transfer items when an identical device is installed and when the identical device transfers an item simultaneously.
- Return a failure code from **IWiaMiniDrv::drvAcquireItemData** if an **IStream** method fails.
- Seek to the correct stream position after the driver begins the transfer or calls **IWiaMiniDrvTransferCallback::GetNextStream** or **IWiaMiniDrvTransferCallback::SendMessage**.
- Function correctly if the WIA service terminates during a transfer and is then restarted, and a new transfer is requested.
- Ignore calls to the **drvNotifyPnPEvent**<element> that contain WIA_EVENT_CANCEL_IO if a transfer is not occurring, and not crash or hang.
- Successfully transfer items after a valid WIA_IPA_TYMED property value is written by using a signed or unsigned variant type.

WIA drivers must not do the following:

- Call a method of an application's **IStream** object other than the **IUnknown::Release** method after an application's transfer callback returns S_FALSE, WIA_STATUS_SKIP_ITEM, or an error.
- Call a method of an application's **IStream** object other than the **IUnknown::Release** method after the driver's **IWiaMiniDrv::drvAcquireItemData** method returns or calls **IWiaMiniDrvTransferCallback::GetNextStream** during a multi-item transfer.
- Call **IWiaMiniDrvTransferCallback::SendMessage** by using WIA_TRANSFER_MSG_END_OF_STREAM and WIA_TRANSFER_MSG_END_OF_TRANSFER messages.
- Call **IWiaMiniDrvTransferCallback::SendMessage** or **IWiaMiniDrvTransferCallback::GetNextStream** after **IWiaMiniDrvTransferCallback::SendMessage** returns S_FALSE or an error.
- Crash or hang if a device requests an upload by using an invalid or empty stream.

Exceptions: Not Specified

Business Justification:

This requirement verifies that a driver does not cause applications or the WIA service to crash or hang in edge conditions when the driver sends a message to an error handler or receives a message to its error handler. This requirement also helps ensure a consistent and quality experience for the user.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2006

Comments: Not Specified

Device.Imaging.Scanner.Base.driverInstallation

Target Feature: Device.Imaging.Scanner.Base

Title: A WIA driver must be installed for scanners and MFPs

Applicable OS Versions:

- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 8 Client x86
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64

Description:

In cases in which a scanner or MFP that has scanning functionality initiates a plug and play installation event, a WIA driver must be installed. On a software-first installation for this device, a WIA driver must be staged to the driver store so that plug and play installations are successful.

This requirement does not prevent the manufacturer from installing other software, such as a TWAIN data source.

Exceptions: Not Specified

Business Justification:

This requirement helps ensure that drivers are implemented correctly.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2009

Comments: Not Specified

Device.Imaging.Scanner.Base.errorHandling

Target Feature: Device.Imaging.Scanner.Base

Title: WIA drivers that support error handling must comply with specified error handling implementations

Applicable OS Versions:

- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 x64
- Windows Server 2008 x86
- Windows 8 Client x86
- Windows Server 2008 Release 2 x64
- Windows 8 Server x64

Description:

Error handling in WIA drivers must comply with the following requirements:

- **IWiaErrorHandler::GetStatusDescription** methods must return S_OK or WIA_STATUS_NOT_HANDLED when the driver calls the methods by using good parameters.
- **IWiaErrorHandler::ReportStatus** and **IWiaErrorHandler::GetStatusDescription** methods must return a standard COM failure code if the driver calls the methods by using a NULL *pWiaItem2* parameter.

WIA drivers must:

- Cancel transfers and return S_FALSE when **IWiaErrorHandler::ReportStatus** is called by using a device status message and returns S_FALSE.
- Cancel transfers and return a standard COM failure code when **IWiaErrorHandler::ReportStatus** is called by using a device status message and returns a failure code.

WIA drivers must not:

- Call **IWiaMiniDrvTransferCallback::SendMessage** by using WIA_STATUS_CLEAR messages.
- Call **ReportStatus** by using failure device status messages when the driver calls **IWiaMiniDrv::drvAcquireItemData** by using good parameters.

IWiaErrorHandler::ReportStatus methods must:

- Return only S_OK or WIA_STATUS_NOT_HANDLED when called by using good parameters and without user input.
- Return S_OK when called by using good parameters and without user input when a device status message is sent that is identical to a message that an active modeless window handles.

- Return a standard COM failure code when called by using good parameters and without user input when a modeless error handler window is open and a device status message is sent that is not identical to the message that the existing window handles.
- Return S_OK when called by using a WIA_STATUS_CLEAR message while an error handler is either active or inactive.

IWiaErrorHandler::ReportStatus methods must not:

- Create or remove any windows when called by using good parameters and without user input when a device status message is sent that is identical to a message that an active modeless window handles.

IWIA driver error handlers must:

- Return without user input when **IWiaErrorHandler::ReportStatus** is called by using a success device status message.
- Consistently choose whether to handle specific device status messages for each item category. For example, a flatbed item may not only sometimes handle the WIA_STATUS_WARMING_UP message.
- Create modeless windows when **IWiaErrorHandler::ReportStatus** returns S_OK after **IWiaErrorHandler::ReportStatus** is called by using a success device status message.
- Remove their active modeless windows when **IWiaErrorHandler::ReportStatus** is called by using a WIA_STATUS_CLEAR message.
- Create modal windows when **IWiaErrorHandler::ReportStatus** returns S_OK after **IWiaErrorHandler::ReportStatus** is called by using a failure device status message.
- Return a valid **pbstrDescription** value when **IWiaErrorHandler::GetStatusDescription** succeeds and does not return WIA_STATUS_NOT_HANDLED.
- Return S_OK and a valid **pbstrDescription** value when **IWiaErrorHandler::GetStatusDescription** is called by using good parameters and a custom device status message that the driver sent during a transfer.
- Return a standard COM failure code when **IWiaErrorHandler::GetStatusDescription** is called by using a NULL *pbstrDescription* parameter.

IWIA driver error handlers must not:

- Return without user input when **IWiaErrorHandler::ReportStatus** is called by using a failure device status message and does not return WIA_STATUS_NOT_HANDLED.
- Create windows when **IWiaErrorHandler::ReportStatus** returns WIA_STATUS_NOT_HANDLED.

Exceptions: Not Specified

Business Justification:

This requirement verifies that a driver does not cause applications or the WIA service to crash or hang in edge conditions when the driver sends a message to an error handler or receives a message to its error handler. This requirement also helps ensure a consistent and quality experience for the user.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2006

Comments: Not Specified

Device.Imaging.Scanner.Base.metadata

Target Feature: Device.Imaging.Scanner.Base

Title: Scanner and MFP driver components must include an authored device metadata document

Applicable OS Versions:

- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows Server 2008 Release 2 x64
- Windows 8 Server x64

Description:

Devices that provide DeviceStage metadata must do so correctly. The metadata must include a photorealistic image of the device, the company logo, and basic task information.

Tasks must only appear when the tasks are available. For example, scanner tasks must not appear on a printer-only connection. Internet tasks must not appear if the Internet is unavailable.

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2009

Comments: Not Specified

Device.Imaging.Scanner.Base.MFPmultiplePorts

Target Feature: Device.Imaging.Scanner.Base

Title: MFP devices that have multiple identical ports must expose the same functionality on all ports

Applicable OS Versions:

- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows Server 2008 Release 2 x64
- Windows 8 Server x64

Description:

If an MFP device has identical multiple ports, the device must expose identical functionality on each port. For example, if one USB port supports print and scan functionality, all other USB ports must support print and scan functionality. This requirement does not extend to bandwidth concerns (that is, a device can have a USB 1.1 port and a USB 2.0 port). All other functionality must be exposed on both ports.

Telephone jacks for fax functionality can behave differently to support line-in and line-out telephony connections.

Exceptions: Not Specified

Business Justification:

Multiple similar ports that have dissimilar functionalities confuse consumers. For example, a printer has two USB ports. One of the USB ports exposes both print and storage functionality. The other port only allows printing. Consumers become confused and generate support calls for Microsoft.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: June 01, 2006

Comments: Not Specified

Device.Imaging.Scanner.Base.RawFileFormat

Target Feature: Device.Imaging.Scanner.Base

Title: A scanning device that supports the WIA Raw Transfer Image file format must implement it correctly

Applicable OS Versions:

- Windows 8 Client x64

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows Server 2008 Release 2 x64
- Windows 8 Server x64

Description:

Devices that support WIA Raw Transfer Image File must support transferring data in all supported **WIA_IPA_DATATYPE**, **WIA_IPA_DEPTH** and **WIA_COMPRESSION_NONE** modes in the WIA Raw Format (**WIA_IPA_FORMAT** set to **WiaImgFmt_RAW**, **WIA_IPA_TYMED** set to **TYMED_FILE**). In other words the uncompressed variant (**WIA_RAW_HEADER::Compression** set to **WIA_COMPRESSION_NONE**) is required.

Exceptions: Not Specified

Business Justification:

TBA

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Imaging.Scanner.Base.scannerInterfaces

Target Feature: Device.Imaging.Scanner.Base

Title: A scanning device must support at least one non-legacy interface

Applicable OS Versions:

- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows Server 2008 Release 2 x64
- Windows 8 Server x64
- Windows Vista Client x64
- Windows Vista Client x86

Description:

Scanners and MFP devices must be able to connect to the computer through a non-legacy interface such as Ethernet, USB, IEEE 1394, or Bluetooth.

Exceptions: Not Specified

Business Justification:

Non-legacy ports provide a much better user experience than legacy ports.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: June 01, 2006

Comments: Not Specified

Device.Imaging.Scanner.Base.statusMessages

Target Feature: Device.Imaging.Scanner.Base

Title: Scanning device sends device status messages correctly

Applicable OS Versions:

- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 x64
- Windows Server 2008 x86
- Windows 8 Client x86
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64

Description:

A scanning device that sends device status messages must do so correctly. The device must also implement the error handler correctly if the device implements an error handler.

Design Notes:

For more information, see the Windows Driver Kit content

on IWiaErrorHandler at http://msdn.microsoft.com/en-us/library/ff543907.aspx

Alternatively, send an e-mail message to wiainfo@microsoft.com.

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2006

Comments: Not Specified

Device.Imaging.Scanner.Base.wia20

Target Feature: Device.Imaging.Scanner.Base

Title: Scanners and multi-function printers that have scanning ability must implement the WIA 2.0 driver architecture according to the Windows Driver Kit guidelines

Applicable OS Versions:

- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 8 Client x86
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64

Description:

Scanners and multi-function printers that have scanning ability must implement the WIA 2.0 driver architecture according to the guidelines in the Windows Driver Kit.

Scanners support stream-based image transfers. In stream-based transfers, the application gives WIA the stream to use, and then the driver reads or writes to the stream. The stream may be a file stream, a memory stream, or any other type of stream. The stream is transparent to the driver. Using streams also provides easy integration with the image processing filter. This helps prevent complications that occur because of the destination type (memory or file).

Scanners need to correctly implement the Windows WIA Scanner Item Architecture for flatbed, ADF, and film scanners and scanners that have storage. The Windows Driver Kit reference documents and tools outline the WIA scanner item architecture. Device manufacturers must implement WIA support as described in the Windows Driver Kit.

Design Notes:

WIA 2.0 enables new stream-based transfer models and certain extensions that include an image-processing filter, a segmentation filter, and error handling. For more information about WIA 2.0, see "Introduction to WIA 2.0" at http://www.microsoft.com/whdc/device/stillimage/WIA20-intro.mspx and "What's new in WIA 2.0" at http://msdn.microsoft.com/en-us/library/ms630379(VS.85).aspx.

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2010

Comments: Not Specified

Device.Imaging.Scanner.Base.WIAArchitecture

Target Feature: Device.Imaging.Scanner.Base

Title: A scanner device driver must implement WIA driver architecture

Applicable OS Versions:

- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 x64
- Windows Server 2008 x86
- Windows 8 Client x86
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64

Description:

Scanner device drivers must support still image devices under WIA architecture or ISO/PRF (formerly PIMA) 15740. The scanner vendor must provide a WIA driver.

A WIA driver is required for all local busses on which a scanner enumerates to Windows. If the device supports network-based scanning using WS-Scan or Distributed Scan Management, the device must do so as outlined in those requirements.

Design Notes:

An optimal user experience is seamless integration of the imaging peripheral with the Windows environment. The operating system detects hot-pluggable WIA devices such as digital cameras, providing a seamless interface with the device. For persistent-connection devices, such as scanners, implementation of device events through buttons and sensors delivers this functionality after initial installation. In addition to WIA, scanners can optionally support TWAIN.

See the Windows Driver Kit, "Still Image Drivers."

Exceptions: Not Specified

Business Justification:

The operating system natively provides WIA support. WIA support helps ensure reliability and an optimal experience with Windows.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: June 01, 2006

Comments: Not Specified

Device.Imaging.Scanner.Base.WIAProperties

Target Feature: Device.Imaging.Scanner.Base

Title: Scanners must implement support for all required WIA properties and property values

Applicable OS Versions:

- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 x64
- Windows Server 2008 x86
- Windows 8 Client x86
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64
- Windows Vista Client x86
- Windows Vista Client x64

Description:

The Windows Driver Kit (WDK) reference documents and tools outline the properties and property values for WIA. Scanners must implement WIA as described in the WDK.

Exceptions: Not Specified

Business Justification:

This requirement allows the device to adequately represent the device's capabilities and functionality to the operating system and applications.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2006

Comments: Not Specified

Device.Imaging.Scanner.DistributedScanManagement

Description: Not Specified

Related Requirements:

- Device.Imaging.Scanner.DistributedScanManagement.DistributedScanManagement

Device.Imaging.Scanner.DistributedScanManagement.DistributedScanManagement

Target Feature: Device.Imaging.Scanner.DistributedScanManagement

Title: A scanner that supports the Distributed Scan Management protocols must implement the protocols correctly

Applicable OS Versions:

- Windows Server 2008 Release 2 x64
- Windows 8 Server x64

Description:

Any device that interacts with Windows Server Active Directory and a Windows Server scan server that implements the Distributed Scan Management Web service protocols must do so correctly.

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2009

Comments: Not Specified

Device.Imaging.Scanner.WSD

Description: Not Specified

Related Requirements:

- Device.Imaging.Scanner.WSD.Rally
- Device.Imaging.Scanner.WSD.WSScan

Device.Imaging.Scanner.WSD.Rally

Target Feature: Device.Imaging.Scanner.WSD

Title: Network-attached scanners and MFPs must implement the Windows Rally Technologies

Applicable OS Versions:

- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86

Description:

Network-connected scanners and MFPs that implement any of the following Windows Rally requirements must comply with the implemented requirement completely.

- Connect-0098 (optional): Devices that implement 802.3 or 802.11 may implement the Link Layer Topology Discovery (LLTD) protocol. This applies only if the device implements LLTD. LLTD implementation is not required.
- Connect-0099 (required): An 802.11 network-enabled device that operates as a station must implement WCN-NET and meet basic 802.11 requirements.
- Connect-0100 (required): A network-enabled device that implements Plug and Play Extensions (PnP-X) must comply with the specification.
- IMAGING-0004 (required): Network-connected devices must implement Windows network-connected Web Services for Devices (WSD) correctly for their device type.

Design Notes:

For more information, see the PnP-X: Plug and Play Extensions for Windows Specification at http://go.microsoft.com/fwlink/?LinkID=123172

Exceptions: Not Specified

Business Justification:

Web Services provides an "it just works" scenario for network scanners.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2008

Comments: Not Specified

Device.Imaging.Scanner.WSD.WSScan

Target Feature: Device.Imaging.Scanner.WSD

Title: Scanners that have a network connection must implement WS-Scan protocol

Applicable OS Versions:

- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86

Description:

This requirement applies to scanners or multifunction printers that have scanning ability, connect to the network, and have a physical scan button on the device. These scanners or multifunction printers must implement the following WS-Scan protocol requirements as outlined in the WS-Scan specification v1.0:

- The device must correctly implement all events and operations that the specification defines.
    - The device must support the **ScanAvailableEvent** so that users can initiate a scan from the scanner and push the document to a scanning application.
- The device must support the Microsoft WSD scan class driver for all device features that WS-Scan exposes.
- If the device has ADF and plate scanning, the device must support those media over WS-Scan.

For more information, see "Implementing Web Services on Devices for Printing and Scanning" at http://www.microsoft.com/whdc/connect/rally/wsdspecs.mspx.

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2011

Comments: Not Specified

Device.Input.FingerPrintReader

Description: Not Specified

Related Requirements:

Device.Input.FingerPrintReader.Base
Device.Input.FingerPrintReader.Extensions
Device.Input.FingerPrintReader.ManagementApps
Device.Input.FingerPrintReader.SensorEngineDB
Device.Input.FingerPrintReader.WBDI

Device.Input.FingerPrintReader.Base

Target Feature: Device.Input.FingerPrintReader

Title: Biometric Fingerprint Reader General Requirement

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client ARM
Windows 8 Server x64
Windows Server 2008 Release 2 x64

Description:

Biometric fingerprint readers certified under this program must be exposed through the Windows Biometric Framework (WBF). Biometric fingerprint reader drivers must expose and adhere to the Windows Biometric Driver Interface (WBDI), as documented on MSDN (http://msdn.microsoft.com/en-us/library/aa973504.aspx). WBF plug-in components must implement all entry points and adhere to the WBF plug-in interfaces that are documented on MSDN (http://msdn.microsoft.com/en-us/library/dd401553(VS.85).aspx).

The following general criteria must be met:

The driver package must contain a WBDI driver, a combination of plug-in adapters, and a fingerprint management application (FMA) that are required all of which are required to enroll a user and log on to Windows.

Devices that operate in advanced mode may implement a compound adapter. A compound adapter may contain a proprietary sensor, engine, and database adapter, or any combination of these adapters.

The WBDI driver and plug-in components must not contain any malicious software such as Trojan horse or backdoor software.

Design Notes:

Biometric devices measure an unchanging physical characteristic to uniquely identify an individual. Fingerprints are one of the most common biometric characteristic measured. Beginning with Windows7, a new set of components, the Windows Biometric Framework, provides support for fingerprint biometric devices. These components, created using the Windows Biometric Framework API, can perform the following tasks:

Capture biometric samples and use them to create a template.

Securely save and retrieve biometric templates.

Map each template to a unique identifier such as a GUID (globally unique identifier) or SID (system identifier).

Enroll new biometric templates.

To expose a device through WBF, the device must be exposed through a driver that implements the WBDI driver and an appropriate combination of Sensor, Engine, and Database plug-in components.

For more complete information about WBF and its components, see the "Introduction to the Windows Biometric Framework (WBF)" white paper at http://www.microsoft.com/whdc/device/biometric/WBFIntro.mspx

Exceptions: Not Specified

Business Justification:

The Windows Biometric Framework provides a common driver and application interface for enrolling, identifying and verifying users as well as a set of common user experiences to allow control of basic settings and offering standard experiences around logon and UAC and common integration points for third party applications on the control panel. The framework offers a common API that allows applications to be developed independent of hardware manufacturer. This in turn allows the OEM more flexibility in selecting hardware and software suppliers. The framework is also extensible and customizable. This allows OEMs to provide their own branded enrolment and logon experiences. It also allows OEMs and IHVs to extending the API for additional features they may wish to support (e.g. PBA).

Scenarios:

Biometric devices support identifying users to log on their system or access data.

Success Metric: Not Specified

Enforcement Date: 12/1/2010

Comments:

INPUT-0066

Device.Input.FingerPrintReader.Extensions

Target Feature: Device.Input.FingerPrintReader

Title: Vendor-supplied drivers or other extension components must not wrap other extension components

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

Drivers and adapter plug-ins that extend the Windows Biometric Framework must not wrap other drivers or adapter plug-ins unless the practice is explicitly permitted by the component documentation supplied by Microsoft.

Exceptions: Not Specified

Business Justification:

Wrapping extension components that do not support wrapping can result in system instability, security vulnerabilities and loss of customer data.

Scenarios:

Biometric devices logging user into system

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments: Not Specified

Device.Input.FingerPrintReader.ManagementApps

Target Feature: Device.Input.FingerPrintReader

Title: Biometric Fingerprint Reader Management Applications

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client ARM
- Windows 8 Server x64

Description:

A fingerprint management application (FMA) must be included in the driver package and should follow the guidelines in "Designing Windows Biometric Framework (WBF) Fingerprint Management Applications" at http://www.microsoft.com/whdc/device/biometric/FMA_Design.mspx

Design Notes:

Biometric devices measure an unchanging physical characteristic to uniquely identify an individual. Fingerprints are the most common biometric characteristic measured. Beginning with Windows7, a new set of components, the Windows Biometric Framework (WBF), provides support for fingerprint biometric devices. These components, created using the Windows Biometric Framework API, can perform the following tasks:

Capture biometric samples and use them to create a template.

Securely save and retrieve biometric templates.

Map each template to a unique identifier such as a GUID (globally unique identifier) or SID (system identifier).

Enroll new biometric templates.

To expose a device through the WBF, the device must be exposed through a driver that implements the Windows Biometric Driver Interface (WBDI) driver as well as an appropriate combination of sensor, engine, and database plug-in components. For a more complete background on WBF and its components, see "Introduction to the Windows Biometric Framework (WBF)" at http://www.microsoft.com/whdc/device/biometric/WBFIntro.mspx.

Exceptions: Not Specified

Business Justification:

The Windows Biometric Framework provides a common driver and application interface for enrolling, identifying and verifying users as well as a set of common user experiences to allow control of basic settings and offering standard experiences around logon and UAC and common integration points for third party applications on the control panel. The framework offers a common API that allows applications to be developed independent of hardware manufacturer. This in turn allows the OEM more flexibility in selecting hardware and software suppliers. The framework is also extensible and customizable. This allows OEMs to provide their own branded enrolment and logon experiences. It also allows OEMs and IHVs to extending the API for additional features they may wish to support (e.g. PBA).

Scenarios:

Biometric devices support identifying users to log on their system or access data.

Success Metric: Not Specified

Enforcement Date: 12/1/2010

Comments:

INPUT-0069

Device.Input.FingerPrintReader.SensorEngineDB

Target Feature: Device.Input.FingerPrintReader

Title: Fingerprint Reader Sensor, Engine and Database Plug-in requirement

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client ARM
- Windows 8 Server x64

Description:

All interface entry points must be implemented, and each element must adhere to the definition of its respective adapter plug-in interface.

Plug-in components must support the sequenced invocation of their interfaces that result from invoking all the top-level Windows Biometric Framework (WBF) APIs.

The sequence of WBF API calls used by the WBF credential provider for Windows logon must be completed in 1.5 seconds or less.

All adapters must be digitally signed, as described in the Windows Biometric Framework: Code-Signing Guidelines white paper at http://www.microsoft.com/whdc/device/biometric/WBF_CodeSign.mspx.

The adapter must run under Application Verifier without generating any errors.

A WBDI driver and plug-in components can support multiple devices but must pass the certification criteria for each of the supported fingerprint readers separately. A separate submission must be made for each supported device.

Design Notes:

Biometric devices measure an unchanging physical characteristic to uniquely identify an individual. Fingerprints are the most common biometric characteristic measured. Beginning with Windows7, a new set of components, the Windows Biometric Framework, provides support for fingerprint biometric devices. These components, created using the Windows Biometric Framework API, can perform the following tasks:

Capture biometric samples and use them to create a template.
Securely save and retrieve biometric templates.
Map each template to a unique identifier such as a GUID (globally unique identifier) or SID (system identifier).
Enroll new biometric templates.

To expose a device through the Windows Biometric Framework, the device must be exposed through a driver that implements the Windows Biometric Driver Interface (WBDI) driver as well as an appropriate combination of sensor, engine, and database plug-in components. For more complete information about WBF and its components, see the "Introduction to the Windows Biometric Framework (WBF)" white paper at http://www.microsoft.com/whdc/device/biometric/WBFIntro.mspx.

Exceptions: Not Specified

Business Justification:

The Windows Biometric Framework provides a common driver and application interface for enrolling, identifying and verifying users as well as a set of common user experiences to allow control of basic settings and offering standard experiences around logon and UAC and common integration points for third party applications on the control panel. The framework offers a common API that allows applications to be developed independent of hardware manufacturer. This in turn

allows the OEM more flexibility in selecting hardware and software suppliers. The framework is also extensible and customizable. This allows OEMs to provide their own branded enrolment and logon experiences. It also allows OEMs and IHVs to extending the API for additional features they may wish to support (e.g. PBA).

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 12/1/2010

Comments:

INPUT-0068

Device.Input.FingerPrintReader.WBDI

Target Feature: Device.Input.FingerPrintReader

Title: Biometric Fingerprint Reader WBDI Driver Requirement

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client ARM
Windows 8 Server x64

Description:

(Change note for Version 2: Deleted references to Terminal Services.)

Windows Biometric Driver Interface (WBDI) drivers must adhere to the following criteria:

All mandatory IOCTLs must be fully implemented and adhere to the WBDI specification.

If an optional IOCTL is implemented, it must adhere to the WBDI specification.

The WBDI driver must support the biometric IOCTL calling sequence.

If the biometric fingerprint reader is not a PnP device, it must create arrival and departure events as recommended in the WBDI specification.

The WBDI INF installer must set up the associated plug-in components and fingerprint management applications that are included in the driver package.

Guidelines in this requirement apply equally well to the kernel-mode and user-mode drivers. The following must be supported:

Backward and forward compatibility. Drivers must have a mechanism that determines different versions of user-mode and kernel-mode components. When two versions of the same driver have been installed on the PC, the kernel component must determine the correct driver to talk to. In remoting scenarios, the mismatch may occur even with a single driver.

The following items must not be used:

Out-of-band communications. Because the user-mode and kernel-mode drivers are running on the same PC, they might be coupled together through channels different from the I/O manager APIs. The goal is to ensure that drivers do not have out-of-band communication, implicit or explicit.

Here are some specific examples that would prevent terminal services redirection:

Shared memory must not be used directly between user-mode applications and either user-mode or kernel-mode drivers. For every data item sent and received, there must be a corresponding I/O request. Share the memory either through a mapped file or through locked pages of one direct I/O call.

Registry communication (that is, any registry keys that are set from user-mode drivers and read from kernel-mode drivers or vice versa). It is problematic when an application is running on the server and drivers are loaded on the client because the registry is set on the server but not on the client.

Kernel objects (passing any kernel object handles in I/O buffers, such as handles to events or semaphores).

Data pointers (passing data pointers in I/O buffers). For example, a driver may try to read a linked list or other complicated data structure from the kernel.

Incompatibility between 32-bit and 64-bit platform implementations of the I/O request packet (IRP) requests (fields in the data structures that are compiled differently, depending on the bitness). Incompatibility of the structures based on bitness results in different offsets and data size for these structures. The data will not be interpreted correctly when a terminal services client and server are not the same platform.

Reliance on a strict timing expectation about how fast the kernel driver responds. Because drivers are remoting over the network, additional latency is added to the response from the hardware. That latency may range from 10 ms to several seconds. A possible cause for this could be slow network or packet losses. If a driver is programmed for a response that comes in time less than usual network latency, the remoting fails.

Setting an access control list (ACL) or any other run-time security check that would prevent any application from opening a device driver. For example, a driver is allowed to accept calls only from particular service. Because all the calls are done on a TS client PC under security context of the remoting client process, they can fail.

Design Notes: Biometric devices measure an unchanging physical characteristic to uniquely identify an individual. Fingerprints are the most common biometric characteristic measured. Beginning with

Windows7, a new set of components, the Windows Biometric Framework (WBF), provides support for fingerprint biometric devices. These components, created with the WBF API can perform the following tasks:

Capture biometric samples and uses them to create a template.

Securely save and retrieve biometric templates.

Map each template to a unique identifier such as a GUID (globally unique identifier) or SID (system identifier).

Enroll new biometric templates.

To expose a device through WBF, the device must be exposed through a driver that implements the WBDI driver as well as an appropriate combination of sensor, engine, and database plug-in components. For more complete information about WBF and its components, see the "Introduction to the Windows Biometric Framework (WBF)" white paper at http://www.microsoft.com/whdc/device/biometric/WBFIntro.mspx.

Exceptions: Not Specified

Business Justification:

The Windows Biometric Framework provides a common driver and application interface for enrolling, identifying and verifying users as well as a set of common user experiences to allow control of basic settings and offering standard experiences around logon and UAC and common integration points for third party applications on the control panel. The framework offers a common API that allows applications to be developed independent of hardware manufacturer. This in turn allows the OEM more flexibility in selecting hardware and software suppliers. The framework is also extensible and customizable. This allows OEMs to provide their own branded enrolment and logon experiences. It also allows OEMs and IHVs to extending the API for additional features they may wish to support (e.g. PBA).

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 12/1/2010

Comments:

INPUT-0067

Device.Input.GameController.CommonController

Description:

Device supports use as a Common Controller Game Device

Related Requirements:

Device.Input.GameController.CommonController.XInput

Device.Input.GameController.CommonController.XInput

Target Feature: Device.Input.GameController.CommonController

Title: Microsoft Common Controller for Windows API support (XINPUT)

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client ARM

Description:

1. Microsoft Common Controller for Windows complies with requirements defined in the XUSB specification:

A common controller must meet all requirements and comply with the XUSB Specification.

Support for audio is optional for the common controller. If the controller supports audio, it must also comply with the device interfaces for audio defined in the XUSB Specification.

2. Microsoft Common Controller for Windows uses the XUSB22.SYS driver:

The common controller must use the Microsoft provided XUSB22.SYS driver to interface with Windows. Custom drivers or filters are not allowed.

3. Microsoft Common Controller for Windows functions in conjunction with the XInput application programming interfaces:

The common controller must function correctly when interfacing with the Microsoft XInput APIs.

Exceptions: Not Specified

Business Justification:

The Microsoft Common Controller Driver is a game input standard that is used for both the Xbox360 console and for Microsoft Windows. The Xbox360 controller or any controllers that utilize this standard will enable the device to be used on Windows as well.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2007

Comments:

INPUT-0003; Version update in paragraph 2 was Xnacc.sys now is Xusb22.sys

Device.Input.GameController.GenericController

Description:

Device supports use as a generic USB game controller

Related Requirements:

Device.Input.GameController.GenericController.DirectInput

Device.Input.GameController.GenericController.DirectInput

Target Feature: Device.Input.GameController.GenericController

Title: Generic USB Game Controller API Support (DINPUT)

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 7 Client x86
Windows 7 Client x64

Description:

An input gaming device must be HID-compliant. The driver must support required functionality defined for Direct Input.

Exceptions: Not Specified

Business Justification:

The Microsoft Common Controller driver is a game input standard that is used for both the Xbox360 console and for Microsoft Windows. The Xbox360 controller or any controllers that utilize this standard will enable the device to be used on Windows as well.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

INPUT-0002

Device.Input.HID

Description: Not Specified

Related Requirements:

Device.Input.HID.I2CDeviceUniqueHWID

Device.Input.HID.I2CProtocolSpecCompliant
Device.Input.HID.MCEClassDriver
Device.Input.HID.MCEClassDriverWindows7
Device.Input.HID.MCERemoteControlCompliance
Device.Input.HID.UsbSpecificationCompliant

Device.Input.HID.I2CDeviceUniqueHWID

Target Feature: Device.Input.HID

Title: I2C connected HID devices must have a Unique HWID along with a HID compatible ID

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64

Description:

All I2C connected HID devices must have a unique HWID and a HID compatible ID that will allow WU to identify the device (when needed) and allow drivers to be loaded from WU.

Design Notes: See Microsoft published HID I2C protocol specification.

Exceptions:

This requirement is only enforced for HID I2C devices and not generalized for SPB.

Business Justification:

To enable the correct drivers to be loaded by WU

Scenarios:

Input devices connected over I2C

Success Metric: Third party drivers for I2C devices are available on WU

Enforcement Date: Windows 8 RC

Comments:

New

Device.Input.HID.I2CProtocolSpecCompliant

Target Feature: Device.Input.HID

Title: All HID devices connected over I2C must comply with MSFT HID I2C Protocol specification

Applicable OS Versions:

Windows 8 Client x64

Windows 8 Client x86

Description:

All HID over I2C compatible devices should comply with the Microsoft published documentation for the HID I2C protocol specification

Design Notes: See Microsoft published HID I2C protocol specification (link not provided yet)

Exceptions:

This requirement is only enforced for HID I2C devices and not generalized for SPB.

Business Justification:

To maintain a standardized way in which HID I2C devices report data and so that all HID I2C device vendors and OEMs can utilize the Microsoft provided HID I2C miniport driver instead of writing and using multiple third party drivers

Scenarios:

Touch, Sensors, Input devices connected over I2C

Success Metric: The device reports data in compliance with the HID I2C specification that the HID I2C miniport driver is able to understand and pass up to HIDClass.sys

Enforcement Date: Windows 8 RC

Comments:

New

Device.Input.HID.MCEClassDriver

Target Feature: Device.Input.HID

Title: IR Receiver/Transceiver interfaces with the Windows Media Center class driver

Applicable OS Versions:

Windows 7 Client x86
Windows 7 Client x64

Description:

For WMC AQ, this requirement is

REQUIRED for Desktop systems

REQUIRED for Laptop systems if Laptop system includes remote and receiver.

For non AQ systems, this requirement is

REQUIRED If system (either Desktop or Laptop) includes remote and receiver.

IR Receivers (input only) and IR Transceivers (input and IR emitting/learning) interface with the Windows Media Center class driver. In a system with a TV tuner, IR Transceivers must be used that support all functions of the ***Remote Control and Receiver-Transceiver Specifications and Requirements for Windows Media Center in Windows Operating Systems*** document.

For tunerless systems with an IR receiver only IR Input and wake functions are required. The IR learning and emitting functions are optional.

DVB-T solutions are not required to support learning and emitting. These functions are optional.

In locales that do not use set top boxes, the learning and emitting functions are optional.

In those regions that support secondary 10 foot application, the IR Receiver/Transceivers are not required to meet the IR Learning requirement.

In those regions that support DVB-S Microsoft strongly recommends the use of IR Transceivers (input and IR emitting/learning)

If shipping a laptop system, IR learning and IR emitting is optional

For IR receivers (input only) that use Microsoft authorized IR protocols and all IR Transceiver (input and emitter functions), the device will return IR waveform envelope to software for software decoding of IR signal. IR signal cannot be decoded in the hardware. The only exception to this is the wake-from-remote power key.

An IR receiver (input only) is allowed to perform hardware decoding of an IR signal. These IR receivers (input only) must not receive and respond to any currently authorized Microsoft IR protocol. IR receivers that use hardware decoding of IR signal also need to support the wake-from-remote functionality. These devices must comply with the Remote ***Control and Receiver-Transceiver Specifications and Requirements for Windows Media Center in Windows Operating Systems*** document.

Design Notes:

Microsoft recommends that IR cables be labeled and well documented for end users. An insert showing a small diagram of the IR control cable and how it connects to the digital cable or satellite receiver could help prevent support calls.

Exceptions: Not Specified

Business Justification:

Products that follow these requirements are compatible with Windows Media Center in Windows.

Scenarios:

The ability to use a remote control and receiver with Windows Media Center.

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

INPUT-0007

Device.Input.HID.MCEClassDriverWindows7

Target Feature: Device.Input.HID

Title: IR Receiver/Transceiver interfaces with the Windows Media Center class driver Windows 7.

Applicable OS Versions:

Windows 7 Client x86
Windows 7 Client x64

Description:

For WMC AQ, this requirement is

REQUIRED for Desktop systems

REQUIRED for Laptop systems if Laptop system includes remote and receiver.

For non AQ systems, this requirement is

REQUIRED If system (either Desktop or Laptop) includes remote and receiver.

The specifications for these requirements are available now from Microsoft Connect as "Windows Media Center IR Receiver-Transceiver and Remote Control Specifications".

The IR receiver components must accurately receive the current Windows Media Center supported IR protocols and the new Windows Media Center Quatro Pulse (QP) protocol.

The IR receiver components must be able to wake from Windows Media Center RC6 and Windows Media Center QP protocol from the sleep toggle button and discrete on button. You have the option of accomplishing this by allowing the device to be programmed, or ensuring the firmware can recognize all possible buttons and protocols for wake.

The IR receiver components must conform to the version 2 Device Driver Interface (DDI) which will have the following capabilities: IR input (narrowband or wideband), IR output and IR learning or combinations of the above.

Laptops have the option to be an IR input only solution regardless of tuner configuration.

If building a TV tuner that has integrated CIR capabilities which allows analog TV signal input you are required to support IR output with one emitter per analog signal input.

For IR receiver components that allow IR input and that use Microsoft authorized IR protocols, the device will return IR waveform envelope to software for software decoding of IR signal. IR signal cannot be decoded in the hardware. The only exception to this is the wake-from-remote sleep toggle key (or discrete sleep).

The IR receiver components must accurately modulate the carrier signal when blasting IR and must accurately report the carrier of the signal when receiving IR through a learning receiver.

We recommend that the IR receiver component flash an LED when requested to do so by the operating system or when an IR signal is received.

Windows Media Center does support remote input from non-Consumer IR compliant devices only through our Hardware Specification for Human Interface Devices (HID) as described in the Windows Media Center IR Receiver-Transceiver and Remote Control Specifications. Examples would include: RF, Bluetooth, and hardware decoded IR.

These devices cannot use the currently supported protocols: Windows Media Center RC 6 or Windows Media Center QP IR protocols.

These devices must support wake from remote functionality.

These devices must support accurate numeric input for all regions.

In order to do this the device manufacturer must register their device ID according to our specifications. Note: The device ID may be the device ID of the remote control rather than the receiver (example: Bluetooth)

Design Notes:

Microsoft recommends that IR cables be labeled and well documented for end users. An insert showing a small diagram of the IR control cable and how it connects to the digital cable or satellite receiver could help prevent support calls.

Details for the protocol specifications and samples will be provided through connect site .

Exceptions: Not Specified

Business Justification:

Products that follow these requirements are compatible with Windows Media Center in Windows.

Scenarios:

The ability to use a remote control and receiver with Windows Media Center.

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

INPUT-0045

Device.Input.HID.MCERemoteControlCompliance

Target Feature: Device.Input.HID

Title: Remote control that supports Media Center functionality complies with the appropriate Windows Remote Control Specification

Applicable OS Versions:

Windows 7 Client x86
Windows 7 Client x64

Description:

Beginning on January 1, 2008 all Windows Media Center remote controls must include the Windows Media Center Green Button Design as specified in the ***Remote Control and Receiver-Transceiver Specifications and Requirements for Windows Media Center in Windows Operating Systems***, and associated artwork.

A remote control is required if shipping with a system that exposes Windows Media Center and includes a TV tuner (including All-In-One and Touch enabled systems). ***

A remote control is optional with any laptop or if shipping tunerless systems including desktops and All-In-Ones.

If a remote control is included with any system that ships with Windows Media Center, the remote control and its matching receivers/transceiver components must implement Window Media Center buttons and functionality as described in the Windows Media Center Remote Control and Receiver/Transceiver Specification.

******After June 1, 2009, a Windows Media Center remote control is optional. If any remote is shipped with a system with Windows Media Center, then it must comply with the "Remote Control and Receiver-Transceiver Specifications and Requirements for Windows Media Center in Windows Operating Systems" specification.***

For WMC AQ in Windows 7 as a remote device: The device must pass all associated tests for INPUT-0006, SYSFUND-0217 and the WMC AQ SYSFUND-0216.

Design Notes:

Microsoft recommends that remote controls use either MC RC 6 IR protocol or the WMCQPprotocol.

Exceptions: Not Specified

Business Justification:

Products that follow these requirements are compatible with Windows Media Center in Windows.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

INPUT-0006

Device.Input.HID.UsbSpecificationCompliant

Target Feature: Device.Input.HID

Title: All HID devices that are connected over USB , should comply with the USB HID Specification (V1.1 or later)

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64

Description:

:

Version 5: Defines WMC AQ requirements

For WMC AQ, this requirement is

REQUIRED for Desktop systems

REQUIRED for Laptop systems if Laptop system includes remote and receiver.

For non AQ systems, this requirement is

REQUIRED If system (either Desktop or Laptop) includes remote and receiver.

IR Receivers (input only) and IR Transceivers (input and IR emitting/learning) interface with the Windows Media Center class driver. In a system with a TV tuner, IR Transceivers must be used that support all functions of the *Remote Control and Receiver-Transceiver Specifications and Requirements for Windows Media Center in Windows Operating Systems* document.

For tunerless systems with an IR receiver only IR Input and wake functions are required. The IR learning and emitting functions are optional.

DVB-T solutions are not required to support learning and emitting. These functions are optional.

In locales that do not use set top boxes, the learning and emitting functions are optional.

In those regions that support secondary 10 foot application, the IR Receiver/Transceivers are not required to meet the IR Learning requirement.

In those regions that support DVB-S Microsoft strongly recommends the use of IR Transceivers (input and IR emitting/learning)

If shipping a laptop system, IR learning and IR emitting is optional

For IR receivers (input only) that use Microsoft authorized IR protocols and all IR Transceiver (input and emitter functions), the device will return IR waveform envelope to software for software decoding of IR signal. IR signal cannot be decoded in the hardware. The only exception to this is the wake-from-remote power key.

An IR receiver (input only) is allowed to perform hardware decoding of an IR signal. These IR receivers (input only) must not receive and respond to any currently authorized Microsoft IR protocol. IR receivers that use hardware decoding of IR signal also need to support the wake-from-remote functionality. These devices must comply with the Remote *Control and Receiver-Transceiver Specifications and Requirements for Windows Media Center in Windows Operating Systems* document.

Design Notes:

Microsoft recommends that IR cables be labeled and well documented for end users. An insert showing a small diagram of the IR control cable and how it connects to the digital cable or satellite receiver could help prevent support calls.

Exceptions: Not Specified

Business Justification:

User Experience

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2006

Comments:

INPUT-0008

Device.Input.Keyboard

Description:

Certification requirements detailing the implementation details of a keyboard important to Microsoft Operating Systems

Related Requirements:

Device.Input.Keyboard.BrowserMultimediaKeysUseMSApis
Device.Input.Keyboard.DynamicKeyboards
Device.Input.Keyboard.HotKeyFunctionAPI
Device.Input.Keyboard.KernelModeDriversUseWdfKmdf
Device.Input.Keyboard.LogoFlagKey
Device.Input.Keyboard.MultipleKeyboard
Device.Input.Keyboard.PS2UniqueHWID
Device.Input.Keyboard.ScanCode

Device.Input.Keyboard.BrowserMultimediaKeysUseMSApis

Target Feature: Device.Input.Keyboard

Title: Keys for Internet browser and multimedia use Microsoft APIs

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64

Description:

If a keyboard or peripheral implements multimedia or Internet browser keys, it must use the registry keys associated with the WM_APPCOMMAND API to access those functions as described in the Windows Driver Kit. Registry keys can be programmed by using INF files to install special entries as defaults or through a customized interface provided to the user.

Design Notes:

See the Microsoft Platform SDK, "WM_APPCOMMAND."

Exceptions: Not Specified

Business Justification:

The use of this API creates a predictable platform for applications can respond appropriately to input.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

INPUT-0017

Device.Input.Keyboard.DynamicKeyboards

Target Feature: Device.Input.Keyboard

Title: Dynamic Keyboards meet the requirements listed here.

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client x86
Windows 8 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Server x64

Description:

When system is displaying the security desktop, a keyboard capable of altering the keycaps to reflect different glyphs or legends dynamically must present a keyboard layout to match the language the current user has active on the system.

When using a keyboard capable of altering the keycaps to reflect different glyphs or legends dynamically, the keyboards must reboot in to the default system language layout as selected in Control Panel -> Regional and Language Options -> Keyboards and Languages.

When using a keyboard capable of altering the keycaps to reflect different glyphs or legends dynamically, the keyboards must change keyboard layout and language as selected by a user at the login screen

When using a keyboard capable of altering the keycaps to reflect different glyphs or legends dynamically, the keyboards reflect the currently selected layout and language preference when the Windows desktop has focus.

When using a keyboard capable of altering the keycaps to reflect different glyphs or legends dynamically, the keyboard may allow for the repositioning of the Windows key, however that key must remain visible to user in all configurations/layouts. By default this key must be present in the lower left of the keyboard, between the control and alternate keys when at a login, welcome or password entry screen. Once the user has logged in the location of the key may be repositioned by user preference.

When using a keyboard capable of altering the keycaps to reflect different glyphs or legends dynamically, the displayed keyboard layout and language match the installed language of the Operating System.

A self-powered keyboard capable of altering the keycaps to reflect different glyphs or legends dynamically, must allow for the keyboard to be reset via a switch or keystroke sequence, independently of a software reset or power cycling of the device.

Exceptions: Not Specified

Business Justification:

For login accessibility, if a user needs to enter a password, the keyboard layout should represent the current language layout for text input. There are implications with passwords in a networked environment as well, not every system may be equipped the same type of keyboard. When a system

is rebooted a Dynamic Key-cap keyboard to go allow any user to walk up and enter login credentials based on a standard layout. This is especially true for self-powered keyboards which may not be reset during a system reboot. If the user changes the keyboard layout while the security desktop is active, the keyboard should respond to the layout change Keyboard shortcuts must function when Windows has focus. Keyboard must give the user a standard input set as recognized by the OS. Usability and Marketing value of having the Windows key consistently located in the lower quadrant of the keyboard is rationale for the positioning of the key. User preferences can be reflected but only when the system is in an unlocked state. Safe Mode is a diagnostic level of the operating system, designed to provide the customer an environment to make system changes and/or repairs. Maintaining a layout reflecting the OS language choice during install on a keyboard helps maintain the basic functionality expected in safe mode. Keyboards that are self-powered will not necessarily be reset when a system reboots. If the keyboard is 'stuck' in a broken state without the ability to reset the keyboard after a reboot the only recourse for the user is to power cycle the keyboard.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: Not Specified

Comments:

INPUT-0037

Device.Input.Keyboard.HotKeyFunctionAPI

Target Feature: Device.Input.Keyboard

Title: Devices that implement Hot-Key functionality implement the corresponding API notifications.

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64

Description:

Pointing and Drawing devices and Keyboards, that implement buttons used as application or function hot keys for which there exists WM_APPCOMMAND API shall implement the corresponding API notification.

This includes but is not limited to the following functions:

Volume Up/Down

Mute

Browser Back/Forward

Play/Pause

Design Notes:

Best practices for supporting Pointing and Drawing devices and Keyboards can be found at http://msdn.microsoft.com/en-us/library/ms997498.aspx.

API for WM_APPCOMMAND notifications can be found at http://msdn.microsoft.com/en-us/library/ms646275(VS.85).aspx.

HID Usage Page and ID information for these functions can be found at http://www.microsoft.com/whdc/archive/scancode.mspx and http://www.usb.org/developers/hidpage.

Exceptions: Not Specified

Business Justification:

The goal of this requirement is to ensure the devices work before additional drivers or software are installed. The scroll wheel working and standard buttons like the calculator key or media keys to work immediately after the device is plugged in. If IHVs have generic buttons, or extra functionality we want it enable it with value add software.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 12/1/2010

Comments:

INPUT-0065

Device.Input.Keyboard.KernelModeDriversUseWdfKmdf

Target Feature: Device.Input.Keyboard

Title: Keyboard kernel-mode drivers use the WDF-KMDF

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86

Windows 8 Client x64
Windows 8 Server x64

Description:

Third-party keyboard kernel-mode drivers must be ported to the WDF KMDF model.

Exceptions: Not Specified

Business Justification:

KMDF provides a well-defined object model and controls the lifetime of objects and memory allocations.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

INPUT-0010

Device.Input.Keyboard.LogoFlagKey

Target Feature: Device.Input.Keyboard

Title: Windows Logo flag key is implemented on all keyboards supporting more than 50 keys, the Windows Start Button design is required after June 30th 2007

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64

Description:

All keyboards supporting more than 50 keys must implement the Windows Logo flag key. The printed Windows flag logo version of the key design may be logo qualified on mobile systems and standalone keyboards until the transition date. The Windows flag trademark must be clearly distinguished on the key top according to *The Microsoft Windows Logo Key Logo License Agreement* and the "Key Support, Keyboard Scan Codes, and Windows" document at http://go.microsoft.com/fwlink/?LinkId=116451.

Mobile systems and keyboards must implement the new Windows Vista Start Button and logo artwork on the required Windows flag key (start key) by the transition date above. The keyboard must support the dome, graphics and other design requirements as defined in *The Microsoft Windows Logo Key Logo License Agreement*, and the "Key Support, Keyboard Scan Codes, and Windows" and "Windows Vista Hardware Start Button" documents. Vendors of systems or keyboards are encouraged to implement the dome design prior to this date. All designs must meet the requirements outlined in the specifications. It is recommended that keyboards supporting 50 or fewer keys implement a Windows Vista Start Button.

Design Notes:

The requirements defined in the "Windows Vista Hardware Start Button" paper call for a polished dome with a chamfer and a monochrome Windows Logo applied. The dome can be molded directly into the keycap. Optionally a domed full color Windows Flag insert can be implemented instead of the polished dome. Laptop and ultra mobile PC options are also defined.
All Windows Flag keys on a keyboard must use the same design implementation option. The updated Windows Vista Hardware Start Button is implemented using the same PS/2 Scan Codes and USB Usages as the previous Windows Key. A draft of the "Windows Vista Hardware Start Button" white paper can obtained on the WHDC website at http://go.microsoft.com/fwlink/?linkid=3757.

Exceptions: Not Specified

Business Justification:

The Start button is designed to be an easily discoverable to launch both the Windows Start menu and other experiences in Microsoft Windows starting with Windows Vista and newer operating systems.

Scenarios:

Launch the start menu as well as different Windows experiences.

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

INPUT-0016

Device.Input.Keyboard.MultipleKeyboard

Target Feature: Device.Input.Keyboard

Title: No interference occurs between multiple keyboards

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86

Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64

Description:

If the system includes more than one keyboard, there must be no conflicts. For example, a docked mobile computer can have more than one keyboard attached to the system. The keyboard ports on a mobile computer and a docking station must be able to resolve conflicts between the two ports when the mobile computer is docked.

Exceptions: Not Specified

Business Justification:

Microsoft Windows supports multiple keyboards connected through the registry and determines which keyboard to enable.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

INPUT-0014

Device.Input.Keyboard.PS2UniqueHWID

Target Feature: Device.Input.Keyboard

Title: All PS/2 connected devices (such as internal keyboards) must have a unique hardware ID

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64

Description:

All PS/2 connected devices (such as touchpads and keyboards) must have a unique hardware ID that enables the third party driver to ship with WU.

Design Notes:

See Microsoft unique hardware ID whitepaper
http://www.microsoft.com/whdc/device/input/mobileHW-IDs.mspx

Exceptions: Not Specified

Business Justification:

This requirement is needed to enable third-party touchpad drivers to be eligible for WU. A unique hardware ID will enable the system to determine the corrected driver needed for a particular device and download it through WU

Scenarios:

Internally attached keyboards show the correct and unique HWID and correct compatible ID.

Success Metric: The unique hardware ID should be in a format that allows WU to identify the device and load the correct driver for it. Success is third party touchpad drivers shipping with WU

Enforcement Date: June 1, 2012

Comments: Not Specified

Device.Input.Keyboard.ScanCode

Target Feature: Device.Input.Keyboard

Title: Scan codes comply with industry standard

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

The following are requirements for a keyboard design that includes any Windows logo keys:

- The keyboard must be developed according to technical requirements in "Key Support, Keyboard Scan Codes, and Windows."
- The keyboard must be compatible at the Windows virtual key-code level.
- The Windows logo key must function as a modifier (CTRL, SHIFT, or ALT).
- Keyboard manufacturers must use consumer control or vendor-specific, top-level collections for HID hot buttons.

Design Notes:

See "Key Support, Keyboard Scan Codes, and Windows" at http://go.microsoft.com/fwlink/?LinkId=36767.

Additional software or drivers can be written to provide software remapping functionality.

Exceptions: Not Specified

Business Justification:

Microsoft has defined extended scan codes for PS/2-compatible multimedia keyboards, and the USB HID Device Working Group has defined the consumer controls page. Hardware vendors must comply with these defined values and use their default functionality to ensure a good user experience following an upgrade or if the user does not install any supplemental software.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

INPUT-0016

Device.Input.PointDraw

Description:

Applies to Mice, touch pads, and other input devices used to move the pointer

Related Requirements:

Device.Input.PointDraw.KernelModeDriversUseWdfKmdf
Device.Input.PointDraw.PS2UniqueHWID

Device.Input.PointDraw.KernelModeDriversUseWdfKmdf

Target Feature: Device.Input.PointDraw

Title: Mouse kernel-mode drivers use the WDF-KMDF

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64

Description:

Third-party mouse kernel-mode drivers must be ported to the WDF KMDF model.

Exceptions: Not Specified

Business Justification:

KMDF drivers require minimal common code for default operations because most such code resides in the framework, where Microsoft can ensure that it is compatible with each successive release of Windows.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

INPUT-0010

Device.Input.PointDraw.PS2UniqueHWID

Target Feature: Device.Input.PointDraw

Title: All PS/2 connected embedded devices (such as touchpads) must have a unique hardware ID

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64

Description:

All PS/2 connected embedded devices (such as touchpads and keyboards) must have a unique hardware ID that enables the third party driver to ship with WU.

Design Notes: See Microsoft unique hardware ID whitepaper
http://www.microsoft.com/whdc/device/input/mobileHW-IDs.mspx

Exceptions: Not Specified

Business Justification:

This requirement is needed to enable third-party touchpad drivers to be eligible for WU. A unique hardware ID will enable the system to determine the corrected driver needed for a particular device and download it through WU

Scenarios:

Devices just work

Success Metric: The unique hardware ID should be in a format that allows WU to identify the device and load the correct driver for it. Success is third party touchpad drivers shipping with WU

Enforcement Date: June 1, 2012

Comments: Not Specified

Device.Input.Sensor.Accelerometer

Description: Not Specified

Related Requirements:

Device.Input.Sensor.Accelerometer.SensorDataType
Device.Input.Sensor.Accelerometer.SensorReportInterval
Device.Input.Sensor.Accelerometer.ShakeEvent

Device.Input.Sensor.Accelerometer.SensorDataType

Target Feature: Device.Input.Sensor.Accelerometer

Title: Accelerometer device drivers shall accurately report the right Data Types for Accelerometers as defined for the Sensors and Location Platform

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Client ARM

Description:

All accelerometer class sensors need to ensure that they accurately report the following Data Types to be seamlessly integrated with Windows (through the sensors platform) and exposed to applications.

Data type	Type	Meaning
SENSOR_DATA_TYPE_ACCELERATION_X_G	**VT_R8**	X-axis acceleration, in *g*s.
SENSOR_DATA_TYPE_ACCELERATION_Y_G	**VT_R8**	Y-axis acceleration, in *g*s.
SENSOR_DATA_TYPE_ACCELERATION_Z_G	**VT_R8**	Z-axis acceleration, in *g*s.

*Clarify what G = Gravitational Constant = 32.2 ft/sec2 and 9.8 m/sec2

Note: Sensor Connection Type = SENSOR_CONNECTION_TYPE_PC_INTEGRATED for hardware that is built-in to the PC enclosure.

For detailed information regarding sensor driver development, please see the Sensors topic in the Device and Driver Technologies section of the Windows Driver Kit (WDK).

Exceptions: Not Specified

Business Justification:

To ensure the accelerometer reports data in the standardized windows Data Types. This will enable smooth integration into the screen orientation rotation manager and platform APIs.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: Windows 8 RC

Comments:

New

Device.Input.Sensor.Accelerometer.SensorReportInterval

Target Feature: Device.Input.Sensor.Accelerometer

Title: Accelerometer function driver and firmware report data with minimum report interval of 16ms (for a 60Hz frequency for gaming)

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Client ARM

Description:

All accelerometer class sensors need to ensure that they accurately report the data at the required sampling rates to be efficient for gaming applications as well as power managed when not in full use.

Sensor Property	Type	Meaning	
SENSOR_PROPERTY_MIN_REPORT_INTERVAL	VT_R8	The minimum elapsed time setting that the hardware supports for sensor data report generation, in milliseconds	
Application Scenario	**Fidelity**	**Hertz**	**Report Interval (msec)**
Augmented Reality / Rich Gaming	High_Fidelity	60	16
Casual Gaming	Medium_Fidelity	30	33
Rotation Manager	Low_Fidelity	15	67

All accelerometer type sensors must support these report intervals to ensure a consistent experience for application categories. Intermediate report intervals may be optionally supported.

Exceptions: Not Specified

Business Justification:

To ensure the accelerometer reports data in the standardized way that all applications can rely on. These three categories of report intervals (sampling rate) should satisfy majority of the accelerometer application categories. Vertical solutions are allowed to support other report interval rates needed for the specific application.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: Windows 8 RC

Comments:

New

Device.Input.Sensor.Accelerometer.ShakeEvent

Target Feature: Device.Input.Sensor.Accelerometer

Title: If an accelerometer device driver (optionally) supports a shake gesture, it must use the correct Event ID and expose to the Sensors and Location Platform.

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Client ARM

Description:

An accelerometer r(or related) sensors may optionally report a shake gesture to the sensor platform, when the user shakes the embedded sensor in the system. The following is the Data Event to be seamlessly integrated with Windows (through the sensor's platform) and exposed to applications.

Data Event	Meaning
SENSOR_EVENT_ACCELEROMETER_SHAKE	The user has shaken the system to trigger an application event.

This is validated in code by calling: ISensor::SupportsEvent()

For detailed information regarding sensor driver development, please see the Sensors topic in the Device and Driver Technologies section of the Windows Driver Kit (WDK).

Exceptions: Not Specified

Business Justification:

This is an optional properly that could be implemented by an accelerometer. When the system is shaken, it sends a shake event to the application. Having a consistent way to recognizing shake allows multiple applications to have a consistent experience. For more details on this event and how the sensor should detect shake, please visit the Sensors Design Guide.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: Windows 8 RC

Comments:

New

Device.Input.Sensor.ALS

Description: Not Specified

Related Requirements:

Device.Input.Sensor.ALS.SupportRequiredData

Device.Input.Sensor.ALS.SupportRequiredData

Target Feature: Device.Input.Sensor.ALS

Title: Ambient Light Sensors must support and generate the required data

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client ARM

Description:

Data Field Support for Ambient Light Sensor

Ambient light sensors identify themselves as SENSOR_TYPE_AMBIENT_LIGHT though Device Driver Interfaces ISensorDriver::OnGetSupportedSensorObjects and ISensorDriver::OnGetProperties()).

Ambient Light sensors must support light readings in lux to ensure all light-aware applications can expect consistent data. This data is queried from Device Driver Interface ISensorDriver::OnGetSupportedDataFields() and ISensorDriver::OnGetDataFields()).

Built in ambient light sensors can be used by the Adaptive Brightness feature in Windows if they also set the SENSOR_PROPERTY_CONNECTION_TYPE property (of data type VT_UI4) to SENSOR_CONNECTION_TYPE_PC_INTEGRATED. This is optional, but recommended for built-in sensors.

Data field	Data type	Definition
SENSOR_DATA_TYPE_LIGHT_LEVEL_LUX	VT_R4	The current light level being measured (in lux).

The requirements below ensure that the driver raises ALS report events in the correct manner. If the correct behavior is not followed, client applications will not receive data.

Generating Reports

The device must be able to generate a series of reports containing SENSOR_DATA_TYPE_LIGHT_LEVEL_LUX.

Verify Sensor can provide dynamic data report demonstrating a decrease and subsequent increase in LUX.

Design Notes: The new Sensor and Location Platform enables sensor and location devices to be easily accessible through two new APIs: the Sensor API and the Location API. The Sensor API provides consistent access to information from physical and logical sensor devices. The Location API is built on the Sensor API and streamlines access to location specific information.

The Sensor and Location Platform will rely on data provided to the platform via device drivers that have implemented the Sensor Device Driver Interface. This document details the Windows Logo tests which will be applied against devices (and their supporting drivers) applying for logo certification for the Sensor and Location Platform. In most cases devices will be verified by using the Sensor and Location Platform.

Ambient Light sensors must comply with Logo Requirement INPUT-0048 (*Sensor and Location Platform devices must properly support the required set of data and properties*) in addition to the requirements described here. These requirements are designed to help ensure the following goals are met:

Sensor devices have high quality hardware and drivers.

Sensor devices interact with the APIs in a consistent and reliable manner.

Exceptions: Not Specified

Business Justification:

To ensure the ALS sensor drivers that receive a logo meet a high quality bar, it is necessary that the requirements include tests for the behavior of the device. The proposed additions ensure that the driver raises ALS report events in the correct manner. If the correct behavior is not followed, client applications will not receive data.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2009

Comments:

INPUT-0050

Device.Input.Sensor.Base

Description: Not Specified

Related Requirements:

Device.Input.Sensor.Base.SupportDataTypesAndProperties

Device.Input.Sensor.Base.SupportDataTypesAndProperties

Target Feature: Device.Input.Sensor.Base

Title: Sensor and Location Platform devices support the set of data types and properties as defined in this requirement

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client ARM

Description:

All sensor devices that implement the Sensor Device Driver Interface must meet the following requirements. See any additional requirements that are specific to the sensor type being tested. For detailed information regarding sensor driver development, please see the Sensors topic in the Device and Driver Technologies section of the Windows Driver Kit (WDK).

Required Properties

Applications that use the Sensor API use these properties to manage devices and show meaningful data to end users. The Sensor and Location Platform uses these properties to populate the details in Control Panel.

These properties are queried using Device Driver Interfaces ISensorDriver::OnGetSupportedSensorObjects, ISensorDriver::OnGetSupportedProperties() and ISensorDriver::OnGetProperties(). Explicit type matching is required data types for these properties must be the same as in the chart below.

Property	Data type	Details
WPD_FUNCTIONAL_OBJECT_CATEGORY	VT_CLSID	Used by ISensorManager::GetSensorsByCategory() to enable search by sensor category (such as Location)
SENSOR_PROPERTY_TYPE	VT_CLSID	Used by ISensorManager::GetSensorsByType() to enable search by sensor type (such as GPS)
SENSOR_PROPERTY_STATE	VT_UI4	The current value of the SensorState enum. Indicates the state of the sensor (such as Ready or Error)
SENSOR_PROPERTY_PERSISTENT_UNIQUE_ID	VT_CLSID	Enables the Sensor API and clients uniquely identify the sensor device
SENSOR_PROPERTY_MANUFACTURER	VT_LPWSTR	Metadata (used by Location and Other Sensors in Control Panel)

SENSOR_PROPERTY_MODEL	VT_LPWSTR	Metadata (used by Location and Other Sensors in Control Panel)
SENSOR_PROPERTY_SERIAL_NUMBER	VT_LPWSTR	Metadata for applications
SENSOR_PROPERTY_FRIENDLY_NAME	VT_LPWSTR	Metadata (used by Location and Other Sensors in Control Panel)
SENSOR_PROPERTY_MIN_REPORT_INTERVAL	VT_UI4	Metadata for applications

Required Static Properties

The following properties must not change value over time, so that the Sensor and Location Platform (and applications) can use them to select and manage sensors. These properties are queried using Device Driver Interfaces ISensorDriver::OnGetSupportedProperties() and ISensorDriver::OnGetProperties().

Data types for these properties must match those in the following chart.

The following properties are static in the Sensor API.

Property	Data type
WPD_FUNCTIONAL_OBJECT_CATEGORY	VT_CLSID
SENSOR_PROPERTY_TYPE	VT_CLSID
SENSOR_PROPERTY_PERSISTENT_UNIQUE_ID	VT_CLSID
SENSOR_PROPERTY_MANUFACTURER	VT_LPWSTR
SENSOR_PROPERTY_MODEL	VT_LPWSTR
SENSOR_PROPERTY_SERIAL_NUMBER	VT_LPWSTR
SENSOR_PROPERTY_FRIENDLY_NAME	VT_LPWSTR
SENSOR_PROPERTY_MIN_REPORT_INTERVAL	VT_UI4

Settable Properties

Applications can use settable properties to configure the driver (such as optimize for power or other factors). Other settable properties can be exposed besides the following one, but if this property is exposed, it must be settable.

Settable properties are optional. It is not required that sensor devices provide settable properties. These properties are queried by using Device Driver Interface ISensorDriver::OnGetProperties and set by using Device Driver Interface ISensorDriver::OnSetProperties.

The following is considered a Settable Properties in the Sensor API.

Property	Data type	Details
SENSOR_PROPERTY_CURRENT_REPORT_INTERVAL	VT_UI4	Sets the minimum frequency (in milliseconds) that a client wants to receive data reports from the

		sensor. This property should be tracked on a per client basis.

The SENSOR_PROPERTY_CHANGE_SENSITIVITY (VT_UNKNOWN) should be settable when present but is not part of the requirement test.

The SENSOR_PROPERTY_CHANGE_SENSITIVITY property is used to set the threshold of how much a data field must change before an event is fired.

Data Fields

Sensor devices are useful only when they report data. Each device must report at least one data field in addition to SENSOR_DATA_TYPE_TIMESTAMP (VT_FILETIME). Data fields are exposed to the Sensor API by using Device Driver Interface ISensorDriver::OnGetSupportedDataFields() and ISensorDriver::OnGetDataFields().

Missing Properties

The device driver must correctly handle properties that a sensor device does not support. This enables the Sensor and Location Platform to correctly notify applications when sensor properties are missing.

Drivers must return S_FALSE from Device Driver Interfaces ISensorDriver::OnGetProperties and ISensorDriver::OnSetProperties if one or more of the requested properties is not present on the device.

For each missing property the PropVariant for the requested property must have a type of VT_ERROR and a value of HRESULT_FROM_WIN32(ERROR_NOT_FOUND). The driver can return valid values for properties it does have alongside invalid values.

The follow requirements ensure the sensor drivers that receive a certification can report data reliably, obey the sensor driver state model and implement data timestamps correctly.

Reliability

The driver must continue to operate after four hours of "get data" and "get property" calls.

The driver must continue to operate when readable/writeable properties are set and read.

The driver must provide data and properties after completion of sleep/resume or hibernation/resume cycle(s)

Timestamp

The driver must report an accurate relative timestamp with each report. This timestamp must be greater than the initial prompt to provide a timestamp and must be less than or equal to the time the event is received.

Ready State Validation

The driver must properly report SENSOR_STATE_READY within five minutes after disabling and enabling the sensor in device manager.

The sensor will only report SENSOR_STATE_READY if there is a valid GPS fix available.

Exceptions: Not Specified

Business Justification:

To ensure the sensor drivers that receive a logo meet a high quality bar, it is necessary that the requirements include tests for the behavior of the device. The proposed additions ensure that the driver can report data reliably, obey the sensor driver state model and implement data timestamps correctly.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2009

Comments:

INPUT-0048

Device.Input.Sensor.Compass

Description: Not Specified

Related Requirements:

Device.Input.Sensor.Compass.InclinometerDataType
Device.Input.Sensor.Compass.SensorDataType
Device.Input.Sensor.Compass.SensorReportInterval

Device.Input.Sensor.Compass.InclinometerDataType

Target Feature: Device.Input.Sensor.Compass

Title: A tilt compensated compass device driver shall accurately report the right Data Types for an inclinometer (leveraging the accelerometer and compass together) as defined for the Sensors and Location Platform

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Client ARM

Description:

A device that exposes a tilt compensated compass shall also properly expose a 3Daccelerometer and a 3D inclinometer. This enables the platform to obtain 3D inclinometer data types. This will be calculated in hardware (or device driver) using the accelerometer and compass data elements and reported to the sensor platform using the Data Types below.

Data Type	Type	Mandatory / Optional	Meaning
SENSOR_DATA_TYPE_TILT_X_DEGREES (Pitch)	**VT_R8**	Mandatory	Pitch inclination, in *degree*s
SENSOR_DATA_TYPE_TILT_Y_DEGREES (Roll)	**VT_R8**	Mandatory	Roll inclination, in *degree*s
SENSOR_DATA_TYPE_TILT_Z_DEGREES (Yaw)	**VT_R8**	Mandatory	Yaw inclination, in *degree*s

Please follow this order for operation:

Z, then X, then Y (Yaw, then Pitch , then Roll)

http://dev.w3.org/geo/api/spec-source-orientation.html

Note: Sensor Connection Type = SENSOR_CONNECTION_TYPE_PC_INTEGRATED for hardware that is built-in to the PC enclosure.

For detailed information regarding sensor driver development, please see the Sensors topic in the Device and Driver Technologies section of the Windows Driver Kit (WDK).

Exceptions: Not Specified

Business Justification:

To ensure the accelerometer and compass reports data in the standardized windows Data Types. This will enable smooth integration into the windows rotation manager and exposed to applications. Critical for gaming scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: Windows 8 RC

Comments:

New

Device.Input.Sensor.Compass.SensorDataType

Target Feature: Device.Input.Sensor.Compass

Title: A tilt compensated compass sensor shall accurately report the right data types for a tilt compensated compass as defined for the Sensors and Location Platform

Applicable OS Versions:

Windows 8 Client x86

Windows 8 Client x64
Windows 8 Client ARM

Description:

All tilt compensated compass class sensors need to ensure that they accurately report the following Data Types to be seamlessly integrated with Windows (through the sensors platform) and exposed to applications. A tilt compensated compass must be able to report degrees while tilted at angles, requiring internal accelerometer data to orient the compass when held at an angle.

Data type	Type	Mandatory / Optional	Meaning
SENSOR_DATA_TYPE_MAGNETIC_HEADING_COMPENSATED_MAGNETIC_NORTH_DEGREES	**VT_R4**	Mandatory	Tilt compensated compass reading with respect to Magnetic North, in *degrees*.
SENSOR_DATA_TYPE_MAGNETIC_HEADING_COMPENSATED_TRUE_NORTH_DEGREES	**VT_R4**	Optional	Tilt compensated compass reading with respect to True North, in *degrees*. (if the device and/or driver have access to location data).

Note: Sensor Connection Type = SENSOR_CONNECTION_TYPE_PC_INTEGRATED for hardware that is built-in to the PC enclosure.

Tilt compensation is performed by means of integration with 3D accelerometer hardware.

For detailed information regarding sensor driver development, please see the Sensors topic in the Device and Driver Technologies section of the Windows Driver Kit (WDK).

Exceptions: Not Specified

Business Justification:

To ensure the compass reports data in the standardized windows Data Types. This will enable smooth integration into the windows location and heading projections to applications.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: Windows 8 RC

Comments:

New

Device.Input.Sensor.Compass.SensorReportInterval

Target Feature: Device.Input.Sensor.Compass

Title: Compass function driver and firmware report data with minimum report interval of 200ms (5Hz frequency) as minimum - this value can have a lower (faster) value

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Client ARM

Description:

All compass class sensors need to ensure that they accurately report the data at the required sampling rates to be efficient for mapping applications as well as power managed when not in full use.

Sensor Property	Type	Meaning	
SENSOR_PROPERTY_CURRENT_REPORT_INTERVAL	VT_R8	The current elapsed time for sensor data report generation, in milliseconds.	
Application Scenario	**Fidelity**	**Hertz**	**Report Interval (msec)**
Mapping	Low_Fidelity	5	200

Compass sensors may report data more frequently if the hardware capabilities exist, but they must meet the above requirement as a minimum reporting rate.

Exceptions: Not Specified

Business Justification:

Ensure the compass and inclinometer reports data in the standardized way that all applications can rely on. These categories of report intervals (sampling rate) should satisfy majority of the application

categories. Vertical solutions are allowed to support other report interval rates needed for the specific application.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: Windows 8 RC

Comments:

New

Device.Input.Sensor.DeviceOrientation

Description: Not Specified

Related Requirements:

Device.Input.Sensor.DeviceOrientation.SensorDataType

Device.Input.Sensor.DeviceOrientation.SensorDataType

Target Feature: Device.Input.Sensor.DeviceOrientation

Title: A device that contains a Gyroscope, Compass and Accelerometer, shall accurately report the right Data Types of the individual sensors along with a singular Device Orientation Data Type as defined for the Sensors and Location Platform

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Client ARM

Description:

All device that exposes a device orientation sensor (SENSOR_TYPE_AGGREGATED_DEVICE_ORIENTATION) needs to ensure that it accurately report the following Data Types to be seamlessly integrated with Windows (through the sensors platform) and exposed to applications.

Sensor data types

SENSOR_DATA_TYPE_ROTATION_MATRIX	**{1637D8A2-4248-4275-865D-558DE84AEDFD}, 16**	**VT_VECTOR|VT_UI1 Counted array representing the orientation of the device in 3D space as a 3x3 rotation matrix. Data for vector types is always serialized as VT_UI1 (an array of unsigned, 1-byte characters). This data field must contain**

		each value as a single-precision float (VT_R4).
SENSOR_DATA_TYPE_QUATERNION	{1637D8A2-4248-4275-865D-558DE84AEDFD}, 17	VT_VECTOR|VT_UI1 The x, y, z, w values of a quaternion representing the orientation of the device in 3D space. Data for vector types is always serialized as VT_UI1 (an array of unsigned, 1-byte characters). This data field must contain each value as a single-precision float (VT_R4).

Note: Sensor Connection Type = SENSOR_CONNECTION_TYPE_PC_INTEGRATED for hardware that is built-in to the PC enclosure.

For detailed information regarding sensor driver development, please see the Sensors topic in the Device and Driver Technologies section of the Windows Driver Kit (WDK).

Exceptions: Not Specified

Business Justification:

To ensure the combined sensors (Gyroscope, Compass, Accelerometer)sensor reports data in the standardized windows Data Types. If a sensor does not report these data fields, it will not be treated as a human presence sensor and will not be exposed to applications as a human presence sensor.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: Windows 8 RC

Comments:

New

Device.Input.Sensor.Gyroscope

Description: Not Specified

Related Requirements:

Device.Input.Sensor.Gyroscope.SensorDataType
Device.Input.Sensor.Gyroscope.SensorReportInterval

Device.Input.Sensor.Gyroscope.SensorDataType

Target Feature: Device.Input.Sensor.Gyroscope

Title: Gyroscope device drivers shall accurately report the right Data Types for Gyroscopes as defined for the Sensors and Location Platform

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Client ARM

Description:

All gyroscope class sensors need to ensure that they accurately report the following Data Types to be seamlessly integrated with Windows (through the sensors platform) and exposed to Modern Applications.

Data type	Type	Meaning
SENSOR_DATA_TYPE_ANGULAR_VELOCITY_X_DEGREES_PER_SECOND	**VT_R8**	Angular velocity around X-axis , in degrees per second.
SENSOR_DATA_TYPE_ANGULAR_VELOCITY_Y_DEGREES_PER_SECOND	**VT_R8**	Angular velocity around Y-axis , in degrees per second.
SENSOR_DATA_TYPE_ANGULAR_VELOCITY_Z_DEGREES_PER_SECOND	**VT_R8**	Angular velocity around Z-axis , in degrees per second.

Sensor Connection Type = SENSOR_CONNECTION_TYPE_PC_INTEGRATED for hardware that is built-in to the PC enclosure.

For detailed information regarding sensor driver development, please see the Sensors topic in the Device and Driver Technologies section of the Windows Driver Kit (WDK).

Exceptions: Not Specified

Business Justification:

To ensure the gyroscope reports data in the standardized windows Data Types. If a sensor does not report these data fields, it will not be treated as a gyroscope and will not be exposed to applications as a gyroscope sensor.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: Windows 8 RC

Comments:

New

Device.Input.Sensor.Gyroscope.SensorReportInterval

Target Feature: Device.Input.Sensor.Gyroscope

Title: Gyroscope function driver and firmware report data with minimum report interval of 16ms (for a 60Hz frequency for gaming)

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Client ARM

Description:

All gyroscope class sensors need to ensure that they accurately report the data at the required sampling rates to be efficient for gaming applications as well as power managed when not in full use.

Sensor Property	Type		Meaning
SENSOR_PROPERTY_CURRENT_REPORT_INTERVAL	VT_R8		The current elapsed time for sensor data report generation, in milliseconds.
Application Scenario	**Fidelity**	**Hertz**	**Report Interval (msec)**
Augmented Reality / Rich Gaming	High_Fidelity	60	16
Casual Gaming	Medium_Fidelity	30	33
Rotation Manager	Low_Fidelity	15	67

All gyroscope type sensors must support these report intervals to ensure a consistent experience for application categories. Intermediate report intervals may be optionally supported.

Exceptions: Not Specified

Business Justification:

To ensure the gyroscope reports data in the standardized way that all apps can rely on. These categories of report intervals (sampling rate) should satisfy majority of the application categories. Vertical solutions are allowed to support other report interval rates needed for the targeted application.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: Windows 8 RC

Comments:

New

Device.Input.Sensor.Location

Description: Not Specified

Related Requirements:

Device.Input.Sensor.Location.SupportRequiredDataFieldsForReport

Device.Input.Sensor.Location.SupportRequiredDataFieldsForReport

Target Feature: Device.Input.Sensor.Location

Title: Location Sensors must support and generate the required data fields for at least one built-in report

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client ARM

Description:

All location devices need to ensure that they report location data at the required report interval taking into account the application desired accuracy to provide location data as efficiently as possible. Location devices should maintain the lowest power state possible when not in use. Location devices should enter a low power state (if possible) when not in the process of acquiring location data.

Data Field Support for Location Reports

- The Location API exposes location data through standard built-in reports. These reports are populated from location sensors that the Location API manages automatically.
- Location sensor devices that report their category as SENSOR_CATEGORY_LOCATION (though Device Driver Interfaces ISensorDriver::OnGetSupportedSensorObjects and ISensorDriver::OnGetProperties()) must support one of the built-in reports so that the location API can manage the sensor and report its data. Each built-in report has a set of expected data fields (though Device Driver Interface ISensorDriver::OnGetSupportedDataFields() and ISensorDriver::OnGetDataFields()) that the Location API looks for.
- As stated in Logo Requirement Device.Input.Sensor.Base.SupportDataTypesAndProperties, all built-in reports require SENSOR_DATA_TYPE_TIMESTAMP to be provided in addition to the designated fields. This field reports the time the data was collected in UTC.

The Location API supports two built-in reports: LatLongReport and CivicAddressReport. A sensor can support either or both of these reports simultaneously.

Important: If a sensor supports both reports, the data reported to each should correspond to the same location. For example, do not report a latitude and longitude that does not match the civic address data.

Lattitude/Longitude (lat/long) sensors are required.LatLongReport is required for all location sensors.

To support LatLongReport, the following fields are required.

Data field	Data type	Details
SENSOR_DATA_TYPE_LATITUDE_DEGREES	VT_R8	Shows the current latitude in degrees, which must be in the range [-90, +90]
SENSOR_DATA_TYPE_LONGITUDE_DEGREES	VT_R8	Shows the current longitude in degrees, which must be in the range [-180, +180]
SENSOR_DATA_TYPE_ERROR_RADIUS_METERS	VT_R8	The actual latitude and longitude must be within a circle, with this radius (in meters) drawn around the reported (latitude, longitude). Enables the Location API to prioritize sensors; it should be updated dynamically and must be non-zero and positive.

As part of the LatLongReport, the following fields are optional, but must be reported when available.

Optional Data field	Data type	Details
SENSOR_DATA_TYPE_ALTITUDE_ELLIPSOID_METERS	VT_R8	The altitude with regards to the WGS84 ellipsoid (in meters)
SENSOR_DATA_TYPE_ALTITUDE_ELLIPSOID_ERROR_METERS	VT_R8	The error of the current altitude measurement (in meters)

Important: The Location API supports an additional set of data fields (which correspond to NMEA data fields). For more information, see the Sensors topic in the Device and Driver Technologies section of the WDK.

All CivicAddressReport reportsmust contain the SENSOR_DATA_TYPE_COUNTRY_REGION property:

Data field	Data type	Details
SENSOR_DATA_TYPE_COUNTRY_REGION	VT_LPWSTR	Shows the ISO 3166 string representation of the country or region where the sensor is located.

As part of the CivicAddressReport, the following fields are optional, but must be reported when available.

Optional Data field	Data type	Details

SENSOR_DATA_TYPE_ADDRESS1	VT_LPWSTR	Shows the 1st line of the address where the sensor is located.
SENSOR_DATA_TYPE_ADDRESS2	VT_LPWSTR	Shows the 2nd line of the address where the sensor is located.
SENSOR_DATA_TYPE_CITY	VT_LPWSTR	Shows the city where the sensor is located.
SENSOR_DATA_TYPE_STATE_PROVINCE	VT_LPWSTR	Shows the state or province where the sensor is located.
SENSOR_DATA_TYPE_POSTALCODE	VT_LPWSTR	Shows the postal code where the sensor is located.

The requirement below ensures that the driver raises location report events in the correct manner. If the correct behavior is not followed, the Location API will block the sensor, and client applications will not receive data.

Generating Sensor Data Reports

Device must be able to generate a series of valid ISensorDataReports for one of or both of the two Location Reports types. Each report generated must contain at least the minimum data fields for the report type.

The Sensor and Location Platform will rely on data provided to the platform via device drivers that have implemented the Sensor Device Driver Interface. In most cases devices will be verified by using the Sensor and Location Platform.

Location sensors must comply with certification requirement Device.Input.Sensor.Base.SupportDataTypesAndProperties (Sensor and Location Platform devices must properly support the required set of data and properties) in addition to the requirements described here. These requirements are designed to help ensure the following goals are met:

Sensor devices have high quality hardware and drivers.

Sensor devices interact with the APIs in a consistent and reliable manner.

Sensor drivers must monitor connect client count and put their respective device into the lowest power state possible (preferably D3) when connected clients = 0 as illustrated in the HID Sensor Sample in the WDK.

Notes about settable properties:

The following settable properties must be utilized in the device driver as filtering and power management criteria. Both properties must be tracked on a per-client basis.

The properties are required to be implemented as settable.

Property	Data type	Details
SENSOR_PROPERTY_CURRENT_REPORT_INTERVAL	VT_UI4	From Requirement-Device.Input.Sensor.Base.SupportDataTypesAndProperties - Sets the minimum frequency (in milliseconds)

		that a client wants to receive data reports from the sensor.
SENSOR_PROPERTY_LOCATION_DESIRED_ACCURACY	VT_UI4	Sets a hint as to how accurate the driver should strive to be. DESIRED_ACCURACY_DEFAULT: The sensor should optimize power and other cost considerations. GPS sensors will not be enumerated by the Location API unless location data is unavailable from other providers on the system or data accuracy is less than 500m. DESIRED_ACCURACY_HIGH: The sensor should deliver the highest accuracy report possible. The Location API will always connect to all location sensors (including GPS) in order to acquire the most accurate position possible.

Exceptions: Not Specified

Business Justification:

To ensure the location sensor drivers that receive a logo meet a high quality bar, it is necessary that the requirements include tests for the behavior of the device. The proposed additions ensure that the driver raises location report events in the correct manner. If the correct behavior is not followed, the Location API will block the sensor, and client applications will not receive data.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2009

Comments:

INPUT-0049

Device.Input.Sensor.Presence

Description: Not Specified

Related Requirements:

Device.Input.Sensor.Presence.SensorDataType

Device.Input.Sensor.Presence.SensorDataType

Target Feature: Device.Input.Sensor.Presence

Title: Human Presence device drivers shall accurately report the right Data Types for Human Presence as defined for the Sensors and Location Platform

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Client ARM

Description:

All human presence class sensors need to ensure that they accurately report the following Data Types to be seamlessly integrated with Windows (through the sensors platform).

Data type	Type	Meaning
SENSOR_DATA_TYPE_HUMAN_PRESENCE	**VT_R8**	Boolean value indicating presence of an object (presumed to be a human).
SENSOR_DATA_TYPE_HUMAN_PROXIMITY_METERS	**VT_R8**	[Optional] Range value indicating distance of an object (presumed to be a human) from the proximity sensor. Value reported in meters. The value 0.0 meters should only be used for situations where zero-distance events (such as a case closing on tablet) have been detected).

Note: Sensor Connection Type = SENSOR_CONNECTION_TYPE_PC_INTEGRATED for hardware that is built-in to the PC enclosure. Note that presence sensors with connection type = SENSOR_CONNECTION_TYPE_PC_ATTACHED can also be used for power management features (if integrated into connected peripheral).

For detailed information regarding sensor driver development, please see the Sensors topic in the Device and Driver Technologies section of the Windows Driver Kit (WDK).

Exceptions: Not Specified

Business Justification:

To ensure the human presence sensor reports data in the standardized windows Data Types. If a sensor does not report these data fields, it will not be treated as a human presence sensor and will not be exposed to applications as a human presence sensor.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: Windows 8 RC

Comments:

New

Device.Input.SmartCardMiniDriver

Description:

MiniDriver program for SmartCards

Related Requirements:

Device.Input.SmartCardMiniDriver.DoNotStopWhenResourcesAreUnavailable
Device.Input.SmartCardMiniDriver.SpecsAndCertifications
Device.Input.SmartCardMiniDriver.SupportMultipleInstancesOnASystem

Device.Input.SmartCardMiniDriver.DoNotStopWhenResourcesAreUnavailable

Target Feature: Device.Input.SmartCardMiniDriver

Title: Smart card driver does not stop the system if required resources are not available

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client ARM
Windows 8 Server x64
Windows Server 2008 Release 2 x64

Description:

A smart card driver must not interrupt system operation if resources that are required by the reader are not available.

Exceptions: Not Specified

Business Justification:

This requirement helps ensure that smart card reader minidrivers are reliable and do not negatively affect system operation.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2006

Comments:

INPUT-0018

Device.Input.SmartCardMiniDriver.SpecsAndCertifications

Target Feature: Device.Input.SmartCardMiniDriver

Title: Windows Smart Card Minidrivers meet Windows Smart Card Minidriver Version 5 Specifications and Certification Criteria

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM
- Windows Server 2008 Release 2 x64
- Windows 8 Server x64

Description:

Smart Card Minidrivers are pluggable security components that provide an abstraction layer between the base CSP and the smartcard to provide secure storage for cryptographic keys and certificates. Smart Card Minidrivers perform secure cryptographic operations including encryption, decryption, key establishment, key exchange and digital signatures. Smart Card Minidrivers also include other form factors, such as a USB tokens or other personal trusted devices.

Smart Card Minidrivers must adhere to the following specifications:

- Smart Card Minidriver Specification for Windows Base Cryptographic Service Provider (Base CSP) and Smart Card Key Storage Provider (KSP), Version 5.06 (or later).
- Smart Card Minidriver Certification Criteria, Version 5.06 (or later).

Smart Card Minidrivers must adhere to the following basic criteria:

- If the device submitted for testing is a smart card and has a ISO 7816 ID-1 smart card form factor, it must be tested with a smart card reader that has passed the WHQL Testing Requirements for smart cards.
- If the device is a multi-function device, it must pass the certification requirements for each device category if a logo program exists.
- The card minidriver may not implement additional functionality beyond that specified in the Card Minidriver Specification.
- The card minidriver may not contain any Trojans or "backdoors".
- The card minidriver may not present any UI to the end user.
- All cryptographic operations must take place on the device.

All cryptographic keys must be stored on the device and must not be exportable from the device.

Smart Card Minidrivers must adhere to the following general criteria in the Card Minidriver Certification Criteria:

Card Minidriver Management and Installation

Card Minidriver Logical File System Requirements

Card Minidriver General Conventions

Card Minidriver Memory Management

Optional General Requirements

Smart Card Minidriver must adhere to the criteria governing each of the Functional Exports for each function in the Card Minidriver specification, including:

Functionality

Performance

Error Handling

Smart Card Minidrivers must support the sequenced invocation of card minidriver functions.

A Smart Card Minidriver may support multiple smart cards, but must pass the certification criteria for each of the supported smart cards separately.

Design Notes:

See Smart Card Minidriver Specification for Windows Base Cryptographic Service Provider (Base CSP) and Smart Card Key Storage Provider (KSP), Version specifications at http://msdn.microsoft.com/en-us/windows/hardware/gg487500.aspx

See Smart Card Minidriver Certification Criteria, at http://msdn.microsoft.com/en-us/windows/hardware/gg487504.

The following table describes the minimum and maximum specification version that must be supported on any given OS family:

Operating system family	Allowed specification versions
Windows 7 client	5,6,7 (any combination allowed such as 5 and 7 only, 5 only, 7 only, 5 and 6 only, 6 and 7 only etc.)
Windows Server 2008	5,6,7 (any combination allowed such as 5 and 7 only, 5 only, 7 only, 5 and 6 only, 6 and 7 only etc.)
Windows 8 client	6,7,8 (any combination allowed such as 6 and 8 only, 6 only, 7 only, 6 and 6 only, 7 and 8 only etc.)
Windows Server	6,7,8 (any combination allowed such as 6 and 8 only, 6 only, 7 only, 6 and 6

v.Next	only, 7 and 8 only etc.)

Exceptions: Not Specified

Business Justification:

This requirement helps ensure that smart card reader minidrivers are reliable and do not negatively affect system operation.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 1/31/2008

Comments:

INPUT-0030

Device.Input.SmartCardMiniDriver.SupportMultipleInstancesOnASystem

Target Feature: Device.Input.SmartCardMiniDriver

Title: Smart card driver can support multiple instances of the same device on a system

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client ARM
Windows 8 Server x64
Windows Server 2008 Release 2 x64

Description:

The smart card driver must be able to function properly if more than one instance of the devices is installed on a system. Functionality of each device instance must be consistent, loading a separate instance cannot reduce functionality.

Exceptions: Not Specified

Business Justification:

This requirement helps ensure that smart card reader minidrivers are reliable and do not negatively affect system operation.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2006

Comments:

INPUT-0019

Device.Input.SmartCardReader

Description: Not Specified

Related Requirements:

Device.Input.SmartCardReader.PinDataEntryKeyboardCompliesWithIso
Device.Input.SmartCardReader.SmartCardService
Device.Input.SmartCardReader.Supports258And259BytePackets
Device.Input.SmartCardReader.SupportsDirectAndInverseConvention
Device.Input.SmartCardReader.SupportsInsertionAndRemovalMonitor
Device.Input.SmartCardReader.SupportsMinClockFrequency
Device.Input.SmartCardReader.SupportsMinDataRateOf9600bps
Device.Input.SmartCardReader.SupportsNegotiableAndSpecificModes
Device.Input.SmartCardReader.SupportsResetCommand
Device.Input.SmartCardReader.UsbCcidCompliesWithUsbDeviceClassSpec
Device.Input.SmartCardReader.UsbCcidIssuesNak

Device.Input.SmartCardReader.PinDataEntryKeyboardCompliesWithIso

Target Feature: Device.Input.SmartCardReader

Title: Input device that implements a PIN data-entry keyboard complies with ISO

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client ARM

Description:

An input device that uses a keyboard for PIN entry must comply with ISO 13491-1:1998 Banking--Secure Cryptographic Devices (retail) Part 1: Concepts, Requirements, and Evaluation Methods.

Exceptions: Not Specified

Business Justification:

Required for smart card reader to function properly with windows

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2006

Comments:

INPUT-0029

Device.Input.SmartCardReader.SmartCardService

Target Feature: Device.Input.SmartCardReader

Title: Smart Card Service must start after Smart Card inserted into reader

Applicable OS Versions:

- Windows 8 Server x64
- Windows 8 Client ARM
- Windows 8 Client x64
- Windows 8 Client x86
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86
- Windows Server 2008 x64

Description:

The Smart Card Service must be started after a Smart Card is inserted into the Smart Card reader.

Exceptions: Not Specified

Business Justification:

This requirement is necessary for reliability of the smart card function.

Scenarios:

Usage of smart card

Success Metric: pass/fail

Enforcement Date: Windows 8 RC

Comments: Not Specified

Device.Input.SmartCardReader.Supports258And259BytePackets

Target Feature: Device.Input.SmartCardReader

Title: Reader supports 258-byte packets in T=0 and 259-byte packets in T=1

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client ARM

Description:

A smart card reader must support the exchange of the following in a single transmission:

.258-byte packets in T=0; that is, 256 data bytes plus the two status words SW1 and SW2.

259-byte packets in T=1; that is, 254 information bytes plus node address, packet control bytes, length, and two error detection code bytes.

Exceptions: Not Specified

Business Justification:

Required for smart card reader to function properly with windows

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2006

Comments:

INPUT-0021

Device.Input.SmartCardReader.SupportsDirectAndInverseConvention

Target Feature: Device.Input.SmartCardReader

Title: Reader supports direct and inverse-convention smart cards

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client ARM

Description:

A smart card reader must support both direct and inverse-convention smart cards either in hardware or in the operating system driver.

Exceptions: Not Specified

Business Justification:

required for smart card reader to function properly with windows

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2006

Comments:

INPUT-0020

Device.Input.SmartCardReader.SupportsInsertionAndRemovalMonitor

Target Feature: Device.Input.SmartCardReader

Title: Reader supports smart card insertion and removal monitor

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

A smart card reader must be able to detect and report smart card insertions and removals with no user intervention other than removing or inserting the smart card itself. The reader must use an interrupt mechanism to report the smart card insertion or removal to the system. A driver polling method to detect smart card insertion and removals is not an acceptable way to meet this requirement.

Exceptions: Not Specified

Business Justification:

Required for smart card reader to function properly with windows

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2006

Comments:

INPUT-0022

Device.Input.SmartCardReader.SupportsMinClockFrequency

Target Feature: Device.Input.SmartCardReader

Title: Reader supports 3.5795-MHz minimum clock frequency

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

A smart card reader must support a minimum clock frequency of 3.5795 MHz.

Exceptions: Not Specified

Business Justification:

Required for smart card reader to function properly with windows

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2006

Comments:

INPUT-0024

Device.Input.SmartCardReader.SupportsMinDataRateOf9600bps

Target Feature: Device.Input.SmartCardReader

Title: Reader supports minimum data rate of 9600 bps

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64

Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client ARM

Description:

A smart card reader must be able to transfer data at a rate of 9600 bps or higher.

Exceptions: Not Specified

Business Justification:

Required for smart card reader to function properly with windows

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2006

Comments:

INPUT-0025

Device.Input.SmartCardReader.SupportsNegotiableAndSpecificModes

Target Feature: Device.Input.SmartCardReader

Title: Reader supports negotiable and specific modes according to the ISO/IEC specification

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client ARM

Description:

To support multiple-protocol smart cards and smart cards that use higher data rates and higher clock frequencies, the reader must support negotiable and specific modes according to ISO/IEC 7816-3 (1997-12-15), Sections 5.4 and7.

The **Power Down** command for ISO 7816-3 is optional, but the **Reset** command is required.

PTS is not required.

Exceptions: Not Specified

Business Justification:

Required for smart card reader to function properly with windows

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2006

Comments:

INPUT-0023

Device.Input.SmartCardReader.SupportsResetCommand

Target Feature: Device.Input.SmartCardReader

Title: Reader supports Reset command

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

A smart card reader must support the asynchronous protocols T=0 and T=1 as described in either the hardware or the driver. Both protocols must be fully supported. The smart card reader and the driver must support cards that can handle both protocols. Support is not required for protocols other than T=0 and T=1.

The following protocol rules apply for the T=1 protocol:

- A transmission is defined as sending a command to a smart card by using one or more T=1 blocks and receiving the corresponding answer by using one or more T=1 blocks as defined in ISO/IEC 7816-3.
- For cards that support IFSC requests, the first transmission after a reset of the smart card must begin with an IFSD request, as defined in ISO/IEC 7816-3, Amendment 1, Section 9.5.1.2.
- For cards that do not support an IFSD request (that is, the card replies with an R-Block indicating "Other error"), the transmission must continue with an I-Block.
- After a successful RESYNCH request, the transmission must restart from the beginning with the first block with which the transmission originally started.

Exceptions: Not Specified

Business Justification:

Required for smart card reader to function properly with windows

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2006

Comments:

INPUT-0026

Device.Input.SmartCardReader.UsbCcidCompliesWithUsbDeviceClassSpec

Target Feature: Device.Input.SmartCardReader

Title: USB smart card CCID reader complies with USB Device Class Specification for USB Chip/Smart Card Interface Devices

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

If the reader supports USB connectivity, CCID is required. To ensure that USB smart card readers function properly with the USB host, smart card CCID readers must comply with USB Device Class: Smart Card Specification for Integrated Circuit(s) Cards Interface Devices, Revision1.00 or later.

Exceptions: Not Specified

Business Justification:

We are creating a driver for USB smartcards and hence need to alert the consumers to follow the latest version of the USB CCID spec to be compliant with the MSFT driver.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2006

Comments:

INPUT-0027

Device.Input.SmartCardReader.UsbCcidIssuesNak

Target Feature: Device.Input.SmartCardReader

Title: USB CCID reader issues NAK on interrupt pipe if device has no interrupt data to transmit

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

USB CCIDs must issue NAK on interrupt pipe, unless the state changes. This prevents the necessity of repeatedly polling the device for status.

Design Notes:

See USB Device Class Specification for USB Chip/Smart Card Interface Devices, Revision1.00 or later, Chapter3. See USB Specification, Revision1.1 or later, Sections 5.7.4 and 8.5.4.

Exceptions: Not Specified

Business Justification:

We don't want to see smartcard readers that need to be polled as it adversely affects laptop battery life if they were to be implemented this way.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2006

Comments:

INPUT-0028

Device.Media.DMR.AV

Description:

Devices that can serve A/V content need to implement these requirements

Related Requirements:

- Device.Media.DMR.AV.AVC

Device.Media.DMR.AV.WMV

Device.Media.DMR.AV.AVC

Target Feature: Device.Media.DMR.AV

Title: AVC requirements for a DMR

Applicable OS Versions:

Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client x86
Windows 8 Client x64

Description:

Req. AVC-01
Applies to AV DMR
A DMR must decode and render content identified with the following Profile IDs: Profiles for AVC content (SD) are aligned to the current and any future DLNA requirements. Profiles for AVC content (HD) are aligned to the current and any future DLNA requirements. Note: At the time of this publication, the profiles have not been published by DLNA. A DMR must decode and render content identified with the following Profile IDs: Profile for AVC content (SD) is TBD Profile for AVC content (HD) is TBD Note: These profiles will be published before the end of 2011 once related industry standardization efforts reach stability. For early implementers, we recommend the following guidelines: AVC SD content: MP@L3, fragmented and non-fragmented, AAC-LC audio, MP4 files AVC HD content: MP@L4, fragmented and non-fragmented, AAC-LC audio, MP4 files
Req. AVC-05
Applies to A/V DMR
A DMR must declare these Profile IDs in the list of supported profiles exposed in the *SinkProtocolInfo* state variable available as part of the Connection Manager Service (CMS).
Req. AVC-10

Applies to A/V DMR
A DMR device must be able to decode the video rotation angle from MP4 files and perform video rotation during composition. The DMR must be able to perform rotations for angles of 0, 90, 180, and 270 degrees.

Design Notes

As described in the DLNA Guidelines, a DMR declares in the *SinkProtocolInfo* state variable the list of all the media types it can play. A DMC connected to the network reads the value of this state variable to determine if content can be played or not in a DMR.

The video rotation angle is defined in the MP4 File Format specification as a transformation matrix that decoders apply during composition, see Section 6.2.2 of ISO/IEC 14496-12:2005(E). In the matrix, coefficients a, b, c, and d define a rotation transformation for an angle of R degrees (measured counterclockwise) if

a = cos R, b = sin R, c = -sin R, and d = cos R

There are two locations for this matrix; one is in the movie header (mvhd) and the other is in the track header (tkhd). For any track, the actual video rotation is given by the sum of the angle defined in the movie header and the angle defined in the corresponding track header. For example, the following 3 cases show a rotation of 90 degrees:

Case 1: movie header rotation: 0 degrees, track header rotation: 90 degrees

Case 2: movie header rotation: 90 degrees, track header rotation: 0 degrees

Case 3: movie header rotation: 20 degrees, track header rotation 70 degrees

Cameras normally use Case 1 and possibly Case 2. Case 3 is allowed in the MP4 specifications but it is not normally used. The tests will cover only cases 1 and 2.

Exceptions:

Required

Business Justification:

This entry requires DMR AV devices to support playback of AVC-encoded content. In addition to the Windows 8 built-in camera capture support optimized for AVC output, many personal digital cameras and cell phones are already capable of recording AVC content. Also, many video distribution web sites are switching from traditional formats into newer formats that use AVC. They are driven not only by availability of content generation tools but also because standards like HTML5 will facilitate media consumption. In addition, Windows 8 Metro style applications are strongly encouraged to use these formats because of the availability of encoder and decoder hardware in mobile form-factors such as tablets.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

NETMEDIA-0110; Updated

Device.Media.DMR.AV.WMV

Target Feature: Device.Media.DMR.AV

Title: WMV requirements for a DMR

Applicable OS Versions:

- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows 8 Client x64

Description:

Req. WMV-01
Applies to AV DMR
A DMR must decode and render content identified with the following Profile IDs: WMVMED_BASE WMVMED_FULL WMVHIGH_FULL
Req. WMV-05
Applies to AV DMR
A DMR must declare these Profile IDs in the list of supported profiles exposed in the *SinkProtocolInfo* state variable available as part of the Connection Manager Service (CMS).

Design Notes

The WMV Profile IDs referenced in this requirement are defined in the DLNA Media Formats Guidelines.

As described in the DLNA Guidelines, a DMR declares in the *SinkProtocolInfo* state variable the list of all the media format profiles that it can play. A DMC connected to the network reads the value of this state variable to determine if content can be played or not by a DMR.

Exceptions:

Required

Business Justification:

This entry requires DMR A/V devices to support playback of WMV-encoded content. Although Windows computers that act as content sources can transcode WMV content into other types, users prefer to watch the content with the highest quality possible. Real-time transcoding reduces video quality and battery life, which is especially important on mobile form-factors such as tablets. Furthermore, WMV is still a popular format in users libraries, on the Web and as an output option for the Windows 8 built-in camera capture feature; which are reasons for having this requirement.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

NETMEDIA-0114; Update

Device.Media.DMR.Base

Description:

Baseline requirements for all DMR devices

Related Requirements:

- Device.Media.DMR.Base.ChangingFriendlyName
- Device.Media.DMR.Base.ContentPlaybackWithoutUserInput
- Device.Media.DMR.Base.DeviceInformation
- Device.Media.DMR.Base.DisplayedMetadata
- Device.Media.DMR.Base.DLNA15CertificationCompliance
- Device.Media.DMR.Base.DMPPlayback
- Device.Media.DMR.Base.DMPSelectionOfAdvertisedResources
- Device.Media.DMR.Base.DMRStateAfterFindingAnError
- Device.Media.DMR.Base.EndOfStream
- Device.Media.DMR.Base.Events
- Device.Media.DMR.Base.MetaDataPackage
- Device.Media.DMR.Base.MetadataSize
- Device.Media.DMR.Base.OptionalFieldsEntries
- Device.Media.DMR.Base.PausingAStream
- Device.Media.DMR.Base.PlaybackPerformance
- Device.Media.DMR.Base.PlayBackPerformanceConstrainedBandwidth
- Device.Media.DMR.Base.PlaysMediaContinuously
- Device.Media.DMR.Base.ProtocolInfoField
- Device.Media.DMR.Base.ResponseToSetAVTransportURIActions
- Device.Media.DMR.Base.SeekOperations
- Device.Media.DMR.Base.SendsSSDPByeByeMessage

Device.Media.DMR.Base.SetNextAVTransportURI
Device.Media.DMR.Base.VolumeControl
Device.Media.DMR.Base.WakeOnLAN
Device.Media.DMR.Base.WiFiDirectSupport
Device.Media.DMR.Base.WMDRMNDLinkProtectionSupport

Device.Media.DMR.Base.ChangingFriendlyName

Target Feature: Device.Media.DMR.Base

Title: DMR supports changing the Friendly Name

Applicable OS Versions:

Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client x86
Windows 8 Client x64

Description:

Req. NAME-01
Applies to AV DMR, and Audio DMR
A Digital Media Renderer (DMR) must allow the user to update the DMR friendly name using any configuration method. The DMR friendly name is defined as the string value included in element *friendlyName* in the Device Description Document.

Design Notes

Some DMR devices may choose to implement this requirement using the *presentationURL* element in the Device Description Document. The URL in this element points to a device web page that may define configuration options. One of the configuration options can give users the opportunity to change the friendly name.

Other DMR devices may choose different means to give users the option to change the DMR friendly name.

The use of a presentation URL to expose an HTML presentation page is described in the UPnP Device Architecture (DA) specification at http://www.upnp.org.

Exceptions:

Required

Business Justification:

Windows computers display the DMR friendly name in user interfaces that expose such information. If a user buys two identical DMR devices from the same manufacturer, these devices typically have

identical friendly names. Windows will display the two devices with the same name, which can be confusing for users. The ability to change the friendly name gives users the ability to correct this problem.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

NETMEDIA-0018; Updated

Device.Media.DMR.Base.ContentPlaybackWithoutUserInput

Target Feature: Device.Media.DMR.Base

Title: DMR content playback without user input

Applicable OS Versions:

- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows 8 Client x64

Description:

Req. INPUT-01
Applies to A/V DMR, Audio DMR
A Digital Media Renderer must advertise its presence on the network via *ssdp:alive* messages after the device has been turned on and once the user has completed any required network configuration options.
Req. INPUT-05
Applies to A/V DMR, Audio DMR
If a Digital Media Renderer advertises its presence on the network via *ssdp:alive* messages, it must be capable of accepting actions, including *SetAVTransportURI()* and *Play()* from a Digital Media Controller without requesting any additional user input. The only exception for this requirement is if the user is performing interactive operations with the device such as configuring the device, playing games, etc. Only in these cases, the device may respond with error codes that indicate that the device is busy.
Req. INPUT-10

Applies to A/V DMR, Audio DMR
A Digital Media Renderer that implements additional functionality for *passive media consumption* must also advertise its presence on the network when the device is used for *passive media consumption* activities. Examples of *passive media consumption* activities are: watching TV, listening to AM/FM radio, listening to CDs, listening to Internet music, watching a DVD or BD disk, watching a movie from the Internet, operating the device as a DMP, etc. *Active media consumption* is the opposite of *passive media consumption*. Playing games is an example of an *active media consumption* activity. Using menus to configure device options is another example of *active media consumption*. This requirement does not apply to *active media consumption* scenarios.

Design Notes

The moment a DMR advertises its presence to the network, any DMC device connected to the network expects the DMR to operate without further delays. Some DMRs could be designed for example to authorize every stream request received from a DMC, but this approach disrupts the interactions of DMCs and DMRs. This requirement forces DMRs to perform any configuration operations before they start sending *ssdp:alive* messages.

Requirement INPUT-10 applies to multi-function devices. For example, it applies to devices that implement a DMR, a TV, and an AM/FM radio receiver. This requirement indicates that the DMR needs to be advertised even if the device operates as a TV, as an AM/FM radio receiver, etc. In addition, requirement INPUT-05 indicates that once the DMR is advertised on the network, it needs to be fully operational. In other words, if the user is watching TV or listening to AM/FM radio, the DMR is fully operational in the background. If the user picks a controller and pushes a picture to the device, the device needs to transition from TV mode or AM/FM radio mode into DMR mode automatically.

Although the examples described in the Design Notes mention TV experiences and AM/FM radio experiences, they apply to any other types of *passive media consumption* like playing a DVD/BD or listening to Internet radios.

Exceptions:

Required

Business Justification:

The Internet has given users an expectation that services can be always on and always available. Unless there are some catastrophic failures, Internet services now operate in always-on and always-connected mode. Users expect the same type of behavior for services in a home network. If a networked TV is on, the user expects the TV/DMR to be always connected. Similarly, if a networked DMS is on, the user expects it to be always connected. Because DMS and DMR are headless network components (no UI except for configuration), they need to be always connected and waiting for

service requests. Hence, there is no strong reason why a DMR should be blocked when a device operates as a TV or as an AM/FM radio. Users like the idea of always connected devices.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

NETMEDIA-0042; Update

Device.Media.DMR.Base.DeviceInformation

Target Feature: Device.Media.DMR.Base

Title: Availability of DMR Information

Applicable OS Versions:

Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client x86
Windows 8 Client x64

Description:

Req. INFO-01

The following device information must be available at the time of testing:

If the device uses firmware (or middleware): firmware vendor and firmware version
If the device uses an application model: App name, App vendor, App version, and App platform
If the device uses an Operating System: OS name and OS version,
Hardware Platform (e.g ARM, x86, x64, etc),
Manufacturer Name,
Model Number

Design Notes

The information is collected to identify certified devices, track bugs, identify areas of improvement, etc.

Exceptions:

Required

Business Justification:

To identify certified devices, track bugs more accurately and identify areas of improvement.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

INFO-01

Device.Media.DMR.Base.DisplayedMetadata

Target Feature: Device.Media.DMR.Base

Title: Displayed Metadata in DMR Operations

Applicable OS Versions:

Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client x64
Windows 8 Client x86

Description:

Req. DISP-01
Applies to A/V DMR, Audio DMR
If a DMR displays the title of an item during playback, by default the title must be extracted from the dc:title value associated with the item or from an internal metadata field within the file. If the DMR displays the date of a picture during playback, by default the date must be extracted from the dc:date value associated with the picture or from an internal metadata field within the file. If the DMR displays the size of a file during playback, the value must describe the correct file size. A DMR can obtain the file size from the res@size attribute, from an internal metadata field in the file, or from its own size computation. After determining the file size, the DMR may round the value to any desired precision (e.g. zero decimal digits, one decimal digit, etc.) in order to show the value in a User Interface. If an average user measures the duration of an A/V or audio item using a standard clock as T_{user}, and if the DMR displays the content duration as T_{disp}, then for all cases, the displayed duration must comply with: $0.9\ T_{user} < T_{disp} < 1.1\ T_{user}$ Note: These requirements describe the default behavior expected from DMR devices. At any time, a user can bypass the default behavior and register user-defined titles, or dates, or event duration values.

Design Notes

Many of the popular file formats like MP4 and ASF include metadata fields that describe title information. Similarly, the file formats for JPEG and PNF also include metadata fields that describe the image creation date. These metadata fields together with metadata properties received from a controller (dc:title, dc:date) represent the best source for obtaining correct values.

Exceptions:

Required

Business Justification:

Users want to see correct content information displayed on their DMRs. Users also expect consistency between devices that expose information like title, date, and duration.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

NETMEDIA-0062; Updated

Device.Media.DMR.Base.DLNA15CertificationCompliance

Target Feature: Device.Media.DMR.Base

Title: DMR must compliance with DLNA 1.5 Certification

Applicable OS Versions:

- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows 8 Client x64

Description:

Req. DLNA-01
Applies to A/V DMR, Audio DMR
A Digital Media Renderer must demonstrate compliance with the DLNA protocols using one of the following alternatives: The DMR passes the DLNA Certification Program and obtains a DLNA certificate. The DMR uses an OEM reference design (hardware/software) that has already passed the DLNA Certification Program and has obtained a DLNA certificate. The DMR uses middleware from an independent software provider. The middleware has passed

the DLNA Certification Program and has obtained a DLNA certificate. If the device implements multiple network interfaces (Ethernet, Wi-Fi, etc.), the DLNA Certificate must indicate a satisfactory evaluation using all interfaces. If the device implements additionally a DMP, the device must demonstrate compliance with the DLNA protocols for the DMP class using one of the alternatives mentioned above. If the device implements additionally a DMS, the device must demonstrate compliance with the DLNA protocols for the DMS class using one of the alternatives mentioned above.
Req. DLNA-05
Applies to A/V DMR
An A/V DMR is a renderer that plays images, audio, and videos. The DLNA certification for an A/V DMR must include the Image Class, the Audio Class, and the A/V Class.
Req. DLNA-10
Applies to Audio DMR
An Audio DMR is a renderer that plays audio but not video. The DLNA certification for an Audio DMR must include the Audio Class. If an Audio DMR plays images, the DLNA certification for this Audio DMR must include the Image Class.

Design Notes

The Windows Device Certification Program recognizes two types of renderers:

A/V DMR: A Digital Media Renderer that plays images, audio, and A/V content. For example, a TV, a BD players, a Set-Top Box, etc.

Audio DMR: A Digital Media Renderer that plays audio content but not videos. An Audio DMR could play images.

Exceptions:

Required

Business Justification:

DLNA defines the baseline specifications for interconnected media devices. The DLNA organization also maintains a strict certification program that ensures good implementations of the protocols. However, DLNA does not define user experiences, performance, and product quality, which are the areas covered by the Windows Device Certification Program.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

NETMEDIA-0003; Update

Device.Media.DMR.Base.DMPPlayback

Target Feature: Device.Media.DMR.Base

Title: DMP Playback Functionality

Applicable OS Versions:

- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows 8 Client x64

Description:

Req. DMP-01
Applies to A/V DMR, Audio DMR
If a DMR device also implements a Digital Media Player (DMP), the DMP must play all content types that can be played as a DMR. Similarly, the DMR must play all content types that can be played as a DMP. The different types of content are defined using a Profile ID, a MIME type, or both.
Req. DMP-05
Applies to A/V DMR, Audio DMR
If a DMR device also implements a Digital Media Player (DMP), the DMP and the DMR must work jointly as a single combined unit in the network: If the DMP plays content, the DMR state variables must change accordingly to reflect the playback conditions. If the DMP stops or pauses playback, the DMR state variables must change accordingly to advertise the changes. The DMR state variables advertise conditions such as current status, current state, current position, current transport actions, etc.
Req. DMP-10
Applies to AV DMR, Audio DMR
If a DMR device also implements a Digital Media Player (DMP), the DMP must be capable to navigate

a Reference Windows DMS in compliance with the following performance procedure and requirement:

Using the DMP controls find the UI from where users start navigating the content in a DMS

Using the DMP controls navigate the UI menus to find the first music item

Using the DMP controls play the first music item

The time to complete tasks 2 and 3 must not exceed 10 seconds (under ideal network conditions).

Design Notes

A Reference Windows DMS is defined as a Windows 8 PC, with WinSAT score of 4 or higher, that shares content with DLNA devices. The content available in a reference Windows DMS is 300 pictures in the Pictures library, 300 music files in the Music library, and 30 video files in the Videos library.

Exceptions:

If a DMR device also works as a DMP then this requirement applies. A DMR that does not work as a DMP needs not implement this requirement.

Business Justification:

Users normally do not understand the technical differences between DMPs and DMRs. Consequently, any differences between the two implementations are perceived as device failures. Users have also complained about devices that cannot browse efficiently medium and large-size libraries.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

NETMEDIA-0112; Update

Device.Media.DMR.Base.DMPSelectionOfAdvertisedResources

Target Feature: Device.Media.DMR.Base

Title: DMP selection of advertised resources

Applicable OS Versions:

Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client x86
Windows 8 Client x64

Description:

Req. DMP-20
Applies to A/V DMR, Audio DMR
This requirement only applies if a DMR device also implements a DMP. Furthermore, this requirement only applies if all the following conditions are satisfied: A DMP finds an item with multiple <res> elements. A DMP selects an A/V or an Audio resource in streaming mode, or a DMP selects an Image resource in interactive mode. A DMP connects to a DMS under ideal network conditions The DMP must select a <res> element according to the following procedures: **Audio Class** If the server exposes a non-transcoded resource with a Profile ID that matches one of the Profile IDs advertised by the DMP/DMR device, then the DMP device must select this resource. If the DMP does not support the Profile ID for the non-transcoded resource, the DMP must select a transcoded resource with a supported Profile ID and play this resource. **A/V Class:** If the server exposes a non-transcoded resource with a Profile ID that matches one of the Profile IDs advertised by the DMP/DMR device, then the DMP device must select this resource. If the DMP does not support the Profile ID for the non-transcoded resource, the DMP must select a transcoded resource with a supported Profile ID and play this resource. **Image Class** The DMP device must select a <res> element for an image with a resolution of 1920x1080 or smaller. There is one exception: a DMP device may select an image resource with a resolution higher than 1920x1080 if the DMP can display the image within 2 seconds. The 2 seconds are measured from the moment the user selects an image to the moment the image is displayed on the screen.

Design Notes

A Windows DMS often exposes content items using multiple <res> elements. Each element identifies the same item but using different encoding formats and different bitrates. When a DMP starts the process to play the content, the DMP needs to select one <res> element for playback.

The native (non-transcoded) content can be identified from the DLNA.ORG_CI flag. A value of 1 indicates transcoded (non-native) content. A value of 0 indicates the opposite. The absence of this flag is equivalent to a value of 0.

This requirement does not apply in the following conditions:

The exchange happens under poor network conditions
The request uses background transfer
The <item> element has only one <res> element.

For Image Class behavior, it is acceptable to provide a configuration option (a menu or a UI dialog window) to bypass the recommended behavior (default behavior), so that the DMP always selects the highest quality image at the expense of transfer time.

Exceptions:

If a DMR also works as a DMP then this requirement applies. A DMR that does not work as a DMP needs not implement this requirement.

Business Justification:

In the case of audio and video, users prefer to play the resource that has the best possible quality. The native (non-transcoded) resource is typically the one with the highest quality. In the case of images, users prefer a balance between high quality and low latency. A majority of the display devices have a maximum resolution of 1920x1080. This resolution is small compared to the large images captured by modern-day portable cameras. For most devices, the transfer of images with higher resolutions does not improve the displayed quality and it simply increases the latency due to network transfer.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

NETMEDIA-0120; Update

Device.Media.DMR.Base.DMRStateAfterFindingAnError

Target Feature: Device.Media.DMR.Base

Title: DMR state after finding an error

Applicable OS Versions:

Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client x86
Windows 8 Client x64

Description:

Req. ERROR-01
Applies to A/V DMR, Audio DMR
If a Digital Media Renderer (DMR) is in the PLAYING, TRANSITIONING, or PAUSED_PLAYBACK state and encounters a non-recoverable error, the DMR must enter a STOPPED state and wait for instructions from the Digital Media Controller (DMC). At the same time, the DMR must report the error setting the value of the *TransportStatus* state variable to ERROR_OCCURRED. As a consequence of this requirement, a DMR with a *TransportState* value equal to STOPPED and its *TransportStatus* value equal to ERROR_OCCURRED implies that content playback has been terminated due to an error.
Applies to A/V DMR, Audio DMR
If the DMC provides another resource using *SetAVTransportURI(),* the DMR must suppress any error message (i.e., change the value of *TransportStatus* to OK) and begin the process to play the new resource.

Design Notes

This requirement is simply a clarification of behavior described in the UPnP AVTransport:1 specifications. In particular, Fig. 1 of the AVTransport:1 specifications describes the behavior after error conditions together with the specifications for the *TransportStatus* and *TransportState* state variables .

Exceptions:

Required

Business Justification:

Any device that enters an error state needs to recover gracefully. Users do not tolerate devices that become non-responsive after error conditions.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2011

Comments:

NETMEDIA-0041; Update

Device.Media.DMR.Base.EndOfStream

Target Feature: Device.Media.DMR.Base

Title: DMR requirement for end of stream

Applicable OS Versions:

Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client x64
Windows 8 Client x86

Description:

Req. EOS-01
Applies to A/V DMR, Audio DMR
A DMR that finishes playing a media resource must enter the STOPPED state. The resource URI remains associated with the DMR. Consequently, if the DMR receives a new *Play()* action, the DMR must start playing the same resource. If the resource is either audio or A/V, playback starts from the beginning (play time of 0).

Design Notes

If the DMR device finishes playing the stream, it needs to enter a stopped state and wait for instructions from any DMC. This requirements exists in the UPnP AVTransport:1 specifications .

Exceptions:

Required

Business Justification:

Some devices fail to implement this behavior. DLNA currently does not test this condition.The requirement insures a good user experience.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

NETMEDIA-0062; Updated

Device.Media.DMR.Base.Events

Target Feature: Device.Media.DMR.Base

Title: DMR Events

Applicable OS Versions:

Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client x64

Windows 8 Client x86

Description:

Req. EVENT-01
Applies to A/V DMR, Audio DMR
The DMR must send AVT LastChange events and RCS LastChange events as described in the related UPnP and DLNA specifications.

Design Notes

The UPnP specifications for the AV Transport Service (AVT) define the use of the AVT LastChange state variable. Similarly, the UPnP specifications for the Rendering Control Service (RCS) define the use of the RCS LastChange state variable. DLNA clarifies several issues such as timing of the different events. UPnP and DLNA require DMRs to implement eventing of these state variables.

The DLNA certification program verifies the implementation of events but they do not cover all cases. The Windows Certification tests for this requirement cover many more cases including events for state transitions, URI value changes, and others.

Exceptions:

Required

Business Justification:

The controller implemented in Windows 8 relies on AVT and RCS LastChange events to make decisions about devices.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

NETMEDIA-0062; Updated

Device.Media.DMR.Base.MetaDataPackage

Target Feature: Device.Media.DMR.Base

Title: The device has a metadata package that meets Windows 8 metadata specifications for the specific device category; and the metadata package is posted for public usage.

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64

Description:

Req. META-01
Applies to A/V DMR, Audio DMR
The device must have a metadata package that meets Windows 8 metadata specifications for the specific device category. The metadata package must be posted for public distribution through the Windows Metadata Information Services (WMIS) once the hardware certification submission is approved and/or available for distribution with the device.

Exceptions:

Required

Business Justification:

This requirement creates a richer end user experience by using this metadata to integrate the experience with the OS.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

New

Device.Media.DMR.Base.MetadataSize

Target Feature: Device.Media.DMR.Base

Title: DMR metadata size requirements

Applicable OS Versions:

- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows 8 Client x64

Description:

Req. META-20
Applies to A/V DMR, Audio DMR
The DMR must be capable of receiving a DIDL-Lite fragment in *CurrentURIMetaData* with at least 30 <res> elements and metadata inclusive. The DMR must tolerate the presence of at least 30 <res>

elements in a message. However, the DMR decides if it parses (and uses) the information in the <res> elements.
Req. META-25
Applies to AV DMR, Audio DMR
If the DMR receives more than 30 <res> elements in the *CurrentURIMetaData* argument of *SetAVTransportURI()*, the DMR must do one of the following: Return UPnP error 501 (Action Failed) Accept the metadata

Design Notes

There is no conflict between this requirement and the current DLNA Guidelines. The DLNA Guidelines require a DMR to accept SOAP requests of up to 20,480 bytes in size. Consequently, a *SetAVTransportURI()* request can be as large as 20 KB in size, which is sufficient to accommodate more than 30 <res> elements.

Exceptions:

Required

Business Justification:

A Windows DMS can transcode content and offer the content using multiple media format profiles (multiple elements). With increased computational power, computers will offer in the future more transcoding alternatives. However, it is also true that many DMRs do not have enough computational power to process large XML files and select specific media resources. For this reason, it is important that devices respond with error 501 if the message cannot be processed. This error message is used by a Windows PC to reduce the number of elements transferred to a DMR.

Scenarios: Not Specified

Success Metric: This detail provides a good user experience handling a common media processing error.

Enforcement Date: Windows RC

Comments:

NETMEDIA-0050; Update

Device.Media.DMR.Base.OptionalFieldsEntries

Target Feature: Device.Media.DMR.Base

Title: DMR must provide non-empty and valid entries for optional fields

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64

Description:

Req. DDD-01
Applies to A/V DMR, Audio DMR
In addition to the normative fields in a Device Description Document (DDD), a DMR must provide non-empty and valid entries for the following optional fields: The manufacturers name in the <manufacturer> element The model name in the <modelName> element The model number in the <modelNumber> element A user-friendly name in the <friendlyName> element

Design Notes

The UPnP Device Architecture (DA) specification defines the structure of the Device Description Document.

Exceptions:

Required

Business Justification:

Having these entries allows for a better diagnosis of device issues. The friendly name appears in different locations in Windows UI components.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: Windows RC

Comments:

New

Device.Media.DMR.Base.PausingAStream

Target Feature: Device.Media.DMR.Base

Title: DMR supports pausing a stream

Applicable OS Versions:

Windows 7 Client x86
Windows 7 Client x64

Windows 8 Client x86
Windows 8 Client x64

Description:

Req. PAUSE-01
Applies to A/V DMR, Audio DMR
The DMR must support pausing Audio or A/V streams. A DMR that receives a *Pause()* action must pause the rendering process. After a *Pause()* action, a DMR that receives a *Play()* action must resume playback from the same position. A DMR must be capable of pause/resume operations for any resources of the following types: Audio or A/V resources for which the DMS supports time-range requests, or byte-range requests, or connection stalling.
Req. PAUSE-05
Applies to A/V DMR, Audio DMR
The Pause operation must be available while the DMR is in the PLAYING state (for audio and A/V resources). As defined in the DLNA Guidelines, the DMR must advertise availability of this operation by inserting the keyword Pause in the list of *CurrentTransportActions*.

Design Notes

A DMR can implement pause/resume operations using the following methods:

Local caching The DMR caches the content and performs pause/resume operations using cached data.

Byte-range requests: Upon receiving a Pause request, the DMR stores the byte position. Upon receiving a request to resume playback, the DMR issues a byte-range request against the server starting from the stored byte position.

Time-range requests: Upon receiving a Pause request, the DMR stores the current play time position. Upon receiving a request to resume playback, the DMR issues a time-range request against the server starting from the stored time position.

Connection stalling: The DMR stalls the HTTP connection until the user resumes playback. Servers usually do not keep stalled connections open for a long time. If a server closes the connection, the DMR needs to use either time-range or byte-range requests.

Exceptions:

Required

Business Justification:

Pause/Resume operations are optional in the DLNA framework. However, users expect any modern device to support Pause/Resume.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

NETMEDIA-0071; Update

Device.Media.DMR.Base.PlaybackPerformance

Target Feature: Device.Media.DMR.Base

Title: DMR Playback Performance

Applicable OS Versions:

- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows 8 Client x64

Description:

Req. PERF-01
Applies to A/V DMR, Audio DMR
A DMR device must comply with the performance specifications indicated in Table PERF below. The test content for this requirement has the following characteristics: A/V: AVC and AAC using an MP4 file. The video resolution is 1280 x 720 at 30 fps. Maximum system bitrate of 5 Mbps Audio: LPCM content, Maximum bitrate of 1.5 Mbps Image: 1920x1080, Maximum size of 500 Kbytes All tests require ideal network conditions.
Req. PERF-05
Applies to A/V DMR, Audio DMR
A DMR that needs more than 3 seconds to play Audio, A/V, or Images for any of the cases described in Table PERF must display a visual indicator to indicate that data is being processed and that it will start playing content soon.

Table PERF: Expected performance values for DMR devices

DMR state	Task starts	Task ends	Max Latency
NO_MEDIA_PRESENT	User hits a play button (video)	DMR starts displaying video	5 sec
PLAYING	User hits a play button for new content (video)	DMR starts displaying the new video	5 sec
STOPPED	User hits a play button for new content (video)	DMR starts playing the new video	5 sec
PLAYING	User seeks to a different position in the stream (video)	DMR starts displaying video from the new position	5 sec
PLAYING	User hits the pause button (audio or video)	DMR pauses playback	2 sec
PAUSED_PLAYBACK	User hits the play button to resume (audio or video)	DMR resumes playing the content	2 sec
NO_MEDIA_PRESENT	User hits a play button (audio)	DMR starts playing audio.	3 sec
PLAYING	User hits a play button for new content (audio)	DMR starts playing the new audio.	3 sec
STOPPED	User hits a play button for new content (audio)	DMR starts playing the new audio.	3 sec
PLAYING	User seeks to a different position in the stream (audio)	DMR starts playing audio from the new position.	3 sec
NO_MEDIA_PRESENT	User hits a play button (image)	DMR displays the image.	3 sec
STOPPED	User hits a play button with new content (image)	DMR displays the new image	3 sec
NO_MEDIA_PRESENT, STOPPED, PLAYING, PAUSED_PLAYBACK	User changes the volume level	DMR plays (or can play) content at the new level	2 sec
NO_MEDIA_PRESENT, STOPPED, PLAYING, PAUSED_PLAYBACK	User applies mute (or removes mute).	DMR is muted (or is not muted)	2 sec

Design Notes

The DLNA Guidelines define the term ideal network conditions as a connection with virtually unlimited bandwidth and negligible congestion.

Exceptions:

Required

Business Justification:

Users are looking now for high quality devices and expect their devices to be always responsive and show minimal delays.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

NETMEDIA-0132; Update

Device.Media.DMR.Base.PlayBackPerformanceConstrainedBandwidth

Target Feature: Device.Media.DMR.Base

Title: DMR Playback Performance under constrained bandwidth

Applicable OS Versions:

- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x64
- Windows 8 Client x86

Description:

Req. PLAYBACK-01
Applies to AV DMR
This requirement applies to a network with approximately constant throughput of 16 Mbps (i.e. a network that simulates an interference-free 802.11g network). This requirement uses A/V content with an instantaneous system bitrate that never exceeds 11 Mbps. A DMR that receives such content normally buffers a few seconds of data before rendering the content. Once the DMR starts rendering the content, the DMR must be able to play the entire content without re-buffering.

Design Notes

A good DMR implementation does not need to re-buffer if the network has enough bandwidth to transfer the content in real time or at rates higher than real-time. This requirement simply tests this assumption.

Exceptions:

Required

Business Justification:

Users dislike devices that continuously have to re-buffer content acquired from the network. A good device is one that has been designed to receive and process content in real time.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

NETMEDIA-0062; Updated

Device.Media.DMR.Base.PlaysMediaContinuously

Target Feature: Device.Media.DMR.Base

Title: DMR plays media continuously

Applicable OS Versions:

- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows 8 Client x64

Description:

Req. STRESS-01
Applies to AV DMR, Audio DMR
A DMR must be able to play a mixture of content types continuously without errors, without user intervention, and without degradation in quality, for at least 6 hours. In practice the test will verify that the device neither crashes nor responds with long delays.

Design Notes

A DMR device needs to work autonomously for long periods of time. The test associated with this requirement plays a mixture of content for a period of 6 hours. However, in practice, DMR devices should be designed to perform consistently for longer periods of time.

Exceptions:

Required

Business Justification:

Users expect their devices to work with the highest quality for long periods of time.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 6/1/2011

Comments:

NETMEDIA-0048; Updated

Device.Media.DMR.Base.ProtocolInfoField

Target Feature: Device.Media.DMR.Base

Title: DMR Use of the protocolInfo field

Applicable OS Versions:

Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client x86
Windows 8 Client x64

Description:

Req. PROT-01
Applies to A/V DMR, Audio DMR
A DMR that exposes in *SinkProtocolInfo* a *protocolInfo* value with a MIME Type and a DLNA Profile ID must be able to play content encoded according to the DLNA specifications for the Profile ID.
Req. PROT-05
Applies to A/V DMR, Audio DMR
A DMR that exposes in *SinkProtocolInfo* a *protocolInfo* value with a MIME Type but without a DLNA Profile ID must be able to play content as specified in Table PROT.

Table PROT: List of supported protocols associated with MIME types (for DMR devices that advertise a protocolInfo value with a MIME type but without a DLNA Profile ID)

MIME type	DMR plays at least this type of content
audio/mpeg	Profile ID: MP3
video/x-ms-wmv	Profile IDs: WMVHIGH_FULL, WMVHIGH_PRO, WMVMED_BASE, WMVMED_FULL, WMVMED_PRO, WMVSPML_BASE, WMVSPLL_BASE
audio/x-ms-wma	Profile IDs: WMABASE, WMAFULL, WMAPRO, WMALSL
audio/mp4 or audio/3gpp	Profile IDs: AAC_ISO_192, AAC_ISO_320, AAC_ISO
audio/wav	Content encoded for a Profile ID of LPCM but using a WAV file format to carry the binary stream.
audio/vnd.dlna.adts	Profile IDs: AAC_ADTS_192, AAC_ADTS_320, AAC_ADTS
video/mp4	In addition to content described in Req. Device.Media.DMR.AV.AVC, the device must decode the following Profile IDs: AVC_MP4_MP_SD_AAC_MULT5, AVC_MP4_BL_L3_SD_AAC, AVC_MP4_BL_L2_CIF30_AAC, AVC_MP4_MP_HD_720p_AAC,

	AVC_MP4_MP_HD_1080i_AAC, AVC_MP4_HP_HD_AAC
video/quicktime	A/V content encoded using Apples Quicktime specifications using the MOV file format.

Design Notes

A Digital Media Renderer exposes a list of *protocolInfo* values using the *SinkProtocolInfo* state variable. These values represent the collection of protocols that the DMR can play. According to DLNA specifications, each *protocolInfo* entry always carries a Profile ID to identify decoding formats.

Many DMR devices use *protocolInfo* values without a Profile ID (DLNA considers this behavior as out of scope of its specifications). This requirement provides minimum playback requirements for these cases to attempt certain level of interoperability.

Exceptions:

Required

Business Justification:

The Windows Play To functionality sends content to a DMR only if the DMR claims to play the content type. Unfortunately, in many cases the DMR cannot play the content type, which creates poor user experiences. This problem happens especially if the DMR advertises content types without a DLNA Profile ID. This requirement attempts to minimize the problem defining at least some expected behavior for these cases.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

NETMEDIA-0025; Updated

Device.Media.DMR.Base.ResponseToSetAVTransportURIActions

Target Feature: Device.Media.DMR.Base

Title: DMR Response to SetAVTransportURI actions

Applicable OS Versions:

- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows 8 Client x64

Description:

Req. SETAVT-01

Applies to A/V DMR, Audio DMR
If a DMR is in the PLAYING state and receives a *SetAVTransportURI()* action with a valid non-empty URL, the DMR must stop playing the content and start playing the new content defined by the URL.

Design Notes

The behavior described in this requirement corresponds to the original desired behavior defined in the UPnP AVTransport:1 specifications.

DLNA allows the same behavior but it also allows a relaxed option. In the relaxed option, a DMR that is in the PLAYING state can respond with an error if it receives a *SetAVTransportURI()* action. These DMRs need to receive a *Stop()* action before accepting a new *SetAVTransportURI()*.

Unfortunately, the relaxed option introduced by DLNA creates an implementation problem. This requirement mandates that DMR devices follow the original UPnP AVTransport:1 specifications.

Exceptions:

Required

Business Justification:

According to DLNA, DMRs that are currently playing content have two options to respond to a new SetAVTransportURI() request. One option is consistent with the original UPnP AVTransport:1 specifications. The second option is a relaxed behavior introduced by DLNA.Unfortunately the relaxed behavior introduces an implementation problem. If a user sends a playlist of images, the Stop() actions required by the relaxed option cause the DMR to insert UI views during the transition from one image to the next one. This requirement avoids this problem.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

NETMEDIA-0118; Update

Device.Media.DMR.Base.SeekOperations

Target Feature: Device.Media.DMR.Base

Title: DMR supports Seek operations

Applicable OS Versions:

- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows 8 Client x64

Description:

Req. SEEK-01
Applies to A/V DMR, Audio DMR
A DMR device must support seek operations using the method known as controller-time seeking defined in the DLNA Guidelines. Using this method, a controller sends *Seek()* actions with playtime values to the DMR.
Req. SEEK-05
Applies to AV DMR, Audio DMR
As defined in the DLNA Guidelines, a DMR device that supports controller-time seeking must include the keywords Seek and X_DLNA_SeekTime in the *CurrentTransportActions* state variable. These keywords must be listed in the state variable while the DMR is in the PLAYING state or in the PAUSED_PLAYBACK state. This requirement does not specify if seek operations are available when the DMR is in the STOPPED state.
Req. SEEK-10
Applies to AV DMR, Audio DMR
A Digital Media Renderer must implement controller-time seek at least for the following types of media resources: A/V or Audio media resources for which a DMS indicates support for time-range requests (a DLNA.ORG_OP value of 10 or 11 indicates a resource for which the DMS supports time-range requests) A DMR device should implement controller-time seek for media resources beyond the type indicated in this requirement.
Req. SEEK-15
Applies to A/V DMR, Audio DMR
If a Digital Media Renderer is in the PLAYING state and receives a *Seek()* action with a valid target playtime, the DMR must resume playback from the target playtime. If a Digital Media Renderer is in the PAUSED_PLAYBACK state and receives a *Seek()* action with a valid target playtime, the DMR must resume playback from the target playtime. Notice that if a DMR is in the PAUSED_PLAYBACK state, a *Seek()* action restarts the playback process and moves the DMR into the PLAYING state.

Design Notes

The controller-time seek protocol defines the interaction between the controller and the DMR. The controller sends a *Seek()* action that has two arguments:

Unit: This argument includes the value REL_TIME to identify seek operations using playtime values.

Target: This argument includes the actual playtime value.

Upon receiving this action, the DMR can use one of the following strategies to satisfy the request:

The DMR has already cached the content. Consequently, the DMR performs the seek operations locally and plays content from cache.

The DMR sends an HTTP request to the DMS using TimeSeekRange.dlna.org header. This is possible only if the DMS supports the header (DLNA.ORG_OP = 10 or 11).

The DMR converts time information into byte information and sends an HTTP request to the DMS using the Range header. This strategy is possible if the DMS supports the header (DLNA.ORG_OP = 01 or 11).

Exceptions:

Required

Business Justification:

Seek operations are considered an optional feature in the context of DLNA implementations. However, any modern media device includes now support for seeking. This is true for portable devices (music players, smart phones) and also for Internet-based players. Users want to have high quality devices with features comparable to their other media devices. Users will qualify a device as low quality if it does not implement seeking, or if the device implements seeking only for a reduced number of content types.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

NETMEDIA-0070; Updated

Device.Media.DMR.Base.SendsSSDPByeByeMessage

Target Feature: Device.Media.DMR.Base

Title: DMR sends ssdp:byebye messages

Applicable OS Versions:

Windows 7 Client x86
Windows 7 Client x64

Windows 8 Client x86
Windows 8 Client x64

Description:

Req. BYEBYE-01
Applies to A/V DMR, Audio DMR
A device that implements DMR functionality must send ssdp:byebye messages when the device turns off the DMR functionality. This requirement does not apply to error conditions like accidental disconnections or unexpected malfunctioning. This requirement applies to all cases when a user gracefully turns off the DMR function.
Req. BYEBYE-05
Applies to A/V DMR, Audio DMR
A DMR device that supports a low power (sleep) mode must send ssdp:byebye messages when the device enters low power mode. For devices that support multiple levels of low power modes, this requirement applies to any modes where the device can no longer accept UPnP actions from controllers in the network.
Req. BYEBYE-10
Applies to A/V DMR, Audio DMR
A DMR device must send ssdp:byebye messages in compliance with UPnP and DLNA specifications. In particular, a DMR device sends one ssdp:bybye message for each corresponding ssdp:alive message.

Design Notes

Devices implement ssdp:byebye messages according to the UPnP Device Architecture 1.0 specification with the additional clarifications defined in the DLNA Guidelines.

Exceptions:

Required

Business Justification:

Users expect their controller devices (smart phones, tablet PCs, etc.) to detect the presence or absence of DMR devices in real time. If a user turns off a DMR, the user expects this information to propagate immediately to controllers in the network showing the list of available DMRs. The use of ssdp:bybye messages is critical to give users real-time information about departing devices.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

NETMEDIA-0128; Update

Device.Media.DMR.Base.SetNextAVTransportURI

Target Feature: Device.Media.DMR.Base

Title: DMR supports SetNextAVTransportURI

Applicable OS Versions:

- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows 8 Client x64

Description:

Req. SETNEXT-01
Applies to A/V DMR, Audio DMR
A DMR device must implement the *SetNextAVTransportURI()* action and its related state variables *NextAVTransportURI* and *NextAVTransportURIMetaData*.
Req. SETNEXT-05
Applies to A/V DMR, Audio DMR
If the DMR is playing an audio resource, then as soon as the resource finishes playing, the DMR must play the resource described in the *NextAVTransportURI* state variable. If the DMR is playing an A/V resource, then as soon as the resource finishes playing, the DMR must play the resource described in the *NextAVTransportURI* state variable. If the DMR is playing an image resource, then as soon as the DMR receives a *Next()* action, the DMR must play the resource described in the *NextAVTransportURI* state variable.
Req. SETNEXT-10
Applies to AV DMR, Audio DMR
If the DMR is playing an audio, A/V, or image resource and if the DMR receives a *Next()* action, the DMR must play the resource defined in the *NextAVTransportURI* state variable (if available). If the DMR receives a *Next()* action but the *NextAVTransportURI* state variable is empty, then the DMR must respond with error 401 (Invalid Action). This requirement does not apply if the DMR is playing a resource from media collections (DIDL_S,

DIDL_V, or any other playlist file format) or PlayContainer operations. In the case of media collections or PlayContainer requests, DLNA defines the proper use of *Next()* and *Previous()* actions. These actions are used to traverse back and forth items in the collection or container. The current version of the Play To controller does not support media collections or PlayContainer operations.
Req. SETNEXT-15
Applies to AV DMR, Audio DMR
If the DMR is playing an audio, A/V, or image resource and if the DMR receives a *Stop()* action, the DMR must stop playing the current resource. If the *NextAVTransportURI* state variable is non-empty, the DMR must not play the next resource.

Design Notes

The UPnP AVTransport specifications define the use of the *SetNextAVTransportURI()* action. In general, this action can be used with audio and A/V resources with few additional clarifications.

The use of this action for images is less clear in the UPnP AVTransport specifications. This document clarifies the use of such action. Specifically:

If a DMR receives a *SetAVTransportURI()* and a *Play()* action to play an image, the DMR plays the image for an indefinite time (see Requirement Device.Media.DMR.Image.JPEG).

At any time, the DMR can receive a *SetNextAVTransportURI()* action. This action conveys the URI of the next content item (images, A/V, or audio).

The controller needs to send a *Next()* action in order to trigger a change from the current image to the next item.

DLNA has clarified the use of the *Next()* action when used in the context of media collections (DIDL_S, DIDL_V, or any other type of playlist format) and also in the context of PlayContainer operations. The *Previous()* and *Next()* actions are used to play the previous and next items in the collection or in a container. However, DLNA acknowledges that the use of a *Next()* action for single resources is unspecified.

This requirement clarifies the use of a *Next()* action when the device has acquired a resource URI that will be played after the current resource.

Exceptions:

Required

Business Justification:

This requirement helps DMRs to become more responsive when the user plays a list of content using Play To. The transition latency can be defined as the wait time from the moment the current content finishes playing to the moment when the next content item starts playing. The transition latency can be minimized if the DMR knows the new URI ahead of time. The DMR can download the new content (or a segment of the new content) before it needs to render such content.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

NETMEDIA-0126; Update

Device.Media.DMR.Base.VolumeControl

Target Feature: Device.Media.DMR.Base

Title: DMR supports volume control

Applicable OS Versions:

- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows 8 Client x64

Description:

Req. VOLUME-01
Applies to A/V DMR, Audio DMR
A DMR device must support volume control requests from a Digital Media Controller and adjust the volume output accordingly. A DMC requests volume adjustments using the *SetVolume()* action with a level between 0 (silence) and 100 (loudest). A Digital Media Renderer needs not support volume control for channels other than the master channel.
Req. VOLUME-05
Applies to A/V DMR, Audio DMR
A DMR that receives a *SetVolume()* action must adjust the volume to the requested level while the DMR is in the following states: NO_MEDIA_PRESENT, STOPPED, PLAYING, PAUSED_PLAYBACK.
Req. VOLUME-10
Applies to A/V DMR, Audio DMR
A DMR must declare the implementation of the *Volume* state variable, the *SetVolume()* action, and the *GetVolume()* action in the Service Description Document. In particular, the allowed range value for the state variable must indicate a minimum of 0 and a maximum of 100.

Design Notes

This requirement is in compliance with current UPnP specifications and DLNA Guidelines. DLNA tests for volume control but not in all applicable states.

Exceptions:

Required

Business Justification:

Any modern media device needs to support volume control. Also, from a usability perspective, volume control needs to be implemented across all different states.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

NETMEDIA-0072; Update

Device.Media.DMR.Base.WakeOnLAN

Target Feature: Device.Media.DMR.Base

Title: DMR supports Wake on LAN

Applicable OS Versions:

- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows 8 Client x64

Description:

Req. WOL-01
Applies to A/V DMR, Audio DMR
If a DMR implements a low power mode (sleep mode), the DMR must be capable of waking up to a normal power mode when it receives a Magic Packet. A Magic Packet is a broadcast UDP packet on port 0 with a payload that contains six bytes of 0xFF followed by the MAC address of the DMR repeated 16 times. Within 10 seconds of receiving a Magic Packet, the DMR must send an ssdp:alive message and be capable of accepting a connection. This requirement only applies to the wired Ethernet interface of a DMR.

Req. WOL-02
Applies to AV DMR, Audio DMR
If a DMR implements a low power mode (sleep mode), the DMR must include the following XML fragment in its Device Description Document: <microsoft:magicPacketWakeSupported xmlns:microsoft="urn:schemas-microsoft-com:WMPNSS-1-0"> 1 </microsoft:magicPacketWakeSupported> This requirement only applies to the wired Ethernet interface of a DMR.

Design Notes

This requirement is necessary to allow communications with a DMR operating in a low power mode.

For additional information on Wake-on-LAN see Network Device Class Power Management Reference Specification, Version 2.0, at http://go.microsoft.com/fwlink/?LinkId=236996

The XML fragment described in the requirement includes blank spaces for clarity. Any unnecessary spaces are removed when the fragment is inserted into a DDD document.

Some devices implement multiple levels of low-power modes. In some levels, the device remains responsive to UPnP actions received over the network. In other levels the device no longer responds to UPnP actions. When a device no longer responds to UPnP actions, then the device is considered to be in sleep mode. As defined in this specification, a device that enters a sleep mode needs to implement a Wake-On-LAN protocol because users expect their devices to wake up automatically when making connections.

Exceptions:

If the DMR implements a low power (sleep) mode, then it must satisfy this requirement.

Business Justification:

Because of energy consumption constraints, DMRs like TVs and AV Receivers are often designed with low power modes (sleep modes). Although this is a good design option, if a user sends content to the DMR, the user expects the DMR to wake up automatically and play the content with minimal delay.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

NETMEDIA-0124; Update

Device.Media.DMR.Base.WiFiDirectSupport

Target Feature: Device.Media.DMR.Base

Title: Wi-Fi Direct support

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64

Description:

Req. WFD-01
Applies to A/V DMR, Audio DMR
If a device implements Wi-Fi Direct, the DMR functionality must be available over the Wi-Fi Direct channel. All requirements described in this document remain valid when the DMR operates using a Wi-Fi Direct connection.

Design Notes

Wi-Fi Direct is a new peer-to-peer connection technology that allows connections between endpoints without the need to use an intermediate access point.

Exceptions:

If Implemented

Business Justification:

Many new computers will implement Wi-Fi Direct and will be capable of streaming content over Wi-Fi Direct (using the Play To functionality). Users want features like Play To to work independently of the connection channel between devices.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

New

Device.Media.DMR.Base.WMDRMNDLinkProtectionSupport

Target Feature: Device.Media.DMR.Base

Title: DMR supports WMDRM-ND link protection

Applicable OS Versions:

Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client x86
Windows 8 Client x64

Description:

Req. WMDRM-01
Applies to A/V DMR, Audio DMR
If a device implements WMDRM-ND, then the implementation must adhere to the DLNA Guidelines for WMDRM-ND Link Protection.
Req. WMDRM-05
Applies to A/V DMR, Audio DMR
If a device implements WMDRM-ND, then the implementation must decode and play all of the following audio profiles: WMDRM_WMALSL WMDRM_WMABASE WMDRM_WMAFULL
Req. WMDRM-10
Applies to A/V DMR
If a device implements WMDRM-ND, then the implementation must decode and play all of the following A/V profiles: WMDRM_WMVMED_BASE WMDRM_WMVMED_FULL WMDRM_WMVHIGH_FULL

Design Notes

WMDRM-ND provides the protection necessary to stream high-value content from a PC to a device.

Exceptions:

If a Digital Media Renderer implements WMDRM-ND, then the device must comply with this requirement.

Business Justification:

Several applications allow streaming protected content from a PC to a device as long as the device implements WMDRM-ND.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

NETMEDIA-0051; Updated

Device.Media.DMR.Image

Description:

Devices that can serve images need to implement these requirements

Related Requirements:

Device.Media.DMR.Image.JPEG

Device.Media.DMR.Image.JPEG

Target Feature: Device.Media.DMR.Image

Title: JPEG requirements for a DMR

Applicable OS Versions:

Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client x86
Windows 8 Client x64

Description:

Req. JPEG-01
Applies to A/V DMR, Audio DMR (if it play images)
A Digital Media Renderer that receives an action to play a valid JPEG image must play the image for an indefinite time. Hence, the Play To controller decides the playback duration for any image. This requirement applies while the device is active outside low-power mode. If the device enters low-power mode, this requirement no longer applies. This requirement only applies while the device operates in DMR mode. If the user changes the mode to watch television or optical disk content, the requirement no longer applies.

Design Notes

A basic principle of DMR operation is that users interact with the controller UI and not with a UI that may be available in the DMR/DMP device. For this reason, the duration time should be defined by the controller and not by the DMR.

Exceptions:

Required

Business Justification:

The behavior described in the requirement enables good consistent experiences for controllers that implement advanced functions like image playlists, mixed playlists, and the use of actions that optimize performance like SetNextAVTransportURI().

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

NETMEDIA-0116; Update

Device.Media.DMS.Audio

Description:

Devices that can serve audio need to implement these requirements

Related Requirements:

Device.Media.DMS.Audio.AACAudioStreaming
Device.Media.DMS.Audio.AlbumArt
Device.Media.DMS.Audio.MP3AudioStreaming
Device.Media.DMS.Audio.WMAStreaming

Device.Media.DMS.Audio.AACAudioStreaming

Target Feature: Device.Media.DMS.Audio

Title: DMS supports AAC audio streaming

Applicable OS Versions:

Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client x86
Windows 8 Client x64

Description:

Req. AAC-51
A DMS must be able to expose and stream the following audio profile: **AAC_ISO_320**
Req. AAC-55
In order to adequately support the AAC profile, the DMS must include in the CDS the following properties: DLNA Profile ID *dc:title* *upnp:class* *res@size* *res@duration* Additionally, a DMS should provide in the CDS the following values: *dc:creator* *upnp:album* *upnp:genre*
Req. AAC-60
In compliance with the DLNA Guidelines, if the DMS receives an HTTP GET or HEAD request that includes the *getContentFeatures.dlna.org: 1* header, the DMS must include in the response the *contentFeatures.dlna.org* header carrying the fourth field of the *protocolInfo* property for the requested resource.

Design Notes

This requirement describes the necessary properties to expose an AAC-encoded audio resource. When a media resource is exposed in the CDS, the corresponding *res*element carries several attributes including *protocolInfo*, *size*, and *duration*.

The fourth field of *protocolInfo* carries the Profile ID, which is important because DMC and DMP devices rely on this value to make decisions about the ability to play the resource.

The *res@size* attribute is important because DMPs and DMCs use this information for seek operations using byte-range requests. Similarly, the *res@duration* attribute is important because DMPs and DMCs use this information for seek operations using time-range requests. The *res@duration* property is also important because DMPs and DMCs use this information to build a user interface for playback.

Exceptions:

Required

Business Justification:

The AAC format has become a popular method to encode audio distributed over the Internet.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

NETMEDIA-0043; Update

Device.Media.DMS.Audio.AlbumArt

Target Feature: Device.Media.DMS.Audio

Title: Album Art requirements for a DMS

Applicable OS Versions:

- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows 8 Client x64

Description:

Req. ALBUMART-51
A DMS must expose album art images for all items described with the *object.item.musicItem* class, as specified in section 7.3.61 of the DLNA Guidelines. If an album art image is available as file metadata, the DMS should use this image to expose album art in the CDS. If album art image is not available as file metadata, the DMS should generate some representative icon for exposure in the CDS.
Req. ALBUMART-55
If a DMS exposes a CDS object of class *object.container.album.musicAlbum* or any derived classes, the DMS must expose album art image for the container as specified in section 7.3.61 of the DLNA Guidelines. If album art image is available as file metadata, the DMS should use this image to expose album art in the CDS. If album art image is not available as file metadata, the DMS should generate some representative icon for exposure in the CDS.

Design Notes

The DLNA Guidelines (August 2009 version) define the method to expose album art in the CDS.

Exceptions:

Required

Business Justification:

Displaying album art has become an integral part of the music browsing/playback experience. Many DMPs and DMCs rely on album art exposed in the CDS to provide rich graphical designs for user interfaces.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

NETMEDIA-0059; Update

Device.Media.DMS.Audio.MP3AudioStreaming

Target Feature: Device.Media.DMS.Audio

Title: DMS supports MP3 audio streaming

Applicable OS Versions:

- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows 8 Client x64

Description:

Req. MP3-51
A DMS must be able to expose and stream the following audio profile: **MP3**
Req. MP3-55
In order to adequately support the MP3 profile, the DMS must include in the CDS the following properties: DLNA Profile ID *dc:title* *upnp:class* *res@size* *res@duration*

Additionally, a DMS should provide in the CDS the following values: *dc:creator* *upnp:album* *upnp:genre*
Req. MP3-60
In compliance with the DLNA Guidelines, if the DMS receives an HTTP GET or HEAD request that includes the *getContentFeatures.dlna.org: 1* header, the DMS must include in the response the *contentFeatures.dlna.org* header carrying the fourth field of the *protocolInfo* property for the requested resource.

Design Notes

This requirement describes the necessary properties to expose an MP3-encoded audio resource. When a media resource is exposed in the CDS, the corresponding *res*element carries several attributes including *protocolInfo*, *size*, and *duration*.

The fourth field of *protocolInfo* carries the Profile ID, which is important because DMC and DMP devices rely on this value to make decisions about the ability to play the resource.

The *res@size* attribute is important because DMPs and DMCs use this information for seek operations using byte-range requests. Similarly, the *res@duration* attribute is important because DMPs and DMCs use this information for seek operations using time-range requests. The *res@duration* property is also important because DMPs and DMCs use this information to build a user interface for playback.

Exceptions:

Required

Business Justification:

The MP3 format has become a popular method to encode audio distributed over the Internet.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

NETMEDIA-0036; Update

Device.Media.DMS.Audio.WMAStreaming

Target Feature: Device.Media.DMS.Audio

Title: DMS supports WMA streaming

Applicable OS Versions:

Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client x86
Windows 8 Client x64

Description:

Req. WMA-51
A DMS must be able to expose and stream the following audio profile: **WMABASE** **WMAFULL** **WMALSL**
Req. WMA-55
In order to adequately support the WMA profiles, the DMS must include in the CDS the following properties: DLNA Profile ID *dc:title* *upnp:class* *res@size* *res@duration* Additionally, a DMS should provide in the CDS the following values: *dc:creator* *upnp:album* *upnp:genre*
Req. WMA-60
In compliance with the DLNA Guidelines, if the DMS receives an HTTP GET or HEAD request that includes the *getContentFeatures.dlna.org: 1* header, the DMS must include in the response the *contentFeatures.dlna.org* header carrying the fourth field of the *protocolInfo* property for the requested resource.

Design Notes

This requirement describes the necessary properties to expose WMA-encoded audio resources. When a media resource is exposed in the CDS, the corresponding *res*element carries several attributes including *protocolInfo*, *size*, and *duration*.

The fourth field of *protocolInfo* carries the Profile ID, which is important because DMC and DMP devices rely on this value to make decisions about the ability to play the resource.

The *res@size* attribute is important because DMPs and DMCs use this information for seek operations using byte-range requests. Similarly, the *res@duration* attribute is important because DMPs and DMCs use this information for seek operations using time-range requests. The *res@duration* property is also important because DMPs and DMCs use this information to build a user interface for playback.

WMA content is stored in files encoded using the Advanced System Format (ASF). The table below indicates the location of the relevant metadata information within an ASF file. DMS devices that comply with this requirement extract metadata from the file to expose the corresponding CDS properties.

CDS Property	ASF field	ASF location
dc:title	Title	Title is a field available in the Content Description Object
dc:creator	Author	Author is a field available in the Content Description Object
dc:date	Creation Date	Creation Date is a field available in the File Properties Object
upnp:genre	WM/Genre	WM/Genre is a descriptor added to the Extended Content Description Object.
upnp:album	WM/AlbumTitle	WM/AlbumTitle is a descriptor added to the Extended Content Description Object.
res@size	File Size	File Size is a field available in the File Properties Object
res@duration	Play Duration	Play Duration is a field available in the File Properties Object

Exceptions:

Required

Business Justification:

The WMA format has become a popular method to encode audio that plays in computers and in other media devices.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

NETMEDIA-0022; Update

Device.Media.DMS.AV

Description:

Devices that can serve A/V content need to implement these requirements

Related Requirements:

Device.Media.DMS.AV.AVCVideoStreaming
Device.Media.DMS.AV.ThumbnailImagesForTheAVMediaClass
Device.Media.DMS.AV.WMVStreaming

Device.Media.DMS.AV.AVCVideoStreaming

Target Feature: Device.Media.DMS.AV

Title: DMS supports AVC video streaming

Applicable OS Versions:

Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client x86
Windows 8 Client x64

Description:

Req. AVC-51
A DMS must be able to expose and stream the following video profiles: **SD profile is TBD** **HD profile is TBD**
Req. AVC-55
In order to adequately support the AVC profiles, the DMS must include in the CDS the following properties: DLNA Profile ID *dc:title* *upnp:class* *res@size* *res@duration* Additionally, a DMS should provide in the CDS the following values:

dc:creator *dc:date* *upnp:genre*
Req. AVC-60
In compliance with the DLNA Guidelines, if the DMS receives an HTTP GET or HEAD request that includes the *getContentFeatures.dlna.org: 1* header, the DMS must include in the response the *contentFeatures.dlna.org* header carrying the fourth field of the *protocolInfo* property for the requested resource.

Design Notes

This requirement describes the necessary properties to expose an AVC-encoded audiovisual resource. When a media resource is exposed in the CDS, the corresponding *res*element carries several attributes including *protocolInfo*, *size*, and *duration*.

The fourth field of *protocolInfo* carries the Profile ID, which is important because DMC and DMP devices rely on this value to make decisions about the ability to play the resource.

The *res@size* attribute is important because DMPs and DMCs use this information for seek operations using byte-range requests. Similarly, the *res@duration* attribute is important because DMPs and DMCs use this information for seek operations using time-range requests. The *res@duration* property is also important because DMPs and DMCs use this information to build a user interface for playback.

Exceptions:

Required

Business Justification:

The number of devices capable of generating video using the AVC format continues to expand. Simultaneously, with the adoption of HTML5, many of the web sites that share video content will migrate to AVC because it is one of the recognized formats in HTML5.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

NETMEDIA-0037; Update

Device.Media.DMS.AV.ThumbnailImagesForTheAVMediaClass

Target Feature: Device.Media.DMS.AV

Title: Thumbnail Images for the A/V media class

Applicable OS Versions:

Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client x86
Windows 8 Client x64

Description:

Req. VTHUMB-51
A DMS must generate a thumbnail image that represents the A/V item. The thumbnail image should correspond to the re-sized version of one of the video frames.
Req. VTHUMB-55
The thumbnail image must adhere to the DLNA requirements for JPEG thumbnails: The thumbnail image uses a Profile ID of **JPEG_TN** as defined in section 7.1.5, Volume 2, DLNA Guidelines The thumbnail image is exposed in the CDS as an alternative *res* element within the *item* element that describes the video item, as defined in section 7.3.60, Volume 1, DLNA Guidelines

Design Notes

The DLNA Guidelines (August 2009 version) define the method to expose thumbnail images in the CDS.

Exceptions:

Required

Business Justification:

Displaying thumbnails has become an integral part of the video browsing/playback experience. Many DMPs and DMCs rely on thumbnails exposed in the CDS to provide rich graphical designs for user interfaces.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

NETMEDIA-0061; Update

Device.Media.DMS.AV.WMVStreaming

Target Feature: Device.Media.DMS.AV

Title: DMS supports WMV streaming

Applicable OS Versions:

Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client x86
Windows 8 Client x64

Description:

Req. WMV-51
A DMS must be able to expose and stream the following video profiles: **WMVMED_BASE** **WMVMED_FULL** **WMVHIGH_FULL**
Req. WMV-55
In order to adequately support the AVC profiles, the DMS must include in the CDS the following properties: DLNA Profile ID *dc:title* *upnp:class* *res@size* *res@duration* Additionally, a DMS should provide in the CDS the following values: *dc:creator* *dc:date* *upnp:genre*
Req. WMV-60
In compliance with the DLNA Guidelines, if the DMS receives an HTTP GET or HEAD request that includes the *getContentFeatures.dlna.org: 1* header, the DMS must include in the response the *contentFeatures.dlna.org* header carrying the fourth field of the *protocolInfo* property for the

requested resource.

Design Notes

This requirement describes the necessary properties to expose WMV-encoded audiovisual resources. When a media resource is exposed in the CDS, the corresponding *res*element carries several attributes including *protocolInfo*, *size*, and *duration*.

The fourth field of *protocolInfo* carries the Profile ID, which is important because DMC and DMP devices rely on this value to make decisions about the ability to play the resource.

The *res@size* attribute is important because DMPs and DMCs use this information for seek operations using byte-range requests. Similarly, the *res@duration* attribute is important because DMPs and DMCs use this information for seek operations using time-range requests. The *res@duration* property is also important because DMPs and DMCs use this information to build a user interface for playback.

WMV content is stored in files encoded using the Advanced System Format (ASF). The table below indicates the location of the relevant metadata information within an ASF file. DMS devices that comply with this requirement extract metadata from the file to expose the corresponding CDS properties.

CDS Property	ASF field	ASF location
dc:title	Title	Title is a field available in the Content Description Object
dc:creator	Author	Author is a field available in the Content Description Object
dc:date	Creation Date	Creation Date is a field available in the File Properties Object
upnp:genre	WM/Genre	WM/Genre is a descriptor added to the Extended Content Description Object.
upnp:album	WM/AlbumTitle	WM/AlbumTitle is a descriptor added to the Extended Content Description Object.
res@size	File Size	File Size is a field available in the File Properties Object
res@duration	Play Duration	Play Duration is a field available in the File Properties Object

Exceptions:

Required

Business Justification:

The WMV format has become a popular method to encode audiovisual content that plays in computers and in other media devices.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

NETMEDIA-0021; Update

Device.Media.DMS.Base

Description:

Baseline requirements for all DMS devices

Related Requirements:

Device.Media.DMS.Base.ChangingFriendlyName
Device.Media.DMS.Base.DeviceInformation
Device.Media.DMS.Base.DLNA15CertificationCompliance
Device.Media.DMS.Base.LowPowerModeSupport
Device.Media.DMS.Base.MetaDataPackage
Device.Media.DMS.Base.MultipleHTTPConnections
Device.Media.DMS.Base.OptionalFieldsEntries
Device.Media.DMS.Base.Performance
Device.Media.DMS.Base.RangeRequests
Device.Media.DMS.Base.SearchOperations
Device.Media.DMS.Base.StreamsContinuously
Device.Media.DMS.Base.WakeOnLAN

Device.Media.DMS.Base.ChangingFriendlyName

Target Feature: Device.Media.DMS.Base

Title: DMS supports changing the Friendly Name

Applicable OS Versions:

Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client x86
Windows 8 Client x64

Description:

Req. NAME-50
A Digital Media Server (DMS) must allow the user to update the DMS friendly name using any configuration method. The DMS friendly name is defined as the string value included in element friendlyName in the Device Description Document.

Design Notes

Some DMS devices may choose to implement this requirement using the presentationURL element in the Device Description Document. The URL in this element points to a device web page that may define configuration options. One of the configuration options can give users the opportunity to change the friendly name.

Other DMS devices may choose different means to give users the option to change the DMS friendly name.

The use of a presentation URL to expose an HTML presentation page is described in the UPnP Device Architecture (DA) specification at http://www.upnp.org.

Exceptions:

Required

Business Justification:

Windows computers display the DMS friendly name in user interfaces that expose such information. If a user buys two identical DMS devices from the same manufacturer, these devices typically have identical friendly names. Windows will display the two devices with the same name, which can be confusing for users. The ability to change the friendly name gives users the ability to correct this problem.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

NETMEDIA-0017; Update

Device.Media.DMS.Base.DeviceInformation

Target Feature: Device.Media.DMS.Base

Title: Availability of DMS Information

Applicable OS Versions:

Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client x86
Windows 8 Client x64

Description:

Req. INFO-01

The following device information must be available at the time of testing:

If the device uses firmware (or middleware): firmware vendor and firmware version
If the device uses an application model: App name, App vendor, App version, and App platform
If the device uses an Operating System: OS name and OS version
Hardware Platform (e.g ARM, x86, x64, etc)
Manufacturer Name

Model Number

Design Notes

The information is collected to identify certified devices, track bugs, identify areas of improvement, etc.

Exceptions:

Required

Business Justification:

To identify certified devices, track bugs more accurately and identify areas of improvement.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

INFO-01

Device.Media.DMS.Base.DLNA15CertificationCompliance

Target Feature: Device.Media.DMS.Base

Title: DMS complies with DLNA 1.5 Certification

Applicable OS Versions:

Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client x86
Windows 8 Client x64

Description:

Req. DLNA-50
A Digital Media Server (DMS) must demonstrate compliance with the DLNA protocols using one of the following alternatives: The DMS passes the DLNA Certification Program and obtains a DLNA Certificate The DMS uses an OEM reference design (hardware/software) that has already passed the DLNA Certification Program and has obtained a DLNA Certificate The DMS uses middleware from an independent software provider. The middleware has passed the DLNA Certification Program and has obtained a DLNA certificate.

If the device implements multiple network interfaces (Ethernet, Wi-Fi, etc.), the DLNA Certificate must indicate a satisfactory evaluation using all interfaces. If the device implements additionally a DMP, the device must demonstrate compliance with the DLNA protocols for the DMP class using one of the alternatives mentioned above. If the device implements additionally a DMS, the device must demonstrate compliance with the DLNA protocols for the DMS class using one of the alternatives mentioned above.
Req. DLNA-55
The DLNA Certification for a DMS must include the Image class, the Audio class, and the A/V class.

Design Notes

The Windows Certification Program for DMS devices has been designed for generic devices that store content and make the content available to the network (NAS devices, Personal Video Recorders, devices that support USB storage, etc.)

The Windows Certification Program currently can neither test nor verify compliance for DMS devices that get content from external broadcast sources and make such content available in the network (Set-Top Boxes, TVs, etc).

Exceptions:

Required

Business Justification:

DLNA defines the baseline specifications for interconnected media devices. The DLNA organization also maintains a strict certification program that ensures good implementations of the protocols. However, DLNA does not define user experiences, performance, and product quality, which are the areas covered by the Windows Device Certification Program.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

NETMEDIA-0002; Update

Device.Media.DMS.Base.LowPowerModeSupport

Target Feature: Device.Media.DMS.Base

Title: DMS declares low-power mode support

Applicable OS Versions:

Windows 7 Client x86

Windows 7 Client x64
Windows 8 Client x86
Windows 8 Client x64

Description:

Req. LOWPOW-51
If a DMS implements a low power mode (sleep mode), the DMS must include the following XML fragment in its Device Description Document: <microsoft:magicPacketWakeSupported xmlns:microsoft="urn:schemas-microsoft-com:WMPNSS-1-0"> 1 </microsoft:magicPacketWakeSupported> This requirement only applies to the wired Ethernet interface of a DMS.

Design Notes

This requirement is necessary to allow communications with a DMS operating in a low power mode.

The XML fragment described in the requirement includes blank spaces for clarity. Any unnecessary spaces are removed when the fragment is inserted into a DDD document.

Some devices implement multiple levels of low-power modes. In some levels, the device remains responsive to UPnP actions received over the network. In other levels the device no longer responds to UPnP actions. When a device no longer responds to UPnP actions, then the device is considered to be in sleep mode. As defined in this specification, a device that enters a sleep mode needs to implement a Wake-On-LAN protocol because users expect their devices to wake up automatically when making connections.

Exceptions:

Required

Business Justification:

Because of energy consumption constraints, DMS devices are often designed with low power modes (sleep modes). Although this is a good design option, if a user attempts to browse or get content from the server, the user expects the DMS to wake up automatically and serve the information with minimal delay.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

NETMEDIA-0078; Update

Device.Media.DMS.Base.MetaDataPackage

Target Feature: Device.Media.DMS.Base

Title: The device has a metadata package that meets Windows 8 metadata specifications for the specific device category; and the metadata package is posted for public usage.

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64

Description:

The device has a metadata package that meets Windows 8 metadata specifications for the specific device category; And the metadata package is posted for public distribution through the Windows Metadata Information Services (WMIS) once the hardware certification submission is approved and/or available for distribution with the device.

Exceptions:

Required

Business Justification:

Supporting the metadata requirements will assure Windows supports a complete experience when using DMS devices.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

New

Device.Media.DMS.Base.MultipleHTTPConnections

Target Feature: Device.Media.DMS.Base

Title: DMS supports multiple HTTP connections

Applicable OS Versions:

Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client x86
Windows 8 Client x64

Description:

Req. HTTP-51
A Digital Media Server (DMS) must support a minimum of two concurrent HTTP connections and be able to respond to two simultaneous requests without error (either GET or POST requests).

Design Notes

In the test, the test tool requests the DMS device to streams two content binaries simultaneously.

Exceptions:

Required

Business Justification:

If a DMS starts streaming long content (e.g., a movie), then one HTTP channel remains busy for a long duration of time. A second channel becomes necessary to communicate with the DMS and request information, or support a second stream for a different receiver.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

NETMEDIA-0054; Update

Device.Media.DMS.Base.OptionalFieldsEntries

Target Feature: Device.Media.DMS.Base

Title: DMS must provide non-empty and valid entries for optional fields

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64

Description:

Req. DDD-51
In addition to the normative fields in a Device Description Document (DDD), a DMS must provide non-empty and valid entries for the following optional fields: The manufacturers name in the <manufacturer> element The model name in the <modelName> element

The model number in the <modelNumber> element A user-friendly name in the <friendlyName> element

Design Notes

The UPnP Device Architecture (DA) specification defines the structure of the Device Description Document.

Exceptions:

Required

Business Justification:

Having these entries allows for a better diagnosis of device issues. The friendly name appears in different locations in Windows UI components.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: Windows RC

Comments:

New

Device.Media.DMS.Base.Performance

Target Feature: Device.Media.DMS.Base

Title: DMS PERFORMANCE REQUIREMENTS

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64

Description:

Req. PERF-51
A DMS device must comply with the performance specifications indicated in Table PERF below. Test constraints: The test content includes 300 music items and 300 picture items. The DMS must include all the test files. At least one container must include 300 items. All tests require ideal network conditions.

Table PERF: Expected performance values for DMS devices

No.	Operation	Performance requirement	Probability of success	Applicability

OP10	From: The moment when the user has enabled the device as a DMS To: The moment when the DMS sends the first advertisement message to the network	10 seconds	90%	All DMS devices
OP15	From: The momen when the user disables DMS functionality To: The moment when the DMS sends byebye messages to the network.	10 seconds	90%	All DMS devices
OP20	From: The moment when a DMP issues a Browse() request to obtain the list of children objects in the Root Container To: The moment when the DMP receives the response	3 seconds	70%	All DMS devices
OP30	From: The moment when a DMP issues a Browse() request to read the metadata associated with any CDS object To: The moment when the DMP receives the response	3 seconds	70%	All DMS devices
OP40	From: The moment when a DMP issues a Browse() request to obtain the list of children objects in a container with 300 items To: The moment when the DMP starts receiving the response.	10 seconds	70%	All DMS devices
OP50	From: The moment when a DMP issues a GET request to obtain content bytes To: The moment when the DMP starts receiving content bytes.	3 seconds	70%	All DMS devices
OP60	From: The moment when a DMP issues a Search() request to search for one item To: the moment when the DMP receives the response for this single item	4 seconds	80%	DMS devices that support Search operations.

Design Notes

The probability of success is defined as the estimated number of cases where the device meets the performance requirement. For example, if the performance requirement is 2 seconds, and the probability of success is 90%, then in a series of 10 tests, the device must respond within 2 seconds in at least 9 out of those 10 tests.

The DLNA Guidelines define the term ideal network conditions as a connection with virtually unlimited bandwidth and negligible congestion.

Exceptions:

Required

Business Justification:

Users are looking now for high quality devices and expect their devices to be always responsive and show minimal delays.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

New

Device.Media.DMS.Base.RangeRequests

Target Feature: Device.Media.DMS.Base

Title: DMS supports range requests

Applicable OS Versions:

Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client x86
Windows 8 Client x64

Description:

Req. RANGE-51
For any media resource whose profile matches any of the profiles described in this document the DMS device must support range-retrieval operations using at least one of the two defined DLNA methods: Time-range requests that use the *TimeSeekRange.dlna.org* HTTP header Byte-range requests that use the *Range* HTTP header
Req. RANGE-55
As defined in the DLNA Guidelines, the DMS must declare the type of supported range requests using the *DLNA.ORG_OP* parameter for any media resource exposed in the CDS.

Design Notes

The use of the Range HTTP header is defined in the HTTP/1.1 specifications (RFC 2616). The use of the TimeSeekRange.dlna.org HTTP header is defined in section 7.4.40 of the DLNA Guidelines, Volume 1, August 2009.

Exceptions:

Required

Business Justification:

Client devices (DMR, DMP) use time-range or byte-range requests to implement playback trick modes like Pause, Seek, Fast Forward, or Rewind.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

NETMEDIA-0066; Update

Device.Media.DMS.Base.SearchOperations

Target Feature: Device.Media.DMS.Base

Title: DMS supports search operations

Applicable OS Versions:

- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows 8 Client x64

Description:

Req. SEARCH-51
If the DMS implements Search, the DMS must accept search queries and return the requested values for the operators listed in table OPSEARCH below. The second column in table OPSEARCH clarifies if a DMS is required or not to implement a property.
Req. SEARCH-55
If the DMS implements Search, the DMS must respond to valid queries that use up to two concurrent expressions. The expressions are joined by the conjunctions: and, or. Each expression uses the properties and operators defined in table OPSEARCH below. The following three examples show queries that use two expressions: dc:title contains Lake and refID exists false dc:creator contains Brahms or dc:description contains Brahms upnp:genre = Rock and dc:date > 1990-01-01 A DMS should support additional properties (beyond those described in table OPSEARCH) and it should support queries with more than two expressions.

Req. SEARCH-60
If the DMS implements Search, the DMS must indicate the list of searchable properties in response to a *GetSearchCapabilities()* action request.

Table OPSEARCH: List of search operators per CDS properties

Property	Required/Optional	Search operators
upnp:class	Required by UPnP/DLNA	derivedfrom, exists
@refID	Required by UPnP/DLNA for duplicate items.	=, exists
dc:title	Required by UPnP/DLNA	contains, =, exists
dc:creator	Optional	contains, =, exists
upnp:artist	Optional	contains, =, exists
upnp:actor	Optional	contains, =, exists
upnp:album	Optional	contains, =, exists
upnp:producer	Optional	contains, =, exists
upnp:director	Optional	contains, =, exists
dc:description	Optional	contains, =, exists
upnp:longDescription	Optional	contains, =, exists
upnp:genre	Optional	contains, =, exists
dc:date	Required by the Windows Certification Program for images.	=, <, <=, >, <=, !=, exists

Design Notes

The DLNA Guidelines (Volume 1, August 2009 version) define Search as an optional action for a DMS. The DLNA Guidelines require that any DMS that implements Search for string-based properties support three operators: contains, =, and exist. This specification identifies the relevant string properties for user-based search queries.

A CDS entry with the @refID property indicates a duplicate item in the CDS. Consequently, a search query that includes refID exists false is used to prevent the DMS from returning duplicates.

Exceptions:

If ImplementedIf a Digital Media Server (DMS) implements the optional CDS Search () action, then the DMS must satisfy this requirement.

Business Justification:

The average number of items in the user's media library continues to increase. There are users that include already thousands of items in a media library. The use of Search provides better performance than Browse when the libraries become large. In general, implementing full support for Search can be a complex operation, but this specification facilitates the implementation by defining a small set of relevant properties.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

NETMEDIA-0026; Update

Device.Media.DMS.Base.StreamsContinuously

Target Feature: Device.Media.DMS.Base

Title: DMS streams continuously

Applicable OS Versions:

Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client x86
Windows 8 Client x64

Description:

Req. STRESS-51
A DMS must be able to stream a mixture of content types continuously without errors, without user intervention, and without degradation in quality, for at least 6 hours. In practice the test will verify that the device neither crashes nor responds with long delays.

Design Notes

A DMS device needs to work autonomously for long periods of time. The test associated with this requirement makes sequential requests for content to the DMS for a period of 6 hours. However, DMS devices should be designed to perform consistently for longer periods of time.

Exceptions:

Required

Business Justification:

Users expect their devices to work with the highest quality for long periods of time.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

NETMEDIA-0047; Update

Device.Media.DMS.Base.WakeOnLAN

Target Feature: Device.Media.DMS.Base

Title: DMS supports Wake on LAN

Applicable OS Versions:

Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client x86
Windows 8 Client x64

Description:

Req. WOL-51
If a DMS implements a low power mode (sleep mode), the DMS must be capable of waking up to a normal power mode when it receives a Magic Packet. A Magic Packet is a broadcast UDP packet on port 0 with a payload that contains six bytes of 0xFF followed by the MAC address of the DMS repeated 16 times. Within 10 seconds of receiving a Magic Packet, the DMS must send an ssdp:alive message and be capable of accepting requests. This requirement only applies to the wired Ethernet interface of a DMR.

Design Notes

This requirement is necessary to allow communications with a DMS operating in a low power mode.

For additional information on Wake-On-LAN see the Network Device Class Power Management Reference Specification, Version 2.0, at http://go.microsoft.com/fwlink/?LinkId=236996

Some devices implement multiple levels of low-power modes. In some levels, the device remains responsive to UPnP actions received over the network. In other levels the device no longer responds to UPnP actions. When a device no longer responds to UPnP actions, then the device is considered to be in sleep mode. As defined in this specification, a device that enters a sleep mode needs to implement a Wake-On-LAN protocol because users expect their devices to wake up automatically when making connections.

Exceptions:

If ImplementedIf the DMS implements a low power mode, then it must satisfy this requirement.

Business Justification:

Because of energy consumption constraints, DMS devices are often designed with low power modes (sleep modes). Although this is a good design option, if a user attempts to browse or get content from the server, the user expects the DMS to wake up automatically and serve the information with minimal delay.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

NETMEDIA-0011; Update

Device.Media.DMS.Image

Description:

Devices that can serve images need to implement these requirements

Related Requirements:

Device.Media.DMS.Image.JPEGImageTransfer
Device.Media.DMS.Image.ThumbnailImagesForTheImageMediaClass

Device.Media.DMS.Image.JPEGImageTransfer

Target Feature: Device.Media.DMS.Image

Title: DMS supports JPEG image transfer

Applicable OS Versions:

Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client x86
Windows 8 Client x64

Description:

Req. JPEG-51
A DMS must be able to expose and stream the following image profiles: **JPEG_SM** **JPEG_MED** **JPEG_LRG**
Req. JPEG-55
In order to adequately support the JPEG profiles, the DMS must include in the CDS the following properties: DLNA Profile ID *dc:title*

upnp:class

res@size

res@resolution

dc:date

If any of these properties is available as EXIF metadata within a JPEG file, the CDS property value must match the value of the related EXIF metadata property.

Design Notes

Pictures are normally acquired using conventional digital cameras. These cameras include rich metadata values as part of the EXIF structure within a JPEG file. These metadata values include possibly title, size, resolution and date.

Exceptions:

Required

Business Justification:

The JPEG format has become the most popular method to encode images for daily use. It is important to extract metadata values from EXIF tags because users move their images from device to device and expect consistency of metadata descriptions.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

NETMEDIA-0049; Update

Device.Media.DMS.Image.ThumbnailImagesForTheImageMediaClass

Target Feature: Device.Media.DMS.Image

Title: Thumbnail images for the image media class

Applicable OS Versions:

Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client x86
Windows 8 Client x64

Description:

Req. ITHUMB-51
A Digital Media Server (DMS) must generate a thumbnail image as a re-sized version of the original picture.
The thumbnail image must adhere to the DLNA requirements for JPEG thumbnails: The thumbnail image uses a Profile ID of JPEG_TN as defined in section 7.1.5, Volume 2, DLNA Guidelines The thumbnail image is exposed in the CDS as an alternative *res* element within the *item* element that describes the image item, as defined in section 7.3.60, Volume 1, DLNA Guidelines

Design Notes

The DLNA Guidelines (August 2009 version) define the method to expose thumbnail images in the CDS.

Exceptions:

Required

Business Justification:

Displaying thumbnails has become an integral part of the image browsing/playback experience. Many DMPs and DMCs rely on thumbnails exposed in the CDS to provide rich graphical designs for user interfaces.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

NETMEDIA-0060; Update

Device.Network.DevFund

Description:

Network requirements

Related Requirements:

Device.Network.DevFund.NdisVersion
Device.Network.DevFund.NdisVersionLegacy
Device.Network.DevFund.NPOS
Device.Network.DevFund.SelectiveSuspend

Device.Network.DevFund.NdisVersion

Target Feature: Device.Network.DevFund

Title: NDIS devices must conform to the NDIS 6.x requirements in the Windows Driver Kit

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

All NDIS device must conform to NDIS 6.x specified in the Windows Driver Kit.

Design Notes:

See the Windows Driver Kit, "NDIS."

Exceptions: Not Specified

Business Justification:

To ensure successful functionality of devices with Windows.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2007

Comments:

NETWORK-0040

Device.Network.DevFund.NdisVersionLegacy

Target Feature: Device.Network.DevFund

Title: NDIS devices must conform to the NDIS requirements in the Windows Driver Kit

Applicable OS Versions:

- Windows Server 2008 x64
- Windows Server 2008 x86
- Windows 7 Client x64
- Windows 7 Client x86
- Windows Server 2003 x86
- Windows Server 2003 x64
- Windows Server 2008 Release 2 x64

Description:

All NDIS devicemust conform to NDIS specified in the Windows Driver Kit.

Design Notes:

See the Windows Driver Kit, "NDIS."

Exceptions:

Devices targeted for OS's that don't support NDIS 6.x will be exempted.
10/100 Ethernet devices targeted for legacy OS's will be exempted.

Business Justification:

To ensure successful functionality devices with Windows.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2007

Comments:

NETWORK-0040

Device.Network.DevFund.NPOS

Target Feature: Device.Network.DevFund

Title: Network Devices must support No Pause On Suspend (NPOS)

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64
Windows 8 Client ARM
Windows Server 2003 x86
Windows Server 2003 x64
Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64

Description:

Network devices must support No Pause On Suspend (NPOS).

Design Notes:

See the No Pause On Suspend Specification.

Exceptions: Not Specified

Business Justification:

To improve stability of device.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2009

Comments: Not Specified

Device.Network.DevFund.SelectiveSuspend

Target Feature: Device.Network.DevFund

Title: NDIS devices must meet Selective Suspend requirements

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

NDIS devices must meet Selective Suspend requirements.

Design Notes:

Exceptions: Not Specified

Business Justification:

To improve battery life and performance.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2007

Comments:

NETWORK-0040

Device.Network.LAN.Base

Description:

LAN requirements

Related Requirements:

Device.Network.LAN.Base.100MbOrGreater
Device.Network.LAN.Base.32MulticastAddresses
Device.Network.LAN.Base.AdvProperties
Device.Network.LAN.Base.AnyBoundary
Device.Network.LAN.Base.IPv4AndIPv6OffloadParity
Device.Network.LAN.Base.NDISCalls
Device.Network.LAN.Base.NDISRequirements
Device.Network.LAN.Base.PacketFiltering
Device.Network.LAN.Base.PreserveOSServices
Device.Network.LAN.Base.PriorityVLAN
Device.Network.LAN.Base.ShortPacketPadding
Device.Network.LAN.Base.SupportIEEEE8023

Device.Network.LAN.Base.100MbOrGreater

Target Feature: Device.Network.LAN.Base

Title: Ethernet devices must support 100-Mb or greater link speeds

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows Server 2008 x64
Windows Server 2008 x86
Windows 7 Client x64
Windows 7 Client x86
Windows Server 2003 x86
Windows Server 2003 x64
Windows Server 2008 Release 2 x64
Windows 8 Client ARM
Windows 8 Server x64

Description:

Devices must be able to link at 100 Mb or higher speeds.

Exceptions: Not Specified

Business Justification:

To ensure networking devices function properly.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2006

Comments:

NETWORK-0038

Device.Network.LAN.Base.32MulticastAddresses

Target Feature: Device.Network.LAN.Base

Title: Ethernet devices must support filtering for at least 32 multicast addresses

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x64
- Windows Server 2008 x86
- Windows 7 Client x64
- Windows 7 Client x86
- Windows Server 2003 x86
- Windows Server 2003 x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client ARM
- Windows 8 Server x64

Description:

An Ethernet device must support filtering of 32 or more multicast addresses.

Design Notes:

See the Windows Driver Kit, multicast.

See the Windows Driver Kit, "NdisReadNetworkAddress" and "MAC Address."

Exceptions: Not Specified

Business Justification:

To ensure Ethernet devices function properly.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2006

Comments:

NETWORK-0002

Device.Network.LAN.Base.AdvProperties

Target Feature: Device.Network.LAN.Base

Title: Ethernet devices must safeguard advanced properties and provide complete configurability through advanced properties.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x64
- Windows Server 2008 x86
- Windows 7 Client x64
- Windows 7 Client x86
- Windows Server 2003 x86
- Windows Server 2003 x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client ARM
- Windows 8 Server x64

Description:

Ethernet devices must adhere to the standardized registry keywords for controlling advanced features as documented on MSDN. Devices must also safeguard both Microsoft standardized and private keywords from malicious values.

Exceptions: Not Specified

Business Justification:

To ensure Ethernet devices function properly.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2007

Comments:

NETWORK-0007

Device.Network.LAN.Base.AnyBoundary

Target Feature: Device.Network.LAN.Base

Title: Ethernet devices must be able to transmit packets from buffers aligned on any boundary

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64

Windows Server 2008 x64
Windows Server 2008 x86
Windows 7 Client x64
Windows 7 Client x86
Windows Server 2003 x86
Windows Server 2003 x64
Windows Server 2008 Release 2 x64
Windows 8 Client ARM
Windows 8 Server x64

Description:

Buffer alignment refers to whether a buffer begins on an odd-byte, word, double-word, or other boundary. Devices must be able to transmit packets with any of the packets fragments beginning on an odd-byte boundary. For performance reasons, packets must be received into contiguous buffers on a double-word boundary.

Exceptions: Not Specified

Business Justification:

To ensure Ethernet devices function properly.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2006

Comments:

NETWORK-0001

Device.Network.LAN.Base.IPv4AndIPv6OffloadParity

Target Feature: Device.Network.LAN.Base

Title: Ethernet Devices that implement offloads must do so consistently for both IPv4 and IPv6

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64
Windows 8 Client ARM

Description:

Network offloads implemented by Ethernet devices need to operate consistently, irrespective of the IP protocol used. Having offload parity allows Windows customers to have a consistent and predictable experience across both IPv4 and IPv6.

Exceptions: Not Specified

Business Justification:

To ensure Ethernet devices function properly and ensure a consistent experience on all IP protocols.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows 8

Comments: Not Specified

Device.Network.LAN.Base.NDISCalls

Target Feature: Device.Network.LAN.Base

Title: Ethernet devices must make only NDIS library or WDF system calls

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x64
- Windows Server 2008 x86
- Windows 7 Client x64
- Windows 7 Client x86
- Windows 8 Server x64
- Windows Server 2003 x86
- Windows Server 2003 x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client ARM

Description:

A driver for an Ethernet device must make only NDIS or WDF calls. Any calls to other kernel mode components are not allowed.

Design Notes:

See the Windows Driver Kit, "NDIS" and "WDF."

Exceptions: Not Specified

Business Justification:

To ensure network devices function properly.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2006

Comments:

NETWORK-0036

Device.Network.LAN.Base.NDISRequirements

Target Feature: Device.Network.LAN.Base

Title: Ethernet devices must conform to the NDIS requirements in the Windows Driver Kit

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x64
- Windows Server 2008 x86
- Windows 7 Client x64
- Windows 7 Client x86
- Windows 8 Server x64
- Windows Server 2003 x86
- Windows Server 2003 x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client ARM

Description:

All Ethernet device drivers must conform to NDIS specified in the Windows Driver Kit.

Design Notes:

See the Windows Driver Kit, "NDIS."

Exceptions: Not Specified

Business Justification:

To ensure successful functionality of Windows.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2007

Comments:

NETWORK-0040

Device.Network.LAN.Base.PacketFiltering

Target Feature: Device.Network.LAN.Base

Title: Ethernet devices must support packet filtering

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x64
- Windows Server 2008 x86
- Windows 7 Client x64
- Windows 7 Client x86
- Windows 8 Server x64
- Windows Server 2003 x86
- Windows Server 2003 x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client ARM

Description:

The miniport driver must support all filter types in the Windows Driver Kit.

Note: Filtering should be performed in Hardware/Firmware.

Exceptions: Not Specified

Business Justification:

To support proper functioning of Ethernet devices.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2006

Comments:

NETWORK-0035

Device.Network.LAN.Base.PreserveOSServices

Target Feature: Device.Network.LAN.Base

Title: Ethernet devices Miniport Driver/Driver Software must not disable OS services

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64

Windows 7 Client x86
Windows Server 2008 x64
Windows Server 2008 x86
Windows Server 2003 x86
Windows Server 2003 x64

Description:

Ethernet devices Miniport Driver/Driver Software must not disable OS services.

Some devices tend to shutoff services such as the Base Filtering Engine (BFE). This leaves the system vulnerable to attack due to lack of security capabilities.

Design Notes:

Exceptions: Not Specified

Business Justification:

To improve stability of devices.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2009

Comments: Not Specified

Device.Network.LAN.Base.PriorityVLAN

Target Feature: Device.Network.LAN.Base

Title: Ethernet devices that implement link speeds of gigabit or greater must implement Priority & VLAN tagging according to the IEEE 802.1q specification

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows Server 2008 x64
Windows Server 2008 x86
Windows 7 Client x64
Windows 7 Client x86
Windows 8 Server x64
Windows 8 Client ARM
Windows Server 2008 Release 2 x64
Windows Server 2003 x86
Windows Server 2003 x64

Description:

This requirement only applies to Ethernet devices that implement link speeds of gigabit or greater. If the Ethernet device does not implement link speeds of gigabit or greater than this requirement does not apply.

Ethernet device & driver must support inserting and removing of priority and VLAN tags.

Exceptions: Not Specified

Business Justification:

To comply with standards.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2007

Comments:

NETWORK-0049

Device.Network.LAN.Base.ShortPacketPadding

Target Feature: Device.Network.LAN.Base

Title: Ethernet devices must pad short packets with constant data

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x64
- Windows Server 2008 x86
- Windows 7 Client x64
- Windows 7 Client x86
- Windows 8 Server x64
- Windows 8 Client ARM
- Windows Server 2008 Release 2 x64
- Windows Server 2003 x86
- Windows Server 2003 x64

Description:

Padding that is added to short Ethernet packets to bring that packet size to the minimum IEEE 802.3, Section4.2.3.3, packet size must be initialized to either zeros or an 8-bit repeating pattern before the packets are transmitted. The 802.3 Ethernet specification requires packets that are shorter than the minimum packet size to be padded up to the minimum size before the packets are transmitted onto the wire. The 802.3 Ethernet specification does not specify the padding content; however, Microsoft requires the padding to be zeros or another constant value to address security concerns. Some drivers pad the packets with data from previously sent packets; this practice is not acceptable.

Design Notes:

New solutions are recommended to implement a padding of zeros. However, some devices that implement the padding in hardware use 0xffs, which addresses the security concern

Exceptions: Not Specified

Business Justification:

To improve security.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2006

Comments:

NETWORK-0044

Device.Network.LAN.Base.SupportIEEEE8023

Target Feature: Device.Network.LAN.Base

Title: Ethernet devices must comply with IEEE 802.3

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x64
- Windows Server 2008 x86
- Windows 7 Client x64
- Windows 7 Client x86
- Windows 8 Server x64
- Windows Server 2003 x86
- Windows Server 2003 x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client ARM

Description:

All 802.3 Ethernet devices must implement and comply with the IEEE 802.3 specification.

Exceptions: Not Specified

Business Justification:

To comply with standards.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2006

Comments:

NETWORK-0043

Device.Network.LAN.ChecksumOffload

Description:

Network requirements

Related Requirements:

Device.Network.LAN.ChecksumOffload.ChecksumOffload

Device.Network.LAN.ChecksumOffload.ChecksumOffload

Target Feature: Device.Network.LAN.ChecksumOffload

Title: Ethernet devices implement Checksum Offloads

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64
Windows 8 Client ARM
Windows Server 2008 Release 2 x64
Windows 7 Client x64
Windows 7 Client x86
Windows Server 2008 x64
Windows Server 2008 x86
Windows Server 2003 x86
Windows Server 2003 x64

Description:

Send and Receive TCP Checksum Offload for IPv4 and IPv6

Send and Receive IP Checksum Offload for IPv4

Send and Receive UDP Checksum offload for IPv4 and IPv6

Support for TCP Checksum Standardized Keywords is mandatory.

Ethernet devices implement Checksum Offloads must expose the NDIS Enumeration Keywords.

Exceptions: Not Specified

Business Justification:

To ensure Ethernet devices function properly.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2009

Comments: Not Specified

Device.Network.LAN.DCB

Description:

LAN requirements

Related Requirements:

Device.Network.LAN.DCB.DCB

Device.Network.LAN.DCB.DCB

Target Feature: Device.Network.LAN.DCB

Title: Ethernet devices that implement Data Center Bridging (DCB) must comply with the DCB Specification.

Applicable OS Versions:

Windows 8 Server x64

Description:

This requirement only applies to Ethernet devices that support Data Center Bridging (DCB).

Ethernet devices that implement Data Center Bridging (DCB) must comply with the following requirements:

MaxNumTrafficClasses must be between 3 and 8 inclusive.

MaxNumEtsCapableTrafficClasses must be at least 2.

MaxNumPfcEnabeldTrafficClasses must be at least 1.

ETS Must be supported

PFC Must be supported

Strict Priority Must be supported

iSCSI CNAs must support classification element for iSCSI traffic (TCP ports 860 and 3260, both src and dest port)

FCoE CNAs must support classification element for FCoE traffic (Ethertype of 0x8906)

Design Notes:

See the Data Center Bridging Specification.

Exceptions: Not Specified

Business Justification:

To improve data center bridging network traffic.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Next version of Windows

Comments: Not Specified

Device.Network.LAN.GRE

Description:

LAN requirements

Related Requirements:

Device.Network.LAN.GRE.GREPacketTaskOffloads

Device.Network.LAN.GRE.GREPacketTaskOffloads

Target Feature: Device.Network.LAN.GRE

Title: Ethernet Devices that implement GRE Encapsulated Packet Task Offloads must comply with the specification

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64

Description:

This requirement only applies to Ethernet devices that implement GRE encapsulated packet task offloads. Ethernet devices that implement GRE encapsulated packet task offloads must comply with the specification.

Exceptions: Not Specified

Business Justification:

To enable VMQ, Checksum Offload, Large Send Offload version 2, and RSS support for GRE encapsulated packets

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Windows 8

Comments: Not Specified

Device.Network.LAN.IPsec

Description:

LAN requirements

Related Requirements:

Device.Network.LAN.IPsec.IPsec

Device.Network.LAN.IPsec.IPsec

Target Feature: Device.Network.LAN.IPsec

Title: Ethernet devices that implement IPsec task offload must support required modes and protocols

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 7 Client x64
Windows 7 Client x86
Windows Server 2008 x64
Windows Server 2008 x86
Windows 8 Server x64
Windows 8 Client ARM
Windows Server 2008 Release 2 x64
Windows Server 2003 x86
Windows Server 2003 x64

Description:

Ethernet devices that support IPsec task offload must support the following:

Version: IPsec Task offload v2

NDIS version: 6.1, 6.20, 6.30

Ethernet devices that support IPsec task offload for Windows 8 must use NDIS 6.30.

Mode:

- IPv4 and IPv6 Transport

- IPv4 and IPv6 Tunnel

Protocol: ESP

Crypto Algorithm:

- Must implement: AES-GCM (128 or higher)

- May implement additional algorithms as stated in MSDN

Coexistence: Ethernet devices that support IPsec task offload and any the following offload technologies:

- Large Send Offload

- Receive Side Scaling

- Checksum offload.

Must implement them in a coexisting manner, such that the use of IPsec task offload does not preclude the use of the other offload technologies implemented for each IPsec mode.

Exceptions:

Business Justification:

To enable security features in Server and Domain Isolation scenarios as well as Datacenter and Cloud Computing scenarios.

Scenarios:

Server and Domain Isolation, Datacenter and Cloud Computing.

Success Metric: Not Specified

Enforcement Date: 02/27/2008

Comments:

NETWORK-0228

Device.Network.LAN.KRDMA

Description:

LAN requirements

Related Requirements:

Device.Network.LAN.KRDMA.KRDMA

Device.Network.LAN.KRDMA.KRDMA

Target Feature: Device.Network.LAN.KRDMA

Title: Devices that implement the NetworkDirect Kernel Mode Interface (NDKPI) (a.k.a., Kernel-mode RDMA, kRDMA) must comply with the Network Direct Kernel Mode Interface (NDKPI) Specification.

Applicable OS Versions:

Windows 8 Server x64

Description:

This requirement only applies to Ethernet devices that implement the Network Direct Kernel Mode Interface (NDKPI Specification. Devices that implement NetworkDirect Kernel Mode Interface (NDKPI) (a.k.a., Kernel-mode RDMA) must comply with the Network Direct Kernel Mode Interface (NDKPI) Specification.

This interface applies to Windows Server 8. It is not supported in downlevel operating systems.

Design Notes:

See the Network Direct Kernel Mode Interface (NDKPI) Specification.

Exceptions: Not Specified

Business Justification:

To support Kernel-Model RDMA.

Scenarios:

The NDKPI interface provides a way for the SMB2-Direct feature to make use of the RDMA capabilities of a NIC. The interface works equally well no matter whether the NIC implements RoCE/RoCEE, iWarp, or RDMA over Infiniband; the NIC must support at least one of these standardized RDMA technologies.

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.LAN.LargeSendOffload

Description:

Network requirements

Related Requirements:

Device.Network.LAN.LargeSendOffload.LargeSendOffload

Device.Network.LAN.LargeSendOffload.LargeSendOffload

Target Feature: Device.Network.LAN.LargeSendOffload

Title: Ethernet devices must implement large send offloads

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows Server 2008 x64
Windows Server 2008 x86
Windows 7 Client x64
Windows 7 Client x86
Windows 8 Server x64
Windows Server 2003 x86
Windows Server 2003 x64
Windows Server 2008 Release 2 x64
Windows 8 Client ARM

Description:

Ethernet devices must support the following task offloads:

Large Send Offload version 2 for IPv4 and IPv6.

Design Notes:

See the Windows Driver Kit, "NDIS."

Exceptions: Not Specified

Business Justification:

Stateless offloads functionality improves performance of network connectivity by performing stateless tasks in hardware. Particularly in a Server environment where there is high network volume, stateless offloads decreases CPU utilization.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2006

Comments:

NETWORK-0046

Device.Network.LAN.NetworkDirect

Description:

Network Direct requirements

Related Requirements:

Device.Network.LAN.NetworkDirect.KeyMPIUsagePatterns
Device.Network.LAN.NetworkDirect.LowDataErrors
Device.Network.LAN.NetworkDirect.MeetMinRates
Device.Network.LAN.NetworkDirect.NDISInterface
Device.Network.LAN.NetworkDirect.NDSPISpec
Device.Network.LAN.NetworkDirect.RegisterMemoryRates
Device.Network.LAN.NetworkDirect.RemoteMemoryViaByteBoundaries
Device.Network.LAN.NetworkDirect.WIMDriverInjection

Device.Network.LAN.NetworkDirect.KeyMPIUsagePatterns

Target Feature: Device.Network.LAN.NetworkDirect

Title: NetworkDirect devices must properly services key MPI usage patterns

Applicable OS Versions:

Windows Server 2008 x64
Windows Server 2008 Release 2 x64
Windows 7 Client x64
Windows 7 Client x86
Windows Server 2008 x86

Description:

ND certification tests will include MPI tests for the following areas: Send/Receive, All Gather, All Reduce, Barrier, Collective Message, Gather, Reduce, Scatter, Scan, RMA Put, RMA Get, RMA actions with fence.

Design Notes:

Each separate test uses pass/fail criteria based on functionality (not performance).

Exceptions: Not Specified

Business Justification:

To support Network Direct scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2008

Comments:

NETWORK-0235

Device.Network.LAN.NetworkDirect.LowDataErrors

Target Feature: Device.Network.LAN.NetworkDirect

Title: NetworkDirect devices must exhibit fewer than specified maximum number of data errors during data integrity tests

Applicable OS Versions:

- Windows Server 2008 x64
- Windows 7 Client x86
- Windows Server 2008 x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64

Description:

Data integrity for set of 2-way communications (pingPong) is tested and reported for a range of packet sizes from 1 byte to 4 Mbytes.

Exceptions: Not Specified

Business Justification:

To support Network Direct scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2008

Comments:

NETWORK-0240

Device.Network.LAN.NetworkDirect.MeetMinRates

Target Feature: Device.Network.LAN.NetworkDirect

Title: NetworkDirect devices must meet the specified minimum for data transfer rate, latency, and connection creation rate

Applicable OS Versions:

Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows Server 2008 x86

Description:

Data throughput is tested and reported for a range of packet sizes from 1 byte to 4 Mbytes. Maximum measured throughput must meet/exceed required throughput in ALL the following conditions:
Verified for: Polling and blocking notification modes
Verified for: Send/Receive and RDMA transfers
Verified for: One-way (ping) and two-way (pingPong)

Data latency is tested and reported for a range of packet sizes from 1 byte to 4 Mbytes. Minimum measured latency must be less than or equal to the required latency in ALL the following conditions:
Verified for: Polling and blocking notification modes
Verified for: Send/Receive and RDMA transfers
Verified for: One-way (ping) and two-way (pingPong)

Connection establishment rates of both client and server components of ND will be verified in certification testing:
Client: Single-threaded client successfully connects to multi-threaded server at minimum connection rate of 1000 cps.
Server: Multi-threaded client successfully connects to single-threaded server at minimum connection rate of 1000 cps.

Design Notes:

Source code for this test is available in:
The Beta - HPC Server 2008 Software Development Kit (SDK) is available on MS Connect (https://connect.microsoft.com/) as part of the "Microsoft Windows HPC Server 2008 Beta" program.

Exceptions: Not Specified

Business Justification:

To support Network Direct scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2008

Comments:

NETWORK-0236

Device.Network.LAN.NetworkDirect.NDISInterface

Target Feature: Device.Network.LAN.NetworkDirect

Title: NetworkDirect devices must implement an NDIS interface

Applicable OS Versions:

- Windows Server 2008 x64
- Windows Server 2008 x86
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64

Description:

Device driver package must include "normal" Network Driver Interface Specification (NDIS) 6.0 compliant driver and a NetworkDirect provider. Installers for these components and any needed network management software components must also be provided.

Design Notes:

See the Windows Driver Kit, "NDIS."

Exceptions: Not Specified

Business Justification:

To support Network Direct scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2008

Comments:

NETWORK-0231

Device.Network.LAN.NetworkDirect.NDSPISpec

Target Feature: Device.Network.LAN.NetworkDirect

Title: NetworkDirect devices must adhere to the NetworkDirect SPI specification

Applicable OS Versions:

- Windows Server 2008 x64
- Windows Server 2008 x86
- Windows 7 Client x86
- Windows 7 Client x64

Windows Server 2008 Release 2 x64

Description:

NetworkDirect device adheres to the most recent version of the NetworkDirect Service Provider Interface (SPI) specification as delivered in the HPC Server 2008 Software Development Kit (SDK).

When encountering error conditions described in the ND_RESULTS section of the NetworkDirect Service Provider Interface (SPI), the ND Provider must return the expected status value.

Design Notes:

The ND SPI interfaces is currently included in:

The HPC Server 2008 Software Development Kit (SDK) will be available from the Microsoft Download center at: http://download.microsoft.com/

Exceptions: Not Specified

Business Justification:

To support Network Direct scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2008

Comments:

NETWORK-0229

Device.Network.LAN.NetworkDirect.RegisterMemoryRates

Target Feature: Device.Network.LAN.NetworkDirect

Title: NetworkDirect devices must register memory to be used for communication at a rate and scale that meets the specified minimum

Applicable OS Versions:

Windows Server 2008 x64
Windows 7 Client x86
Windows Server 2008 Release 2 x64
Windows 7 Client x64
Windows Server 2008 x86

Description:

The rate at which an ND Provider can register and de-register memory (complete a set of INDAdapter::RegisterMemory, INDAdapter::DeregisterMemory operations) must meet a specified minimum rate and scale.

Design Notes:

Source code for this test is available in the download center
Memory registrations are fundamental to data transfer operations with Network Direct adapters. Memory registrations map application buffers to a Network Direct adapter and provide the Network Direct adapter the ability to access application buffers directly without OS involvement.

Exceptions: Not Specified

Business Justification:

To support Network Direct scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2008

Comments:

NETWORK-0239

Device.Network.LAN.NetworkDirect.RemoteMemoryViaByteBoundaries

Target Feature: Device.Network.LAN.NetworkDirect

Title: NetworkDirect devices must access remote memory solely within prescribed byte-level boundaries

Applicable OS Versions:

- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows Server 2008 x86

Description:

Security for Remote Memory Access (RMA) should be provided with ND provider checks on memory registration and use.

Memory Registration: INDAdapter::RegisterMemory will succeed ONLY when the calling application/process has read/write access to the memory being registered; else ND_ACCESS_VIOLATION is returned.

Memory Window Creation: INDEndpoint::Bind must bind memory windows only to successfully registered memory AND a memory window must be bound to one endpoint at a time. (Windows may be reused after being invalidated for its previous end point).

Memory Window Usage: Communications with an endpoint can use only the memory window(s) [one or more] bound to that endpoint.

Design Notes:

To elaborate on proper memory window operation, an endpoint may not successfully process an inbound RDMA request on a memory window that is not bound to it. For example, given endpoints A and A in process A on system A, and enpoints B and B in process B on system B. And further that EP A is connected to EP B, and EP A is connected to EP B. If a memory window W is bound to EP A, and the MW descriptor sent to application B (over either EP A or A), application B can only use the MW descriptor to access memory exposed by the MW on EP B. Attempts to use it on EP B (and thus A) must fail.

The provider should use MmProbeAndLockPages (http://msdn2.microsoft.com/en-us/library/ms802929.aspx) which probes access rights for a given buffer (in virtual memory), makes those pages resident, and locks those pages into memory; or equivalent to ensure that only memory pages to which the application calling ND SPI has proper access are registered for ND use.

Exceptions: Not Specified

Business Justification:

To support Network Direct scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2008

Comments:

NETWORK-0241

Device.Network.LAN.NetworkDirect.WIMDriverInjection

Target Feature: Device.Network.LAN.NetworkDirect

Title: NetworkDirect devices must be installable with WIM driver injection

Applicable OS Versions:

- Windows Server 2008 x64
- Windows 7 Client x86
- Windows Server 2008 x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64

Description:

ND Device installation must be compatible with injection into a Windows Imaging Format (WIM) OS image to facilitate OEM usage of ND devices and usage in compute clusters where re-imaging of

compute nodes can be frequent. Device driver must have a properly formatted INF for its device class as a pre-requisite to WIM driver injection compatibility.

Design Notes:

Driver injection is performed with the "imageX" and "peimg /inf" commands which are available in the Automated Installation Kit (AIK) for Windows Vista SP1 and Windows Server 2008 here: http://www.microsoft.com/downloads/details.aspx?FamilyID=94bb6e34-d890-4932-81a5-5b50c657de08&DisplayLang=en

INF-based installation is described in the Windows Driver Kit (WDK) "Creating an INF File".

Exceptions: Not Specified

Business Justification:

To support Network Direct scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2008

Comments:

NETWORK-0230

Device.Network.LAN.PM

Description:

LAN requirements

Related Requirements:

Device.Network.LAN.PM.PowMgmtNDIS
Device.Network.LAN.PM.WakeOnLANPatterns
Device.Network.LAN.PM.WakePacket

Device.Network.LAN.PM.PowMgmtNDIS

Target Feature: Device.Network.LAN.PM

Title: Ethernet devices that implement network presence offloads must conform to the Power Management specification on the NDIS Program

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows Server 2008 x64

Windows Server 2008 x86
Windows 7 Client x64
Windows 7 Client x86
Windows 8 Server x64
Windows Server 2003 x86
Windows Server 2003 x64
Windows Server 2008 Release 2 x64
Windows 8 Client ARM

Description:

Support of this feature is Required. There are two variants referenced below. The designer is free to implement all or any subset.

Ethernet devices that implement ARP offload must implement it as defined in the Power Management specification on MSDN. Specifically, the offload must respond to an ARP Request (operation = 1) by responding with an ARP Reply (operation = 2) as defined in RFC 826.

Ethernet devices that implement IPv6 NS offload must implement it as defined in Power Management specification on the MSDN. Specifically, the offload must respond to an Neighbor Solicitation (operation = 135) by responding with an NS Advertisement (operation = 136) as defined in RFC 2461. Devices must support at least 2 NS offloads, each with up to 2 target IPv6 addresses.

Design Notes:

See the Power Management specification on MSDN.

See RFC 826 at http://go.microsoft.com/fwlink/?LinkId=108718

Exceptions:

Exceptions to this requirement include: PC Card, CardBus devices and for external USB network devices operating on bus power. Devices with transmission speeds in excess of 1 gigabitDevices with fiber optic medium

Business Justification:

To improve power management.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2009

Comments:

NETWORK-0212

Device.Network.LAN.PM.WakeOnLANPatterns

Target Feature: Device.Network.LAN.PM

Title: Ethernet devices must implement Wake on LAN patterns according to the specification

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x64
- Windows 7 Client x86
- Windows Server 2003 x86
- Windows Server 2003 x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client ARM
- Windows 8 Server x64

Description:

Implementation must comply with Network Device Class Power Management Reference Specification.

Ethernet devices and drivers are required support at least eight (8) bitmap patterns since Windows uses up to eight patterns.

Magic packet wake support is required and is not included in the 8 bitmap pattern requirement.

Design Notes:

Implementation details are in the Power Management specification on MSDN.

Exceptions:

Exceptions to this requirement include: PC Card, CardBus devices and for external USB network devices operating on bus power. Devices with transmission speeds in excess of 1 gigabit Devices with fiber optic medium. Note: If the device implement Wake on LAN – it must implement it correctly based on this requirement regardless whether the device is on the exception list or not.

Business Justification:

To improve power management.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2009

Comments:

NETWORK-0227

Device.Network.LAN.PM.WakePacket

Target Feature: Device.Network.LAN.PM

Title: Ethernet devices that implement Wake Packet Detection must comply with Network Device Class Power Management Reference Specification.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86
- Windows Server 2003 x86
- Windows Server 2003 x64
- Windows Server 2008 x86
- Windows Server 2008 x64

Description:

Ethernet devices that implement Wake Packet Detection must comply with Network Device Class Power Management Reference Specification.

Implementation must comply with Network Device Class Power Management Reference Specification discussed in the Windows WDK. The NIC is required to capture at least the first 128 bytes of the packet causing the network wake and generate a status indication to the operating system.

Design Notes:

See the WDK, Network Device Class Power Management Reference Specification.

Exceptions: Not Specified

Business Justification:

To improve power management.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2009

Comments:

NETWORK-8355

Device.Network.LAN.RSC

Description:

LAN requirements

Related Requirements:

Device.Network.LAN.RSC.RSC

Device.Network.LAN.RSC.RSC

Target Feature: Device.Network.LAN.RSC

Title: Ethernet devices that implement Receive Segment Coalescing (RSC) must comply with the RSC Specification.

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64
Windows 8 Client ARM

Description:

Ethernet devices that implement Receive Segment Coalescing (RSC) must comply with the RSC Specification and require both IPv4 and IPv6 support.

Design Notes:

NDIS version: 6.30

Exceptions: Not Specified

Business Justification:

To comply with NDIS 6.30-RSC requirements that improves CPU utilization.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2009

Comments: Not Specified

Device.Network.LAN.RSS

Description:

LAN requirements

Related Requirements:

Device.Network.LAN.RSS.RSS
Device.Network.LAN.RSS.SetHashFunctionTypeAndValue
Device.Network.LAN.RSS.SupportHashTypeNDISRSSAPSHASHTYPETCPIPV4
Device.Network.LAN.RSS.SupportIndirectionTablesSizes
Device.Network.LAN.RSS.SupportToeplitzHashFunction
Device.Network.LAN.RSS.SupportUpdatesToRSSInfo

Device.Network.LAN.RSS.RSS

Target Feature: Device.Network.LAN.RSS

Title: Ethernet devices that implement RSS

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64
Windows 8 Client ARM
Windows Server 2008 Release 2 x64

Description:

Ethernet devices that implement RSS must support:

Hash types:IPv4, TCP IPv4, IPv6, and TCP IPv6

Multiple processor groups

Number of queues (depending on link speed):

Less than 10 gigabit: 2

10 gigabit or greater: 4

Implement all RSS standardized keywords

Message Signaled Interrupts Extended (MSI-X) as defined in the PCI v3.0 specification. The device must have an MSI-X vector for each RSS hardware queue. For example if the device advertises support for 8 RSS hardware queues then it must have at least 8 MSI-X vectors in the hardware.

Design Notes:

See RSS Standardized Keywords Specification http://msdn.microsoft.com/en-us/library/windows/hardware/ff570864(v=vs.85).aspx

In addition the device must allocate as many MSI-X table entries as there are CPUs in the system. See the NDIS documentation section on MSI-X for more detailshttp://msdn.microsoft.com/en-us/library/bb961702.aspx

Exceptions: Not Specified

Business Justification:

To enable RSS functionality.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2009

Comments: Not Specified

Device.Network.LAN.RSS.SetHashFunctionTypeAndValue

Target Feature: Device.Network.LAN.RSS

Title: Ethernet devices that implement RSS must set the hash function, hash type, and hash value on each indicated packet

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64

Description:

This requirement only applies to Ethernet devices that implement RSS. If the Ethernet device does not implement RSS then this requirement does not apply.

If the network device supports RSS, all packets for which the RSS implementation was able to calculate the hash the RSS implementation must return the full 32-bit hash value, the hash function, and the hash type, for each received packet it indicates. For any packets it received without error for which it was not able to generate the hash function, it must clear the hash type. Macros NET_BUFFER_LIST_SET_HASH_TYPE,NET_BUFFER_LIST_SET_HASH_FUNCTION, and NET_BUFFER_LIST_SET_HASH_VALUE must be used to set the associated values.

Design Notes:

See the MSDN page for more information: http://msdn.microsoft.com/en-us/library/windows/hardware/ff570726(v=vs.85).aspx

See the Windows Driver Kit, "Indicating RSS Receive Data."

Exceptions: Not Specified

Business Justification:

To support correct functionality.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2007

Comments:

NETWORK-0120

Device.Network.LAN.RSS.SupportHashTypeNDISRSSAPSHASHTYPETCPIPV4

Target Feature: Device.Network.LAN.RSS

Title: Ethernet devices that implement RSS must support hash type NDIS_RSS_CAPS_HASH _TYPE_TCP_IPV4

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 8 Server x64

Description:

This requirement only applies to Ethernet devices that implement RSS. If the Ethernet device does not implement RSS then this requirement does not apply.

If the network device supports RSS, the RSS implementation must support at least hash type NDIS_RSS_CAPS_HASH _TYPE_TCP_IPV4, as advertised in OID_GEN_RECEIVE_SCALE_CAPABILITIES.

Exceptions: Not Specified

Business Justification:

To support Hash type.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2007

Comments:

NETWORK-0122

Device.Network.LAN.RSS.SupportIndirectionTablesSizes

Target Feature: Device.Network.LAN.RSS

Title: Ethernet devices that implement RSS must support specific Indirection Table sizes

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64

Description:

This requirement only applies to Ethernet devices that implement RSS. If the Ethernet device does not implement RSS then this requirement does not apply.

If the network device supports RSS, the RSS implementation must support entries for size 2, 4, 8, 16, 32, 64, and 128 Indirection Tables. The lookup into the Indirection Table to find the destination CPU must be accomplished by using only the least significant bits as specified by the last value set in the OID_GEN_RECEIVE_SCALE_PARAMETERS, NumberOfLsbs variable. An RSS implementation must support the host protocol stack setting NumberOfLsbs to any value between 1 and 7, inclusive.

Design Notes:

See the Windows Driver Kit, "OID_GEN_RECEIVE_SCALE_PARAMETERS."

Exceptions: Not Specified

Business Justification:

To support Indirection Table sizes.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2007

Comments:

NETWORK-0123

Device.Network.LAN.RSS.SupportToeplitzHashFunction

Target Feature: Device.Network.LAN.RSS

Title: Ethernet devices that implement RSS must support the Toeplitz hash function

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64

Description:

This requirement only applies to Ethernet devices that implement RSS. If the Ethernet device does not implement RSS then this requirement does not apply.

If the network device supports RSS, the RSS implementation must at least support the Toeplitz hash function for the types of packets for which it advertised as being able to generate the hash (as specified in OID_GEN_RECEIVE_SCALE_CAPABILITIES). This includes support for the HashSecretKey length of 40 bytes.

Design Notes:

See Windows Driver Kit, "RSS Hashing Functions." Also, refer to MSDN for more information http://msdn.microsoft.com/en-us/library/windows/hardware/ff570725(v=vs.85).aspx

Exceptions: Not Specified

Business Justification:

To support Toeplitz has function.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2007

Comments:

NETWORK-0121

Device.Network.LAN.RSS.SupportUpdatesToRSSInfo

Target Feature: Device.Network.LAN.RSS

Title: Ethernet devices that implement RSS must support updates to RSS information at any time

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 x86
Windows Server 2008 x64
Windows 8 Server x64
Windows Server 2008 Release 2 x64

Description:

This requirement only applies to Ethernet devices that implement RSS. If the Ethernet device does not implement RSS then this requirement does not apply.

At any time a network device that supports RSS, must support setting OID_GEN_RECEIVE_SCALE_PARAMETERS, including updating the Indirection Table, NumberOfLsbs, SecretKey, and HashInformation (hash function and hash type). The RSS implementation can post packets out of order during the transition from the prior state to the new state and can perform a hardware reset if the HashInformation, SecretKey, or NumberOfLsbs changed. It must not perform a hardware reset if only the Indirection Table contents are changed.

Design Notes:

See the Windows Driver Kit, OID_GEN_RECEIVE_SCALE_PARAMETERS.

Exceptions: Not Specified

Business Justification:

To ensure proper functionality.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2006

Comments:

NETWORK-0124

Device.Network.LAN.SRIOV

Description:

Network requirements

Related Requirements:

Device.Network.LAN.SRIOV.SRIOV

Device.Network.LAN.SRIOV.SRIOV

Target Feature: Device.Network.LAN.SRIOV

Title: Ethernet devices that implement Single Root I/O Virtualization (SR-IOV) must support required functionalities

Applicable OS Versions:

Windows 8 Server x64

Description:

Requirements for an IOV NIC

If 10GbE+ hardware requirements

Minimum of 64 functions (1 PF + 63 VFs)

Minimum of 128 queues

Minimum of 128 MAC filters and optional VLAN ID

If 1GbE or less hardware requirements

Minimum of 8 functions (1 PF + 7 VFs)

Minimum of 16 queues

Minimum of 16 MAC filters and optional VLAN ID

Driver must use MS software back-channel to PF for VF driver PCIe configuration space requests.

The default initial switch configuration must be in the INF file.

The PF INF must set the UpperRangeto ndis5 and LowerRange to ethernet

The INF must not include the *SriovPreferred keyword

Coalesced filters must be set in NDIS_REECIVED_FILTER_CAPABILITIES

SR-IOV NIC must include a Virtual Ethernet Bridge (VEB)

If RSS is supported for VF miniports, the NIC must be able to support the same number of RSS as it can of VFs

Both the PF and VF miniport drivers must be able to pass the LAN certification tests

The default vPort cannot be deleted; non-default vPorts on a VF can be deleted

If SRIOV is disabled, the NIC and miniport must function as a standard Ethernet NIC

An SRIOV NIC must also advertise and implement VMQ. Queue pair not allocated to IOV are to be available for VMQ

On report of media connected, he VF miniport must be ready to accept traffic

A VF miniport must specify an UpperRange of ndisvf and a LowerRange of iovvf

Design Notes:

See the Single Root I/O Virtualization Specification.

Exceptions: Not Specified

Business Justification:

To support virtualization scenarios.

Scenarios:

Virtualization

Success Metric: Not Specified

Enforcement Date: Windows 8 RC

Comments: Not Specified

Device.Network.LAN.TCPChimney

Description:

TCP Chimney requirements

Related Requirements:

Device.Network.LAN.TCPChimney.ComplyWithNDIS
Device.Network.LAN.TCPChimney.ComplyWithTCPIPProtocol
Device.Network.LAN.TCPChimney.HandlesOutOfOrderData
Device.Network.LAN.TCPChimney.ImplementSufficientlyGranularTimers
Device.Network.LAN.TCPChimney.NeighborStateObjTimestampsComplyWithWDK
Device.Network.LAN.TCPChimney.Support1024Connections
Device.Network.LAN.TCPChimney.Support64bitAddresses

Device.Network.LAN.TCPChimney.ComplyWithNDIS

Target Feature: Device.Network.LAN.TCPChimney

Title: Ethernet devices that implement TCP Chimney must comply with the latest NDIS miniport driver model

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows Server 2008 x64
Windows Server 2008 x86
Windows 7 Client x86
Windows 7 Client x64
Windows 8 Server x64

Description:

This requirement only applies to Ethernet devices that implement TCP Chimney. If the Ethernet device does not implement TCP Chimney then this requirement does not apply.

The TCP Chimney portion of the device must comply with the TCP Chimney specification on Connect.

Ethernet devices that implement TCP Chimney must comply with NDIS 6.0 for Vista.
Ethernet devices that implement TCP Chimney must comply with NDIS 6.1 for WS2008.
Ethernet devices that implement TCP Chimney must comply with NDIS 6.2 for Win7.

Design Notes:

See Windows Driver Kit, "Network Devices and Protocols."

Exceptions: Not Specified

Business Justification:

To support capability.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2007

Comments:

NETWORK-0092

Device.Network.LAN.TCPChimney.ComplyWithTCPIPProtocol

Target Feature: Device.Network.LAN.TCPChimney

Title: Ethernet devices that implement TCP Chimney must comply with the IETF standard RFCs for the TCP/IP protocol family and behaves as the Microsoft Windows (host) TCP/IP protocol implementation

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows Server 2008 x64

Windows Server 2008 x86
Windows 7 Client x86
Windows 7 Client x64
Windows 8 Server x64

Description:

This requirement only applies to Ethernet devices that implement TCP Chimney. If the Ethernet device does not implement TCP Chimney then this requirement does not apply.

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT","SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this requirement are to be interpreted as described RFC 2119.

A TCP chimney NIC MUST implement the TCP/IP protocol family such that:

1. The TCP/IP protocol implementation conforms to the IETF standard RFCs and

2. The TCP/IP protocol implementation behaves in the same way as the Microsoft Windows TCP/IP protocol implementation

This requirement specifies which RFCs must be implemented by the TCP chimney NIC and clarifies the expected behavior in cases where an RFC is ambiguous.

Table 1 lists the RFCs that a TCP Chimney NIC must implement.

Descriptive name	Specification
Transmission Control Protocol RFCs	RFC 793 Transmission Control Protocol RFC 813 Window and Acknowledgment Strategy in TCP RFC 1122* Requirements for Internet Hosts Communication Layers - entire section 4.2 RFC 1323 TCP Extensions for High Performance RFC 2923* TCP Problems with Path MTU Discovery RFC 2988* Computing TCPs RTO RFC 3465* - Appropriate Byte Counting
TCP congestion control	RFC 2581* TCP Congestion Control RFC 2582* - NewReno Modification to TCP's Fast Recovery Alg. RFC 3042 Limited Transmit
TCP loss recovery	RFC 2018* TCP Selective Acknowledgment Options RFC 3517* Conservative Selective Acknowledgment (SACK)-based Loss Recovery Algorithm for TCP
TCP security	RFC tcpm-tcpsecure-09* - Improving TCP's Robustness to Blind In-Window Attacks
Internet Protocol v4 RFCs**	RFC 791 RFC 894 RFC 1042 RFC 1191 RFC 1122 entire section 3.2
Internet Protocol v6 RFCs**	RFC 1752 RFC 1981 RFC 2374

	RFC 2460 RFC 2461 RFC 2675 RFC 2711 RFC 3122 RFC 3513
*** - There are associated clarifications for the RFC which must be followed. They are outlined below.** **** - TCP Chimney NICs MUST NOT implement the entire set of IP related RFCs. Instead the TCP Chimney Driver Kit guidelines for the Internet Protocol RFC implementation must be followed.**	

Table 1 - Lists of RFCs that a TCP Chimney NIC must implement

RFC Clarifications

The following clarifications must be followed by the TCP/IP implementation in the TCP Chimney NIC.

RFC 1122

1. Section 4.2.3.4 specifies that the Nagle algorithm SHOULD be implemented as a method to avoid the Silly Window Syndrome. The TCP chimney NIC MUST implement the Nagle algorithm and the implementation must follow this pseudo code:

a. When sending a segment the first stage of SWS avoidance MUST be implemented as:

```
Send()

{

..

If (BytesToSend > MSS ||

BytesToSend > MaxSndWnd /2 ||

BytesToSend >= BytesInCurReq ||

ForceOutput)

{

BeginSend();

}else

{

StartSwsTimer();

}

...

...
```

BytesToSend Number of available Bytes that can be sent as allowed by the current send window

MSS Maximum Segment Size

MaxSndWnd Maximum receive window that the TCP peer ever advertised

BytesInCurReq Bytes left in the current send request

ForceOutput Variable that determines if the segment MUST be sent, due to SWS timer expiring as an example.

The line in red specifies the deviation from the SWS avoidance that MUST be implemented.

Note: This pseudo code defines the behavior at the time of sending, not at the time when the send request is offloaded by the host TCP/IP stack.

b. The reason why the MS TCPIP stack deviates from the SWS algorithm in the way described above is:

1. CWND can grow in Bytes. More precisely, CWND is not constrained to grow or shrink in multiples of MSS or PUSH boundaries. Because the TCP implementation in Windows implements Appropriate Byte Counting (RFC 3465) this point is strengthened even further.

2. The PUSH boundary is determined by the TCP application posting data to be sent so it is not guaranteed to be aligned with the MSS size.

3. Because of #1 and #2 it is very likely for the TCP state machine to reach a point at which one MSS has been placed on the wire and there is a sub-MSS segment which, if sent, will complete the block of data up to the PUSH boundary.

a. In this case it is favorable for TCP to send this one sub-MSS segment in order to complete the transmission of the apps buffer up to the PUSH boundary. The reason why it is favorable to do this is because the data will be delivered to the receiving application faster than if the SWS algorithm was followed to the letter. At the same time the deviation does not re-introduce any of the problems SWS addressed in the first place.

2. As described in section 4.2.2.17 a TCP Chimney NIC MUST use the connection RTT as a trigger to send a zero window probe and then exponentially increase the interval between successive probes. In addition, the probe MUST contain 1 new Byte of data.

3. TCP Chimney NIC MUST support filling at least two reassembly holes.

RFC 2018

1. The TCP Chimney NIC MAY implement RFC 2018. If a TCP Chimney NIC implements RFC 2018 then it MUST also implement RFC 3517.

2. A TCP Chimney NIC that DOES NOT implement RFC 2018 MUST properly process pure ACK packets, which contain SACK blocks, as described in section 3 of RFC 793.

RFC 2581

1. The TCP Chimney NIC MUST be able to transition from using the slow-start algorithm to using the congestion avoidance algorithm as specified in Section 3.1. In addition it MUST implement Congestion Window (cwnd) = Slow Start Threshold (ssthresh) instead of Congestion Window > Slow Start Threshold.

RFC 2582

1. The TCP Chimney NIC MUST use the following equation instead of the one described in RFC 2582, section 3 point 1:

SsThresh = max(2*mss, min(cwnd,window_advertised_by_peer)/2)

RFC 2923

1. The TCP Chimney NIC is NOT required to implement the recommendations outlined in RFC 2923. Instead, the TCP Chimney NIC must upload the TCP connection to allow the host stack to execute the black hole detection state machine. See the Windows Driver Kit for details.

RFC 2988

1. See RFC 1122 section 4.2.3.1 and RFC 2988 for background information. TCP Chimney NIC MUST implement RTO calculation using the following algorithm, which is the same as RFC 2988 with minor exceptions that are qualified below:

function CalculateRto (first, byRef srtt, byRef rttvar, m)

rttSample = Minimum (m, 30s)

if first then

rttvar = m/2

srtt = m

else

' notice that rttvar is calculated first, using the old

' value of srtt

rttvar = (3/4) * rttvar + (1/4) * abs(srtt - rttSample)

srtt = (7/8)*srtt + (1/8) * rttSample

end if

CalculateRto = srtt + 4 * rttvar

CalculateRto = Minimum (CalculateRto, 60s)

CalculateRto = Maximum (CalculateRto, 300ms)

The two lines in red capture the deviation from the RFC. Specifically, it is expected that the TCP Chimney NIC has an upper bound when calculating the RTT value.

RFC 3465

1. Section 2.1 describes the changes to CWND during congestion avoidance. A TCP Chimney NIC MUST use the following formula to calculate CWND during congestion avoidance:

// L is 4CWnd += max((MaxMss * min(MaxMss * L, BytesAcked)) /CWnd, 1)

Note that if BytesAcked is always 1 the above equation becomes max((MaxMss, MaxMss)/Cwnd, 1)which is equivalent to equation 2 in RFC 2581.

2. Section 2.3 in RFC 3465 discusses the limit, L, chosen for the CWND increase during slow start and congestion avoidance, which controls the aggressiveness of the algorithm. A TCP Chimney NIC MUST use a value of 4 for L in order for it to exhibit the same behavior as the Windows TCP/IP protocol implementation.

RFC tcpm-tcpsecure-09

1. TCP Chimney NICs MUST follow the security guidelines outlined in sections 3, 4, and 5 of the TCP Security internet draft RFC (http://tools.ietf.org/html/draft-ietf-tcpm-tcpsecure-12).

2. TCP Chimney NICSs SHOULD follow the Windows specific implementation details described in the WDK.

Design Notes:

See the full text of the RFCs at http://go.microsoft.com/fwlink/?LinkId=36702.

Exceptions: Not Specified

Business Justification:

To comply with TCP/IP standards.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2007

Comments:

NETWORK-0113

Device.Network.LAN.TCPChimney.HandlesOutOfOrderData

Target Feature: Device.Network.LAN.TCPChimney

Title: Ethernet devices that implement TCP Chimney must properly handle the Out Of Order data scenarios

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64

Description:

Ethernet devices that implement TCP Chimney must properly handle Out Of Order data scenarios describe below:

1. If anything is placed in the reassembly queue after an inorder FIN then the reassembly queue MUST be flushed by discarding all of its contents.

2. If a TCP Chimney NIC stores an OOO FIN in the reassembly queue, then it MUST not store OOO data or OOO FIN beyond another OOO FIN in the reassembly queue. If it receives OOO data or OOO FIN segment that would lead to such a conflict, then the TCP Chimney NIC MUST drop that segment and flush the reassembly queue by discarding all of its contents.

Exceptions: Not Specified

Business Justification:

To support handling out of order data scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2010

Comments:

NETWORK-0294

Device.Network.LAN.TCPChimney.ImplementSufficientlyGranularTimers

Target Feature: Device.Network.LAN.TCPChimney

Title: Ethernet devices that implement TCP Chimney must implement sufficiently granular timers

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64

Windows 8 Server x64
Windows Server 2008 Release 2 x64
Windows Server 2008 x86
Windows Server 2008 x64

Description:

This requirement only applies to Ethernet devices that implement TCP Chimney. If the Ethernet device does not implement TCP Chimney then this requirement does not apply.

The TCP chimney NIC must have access to timers (implemented on the NICs hardware) with precise enough granularity and skew such that it can drive the TCP/IP state machine correctly. The timer granularity must be 10 milliseconds or better (lower than 10 ms) and the timer skew must be as good as what general purpose CPU timer provides

Exceptions: Not Specified

Business Justification:

To ensure proper functionality.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2009

Comments:

NETWORK-0274

Device.Network.LAN.TCPChimney.NeighborStateObjTimestampsComplyWithWDK

Target Feature: Device.Network.LAN.TCPChimney

Title: Neighbor state object timestamps are implemented according to details in the Windows Driver Kit

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows Server 2008 x64
Windows Server 2008 x86
Windows 7 Client x86
Windows 7 Client x64
Windows 8 Server x64
Windows Server 2008 Release 2 x64

Description:

A network device that implements TCP Chimney must ensure that TCP Chimney maintains a timestamp for each neighbor state object and perform checks against the timestamp on each incoming and outgoing packet.

Design Notes:

See the Windows Driver Kit, "OID_TCP_OFFLOAD.

Exceptions: Not Specified

Business Justification:

To comply with standards.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2007

Comments:

NETWORK-0096

Device.Network.LAN.TCPChimney.Support1024Connections

Target Feature: Device.Network.LAN.TCPChimney

Title: Ethernet devices that implement TCP Chimney must support at least 1024 connections and not advertise more offload capacity than what the HW can support

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64

Description:

This requirement only applies to Ethernet devices that implement TCP Chimney. If the Ethernet device does not implement TCP Chimney then this requirement does not apply.

Ethernet devices that implement TCP Chimney must support at least 1024 connections and not advertise more offload capacity than what the HW can support.

Exceptions: Not Specified

Business Justification:

To support at least 1024 connections.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2009

Comments:

NETWORK-0273

Device.Network.LAN.TCPChimney.Support64bitAddresses

Target Feature: Device.Network.LAN.TCPChimney

Title: Ethernet devices that implement TCP Chimney must support 64-bit addresses

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows Server 2008 x64
- Windows Server 2008 x86
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Server x64
- Windows Server 2008 Release 2 x64

Description:

This requirement only applies to Ethernet devices that implement TCP Chimney. If the Ethernet device does not implement TCP Chimney then this requirement does not apply.

If the device uses PCI, it must support 64-bit addresses; 64-bit data support is not required

Exceptions: Not Specified

Business Justification:

To support 64-bit addresses.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2007

Comments:

NETWORK-0118

Device.Network.LAN.VMQ

Description:

LAN requirements

Related Requirements:

Device.Network.LAN.VMQ.VirtualMachineQueues

Device.Network.LAN.VMQ.VirtualMachineQueues

Target Feature: Device.Network.LAN.VMQ

Title: Ethernet devices that implement Virtual Machine Queues comply with specification

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 7 Client x64
Windows 7 Client x86
Windows 8 Server x64
Windows Server 2008 Release 2 x64

Description:

1) Implementation must comply with Programmable Machine Queues Reference Specification.

2) At least four queues with filters must be supported. Support for at least 16 queues with filters is recommended. The number of queues required will be 16 by December 1, 2009 for 10 Gigabit parts. The number of queues required is inclusive of the default queue.

3) MSI-X Support (NDIS_RECEIVE_FILTER_MSI_X_SUPPORTED) is mandatory:

Queues	MSI-X to Queues Ratio
1 to 16	1:1
17-64	1:2 (Min 16)
65-unlimited	1:16 (Min 32)

4) Filtering:

a. Support for VLAN filtering in HW (NDIS_RECEIVE_FILTER_MAC_HEADER_VLAN_ID_SUPPORTED) is optional. If implemented, VLANs per VM Queue (NumVlansPerVMQueue) must be >= 1

b. Support for MAC filtering in HW (NDIS_RECEIVE_FILTER_MAC_HEADER_DEST_ADDR_SUPPORTED) is mandatory.

5) Support for NDIS_RECEIVE_FILTER_TEST_HEADER_EQUAL_SUPPORTED is mandatory

6) The maximum number of MAC header filters(MaxMacHeaderFilters) must be >= Number of queues

7) Total MAC addresses (NumTotalMacAddresses)must be >= Number of queues

8) MAC addresses per VM Queue(NumMacAddressesPerVMQueue) must be >= 1

9) Per-queue receive indication must be supported (NDIS_RECEIVE_QUEUE_PARAMETERS_PER_QUEUE_RECEIVE_INDICATION)

10) Look-ahead split NDIS_RECEIVE_FILTER_LOOKAHEAD_SPILT_SUPPORTED) support is optional. If implemented, implementations MUST split the incoming packet in hardware and **DMA** the lookahead portion of the packet to the lookahead buffer and post-lookahead portion of the packet to the post-lookahead buffer. It is acceptable for the device to DMA the lookahead portion of the packet to the backfill portion of the post-lookahead buffer and also DMA it to the lookahead buffer.

a. If present, must support MinLookAheadSplitSize <= 128 bytes, MaxLookAheadSplitSize >= 14

b. Post December 1, 2009 support is mandatory for new hardware.

11) Dynamic VMQ support is required for Windows 8 Server only.

Design Notes:

Implementation details are in the ProgrammableMachine Queues specification, on the NDIS Program, Connect site https://connect.microsoft.com/Downloads/DownloadDetails.aspx?SiteID=238&DownloadID=18742.

Exceptions: Not Specified

Business Justification:

To enable Virtual Machine Queues.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2009

Comments:

NETWORK-0226

Device.Network.MobileBroadband.CDMA

Description:

Mobile Broadband

Related Requirements:

Device.Network.MobileBroadband.CDMA.ComplyWithBaseReq
Device.Network.MobileBroadband.CDMA.FWComplyWithMBSpec
Device.Network.MobileBroadband.CDMA.ImplementSMS
Device.Network.MobileBroadband.CDMA.MultiCarrierFunctionality
Device.Network.MobileBroadband.CDMA.SupportUSBSelectiveSuspend
Device.Network.MobileBroadband.CDMA.SupportWakeOnMB

Device.Network.MobileBroadband.CDMA.ComplyWithBaseReq

Target Feature: Device.Network.MobileBroadband.CDMA

Title: Mobile Broadband devices must comply with the following base requirements

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client ARM

Description:

Mobile Broadband devices must comply with the following base requirements:

MUST conform to NDIS 6.30 and Microsoft Mobile Broadband Driver Model Specification requirements in the Windows Driver Kit.

MUST comply with power management specifications.

MUST meet the performance target for various operation specified for various class of devices in the Mobile Broadband Driver Model Specification

Mobile Broadband device driver must implement and conform to the NDIS 6.30 and Microsoft Mobile Broadband Driver Model Specifications. All recommended implementation specified in the Mobile Broadband Driver Model Specifications needs to be implemented. Note that Microsofts MB class driver is compliant to above requirements.

Mobile Broadband Device must support the Power Management Policy as outline in the Network Device Class Power Management Reference Specification, Version 2.0. Device must be functional after various OS Power Management operations.

Device must meet the performance targets describe in the Mobile Broadband Driver Model Specification.

Design Notes:

Helpful links:

Mobile Broadband Driver Model Specifications

http://msdn.microsoft.com/en-us/library/ff560543(v=VS.85).aspx

Network Device Class Power Management Reference Specification

http://download.microsoft.com/download/1/6/1/161ba512-40e2-4cc9-843a-923143f3456c/netpmspc.rtf

Exceptions: Not Specified

Business Justification:

To ensure support Mobile Broadband scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2010

Comments:

Base Requirements

Device.Network.MobileBroadband.CDMA.FWComplyWithMBSpec

Target Feature: Device.Network.MobileBroadband.CDMA

Title: USB interface based CDMA class of Mobile Broadband device firmware must comply with Microsoft's Mobile Broadband Interface Model Specification.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

USB interface based CDMA class of Mobile Broadband device firmware implementation must comply with the Microsoft's Mobile Broadband Interface Model Specification.

No additional IHV drivers are needed for the functionality of the device and the device must work with Microsoft's generic class driver implementation. Note that Microsoft generic class driver does not support WiMax and nonUSB interface devices. For those devices, IHV driver is required.

Exceptions:

- Device models that are announced as End of life (EOL) as of December, 2011.- Device models that are no longer in production line.

Business Justification:

To improve user experience.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.MobileBroadband.CDMA.ImplementSMS

Target Feature: Device.Network.MobileBroadband.CDMA

Title: CMDA class of Mobile Broadband devices must implement all SMS functionality as defined in the Microsoft Mobile Broadband Driver Model Specification.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

Driver must support the all the SMS OIDs defined in the Microsoft Mobile Broadband Driver Model Specification.

Design Notes:

Here is the link to the Mobile Broadband Driver Model Specification

http://msdn.microsoft.com/en-us/library/ff560543(v=VS.85).aspx

Exceptions: Not Specified

Business Justification:

To support SMS functionality.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.MobileBroadband.CDMA.MultiCarrierFunctionality

Target Feature: Device.Network.MobileBroadband.CDMA

Title: Mobile Broadband devices that support multi-carrier feature must support the multi-carrier functionality.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

Mobile Broadband devices that support multi-carrier feature must support the multi-carrier functionality and should also do the following:

Must meet the multi-carrier performance requirements available in the Mobile Broadband Driver Model Specification.

Must stay on the bus when changing home providers.

Must successfully pass all applicable certification tests covering all the different cellular class technologies that the device is capable of connecting to.

Mobile Broadband devices supporting multi-carrier feature must meet the multi-carrier performance requirements specified in mobile broadband driver model specification.

Mobile Broadband devices that support multi-carrier feature must not do a bus / device re-enumeration or power reset the device resulting in PnP re-enumeration to the Windows when changing the home providers.

If the device is capable of supporting GSM and CDMA cellular class technologies, then the device must execute both GSM as well as CDMA certification tests. For this to be covered correctly, the location of certification test execution must be in the coverage area of at least one GSM and one CDMA cellular class technologies.

Exceptions: Not Specified

Business Justification:

To support Mobile Broadband scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.MobileBroadband.CDMA.SupportUSBSelectiveSuspend

Target Feature: Device.Network.MobileBroadband.CDMA

Title: USB based Mobile Broadband devices must support Windows implementation of USB selective suspend.

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Client ARM

Description:

USB based Mobile Broadband devices must support Windows implementation of USB selective suspend (SS). No alternate USB SS implementation is allowed.

Exceptions: Not Specified

Business Justification:

To support Mobile Broadband scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.MobileBroadband.CDMA.SupportWakeOnMB

Target Feature: Device.Network.MobileBroadband.CDMA

Title: Mobile Broadband class of devices MUST support the following wake on mobile broadband capabilities.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

Mobile Broadband class of devices MUST support the following wake on mobile broadband capabilities.

- Devices MUST support 16 bitmap wake patterns of 128 byte each.
- Devices MUST wake the system on register state change.
- Devices MUST wake the system on media connect.
- Devices MUST wake the system on media disconnect.
- GSM and CDMA class of Devices MUST wake the system on receiving an incoming SMS message.
- Devices that support USSD MUST wake the system on receiving USSD message.
- Devices MUST support wake packet indication. NIC should cache the packet causing the wake on hardware and pass it up when the OS is ready for receives.

Mobile Broadband class of devices must support wake on mobile broadband. Device should wake the system on above mentioned events. Note that wake on USSD is mandatory only if the device

reports that it supports USSD, else it is optional. See the following MSDN documentation for more information on the SMS and register state wake events.

NDIS_STATUS_WWAN_REGISTER_STATE

NDIS_STATUS_WWAN_SMS_RECEIVE

Exceptions: Not Specified

Business Justification:

To support Mobile Broadband scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.MobileBroadband.GSM

Description:

Mobile Broadband

Related Requirements:

Device.Network.MobileBroadband.GSM.ComplyWithBaseReq
Device.Network.MobileBroadband.GSM.EAPSIM
Device.Network.MobileBroadband.GSM.FWComplyWithMBSpec
Device.Network.MobileBroadband.GSM.ImplementSMS
Device.Network.MobileBroadband.GSM.MultiCarrierFunctionality
Device.Network.MobileBroadband.GSM.SupportFastDormancy
Device.Network.MobileBroadband.GSM.SupportUSBSelectiveSuspend
Device.Network.MobileBroadband.GSM.SupportWakeOnMB
Device.Network.MobileBroadband.GSM.USSD

Device.Network.MobileBroadband.GSM.ComplyWithBaseReq

Target Feature: Device.Network.MobileBroadband.GSM

Title: Mobile Broadband devices must comply with the following base requirements

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 7 Client x86
Windows 7 Client x64

Windows 8 Client ARM

Description:

Mobile Broadband devices must comply with the following base requirements:

MUST conform to NDIS 6.30 and Microsoft Mobile Broadband Driver Model Specification requirements in the Windows Driver Kit.

MUST comply with power management specifications.

MUST meet the performance target for various operation specified for various class of devices in the Mobile Broadband Driver Model Specification

Mobile Broadband device driver must implement and conform to the NDIS 6.30 and Microsoft Mobile Broadband Driver Model Specifications. All recommended implementation specified in the Mobile Broadband Driver Model Specifications needs to be implemented. Note that Microsofts MB class driver is compliant to above requirements.

Mobile Broadband Device must support the Power Management Policy as outline in the Network Device Class Power Management Reference Specification, Version 2.0. Device must be functional after various OS Power Management operations.

Device must meet the performance targets describe in the Mobile Broadband Driver Model Specification.

Design Notes:

Helpful links:

Mobile Broadband Driver Model Specifications

http://msdn.microsoft.com/en-us/library/ff560543(v=VS.85).aspx

Network Device Class Power Management Reference Specification

http://download.microsoft.com/download/1/6/1/161ba512-40e2-4cc9-843a-923143f3456c/netpmspc.rtf

Exceptions: Not Specified

Business Justification:

To support Mobile Broadband scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2010

Comments:

Base Requirements

Device.Network.MobileBroadband.GSM.EAPSIM

Target Feature: Device.Network.MobileBroadband.GSM

Title: GSM class of Mobile Broadband devices that support extensible authentication protocol method for GSM Subscriber Identity Module (EAP-SIM) must support EAP-SIM defined in RFC 4186.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

GSM devices that support EAP-SIM must support EAP-SIM defined in RFC 4186.

Exceptions: Not Specified

Business Justification:

To support Mobile Broadband scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.MobileBroadband.GSM.FWComplyWithMBSpec

Target Feature: Device.Network.MobileBroadband.GSM

Title: USB interface based GSM class of Mobile Broadband device firmware must comply with Microsoft's Mobile Broadband Interface Model Specification.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

USB interface based GSM class of Mobile Broadband device firmware implementation must comply with the Microsoft's Mobile Broadband Interface Model Specification.

No additional IHV drivers are needed for the functionality of the device and the device must work with Microsoft's generic class driver implementation. Note that Microsoft generic class driver does not support WiMax and nonUSB interface devices. For those devices, IHV driver is required.

Exceptions:

- Device models that are announced as End of life (EOL) as of December, 2011.- Device models that are no longer in production line.

Business Justification:

To improve user experience.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.MobileBroadband.GSM.ImplementSMS

Target Feature: Device.Network.MobileBroadband.GSM

Title: GSM class of Mobile Broadband devices must implement all SMS functionality as defined in the Microsoft Mobile Broadband Driver Model Specification.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

Driver must support the all the SMS OIDs defined in the Microsoft Mobile Broadband Driver Model Specification.

Design Notes:

Here is the link to the Mobile Broadband Driver Model Specification

http://msdn.microsoft.com/en-us/library/ff560543(v=VS.85).aspx

Exceptions: Not Specified

Business Justification:

To support SMS functionality.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.MobileBroadband.GSM.MultiCarrierFunctionality

Target Feature: Device.Network.MobileBroadband.GSM

Title: Mobile Broadband devices that support multi-carrier feature must support the multi-carrier functionality.

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Client ARM

Description:

Mobile Broadband devices that support multi-carrier feature must support the multi-carrier functionality and should also do the following:

Must meet the multi-carrier performance requirements available in the Mobile Broadband Driver Model Specification.

Must stay on the bus when changing home providers.

Must successfully pass all applicable certification tests covering all the different cellular class technologies that the device is capable of connecting to.

Mobile Broadband devices supporting multi-carrier feature must meet the multi-carrier performance requirements specified in mobile broadband driver model specification.

Mobile Broadband devices that support multi-carrier feature must not do a bus / device re-enumeration or power reset the device resulting in PnP re-enumeration to the Windows when changing the home providers.

If the device is capable of supporting GSM and CDMA cellular class technologies, then the device must execute both GSM as well as CDMA certification tests. For this to be covered correctly, the location of certification test execution must be in the coverage area of at least one GSM and one CDMA cellular class technologies.

Exceptions: Not Specified

Business Justification:

To support Mobile Broadband scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.MobileBroadband.GSM.SupportFastDormancy

Target Feature: Device.Network.MobileBroadband.GSM

Title: GSM class of Mobile Broadband devices MUST support Fast Dormancy mechanism defined by 3GPP in release 8.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

Mobile Broadband devices must implement fast dormancy mechanism defined by 3GPP in revision 8. Fast Dormancy is a battery life savings mechanism for UE (User Equipment) devices that allows the devices to request the network to put them in a low power channel. UE sends a SIGNALLING CONNECTION RELEASE INDICATION (SCRI) message (sent by the UE to the network) with the IE "Signaling Connection Release Indication Cause" present and set to "UE Requested PS Data session end".

Exceptions: Not Specified

Business Justification:

To support Mobile Broadband scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.MobileBroadband.GSM.SupportUSBSelectiveSuspend

Target Feature: Device.Network.MobileBroadband.GSM

Title: USB based Mobile Broadband devices must support Windows implementation of USB selective suspend.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

USB based Mobile Broadband devices must support Windows implementation of USB selective suspend (SS). No alternate USB SS implementation is allowed.

Exceptions: Not Specified

Business Justification:

To support USB Selective Suspend scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.MobileBroadband.GSM.SupportWakeOnMB

Target Feature: Device.Network.MobileBroadband.GSM

Title: Mobile Broadband class of devices MUST support the following wake on mobile broadband capabilities.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

Mobile Broadband class of devices MUST support the following wake on mobile broadband capabilities.

- Devices MUST support 16 bitmap wake patterns of 128 byte each.
- Devices MUST wake the system on register state change.
- Devices MUST wake the system on media connect.
- Devices MUST wake the system on media disconnect.
- GSM and CDMA class of Devices MUST wake the system on receiving an incoming SMS message.
- Devices that support USSD MUST wake the system on receiving USSD message.
- Devices MUST support wake packet indication. NIC should cache the packet causing the wake on hardware and pass it up when the OS is ready for receives.

Mobile Broadband class of devices must support wake on mobile broadband. Device should wake the system on above mentioned events. Note that wake on USSD is mandatory only if the device reports that it supports USSD, else it is optional. See the following MSDN documentation for more information on the SMS and register state wake events.

NDIS_STATUS_WWAN_REGISTER_STATE

NDIS_STATUS_WWAN_SMS_RECEIVE

Exceptions: Not Specified

Business Justification:

To support Mobile Broadband scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.MobileBroadband.GSM.USSD

Target Feature: Device.Network.MobileBroadband.GSM

Title: GSM class of Mobile Broadband devices that implement Unstructured Supplementary Service Data (USSD) must support USSD based on Mobile Broadband Driver Model.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

Windows Mobile Broadband Driver Model is updated to include the full support of sending and receiving USSD messages. Devices that implement USSD must support USSD based on Mobile Broadband Driver Model.

Exceptions: Not Specified

Business Justification:

To support Mobile Broadband scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.MobileBroadband.WiMax

Description:

Mobile Broadband

Related Requirements:

Device.Network.MobileBroadband.WiMax.ComplyWithBaseReq
Device.Network.MobileBroadband.WiMax.ImplementIPPacket
Device.Network.MobileBroadband.WiMax.MultiCarrierFunctionality
Device.Network.MobileBroadband.WiMax.SupportUSBSelectiveSuspend
Device.Network.MobileBroadband.WiMax.SupportWakeOnMB

Device.Network.MobileBroadband.WiMax.ComplyWithBaseReq

Target Feature: Device.Network.MobileBroadband.WiMax

Title: Mobile Broadband devices must comply with the following base requirements

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Client ARM

Description:

Mobile Broadband devices must comply with the following base requirements:

MUST conform to NDIS 6.30 and Microsoft Mobile Broadband Driver Model Specification requirements in the Windows Driver Kit.

MUST comply with power management specifications.

MUST meet the performance target for various operation specified for various class of devices in the Mobile Broadband Driver Model Specification

Mobile Broadband device driver must implement and conform to the NDIS 6.30 and Microsoft Mobile Broadband Driver Model Specifications. All recommended implementation specified in the Mobile Broadband Driver Model Specifications needs to be implemented. Note that Microsofts MB class driver is compliant to above requirements.

Mobile Broadband Device must support the Power Management Policy as outline in the Network Device Class Power Management Reference Specification, Version 2.0. Device must be functional after various OS Power Management operations.

Device must meet the performance targets describe in the Mobile Broadband Driver Model Specification.

Design Notes:

Helpful links:

Mobile Broadband Driver Model Specifications

http://msdn.microsoft.com/en-us/library/ff560543(v=VS.85).aspx

Network Device Class Power Management Reference Specification

http://download.microsoft.com/download/1/6/1/161ba512-40e2-4cc9-843a-923143f3456c/netpmspc.rtf

Exceptions: Not Specified

Business Justification:

To support Mobile Broadband scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2010

Comments:

Base Requirements

Device.Network.MobileBroadband.WiMax.ImplementIPPacket

Target Feature: Device.Network.MobileBroadband.WiMax

Title: WiMAX class of Mobile Broadband devices must implement the IP packet handling capability (Raw IP) and support address acquisition through Windows based DHCP as defined for WiMAX in the Mobile Broadband Specification for Windows

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

This requirement is applicable to WiMAX category of devices only.

WiMAX devices must confirm to NDIS 6.30 and Microsoft Mobile Broadband Driver Model Specification requirements for Windows.

All recommended implementation for WiMAX as specified in the Mobile Broadband Driver Specification must be implemented.

Exceptions: Not Specified

Business Justification:

To support Mobile Broadband scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.MobileBroadband.WiMax.MultiCarrierFunctionality

Target Feature: Device.Network.MobileBroadband.WiMax

Title: Mobile Broadband devices that support multi-carrier feature must support the multi-carrier functionality.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

Mobile Broadband devices that support multi-carrier feature must support the multi-carrier functionality and should also do the following:

- Must meet the multi-carrier performance requirements available in the Mobile Broadband Driver Model Specification.
- Must stay on the bus when changing home providers.
- Must successfully pass all applicable certification tests covering all the different cellular class technologies that the device is capable of connecting to.

Mobile Broadband devices supporting multi-carrier feature must meet the multi-carrier performance requirements specified in mobile broadband driver model specification.

Mobile Broadband devices that support multi-carrier feature must not do a bus / device re-enumeration or power reset the device resulting in PnP re-enumeration to the Windows when changing the home providers.

If the device is capable of supporting GSM and CDMA cellular class technologies, then the device must execute both GSM as well as CDMA certification tests. For this to be covered correctly, the location of certification test execution must be in the coverage area of at least one GSM and one CDMA cellular class technologies.

Exceptions: Not Specified

Business Justification:

To support Mobile Broadband scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.MobileBroadband.WiMax.SupportUSBSelectiveSuspend

Target Feature: Device.Network.MobileBroadband.WiMax

Title: USB based Mobile Broadband devices must support Windows implementation of USB selective suspend.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

USB based Mobile Broadband devices must support Windows implementation of USB selective suspend. No alternate USB SS implementation is allowed.

Exceptions: Not Specified

Business Justification:

To support USB Selective Suspend scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.MobileBroadband.WiMax.SupportWakeOnMB

Target Feature: Device.Network.MobileBroadband.WiMax

Title: Mobile Broadband class of devices MUST support the following wake on mobile broadband capabilities.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client ARM

Description:

Mobile Broadband class of devices MUST support the following wake on mobile broadband capabilities.

Devices MUST support 16 bitmap wake patterns of 128 byte each.

Devices MUST wake the system on register state change.

Devices MUST wake the system on media connect.

Devices MUST wake the system on media disconnect.

GSM and CDMA class of Devices MUST wake the system on receiving an incoming SMS message.

Devices that support USSD MUST wake the system on receiving USSD message.

Devices MUST support wake packet indication. NIC should cache the packet causing the wake on hardware and pass it up when the OS is ready for receives.

Mobile Broadband class of devices must support wake on mobile broadband. Device should wake the system on above mentioned events. Note that wake on USSD is mandatory only if the device reports that it supports USSD, else it is optional. See the following MSDN documentation for more information on the SMS and register state wake events.

NDIS_STATUS_WWAN_REGISTER_STATE

NDIS_STATUS_WWAN_SMS_RECEIVE

Exceptions: Not Specified

Business Justification:

To support Mobile Broadband scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.Router

Description:

Feature required to be a router

Related Requirements:

Device.Network.Router.BasicCompatibility
Device.Network.Router.BasicPerf
Device.Network.Router.ConeOrRestrictedNAT
Device.Network.Router.GetTotalBytesPerf

Device.Network.Router.GetTotalBytesSupport
Device.Network.Router.NATLoopback
Device.Network.Router.PnPXUPnPSupport
Device.Network.Router.PresentationURLPnPProperty
Device.Network.Router.UPnPIGD
Device.Network.Router.UPnPPortMappings
Device.Network.Router.WCNDynamicPIN
Device.Network.Router.WFACertified
Device.Network.Router.WPSVer2
Device.Network.Router.WPSVer2PushButton

Device.Network.Router.BasicCompatibility

Target Feature: Device.Network.Router

Title: Routers must meet basic Microsoft product compatibility requirements

Applicable OS Versions:

Windows 7 Client x86
Windows 8 Client x86

Description:

A router must meet basic Microsoft product compatibility requirements. These include the following:

MTU size: The maximum transmission unit (MTU) size must not exceed 1365.

ICMP echo: The router must avoid resetting port mappings if an Internet Control Message Protocol (ICMP) Echo message fails or times out. The router must support correct responses to ICMP Destination Unreachable and Port Unreachable messages.

DHCP lease: A Dynamic Host Configuration Protocol (DHCP) client behind the routing device can receive the same IP address with a lease duration of longer than five minutes when an IP address is renewed repeatedly.

UDP packet handling: UDP packets from separate WAN-side IP addresses must be able to traverse the network address translation (NAT) component of the router. For session policy, devices must keep a UDP port association open when the only traffic that the router receives is the keep-alive traffic that is generated through UDP.

TCP sockets: The router must be able to download packets on TCP ports 80 and 307.4

FIN segment response: For the TCP finish (FIN) segment response, the device must keep a TCP socket association open until a download is complete, even after an internal client sends a TCP FIN packet.

Windows Scaling

ECN

Teredo

Exceptions: Not Specified

Business Justification:

This requirement helps ensure overall compatibility testing for home routers to support Windows.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

NETWORK-0082

Device.Network.Router.BasicPerf

Target Feature: Device.Network.Router

Title: Wireless router must meet basic wireless performance requirements.

Applicable OS Versions:

Windows 7 Client x86
Windows 8 Client x86

Description:

A wireless router must meet be able to sustain a minimum wireless throughput of 18 megabits per second (Mbps) (for 11g routers) and 60 Mbps (for 11n routers) over a Wi-Fi Protected Access version 2 (WPA2) Advanced Encryption Standard (AES) pre-shared key (PSK) secured connection based on the following test requirements and metrics:

The test range is at least 10 feet.

The test duration is one hour.

The packet loss during the test is 1% or less.

The test environment is open air, or non-chamber.

Exceptions: Not Specified

Business Justification:

This ensures acceptable performance for end users.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

NETWORK-0057

Device.Network.Router.ConeOrRestrictedNAT

Target Feature: Device.Network.Router

Title: Routers must implement a cone or restricted NAT type

Applicable OS Versions:

Windows 7 Client x86
Windows 8 Client x86

Description:

Routers must not use a symmetric NAT type.

After traversal, the NAT must store mappings between the private address and port pair and the public address and port pair. After the NAT translation table entry is established, inbound traffic to an external address and port pair is allowed from any source address and port pair.

Exceptions: Not Specified

Business Justification:

This helps ensure industry standardization on NAT

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

NETWORK-0081

Device.Network.Router.GetTotalBytesPerf

Target Feature: Device.Network.Router

Title: Routers must respond to the GetTotalBytesSent and GetTotalBytesReceived actions within 1000ms

Applicable OS Versions:

Windows 7 Client x86
Windows 8 Client x86

Description:

When the WAN port of the device is saturated, or under 95% load of rated port speed,the device must respond to **GetTotalBytesSent** and **GetTotalBytesReceived** queries within 1000 milliseconds (ms). The device must be able to respond to five simultaneous requests without reporting any errors. The total time to process the requests is cumulative. Five simultaneous requests may take 1000 ms x 5, or 5000 ms, to complete. These actions are contained within the UPnP Internet Gateway Device (IGD) device control protocol (DCP). These actions must be turned on and enabled by default. See requirement Network-0080.

Design Notes:

See the UPnP IGD Specification revision1.0 at http://go.microsoft.com/fwlink/?LinkId=58381

Exceptions: Not Specified

Business Justification:

Background Intelligent Transfer Service (BITS) is used by Windows Update, Microsoft Update, Systems Management Server (SMS), MOM, and many other applications to distribute and maintain software on Windows systems. Many of these computers are found in homes, small businesses, and branch offices behind low-cost gateway devices. These computers must be maintained without interrupting normal business operations or home users who are playing video games or listening to music.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

NETWORK-0050

Device.Network.Router.GetTotalBytesSupport

Target Feature: Device.Network.Router

Title: Routers must support GetTotalBytesSent and GetTotalBytesReceived as defined in the UPnP IGD 1.0 specification

Applicable OS Versions:

Windows 7 Client x86
Windows 8 Client x86

Description:

All routers must correctly implement the **GetTotalBytesSent** and **GetTotalBytesReceived** actions of the **urn:schemas-upnp-org:service:WANCommonInterfaceConfig:1** <element> as defined in the

UPnP IGD 1.0 specification. According to the standard, the counters must be an unsigned 32-bit number. The counters must also correctly handle values that are larger than 2 gigabytes (GB).

Design Notes:

See the UPnP IGD Specification Revision 1.0 at http://go.microsoft.com/fwlink/?LinkId=58381

Exceptions: Not Specified

Business Justification:

Background Intelligent Transfer Service (BITS) is used by Windows Update, Microsoft Update, Systems Management Server (SMS), MOM, and many other applications to distribute and maintain software on Windows systems. Many of these computers are found in homes, small businesses, and branch offices behind low-cost gateway devices. These computers must be maintained without interrupting normal business operations or home users who are playing video games or listening to music.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

NETWORK-0065

Device.Network.Router.NATLoopback

Target Feature: Device.Network.Router

Title: Router must support NAT Loopback.

Applicable OS Versions:

Windows 7 Client x86
Windows 8 Client x86

Description:

A NAT device must support hairpin or loopback functionality. This means the NAT must carry out a "Twice-NAT" translation of addresses for local systems, allowing them to communicate with one another. When a host on the private side of a NAT device attempts to connect with another host behind the same NAT device by using the public address of the target host, the NAT device must perform the equivalent of a "Twice-NAT" translation on the packet. The originating host's private endpoint must be translated into its assigned public endpoint and the target host's public endpoint must be translated into its private endpoint, before the packet is forwarded to the target host

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

NETWORK-0075

Device.Network.Router.PnPXUPnPSupport

Target Feature: Device.Network.Router

Title: IGDs must support PnP-X extension to UPnP

Applicable OS Versions:

Windows 8 Client x86

Description:

All IGDs must support PnP-X extension to UPnP. The specific tags that need to be implemented are:

Device category

Wireless routers:

<pnpx:X_deviceCategory>**NetworkInfrastructure.Router**</pnpx:X_deviceCategory>

<df:X_deviceCategory>**Network.Router.Wireless**</df:X_deviceCategory>

Wired routers:

<pnpx:X_deviceCategory>**NetworkInfrastructure.Router**</pnpx:X_deviceCategory>

<df:X_deviceCategory>**Network.Router**</df:X_deviceCategory>

Hardware ID

<pnpx:X_hardwareId>**ROUTER_IHV_TO_ADD_DEVICE_SPECIFIC_HARDWARE_ID_PLUS_VEN_0033&DEV_0008&REV_01**</pnpx:X_hardwareId>

Compatible ID

<pnpx:CompatibleId>**urn:schemas-upnp-org:device:InternetGatewayDevice:1**</pnpx:CompatibleId>

Design Notes (if any):

For more information about PnP-X requirements, see Using Windows Rally Vertical Pairing to Automatically Install Wi-Fi Devices.

Exceptions: Not Specified

Business Justification:

This requirement allows IGDs to be installed on the computer so that the IHV can create and deploy a Device Stage package.

Scenarios: Not Specified

Success Metric: If device passes certification tests there would be no yellow exclamations for IGD Devices in the Device Hub.

Enforcement Date: Windows RC

Comments:

New

Device.Network.Router.PresentationURLPnPProperty

Target Feature: Device.Network.Router

Title: All IGDs must contain the 'Presentation URL' UPnP property

Applicable OS Versions:

Windows 8 Client x86

Description:

All IGDs must contain the **Presentation URL** UPnP property. The URL must be the address of the IGD management web-page

Exceptions: Not Specified

Business Justification:

This requirement allows IGDs to be installed on the computer. The requirement also helps ensure that the default double-click action is to open the router management webpage.

Scenarios: Not Specified

Success Metric: If the device passes certification tests, the IGD device appears in the device hub. The default double-click action opens the default browser to the router management webpage.

Enforcement Date: Windows RC

Comments:

New

Device.Network.Router.UPnPIGD

Target Feature: Device.Network.Router

Title: Router must implement UPnP IGD v1.0 and ship with UPnP IGD enabled (turned on) by default.

Applicable OS Versions:

Windows 7 Client x86
Windows 8 Client x86

Description:

The device must implement UPnP Internet Gateway Device (IGD) 1.0 or greater. Device must have passed all UPnP Implementers Corporation tests for IGD. UPnP IGD must be on (enabled) by default, that is, in the factory-shipping state and following a physical (reset to factory conditions) reset.

Exceptions: Not Specified

Business Justification:

Required for device discovery and programmatic access to create / delete port mappings.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

NETWORK-0080

Device.Network.Router.UPnPPortMappings

Target Feature: Device.Network.Router

Title: Router must support at least 25 UPnP port mappings.

Applicable OS Versions:

Windows 7 Client x86
Windows 8 Client x86

Description:

A router must support at least 25 individual port mappings that can be configured remotely via UPnP.

Exceptions: Not Specified

Business Justification:

For SBS to function successfully, this number of ports must be equal or greater to 25. Without this, core remote access functionality will not be available.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

NETWORK-0055

Device.Network.Router.WCNDynamicPIN

Target Feature: Device.Network.Router

Title: Router that implements a display must generate a dynamic WCN PIN and display it correctly.

Applicable OS Versions:

Windows 7 Client x86
Windows 8 Client x86

Description:

A wireless router that implements a display (ex. OLED or LCD) that can support a 4 or 8 digit WCN PIN must generate a dynamic WCN PIN and displaying it on the screen correctly.

The device must indicate support for display in the WPS configuration methods.

Design Notes:

See the WCN Specification at http://go.microsoft.com/fwlink/?LinkId=50323.

Exceptions: Not Specified

Business Justification:

Dynamic WCN Pins provide additional wireless security.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

NETWORK-0064

Device.Network.Router.WFACertified

Target Feature: Device.Network.Router

Title: A wireless router must be WFA (Wi-Fi Alliance) certified.

Applicable OS Versions:

Windows 7 Client x86
Windows 8 Client x86

Description:

A wireless router must be WFA (Wi-Fi Alliance) certified for:

All IEEE radio standards supported in the device: 802.11a, 802.11b, 802.11g, and 802.11n

Wi-Fi wireless network security - WPA (Wi-Fi Protected Access) and WPA2 (Wi-Fi Protected Access 2)

WiFi Protected Setup (WPS PIN and WPS PBC)

WiFi Multimedia QoS (WMM)

The WiFi Certification ID for the device is required to verify these requirements.

Exceptions: Not Specified

Business Justification:

LLTD-QoS/qWave and WCN align with WFA's requirements for WMM and WPS.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

NETWORK-0052

Device.Network.Router.WPSVer2

Target Feature: Device.Network.Router

Title: Routers must support WPS-Version 2

Applicable OS Versions:

Windows 7 Client x86
Windows 8 Client x86

Description:

A wireless router must support the Wi-Fi Protected Setup Specification version 2.0 from the Wi-Fi Alliance. The router must also pass the Wi-Fi Alliance certification program.

Exceptions: Not Specified

Business Justification:

The Wi-Fi Protected Setup Specification version 2.0 describes a simple and protected method that is used to set up and connect to networks.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

NETWORK-0085; Updated

Device.Network.Router.WPSVer2PushButton

Target Feature: Device.Network.Router

Title: Wireless routers must support a hardware push-button for WPS Version 2

Applicable OS Versions:

Windows 7 Client x86
Windows 8 Client x86

Description:

A wireless router must have a hardware push button that the user can clearly identify as the push button for setting up wireless security. The push button must comply with the WCN specification.

Exceptions: Not Specified

Business Justification:

This requirement helps complete the WCN-NET push button experience.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

NETWORK-0088; Update

Device.Network.WLAN.Base

Description:

Wireless LAN

Related Requirements:

Device.Network.WLAN.Base.ConformToNDIS
Device.Network.WLAN.Base.ImplementD0PacketCoalescing
Device.Network.WLAN.Base.ImplementVoicePersonalWMMPowerSave
Device.Network.WLAN.Base.MeetPerformanceReq
Device.Network.WLAN.Base.MeetScanAndConnReq
Device.Network.WLAN.Base.MinimizeCPUUtilization
Device.Network.WLAN.Base.OnlyWDFOrNDIS630Calls
Device.Network.WLAN.Base.PassWiFiAllianceCertification
Device.Network.WLAN.Base.PermitIEToRequestAndResponseAF
Device.Network.WLAN.Base.SupportFiltering32MulticastAddresses
Device.Network.WLAN.Base.SupportIEEE80211w
Device.Network.WLAN.Base.SupportMultiDeviceInstances
Device.Network.WLAN.Base.SupportPromiscuousAndMulticastPacketFiltering
Device.Network.WLAN.Base.SupportSeparateBeaconAndProbeIE
Device.Network.WLAN.Base.SupportVirtualWiFi
Device.Network.WLAN.Base.SupportWiFiAutoSaveMode
Device.Network.WLAN.Base.TransmitPacketsOnAnyBoundary

Device.Network.WLAN.Base.ConformToNDIS

Target Feature: Device.Network.WLAN.Base

Title: WLAN devices MUST conform to NDIS requirements in the Windows Driver Kit.

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 7 Client x86
Windows 7 Client x64

Description:

All WLAN device drivers MUST conform to NDIS 6.30 and the Native Wi-Fi driver model specified in the Windows Driver Kit.

Exceptions: Not Specified

Business Justification:

To support WLAN functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.WLAN.Base.ImplementD0PacketCoalescing

Target Feature: Device.Network.WLAN.Base

Title: WLAN devices that implement D0 Packet Coalescing MUST support D0 Packet Coalescing.

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64

Description:

Windows will optimize the networking power efficiency by allowing the device to aggregate and delay certain network protocols. The device that implements D0 Packet Coalescing is expected to queue packets and indicate to the OS on a periodic basis.

Exceptions: Not Specified

Business Justification:

To support WLAN functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.WLAN.Base.ImplementVoicePersonalWMMPowerSave

Target Feature: Device.Network.WLAN.Base

Title: WLAN devices that implement Voice-Personal, and/or WMM Power Save MUST obtain respective Wi-Fi Alliance Certifications for Voice-Personal, and/or WMM Power Save.

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 7 Client x86
Windows 7 Client x64

Description:

This requirement only applies to WLAN devices that implement and/or Voice-Personal.

Voice-Personal: If Voice-Personal is implemented, the 802.11 device MUST pass current WFA Voice-Personal certification.

WMM Power Save: If WMM power save is implemented, the 802.11 device must successfully pass the current WFA certification for WMM Power Save.

Exceptions: Not Specified

Business Justification:

To support WLAN functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.WLAN.Base.MeetPerformanceReq

Target Feature: Device.Network.WLAN.Base

Title: WLAN devices MUST meet performance requirements.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64

Description:

All 802.11 devices MUST meet the following performance requirements:

Virtual Wi-Fi: For Virtual Wi-Fi, all 802.11 devices MUST NOT have more than 15% aggregate throughput degradation when data flow is divided between multiple Virtual Wi-Fi ports compared with aggregate throughput when only one Virtual Wi-Fi port is connected.

Rate Negotiation: Driver should not drop physical rate more than 25% when under congestion (same channel cross traffic). And it should recover to within 5% of original rate within 2 seconds after congestion ceases.

Throughput: All drivers that claim 11n capability MUST sustain 20 Mbps throughput for at least 15 minutes.

Exceptions: Not Specified

Business Justification:

To support WLAN functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.WLAN.Base.MeetScanAndConnReq

Target Feature: Device.Network.WLAN.Base

Title: WLAN devices MUST meet scanning and connection requirements.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64

Description:

Preferred channels set: When supported and permitted by the regulatory domain, the miniport driver MUST prefer the following channels when scanning for available networks or roaming to find a candidate access point:

- 2.4 Ghz channels: 1 to 14
- 5GHz U-NII Low channels: 36, 40, 44, 48
- 5GHz U-NII Mid channels: 52, 56, 60,64
- 5GHz U-NII Worldwide channels: 100, 104, 108, 112, 116, 120, 124, 128, 132, 136, 140
- 5GHz U-NII Upper channels: 149, 153, 157, 161,165

The driver should report support for these and any other channels it supports through OID_DOT11_SUPPORTED_DSSS_CHANNEL_LIST and OID_DOT11_SUPPORTED_OFDM_FREQUENCY_LIST.

Association Time: On WPA2-PSK networks, WLAN devices should finish the association within 200ms. It is measured as the time between the events when OS issues an OID OID_DOT11_CONNECT_REQUEST and miniport sends an NDIS_STATUS_DOT11_ASSOCIATION_COMPLETION indication to the OS.

General Scanning: WLAN device should start scanning when it receives OID OID_DOT11_SCAN_REQUEST from OS or the device resumes to D0 state from low power state and reply back with an indication NDIS_STATUS_DOT11_SCAN_CONFIRM to the OS as soon as it completes the scan. In case of active scanning, miniport is expected to send the active wildcard probes to the network channels to meet the scanning goals. In case of passive scanning, miniport is not expected to send any probes to the network channels.

Following priority order should be followed for scanning.

NLO channel hints

Preferred channels

Any remaining channels

The timings listed below will be measured from the time stamp when the device receives OID OID_DOT11_SCAN_REQUEST from OS to the time stamp when the respective indication is provided to the OS by the miniport.

If the Network list offload hints are available, the device should leverage the network list offload hints to optimize scanning behavior and return NDIS_STATUS_DOT11_OFFLOAD_NETWORK_STATE_CHANGED indication when a matching profile is found within the following timings:

Scanning a network were active scanning is allowed 20ms/channel

Scanning a network were only passive scanning is allowed 120ms/channel

If there is no match found using Network List Offload hints, WLAN device should next scan the preferred channels in the list above. For scanning the channels in the preferred channel list, WLAN device should not take more than 3.5 seconds (time includes scanning for both active and passive channels). The newly created list of surrounding BSS entries should be returned on the next BSS list query from the OS.

If there is no match found in above 2 cases, WLAN device should next scan any remaining channels. WLAN device should not take more than 4 sec for the entire scan operation.

Resume from Sleep/Screen Off:

When resuming from sleep/screen off, the WLAN device is expected to reconnect to the same AP that it was connected to before going to sleep, if it is available. WLAN devices should meet the following timings for detecting its presence:

Reconnecting to a channel were active scanning is allowed 50ms

Reconnecting to a channel were only Passive scanning is allowed 120ms

If the last connected network is not found, WLAN device should scan for networks in the priority order listed below.

NLO channel hints

Preferred channels

Any remaining channels

WLAN device should follow the similar timing constraints as defined above in the general scanning section. The timings will be measured from the time stamp when the device reports as being in D0 state (protocol driver reporting NetDeviceStateD0 through NetEventSetPower PnP event) to the time stamp when the respective indication is provided to the OS by the miniport.

Roaming: Driver MUST detect and indicate the loss of AP (no beacons) within 20 beacon intervals.

Exceptions: Not Specified

Business Justification:

To support WLAN functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.WLAN.Base.MinimizeCPUUtilization

Target Feature: Device.Network.WLAN.Base

Title: WLAN devices MUST minimize CPU utilization.

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64

Description:

The Wi-Fi device should conform to the following requirements for minimizing CPU utilization.

While in EXTSTA mode and in D0 state, WLAN devices must not interrupt OS more than 1 time per data packet (non D0 coalesced packets only. If WLAN device supports D0 packet coalescing, WLAN device should comply with the D0 packet coalescing spec for D0 coalesced packets). Note that if WLAN device is using SDIO bus interface, interrupts generated by SDIO host controller per data packet are exempted from this requirement. Non data packet-related interrupts, if needed, must not exceed an average of 3 interrupts per second when measured over a 2 minute period. Also, as a best practice, in active state (when there is data traffic), the non-data packet related interrupts should be issued within 1 millisecond of a valid packet-related interrupt.

In D0 state, all (if any) miniport specific periodic maintenance timers must be specified using an available timer coalescing API, with a minimum 2 second period and 1 second tolerance. This requirement will be tested in a long running connected state when there is no change in connectivity. All such timers must be cancelled in D3 state. Note that this requirement does not apply to timers defined in IEEE 802.11 specification e.g., connection timers such as association timers, roaming timers etc.

An individual DPC (Deferred Procedural Call) duration MUST not exceed 2 milliseconds. Accumulated DPC duration should be less than 4 milliseconds over any 10 millisecond window.

In low power states, if the device is not Wake on Wireless capable, WLAN device must not interrupt the CPU. If the device is Wake on Wireless capable, WLAN device must not interrupt the CPU except on wake triggers indicated by NDIS for Wake on Wireless LAN. All interrupts not related to wake triggers must be cancelled.

Exceptions: Not Specified

Business Justification:

To support WLAN functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.WLAN.Base.OnlyWDFOrNDIS630Calls

Target Feature: Device.Network.WLAN.Base

Title: WLAN devices MUST make only NDIS 6.30 or WDF system calls.

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 7 Client x86
Windows 7 Client x64

Description:

Network device drivers must make only Network Driver Interface Specification (NDIS) 6.30 or Windows Driver Foundation (WDF) calls. Any calls to kernel mode components are not allowed.

See the "NDIS" and "WDF" topics in the Windows Driver Kit.

Exceptions: Not Specified

Business Justification:

To support WLAN functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.WLAN.Base.PassWiFiAllianceCertification

Target Feature: Device.Network.WLAN.Base

Title: WLAN Device MUST successfully pass the current Wi-Fi Alliance certification for 802.11/WPA2/WPA, 802.11n, Wi-Fi Protected Setup (STA), WMM and Wi-Fi Direct.

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 7 Client x86
Windows 7 Client x64

Description:

WLAN Device must successfully pass the current Wi-Fi Alliance (WFA) certification for 802.11/WPA2/WPA, 802.11n, Wi-Fi Protected Setup (STA), WMM (STA) and Wi-Fi Direct.

802.11/WPA2/WPA: The device must pass the WFA certification for 802.11/WPA2/WPA. Support for IEEE 802.11g is required. 802.11a-only implementations are not permitted.

802.11n: The device must pass the WFA certification for 802.11n.

Wi-Fi Protected Setup: The device must successfully pass the current WFA certification for Wi-Fi Protected Setup.

WMM: The device must successfully pass the current WFA certification for WMM.

Wi-Fi Direct: The device must successfully pass the current WFA certification for Wi-Fi Direct.

Exceptions: Not Specified

Business Justification:

To support WLAN functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2009

Comments:

NETWORK - 0224

Device.Network.WLAN.Base.PermitIEToRequestAndResponseAF

Target Feature: Device.Network.WLAN.Base

Title: WLAN devices MUST permit addition of Information Elements to request and response association frames.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64

Description:

All 802.11 devices MUST permit Information Elements to be added to association frames, both requests and responses. This includes adding currently specified Information Elements, such as Wi-Fi Protected Setup as well as other vendor extended Information Elements.

Exceptions: Not Specified

Business Justification:

To support WLAN functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.WLAN.Base.SupportFiltering32MulticastAddresses

Target Feature: Device.Network.WLAN.Base

Title: WLAN devices MUST support filtering for at least 32 multicast addresses on each port.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64

Description:

WLAN hardware MUST support at least 32 multicast addresses on each port. Both STA and Wi-Fi-Direct ports need to support filtering 32 multicast addresses separately.

Exceptions: Not Specified

Business Justification:

To support WLAN functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.WLAN.Base.SupportIEEE80211w

Target Feature: Device.Network.WLAN.Base

Title: WLAN devices MUST Support IEEE 802.11w standard for protected management frames.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64

Description:

IEEE 802.11w is an addition to IEEE 802.11 suite of standards to enhance the security of management frames. All WLAN devices must support the IEEE 802.11w standard.

Exceptions: Not Specified

Business Justification:

To support WLAN functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.WLAN.Base.SupportMultiDeviceInstances

Target Feature: Device.Network.WLAN.Base

Title: WLAN devices MUST support multiple device instances.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64

Description:

Plug and Play can support multiple instances of a device. Multiple instances of the device can exist and function in the same system at the same instance. For network communications devices, the Plug and Play IDs and resource support MUST be sufficient to allow multiple network

communications devices to be added automatically to the system. This requirement implies that all device resources MUST be set and read through the standard interfaces provided by the bus on which the device resides. For PCI devices, this interface is the PCI configuration space. Also, device parameter settings MUST be stored in the registry.

Exceptions: Not Specified

Business Justification:

To support WLAN functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.WLAN.Base.SupportPromiscuousAndMulticastPacketFiltering

Target Feature: Device.Network.WLAN.Base

Title: WLAN devices MUST support promiscuous and multicast packet filtering.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64

Description:

WLAN device and driver MUST support promiscuous and multicast packet filtering. The miniport driver MUST support all filter types identified in the Windows Driver Kit. By default, multicast promiscuous mode is not enabled.

Exceptions: Not Specified

Business Justification:

To support WLAN functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.WLAN.Base.SupportSeparateBeaconAndProbeIE

Target Feature: Device.Network.WLAN.Base

Title: WLAN devices MUST support separate beacon and probe Information Elements.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64

Description:

All 802.11 devices MUST separately indicate the Wi-Fi Protected Setup IEs that are received in Beacon frames and probe-response frames. If the device has received both a beacon frame and a probe-response frame from a particular BSSID, then it MUST provide two instances of the IE, where one instance is the most recently received WPS IE from the Beacon and one instance is the most recently received WPS IE from the probe response.

Exceptions: Not Specified

Business Justification:

To support WLAN functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.WLAN.Base.SupportVirtualWiFi

Target Feature: Device.Network.WLAN.Base

Title: WLAN devices MUST support Virtual Wi-Fi.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64

Description:

All 802.11 devices MUST support Virtual Wi-Fi and enable simultaneous infrastructure STA connection and Soft AP hosting OR infrastructural STA and Wi-Fi Direct ports. The Virtual Wi-Fi interface is specified in the Extensible WLAN driver specification document.

Exceptions: Not Specified

Business Justification:

To support WLAN functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.WLAN.Base.SupportWiFiAutoSaveMode

Target Feature: Device.Network.WLAN.Base

Title: WLAN devices MUST Support Wi-Fi Auto Power Save Mode.

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64

Description:

The Wi-Fi driver is required to perform detection and negotiation of proper Wi-Fi Power Save Mode (PSM) between the device and the Wi-Fi Access Point and report it in driver capability during initialization in DOT11_EXTSTA_ATTRIBUTES. If the driver reports that it supports PSM detection, WLAN service will delegate the PSM decision to the driver by default.

If the driver supports the AUTO-PSM capability the Wi-Fi service will no longer set the broadcast management filter to receive beacons from the miniport and will instead set an OID to turn on Auto-PSM in the driver. When in Auto-PSM, the driver should always negotiate PSM mode when it detects that the AP supports it and manage the use of PSM between the device and the Wi-Fi Access Point to ensure optimal connectivity while using the least amount of power.

Exceptions: Not Specified

Business Justification:

To support WLAN functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.WLAN.Base.TransmitPacketsOnAnyBoundary

Target Feature: Device.Network.WLAN.Base

Title: WLAN devices MUST be able to transmit packets from buffers aligned on any boundary.

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64

Description:

Buffer alignment refers to whether a buffer begins on an odd-byte, word, double-word, or other boundary. Devices MUST be able to transmit packets with any of the packets fragments beginning on an odd-byte boundary. For performance reasons, packets MUST be received into contiguous buffers on a double-word boundary.

Exceptions: Not Specified

Business Justification:

To support WLAN functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.WLAN.CSBBase

Description:

Wireless LAN

Related Requirements:

- Device.Network.WLAN.CSBBase.ConformToNDIS
- Device.Network.WLAN.CSBBase.ImplementVoicePersonalWMMPowerSave
- Device.Network.WLAN.CSBBase.MeetPerformanceReq
- Device.Network.WLAN.CSBBase.MeetScanAndConnReq
- Device.Network.WLAN.CSBBase.MinimizeCPUUtilization
- Device.Network.WLAN.CSBBase.MustSupportD0PacketCoalescing
- Device.Network.WLAN.CSBBase.OnlyWDFOrNDIS630Calls
- Device.Network.WLAN.CSBBase.PassWiFiAllianceCertification
- Device.Network.WLAN.CSBBase.PermitIEToRequestAndResponseAF
- Device.Network.WLAN.CSBBase.SupportFiltering32MulticastAddresses

Device.Network.WLAN.CSBBase.SupportIEEE80211w
Device.Network.WLAN.CSBBase.SupportPromiscuousAndMulticastPacketFiltering
Device.Network.WLAN.CSBBase.SupportSeparateBeaconAndProbeIE
Device.Network.WLAN.CSBBase.SupportVirtualWiFi
Device.Network.WLAN.CSBBase.SupportWiFiAutoSaveMode
Device.Network.WLAN.CSBBase.TransmitPacketsOnAnyBoundary

Device.Network.WLAN.CSBBase.ConformToNDIS

Target Feature: Device.Network.WLAN.CSBBase

Title: WLAN devices that go into systems that support connected standby MUST conform to NDIS requirements in the Windows Driver Kit.

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Client ARM

Description:

All WLAN devices that go into systems that support connected standby MUST conform to NDIS 6.30 and the Native Wi-Fi driver model specified in the Windows Driver Kit.

Exceptions: Not Specified

Business Justification:

To support WLAN functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.WLAN.CSBBase.ImplementVoicePersonalWMMPowerSave

Target Feature: Device.Network.WLAN.CSBBase

Title: WLAN devices that go into systems that support connected standby and implement Voice-Personal and/or WMM Power Save MUST obtain respective Wi-Fi Alliance Certifications for Voice-Personal and/or WMM Power Save.

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64

Windows 8 Client ARM

Description:

This requirement only applies to WLAN devices that implement and/or Voice-Personal.

Voice-Personal: If Voice-Personal is implemented, the 802.11 device MUST pass current WFA Voice-Personal certification.

WMM Power Save: If WMM power save is implemented, the 802.11 device must successfully pass the current WFA certification for WMM Power Save.

Exceptions: Not Specified

Business Justification:

To support WLAN functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.WLAN.CSBBase.MeetPerformanceReq

Target Feature: Device.Network.WLAN.CSBBase

Title: WLAN devices that go into systems that support connected standby MUST meet performance requirements.

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Client ARM

Description:

All 802.11 devices MUST meet the following performance requirements:

Virtual Wi-Fi: For Virtual Wi-Fi, all 802.11 devices MUST NOT have more than 15% aggregate throughput degradation when data flow is divided between multiple Virtual Wi-Fi ports compared with aggregate throughput when only one Virtual Wi-Fi port is connected.

Rate Negotiation: Driver should not drop physical rate more than 25% when under congestion (same channel cross traffic). And it should recover to within 5% of original rate within 2 seconds after congestion ceases.

Throughput: All drivers that claim 11n capability MUST sustain 20 Mbps throughput for at least 15 minutes.

Exceptions: Not Specified

Business Justification:

To support WLAN functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.WLAN.CSBBase.MeetScanAndConnReq

Target Feature: Device.Network.WLAN.CSBBase

Title: WLAN devices that go into systems that support connected standby MUST meet scanning and connection requirements.

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Client ARM

Description:

Preferred channels set: When supported and permitted by the regulatory domain, the miniport driver MUST prefer the following channels when scanning for available networks or roaming to find a candidate access point:

2.4 Ghz channels: 1 to 14

5GHz U-NII Low channels: 36, 40, 44, 48

5GHz U-NII Mid channels: 52, 56, 60,64

5GHz U-NII Worldwide channels: 100, 104, 108, 112, 116, 120, 124, 128, 132, 136, 140

5GHz U-NII Upper channels: 149, 153, 157, 161,165

The driver should report support for these and any other channels it supports through OID_DOT11_SUPPORTED_DSSS_CHANNEL_LIST and OID_DOT11_SUPPORTED_OFDM_FREQUENCY_LIST.

Association Time: On WPA2-PSK networks, WLAN devices should finish the association within 200ms. It is measured as the time between the events when OS issues an OID OID_DOT11_CONNECT_REQUEST and miniport sends an NDIS_STATUS_DOT11_ASSOCIATION_COMPLETION indication to the OS.

General Scanning: WLAN device should start scanning when it receives OID OID_DOT11_SCAN_REQUEST from OS or the device resumes to D0 state from low power state and reply back with an indication NDIS_STATUS_DOT11_SCAN_CONFIRM to the OS as soon as it completes the scan. In case of active scanning, miniport is expected to send the active wildcard probes to the network channels to meet the scanning goals. In case of passive scanning, miniport is not expected to send any probes to the network channels.

Following priority order should be followed for scanning.

NLO channel hints

Preferred channels

Any remaining channels

The timings listed below will be measured from the time stamp when the device receives OID OID_DOT11_SCAN_REQUEST from OS to the time stamp when the respective indication is provided to the OS by the miniport.

If the Network list offload hints are available, the device should leverage the network list offload hints to optimize scanning behavior and return NDIS_STATUS_DOT11_OFFLOAD_NETWORK_STATE_CHANGED indication when a matching profile is found within the following timings:

Scanning a channel were active scanning is allowed 20ms/channel

Scanning a channel were only passive scanning is allowed 120ms/channel

If there is no match found using Network List Offload hints, WLAN device should next scan the preferred channels in the list above. For scanning the channels in the preferred channel list, WLAN device should not take more than 3.5 seconds (time includes scanning for both active and passive channels). The newly created list of surrounding BSS entries should be returned on the next BSS list query from the OS.

If there is no match found in above 2 cases, WLAN device should next scan any remaining channels. WLAN device should not take more than 4 sec for the entire scan operation.

Resume from Sleep/Screen Off:

When resuming from sleep/screen off, the WLAN device is expected to reconnect to the same AP that it was connected to before going to sleep, if it is available. WLAN devices should meet the following timings for detecting its presence:

Reconnecting to a channel were active scanning is allowed 50ms

Reconnecting to a channel were only Passive scanning is allowed 120ms

If the last connected network is not found, the WLAN device should scan for networks in the priority order listed below.

NLO channel hints

Preferred channels

Any remaining channels

WLAN device should follow the similar timing constraints as defined above in the general scanning section. The timings in this case will be measured from the time stamp when the device reports as being in D0 state (protocol driver reporting NetDeviceStateD0 through NetEventSetPower PnP event) to the time stamp when the respective indication is provided to the OS by the miniport.

Roaming: Driver MUST detect and indicate the loss of AP (no beacons) within 20 beacon intervals.

Exceptions: Not Specified

Business Justification:

To support WLAN functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.WLAN.CSBBase.MinimizeCPUUtilization

Target Feature: Device.Network.WLAN.CSBBase

Title: WLAN devices that go into systems that support connected standby MUST minimize CPU utilization.

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Client ARM

Description:

The Wi-Fi device should conform to the following requirements for minimizing CPU utilization.

While in EXTSTA mode and in D0 state, WLAN devices must not interrupt OS more than 1 time per data packet (non D0 coalesced packets only. WLAN device should comply with the D0 packet coalescing spec for D0 coalesced packets). Note that if WLAN device is using SDIO bus interface, interrupts generated by SDIO host controller per data packet are exempted from this requirement. Non data packet related interrupts must not interrupt the CPU and should be handled by the device.

In D0 state, all (if any) miniport specific periodic maintenance timers must be specified using an available timer coalescing API, with a minimum 5 second period and 10 second tolerance.

This requirement will be tested in a long running connected state when there is no change in connectivity. All such timers must be cancelled in D3 state. Note that this requirement does not apply to timers defined in IEEE 802.11 specification e.g., connection timers such as association timers, roaming timers etc.

While in ExtSTA mode and in D0 state, WLAN device must not indicate beacons to OS unless configured to do so by the OS.

An individual DPC (Deferred Procedural Call) duration MUST not exceed 2 milliseconds. Accumulated DPC duration should be less than 4 milliseconds over any 10 millisecond window.

In low power states when the device is armed for Wake, WLAN device must not interrupt the CPU except on the wake triggers indicated by NDIS for Wake on Wireless LAN. All interrupts not related to wake triggers must be cancelled.

Exceptions: Not Specified

Business Justification:

To support WLAN functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.WLAN.CSBBase.MustSupportD0PacketCoalescing

Target Feature: Device.Network.WLAN.CSBBase

Title: WLAN devices that go into systems that support connected standby MUST support D0 Packet Coalescing.

Applicable OS Versions:

Windows 8 Client ARM
Windows 8 Client x64
Windows 8 Client x86

Description:

Windows will optimize the networking power efficiency by allowing the device to aggregate and delay certain network protocols. The device is expected to queue packets and indicate to the OS on a periodic basis.

Exceptions: Not Specified

Business Justification:

To support WLAN functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.WLAN.CSBBase.OnlyWDFOrNDIS630Calls

Target Feature: Device.Network.WLAN.CSBBase

Title: WLAN devices that go into systems that support connected standby MUST make only NDIS 6.30 or WDF system calls.

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Client ARM

Description:

Network device drivers must make only Network Driver Interface Specification (NDIS) 6.30 or Windows Driver Foundation (WDF) calls. Any calls to kernel mode components are not allowed.

See the "NDIS" and "WDF" topics in the Windows Driver Kit.

Exceptions: Not Specified

Business Justification:

To support WLAN functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.WLAN.CSBBase.PassWiFiAllianceCertification

Target Feature: Device.Network.WLAN.CSBBase

Title: WLAN Devices that go into systems that support connected standby MUST successfully pass the current Wi-Fi Alliance certification for 802.11/WPA2/WPA, 802.11n, Wi-Fi Protected Setup (STA), WMM and Wi-Fi Direct.

Applicable OS Versions:

Windows 8 Client x86

Windows 8 Client x64
Windows 8 Client ARM

Description:

WLAN Device must successfully pass the current Wi-Fi Alliance (WFA) certification for 802.11/WPA2/WPA, 802.11n, Wi-Fi Protected Setup (STA), WMM (STA) and Wi-Fi Direct.

802.11/WPA2/WPA: The device must pass the WFA certification for 802.11/WPA2/WPA. Support for IEEE 802.11g is required. 802.11a-only implementations are not permitted.

802.11n: The device must pass the WFA certification for 802.11n.

Wi-Fi Protected Setup: The device must successfully pass the current WFA certification for Wi-Fi Protected Setup.

WMM: The device must successfully pass the current WFA certification for WMM.

Wi-Fi Direct: The device must successfully pass the current WFA certification for Wi-Fi Direct.

Exceptions: Not Specified

Business Justification:

To support WLAN functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: 06/01/2009

Comments:

NETWORK - 0224

Device.Network.WLAN.CSBBase.PermitIEToRequestAndResponseAF

Target Feature: Device.Network.WLAN.CSBBase

Title: WLAN devices that go into systems that support connected standby MUST permit addition of Information Elements to request and response association frames.

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Client ARM

Description:

All 802.11 devices MUST permit Information Elements to be added to association frames, both requests and responses. This includes adding currently specified Information Elements, such as Wi-Fi Protected Setup as well as other vendor extended Information Elements.

Exceptions: Not Specified

Business Justification:

To support WLAN functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.WLAN.CSBBase.SupportFiltering32MulticastAddresses

Target Feature: Device.Network.WLAN.CSBBase

Title: WLAN devices that go into systems that support connected standby MUST support filtering for at least 32 multicast addresses on each port.

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Client ARM

Description:

WLAN hardware MUST support at least 32 multicast addresses on each port. Both STA and Wi-Fi-Direct ports need to support filtering 32 multicast addresses separately.

Exceptions: Not Specified

Business Justification:

To support WLAN functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.WLAN.CSBBase.SupportIEEE80211w

Target Feature: Device.Network.WLAN.CSBBase

Title: WLAN devices that go into systems that support connected standby MUST Support IEEE 802.11w standard for protected management frames.

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Client ARM

Description:

IEEE 802.11w is an addition to IEEE 802.11 suite of standards to enhance the security of management frames. All WLAN devices must support the IEEE 802.11w standard.

Exceptions: Not Specified

Business Justification:

To support WLAN functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.WLAN.CSBBase.SupportPromiscuousAndMulticastPacketFiltering

Target Feature: Device.Network.WLAN.CSBBase

Title: WLAN devices that go into systems that support connected standby MUST support promiscuous and multicast packet filtering.

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Client ARM

Description:

WLAN device and driver MUST support promiscuous and multicast packet filtering. The miniport driver MUST support all filter types identified in the Windows Driver Kit. By default, multicast promiscuous mode is not enabled.

Exceptions: Not Specified

Business Justification:

To support WLAN functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.WLAN.CSBBase.SupportSeparateBeaconAndProbeIE

Target Feature: Device.Network.WLAN.CSBBase

Title: WLAN devices that go into systems that support connected standby MUST support separate beacon and probe Information Elements.

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Client ARM

Description:

All 802.11 devices MUST separately indicate the Wi-Fi Protected Setup IEs that are received in Beacon frames and probe-response frames. If the device has received both a beacon frame and a probe-response frame from a particular BSSID, then it MUST provide two instances of the IE, where one instance is the most recently received WPS IE from the Beacon and one instance is the most recently received WPS IE from the probe response.

Exceptions: Not Specified

Business Justification:

To support WLAN functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.WLAN.CSBBase.SupportVirtualWiFi

Target Feature: Device.Network.WLAN.CSBBase

Title: WLAN devices that go into systems that support connected standby MUST support Virtual Wi-Fi.

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64

Windows 8 Client ARM

Description:

All 802.11 devices MUST support Virtual Wi-Fi and enable simultaneous infrastructure STA connection and Soft AP hosting OR simultaneous infrastructure STA connection and Wi-Fi Direct ports. The Virtual Wi-Fi interface is specified in the Extensible WLAN driver specification document.

Exceptions: Not Specified

Business Justification:

To support WLAN functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.WLAN.CSBBase.SupportWiFiAutoSaveMode

Target Feature: Device.Network.WLAN.CSBBase

Title: WLAN devices that go into systems that support connected standby MUST Support Wi-Fi Auto Power Save Mode.

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Client ARM

Description:

The Wi-Fi driver is required to perform detection and negotiation of proper Wi-Fi Power Save Mode (PSM) between the device and the Wi-Fi Access Point and report it in driver capability during initialization in DOT11_EXTSTA_ATTRIBUTES. If the driver reports that it supports PSM detection, WLAN service will delegate the PSM decision to the driver by default.

If the driver supports the AUTO-PSM capability the Wi-Fi service will no longer set the broadcast management filter to receive beacons from the miniport and will instead set an OID to turn on Auto-PSM in the driver. When in Auto-PSM, the driver should always negotiate PSM mode when it detects that the AP supports it and manage the use of PSM between the device and the Wi-Fi Access Point to ensure optimal connectivity while using the least amount of power.

Exceptions: Not Specified

Business Justification:

To support WLAN functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.WLAN.CSBBase.TransmitPacketsOnAnyBoundary

Target Feature: Device.Network.WLAN.CSBBase

Title: WLAN devices that go into systems that support connected standby MUST be able to transmit packets from buffers aligned on any boundary.

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Client ARM

Description:

Buffer alignment refers to whether a buffer begins on an odd-byte, word, double-word, or other boundary. Devices MUST be able to transmit packets with any of the packets fragments beginning on an odd-byte boundary. For performance reasons, packets MUST be received into contiguous buffers on a double-word boundary.

Exceptions: Not Specified

Business Justification:

To support WLAN functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.WLAN.CSBNLO

Description: Not Specified

Related Requirements:

Device.Network.WLAN.CSBNLO.SupportNetworkListOffload

Device.Network.WLAN.CSBNLO.SupportNetworkListOffload

Target Feature: Device.Network.WLAN.CSBNLO

Title: WLAN devices that go into systems that support connected standby MUST support Network List Offloads.

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Client ARM

Description:

WLAN devices MUST support 10 offloaded BSSID profiles. Wi-Fi profiles that are marked auto-connect will be offloaded by the OS to the device. Wi-Fi profiles that are marked auto-connect will be offloaded by the OS to the device. The device should not indicate any new networks to the OS unless it matches the offloaded profiles. The device should also use the network list offload as a hint to optimize scan behaviors and return NDIS_STATUS_DOT11_OFFLOAD_NETWORK_STATE_CHANGED indication when a matching profile is found.

Exceptions: Not Specified

Business Justification:

To support WLAN functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.WLAN.CSBSoftAP

Description:

Wireless LAN

Related Requirements:

Device.Network.WLAN.CSBSoftAP.SupportSoftAP

Device.Network.WLAN.CSBSoftAP.SupportSoftAP

Target Feature: Device.Network.WLAN.CSBSoftAP

Title: WLAN devices that go into systems that support connected standby MUST support Soft AP.

Applicable OS Versions:

Windows 8 Client x86

Windows 8 Client x64
Windows 8 Client ARM

Description:

All 802.11 devices MUST support the Soft AP application with 8 simultaneously associated stations using security (WPA2). All 802.11 devices MUST support the Soft AP by permitting extension of Information Elements in Beacon and Probe Response frames. This includes adding currently specified Information Elements, such as Wi-Fi Protected Setup as well as other vendor extended Information Elements. All 802.11 devices MUST support the Soft AP by permitting stations associated to the Soft AP to utilize power save and providing them with beacon notification to wake up and receive waiting data.

Exceptions: Not Specified

Business Justification:

To support WLAN functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.WLAN.CSBWiFiDirect

Description:

Wireless LAN

Related Requirements:

Device.Network.WLAN.CSBWiFiDirect.SupportAtLeast2WiFiDirectPortsConcurrently
Device.Network.WLAN.CSBWiFiDirect.SupportAtLeast4Clients

Device.Network.WLAN.CSBWiFiDirect.SupportAtLeast2WiFiDirectPortsConcurrently

Target Feature: Device.Network.WLAN.CSBWiFiDirect

Title: WLAN Devices that go into systems that support connected standby MUST support at least 2 Wi-Fi Direct role ports concurrently.

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Client ARM

Description:

WLAN Device must be capable of supporting at least 2 Wi-Fi Direct role ports concurrently in the following configurations in addition to the Wi-Fi Direct device port:

A Group Owner (GO) on one Wi-Fi Direct port and Client on the other Wi-Fi Direct port(s) concurrently.

A Client on each Wi-Fi Direct Port concurrently.

These ports must be supported concurrently with Infrastructure connectivity on different channels.

Exceptions: Not Specified

Business Justification:

To support WLAN functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.WLAN.CSBWiFiDirect.SupportAtLeast4Clients

Target Feature: Device.Network.WLAN.CSBWiFiDirect

Title: WLAN Devices that go into systems that support connected standby MUST support at-least 4 clients being connected simultaneously to the Wi-Fi Direct Group Owner on the Device.

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Client ARM

Description:

WLAN Device must be able to support at-least 4 clients being connected simultaneously to the running Wi-Fi Direct Group Owner on the Device.

Exceptions: Not Specified

Business Justification:

To support WLAN functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.WLAN.CSBWoWLAN

Description:

Wireless LAN

Related Requirements:

Device.Network.WLAN.CSBWoWLAN.MustSupportWakeOnWLAN

Device.Network.WLAN.CSBWoWLAN.MustSupportWakeOnWLAN

Target Feature: Device.Network.WLAN.CSBWoWLAN

Title: WLAN devices that go into systems that support connected standby MUST support Wake on Wireless LAN.

Applicable OS Versions:

Windows 8 Client ARM
Windows 8 Client x86
Windows 8 Client x64

Description:

WLAN devices must support Wake on Wireless LAN (WoWLAN) capability. Partial wake implementations (subset of below list) will not be considered for certification. WLAN devices should do the following:

MUST indicate the specific Wake on Wireless LAN (WoWLAN) capability that is supported.

MUST support wake on Magic Packet. A magic packet is a packet that contains 16 contiguous copies of the receiving NIC's MAC address.

MUST support at least 16 WoWLAN bitmap wake-up patterns of 128 byte each.

MUST be able to perform GTK (WPA/WPA2) and IGTK refresh (WPA2) while in the D3 state.

MUST support wake on GTK and IGTK handshake error.

MUST support wake when 802.1x EAP-Request/Identity Packet is received.

MUST support wake when four way handshake request is received.

MUST support wake on association lost with current AP.

MUST support wake when a network matches NLO (Network list offload) hints.

MUST support wake packet indication. NIC should cache the packet causing the wake on hardware and pass it up when the OS is ready for receives.

MUST support ARP and NS offloads to ensure link local network discovery. WLAN device should be able to respond to ARP and NS requests without interrupting the CPU when the device is in low power (D3) state. Devices must support at least 1 ARP offload and at least 2 NS offloads.

Exceptions: Not Specified

Business Justification:

To support WLAN functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.WLAN.NLO

Description: Not Specified

Related Requirements:

Device.Network.WLAN.NLO.SupportNetworkListOffload

Device.Network.WLAN.NLO.SupportNetworkListOffload

Target Feature: Device.Network.WLAN.NLO

Title: WLAN devices MUST support Network List Offload.

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64

Description:

WLAN devices MUST support 10 offloaded BSSID profiles. It is recommended to implement this feature in firmware, but if the device is incapable of supporting it in firmware, its ok to support it in driver. Wi-Fi profiles that are marked auto-connect will be offloaded by the OS to the device/driver. The device should use the network list offload as a hint to optimize scanning behavior and return NDIS_STATUS_DOT11_OFFLOAD_NETWORK_STATE_CHANGED indication when a matching profile is found.

Exceptions: Not Specified

Business Justification:

To support WLAN functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.WLAN.SoftAP

Description:

Wireless LAN

Related Requirements:

Device.Network.WLAN.SoftAP.SupportSoftAP

Device.Network.WLAN.SoftAP.SupportSoftAP

Target Feature: Device.Network.WLAN.SoftAP

Title: WLAN devices MUST support Soft AP.

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 7 Client x86
Windows 7 Client x64

Description:

All 802.11 devices MUST support the Soft AP application with 8 simultaneously associated stations using security (WPA2). All 802.11 devices MUST support the Soft AP by permitting extension of Information Elements in Beacon and Probe Response frames. This includes adding currently specified Information Elements, such as Wi-Fi Protected Setup as well as other vendor extended Information Elements. All 802.11 devices MUST support the Soft AP by permitting stations associated to the Soft AP to utilize power save and providing them with beacon notification to wake up and receive waiting data.

Exceptions: Not Specified

Business Justification:

To support WLAN functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.WLAN.WiFiDirect

Description:

Wireless LAN

Related Requirements:

Device.Network.WLAN.WiFiDirect.SupportAtLeast2WiFiDirectPortsConcurrently
Device.Network.WLAN.WiFiDirect.SupportAtLeast4Clients

Device.Network.WLAN.WiFiDirect.SupportAtLeast2WiFiDirectPortsConcurrently

Target Feature: Device.Network.WLAN.WiFiDirect

Title: WLAN Devices MUST support at least 2 Wi-Fi Direct role ports concurrently.

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64

Description:

WLAN Device must be capable of supporting at least 2 Wi-Fi Direct role ports concurrently in the following configurations in addition to the Wi-Fi Direct device port:

A Group Owner (GO) on one Wi-Fi Direct port and Client on the other Wi-Fi Direct port(s) concurrently.

A Client on each Wi-Fi Direct Port concurrently.

These ports must be supported concurrently with Infrastructure connectivity on different channels

Exceptions: Not Specified

Business Justification:

To support WLAN functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.WLAN.WiFiDirect.SupportAtLeast4Clients

Target Feature: Device.Network.WLAN.WiFiDirect

Title: WLAN Devices MUST support at-least 4 clients being connected simultaneously to each Wi-Fi Direct Group Owner on the Device.

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64

Description:

WLAN Device must be able to support at-least 4 clients being connected simultaneously to each running Wi-Fi Direct Group Owner on the Device.

Exceptions: Not Specified

Business Justification:

To support WLAN functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Network.WLAN.WoWLAN

Description:

Wireless LAN

Related Requirements:

Device.Network.WLAN.WoWLAN.ImplementWakeOnWLAN

Device.Network.WLAN.WoWLAN.ImplementWakeOnWLAN

Target Feature: Device.Network.WLAN.WoWLAN

Title: WLAN devices that implement Wake on Wireless LAN MUST support Wake on Wireless LAN.

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64

Description:

WLAN devices that implement WoWLAN (Wake on Wireless LAN) must support WoWLAN capability. Implementation of subset of below features will not be considered for certification. WLAN devices should do the following:

MUST indicate the specific Wake on Wireless LAN (WoWLAN) capability that is supported.

MUST support wake on Magic Packet. A magic packet is a packet that contains 16 contiguous copies of the receiving NIC's MAC address.

MUST support at least 16 WoWLAN bitmap wake-up patterns of 128 byte each.

MUST be able to perform GTK (WPA/WPA2) and IGTK refresh (WPA2) while in the D3 state.

MUST support wake on GTK and IGTK handshake error.

MUST support wake when 802.1 x EAP-Request/Identity packets is received. .

MUST support wake when four way handshake request is received.

MUST support wake on association lost with current AP.

MUST support wake when a network matches NLO (Network list offload) hints.

MUST support wake packet indication. NIC should cache the packet causing the wake on hardware and pass it up when the OS is ready for receives.

MUST support ARP and NS offloads to ensure link local network discovery. WLAN device should be able to respond to ARP and NS requests without interrupting the CPU when the device is in low power (D3) state. Devices must support at least 1 ARP offload and at least 2 NS offloads.

Exceptions: Not Specified

Business Justification:

To support WLAN functionality and scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments: Not Specified

Device.Portable.Core

Description:

Core

Related Requirements:

Device.Portable.Core.AudioCodec
Device.Portable.Core.CustomDeviceServices
Device.Portable.Core.DeviceServices
Device.Portable.Core.MediaSync
Device.Portable.Core.ModelID
Device.Portable.Core.MTP
Device.Portable.Core.MTPFunctionality
Device.Portable.Core.MTPMultiSession
Device.Portable.Core.MTPObjectProperties
Device.Portable.Core.MTPStreams
Device.Portable.Core.TransportBluetooth
Device.Portable.Core.TransportIP
Device.Portable.Core.TransportIPDLNA
Device.Portable.Core.TransportUSB
Device.Portable.Core.VideoCodec

Device.Portable.Core.AudioCodec

Target Feature: Device.Portable.Core

Title: If a Portable Device can capture audio content, it must do so using a format supported natively in Windows

Applicable OS Versions:

Windows 8 Client x64
Windows 8 Client x86
Windows 7 Client x86
Windows 7 Client x64

Description:

If a Portable Device can capture audio content, it must do so in a format that Windows understands natively.

The following tables represent the list of in-box formats that Windows will render to; this is not the exhaustive of supported formats in Windows.

Note that for a given format in the tables below it is not required to support all given Bit rates and Sample rates, it is however required to support a format that lines up within the ranges provided.

MP4 Audio Content

	Setting	**Requirement**

AAC	Codec	AAC Standard, AAC Profile level 2 minimum (0x29), compatible with (0x29, 0x2A, 0x2B, 0x2C, 0x2E, 0x2F, 0x30, 0x31, 0x32, 0x33)
	bit rate for CBR	96, 128, 160, 192Kbps
	Sample rate	44.1 or 48kHz
	Channels	2

MP3 Audio Content

	Setting	**Requirement**
MP3	Codec	Version1, Layer3
	Bit rate for CBR files	From 32 to 320kilobits per second (Kbps)
	Sample rate	16, 22.05, 24, 32, 44.1, 48kilohertz (kHz)
	Channels	2

WMA Audio Content

	Setting	**Requirement**
WMA Standard	Codec	WMA9 or later
	Bit rate for CBR files	From 32 to 256kilobits per second (Kbps)
	Average bit rate for VBR files	From 48 to 160Kbps
	Maximum peak bit rate for VBR files	256Kbps
	Sample rate	44.1 or 48kilohertz (kHz)
	Channels	2

Exceptions: Not Specified

Business Justification:

This requirement ensures that end users running Windows have a successful experience extracting media content from the portable device without having to install 3rd party codecs (which can be hard to identify and download).

Scenarios:

This requirement ensures that end users running Windows have a successful experience extracting media content from the portable device without having to install 3rd party codecs (which can be hard to identify and download).

Success Metric: Pass/Fail

Enforcement Date: June 01, 2006

Comments:

PORTABLE-0009, PORTABLE-0067

Device.Portable.Core.CustomDeviceServices

Target Feature: Device.Portable.Core

Title: Portable device that implements custom MTP Services meets requirements defined in the MTP Devices Services Extension Specification

Applicable OS Versions:

- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x64

Description:

The MTP Device Services Extension to the Media Transfer Protocol (MTP) helps an MTP Initiator, in this case Windows, find and access certain types of content stored on the device. Extension mechanisms have been defined that provide greater flexibility for applications that deal with specific content types defined by the device. These mechanisms provide greater extensibility than the existing data code mechanisms currently defined by the Media Transfer Protocol Specification, Revision 1.0.

If the portable device supports a custom device service the implementation must have a valid *ServiceInfo* dataset, a valid *ServiceCapabilities* dataset, and a valid *ServicePropertiesDesc* dataset as defined in the MTP Device Services Extension Specification. All mandatory properties defined in the specification must be supported in the custom service.

The following table is a list of operations that are required when implementing MTP Services:

Operation	MTP Datacode	Description
GetServiceIDs	0x9301	This operation returns an array of ServiceIDs.
GetServiceInfo	0x9302	This operation returns the ServiceInfo dataset for a service.
GetServiceCapabilities	0x9303	All object format and method format information is reported by using the GetServiceCapabilities operation.
GetServicePropDesc	0x9304	This operation returns theServicePropertyDesc dataset for a service.
GetServicePropList	0x9305	This operation is similar to GetObjectPropList in the MTP specification, Revision 1.0. GetServicePropList reads properties

		from a service.
SetServicePropList	0x9306	This operation sets a ServiceProperty by using the ServicePropList dataset. It enables the writing of property values to a service.
UpdateObjectPropList	0x9307	This operation sets the property list for a particular object that will be updated with a new binary object. This operation can be used to replace the binary data of an existing object.
DeleteObjectPropList	0x9308	This operation removes the properties that are specified in the DeleteObjectPropList dataset from the specified object or objects.
DeleteServicePropList	0x9309	This operation removes the properties that are specified in the DeleteServicePropList dataset from the specified service.

If scenarios that can be implemented using capabilities defined in the MTP specification are not implemented using the operations, device properties, etc. defined in the MTP 1.0 specification then the device must define these scenarios in terms of device services and must support these services according to the requirements defined in the MTP Device Services Extension Specification. For example, a media exchange service may be defined to manage media synchronization between the initiator and responder which replaces functionality supported by standard MTP 1.0 behavior.

To expose a custom device service in Device Stage, a custom task must be authored and defined as part of the Device Stage authoring process. This is described in the Microsoft Device Experience Development Kit.

Design Notes:

Refer to the MTP Device Services Extension Specification available at http://go.microsoft.com/fwlink/?LinkID=198516.

Refer to the Microsoft Device Experience Development Kit available at http://go.microsoft.com/fwlink/?LinkID=198537.

Exceptions: Not Specified

Business Justification:

Partners who wish to use the Microsoft MTP Class driver have a way to provide value add functionality by leveraging the class driver as a pass-through for their custom Device Services. This requirement provides guidance on the correct way to implement custom Device Services.

Scenarios:

Partners who wish to use the Microsoft MTP Class driver have a way to provide value add functionality (e.g. Firmware revision information, media pass through, etc) can use this capability without having to update the MTP class driver.

Success Metric: Pass/Fail

Enforcement Date: June 01, 2009

Comments:

PORTABLE-0035

Device.Portable.Core.DeviceServices

Target Feature: Device.Portable.Core

Title: Portable devices that support defined MTP Services implement these services according to the requirements defined in the MTP Device Services for Windows Specification

Applicable OS Versions:

Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client x64

Description:

The MTP Device Services architecture enables Windows to locate and use various services and content types located on a device. By creating extensions to the WPD API and MTP protocol, Windows can locate, consume, and interact with useful content and services on the device.

Supported Personal Information Management (PIM) Services:

Contact Service

Calendar Service

Notes Service

Tasks Service

Other Supported Services:

Status Service

Hints Service

Device Metadata Service

Ringtones Service

If the device supports one or more of the PIM services it must also support one of two synchronization services to actually synchronize the data. Windows supports the following in-box:

Enumeration Sync

Anchor Sync

It is up to the manufacturer to choose the services that is best suited for the device.

Vendors who choose to support custom Device Stage metadata packages, should ensure that the appropriate tasks or device properties are exposed correctly within the package.

For services to be accessible by a service aware initiator the following service related operations must also be supported by the device:

Operation	MTP Datacode	Description
GetServiceIDs	0x9301	This operation returns an array of ServiceIDs.
GetServiceInfo	0x9302	This operation returns the ServiceInfo dataset for a service.
GetServiceCapabilities	0x9303	All object format and method format information is reported by using the GetServiceCapabilities operation.
GetServicePropDesc	0x9304	This operation returns theServicePropertyDesc dataset for a service.
GetServicePropList	0x9305	This operation is similar to GetObjectPropList in the MTP specification, Revision 1.0. GetServicePropList reads properties from a service.
SetServicePropList	0x9306	This operation sets a ServiceProperty by using the ServicePropList dataset. It enables the writing of property values to a service.
UpdateObjectPropList	0x9307	This operation sets the property list for a particular object that will be updated with a new binary object. This operation can be used to replace the binary data of an existing object.
DeleteObjectPropList	0x9308	This operation removes the properties that are specified in the DeleteObjectPropList dataset from the specified object or objects.
DeleteServicePropList	0x9309	This operation removes the properties that are specified in the DeleteServicePropList dataset from the specified service.

Design Notes:

For information on defined MTP Services refer to the MTP Device Services for Windows Specification available at http://msdn.microsoft.com/en-us/windows/hardware/gg463544.

For information on service operations and general services details refer to the MTP Device Services Extension Specification available at http://msdn.microsoft.com/en-us/windows/hardware/gg463545.

See also Metadata Schema and Package Format Specification available at http://msdn.microsoft.com/en-us/windows/hardware/gg463153.

Exceptions: Not Specified

Business Justification:

We wish to ensure that portable devices expose the basic Device Services to ensure a basic experience with ISV applications.

Scenarios:

Windows components, such as Device Stage, and 3rd party applications have access to the rich set of MTP Services supported by devices which are exposed via the Windows Portable Devices platform. End users can take advantage of this capability and are able to manage PIM related data on the

device including synchronizing contacts, calendar, tasks, notes, etc. with their other applications running on Windows.

Success Metric: Pass/Fail

Enforcement Date: June 01, 2009

Comments:

PORTABLE-0034, PORTABLE-0045

Device.Portable.Core.MediaSync

Target Feature: Device.Portable.Core

Title: Portable Devices that support media content must meet basic interoperability requirements to successfully transfer content with an MTP aware media player application on Windows

Applicable OS Versions:

- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86

Description:

An MTP based device must be able to transfer data to and from the PC using a Windows based application that supports MTP.

Note that if the device replaces core MTP media handling functionality with a custom MTP Service then interoperability with Windows Media Player will not function as expected. Devices that implement a custom service must have functional parity with core MTP operations to support media transfer and synchronization with a WPD-based application.

The following requirements must be met when interacting with Windows Media Player:

Supports digital media transfers from device to computer - The device must support digital media transfers to computers through applications that enable such transfers, such as the Portable Device Shell Namespace Extension and Windows Media Player.

Supports digital media transfers from computer to device - The device must support transfer of supported media formats from the computer to the device through Windows Media Player. The device must support transfer of all file formats from the computer to the device through the Windows Namespace Extension.

Supports metadata updates -The device must support updates to MTP metadata after the original track has been copied to the device. The metadata updates are performed using Windows Media Player.

Supports cancellation of transfer and synchronization - The device must support cancellations of transfers and synchronizations mid-task without crashing or hanging of the device.

Properly identifies file types - The device must not rely on the file name extension of a file to discover the file type. The device can use the MTP object format or can parse the file contents to determine the file type.

Provide device object metadata - If a host sends an empty value for Album, Artist, or Title, the device must display meaningful text in place of the empty metadata and allow the user to find the content when searching on the associated field in the device UI. For example, a device might use Unknown Artist for the artist when it receives an empty value for artist metadata. If the device supports displaying genre metadata, then if a host sends an empty value for genre, the device must display meaningful text in place of the empty metadata and allow the user to find the content when searching on the associated field in the device UI.

Playlist transfer - The device must support the transfer of playlists manually created by the user using Windows Media Player. When a manual playlist is transferred using Windows Media Player, both the playlist and the content the playlist references must reside on the device.

To receive playlists from Windows Media Player, a device must support object references. If object references are supported, they must be supported on all formats. Support for object references is indicated by supporting the following commands:

GetObjectReferences (0x9810)

SetObjectReferences (0x9811)

The device must also support the AbstractAudioVideoPlaylist (0xBA05) format. A playlist is sent as a 0-byte object with a list of references to object handles for all objects contained in the playlist.

Exceptions: Not Specified

Business Justification:

This helps meet basic interoperability with an MTP aware media player application on Windows

Scenarios:

An end-user can connect their device to Windows and begin managing their media with native Windows applications such as Windows Media Player or their favorite 3rd party WPD-based media management application.

Success Metric: Not Specified

Enforcement Date: 6/1/2009

Comments:

PORTABLE-0058

Device.Portable.Core.ModelID

Target Feature: Device.Portable.Core

Title: Portable Devices may support the optional ModelID property to uniquely identify logical device functions on a multi-function device

Applicable OS Versions:

Windows 8 Client x64
Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client x86

Description:

Plug and Play includes support for a DevNode property called DEVPKEY_Device_ModelId. This key is used to store the ModelID value that some devices will store in a device-class-specific or manufacturer-specific manner on the device.

The ModelID spans logical device functions on a multi-function device through an internal structure in Windows 7 called the Display Object that aggregates logical devices in a single piece of plastic representation. The ModelID can be as specific or as generic as the manufacturer chooses. For example, ModelID may differ between product models, colors of an individual model, or even individual devices. To support ModelID the device property code 0xD302 along with required properties must be supported.

Design Notes:

Refer to the MTP Device Services Extension Specification that is included with the Windows Portable Device Enabling Kit, available at http://msdn.microsoft.com/en-us/windows/hardware/gg463545.

Exceptions: Not Specified

Business Justification:

Ensure support for ModelID information (needed by Plug and Play to group device functions together)

Scenarios:

The use of ModelID allows Windows to always identify the device as a single piece of plastic.

Success Metric: Pass/Fail

Enforcement Date: 6/1/2009

Comments:

PORTABLE-0054, PORTABLE-0055, PORTABLE-0056

Device.Portable.Core.MTP

Target Feature: Device.Portable.Core

Title: Portable devices must support a core set of MTP operations and devices properties as defined in the Media Transport Protocol revision 1.0 or later, along with device properties and object formats for specific device type.

Applicable OS Versions:

Windows 8 Client x64
Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client x86

Description:

Media Transfer Protocol (MTP) is an extension of the Picture Transfer Protocol (ISO 15740). Both specifications include optional commands, operations, etc.but some of these are required for certification. There is a core set of operations and device properties that must be met by each device category in order to be compliant with Windows. Implementation details for MTP are defined in the Media Transfer Protocol Specification, Revision 1.0 and will be used as the reference for certification of all portable devices as part of the Windows Hardware Certification Program.

Supports MTP drivers

A Portable Device must work with the inbox MTP drivers that ship with Windows. This means that they must install and work immediately without requiring the installation of additional drivers or software.

Supports MTP by default

Portable Devices must ship with MTP enabled as the default connection mode. Devices can support Mass Storage Class (MSC) in addition to MTP.

The device must not have a persistent, user accessible setting to enable or disable the MTP or MSC mode.

All storage volumes on device (both internal and external) must be accessible using the MTP protocol.

A device can have a safe mode that allows the user to boot to MSC mode for device recovery scenarios. After the user disconnects a device connected in safe mode, the device must resume normal operation.

Device Information Dataset

The device must support the MTP DeviceInfo dataset. The following table describes field-specific requirements that must be implemented:

Dataset Field	Requirement	Notes
Standard	R	This identifies the PTP version this device can support in hundredths. For

Version		MTP devices this shall contain the value 100 (representing 1.00).
MTP Vendor Extension ID	R	This identifies the MTP vendor-extension version in use by this device.
MTP Version	R	This identifies the version of the MTP standard this device supports. For MTP devices implemented under MTP 1.0 this shall contain the value 100 (representing 1.00). For MTP devices implemented under MTP 2.0 this shall contain the value 200.
MTP Extensions	I	This string is used to identify any extension sets applied to MTP.
Functional Mode	R	Modes allow the device to express different states with different capabilities. If the device supports only one mode, this field shall contain the value 0x00000000. See MTP spec for details.
Operations Supported	R	This field identifies by datacode all operations that this device supports in the current functional mode.
Events Supported	I	This field identifies by datacode all events that this device can generate in the current functional mode.
Device Properties Supported	R	This field identifies by datacode all device properties that this device supports in the current functional mode.
Capture Formats	I	This field identifies by datacode the object format codes for each format that this device can generate independently (that is, without the content being placed on the device).
Playback Formats	I	This field identifies by datacode the object format codes for each format that this device can understand and parse if placed on the device. If the device can carry unidentified binary objects without understanding them, it shall indicate this by including the Undefined Object (0x3000) code in its Playback Formats.
Manufacturer	R	The MTP specification identifies this as anoptional string, it is required for Hardware Certification. This field contains a human-readable string identifying the manufacturer of this device.
Model	R	The MTP specification identifies this as an optional string, it is required for Hardware Certification. This field contains a human-readable string identifying the model of this device.

Device Version	R	The MTP specification identifies this as an optional string, it is required for Hardware Certification. This field contains a human-readable string identifying the software or firmware version of this device.
Serial Number	R	A serial number must be unique among all devices sharing identical Model and Device Version fields.

Where:

R = Required

I = If Implemented (Optional)

N/A = Not Applicable

Supports Control Requests

Control requests enable an MTP initiator to send out-of-band requests to the MTP responder. The device must support the following requests.

Get Status

Cancel Request

Reset Device Request

Required Operations

This core set of MTP operations and device properties that must be met by each portable device in order to be compliant with Windows. Implementation details for each operation and device property are defined in the Media Transfer Protocol Specification, Revision 1.0 or later. The highlighted operations represent the core set of MTP operations required for all portable devices.

	Property Code	**Mobile Phone**	**Media Player**	**Digital Camera**	**Digital Video Camera**	**Other (Base)**
GetDeviceInfo	0x1001	R	R	R	R	R
OpenSession	0x1002	R	R	R	R	R
CloseSession	0x1003	R	R	R	R	R
GetStorageIDs	0x1004	R	R	R	R	R
GetStorageInfo	0x1005	R	R	R	R	R
GetNumObjects	0x1006	R	R	R	R	R
GetObjectHandles	0x1007	R	R	R	R	R

GetObjectInfo	0x1008	R	R	R	R	R
GetObject	0x1009	R	R	R	R	R
GetDevicePropDesc	0x1014	R	R	R	R	R
GetDevicePropValue	0x1015	R	R	R	R	R
DeleteObject	0x100B	R	R	R	R	I
SetDevicePropValue	0x100A	R	R	I	I	I
SendObjectInfo	0x100C	R	R	I	I	I
SendObject	0x100D	R	R	I	I	I
GetPartialObject	0x101B	R	R	I	I	I
GetObjectPropsSupported	0x9801	R	R	I	I	I
GetObjectPropDesc	0x9802	R	R	I	I	I
GetObjectPropValue	0x9803	R	R	I	I	I
SetObjectPropValue	0x9804	R	R	I	I	I
GetObjectReferences	0x9810	R	R	I	I	I
SetObjectReferences	0x9811	R	R	I	I	I

See **Operations** in the MTP Specification, Revision 1.0 or later for complete list of defined MTP Operations.

The following operations are highly recommended, though not required:

FormatStore (0x100F)

GetObjectPropList (0x9805)

SetObjectPropList (0x9806)

GetInterDependentPropDesc (0x9807)

SendObjectPropList (0x9808) For this command, your device must also support specification of destination, which allows the MTP initiator to dictate a parent handle for that object.

Required Events

The following events must be supported for all objects:

ObjectAdded (0x4002)

ObjectRemoved (0x4003)

If the device supports removable media, it must also support the following events.

StoreAdded (0x4004)

StoreRemoved (0x4005)

Operation Responses

An appropriate response must be returned for any and all operations. If an error is encountered error response codes are also defined and must be provided by the device. For more information, see section 4.8 of the MTP specification, "Responses."

Required Device Properties

	Property Code	Mobile Phone	Media Player	Digital Camera	Digital Video Camera	Other (Base)
Battery Level	0x5001	I	I	I	I	I
Synchronization Partner	0xD401	I	I	I	I	I
Device Friendly Name	0xD402	R	R	I	I	I
Device Icon	0xD405	I	I	I	I	I

See **Device Properties** in the MTP Specification, Revision 1.0 or later for complete list of defined Device Properties.

Device Icon is not required but strongly recommended. High resolution images of the device can be used and exposed in Windows UI.

Object Formats

The following table represents the list of required object formats for a portable device. Each object format also has a list of object properties that must be supported per object type. The highlighted object format is required for all MTP devices and is considered a core MTP requirement for Windows.

	Property Code	Mobile Phone	Media Player	Digital Camera	Digital Video Camera	Other (Base)
Undefined	0x3000	I	I	I	I	I
Association	0x3001	R	R	R	R	R
Abstract Audio Album	0xBA03	I(2)	I(2)	N/A	N/A	I(2)
Abstract Audio & Video	0xBA05	I	I	N/A	N/A	I

Playlist						
WAV	0x3008	R(1)	R(1)	I	I	I
MP3	0x3009	R(1)	R(1)	I	I	I
AVI	0x300A	R(1)	R(1)	I	R(1)	I
MPEG	0x300B	R(1)	R(1)	I	R(1)	I
ASF	0x300C	R(1)	R(1)	I	R(1)	I
EXIF/JPEG	0x3801	I	I	R(1)	I	I
TIFF/EP	0x3802	I	I	R(1)	I	I
BMP	0x3804	I	I	R(1)	I	I
JPEG XR	0xB804	I	I	R(1)	I	I
WMA	0xB901	R(1)	R(1)	I	I	I
AAC	0xB903	R(1)	R(1)	I	I	I
WMV	0xB981	I	I	I	R(1)	I
MP4 Container	0xB982	I	I	I	R(1)	I
3GP Container	0xB984	I	I	I	R(1)	I
3G2	0xB985	I	I	I	R(1)	I
AVCHD	0xB986	I	I	I	R(1)	I

(1) Portable devices that are capable of playing audio and/or video must support at least one of these formats. This table does not represent the complete list of supported formats; it represents the list of commonly used formats.

See **Object Formats** in the MTP Specification, Revision 1.0 or later for complete list of defined Device Properties.

(2)Support for the following object properties is mandatory for AbstractAudioAlbum objects:

Genre (0xDC8C)

Album Artist (0xDC9B)

WMPMetadataRoundTrip (0x9201)

A device that supports both the AbstractAudioAlbum format and an image format may optionally support album art. Album art is attached to an AbstractAudioAlbum object by adding the art to the album's RepresentativeSampleData property.

If a Portable Device supports album art, then the device must support the following object properties:

PurchaseFlag (0xd901), an object property in the Windows Media DRM 10 for Portable Devices MTP Extensions

RepresentativeSampleFormat (0xdc81)

RepresentativeSampleSize (0xdc82)

RepresentativeSampleHeight (0xdc83)

RepresentativeSampleWidth (0xdc84)

RepresentativeSampleData (0xdc86)

User Rating (0xdc8a)

WMPMetadataRoundTrip (0x9201)

Design Notes:

"Picture Transfer Protocol (PTP) for Digital Still Photography Devices," Version 1.0 of the PIMA15740: 2000 Picture Transfer Protocol specification http://go.microsoft.com/fwlink/?LinkId=237005.

The Media Transfer Protocol Specification, Revision 1.0 is available at http://www.usb.org/developers/devclass_docs/MTP_1.0.zip .

Additional implementation details can be found in the Windows 7 Portable Device Enabling Kit for MTP, which is available at http://go.microsoft.com/fwlink/?LinkID=198551.

Exceptions: Not Specified

Business Justification:

Existing requirement to ensure that the device is compatible with the core MTP requirements.

Scenarios:

Existing requirement to ensure that the device is compatible with the core MTP requirements.

Success Metric: Pass/Fail

Enforcement Date: 6/1/2009

Comments:

PORTABLE-0059, PORTABLE-0031, PORTABLE-0041, PORTABLE-0062

Device.Portable.Core.MTPFunctionality

Target Feature: Device.Portable.Core

Title: A device that supports MTP must meet mandatory general functionality requirements to ensure expected behavior in Windows

Applicable OS Versions:

Windows 8 Client x64
Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client x86

Description:

Portable devices must behave according to the requirements defined below. The device must:

Supports transfer and storage of content - The device must allow the transfer and storage of digital content.

Persists all transferred content - The device must not report receiving content which it has not persisted.

Respond as expected - A Portable Device must respond when it receives an MTP *GetDeviceStatus* control request issued by a host.

Support unexpected device disconnects - An unexpected disconnect must not cause the device to stop responding (hang or crash) or to restart.

Exceptions: Not Specified

Business Justification:

Ensure compatibility with the Microsoft MTP Class Driver

Scenarios:

Ensure compatibility with the Microsoft MTP Class Driver

Success Metric: Pass/Fail

Enforcement Date: 6/1/2009

Comments:

PORTABLE-0006, PORTABLE-0007, PORTABLE-0057

Device.Portable.Core.MTPMultiSession

Target Feature: Device.Portable.Core

Title: Portable devices that enable MTP multisession functionality support required object session operations

Applicable OS Versions:

Windows 8 Client x64
Windows 8 Client x86

Description:

In order to properly support MTP multisession functionality and work as expected with a multisession aware Initiator the device must support the following session operations:

CreateSession

GetSessionResponderInfo

The RestrictSession operation is optional. This operation is supported in Windows and will honor restrictions that are defined in a responders RestrictSession dataset.

These operations are required in addition to the operations for basic session operation defined in the MTP 1.0 specification; those operations include OpenSession, CloseSession, TransactionID, SessionID, etc. There is no requirement for the minimum or maximum number of sessions that a device must support if multisession.

Design Notes:

For implementation details see the Media Transfer Protocol Specification Revision 2.0, available at http://www.usb.org/developers/devclass_docs.

Exceptions:

These operations must be supported if the device supports MTP Multisession over USB or IP. Support for Multisession is not required for Windows Hardware Certification, however supporting this feature is strongly recommended.

Business Justification:

This requirement is necessary in order to ensure that a portable device that supports MTP Multisession functionality does so in a way that ensures compliance with other Initiators that the device may interface with. MTP 2.0 is an industry standard specification that is being ratified by the USB organization and represents the standard way that this functionality should be supported.

Scenarios:

Support for MTP Multisession enables multiple applications to communicate with a portable device simultaneously. MTP 1.0 version devices are single session and therefore transactions between the Initiator and the Responder must occur asynchronously. Multisession capable devices alleviate this pipe contention issues by supporting multiple sessions. An application is not required to wait until another applications transaction (bulk media copy for example) has completed; the application is given its own session between the Initiator and Responder. Support for multisession is enabled in the IP and USB transports class drivers in Windows.

Success Metric: Not Specified

Enforcement Date: TBD

Comments:

New Requirement

Device.Portable.Core.MTPObjectProperties

Target Feature: Device.Portable.Core

Title: A MTP device must support object properties for each consumable media format

Applicable OS Versions:

- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86

Description:

The following device capabilities must be supported for each format reported by the device. If an MP4 container is supported, the device capability needs to include the proper properties in this entry. Based on this information, Windows can predict the proper transcode profile and transcode a file to the device which can then be played back on the device.

The following table summarizes Required (R) and recommended If Implemented(I) / Optional object property support for all object formats. This is not an exhaustive list of object properties available, for the complete list refer to the Object Format Summary Table in the MTP specification.

Object property	**MTP Datacode**	**Status**	**Description**
Storage ID	0xDC01	R	The storage area that the object is stored in.
Object Format	0xDC02	R	The data format for the object.
Protection Status	0xDC03	R	The protection status of the object.
Object Size	0xDC04	R	The size of the data component of the object, in bytes.
Object File Name	0xDC07	R	The file name of the object.
Date Created	0xDC08	I(1)	This property contains the date and time when the object was created.
Date Modified	0xDC09	I(1)	The date and time when the object was last altered.
Parent Object	0xDC0B	R	The parent object of the object.

Persistent Unique Object Identifier	0xDC41	R	The persistent unique object identifier of the object.
Name	0xDC44	R	The name of the object.
Non-Consumable	0xDC4F	R	Indicates whether the object was transferred to the portable media player for storage only and is, therefore, not available to be consumed (for example, played) by the portable device.

Where:

R = Required

I = If Implemented (Optional)

N/A = Not Applicable

(1) It is strongly recommended that Date Created and Date Modified be supported in order for applications to be able to distinguish which objects have been previously imported or synchronized. This is particularly important for photo acquisition scenarios.

The following table summarizes required and If Implemented (Optional) object property support for image, audio, and video objects. This is not an exhaustive list of all supported object properties, for a complete list refer to the Object Property Summary Table in the MTP Specification.

Object property	MTP Datacode	Image	Audio	Video
Artist	0xDC46	N/A	R	I
Description	0xDC48	N/A	I	I
Representative Sample Format	0xDC81	I	I	I
Representative Sample Size	0xDC82	I	I	I
Representative Sample Height	0xDC83	I	I	I
Representative Sample Width	0xDC84	I	I	I
Representative Sample Data	0xDC86	I	I	I
Width	0xDC87	R	N/A	R
Height	0xDC88	R	N/A	R
Duration	0xDC89	N/A	I	I
User Rating	0xDC8a	N/A	I	I
Track	0xDC8b	N/A	R	I
Genre	0xDC8c	N/A	I	I
Use Count	0xDC91	N/A	I	I
Parental Rating	0xDC94	N/A	I	I
Original Release Date	0xDC99	N/A	I	I
Album Name	0xDC9A	N/A	R	N/A
Album Artist	0xDC9B	N/A	R(2)	N/A
Bitrate Type	0xDE92	N/A	I	I
Sample Rate	0xDE93	N/A	R	R
Number of Channels	0xDE94	N/A	R	R
ScanType	0xDE97	N/A	I	R

Audio WAVE Codec	0xDE99	N/A	R	R
Audio Bitrate	0xDE9A	N/A	R	R
Video FourCC Codec	0xDE9B	N/A	N/A	R
Video Bitrate	0xDE9C	N/A	N/A	R
Frames Per Thousand Seconds	0xDE9D	N/A	N/A	R
Key Frame Distance	0xDE9E	N/A	N/A	I
Encoding Profile	0xDEA1	N/A	I	R

(2) Note that Album Artist (0xDC9B) is only required if the entire album is available, individual audio files only need to support artist (0xDC46).

Additionally, if the device supports an If Implemented (Optional) object property, the device must do so in conformance with the MTP specification and guidelines. Tests are designed to validate optional functionality if the device supports it.

Design Notes:

For implementation details refer to Media Transfer Protocol Specification Revision 1.0, Appendix B Object Properties available at http://www.usb.org/developers/devclass_docs.

Exceptions: Not Specified

Business Justification:

Object properties enable metadata that describes objects to be exchanged separately from the objects themselves. The primary benefit of this functionality is to permit the rapid enumeration of large storages (e.g. 8GB+), regardless of the file system. For each format that the device supports, the device must support all mandatory MTP object properties for that format.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2009

Comments:

PORTABLE-0064, PORTABLE-0065

Device.Portable.Core.MTPStreams

Target Feature: Device.Portable.Core

Title: Portable devices that implement MTP Streams support required object stream operations

Applicable OS Versions:

Windows 8 Client x64
Windows 8 Client x86

Description:

In order to properly support MTP Stream functionality a device must support the following operations:

Operation	Status	Description
OpenObjectStream	R	This operation opens an object as a stream.
ReadObjectStream	R	This operation reads a portion of data from the previously opened object stream.
WriteObjectStream	R	This operation writes the portion of data to the previously opened object stream.
SeekObjectStream	R	This operation seeks forward reading or writing of the next data block in the stream.
CloseObjectStream	R	This operation closes the previously opened object stream and releases the allocated

Where:

R = Required

I = If Implemented (Optional)

N/A = Not Applicable

Design Notes:

For implementation details refer to Media Transfer Protocol Specification Revision 2.0, available at http://www.usb.org/developers/devclass_docs.

Exceptions:

These operations must be supported if the device supports MTP Streams. Support for Streams is not required for Windows Hardware Certification, however supporting this enhancement is strongly recommended.

Business Justification:

This requirement is necessary in order to ensure that a portable device that supports MTP Stream functionality does so I a way that ensures compliance with other Initiators that the device may interface with. MTP 2.0 is an industry standard specification that is being ratified by the USB organization and represents the standard way that this functionality should be supported.

Scenarios:

Support for MTP Streams enables multiple operations to queue on a single MTP session between the Initiator and the Responder, alleviating pipe contention issues. An operation is not required to wait until another operation has completed before it can be sent between the Initiator and Responder. The second benefit of these streaming enhancement allows random access to a responder's objects by initiator without copying then entire object to the initiator first.

Success Metric: Not Specified

Enforcement Date: TBD

Comments:

New Requirement

Device.Portable.Core.TransportBluetooth

Target Feature: Device.Portable.Core

Title: If a Portable Device leverages the Bluetooth transport, then the device shall support the latest required specification(s) for that transport and related tests: Bluetooth (Specification Version 2.1 or greater)

Applicable OS Versions:

Windows 8 Client x64
Windows 8 Client x86
Windows 7 Client x86
Windows 7 Client x64

Description:

A Portable Device that uses Bluetooth must use the following specifications

Bluetooth Core specification Version 2.1 + EDR (with latest errata) or newer [https://www.bluetooth.org/Technical/Specifications/adopted.htm]

MTP over Bluetooth Profile Specification from Microsoft http://go.microsoft.com/fwlink/?LinkId=237003.

Design Notes:

The connection must be implemented according to requirements defined for Bluetooth devices.

A Bluetooth-capable Portable Device must be able to communicate with **Window's Bluetooth Class Driver** (in a standard MTP conversation over Bluetooth similar to USB and TCP/IP).

A Bluetooth-capable Portable Device must support Bluetooth V2.1+EDR, to ensure that Windows can leverage **Secure Simple Pairing (SSR)** for optimized pairing experience.

A Bluetooth-capable Portable Device must leverage L2CAP Transport MTP Responder Service Definition Record required parameter "**GetFormatCapabilities**" to mitigate format enumeration performance issues.

Exceptions: Not Specified

Business Justification:

This requirement helps ensure that the Bluetooth MTP device is compliant to the specification and works correctly with the Bluetooth MTP Class Driver.

Scenarios:

Ensure that Bluetooth attached MTP devices are spec compliant and work as expected with the Windows Bluetooth MTP Class Driver.

Success Metric: Pass/Fail logo tests

Enforcement Date: June 01, 2012

Comments:

PORTABLE-0028; PORTABLE-0038; PORTABLE-0052

Device.Portable.Core.TransportIP

Target Feature: Device.Portable.Core

Title: If a Portable Device leverages the IP transport, then the device shall support the latest required specification(s) for that transport and related tests: IP (Along with the MTP Network Association Extension specification and related UPnP specifications).

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 7 Client x86
- Windows 7 Client x64

Description:

Windows Portable Device that support MTP over IP must comply with the Camera and Imaging Products Association (CIPA) PTP over Internet Protocol (PTP/IP) specification (CIPA-005/2005), which extends the PTP specification to use TCP/IP to perform PTP operations over wireless networks.

A MTP/IP device must support the following three technologies to ensure compatibility with the PC:

1. UPnP Basic Device

MTP/IP devices must support Simple Service Discovery Protocol (SSDP) discovery as a Universal Plug and Play (UPnP) basic device. Refer to the MTP Network Association Extension definition for details.

2. Picture Transfer Protocol of IP

MTP/IP devices must comply with the Camera and Imaging Products Association (CIPA) PTP over Internet Protocol (PTP/IP) specification (CIPA-005/2005), which extends the PTP specification to use TCP/IP to perform PTP operations over wireless networks.

3. MTP Network Association Extension

MTP/IP devices must comply with the Wi-Fi-provisioning extension to the MTP specification and the MTP Network Association extension to the MTP specification.

Support for this MTP extension is indicated by including the following string in the MTPVendorExtensionDesc field of the MTP DeviceInfo dataset, returned as a response to the MTP GetDeviceInfo operation.

Dataset field	Datatype
MTPVendorExtensionDesc	microsoft.com/WPDNA: 1.0

The device property defined by this extension supports Network Association configuration for devices that support Zero or Nominal Authentication. Secure Authentication is not supported by this extension; secure authentication can be implemented via a custom MTP Service if desired. Refer to the MTP Network Association Extension definition for details.

Recommendations for Network Provisioning

The MTP Wi-Fi Provisioning Extension extends the Windows Connect Now architecture to MTP devices. This extension defines a new MTP object format for WFC (Wireless Configuration File) objects. A WFC object contains the network settings that will allow the responder to join a wireless network. An initiator supporting the WCN architecture will be able to transfer WFC objects to the device. Each WFC object represents the settings for a single wireless network. The responder may receive multiple WFC objects over time. Each object will be named according to the SSID of the network.

The following DataType must be included in the MTPVendorExtensionDesc field of the DeviceInfo dataset.

Dataset field	Datatype
MTPVendorExtensionDesc	microsoft.com/WPDWCN: 1.0

Recommendations for Improved Performance over IP

In order to mitigate format enumeration performance issues, MTP/IP devices should also support the **GetFormatCapabilities** operation as defined in the MTP Device Services Extensions Specification. This operation improves the performance of querying a device for the supported object properties on formats that are associated with classic MTP storages (those formats that appear in the DeviceInfo dataset). GetFormatCapabilities is a bulk operation that duplicates the functionality of GetObjectPropsSupported, GetObjectPropDesc, and GetInterdependentPropDesc. Responders should implement this operation because it replaces multiple calls to the device with a single, more efficient call.

Design Notes:

Refer to the PTP/IP Specification (CIPA-005/2005) "Picture Transfer Protocol over TCP/IP Networks" (http://www.cipa.jp/ptp-ip/index_e.html).

MTP Device Services Extension Specification at http://msdn.microsoft.com/en-us/windows/hardware/gg463545.

MTP Network Association, MTP Windows Connect Now, and MTP Wi-Fi Provisioning Extension documents are available at http://msdn.microsoft.com/en-us/windows/hardware/gg463543.

Exceptions: Not Specified

Business Justification:

This requirement helps ensure that the MTP device that connects over IP, is compliant to the specification and works correctly with the MTP IP Class Driver.

Scenarios:

Ensure that IP attached MTP devices are spec compliant and work as expected with the Windows MTP IP Class Driver.

Success Metric: Pass/Fail logo tests

Enforcement Date: June 01, 2012

Comments:

PORTABLE-0029, PORTABLE-0039; PORTABLE-0053

Device.Portable.Core.TransportIPDLNA

Target Feature: Device.Portable.Core

Title: A Portable Device that functions as a Digital Media Controller, Digital Media Renderer, or Digital Media Server conforms to requirements defined for Networked Media Devices.

Applicable OS Versions:

Windows 8 Client x64
Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client x86

Description:

Portable devices can implement one or more of the DLNA device classes such as:

Digital Media Controller (DMC)

Digital Media Renderer (DMR)

Digital Media Server (DMS)

The device must meet the requirements defined for each device class called out in the Networked Media Device section of the Windows Hardware Certification Program for Hardware. DLNA is managed independent of MTP

Design Notes:

Refer to the Networked Media Device section for requirement details. Information on DLNA Certification can be found at http://www.dlna.org.

Exceptions: Not Specified

Business Justification:

Portable Devices that implement DLNA , must pass the Microsoft DLNA logo requirements and associated tests.

Scenarios:

Portable Devices that implement DLNA , must pass the Microsoft DLNA logo requirements and associated tests.

Success Metric: Pass/Fail

Enforcement Date: 6/1/2009

Comments:

PORTABLE-0036, PORTABLE-0048

Device.Portable.Core.TransportUSB

Target Feature: Device.Portable.Core

Title: If a Portable Device leverages the USB transport, then the device shall support the latest required specification(s) for that transport and related tests: USB (Specification Version 2.0 or greater)

Applicable OS Versions:

Windows 8 Client x64
Windows 8 Client x86
Windows 7 Client x86
Windows 7 Client x64

Description:

A Portable Device that uses USB must use the following:

USB Core specification Version 2.0 (with latest errata) or newer [http://www.usb.org/developers/docs]

USB MTP DWG Specification 1.0 (with latest errata) or newer [http://www.usb.org/developers/devclass_docs]

Design Notes:

USB connected devices must support both High-Speed and Full-Speed. Super-Speed is optional.

The connection must be implemented according to requirements in the appropriate connection type section.

There are no specific Windows Hardware Certification requirements for the type of USB connector used on a portable device.

Exceptions: Not Specified

Business Justification:

Devices are optionally allowed to be USB 3.0 or leverage MTP 2.0 specification. Since both of these specifications are relatively new, they will not be mandated in WLK 2.0, though they may be required for future versions of the Windows Hardware Certification kit.

Scenarios:

Ensure that USB attached MTP devices are spec compliant and work as expected with the Windows USB MTP Class Driver.

Success Metric: Pass/Fail logo tests

Enforcement Date: June 01, 2012

Comments:

PORTABLE-0001; PORTABLE-0020; PORTABLE-0025; PORTABLE-0038; PORTABLE-0044

Device.Portable.Core.VideoCodec

Target Feature: Device.Portable.Core

Title: If a Portable Device has the ability to capture video content, it must do so using a format supported natively in Windows

Applicable OS Versions:

Windows 8 Client x64
Windows 8 Client x86
Windows 7 Client x86
Windows 7 Client x64

Description:

If a Portable Device can capture video content, it must do so in a format that Windows can decode natively without the need for additional codecs to be installed.

The following tables represent the list of in-box formats that Windows will render to; this is not the exhaustive of supported formats.

Note that for a given format in the tables below it is not required to support all given Bit rates and Sample rates, it is however required to support a format that lines up within the ranges provided.

MP4 Container Content

	Setting	Requirement
AAC	Codec	AAC Standard, minimum AAC Profile level 2 (0x29), compatible with 0x29, 0x2A, 0x2B, 0x2C, 0x2E, 0x2F, 0x30, 0x31, 0x32, 0x33
	bit rate for CBR	96, 128, 160, 192Kbps
	Sample rate	44.1 or 48kHz
	Channels	2
H.264	Codec	H.264 Standard
	Codec profile	Baseline, Level 3
	Resolution, pixel aspect ratio	Any (as long as reported)
	Frame rate	Any (as long as reported)
	Average bit rate	Any (as long as reported)

Mobile WMV Video Content

	Setting	Requirement
WMA Standard	Codec	WMA9 or later
	Average bit rate	From 48 to 96kilobits per second (Kbps)
	Maximum peak bit rate	192Kbps
	Sample rate	44.1 or 48kHz
	Channels	2
WMV	Codec	VC-1
	Codec profile	Simple
	Resolution, pixel aspect ratio	One of the following: 176x144 pixels, 1:1 220x176 pixels, 1:1
	Frame rate	15 or 24frames per second
	Average bit rate	From 96 to 384Kbps

	Maximum peak bit rate	768Kbps
	Maximum buffer	3 seconds
	Maximum key-frame distance	15seconds
	Color depth	16bits per pixel

Portable WMV Video Content

	Setting	**Requirement**
WMA Standard	Codec	WMA9 or later
	Average bit rate	From 48 to 128Kbps
	Maximum peak bit rate	256Kbps
	Sample rate	44.1 or 48kHz
	Channels	2
WMV	Codec	VC-1
	Codec profile	Simple
	Resolution, pixel aspect ratio	One of the following: 320x240 pixels, 1:1 480x27"70 pixels, 1:1
	Frame rate	24, 25, or 29.97frames per second
	Average bit rate	From 384 to 850Kbps
	Maximum peak bit rate	1,700Kbps
	Maximum buffer	3 seconds
	Maximum key-frame distance	15seconds
	Color depth	16bits per pixel

Standard WMV Video Content

	Setting	**Requirement**
WMA Standard	Codec	WMA9 or later
	Average bit rate	From 64 to 192Kbps

	Maximum peak bit rate	360Kbps
	Sample rate	44.1 or 48kHz
	Channels	2
WMV	Codec	VC-1
	Codec profile	Main
	Resolution, pixel aspect ratio	One of the following: 640x4″80 pixels, 1:1 720x4″80 pixels, 10:11 720x4″80 pixels, 40:33
	Frame rate	24, 25, or 29.97frames per second
	Average bit rate	From 1,000 to 3,000Kbps
	Maximum peak bit rate	6,000 Kbps
	Maximum buffer	2 seconds
	Maximum key-frame distance	15seconds
	Color depth	16 or 24bits per pixel

High Definition WMV Video Content

	Setting	**Minimum requirement**
WMA Standard	Codec	WMA9 or later
	Average bit rate	From 64 to 192Kbps
	Maximum peak bit rate	360Kbps
	Sample rate	44.1 or 48kHz
	Channels	2
WMA Professional	Codec	WMA9 or later
	Average bit rate	From 320 to 1,500Kbps
	Maximum peak bit rate	2,500Kbps
	Sample rate	44.1, 48, or 96kHz

	Channels	up to 8
WMV	Codec	VC-1
	Codec profile	Advanced
	Resolution, pixel aspect ratio	1280x7□20 pixels, 1:1
	Frame rate	24, 25, or 29.97frames per second
	Average bit rate	From 8,000 to 10,000Kbps
	Maximum peak bit rate	20,000Kbps
	Maximum buffer	2 seconds
	Maximum key-frame distance	15seconds
	Color depth	24bits per pixel

Exceptions: Not Specified

Business Justification:

This requirement ensures that end users running Windows have a successful experience extracting media content from the portable device without having to install 3rd party codecs (which can be hard to identify and download).

Scenarios:

This requirement ensures that end users running Windows have a successful experience extracting media content from the portable device without having to install 3rd party codecs (which can be hard to identify and download).

Success Metric: Pass/Fail

Enforcement Date: June 01, 2006

Comments:

PORTABLE-0010, PORTABLE-0067

Device.Portable.DigitalCamera

Description:

DigitalCamera

Related Requirements:

Device.Portable.DigitalCamera.MTP

Device.Portable.DigitalCamera.MTP

Target Feature: Device.Portable.DigitalCamera

Title: Digital Cameras must support MTP operations and properties as defined in the Media Transport Protocol revision 1.0 or later, along with specific object formats per device type.

Applicable OS Versions:

- Windows 8 Client x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86

Description:

Media Transfer Protocol (MTP) is an extension of the Picture Transfer Protocol (ISO 15740). Both specifications include optional commands, operations, etc., but some of these are required for certification. There is a core set of operations and device properties that must be met by each device category in order to be compliant with Windows. Implementation details for MTP are defined in the Media Transfer Protocol Specification, Revision 1.0 and will be used as the reference for certification of all portable devices as part of the Windows Hardware Certification Program.

Required Operations

This core set of MTP operations and device properties that must be met by each portable device in order to be compliant with Windows. Implementation details for each operation and device property are defined in the Media Transfer Protocol Specification, Revision 1.0 or later.

	Property Code	Mobile Phone	Media Player	Digital Camera	Digital Video Camera	Other (Base)
GetDeviceInfo	0x1001	R	R	R	R	R
OpenSession	0x1002	R	R	R	R	R
CloseSession	0x1003	R	R	R	R	R
GetStorageIDs	0x1004	R	R	R	R	R
GetStorageInfo	0x1005	R	R	R	R	R
GetNumObjects	0x1006	R	R	R	R	R
GetObjectHandles	0x1007	R	R	R	R	R
GetObjectInfo	0x1008	R	R	R	R	R
GetObject	0x1009	R	R	R	R	R

GetDevicePropDesc	0x1014	R	R	R	R	R
GetDevicePropValue	0x1015	R	R	R	R	R
DeleteObject	0x100B	R	R	R	R	I
SetDevicePropValue	0x100A	R	R	I	I	I
SendObjectInfo	0x100C	R	R	I	I	I
SendObject	0x100D	R	R	I	I	I
GetPartialObject	0x101B	R	R	I	I	I
GetObjectPropsSupported	0x9801	R	R	I	I	I
GetObjectPropDesc	0x9802	R	R	I	I	I
GetObjectPropValue	0x9803	R	R	I	I	I
SetObjectPropValue	0x9804	R	R	I	I	I
GetObjectReferences	0x9810	R	R	I	I	I
SetObjectReferences	0x9811	R	R	I	I	I

Where:

R = Required

I = If Implemented (Optional)

N/A = Not Applicable

See **Operations** in the MTP Specification, Revision 1.0 or later for complete list of defined MTP Operations.

The following operations are highly recommended, though not required:

FormatStore (0x100F)

GetObjectPropList (0x9805)

SetObjectPropList (0x9806)

GetInterDependentPropDesc (0x9807)

SendObjectPropList (0x9808) For this command, your device must also support specification of destination, which allows the MTP initiator to dictate a parent handle for that object.

Required Events

The following events must be supported for all objects:

ObjectAdded (0x4002)

ObjectRemoved (0x4003)

If the device supports removable media, it must also support the following events.

StoreAdded (0x4004)

StoreRemoved (0x4005)

Operation Responses

An appropriate response must be returned for any and all operations. If an error is encountered error response codes are also defined and must be provided by the device. For more information, see section 4.8 of the MTP specification, "Responses."

Required Device Properties

	Property Code	Mobile Phone	Media Player	Digital Camera	Digital Video Camera	Other (Base)
Battery Level	0x5001	I	I	I	I	I
Synchronization Partner	0xD401	I	I	I	I	I
Device Friendly Name	0xD402	R	R	I	I	I
Device Icon	0xD405	I	I	I	I	I

See **Device Properties** in the MTP Specification, Revision 1.0 or later for complete list of defined Device Properties.

Device Icon is not required but strongly recommended. High resolution images of the device can be used and exposed in Windows UI.

Object Formats

The following table represents the list of required object formats for a portable device.

Each object format also has a list of object properties that must be supported per object type.

	Property Code	Mobile Phone	Media Player	Digital Camera	Digital Video Camera	Other (Base)
Undefined	0x3000	I	I	I	I	I
Association	0x3001	R	R	R	R	R
Abstract Audio Album	0xBA03	I	I	N/A	N/A	I

Abstract Audio & Video Playlist	0xBA05	I	I	N/A	N/A	I
WAV	0x3008	R(1)	R(1)	I	I	I
MP3	0x3009	R(1)	R(1)	I	I	I
AVI	0x300A	R(1)	R(1)	I	R(1)	I
MPEG	0x300B	R(1)	R(1)	I	R(1)	I
ASF	0x300C	R(1)	R(1)	I	R(1)	I
EXIF/JPEG	0x3801	I	I	R(1)	I	I
TIFF/EP	0x3802	I	I	R(1)	I	I
BMP	0x3804	I	I	R(1)	I	I
JPEG XR	0xB804	I	I	R(1)	I	I
WMA	0xB901	R(1)	R(1)	I	I	I
AAC	0xB903	R(1)	R(1)	I	I	I
WMV	0xB981	I	I	I	R(1)	I
MP4 Container	0xB982	I	I	I	R(1)	I
3GP Container	0xB984	I	I	I	R(1)	I
3G2	0xB985	I	I	I	R(1)	I
AVCHD	0xB986	I	I	I	R(1)	I

(1) Portable devices that are capable of playing audio and/or video must support at least one of these formats. This table does not represent the complete list of supported formats; it represents the list of commonly used formats.

See **Object Formats** in the MTP Specification, Revision 1.0 or later for complete list of defined Device Properties.

Design Notes:

"Picture Transfer Protocol (PTP) for Digital Still Photography Devices," Version 1.0 of the PIMA15740: 2000 Picture Transfer Protocol specification http://go.microsoft.com/fwlink/?LinkId=237005.

The Media Transfer Protocol Specification, Revision 1.0 is available at http://www.usb.org/developers/devclass_docs/MTP_1.0.zip .

Additional implementation details can be found in the Windows 7 Portable Device Enabling Kit for MTP, which is available at http://msdn.microsoft.com/en-us/windows/hardware/gg463543 .

Exceptions: Not Specified

Business Justification:

Existing requirement to ensure that the device is compatible with the core MTP requirements.

Scenarios:

Existing requirement to ensure that the device is compatible with the core MTP requirements.

Success Metric: Pass/Fail

Enforcement Date: 6/1/2009

Comments:

PORTABLE-0059, PORTABLE-0031, PORTABLE-0041, PORTABLE-0062

Device.Portable.DigitalVideoCamera

Description:

DigitalVideoCamera

Related Requirements:

Device.Portable.DigitalVideoCamera.MTP

Device.Portable.DigitalVideoCamera.MTP

Target Feature: Device.Portable.DigitalVideoCamera

Title: Digital Video Cameras must support MTP operations and properties as defined in the Media Transport Protocol revision 1.0 or later, along with specific object formats per device type.

Applicable OS Versions:

Windows 8 Client x64
Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client x86

Description:

Media Transfer Protocol (MTP) is an extension of the Picture Transfer Protocol (ISO 15740). Both specifications include optional commands, operations, etc., but some of these are required for certification. There is a core set of operations and device properties that must be met by each device category in order to be compliant with Windows. Implementation details for MTP are defined in the

Media Transfer Protocol Specification, Revision 1.0 and will be used as the reference for certification of all portable devices as part of the Windows Hardware Certification Program.

Required Operations

This core set of MTP operations and device properties that must be met by each portable device in order to be compliant with Windows. Implementation details for each operation and device property are defined in the Media Transfer Protocol Specification, Revision 1.0 or later.

	Property Code	Mobile Phone	Media Player	Digital Camera	Digital Video Camera	Other (Base)
GetDeviceInfo	0x1001	R	R	R	R	R
OpenSession	0x1002	R	R	R	R	R
CloseSession	0x1003	R	R	R	R	R
GetStorageIDs	0x1004	R	R	R	R	R
GetStorageInfo	0x1005	R	R	R	R	R
GetNumObjects	0x1006	R	R	R	R	R
GetObjectHandles	0x1007	R	R	R	R	R
GetObjectInfo	0x1008	R	R	R	R	R
GetObject	0x1009	R	R	R	R	R
GetDevicePropDesc	0x1014	R	R	R	R	R
GetDevicePropValue	0x1015	R	R	R	R	R
DeleteObject	0x100B	R	R	R	R	I
SetDevicePropValue	0x100A	R	R	I	I	I
SendObjectInfo	0x100C	R	R	I	I	I
SendObject	0x100D	R	R	I	I	I
GetPartialObject	0x101B	R	R	I	I	I
GetObjectPropsSupported	0x9801	R	R	I	I	I
GetObjectPropDesc	0x9802	R	R	I	I	I
GetObjectPropValue	0x9803	R	R	I	I	I
SetObjectPropValue	0x9804	R	R	I	I	I

GetObjectReferences	0x9810	R	R	I	I	I
SetObjectReferences	0x9811	R	R	I	I	I

Where:

R = Required

I = If Implemented (Optional)

N/A = Not Applicable

See **Operations** in the MTP Specification, Revision 1.0 or later for complete list of defined MTP Operations.

The following operations are highly recommended, though not required:

FormatStore (0x100F)

GetObjectPropList (0x9805)

SetObjectPropList (0x9806)

GetInterDependentPropDesc (0x9807)

SendObjectPropList (0x9808) For this command, your device must also support specification of destination, which allows the MTP initiator to dictate a parent handle for that object.

Required Events

The following events must be supported for all objects:

ObjectAdded (0x4002)

ObjectRemoved (0x4003)

If the device supports removable media, it must also support the following events.

StoreAdded (0x4004)

StoreRemoved (0x4005)

Operation Responses

An appropriate response must be returned for any and all operations. If an error is encountered error response codes are also defined and must be provided by the device. For more information, see section 4.8 of the MTP specification, "Responses."

Required Device Properties

	Property Code	**Mobile Phone**	**Media Player**	**Digital Camera**	**Digital Video Camera**	**Other (Base)**

Battery Level	0x5001	I	I	I	I	I
Synchronization Partner	0xD401	I	I	I	I	I
Device Friendly Name	0xD402	R	R	I	I	I
Device Icon	0xD405	I	I	I	I	I

See **Device Properties** in the MTP Specification, Revision 1.0 or later for complete list of defined Device Properties.

Device Icon is not required but strongly recommended. High resolution images of the device can be used and exposed in Windows UI.

Object Formats

The following table represents the list of required object formats for a portable device.

Each object format also has a list of object properties that must be supported per object type.

	Property Code	**Mobile Phone**	**Media Player**	**Digital Camera**	**Digital Video Camera**	**Other(Base)**
Undefined	0x3000	I	I	I	I	I
Association	0x3001	R	R	R	R	R
Abstract Audio Album	0xBA03	I	I	N/A	N/A	I
Abstract Audio & Video Playlist	0xBA05	I	I	N/A	N/A	I
WAV	0x3008	R(1)	R(1)	I	I	I
MP3	0x3009	R(1)	R(1)	I	I	I
AVI	0x300A	R(1)	R(1)	I	R(1)	I
MPEG	0x300B	R(1)	R(1)	I	R(1)	I
ASF	0x300C	R(1)	R(1)	I	R(1)	I
EXIF/JPEG	0x3801	I	I	R(1)	I	I
TIFF/EP	0x3802	I	I	R(1)	I	I
BMP	0x3804	I	I	R(1)	I	I
JPEG XR	0xB804	I	I	R(1)	I	I

WMA	0xB901	R(1)	R(1)	I	I	I
AAC	0xB903	R(1)	R(1)	I	I	I
WMV	0xB981	I	I	I	R(1)	I
MP4 Container	0xB982	I	I	I	R(1)	I
3GP Container	0xB984	I	I	I	R(1)	I
3G2	0xB985	I	I	I	R(1)	I
AVCHD	0xB986	I	I	I	R(1)	I

(1) Portable devices that are capable of playing audio and/or video must support at least one of these formats. This table does not represent the complete list of supported formats; it represents the list of commonly used formats.

See **Object Formats** in the MTP Specification, Revision 1.0 or later for complete list of defined Device Properties.

Design Notes:

"Picture Transfer Protocol (PTP) for Digital Still Photography Devices," Version 1.0 of the PIMA15740: 2000 Picture Transfer Protocol specification http://go.microsoft.com/fwlink/?LinkId=237005.

The Media Transfer Protocol Specification, Revision 1.0 is available at http://www.usb.org/developers/devclass_docs/MTP_1.0.zip .

Additional implementation details can be found in the Windows 7 Portable Device Enabling Kit for MTP, which is available at http://msdn.microsoft.com/en-us/windows/hardware/gg463543 .

Exceptions: Not Specified

Business Justification:

Existing requirement to ensure that the device is compatible with the core MTP requirements.

Scenarios:

Existing requirement to ensure that the device is compatible with the core MTP requirements.

Success Metric: Pass/Fail

Enforcement Date: 6/1/2009

Comments:

PORTABLE-0059, PORTABLE-0031, PORTABLE-0041, PORTABLE-0062

Device.Portable.MediaPlayer

Description:

MediaPlayer

Related Requirements:

Device.Portable.MediaPlayer.MTP

Device.Portable.MediaPlayer.MTP

Target Feature: Device.Portable.MediaPlayer

Title: Media Players must support MTP operations and properties as defined in the Media Transport Protocol revision 1.0 or later, along with specific object formats per device type.

Applicable OS Versions:

Windows 8 Client x64
Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client x86

Description:

Media Transfer Protocol (MTP) is an extension of the Picture Transfer Protocol (ISO 15740). Both specifications include optional commands, operations, etc.but some of these are required for certification. There is a core set of operations and device properties that must be met by each device category in order to be compliant with Windows. Implementation details for MTP are defined in the Media Transfer Protocol Specification, Revision 1.0 and will be used as the reference for certification of all portable devices as part of the Windows Hardware Certification Program.

Required Operations

This core set of MTP operations and device properties that must be met by each portable device in order to be compliant with Windows. Implementation details for each operation and device property are defined in the Media Transfer Protocol Specification, Revision 1.0 or later.

	Property Code	Mobile Phone	Media Player	Digital Camera	Digital Video Camera	Other (Base)
GetDeviceInfo	0x1001	R	R	R	R	R
OpenSession	0x1002	R	R	R	R	R
CloseSession	0x1003	R	R	R	R	R
GetStorageIDs	0x1004	R	R	R	R	R

GetStorageInfo	0x1005	R	R	R	R	R
GetNumObjects	0x1006	R	R	R	R	R
GetObjectHandles	0x1007	R	R	R	R	R
GetObjectInfo	0x1008	R	R	R	R	R
GetObject	0x1009	R	R	R	R	R
GetDevicePropDesc	0x1014	R	R	R	R	R
GetDevicePropValue	0x1015	R	R	R	R	R
DeleteObject	0x100B	R	R	R	R	I
SetDevicePropValue	0x100A	R	R	I	I	I
SendObjectInfo	0x100C	R	R	I	I	I
SendObject	0x100D	R	R	I	I	I
GetPartialObject	0x101B	R	R	I	I	I
GetObjectPropsSupported	0x9801	R	R	I	I	I
GetObjectPropDesc	0x9802	R	R	I	I	I
GetObjectPropValue	0x9803	R	R	I	I	I
SetObjectPropValue	0x9804	R	R	I	I	I
GetObjectReferences	0x9810	R	R	I	I	I
SetObjectReferences	0x9811	R	R	I	I	I

Where:

R = Required

I = If Implemented (Optional)

N/A = Not Applicable

See **Operations** in the MTP Specification, Revision 1.0 or later for complete list of defined MTP Operations.

The following operations are highly recommended, though not required:

FormatStore (0x100F)

GetObjectPropList (0x9805)

SetObjectPropList (0x9806)

GetInterDependentPropDesc (0x9807)

SendObjectPropList (0x9808) For this command, your device must also support specification of destination, which allows the MTP initiator to dictate a parent handle for that object.

Required Events

The following events must be supported for all objects:

ObjectAdded (0x4002)

ObjectRemoved (0x4003)

If the device supports removable media, it must also support the following events.

StoreAdded (0x4004)

StoreRemoved (0x4005)

Operation Responses

An appropriate response must be returned for any and all operations. If an error is encountered error response codes are also defined and must be provided by the device. For more information, see section 4.8 of the MTP specification, "Responses."

Required Device Properties

	Property Code	**Mobile Phone**	**Media Player**	**Digital Camera**	**Digital Video Camera**	**Other (Base)**
Battery Level	0x5001	I	I	I	I	I
Synchronization Partner	0xD401	I	I	I	I	I
Device Friendly Name	0xD402	R	R	I	I	I
Device Icon	0xD405	I	I	I	I	I

See **Device Properties** in the MTP Specification, Revision 1.0 or later for complete list of defined Device Properties.

Device Icon is not required but strongly recommended. High resolution images of the device can be used and exposed in Windows UI.

Object Formats

The following table represents the list of required object formats for a portable device.

Each object format also has a list of object properties that must be supported per object type.

	Property Code	Mobile Phone	Media Player	Digital Camera	Digital Video Camera	Other(Base)
Undefined	0x3000	I	I	I	I	I
Association	0x3001	R	R	R	R	R
Abstract Audio Album	0xBA03	I(2)	I(2)	N/A	N/A	I(2)
Abstract Audio & Video Playlist	0xBA05	I	I	N/A	N/A	I
WAV	0x3008	R(1)	R(1)	I	I	I
MP3	0x3009	R(1)	R(1)	I	I	I
AVI	0x300A	R(1)	R(1)	I	R(1)	I
MPEG	0x300B	R(1)	R(1)	I	R(1)	I
ASF	0x300C	R(1)	R(1)	I	R(1)	I
EXIF/JPEG	0x3801	I	I	R(1)	I	I
TIFF/EP	0x3802	I	I	R(1)	I	I
BMP	0x3804	I	I	R(1)	I	I
JPEG XR	0xB804	I	I	R(1)	I	I
WMA	0xB901	R(1)	R(1)	I	I	I
AAC	0xB903	R(1)	R(1)	I	I	I
WMV	0xB981	I	I	I	R(1)	I
MP4 Container	0xB982	I	I	I	R(1)	I
3GP Container	0xB984	I	I	I	R(1)	I
3G2	0xB985	I	I	I	R(1)	I
AVCHD	0xB986	I	I	I	R(1)	I

(1) Portable devices that are capable of playing audio and/or video must support at least one of these formats. This table does not represent the complete list of supported formats; it represents the list of commonly used formats.

See **Object Formats** in the MTP Specification, Revision 1.0 or later for complete list of defined Device Properties.

(2)Support for the following object properties is mandatory for AbstractAudioAlbum objects:

Genre (0xDC8C)

Album Artist (0xDC9B)

WMPMetadataRoundTrip (0x9201)

A device that supports both the AbstractAudioAlbum format and an image format may optionally support album art. Album art is attached to an AbstractAudioAlbum object by adding the art to the album's RepresentativeSampleData property.

If a Portable Device supports album art, then the device must support the following object properties:

PurchaseFlag (0xd901), an object property in the Windows Media DRM 10 for Portable Devices MTP Extensions

RepresentativeSampleFormat (0xdc81)

RepresentativeSampleSize (0xdc82)

RepresentativeSampleHeight (0xdc83)

RepresentativeSampleWidth (0xdc84)

RepresentativeSampleData (0xdc86)

User Rating (0xdc8a)

WMPMetadataRoundTrip (0x9201)

Design Notes:

"Picture Transfer Protocol (PTP) for Digital Still Photography Devices," Version 1.0 of the PIMA15740: 2000 Picture Transfer Protocol specification http://go.microsoft.com/fwlink/?LinkId=237005.

The Media Transfer Protocol Specification, Revision 1.0 is available at http://www.usb.org/developers/devclass_docs/MTP_1.0.zip .

Additional implementation details can be found in the Windows 7 Portable Device Enabling Kit for MTP, which is available at http://msdn.microsoft.com/en-us/windows/hardware/gg463543 .

Exceptions: Not Specified

Business Justification:

Existing requirement to ensure that the device is compatible with the core MTP requirements.

Scenarios:

Existing requirement to ensure that the device is compatible with the core MTP requirements.

Success Metric: Pass/Fail

Enforcement Date: 6/1/2009

Comments:

PORTABLE-0059, PORTABLE-0031, PORTABLE-0041, PORTABLE-0062

Device.Portable.MobilePhone

Description:

MobilePhone

Related Requirements:

Device.Portable.MobilePhone.MTP

Device.Portable.MobilePhone.MTP

Target Feature: Device.Portable.MobilePhone

Title: Mobile Phones must support MTP operations and properties as defined in the Media Transport Protocol revision 1.0 or later, along with specific object formats per device type.

Applicable OS Versions:

Windows 8 Client x64
Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client x86

Description:

Media Transfer Protocol (MTP) is an extension of the Picture Transfer Protocol (ISO 15740). Both specifications include optional commands, operations, etc., but some of these are required for certification. There is a core set of operations and device properties that must be met by each device category in order to be compliant with Windows. Implementation details for MTP are defined in the Media Transfer Protocol Specification, Revision 1.0 and will be used as the reference for certification of all portable devices as part of the Windows Hardware Certification Program.

Required Operations

This core set of MTP operations and device properties that must be met by each portable device in order to be compliant with Windows. Implementation details for each operation and device property are defined in the Media Transfer Protocol Specification, Revision 1.0 or later.

	Property	Mobile	Media	Digital	Digital Video	Other

	Code	Phone	Player	Camera	Camera	(Base)
GetDeviceInfo	0x1001	R	R	R	R	R
OpenSession	0x1002	R	R	R	R	R
CloseSession	0x1003	R	R	R	R	R
GetStorageIDs	0x1004	R	R	R	R	R
GetStorageInfo	0x1005	R	R	R	R	R
GetNumObjects	0x1006	R	R	R	R	R
GetObjectHandles	0x1007	R	R	R	R	R
GetObjectInfo	0x1008	R	R	R	R	R
GetObject	0x1009	R	R	R	R	R
GetDevicePropDesc	0x1014	R	R	R	R	R
GetDevicePropValue	0x1015	R	R	R	R	R
DeleteObject	0x100B	R	R	R	R	I
SetDevicePropValue	0x100A	R	R	I	I	I
SendObjectInfo	0x100C	R	R	I	I	I
SendObject	0x100D	R	R	I	I	I
GetPartialObject	0x101B	R	R	I	I	I
GetObjectPropsSupported	0x9801	R	R	I	I	I
GetObjectPropDesc	0x9802	R	R	I	I	I
GetObjectPropValue	0x9803	R	R	I	I	I
SetObjectPropValue	0x9804	R	R	I	I	I
GetObjectReferences	0x9810	R	R	I	I	I
SetObjectReferences	0x9811	R	R	I	I	I

Where:

R = Required

I = If Implemented (Optional)

N/A = Not Applicable

See **Operations** in the MTP Specification, Revision 1.0 or later for complete list of defined MTP Operations.

The following operations are highly recommended, though not required:

FormatStore (0x100F)

GetObjectPropList (0x9805)

SetObjectPropList (0x9806)

GetInterDependentPropDesc (0x9807)

SendObjectPropList (0x9808) For this command, your device must also support specification of destination, which allows the MTP initiator to dictate a parent handle for that object.

Required Events

The following events must be supported for all objects:

ObjectAdded (0x4002)

ObjectRemoved (0x4003)

If the device supports removable media, it must also support the following events.

StoreAdded (0x4004)

StoreRemoved (0x4005)

Operation Responses

An appropriate response must be returned for any and all operations. If an error is encountered error response codes are also defined and must be provided by the device. For more information, see section 4.8 of the MTP specification, "Responses."

Required Device Properties

	Property Code	**Mobile Phone**	**Media Player**	**Digital Camera**	**Digital Video Camera**	**Other (Base)**
Battery Level	0x5001	I	I	I	I	I
Synchronization Partner	0xD401	I	I	I	I	I
Device Friendly Name	0xD402	R	R	I	I	I

Device Icon	0xD405	I	I	I	I	I

See **Device Properties** in the MTP Specification, Revision 1.0 or laterfor complete list of defined Device Properties.

Device Icon is not required but strongly recommended. High resolution images of the device can be used and exposed in Windows UI.

Object Formats

The following table represents the list of required object formats for a portable device.

Each object format also has a list of object properties that must be supported per object type.

	Property Code	**Mobile Phone**	**Media Player**	**Digital Camera**	**Digital Video Camera**	**Other(Base)**
Undefined	0x3000	I	I	I	I	I
Association	0x3001	R	R	R	R	R
Abstract Audio Album	0xBA03	I(2)	I(2)	N/A	N/A	I(2)
Abstract Audio & Video Playlist	0xBA05	I	I	N/A	N/A	I
WAV	0x3008	R(1)	R(1)	I	I	I
MP3	0x3009	R(1)	R(1)	I	I	I
AVI	0x300A	R(1)	R(1)	I	R(1)	I
MPEG	0x300B	R(1)	R(1)	I	R(1)	I
ASF	0x300C	R(1)	R(1)	I	R(1)	I
EXIF/JPEG	0x3801	I	I	R(1)	I	I
TIFF/EP	0x3802	I	I	R(1)	I	I
BMP	0x3804	I	I	R(1)	I	I
JPEG XR	0xB804	I	I	R(1)	I	I
WMA	0xB901	R(1)	R(1)	I	I	I
AAC	0xB903	R(1)	R(1)	I	I	I
WMV	0xB981	I	I	I	R(1)	I
MP4 Container	0xB982	I	I	I	R(1)	I

3GP Container	0xB984	I	I	I	R(1)	I
3G2	0xB985	I	I	I	R(1)	I
AVCHD	0xB986	I	I	I	R(1)	I

(1) Portable devices that are capable of playing audio and/or video must support at least one of these formats. This table does not represent the complete list of supported formats; it represents the list of commonly used formats.

See**Object Formats** in the MTP Specification, Revision 1.0 or later for complete list of defined Device Properties.

(2)Support for the following object properties is mandatory for AbstractAudioAlbum objects:

Genre (0xDC8C)

Album Artist (0xDC9B)

WMPMetadataRoundTrip (0x9201)

A device that supports both the AbstractAudioAlbum format and an image format may optionally support album art. Album art is attached to an AbstractAudioAlbum object by adding the art to the album's RepresentativeSampleData property.

If a Portable Device supports album art, then the device must support the following object properties:

PurchaseFlag (0xd901), an object property in the Windows Media DRM 10 for Portable Devices MTP Extensions

RepresentativeSampleFormat (0xdc81)

RepresentativeSampleSize (0xdc82)

RepresentativeSampleHeight (0xdc83)

RepresentativeSampleWidth (0xdc84)

RepresentativeSampleData (0xdc86)

User Rating (0xdc8a)

WMPMetadataRoundTrip (0x9201)

Design Notes:

"Picture Transfer Protocol (PTP) for Digital Still Photography Devices," Version 1.0 of the PIMA15740: 2000 Picture Transfer Protocol specification http://go.microsoft.com/fwlink/?LinkId=237005.

The Media Transfer Protocol Specification, Revision 1.0 is available at http://www.usb.org/developers/devclass_docs/MTP_1.0.zip .

Additional implementation details can be found in the Windows 7 Portable Device Enabling Kit for MTP, which is available at http://msdn.microsoft.com/en-us/windows/hardware/gg463543.

Exceptions: Not Specified

Business Justification:

Existing requirement to ensure that the device is compatible with the core MTP requirements.

Scenarios:

Existing requirement to ensure that the device is compatible with the core MTP requirements.

Success Metric: Pass/Fail

Enforcement Date: 6/1/2009

Comments:

PORTABLE-0059, PORTABLE-0031, PORTABLE-0041, PORTABLE-0062

Device.Storage.Controller

Description:

General feature that applies to all Storage Controllers

Related Requirements:

Device.Storage.Controller.BasicFunction
Device.Storage.Controller.BitLocker
Device.Storage.Controller.ClassCode
Device.Storage.Controller.InfFile
Device.Storage.Controller.MiniportDriverModel

Device.Storage.Controller.BasicFunction

Target Feature: Device.Storage.Controller

Title: Storage Controller Basic Functionality

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64

Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64
Windows 8 Client ARM

Description:

Controller Basic Functionality

PCI-based host controllers and adapters must support PCI bus mastering as the default setting, and virtual direct access memory (DMA) services must be supported in the host adapter option ROM.

Devices that cannot perform their function without other registry modifications, other than those performed automatically by the device class co-installer, are not eligible for the certification.

All commands must be passed to the underlying physical device unless the controller is a RAID adapter.

If a device returns less data than requested, it must correctly indicate an underrun condition and adapters must handle this in accordance with the WDK (adjust DataTransferLength).

Non-RAID Controller

Miniport drivers other than RAID drivers may not create pseudo devices to be used as targets for management commands for the adapter when no actual LUNs are present. Instead, a SCSI miniports INF must specify the CreateInitiatorLu parameter under the services Parameters key and set this DWORD value to 1. This may not be done using a coinstaller. Storage miniport drivers do not use this parameter as the adapter may always be used even if no devices are present.

Values for storage drivers that are documented in the WDK.

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

Excerpted from Storage-0006 & Storage-0022

Device.Storage.Controller.BitLocker

Target Feature: Device.Storage.Controller

Title: BitLocker must not cause failure in SAN Boot thru Storage Controllers

Applicable OS Versions:

Windows 8 Server x64

Description:

BitLocker must be properly enabled to protect Operating System in a SAN Boot configuration

Exceptions: Not Specified

Business Justification:

(1) When a server is placed in an environment without adequate physical security, BitLocker protects data on the server against unauthorized access if a server is stolen; (2)When hosting service providers repurpose or decommission storage arrays, BitLocker Disk Encryption prevents data breach.

Scenarios:

It is part of BitLocker for Windows Server scenarios.

Success Metric: A storage controller needs to pass BitLocker test.

Enforcement Date: August 15, 2011

Comments: Not Specified

Device.Storage.Controller.ClassCode

Target Feature: Device.Storage.Controller

Title: Bus-attached controllers must implement the correct class/subclass code as specified in PCI 2.3, Appendix D.

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64
Windows 8 Client ARM

Description:

Controller Class Code

Storage host controllers and adapters must meet the requirements for the device protocol used and any requirements related to the device storage bus.

Bus-attached controllers must implement the correct class/subclass code as specified in PCI 2.3, Appendix D. This applies to all bus types including, but not limited to, IDE, Fibre Channel, SCSI, and SATA-based devices. Any device which implements RAID functionality regardless of whether the RAID implementation is done in hardware, firmware or in the driver code, must implement the PCI RAID Class Code (01/04) and not use the interconnect class code (for example, a SATA RAID controller must implement the 01/04 class code and not the AHCI class code 01/06/01).

Non-PCI attached storage host controller does not need to report PCI class code. However, it must report the equivalent ACPI Compatibility ID.

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2009

Comments:

Taken from Storage-0022

Device.Storage.Controller.InfFile

Target Feature: Device.Storage.Controller

Title: All host adapters must be installed by using Plug and Play mechanisms and require the use of an INF file

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

Controller INF File

All host adapters must be installed by using Plug and Play mechanisms and require the use of an INF. Installation programs must use INF files, must pass the most current Chkinf tool, and can explicitly modify the registry values only by using INF file constructs that meet the Windows certification program requirements. Changes can be made only under the following keys:

TimeoutValues under the class driver services keys (Disk and Tape).

SpecialTargetList only for SCSIport implementations.

Devices service key (must be HKLM\System\CurrentControlSet\Services\ and the Parameters\Device subkey under that).

Signal BusChangeDetected on any link transition or detection of a hot-plug event. A limited settling is allowed before this is signaled, but it must be settable through a registry parameter.

Implementation of the BusType registry DWORD to correctly set the interface type in accordance with the enumeration in NTDDSTOR.H (see the WDK).This value must be set in the miniports INF under the services Parameters key there is no programmatic way to set it and you may not rely on coinstallers as they do not run under all scenarios.

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2009

Comments:

Taken from Storage-0022

Device.Storage.Controller.MiniportDriverModel

Target Feature: Device.Storage.Controller

Title: Storage Miniport Driver Model

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

Miniport Driver Model

Drivers used with storage controllers must follow the miniport driver model. Storage Miniports must comply with all miniport interface definitions as defined in the WDK. Fibre Channel, iSCSI, and SAS drivers must be Storport miniports. SATA, SATA RAID, SCSI, and SCSI RAID drivers can be either SCSIport miniports or Storport miniports. SATA drivers may also be written as ATAport miniports. Monolithic, full-port drivers or other types of drivers that do not follow the miniport model are not eligible for the certification. All drivers for physical hardware must be implemented to support Plug and Play. Legacy drivers are no longer supported.

Any device that depends on a filter driver for physical disk drive functionality is not eligible for certification. Filter drivers may not be used to bypass any part of the storage stack. For example, a filter driver many not directly access any hardware (such as by using HAL calls) and filter drivers may not be used to link cache manager to the hardware implementation. Filter drivers may not be used to violate any terms of the certification program.

Multipathing drivers may not be tied to specific HBAs except for PCI RAID controllers and must use the MPIO model.

Transient or pseudo-devices may not be exposed to the system. Drivers that specify NODRV may be used to "claim" management devices that report as processor, controller, or MSC device types. Such drivers that do not refer to a service entry are not eligible for the certification, but they can be signed.

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2009

Comments:

Taken from Storage-0022

Device.Storage.Controller.Ata

Description:

Defines the Industry and Microsoft standards that must be met

Related Requirements:

Device.Storage.Controller.Ata.Interface

Device.Storage.Controller.Ata.Interface

Target Feature: Device.Storage.Controller.Ata

Title: PATA Controller Interface

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64

Description:

PATA Interface

PATA controllers must comply with the ATA/ATAPI-7 specification. Bus-mastering DMA capability is required for all PATA controllers with the exception of compact flash and similar flash-RAM device.

The following requirements are also applied to ATA/ATAPI controllers.

- The PACKET command protocol as defined in ATA/ATAPI-7 Volume 2, Section 11.8, must not be implemented in ATA-only controllers.
- PATA controllers that support the PACKET command protocol must be fully implemented as defined in ATA/ATAPI-7 Volume 2, Section 11.8.
- Identify Device data fields (61:60) and (103:100) must not be used to determine 28-bit or 48-bit LBA addressing support. Instead, bit 10 of word 83 and bit 10 of word 86 must be checked for 48-bit LBA addressing support as defined in ATA/ATAPI-7, Volume 1, Section 4.2.1.

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2009

Comments:

Taken from Storage-0022

Device.Storage.Controller.Boot

Description:

Defines requirements that must be met if the Storage Controller support Boot

Related Requirements:

Device.Storage.Controller.Boot.BasicFunction
Device.Storage.Controller.Boot.BitLocker

Device.Storage.Controller.Boot.BasicFunction

Target Feature: Device.Storage.Controller.Boot

Title: If the controller implements boot support it must support Int13h functions

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64
Windows 8 Client ARM

Description:

Controller Boot Support

Option ROMs in host controllers and adapters for any interface type, including RAID controllers, that provide boot support must fully support extended Int13h functions (functions 4xh) as defined in BIOS Enhanced Disk Drive Services - 3 [T13-D1572], Revision 3 or later. Logical block addressing is the only addressing mechanism supported.

It is recommended that controllers also support booting using the Extensible Firmware Interface (EFI) and implement device paths as defined in EDD-3.

SD/eMMC/NAND flash controllers do not have Option ROM, so the first part of this requirement does not apply. EFI support is required.

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2009

Comments:

Taken from Storage-0022

Device.Storage.Controller.Boot.BitLocker

Target Feature: Device.Storage.Controller.Boot

Title: BitLocker must not cause failure in SAN Boot thru Storage Controllers

Applicable OS Versions:

Windows 8 Server x64

Description:

BitLocker must be properly enabled to protect Operating System in a SAN Boot configuration

Exceptions: Not Specified

Business Justification:

(1) When a server is placed in an environment without adequate physical security, BitLocker protects data on the server against unauthorized access if a server is stolen; (2) When hosting service providers repurpose or decommission storage arrays, BitLocker Disk Encryption prevents data breach.

Scenarios:

It is part of BitLocker for Windows Server scenarios.

Success Metric: A storage controller needs to pass BitLocker test.

Enforcement Date: August 15, 2011

Comments: Not Specified

Device.Storage.Controller.BootDeviceGreaterThan

Description:

Defines requirements that must be met if the Storage Controller support 2.2 Terabyte Boot

Related Requirements:

Device.Storage.Controller.BootDeviceGreaterThan.BasicFunction

Device.Storage.Controller.BootDeviceGreaterThan.BasicFunction

Target Feature: Device.Storage.Controller.BootDeviceGreaterThan

Title: Controllers supporting a boot device with a capacity greater than 2.2 terabytes must comply with requirements

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64

Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64
Windows 8 Client ARM

Description:

Controllers supporting a boot device with a capacity greater than 2.2 terabytes must comply with the following requirements:

Small Computer System Interface (SCSI) and SCSI-compatible storage controllers must comply with section 14, "SCSI Driver Model", of UEFI specification version 2.3.1.

The Internet Small Computer System Interface (iSCSI) boot initiator must comply with section 15, "iSCSI Boot", of UEFI specification version 2.3.

The storage controller must support T10 SBC3 Read Capacity (16) command in the UEFI device driver and the Windows device driver. If Advanced Technology Attachment (ATA) or an Advanced Technology Attachment with Packet Interface (ATAPI) storage controller or disk drive is used, the controller firmware or driver must implement SCSI ATA Translation according T10 SAT3 specifications.

The storage controller must report the exact size of the boot disk drive in the EFI shell and in the Windows operating system.

Exceptions: Not Specified

Business Justification:

Disk drive vendors are ready to ship greater than 2.2 TB disks into the market in 2010. OEM vendors are also preparing greater than 2.2 TB boot solutions for their new system platforms. To protect Microsoft partners' investment and provide a good user experience with a greater than 2.2 TB disk drive, controllers must meet these requirements.

Scenarios:

Device compatibility

Success Metric: Pass/Fail

Enforcement Date: December 1, 2010

Comments:

SYSFUND-0229

Device.Storage.Controller.Fc

Description:

Defines the Industry and Microsoft standards that must be met

Related Requirements:

Device.Storage.Controller.Fc.Interface

Device.Storage.Controller.Fc.Interface

Target Feature: Device.Storage.Controller.Fc

Title: Fibre Channel HBA Interface

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64

Description:

Fibre Channel Interface

Fibre channel host bus adapter drivers must support the WMI classes and methods required to implement the FC-HBA or SM-API by using the Microsoft HBAAPI.DLL. Vendors may not replace the Microsoft-provided version of the HBAAPI.DLL file. A subset of Hbapiwmi.mof WMI classes and methods are required for Windows compatibility. Other WMI classes are optional and are grouped to form feature sets. If a driver implements any part of an optional feature set, all related classes in that feature set must be supported. In some cases, some features are grouped into subfeatures. If a driver implements such a subfeature, the driver must correctly support that specific subfeature.

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2009

Comments:

Taken from Storage-0029

Device.Storage.Controller.Fcoe

Description:

Defines the Industry and Microsoft standards that must be met

Related Requirements:

Device.Storage.Controller.Fcoe.Interface
Device.Storage.Controller.Fcoe.Interoperability

Device.Storage.Controller.Fcoe.Interface

Target Feature: Device.Storage.Controller.Fcoe

Title: Fibre Channel over Ethernet Host Bus Adapter

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64

Description:

Fibre Channel over Ethernet Host Bus Adapter

FCoE Adapter must implement FC-BB-5 FC-BB_E specification.

FCoE Adapter miniport must be implemented as a Storport miniport.

FCoE Adapter miniport must define VER_FILEDESCRIPTION_STR and contain the substring [FCoE].

For FCoE Adapter miniport INFs [service-install-section]

DisplayName entry value is required and must contain the substring [FCoE].

Description entry value is optional, if specified, must contain the substring [FCoE].

For FCoE Adapter miniport INFs Models Section [models-section-name] | [models-section-name.TargetOSVersion]

device-description entry value must contain the substring [FCoE].

FCoE Adapter miniport must declare BusTypeFibre as its STORAGE_BUS_TYPE in the INF file. STORAGE_BUS_TYPE Enumeration

FCoE Adapters that expose PCI storage device(s)/function(s) for FCoE frame processing (either egress or ingress), must report 0x0c0400 (Fibre Channel) as its Class Code (Base Class, Sub-Class and Interface code).

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 12/1/2010

Comments:

Strorage-0029

Device.Storage.Controller.Fcoe.Interoperability

Target Feature: Device.Storage.Controller.Fcoe

Title: Fibre Channel over Ethernet Host Bus Adapter - Interoperability

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64

Description:

Interoperability

FCoE Adapter must interoperate with FC SAN devices through FCF.

FCoE Adapter must interoperate with native FCoE devices.

FCoE Adapter must be able to transport network (IP) and storage (FCoE) traffic concurrently.

Disable/removal/loss of FC (storage) functionality must not impact network (Ethernet) connectivity and functionality.

FCoE Adapter must be able to access and address FC and native FCoE devices concurrently (through FCF).

Initiator Coexistence

FCoE Adapter must coexist with FC Adapter without interference on the same system.

FCoE Adapter must coexist with other FCoE Adapters without interference on the same system.

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 12/1/2010

Comments:

Storage-0030

Device.Storage.Controller.Flush

Description:

Defines the Industry and Microsoft standards that must be met

Related Requirements:

Device.Storage.Controller.Flush.BasicFunction

Device.Storage.Controller.Flush.BasicFunction

Target Feature: Device.Storage.Controller.Flush

Title: Flush to Connected Device

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64
Windows 8 Client ARM

Description:

Industry standard spec requirements:

For ATA device, FLUSH CACHE (E7h, Non-Data) 28-bit command is optional for ATA devices and ATAPI devices. FLUSH CACHE EXT (EAh, Non-Data) 48-bit command is mandatory for devices implementing the 48-bit Address feature set

For SCSI Devices, SYNCHRONIZE CACHE (10) command and /or SYNCHRONIZE CACHE (16) command shall be implemented

Windows Design Spec requirements - Controller:

For controllers and device drivers of all bus types (ATA, SCSI and USB), flush cache command shall be sent to connected device without any omission.

Exceptions: Not Specified

Business Justification:

To support Storage Controller functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: Windows 8 RC

Comments:

New

Device.Storage.Controller.Iscsi

Description:

Defines the Industry and Microsoft standards that must be met

Related Requirements:

Device.Storage.Controller.Iscsi.Interface

Device.Storage.Controller.Iscsi.Interface

Target Feature: Device.Storage.Controller.Iscsi

Title: iSCSI Interface

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86

Windows 8 Client x64
Windows 8 Server x64

Description:

iSCSI Interface

iSCSI host bus adapters must be compatible with iSCSI RFC3720 and must implement all mandatory requirements. The only exception to this is that IPsec is not mandatory but, if implemented, will be tested. All optional components, if implemented, must comply with this specification.

Optional behavior must not undermine compliance with the iSCSI specification or the Windows certification program requirements for iSCSI.

The device and drivers must also meet applicable requirements for SCSI controllers and devices.

An iSNS client must be implemented and comply with RFC3723.

An Adapter must be able to receive ping (ICMP) and send ping (ICMP).

Device logons must be consistent with the Microsoft iSCSI Discovery Service.

Any boot device configured by other means must be reported to the service after boot.

Any other persistent target assignments and sessions under control of the HBA must be reported by using WMI to the iSCSI Initiator Service when it is available.

The following logon authentication implementations are both required:

"CHAP-target authenticates initiator"

None

Mutual CHAP, if implemented, must adhere to the specification and will be tested.

IPSec support must adhere to all applicable IPSec requirements in this document.

The HBA driver must implement all required WMI interfaces documented in the WDK.

The initiator must perform an automatic logon to targets assigned to the computer as persistent targets. The initiator will connect to all persistent targets before the targets are enumerated by Windows, which starts with an Inquiry to LUN 0. If a connection drops, it will continue to try to reconnect. Devices cannot depend on the discovery service for this information.

Initiators must:

Maintain the persistent logon information in the registry or in NVRAM.

Support the new WMI class for defining/managing persistent logons.

Persist IP network adapter and discovery configurations (IP configuration information, such as static IP address, static default gateway IP address, static subnet mask, and DNS server) or use DHCP to obtain this information.

For discovery configuration, remember which discovery methods are used and, for iSNS, maintain the address of the iSNS server.

An iSCSI HBA must support changers, disk, tape, and external RAID devices.

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2009

Comments:

Taken from Storage-0022

Device.Storage.Controller.Iscsi.Boot

Description:

Defines the Industry and Microsoft standards that must be met

Related Requirements:

Device.Storage.Controller.Iscsi.Boot.FwTable

Device.Storage.Controller.Iscsi.Boot.FwTable

Target Feature: Device.Storage.Controller.Iscsi.Boot

Title: iSCSI Boot Functionality

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64

Description:

iSCSI Boot Functionality

The iSCSI boot component must:

Be compatible with iSCSI targets complying to iSCSI RFC3720.

Defer all iSCSI functionality to the Microsoft iSCSI software initiator after the Windows boot sequence begins (once ntoskrnl.exe loads).

Support Login Redirection.

Support one-way CHAP and must maintain CHAP secret in non-volatile memory.

Implement iSCSI Boot Firmware Table in firmware or Option ROM per the ACPI SIIG optional table.

Support crashdump.

Support use of IPV6 addressing.

Support iSNS (Internet Server Name Service)

Support RELOGIN as minimum error handling in the case of an error during logon initialization sequence with the iSCSI target.

Support the following DHCP option numbers: 1, 3, 17, 43, 51, 54, 57, 60, 61, 255, 201, 202, 203, 204, 205, and 206.

Support the following DHCP parameters:

DHCPDISCOVER to get an IP address assigned.

DHCPDISCOVER to get an IP address assigned.

DHCPREQUEST to obtain iSCSI protocol parameters.

DHCPINFORM to inquire updates in information from the DHCP server.

In addition:

All requests from iSCSI preboot component DHCP clients to DHCP servers must use option 60 to signal the appropriate vendor scope.

All requests from iSCSI preboot component DHCP clients to DHCP servers must use option 61 to signal the identity of this given client. If the client Alt ID is not defined, then the type field should be set to 0x01 and the EN MAC address must be used to define client identity. If the client Alt Id is defined, then the type field should be set to 0x00 and the CAID field must be used.

All requests from iSCSI preboot component DHCP clients to DHCP servers must use the CHADDR field containing the EN MAC address of the DHCP client.

The use of CIADDR in iSCSI preboot component must conform to the DHCP usage of CIADDR.

The use of YIADDR iSCSI preboot component must conform to the DHCP usage of YIADDR; namely, the iSCSI preboot component DHCP client must accept the YIADDR provided by the DHCP server during the DHCPREQUEST□PACK or DHCPINFORM□PACK transaction sequence.

The use of SIADDR in iSCSI preboot component must conform to the DHCP usage of SIADDR; namely, the iSCSI preboot component DHCP client must use this address to access the DHCP server during DHCPREQUEST□PACK or DHCPINFORM□PACK transactions.

To support DHCP option 1, the subnet mask provided in the DHCPOFFER response from iSCSI pre-boot component must provide the subnet mask.

All transactions between iSCSI preboot component DHCP clients and DHCP servers must be a single-frame transaction.

To avoid conflict with other services, iSCSI preboot component must not use DHCP option 52.

Implementation of multiple option responses in iSCSI preboot component must comply with RFC 3396.

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

Storage-0007

Device.Storage.Controller.Optical

Description:

General feature that applies to all Storage Controllers to ensure optical burning requirements are met

Related Requirements:

Device.Storage.Controller.Optical.BasicFunction

Device.Storage.Controller.Optical.BasicFunction

Target Feature: Device.Storage.Controller.Optical

Title: Storage HBA Drivers must support Optical drives

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

Controller Optical Support

The storage HBA drivers must support the optical device. The CDBs sent to the optical device and the response from the optical device must be handled properly.

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2009

Comments:

Taken from Storage-0022

Device.Storage.Controller.Raid

Description:

Defines the Industry and Microsoft standards that must be met

Related Requirements:

- Device.Storage.Controller.Raid.BasicFunction
- Device.Storage.Controller.Raid.PassThroughDiskSupport

Device.Storage.Controller.Raid.BasicFunction

Target Feature: Device.Storage.Controller.Raid

Title: RAID Controller

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64
Windows 8 Client ARM

Description:

RAID Controller

Miniport drivers for PCI RAID adapters may create a management LU if this device is always available for management commands and has a device type other than disk. Since this device will appear in device manager, a NODRV INF may be submitted to claim this device and prevent user popups (this INF may be signed).

For RAID controllers, the controller itself must correctly interpret the commands and respond in accordance with the applicable SCSI specifications, even if the controller implements RAID on a different interface type such as SATA. Any commands that are not recognized must result in a SCSI check condition with valid sense data.

SCSI Requirements can be found in Device.Storage.SCSI section.

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2009

Comments:

Taken from Storage-0022

Device.Storage.Controller.Raid.PassThroughDiskSupport

Target Feature: Device.Storage.Controller.Raid

Title: Windows Virtualized Standard Storage HBA must meet requirements

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64

Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64
Windows 8 Client ARM

Description:

If the HBA is a RAID card, it must support pass-through disks. When configured as a pass-through disk, the card must report the actual bus type and pass Persistent Reservations (PRs) through to the pass-through disks.

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: Windows 8 RC

Comments:

New

Device.Storage.Controller.Sas

Description:

Defines the Industry and Microsoft standards that must be met

Related Requirements:

Device.Storage.Controller.Sas.Interface

Device.Storage.Controller.Sas.Interface

Target Feature: Device.Storage.Controller.Sas

Title: SAS Controller Interface

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64

Windows 8 Server x64

Description:

SAS Interface

SAS host bus adapter miniport drivers must use the Microsoft hbaapi DLL to support the Windows Management Instrumentation (WMI) methods. The specific required WMI classes and methods are grouped and designated as mandatory or optional. All mandatory classes and methods must be supported. If a group is identified as optional and a miniport driver supports that group, individual methods and classes within that group are also classified as mandatory if the group is implemented or optional if the group is implemented.

Note: The SAS HBA API is currently in draft stage at the T11.5 working group. This support will not be a requirement until the draft document is complete. WHQL will issue an announcement when this support becomes a requirement.

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2009

Comments:

Taken from Storage-0022

Device.Storage.Controller.Sata

Description:

Defines the Industry and Microsoft standards that must be met

Related Requirements:

Device.Storage.Controller.Sata.Interface

Device.Storage.Controller.Sata.Interface

Target Feature: Device.Storage.Controller.Sata

Title: SATA Controller Interface

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86

Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64
Windows 8 Client ARM

Description:

SATA Interface

SATA controllers must comply with the ATA/ATAPI-7 specification

SATA controllers must support hot plug.

SATA controllers shall support Asynchronous Notification as defined in Serial ATA: High Speed Serialized AT Attachment, Version 2.6 or later and AHCI 1.3 or later.

If implemented, NCQ must be supported properly.

If implemented, larger sectors other than 512 bytes must be supported properly.

AHCI SATA controllers must comply with the AHCI 1.0 specification or later.

SATA controllers shall not emulate PATA.

Interfaces for non-AHCI SATA controllers, if implementing an interface other than AHCI, must be supported by a Windows inbox driver or must certification with a supplied driver set. The supplied drivers must meet the driver certification requirements in this document.

Recommendation: SATA controllers should implement interface power management

Recommendation: SATA controllers should implement Native Command Queuing (NCQ) support

Exceptions: Not Specified

Business Justification:

To support Storage Controller functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2009

Comments:

Taken from Storage-0022

Device.Storage.Hd

Description: Not Specified

Related Requirements:

Device.Storage.Hd.BasicFunction
Device.Storage.Hd.PhysicalSectorSizeReportsAccurately

Device.Storage.Hd.BasicFunction

Target Feature: Device.Storage.Hd

Title: All Storage Devices are tested to ensure they work correctly under stress

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64
Windows 8 Client ARM

Description:

The test will then go through the following scenarios on all HDDs

Sequential read

Sequential write

Sequential verify (write followed by read and comparison)

Random read

Random write

Random verify

These tests do not apply to SD/eMMC devices

Exceptions: Not Specified

Business Justification:

Storage devices must reliably read and write data without data loss or data corruption.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

Taken from Storage-0024

Device.Storage.Hd.PhysicalSectorSizeReportsAccurately

Target Feature: Device.Storage.Hd

Title: Reported physical sector size must be the unit of atomic write

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

Following applies to ATA based Hard Disk Drives:

If implemented, support for storage devices with logical sector sizes larger than 512-bytesneed to be implemented as described in the ATA-8 specifications. Please refer to the INCITS T13 specification repository for access to the specification.

Following applies to SCSI based Hard Disk Drives:

If implemented, support for storage devices with logical sector sizes larger than 512-bytes need to be implemented as described in the SBC-3, SPC-4, and SAT-3 specifications. Please refer to the INCITS T10 specification repository for access to the relevant specifications.

Exceptions: Not Specified

Business Justification:

Some Hard Disk Drives report the physical sector size of the disk incorrectly. For example, the drive is released a 4K drive without reporting that it is indeed a 4K drive. Applications use the reported physical sector size as a notion of atomicity and perform I/O based on this. The most basic example is a database-style application will only store one commit record within the unit of atomic write for fear of loss if power is lost or if a physical sector becomes physically bad. When the reported physical sector size is not the unit of atomicity, serious reliability concerns can arise in scenarios where power is lost such as: Applications can fail to recover, and users will need to restore from backup. Applications can fail to recover, but the application will need to perform a lengthy consistency check. Corruption of metadata, log file data, user data, or even data from other applications.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: Windows 8 RC

Comments:

New

Device.Storage.Hd.1394

Description: Not Specified

Related Requirements:

Device.Storage.Hd.1394.Compliance

Device.Storage.Hd.1394.Compliance

Target Feature: Device.Storage.Hd.1394

Title: IEEE 1394 Hard Disk Drive Specification compliance

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64
Windows 8 Client ARM

Description:

1394 Compliance

IEEE-1394 (Firewire) devices must comply with Serial Bus Protocol-2 (SBP-2) and SCSI Primary Commands-2 (SPC-2), and disk devices must comply with SCSI Reduced Block Commands (RBC).

The reference for specification compliance...:

SBP-2, SPC-2, Min:RBC

Exceptions: Not Specified

Business Justification:

To support Storage HD functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

Taken from Storage-0003

Device.Storage.Hd.Alua

Description: Not Specified

Related Requirements:

Device.Storage.Hd.Alua.Compliance

Device.Storage.Hd.Alua.Compliance

Target Feature: Device.Storage.Hd.Alua

Title: Asymmetric Logical Unit Access (ALUA)

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64

Description:

If a device supports asymmetric logical unit access (ALUA), the device must comply with the requirement of implementing target port group support (TPGS) in standard inquiry data as SPC3-r23 section 6.4.2.

The Report Target Port Group command must be supported, if logical units report in the standard Inquiry Data that they support ALUA. The storage device must comply with SPC3 r23 section 6.25 Report Target Port Group command according to its TPGS field code in the standard Inquiry data.

Exceptions: Not Specified

Business Justification:

To support Storage HD functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2007

Comments:

Taken from Storage-0003 & Storage-0007

Device.Storage.Hd.Ata

Description: Not Specified

Related Requirements:

Device.Storage.Hd.Ata.BasicFunction
Device.Storage.Hd.Ata.Dma

Device.Storage.Hd.Ata.BasicFunction

Target Feature: Device.Storage.Hd.Ata

Title: ATA/ATAPI Interface

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64

Description:

PATA Devices (legacy)

(PATA Devices will no longer be accepted for WHQL submission after June 2013.)

Microsoft recommends the use of SATA for new devices. However, in a spirit of compatibility with existing device base, the following requirements are provided for PATA devices.

Shared bus capabilities are required for PATA devices; devices shall be configurable as device 0 or device 1.

Exceptions: Not Specified

Business Justification:

To support Storage HD functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

Taken from Storage-0024

Device.Storage.Hd.Ata.Dma

Target Feature: Device.Storage.Hd.Ata

Title: ATA/ATAPI DMA Mode

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64

Description:

ATA/ATAPI DMA Mode

All PATA controllers and PATA peripherals shall support Ultra-DMA as defined in ATA/ATAPI-7.

Justification:

In addition to improved transfer rates, Ultra-DMA also provides error checking for improved robustness over previous PATA implementations.

Exceptions: Not Specified

Business Justification:

To support Storage HD functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

Taken from Storage-0024

Device.Storage.Hd.AtaProtocol

Description: Not Specified

Related Requirements:

Device.Storage.Hd.AtaProtocol.Performance
Device.Storage.Hd.AtaProtocol.Protocol

Device.Storage.Hd.AtaProtocol.Performance

Target Feature: Device.Storage.Hd.AtaProtocol

Title: ATA Device Performance

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64
Windows 8 Client ARM

Description:

ATA Device Performance

The Windows 7 Windows System Assessment Tool (WinSAT) disk formal test for the block storage device must pass the following performance requirements for any visible storage space utilization up to 95% (% of utilization as % of used space seen through the Windows file system).

Disk Sequential 64K Byte Read >25 MB/s

Disk Random 16K Byte Read >0.5MB/s

Disk Sequential 64K Byte Write >20 MB/s

Average Read Time with Sequential Writes <25 ms

Latency: 95th Percentile <120 ms

Latency: Maximum <700 ms

Average Read Time with Random Writes <40 ms

Exceptions: Not Specified

Business Justification:

To improve ATA Device Performance.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2010

Comments:

Taken from Storage-0024

Device.Storage.Hd.AtaProtocol.Protocol

Target Feature: Device.Storage.Hd.AtaProtocol

Title: ATA/ATAPI Protocol

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64
- Windows 8 Client ARM

Description:

ATA/ATAPI Protocol

ATA/ATAPI controllers and devices shall comply with the following standard(s)

INCITS 397-2005 (1532D): AT Attachment with Packet Interface 7 or later, also referred to in this document as ATA/ATAPI-7. AT Attachment with Packet Interface 8 or later will be required when this revision 8 is final and published.

ATA/ATAPI controllers shall support Windows operating system boot

ATA/ATAPI devices shall not rely on Identify Device data fields (61:60) and (103:100) to determine 28 bit or 48 bit LBA addressing support. ATA/ATAPI shall rely instead on bit 10 of word 83 and bit 10 of word 86 to identify 48 bit LBA addressing support (as per ATA/ATAPI-7).

Design notes

Recommended:

Reporting Nominal Media Rotation Rate

If a device requires Windows defragmentation to be turned off by default, the device should report its Nominal Media Rotation Rate as 0001h Non-rotating media (e.g. solid state device) as per the ATA8-ACS1 specification, section 7.16.7.77.

Justification:

When the Nominal Media Rotation Rate reported by the device is anything but 0001h Non-rotating media, Windows will by default perform defragmentation of the block storage device.

Exceptions: Not Specified

Business Justification:

To support Storage HD functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

Taken from Storage-0024

Device.Storage.Hd.DataVerification

Description:

Disk Verification Tests to ensure there is no data loss or corruption

Related Requirements:

Device.Storage.Hd.DataVerification.BasicFunction

Device.Storage.Hd.DataVerification.BasicFunction

Target Feature: Device.Storage.Hd.DataVerification

Title: All Storage Devices will work correctly on Windows

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64
Windows 8 Client ARM

Description:

Storage devices must reliably read and write data without data loss or data corruption

Exceptions: Not Specified

Business Justification:

Storage devices must reliably read and write data without data loss or data corruption

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

Taken from Storage-0024

Device.Storage.Hd.Ehdd

Description: Not Specified

Related Requirements:

Device.Storage.Hd.Ehdd.Compliance

Device.Storage.Hd.Ehdd.Compliance

Target Feature: Device.Storage.Hd.Ehdd

Title: Encrypted Hard Drive complies with Microsoft and Industry specifications

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64
Windows 8 Client ARM

Description:

Encrypted Hard Drive

eDrives must be compliant with these industry specifications

- IEEE 1667 version IEEE 1667-2009
  - Support Probe silo
  - Support TCG Storage silo
- TCG
  - TCG core specification version2.0
  - OPAL SSC 2.1
    - Programmatic TPer Reset Rev
    - Modifiable CommonName Column
    - SID Authority Disable
  - OPAL SSC Feature Sets
    - Set Additional Data Store Rev 1.05 or later
    - Single User Mode Rev 1.02 or later
- SCSI
  - SPC4
  - SAT2
- ATA
  - ACS2

eDrives must comply with these Windows Design Spec requirements:

- Support at least AES128
- Support one or more of the following cipher modes
  - CBC
  - XTS
- Support at least8 bands
- Support additional data store tables
- Support range crossing
- Support authenticate method
- Support secret protect info

Support modifiable common name

Support TCG stack reset

Support programmatic TPer reset

Support single user mode

If SCSI devices(SPC4):-

The 1667 Version Descriptor, 0xFFC2 (IEEE 1667-2009)should be reported in the INQUIRY data. The Additional Length field of the INQUIRY data must be greater than or equal to0x38.

Security Protocol IN output must report 00, 01, 02, EE in the Supported Security Protocol List payload

If ATA devices (ACS2):-

The TrustedComputer.FeatureSupported (word 48 - bit 0must be set to 1) must be reported in the IDENTIFY data

The AdditionalSupported.IEEE1667IDENTIFY (word 69 - bit 7 should be set to 1) must be reported in the IDENTIFY data

Trusted Receive output should report 00,01, 02, EE in the Supported Security Protocol List payload

If SATA-USB:-

Support SAT2

Command Performance:- The drive must complete the following operations within the specified duration

Operations	Max completion time
Discovery/Enumeration	24 sec (8 bands)
Activate	45 sec
Revert	8 sec
Create Band	1.5 sec
Delete Band	1.5 sec
Erase Band	2sec
Set Metadata	20sec
Get Metadata	14sec
Lock Band	1.5 sec
Unlock Band	1.5 sec

Exceptions: Not Specified

Business Justification:

To ensure Encrypted Hard Drive compliance

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2007

Comments:

Storage-8002

Device.Storage.Hd.EnhancedStorage

Description: Not Specified

Related Requirements:

Device.Storage.Hd.EnhancedStorage.1667Compliance

Device.Storage.Hd.EnhancedStorage.1667Compliance

Target Feature: Device.Storage.Hd.EnhancedStorage

Title: Enhanced Storage Devices comply with the IEEE 1667 defined standards

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64
Windows 8 Client ARM

Description:

IEEE1667-Class (Enhanced Storage) enabled storage devices meet industry standards

Enhanced Storage devices comply with the IEEE 1667 defined standards.

Enhanced Storage device must:

Support authenticating the host

Implement support for IEEE 1667 (version 1.1 or later) defined Probe Silo on the device.

Implement at least one Certificate or Password Silo on the device.

Enhanced Storage device that implements Certificate Silo must:

Load native windows certificate silo driver?

Responds to all commands of the IEEE 1667 version 1.1 specification

Verify certificate-based authentication is used to allow and block access to volume.

Enhanced Storage device that implements Password Silo must:

Load native password silo driver

Respond to all commands in the IEEE 1667 Password Silo specification?

Verify password-based authentication is used to allow and block access to the volume.

Design Notes:

Obtain IEEE 1667 specification from IEEE at the following location:

http://go.microsoft.com/fwlink/?LinkID=110100

Exceptions: Not Specified

Business Justification:

To support Storage HD functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

Storage-0019

Device.Storage.Hd.FibreChannel

Description: Not Specified

Related Requirements:

Device.Storage.Hd.FibreChannel.Compliance

Device.Storage.Hd.FibreChannel.Compliance

Target Feature: Device.Storage.Hd.FibreChannel

Title: Fibre Channel Devices

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64

Description:

Fibre Channel devices must comply with Fibre Channel Protocol for SCSI, Second Version (FCP-2) or later. To ensure interoperability at the electrical and signaling levels, Fibre Channel devices must comply with Third-Generation Fibre Channel Physical and Signaling Interface (formerly ANSI X3.303:1998).

Exceptions: Not Specified

Business Justification:

To support Storage HD functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2007

Comments:

Taken from Storage-0003

Device.Storage.Hd.Flush

Description: Not Specified

Related Requirements:

Device.Storage.Hd.Flush.BasicFunction

Device.Storage.Hd.Flush.BasicFunction

Target Feature: Device.Storage.Hd.Flush

Title: Flush Command Completion

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86

Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64
Windows 8 Client ARM

Description:

Industry standard spec requirements:

For ATA device, FLUSH CACHE (E7h, Non-Data) 28-bit command is optional for ATA devices and ATAPI devices. FLUSH CACHE EXT (EAh, Non-Data) 48-bit command is mandatory for devices implementing the 48-bit Address feature set

For SCSI Devices, SYNCHRONIZE CACHE (10) command and /or SYNCHRONIZE CACHE (16) command shall be implemented

Windows Design Spec requirements - HDD:

Correct Reporting of Completion: When the OS issues a flush cache command, the storage device should synchronously report Completion of the command only when the content of cache has been persisted.

Exceptions: Not Specified

Business Justification:

To support Storage HD functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: Windows 8 RC

Comments:

New

Device.Storage.Hd.Hybrid

Description: Not Specified

Related Requirements:

Device.Storage.Hd.Hybrid.Piton

Device.Storage.Hd.Hybrid.Piton

Target Feature: Device.Storage.Hd.Hybrid

Title: Devices that implement the NV-Cache command set support industry Microsoft standards

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64
Windows 8 Client ARM

Description:

Devices that implement the NV-Cache command set support industry Microsoft standards

A device or system that contains a device that implements a non-volatile memory cache and either the non-volatile memory cache (NV-Cache) command set as defined by T13 ATA-ASC or the NV Cache Manager I/O Control Codes must meet the following certification requirements.

This device or system is known as a device under test (DUT)

A minimum of 50-MB NV Cache must be implemented in the DUT and exposed to Windows.

The NV-Cache must be able to perform according to the following scenarios:

Scenario Block Size (512 byte aligned) Throughput

Random Read 4KB 4MB/sec

Random Write 4KB 4MB/sec

Sequential Read 64KB 16MB/sec

Sequential Write 64KB 8MB/sec (Microsoft recommends 16MB/sec)

Random Read 1MB 16MB/sec

Random Write 1MB 10MB/sec

Immediately following a transition from device power state OFF to ON the DUT must be able to service a 512 bytes IO that is pinned in the NV-Cache in less than 3 seconds. This requirement is used to ensure the boot and resume benefits of the NV-Cache are realized. This requirement does not apply to storage adapters implementing NV-Cache command set.

In normal operation, the maximum read/write latencies for data in the NV-Cache should be:

less than or equal to 3 millisecond for a random 4K block

less than or equal to 4 millisecond for a random 4.5k block

The DUT must fully comply with ATA 7.0 with all additions enumerated in "e05106r7-ACS-NV_Cache_Command_Proposal" or ATA 8.0 when ratified.

Windows must be able to perform the following tasks when the DUT is installed on a system:

Detect that the device supports the NV-Cache commands when installed.

Enter and return from NV-Cache power mode.

Pin boot LBAs to enhance boot performance.

Exceptions: Not Specified

Business Justification:

To ensure standards compliance.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2007

Comments:

Storage-0005

Device.Storage.Hd.Iscsi

Description: Not Specified

Related Requirements:

Device.Storage.Hd.Iscsi.BasicFunction

Device.Storage.Hd.Iscsi.BasicFunction

Target Feature: Device.Storage.Hd.Iscsi

Title: iSCSI Devices

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64

Description:

iSCSI devices

iSCSI devices must comply with RFC3720, RFC3721, and RFC3723.

Device must complete testing by using the Microsoft iSCSI initiator.

Device must be able to receive ping (ICMP) and send ping (ICMP).

The following iSCSI protocol features are required:

Send Targets.

Logon Authentication: CHAP and none. Targets may delegate CHAP authentication to Radius.

Discovery Session Logon key/value pairs: InitiatorName, SessionType, and AuthMethod.

Normal Session Logon key/value pairs: InitiatorName, SessionType, AuthMethod, and TargetName.

DataPDUInOrder.

DataSequenceInOrder.

DefaultTime2Wait.

DefaultTime2Retain

ErrorRecoveryLevel=0.

Targets that allow different shared secrets for different initiator names.

The following iSCSI protocol features must pass testing if they are implemented:

Mutual CHAP.

HeaderDigest: CRC32 and none.

DataDigest: CRC32 and none.

InitialR2T.

IPsec; when using IPsec, Main mode must be available. In addition, the following items are required when IPsec is implemented:

IPsec transport mode must be implemented.

Internet key exchange (IKE) implementations must support main mode and preshared keys. Target portals with the same IP address must expect the identical main mode IKE policy.

Targets and initiators must allow different preshared keys for different identifier payloads.

Targets and initiators must have static IP addresses for main mode.

Additional Standard Inquiry data VERSION DESCRIPTORS (SPC-3) are required

At least one iSCSI VERSION DESCRIPTOR is required (value = 0960h).

Exceptions: Not Specified

Business Justification:

To support Storage HD functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2007

Comments:

Taken from Storage-0003

Device.Storage.Hd.Mpio

Description: Not Specified

Related Requirements:

Device.Storage.Hd.Mpio.BasicFunction

Device.Storage.Hd.Mpio.BasicFunction

Target Feature: Device.Storage.Hd.Mpio

Title: RAID implementations that provide a multipathing solution must comply with Microsoft multipath I/O (MPIO)

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86

Windows 8 Client x64
Windows 8 Server x64

Description:

Internal or external RAID implementations that provide a multipathing solution must comply with Microsoft multipath I/O (MPIO). Windows multipathing solutions must consist of a Device Specific Module (DSM) created by using the Microsoft MPIO DDK and must comply with all requirements set forth in the Multipath I/O Program Agreement.

Following WMI classes must be implemented by 3rd party DSM .

DSM_QuerySupportedLBPolicies

DSM_QueryUniqyeId

3rd party DSM must report minor, major version numbers for the DSM

Exceptions: Not Specified

Business Justification:

To support Storage HD functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2007

Comments:

Taken from Storage-0007

Device.Storage.Hd.OffloadedDataTransfer

Description:

Windows Offloaded Data Transfer

Related Requirements:

Device.Storage.Hd.OffloadedDataTransfer.CopyOffload

Device.Storage.Hd.OffloadedDataTransfer.CopyOffload

Target Feature: Device.Storage.Hd.OffloadedDataTransfer

Title: If Copy Offload is supported these requirements must be implemented

Applicable OS Versions:

Windows 8 Client x86

Windows 8 Client x64
Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client ARM
Windows 8 Server x64
Windows Server 2008 Release 2 x64

Description:

Industry standard spec requirements:

Targets that support Windows Copy Offload feature must implement T10 XCOPY Lite specification (11-059r8):

Supported VPD Pages VPD page (Must include ECOP VPD Page (Page Code 8Fh) in the Supported VPD page list)

ECOP VPD page or ECOP VPD Page (Page Code 8Fh) + Block Device ROD Limits ECOP descriptor (0000h)

Block Limit VPD Page (Page Code B0h)

According to T10 11-059r8 spec, Windows adopts 83h OP Code + 10 Service Action Code for POPULATE TOKEN and 83h OP Code + 11 Service Action Code for WRITE USING TOKEN commands.

According to T10 11-059r8 spec, Windows adopts 84h OP Code + 07 Service Action Code for RECEIVE ROD TOKEN INFORMATION command. Response service action field and command parameters shall be compliant with T10 XCOPY Lite spec (11-059r8)

Windows Design Spec requirements:

During the target device enumeration, Windows will send down an Inquiry for Supported VPD Pages VPD page. If 8F is included in Supported VPD page list, Windows will inquiry for ECOP VPD page and BLOCK LIMITs VPD page.

Implementation and Error Handling with Parameters of ECOP VPD page

The MAXIMUM RANGE DESCRIPTORS - If the number of Block Device Range Descriptors of a POPULATE TOKEN or WRITE USING TOKEN command exceeds the MAXIMUM RANGE DESCRIPTORS, copy manager shall terminate the command with CHECK CONDITION status with the sense key set to ILLEGAL REQUEST and the additional sense code set to TOO MANY SEGMENT DESCRIPTORS.

The MAXIMUM INACTIVITY TIMER (MAXIMUM IAT) - If the INACTIVITY TIMEOUT of a POPULATE TOKEN command exceeds the MAXIMUM INACTIVITY TIMER, copy manager shall terminate the command with CHECK CONDITION status with the

sense key set to ILLEGAL REQUEST and the additional sense code set to INVALID FIELD IN PARAMETER LIST.

The MAXIMUM TOKEN TRANSFER SIZE

If the sum of the NUMBER OF LOGICAL BLOCKS fields in all block device range descriptors of the WRITE USING TOKEN command is greater than the MAXIMUM TOKEN TRANSFER SIZE, copy manager shall terminate the command with CHECK CONDITION status with the sense key set to ILLEGAL REQUEST and the additional sense code set to INVALID FIELD IN PARAMETER LIST.

If the sum of the NUMBER OF LOGICAL BLOCKS fields in all block device range descriptors of the POPULATE TOKEN is greater than the MAXIMUM TOKEN TRANSFER SIZE, copy manager shall terminate the command with CHECK CONDITION status with the sense key set to ILLEGAL REQUEST and the additional sense code set to INVALID FIELD IN PARAMETER LIST.

Implementation and Error Handling with Parameters of Block Limits VPD page

The MAXIMUM TRANSFER LENGTH field indicates the maximum transfer length in blocks that the copy manager accepts for a single BLOCK DEVICE RANGE DESCRIPTOR. If a copy manager receives a request for a NUMBER OF LOGICAL BLOCKS exceeding this maximum, then the copy manager shall terminate the command with CHECK CONDITION status with the sense key set to ILLEGAL REQUEST and the additional sense code set to INVALID FIELD IN PARAMETER LIST.

Storage array must support both synchronous and asynchronous POPULATE TOKEN and WRITE USING TOKEN according to T10 11-059r8, 11-078r4 and 11-204r0 spec.

Storage array must complete synchronous POPULATE TOKEN and WRITE USING TOKEN commands in a very short time (4 seconds) without causing any SCSI command timeout.

User Experience Requirements:

Fall back to Legacy Copy Windows copy offload operation shall be able to fall back legacy copy operation when a copy offload error or limitation is reported.

Drag and drop copy experience must be able support drag and drop copy with copy offload capable storage target device.

Exceptions: Not Specified

Business Justification:

Storage devices must reliably read and write data without data loss or data corruption.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: Windows 8 RC

Comments:

New

Device.Storage.Hd.PersistentReservation

Description: Not Specified

Related Requirements:

Device.Storage.Hd.PersistentReservation.ClusterFailover

Device.Storage.Hd.PersistentReservation.ClusterFailover

Target Feature: Device.Storage.Hd.PersistentReservation

Title: Cluster Failover for RAID Array systems

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64

Description:

Required: All Microsoft MPIO device-specific modules (DSMs) must be Windows Hardware Certification-qualified and support registering and unregistering persistent reservations across all paths.

Design Notes:

All host bus adaptors (HBA) used on failover cluster nodes can use only a Windows Hardware Certification-qualified miniport driver based on the Storport miniport model.

All multipath I/O solutions leveraged on highly available failover clusters must be based on Microsoft MPIO.

It is recommended that in addition to the standard HCT qualification all solutions are also validated with the "Microsoft Cluster Configuration Validation Wizard" (ClusPrep) tool.

FC, iSCSI, and particularly serial-attached SCSI (SAS) failover cluster solutions cannot be built on RAID HBAs where cache and/or RAID configuration is machine/node specific. The RAID set

information and hardware cache must reside in a single shared point that lives in an external storage controller.

SAS, FC, and iSCSI have no restrictions as to the number of nodes they support (which currently is 8nodes).

Note: Legacy parallel-SCSI server clusters were restricted to a maximum size of 2nodes.

Only SAS devices using the Serial SCSI Protocol (SSP) transport will be supported on failover clusters (including SAS JBOD or any SAS SSP RAID systems). SATA devices attached to a SAS domain must be part of a RAID system.

SATA direct attach and SATA JBOD is not supported; the system must include RAID.

If the system disks are attached to a bus type that is not a valid type for shared storage (something other than FC, iSCSI, or SAS), then the system disks and shared storage must be on separate physical controllers/host bus adaptors.

Exceptions: Not Specified

Business Justification:

To improve stability.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

Storage-0006

Device.Storage.Hd.RaidArray

Description: Not Specified

Related Requirements:

Device.Storage.Hd.RaidArray.BasicFunction
Device.Storage.Hd.RaidArray.BitLocker

Device.Storage.Hd.RaidArray.BasicFunction

Target Feature: Device.Storage.Hd.RaidArray

Title: RAID Array Systems

Applicable OS Versions:

Windows Server 2008 x86

Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64

Description:

RAID Requirements

RAID systems and devices must support commands from SBC-2 (or later) regardless of the drive interface implemented.

RAID controllers and RAID systems must support, at a minimum, one of: RAID1, RAID 5, RAID6 or RAID 1/0.

External RAID arrays must allow a failed drive that is redundant to be replaced manually without shutting down or halting the system. This requirement includes, but is not limited to, drives in a mirror set, a physical drive being replaced by a "hot spare," and the first failed drive in a RAID level-5 array. The RAID subsystem must also allow lost data to be rebuilt without interfering with system operations. It is expected that RAID array throughput will be impacted during the rebuild.

Exceptions: Not Specified

Business Justification:

To support Storage HD functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2007

Comments:

Taken from Storage-0007

Device.Storage.Hd.RaidArray.BitLocker

Target Feature: Device.Storage.Hd.RaidArray

Title: BitLocker must not cause data corruption on Storage Arrays

Applicable OS Versions:

Windows 8 Server x64

Description:

BitLocker must be properly enabled to protect Data volumes on Storage Arrays

Exceptions: Not Specified

Business Justification:

(1) When a server is placed in an environment without adequate physical security, BitLocker protects data on the server against unauthorized access if a server is stolen; (2) When hosting service providers repurpose or decommission storage arrays, BitLocker Disk Encryption prevents data breach.

Scenarios:

It is part of BitLocker for Windows Server scenarios.

Success Metric: A storage array needs to pass BitLocker test.

Enforcement Date: August 15, 2011

Comments: Not Specified

Device.Storage.Hd.ReadZeroOnTrimUnmap

Description: Not Specified

Related Requirements:

Device.Storage.Hd.ReadZeroOnTrimUnmap.BasicFunction

Device.Storage.Hd.ReadZeroOnTrimUnmap.BasicFunction

Target Feature: Device.Storage.Hd.ReadZeroOnTrimUnmap

Title: The requirement applies to Hard Disk Drives

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64
Windows 8 Client ARM

Description:

If the logical block provisioning read zeros (LBPRZ) bit is set to one, then the device server shall set all bits to zero in the Data-In Buffer for read operation on an unmapped (deallocated or anchored) LBA

Exceptions: Not Specified

Business Justification:

This requirement is necessary for Storage Devices to function properly

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: Windows 8 RC

Comments: Not Specified

Device.Storage.Hd.Sas

Description: Not Specified

Related Requirements:

Device.Storage.Hd.Sas.ComplyWithIndustrySpec

Device.Storage.Hd.Sas.ComplyWithIndustrySpec

Target Feature: Device.Storage.Hd.Sas

Title: Serial Attached SCSI devices comply with industry specifications

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64
Windows 8 Client ARM

Description:

The reference for specification compliance. Where noted as Min, the baseline specification is mandatory. Rec indicates the preferred version of the specification. If not otherwise specified, the version listed is the minimum required. Unless otherwise indicated, all features of the cited specifications that are classified as mandatory by the standards body must be implemented.

SAS-1, SAM-3, SPC-3, Min:SBC, Rec: SBC-2

Serial Attached SCSI devices comply with the Serial Attached SCSI (SAS) Specification 1 or later.

Exceptions: Not Specified

Business Justification:

To support Storage HD functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2007

Comments:

Storage-0003

Device.Storage.Hd.Sata

Description: Not Specified

Related Requirements:

Device.Storage.Hd.Sata.BasicFunction

Device.Storage.Hd.Sata.BasicFunction

Target Feature: Device.Storage.Hd.Sata

Title: ATA Device Performance

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64
Windows 8 Client ARM

Description:

SATA Devices

Requirement: SATA devices shall meet the requirements of the Serial ATA: High Speed Serialized AT Attachment, Version 2.6 or later.

Requirement: SATA devices support hot-plug functionality

Recommendation: SATA devices should implement interface power management

Recommendation: SATA devices should implement Native Command Queuing (NCQ) support

Exceptions: Not Specified

Business Justification:

To support Storage HD functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: Windows 8 RC

Comments:

New

Device.Storage.Hd.Scsi

Description: Not Specified

Related Requirements:

Device.Storage.Hd.Scsi.Connectors
Device.Storage.Hd.Scsi.ParallelInterface

Device.Storage.Hd.Scsi.Connectors

Target Feature: Device.Storage.Hd.Scsi

Title: SCSI Connectors

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64

Description:

SCSI Connectors

If an external connector is implemented, it must meet the requirements in SCSI or a later specification. The SCSI connector must not use the same connector type as any other non-SCSI connector on the system. All external parallel SCSI connectors must be labeled with ANSI-approved

icon for the bus. For internal and external configurations, the SCSI bus cable must be plugged into shrouded and keyed connectors on the host adapter and devices. This ensures that the cable is properly positioned so the user cannot plug in cables incorrectly. For internal configurations, pin-1 orientation must be designated on one edge of the ribbon cable and also on the keyed connector for the SCSI peripheral device.

Exceptions: Not Specified

Business Justification:

To support Storage HD functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2007

Comments:

Storage-0003

Device.Storage.Hd.Scsi.ParallelInterface

Target Feature: Device.Storage.Hd.Scsi

Title: Parallel SCSI Interface

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64

Description:

Parallel SCSI Interface

Parallel SCSI devices and adapters comply with SCSI Parallel Interface-4 (SPI-4) or later.

Termination: Automatic termination circuit and SCSI terminators meet SPI-4 standard or later. Parallel SCSI host controllers and adapters must use automatic termination that allows a user to add external devices without removing the server case. Terminators used in the SCSI host adapter must be regulated terminators, which are also known as active, SCSI SPI-4, or Boulay terminators. SCSI termination built onto internal cables must also meet the SPI-4 specification.

Terminator power must be supplied to the SCSI bus with overcurrent protection. The host adapter must supply terminator power (TERMPWR) to the SCSI bus for system-board implementations by using PCI or another expansion bus. All terminators on the external SCSI bus must be powered from the TERMPWR lines in the SCSI bus. In addition, the circuit that supplies TERMPWR must have overcurrent protection built into it.

External removable disks, hard drives, and CD/DVD optical drives must provide automatic termination or an accessible on-board termination switch. At a minimum, a mechanical means must be provided for setting termination and the switch must be accessible to the user without opening the device chassis.

All SCSI devices supporting hot plugging must comply with annex D of SPI-4, which addresses SCSI device insertion and removal, with and without command activity.

Differential devices must support DIFFSENS as defined in SPI-4 standard or later.

Exceptions: Not Specified

Business Justification:

To support Storage HD functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2007

Comments:

Storage-0003

Device.Storage.Hd.ScsiEnclosureService

Description: Not Specified

Related Requirements:

Device.Storage.Hd.ScsiEnclosureService.Compliance

Device.Storage.Hd.ScsiEnclosureService.Compliance

Target Feature: Device.Storage.Hd.ScsiEnclosureService

Title: Devices that support Storage Spaces must meet the following requirements

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows Server 2008 x86

Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client ARM
Windows 8 Server x64
Windows Server 2008 Release 2 x64

Description:

All the requirements apply if device supports SES.

All Storage Disks must be certified with the Windows Hardware Certification Kit

It is recommended that a Storage Chassis be used.

All disks in the chassis must be able to be enumerated by all controllers in the chassis.

If there are multiple ports into the chassis, then each port must enumerate the same number of disks.

If the Storage Disks are SAS, then the chassis must support the following subset of SCSI Enclosure System (SES):

The ability to power LEDs to indicate a disk-of-interest, e.g., failed disk

The ability to retrieve the slot number of a particular disk

If the Storage Disks will be used with Windows clustering, then the following are required:

Persistent Reservation (PR) must be honored for every host-accessible disk in the chassis

All Storage Disks must support multiple initiators.

Additionally following requirements must be met

The Disk drives in SES enclosure and the SES enclosureshall support VPD page 0x83 AND need to provide an identifier with association type 1 (port association). The identifier value needs to be the SAS address for the port. This value is mapped against the SAS address reported for the slot through SES commands.

Expect following diagnostic pages to be supported from SES enclosure for drive/slot mapping: 00, 0x1, 0x07, 0x0A

Require that for page 0x0A, the device provides additional element status description with EIP bit = 1. i.e. provide data in format as described in Table-27. Protocol Specific information for SAS protocol is mandatory. i.e. Table 33, 35 and 37 need to be supported. The SAS address field in table 37 needs to match the SAS address specified by the device (refer 2 above).

For identification LED blink, Control page 0x02 is mandatory.

The order of elements specified starts with Disk Slots.

The Disk slots are in ascending order and stored in the ElementIndex field.

If there is a match for a lot in 2 above, then control element to that slot in 0x02 should work.

Receive Diagnostics Results 0x02 page for getting the status.

Response time of these commands must be < 500 ms.

The enclosure must support blinking of both Identify and Failure LED.

Exceptions: Not Specified

Business Justification:

–

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: Windows 8 RC

Comments:

New

Device.Storage.Hd.ScsiProtocol

Description: Not Specified

Related Requirements:

Device.Storage.Hd.ScsiProtocol.ReferenceSpec
Device.Storage.Hd.ScsiProtocol.SamCompliance
Device.Storage.Hd.ScsiProtocol.SpcCompliance

Device.Storage.Hd.ScsiProtocol.ReferenceSpec

Target Feature: Device.Storage.Hd.ScsiProtocol

Title: Reference to Specifications

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86

Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64
Windows 8 Client ARM

Description:

Where noted as Min, the baseline specification is mandatory. Rec indicates the preferred version of the specification. If not otherwise specified, the version listed is the minimum required. Unless otherwise indicated, all features of the cited specifications that are classified as mandatory by the standards body must be implemented.

SPI-4, SAM-3, Min:SPC-2, Rec: SPC-3, Min: SBC, Rec: SBC-2

Exceptions: Not Specified

Business Justification:

To support Storage HD functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2007

Comments:

Storage-0003

Device.Storage.Hd.ScsiProtocol.SamCompliance

Target Feature: Device.Storage.Hd.ScsiProtocol

Title: SCSI Architecture Model SAM-3

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64
Windows 8 Client ARM

Description:

SCSI Architecture Model SAM-3

SCSI Devices must comply with SCSI Architecture Model SAM-3 or later (except as noted in SBP-2 for 1394 devices), including the following requirements:

- All devices must support LUN reset. In particular, if two LUNs L0 and L1 under the same target have outstanding commands, a LUN reset to L0 must clear any outstanding commands to L0 only.
- Following a reset, all devices must return an appropriate unit attention condition to any initiator currently having access to the logical unit.
- All FC, iSCSI, SCSI, and SAS devices must support multiple initiators.
- MODE SELECT commands that change parameters must cause a unit attention condition to be raised for any other initiator consistent with SAM-3.
- LUN 0 must be implemented for all targets. At a minimum, LUN 0 must respond to INQUIRY and all multi-LUN targets must support the REPORT LUNS commands.
- If any LUN is added or removed that is accessible to the initiator(s), the device must report a unit attention condition of (06/3F/0E) REPORT LUNS DATA HAS CHANGED MODE SELECT. Commands that change parameters must cause a unit attention condition to be raised for any other initiator that would be impacted by the change.
- Any unrecognized SCSI command or incorrectly formed command descriptor block (CDB) must result in an immediate CHECK CONDITION reported back to the initiator.

Exceptions: Not Specified

Business Justification:

To support Storage HD functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2007

Comments:

Storage-0003

Device.Storage.Hd.ScsiProtocol.SpcCompliance

Target Feature: Device.Storage.Hd.ScsiProtocol

Title: SCSI Primary Commands-3 (SPC-3)

Applicable OS Versions:

- Windows Server 2008 x86

Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64
Windows 8 Client ARM

Description:

SCSI Primary Commands-3 (SPC-3)

Devices must comply with SCSI Primary Commands-3 (SPC-3): support commands listed as mandatory in the SCSI Primary Commands (SPC-3 or later). In addition, each device type must implement the mandatory command set for that type (SBC for block devices and so on).

For SCSI INQUIRY and REPORT LUNS commands:

All devices must support the SCSI INQUIRY command.

Multi-LUN devices must always respond to an INQUIRY command sent to LUN 0 even if LUN 0 is not implemented. This can be indicated by returning the Device Type Qualifier of 3.

All multi-LUN devices must support the REPORT LUNS command as defined in SPC-2 or later.

Windows supports only single-level logical unit numbers up to 255; see SAM-3. Use of any other format will be incorrectly interpreted, and the device may not be available or data corruption will occur.

All standard INQUIRY data must be correctly set for the device capabilities.

VERSION Field must be 04 or greater. For SAS, this field must be 05 or greater.

Drives with attached media changers must set the MChgnr bit in the standard inquiry data.

Multi-LUN units must return valid REPORT LUNS data for LUN 0.

If LUN 0 is not an accessible PERIPHERAL DEVICE TYPE, the PERIPHERAL QUALIFIER shall be returned as 1. SCSIport will not enumerate the entire target device if a qualifier of 3 is used. It is strongly recommended that LUN 0 not be type 0 because it must be exposed to all initiators. Type 0 is permitted only if the array can map different logical units to LUN 0 for each initiator.

If a device has more than one port, MultiP bit must be set and page 83h descriptors must correctly reflect the port information.

Vital Product Data (VPD) pages:

Page 0 (Supported Pages) is required.

Page 83h (Device Identification) is required. For VPD Page 83, at least one type-3 or one type-2 descriptor must be returned for each logical unit, the value must use Code Set 1 (Binary), the value must be unique for that logical unit, and it must be the same value regardless of the path or port responding to the request. Appropriate descriptors for multiport devices are required in addition to that mandatory descriptor. Devices that support aliases must also support the corresponding descriptor types. Vendor-specific device identifiers, if present, must use type 0 and must follow the specified format, including correct page length; vendor-specific identifiers are not a substitute for the mandatory type 2/3 descriptors. All device identifiers must conform to formatting rules set forth in SPC-3 or later, even if the device claims only conformance to a previous release.

Device must comply with SPC-3 section 7.6.3 Device Identification VPD page 83h

At least one identification descriptor must have the IDENTIFIER TYPE field set to:

2h (i.e., EUI-64-based) as defined in 7.6.3.5

3h (i.e., NAA); or as defined in 7.6.3.6

8h (i.e., SCSI name string) at defined in 7.6.3.11

Page 83h PROTOCOL IDENTIFIER and PIV bit are recommended. SAS Device IDs must set the PROTOCOL IDENTIFIER equal to 06 (SAS).

Page 80h (serial number) is required and must be in ASCII.

SCSI Mode Sense Command and Pages

MODE SENSE (6) is mandatory for all devices except RBC devices, which implement MODE SENSE (10). The DBD bit must be supported.

Mode Page 3f (all pages) is mandatory.

Device type-specific pages listed in the device-specific sections of this document.

Additional commands for all devices are as follows:

All devices must support the TEST UNIT READY and REQUEST SENSE commands.

Block Storage (Disk and RAID) Devices

Block storage (disk and RAID) devices must comply with the following requirements:

SCSI block commands (SBC) or later (RBC for 1394). These requirements apply to any device reporting as Device Type 0, including logical units exposed by a RAID controller or subsystem.

Block Devices must support the SCSI START STOP UNIT command to decrease power consumption.

READ CAPACITY (10) command. If a device has more than 232 - 1 sectors, a value of 0xFFFFFFFF must be returned for the RETURNED LOGICAL BLOCK ADDRESS field and the READ CAPACITY (16) command must be supported (see below).

Any change to capacity must set a unit attention condition of CAPACITY DATA HAS CHANGED for all initiators with access to the logical unit.

READ(10).

WRITE(10). Support for force unit access (FUA) is mandatory for individual physical disk drives or RAID controllers that contain volatile (non-battery-backed) cache memory and must cause the data sent with this command to be committed to physical media before the command completes.

REASSIGN BLOCKS (hard disks only). RAID controllers that handle bad block replacement should succeed this command.

VERIFY (10).

START STOP UNIT. This command must not perform any other action, such as path failover.

SYNCHRONIZE CACHE (10) (no optional fields are used). For a physical disk drive, this command causes all data in the write cache to commit to physical media if write caching is enabled. Failure to follow this can result in data corruption.

Mode pages:

Mode Page 8 (caching page) with the following bits must contain valid information: WCE (Write Cache Enable), CACHE SEGMENT SIZE, and NUMBER OF CACHE SEGMENTS (optional). RBC devices support Page 6 instead of Page 8 (WCD, WRITED, FORMATD, and LOCKD bits).

If a device supports disabling write caching through the use of the WCE bit, this bit must also be reported as changeable and be supported by a MODE SELECT operation which modifies it. The status of the write caching must be visible by reading Mode Page 8. Vendors can implement caching policies outside of the limited SBC ones, and disabling of write cache does not need to be through this mode page.

Disk devices that support greater than 2-TB logical units (including 1394 disks). Devices must conform to SPC-3 and must implement all of the following in accordance with the SCSI Block Commands-2 (SBC-2) specification:

READ CAPACITY (16)

READ (16)

WRITE (16) FUA (bit must be supported if a volatile cache is present on the device)

VERIFY (16)

REASSIGN BLOCKS. LONGLBA field must be supported.

Erasable SCSI Disk Devices

Erasable SCSI disk devices must also support the following commands or features:

ERASE: Full-side and selected-block erase.

Format requirements reported with FORMAT command.

MODE SENSE (6) Total spare blocks available, write protect status.

PREVENT ALLOW MEDIUM REMOVAL and START STOP UNIT.

REASSIGN BLOCKS and READ DEFECT DATA (10).

WRITE without pre-erase, for erasable optical only.

Exceptions: Not Specified

Business Justification:

To ensure compliance.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2007

Comments:

Storage-0003

Device.Storage.Hd.ThinProvisioning

Description: Not Specified

Related Requirements:

Device.Storage.Hd.ThinProvisioning.BasicFunction

Device.Storage.Hd.ThinProvisioning.BasicFunction

Target Feature: Device.Storage.Hd.ThinProvisioning

Title: Thin Provisioning

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64

Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64

Description:

Industry standard spec requirements:

Targets that support thin provisioning feature must implement following T10 SPC4 and SBC3 specification:

Supported VPD Page VPD Page (Page Code 00h)

Block Limit VPD Page (Page Code B0h)

Logical Block Provisioning VPD Page (Page Code B2h)

Logical Block Provisioning Log Page (Page Code 0Ch)

Windows Design Spec requirements:

Target devices with thin provisioning feature must meet the following requirements.

Must return Inquiry command for Supported VPD Page VPD Page with B0h and B2h.

Must return LBPU bit set to one and Provisioning Type field = 3 (010b) of Logical Block Provisioning VPD page (Page Code B2).

Must implement Block Limit VPD Page (Page Code B0h) and support the following parameters.

MAXIMUM UNMAP LBA COUNT

MAXIMUM UNMAP BLOCK DESCRIPTOR COUNT

OPTIMAL UNMAP GRANULARITY

UNMAP GRANULARITY ALIGNMENT

UGAVALID Bit

Must implement Logical Block Provisioning VPD Page (Page Code B2h) and support the following parameters.

Threshold Exponent

LBPU bit

LBPRZ bit

Provisioning Type field

Thin Provisioning target devices should support Log sense command to retrieve Logical Block Provisioning Log Page (Page Code 0Ch) for adding the following information into the threshold notification system event log.

Used LBA mapping resources of a Thin Provisioning LUN.

Available LBA mapping resources to the Thin Provisioning LUN.

Storage array must support UNMAP (10) command, the LBPU bit in LBP VPD page shall set to one.

Must support Get LBA Status (16) command according to T10 SBC3 spec.

If the LBPME bit in ReadCapacity(16) return is set to one or B2h is reported in the Supported VPD Page VPD Page, the storage array must support Logical Block Provisioning VPD page (Page Code B2h)

If the LBPRZ bit in ReadCapacity(16) return is set to one, the storage array must set LBPRZ bit of Logical Block Provisioning VPD page to one.

Storage array must support threshold notification (TN), temporary resource exhaustion (TRE) and permanent resource exhaustion (PRE) through the following sense key, additional sense code and additional sense code qualifier returns as SPC4 and SBC3 specs.

TN Sense Key/ASC/ASCQ (06/38/07)

TRE Sense Key/ASC/ASCQ (02/04/14)

PRE Sense Key/ASC/ASCQ (07/27/07)

User Experience Requirements:

Must be able to set threshold through vendors storage management utility and monitor system event log when the thin provisioning soft threshold is reached.

Must support Log Sense command to retrieve LBP log page for reporting available LBA mapping resource and used LBA mapping resource information to the thin provisioning LUN, if Log Page (Page Code OCh) is implemented.

Exceptions: Not Specified

Business Justification:

To support Storage HD functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2007

Comments:

New / updated some parameters and added to requirement.

Device.Storage.Hd.Trim

Description: Not Specified

Related Requirements:

Device.Storage.Hd.Trim.BasicFunction

Device.Storage.Hd.Trim.BasicFunction

Target Feature: Device.Storage.Hd.Trim

Title: ATA Trim Functionality

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64
Windows 8 Client ARM

Description:

ATA Trim Functionality

If the device implements ATA non-NCQ Trim support:

The Trim implementation shall comply with ATA ACS2 Section 7.10 (Data Set Management Commands).

Command completion time shall not exceed 20ms or 8ms * (number of LBA range entries), whichever is greater, and shall always be less than 600ms.

Device shall return all '0's to Read command before trimmed block(s) is re-written.

Exceptions: Not Specified

Business Justification:

To support Storage HD functionality and scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: Windows 8 RC

Comments:

Taken from Storage-0024. Para #3 is New

Device.Storage.Hd.Uas

Description: Not Specified

Related Requirements:

Device.Storage.Hd.Uas.Compliance

Device.Storage.Hd.Uas.Compliance

Target Feature: Device.Storage.Hd.Uas

Title: USB UAS Storage Devices

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64
Windows 8 Client ARM

Description:

USB UASP Storage devices must also be compliant with the USB UASP v1.0 and SPC4, SBC3

USB UASP Storage devices must support the following:

Mode page code: 0x08 Mode subpage code : 00

Block Limits page - 0xB0 SPC3 6.5.3

Support at least 16 streams

Support task management commands

Note: for further information on mode pages see SPC4: D.6 Mode page codes

Support SPC, SBC version descriptors

Support/report R02. R04 version descriptors

Device must report FIXED if it is not a true removable media (RMB=0)

Note: for further information on mode pages see SPC4: D.6 Mode page codes

Data Devices must perform as indicated:

Minimum sequential write speed: 120MB/s

Minimum sequential read speed: 170MB/s

Exceptions: Not Specified

Business Justification:

To ensure USB UASP v1.0 and SPC4, SBC3 compliance.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: Windows 8 RC

Comments:

Taken from Storage-8003

Device.Storage.Hd.UasOnEHCI

Description: Not Specified

Related Requirements:

Device.Storage.Hd.UasOnEHCI.BasicFunction

Device.Storage.Hd.UasOnEHCI.BasicFunction

Target Feature: Device.Storage.Hd.UasOnEHCI

Title: USB UAS Storage Devices

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64

Windows 8 Client ARM

Description:

If the device supports UASP on XHCI and then it must support UASP on EHCI.

Exceptions: Not Specified

Business Justification:

To ensure smooth operation of UAS devices on UAS

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: Windows 8 RC

Comments:

Taken from Storage-8003

Device.Storage.Hd.Usb

Description: Not Specified

Related Requirements:

Device.Storage.Hd.Usb.Compatibility

Device.Storage.Hd.Usb.Compatibility

Target Feature: Device.Storage.Hd.Usb

Title: USB Compatibility

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64
Windows 8 Client ARM

Description:

USB Compatibility

All USB storage devices must meet the requirements of the Universal Serial Bus Mass Storage Class Specification Overview, V1.2 Revision. This includes all USB Mass Storage class documents, including Bulk Only, Control/Bulk/Interrupt, Bootability, and UFI Command specifications.

BOT1.0, SPC-2, SBC-2

USB 3.0 devices must retain backward compatibility at the Type-A connector to allow Superspeed devices to be used, albeit at a lower speed, with USB 2.0 PCs and allow high speed devices with their existing cables to be connected to the USB 3.0 Superspeed Type-A connectors.

USB storage devices must comply with USB 3.0 Section 11 Interoperability and Power Delivery specs. The following table lists the compatibility matrix for USB3.0 and USB2.0. The implication of identifying a host port as supporting USB3.0 is that both hardware and software support for USB3.0 is in place; otherwise the port shall only be identified as a USB2.0 port.

USB Host Port	USB Device Capability	Connected Mode
USB 2.0	USB 2.0	USB 2.0 high-speed, full-speed, or low-speed
	USB 3.0	USB 2.0 high-speed
USB 3.0	USB 2.0	USB 2.0 high-speed, full-speed, or low-speed
	USB 3.0	USB 3.0 SuperSpeed

USB Storage Devices must comply with certification requirement for USB devices and USB Storage Devices

Note:

Please refer to USB3.0 spec section 3.1.4 USB 3.0 Architecture summary

USB3.0 Super-speed 5 Gb/s

USB2.0 high-speed - 480 Mb/s

Full-speed 12 Mb/s

Low-speed 1.5 Mb/s

Exceptions: Not Specified

Business Justification:

To ensure USB compatibility.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2007

Comments:

Storage-0001 & New UAS details

Device.Storage.Hd.Usb3

Description: Not Specified

Related Requirements:

Device.Storage.Hd.Usb3.Compliance

Device.Storage.Hd.Usb3.Compliance

Target Feature: Device.Storage.Hd.Usb3

Title: USB 3.0 Storage Devices

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64
Windows 8 Client ARM

Description:

USB 3.0 Storage devices support industry specifications as indicated below

All USB 3.0 Storage devices must be compliant with the USB 3.0 Version 1.0 specification

Provide unique product identification through each storage end point (BOT, UASP)

USB VID/PID

Data Devices must perform as indicated:

Minimum sequential write speed: 60MB/s

Minimum sequential read speed: 90MB/s

Exceptions: Not Specified

Business Justification:

To ensure USB 3.0 compliance.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: Windows 8 RC

Comments:

Taken from Storage-8003, UPDATED.

Device.Storage.Hd.WindowsToGoCapableUSBDrive

Description:

Windows To Go Capable USB Drive feature

Related Requirements:

Device.Storage.Hd.WindowsToGoCapableUSBDrive.WindowsToGoCapableUSBDrive

Device.Storage.Hd.WindowsToGoCapableUSBDrive.WindowsToGoCapableUSBDrive

Target Feature: Device.Storage.Hd.WindowsToGoCapableUSBDrive

Title: Windows To Go Capable USB Drive

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64

Description:

This requirement only applies to USB Storage devices that support USB Boot.

USB boot devices must be USB 3.0 and meet industry specifications as indicated below

All USB 3.0 Storage devices must be compliant with the USB 3.0 Version 1.0 specification

USB UASP Storage devices must also be compliant with the USB UASP version 1.04 specification and the SCSI SPC 4 specification, SBC3

USB BOT Storage devices must also be compliant with USB BOT specification

USB boot devices must also:

Include in the MS OS Descriptor extended property the value WindowsBootCapable DWORD value -1

Be at least 32GB in size (25GB useable)

Support Trim/unmap command

Provide unique, consistent product identification

USB VID/PID

Inquiry Serial Number

Inquiry Model Number

Device must report FIXED (RMB=0)

Support the following mode pages

Mode page code: 0x08 Mode subpage code : 00

Meet the following performance requirements:

Random 4k Write IOPs >= 200

Random 4k Read IOPs >= 2000

Sequential write speed >= 40 MB/s

Sequential read speed >= 60 MB/s

Max I/O Latency < 500 milliseconds

Additional I/O Latency requirement:

Maximum of 20 seconds sum-total of user-perceivable I/O latencies over any 1 hour period of a user-representative workload, where a user-perceivable I/O is defined as having a latency of at least 100 millisecond

UASP Storage devices must support the following as well, BOT device need not support

Support at least 16 streams

Support task management commands

Note: for further information on mode pages see SPC4: D.6 Mode page codes

Check version descriptor for R02, R04

Check version for SPC, SBC

Exceptions: Not Specified

Business Justification:

To support USB Boot scenarios.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: Windows Next RC.

Comments: Not Specified

Device.Storage.Optical

Description: Not Specified

Related Requirements:

Device.Storage.Optical.CdRawRecording
Device.Storage.Optical.CommandPerformance
Device.Storage.Optical.DriveDefinition
Device.Storage.Optical.Features
Device.Storage.Optical.MmcVersion
Device.Storage.Optical.Profiles
Device.Storage.Optical.RealTimeStreaming

Device.Storage.Optical.CdRawRecording

Target Feature: Device.Storage.Optical

Title: Optical Drives must support CD RAW Recording

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64
Windows 8 Client ARM
Windows Vista Client x86
Windows Vista Client x64

Description:

Optical drives must support CD RAW Recording Mode for CD-R and CD-RW profiles.

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

Taken from Storage-0026

Device.Storage.Optical.CommandPerformance

Target Feature: Device.Storage.Optical

Title: Optical Drives must complete Performance Command within allowed time frames

Applicable OS Versions:

- Windows Server 2008 x86
- Windows 8 Server x64
- Windows 8 Client x64
- Windows 8 Client x86
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86
- Windows Server 2008 x64
- Windows 8 Client ARM
- Windows Vista Client x86
- Windows Vista Client x64

Description:

Optical Drives must complete all commands within the maximum allowed times according to the following table.

COMMAND	Basic certification requirement (msec)	Exception (exception criteria/ msec)
GET CONFIGURATION	20	
GET EVENT STATUS NOTIFICATION	20	
GET PERFORMANCE	20	
INQUIRY	20	
MECHANISM STATUS	20	
MODE SELECT	20	
MODE SENSE	20	
PREVENT ALLOW MEDIUM REMOVAL	20	
READ TOC PMA ATIP	20	
READ BUFFER CAPACITY	20	
READ CAPACITY	20	
READ CD[1,2,4]	500	
READ DISC INFORMATION	50	
READ FORMAT CAPACITIES	20	

READ TRACK INFORMATION	20	
REQUEST SENSE (when not following a command failed with error status)	20	
SEND OPC INFORMATION	60000	For dual layer media, i.e. DVD+R DL, DVD-R DL, BD-RE DL / 70000
SET READ AHEAD	20	
SET STREAMING	20	
START STOP UNIT (Immed=1b)	20	
START STOP UNIT (Eject, Immed=0b)	7000	
START STOP UNIT (Inject, Immed=0b, until media is ready)	25000	DVD-RAM / 40000 For dual layer media, i.e. DVD+R DL, DVD-R DL, BD-RE DL / 30000
SYNCHRONIZE CACHE (Immed=1b)	20	
SYNCHRONIZE CACHE (Immed=0b)	15000	
TEST UNIT READY	20	
BLANK (Immed=1b)	20	
BLANK (Immed=0b)	N/A	
CLOSE TRACK SESSION (Immed=1b)	20	
CLOSE TRACK SESSION (Immed=0b, close logical track or session, do not finalize disc)	65000	DVD+R DL, DVD-R DL / 300000 DVD-R SL, DVD-RW / 180000
CLOSE TRACK SESSION (Immed=0b, finalize disc)	65000	DVD+R DL, 300000 DVD-R SL, DVD-R DL DVD-RW / 900000
FORMAT UNIT (Immediate)	20	
LOAD/UNLOAD MEDIUM	N/A	

READ10[1,2,4]	500	DVD-RAM / 800
READ12 (not streaming)[1,2,4]	500	DVD-RAM / 800
READ12 (streaming)[1,2,4]	100	CD at 1X / 500; CD at 2x / 350; CD at 4x / 200
READ DISC STRUCTURE[6]	20	
READ MEDIA SERIAL NUMBER[2]	500	
REPORT KEY	20	
RESERVE TRACK	N/A	
SEND CUE SHEET	N/A	
SEND DISC STRUCTURE	N/A	
SEND KEY	20	
SET CD SPEED	20	
WRITE 10 (FUA=0) [1,3]	50	
WRITE 12 (FUA=0) [1,3]	50	
WRITE BUFFER	N/A	
ERASE	N/A	
READ BUFFER	N/A	
READ CD MSF[1,2,4]	500	
REPAIR TRACK	N/A	
SEEK10	N/A	
VERIFY 10	N/A	
WRITE AND VERIFY 10	N/A	

Command completion time is defined as the time between a command leaving the Microsoft port / miniport driver and the command completion being returned to the Microsoft port / miniport driver. If the command is failed with error status, this time also includes the subsequent Request Sense command leaving the Microsoft port / miniport driver and the Request Sense command completion being returned to the Microsoft port / miniport driver.

All the command execution time performance measurement should be performed on media conforming to media physical layer standard specification from associated committees - i.e. DVD Forum, BDA, DVD+RW Alliance. Also, they should be performed under normal temperature and humidity operational condition as declared in the device specification.

Note: Read-Only drives will be retired from the Windows certification Program on June 01, 2010. Hence, certification requirements and tests will cease to exist for Read-Only drives on June 01, 2010. Partners who wish to receive Windows certification on systems with Read-Only drives would still be a able to. However, the Read-Only drive would fall under the "unclassified" category of devices.

[1]: Transfer length for the read and write performance tests is equal or smaller to a single ECC block (32 or 64 KB depending of the current media type, i.e. 64KB for CD and BD, 32KB for DVD).

[2]: Performance tests may be exercised at any speed reported as supported by the device, including 1x media speed if so reported as supported.

[3]: Drive ***must*** make use of write buffer and ***shall not*** delay command completion by any form of media access. If write buffer is full, drive ***must*** fail the write command with long write in progress sense information (02h/04h/08h).

[4]: The first hundred read I/O commands after media arrival or resume from StandBy power state or Set Cd Speed or Set Streaming are permitted a delay up to a cumulative total of60 000 msec to complete to allow for additional spin-up time. These commands individually may take any duration up to a limit of 7 000 msec, but the cumulative time to complete all hundred commands shall not exceed 60 000 msec. Only the time between when a read command is sent to the device and that read command is completed by the device is accounted for, the time between two successive read commands is not accounted for (i.e. host delays are not measured).

[6]: The list of disc structure codes is limited to; physical format information (Format = 0x00, Address = 0), DVD-RAM medium status (Format = 0x09, Address = 0), DVD+RW write inhibit DCB (Format = 0x30 Address = 0x57444300), write protection status (Format = 0xC0 Address = 0)[7]:Dual Layer Write profile *will be required on 1 June 2010* for Blu-Ray drives of 9.5 mm height and smaller as well as DVD drives 7mm height and smaller. This ends the previous exception for these form factors.

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2010

Comments:

Taken from Storage-0026

Device.Storage.Optical.DriveDefinition

Target Feature: Device.Storage.Optical

Title: How Optical Drives are defined for certification

Applicable OS Versions:

- Windows Server 2008 x86
- Windows Server 2008 x64
- Windows 7 Client x86
- Windows 7 Client x64
- Windows Server 2008 Release 2 x64
- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Server x64

Windows 8 Client ARM
Windows Vista Client x86
Windows Vista Client x64

Description:

To be an Optical Drive, the device must be defined as CD (Compact Disc) device, DVD (Digital Versatile Disc or Digital Video Disc) device, BD (Blu-Ray Disc) device or any device which identifies itself as Peripheral Device Type 5 per INCITSs T10s command set SCSI Primary Commands, SPC (any revision).

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

Taken from Storage-0026

Device.Storage.Optical.Features

Target Feature: Device.Storage.Optical

Title: Required Optical Drive Features

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64
Windows 8 Client ARM
Windows Vista Client x86
Windows Vista Client x64

Description:

Optical Drives must support the required Features listed below

Core Feature

Profile List Feature

Mandatory features per profile

Removable medium feature from Mt. Fuji 7

Reporting correct tray status

Power management feature

Morphing Feature

Drive Serial Number Feature

DVD CSS Feature (0106h)

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

Taken from Storage-0026

Device.Storage.Optical.MmcVersion

Target Feature: Device.Storage.Optical

Title: Optical Drives must comply with MMC

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64
Windows 8 Client ARM
Windows Vista Client x86
Windows Vista Client x64

Description:

Optical Drives must conform to INCITSs T10s command set, MultiMedia Command Set 6 (MMC-6), when published. Because the publication of MMC-6 has been delayed, Optical Drives must in the interim conform to the combination of INCITSs T10s command set MultiMedia Command Set 5

(MMC-5) and SFFs Mt. Fuji Commands for Multimedia Devices Version 7 (INF-8090i v7) until publication of MMC-6. If and when MMC-5 and INF-8090i v7 contradict each other, and the following requirements do not specify explicitly the required behavior, compliance to MMC-5 is required (with the exception of features newly defined in INF-8090i v7).

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

Taken from Strorage-0026

Device.Storage.Optical.Profiles

Target Feature: Device.Storage.Optical

Title: Required Optical Drive Profiles

Applicable OS Versions:

- Windows Server 2008 x86
- Windows 8 Server x64
- Windows 8 Client x64
- Windows 8 Client x86
- Windows Server 2008 Release 2 x64
- Windows 7 Client x64
- Windows 7 Client x86
- Windows Server 2008 x64
- Windows 8 Client ARM
- Windows Vista Client x86
- Windows Vista Client x64

Description:

Optical Drives must support the required profiles as listed below

- CD-ROM
- DVD-ROM
- Removable Disk
- CD-R
- CD-RW

DVD-R Sequential Recording

DVD-RW Restricted Overwrite

DVD-R Dual Layer Sequential Recording

DVD+RW

DVD+R

DVD+R DL

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2010

Comments:

Taken from Storage-0026

Device.Storage.Optical.RealTimeStreaming

Target Feature: Device.Storage.Optical

Title: Optical Drives must support Real Time Streaming

Applicable OS Versions:

Windows Server 2008 x86
Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64
Windows 8 Client ARM
Windows Vista Client x86
Windows Vista Client x64

Description:

Optical Drives must support Real Time Streaming as required according to Profile requirements. For all recordable and rewritable profiles, the following fields shall be set accordingly: Stream Writing (SW)=1b and Write Speed Performance Descriptor (WSPD)=1b.

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2006

Comments:

Taken from Storage-0026

Device.Storage.Optical.BluRayReader

Description: Not Specified

Related Requirements:

Device.Storage.Optical.BluRayReader.Profiles

Device.Storage.Optical.BluRayReader.Profiles

Target Feature: Device.Storage.Optical.BluRayReader

Title: Required Profiles for BluRay Readers

Applicable OS Versions:

Windows Server 2008 x64
Windows 7 Client x86
Windows 7 Client x64
Windows Server 2008 Release 2 x64
Windows 8 Client x86
Windows 8 Client x64
Windows 8 Server x64
Windows 8 Client ARM
Windows Vista Client x86
Windows Vista Client x64
Windows Server 2008 x86

Description:

BluRay Reader drives must support BD-ROM profile.

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2010

Comments:

Taken from Storage-0026

Device.Storage.Optical.BluRayWriter

Description: Not Specified

Related Requirements:

Device.Storage.Optical.BluRayWriter.Profiles

Device.Storage.Optical.BluRayWriter.Profiles

Target Feature: Device.Storage.Optical.BluRayWriter

Title: Required Profiles for BluRay Writers

Applicable OS Versions:

Windows Server 2008 x86
Windows 8 Server x64
Windows 8 Client x64
Windows 8 Client x86
Windows Server 2008 Release 2 x64
Windows 7 Client x64
Windows 7 Client x86
Windows Server 2008 x64
Windows 8 Client ARM
Windows Vista Client x86
Windows Vista Client x64

Description:

BluRay Drives that can write must support BD-ROM, BD-R Sequential Recording and BD-RE profiles.

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2010

Comments:

Taken from Storage-0026

Device.Storage.Optical.Sata

Description: Not Specified

Related Requirements:

Device.Storage.Optical.Sata.AsynchronousNotification

Device.Storage.Optical.Sata.AsynchronousNotification

Target Feature: Device.Storage.Optical.Sata

Title: Asynchronous Notification is Required for all SATA connected drives.

Applicable OS Versions:

Windows Server 2008 x86
Windows 8 Server x64
Windows 8 Client x64
Windows 8 Client x86
Windows Server 2008 Release 2 x64
Windows 7 Client x64
Windows 7 Client x86
Windows Server 2008 x64
Windows 8 Client ARM
Windows Vista Client x86
Windows Vista Client x64

Description:

Optical Drives that connect via the SATA bus must support Asynchronous Notification.

Exceptions: Not Specified

Business Justification: Not Specified

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: 6/1/2010

Comments:

Taken from Storage-0026

Device.Streaming.HMFT

Description:

Hardware Media Foundation Transform

Related Requirements:

Device.Streaming.HMFT.Decoding
Device.Streaming.HMFT.Encoding

Device.Streaming.HMFT.Decoding

Target Feature: Device.Streaming.HMFT

Title: Hardware Media Foundation Transform (HMFT) supports video decoding

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Client ARM

Description:

Supported Formats

HMFT video decoder is only supported for MPEG-4 Part 2 and MJPEG.

Media Foundation Compliance

The video decoder Hardware Media Foundation Transform (HMFT) must fully comply with the following Media Foundation Transform (MFT) interfaces:

IMFTransform

IMFMediaEventGenerator

IMFShutdown

IMFQualityAdvise

IMFRealTimeClientEx

The HMFT video decoder must use Media Foundation work queue support (no thread creation) via **IMFRealTimeCLientEx::SetWorkQueue**. This ensures the following:

Multimedia Class Scheduler Service (MMCSS) support for playback and capture/encode

Improved core scalability

The HMFT video decoder must support **IMF2DBuffer2** for enhanced security, but also work with input buffers of type **IMF2DBuffer**, and **IMFMediaBuffer** in that order of preference.

DirectX Rendering

The video decoder HMFT must support both DirectX(DX)X9 and DX11 devices, and it must avoid copies in or out of DX11 components.

On MFT_Set_D3D_Manager, the video decoder HMFT must first query for DirectX Graphics Infrastructure (DXGI) Manager and then query for D3D9 Manager if DXGI Manager is not found.

The HMFT video decoder must support system memory output because some transforms in the pipeline may support only system memory buffers.

If the HMFT video decoder is based on GPU must support DX11.

Memory Usage

The HMFT video decoder must be an asynchronous MFT. This reduces the memory working set by about 50 percent.

Trusted Merit Certification and Verification

The video decoder HMFT must support the Trusted Merit Certification and Verification process, as defined across the Windows Hardware Certification Kit.

Each HMFT video decoder must be provided as a separate binary and must be individually signed.

HMFT video decoders must not advertise support for more than one compression standard.

All HMFTs must set the following Media Foundation attributes while registering the MFT with the system:

MFT_ENUM_HARDWARE_URL_Attribute

MFT_ENUM_HARDWARE_VENDOR_ID_Attribute

Format Requirements

HMFT video decoders must not advertise support for inbox formats supported by DirectX Video Acceleration (DXVA) (H.264, WMV, MPEG2).

If implemented, the HMFT video decoder for MPEG-4 Part 2 must support Simple and Advanced Simple Profile (If Global Motion Compensation (GMC) is not supported, then the media type must be rejected to allow the software decoder to be used for playback), and all levels.

The decoder must be fully conformant to specifications that are defined for the format.

The MPEG-4 Part 2 decoder must fully support H.263baseline content and advertise support for this media type.

In addition to the preceding requirements, we recommend that the decoder support post-processing for deblocking and deringing.

Vendors may provide other HMFTs video decoders for formats that are not supported inbox, but there are no verification tests or logo certification available.

Note: The recommendations and requirements that are defined in this document apply to all formats.

Functionality

The video decoder HMFT must support the following functionalities:

Dynamic format and resolution changes

Trick modes (playback rate control, thinning mode) and seek

Performance

The HMFT video decoder must be able to decode 40 megabits per second (Mbps) at 1080p in real time.

Interlace Support

The HMFT video decoder must support the input format for both interlaced and progressive bit streams. It must not de-interlace. It may support inverse telecine(IVTC).

Multiple Instances

The HMFT video decoder must support multiple instances of the decoder in parallel (both in-process and out of process) to enable multiple concurrent video playback streams in the same or different applications.

Design Notes

The HMFT video decoder must be installed and uninstalled through a device driver that meets Windows security requirements. The driver must not cause the operating system to crash or hang, and must disallow memory violations.

Each HMFT component must be a separate binary, individually certified and signed.

Exceptions: Not Specified

Business Justification:

HMFT is a feature that enables Independent Hardware Vendors (IHVs) to provide to hardware media solutions for non-DXVA supported formats.DX9 support is required for older applications. DX11 support is required for the new features, scenarios and improved performance.

Scenarios:

Hardware accelerated video playback via Media Foundation for playback and transcode scenarios. On the playback side this includes playback in Windows Media Player, Media Center, Windows Live Movie Maker and Video Tag in Internet Explorer 9. On the transcode side, this includes transcoding for Play To home media streaming, and synchronizing to the device.

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments: Not Specified

Device.Streaming.HMFT.Encoding

Target Feature: Device.Streaming.HMFT

Title: Hardware Media Foundation Transform (HMFT) supports video encoding

Applicable OS Versions:

Windows 8 Client x86
Windows 8 Client x64
Windows 8 Client ARM

Description:

A. H.264 Encode

If your hardware supports H.264 Encode, you must:

A.1 On input:

A.1.1 Support NV12

A.1.2 If your hardware supports it:

A.1.2.1 Support IYUV and YUY2

A.1.3 Support input buffers of (and query in this order):

A.1.3.1 IMF2DBuffer2

A.1.3.2 IMF2DBuffer

A.1.3.3 IMFMediaBuffer

A.2 On output:

A.2.1 Support Baseline profile

A.2.2 Support Constrained Baseline profile

A.2.3 Support Main profile

A.3 Your H.264 encoder must expose the following interfaces:

A.3.1IMFTransform

A.3.2 ICodecAPI

A.3.2.1 ICodecAPI requires the following functions to be implemented:

A.3.2.1.1 IsSupported

A.3.2.1.2 GetValue

A.3.2.1.3 SetValue

A.3.2.2 ICodecAPI requires the following properties to be supported:

A.3.2.2.1 Peak-constrained VBR mode

A.3.2.2.2 Quality-based VBR mode

A.3.2.2.3 CBR encoding

A.3.2.2.4 GOP distance

A.3.2.2.5 Frame-QP control

A.3.2.2.6 Adaptive bitrate control

A.3.2.2.7 Force-Key-frame control

A.3.2.2.8 QualityVsSpeed control

A.3.2.2.9 Low-latency encoding

A.3.2.2.10 Temporal-layer encoding

A.3.2.3 It is recommended that you additionally implement the following functions:

A.3.2.3.1 GetDefaultValue

A.3.2.3.2 GetParameterRange

A.3.2.3.3 GetParameterValues

A.3.2.3.4 IsModifiable

A.3.2.3.5 SetAllDefaults

A.3.3 IMFAttributes

A.3.3.1 Via IMFTransform::GetAttributes

A.3.3.1 2 The two required attributes are:

A.3.3.12.1 MFT_ENUM_HARDWARE_URL_Attribute

A.3.3.12.2 MFT_ENUM_HARDWARE_VENDOR_ID_Attribute

A.3.4 It is recommended that you implement an asynchronous MFT

A.3.4.1 Asynchronous MFTs require the following additional interfaces:

A.3.4.1.1 IMFMediaEventGenerator

A.3.4.1.2 IMFShutdown

A.3.4.2 Asynchronous MFTs are encouraged to avoid creating threads and are recommended to use the MF Thread Pool

A.3.4.2.1 Registration with MMCS via IMFRealTimeCLientEx::SetWorkQueue is critical to meet performance goals

A.3.5 IMFRealTimeClientEx is recommended

A.4 Encoding settings

A.4.1 Through input media type negotiation, the H.264 Encoder must support:

A.4.1.1 MF_MT_INTERLACE_MODE

A.4.1.1.1 The encoder must preserve interlace from input to output or reject interlace

A.4.1.2 MF_MT_MINIMUM_DISPLAY_APERTURE

A.4.2 Through output media type negotiation, the H.264 Encoder must support:

A.4.2.1 MF_MT_SUBTYPE

A.4.2.2 MF_MT_MINIMUM_DISPLAY_APERTURE

A.4.2.3 MF_MT_FRAME_RATE

A.4.2.4 MF_MT_AVG_BITRATE

A.4.2.5 MF_MT_MPEG2_PROFILE

A.4.2.6 MF_MT_MPEG2_LEVEL

A.4.2.7 MF_MT_PIXEL_ASPECT_RATIO

A.4.3 It is recommended that your H.264 Encoder supports:

A.4.3.1 B frame encoding

A.5 Multiple Instances

A.5.1 It is required that your H.264 encoder must support a minimum of 3 concurrent instances

A.5.1.1 These instances may be in the same process or in different processes

A.6 Merit Validation

A.6.1 It is required that your H.264 encoder supports the trusted merit verification process

A.6.2 It is required that your H.264 encoder be a separate binary, individually certified and signed

A.7 Additional Requirements

A.7.1 Your H.264 encoder must work with the Windows MP4 file sink

A.7.2 Your H.264 encoder must implement proper order of encoding configuration

A.8 Installation

A.8.1 It is required that your H.264 encoder is registered and unregistered along with the device driver used in the encoder

A.9 Performance

A.9.1 It is required that your H.264 encoder must be capable of real-time encoding 1920x1080x24fps up to 12 Mbps

B. MPEG-2 Encode

On x86 and x64 systems, if your hardware supports MPEG-2 Encode, you must:

(MPEG-2 Encode is not supported on Windows on ARM systems)

B.1 On input:

B.1.1 Support NV12

B.1.2 If your hardware supports it:

B.1.2.1 Support IYUV and YUY2

B.1.3 Support input buffers of (and query in this order):

B.1.3.1 IMF2DBuffer2

B.1.3.2 IMF2DBuffer

B.1.3.3 IMFMediaBuffer

B.2 On output:

B.2.1 Support Simple profile

B.2.2 Support Main profile

B.2.3 Support High profile

B.3 Your MPEG-2 encoder must expose the following interfaces:

B.3.1 IMFTransform

B.3.2 ICodecAPI

B.3.2.1 ICodecAPI requires the following functions to be implemented:

B.3.2.1.1 IsSupported

B.3.2.1.2 GetValue

B.3.2.1.3 SetValue

B.3.2.2 ICodecAPI requires the following properties to be supported:

B.3.2.2.1 Peak-constrained VBR mode

B.3.2.2.2 Quality-based VBR mode

B.3.2.2.3 CBR encoding

B.3.2.2.4 GOP distance

B.3.2.2.5 Frame-QP control

B.3.2.2.6 Adaptive bitrate control

B.3.2.2.7 Force-Key-frame control

B.3.2.2.8 QualityVsSpeed control

B.3.2.2.9 Low-latency encoding

B.3.2.3 It is recommended that you additionally implement the following functions:

B.3.2.3.1 GetDefaultValue

B.3.2.3.2 GetParameterRange

B.3.2.3.3 GetParameterValues

B.3.2.3.4 IsModifiable

B.3.2.3.5 SetAllDefaults

B.3.3 IMFAttributes

B.3.3.1 Via IMFTransform::GetAttributes

B.3.3.2 The two required attributes are:

B.3.3.2.1 MFT_ENUM_HARDWARE_URL_Attribute

B.3.3.2.2 MFT_ENUM_HARDWARE_VENDOR_ID_Attribute

B.3.4 It is recommended that you implement an asynchronous MFT

B.3.4.1 Asynchronous MFTs require the following additional interfaces:

B.3.4.1.1 IMFMediaEventGenerator

B.3.4.1.2 IMFShutdown

B.3.4.2 Asynchronous MFTs are encouraged to avoid creating threads and are recommended to use the MF Thread Pool

B.3.4.2.1 Registration with MMCS via IMFRealTimeCLientEx::SetWorkQueue is critical to meet performance goals

A.3.5 IMFRealTimeClientEx is recommended

B.4 Encoding Settings

B.4.1 Through input media type negotiation, the MPEG-2 Encoder must support:

B.4.1.1 MF_MT_INTERLACE_MODE

B.4.1.1.1 The encoder must preserve interlace from input to output or reject interlace

B.4.1.2 MF_MT_MINIMUM_DISPLAY_APERTURE

B.4.2 Through output media type negotiation, the MPEG-2 Encoder must support:

B.4.2.1 MF_MT_SUBTYPE

B.4.2.2 MF_MT_MINIMUM_DISPLAY_APERTURE

B.4.2.3 MF_MT_FRAME_RATE

B.4.2.4 MF_MT_AVG_BITRATE

B.4.2.5 MF_MT_MPEG2_PROFILE

B.4.2.6 MF_MT_MPEG2_LEVEL

B.4.2.7 MF_MT_PIXEL_ASPECT_RATIO

B.4.3 It is recommended that your MPEG-2 Encoder supports:

B.4.3.1 B frame encoding

B.5 Multiple Instances

B.5.1 It is required that your MPEG-2 encoder must support a minimum of 3 concurrent instances

B.5.1.1 These instances may be in the same process or in different processes

B.6 Merit Validation

B.6.1 It is required that your MPEG-2 encoder supports the trusted merit verification process

B.6.2 It is required that your MPEG-2 encoder be a separate binary, individually certified and signed

B.7 Additional Requirements

B.7.1 Your MPEG-2 encoder must work with the Windows MPEG PS/TS file sink

B.7.2 Your MPEG-2 encoder must implement proper order of encoding configuration

B.8 Installation

B.8.1 It is required that your MPEG-2 encoder is registered and unregistered along with the device driver used in the encoder

B.9 Performance

B.9.1 It is required that your MPEG-2 encoder must be capable of real-time encoding 1280x720x30fps up to 7 Mbps

C. VC-1 Encode

If your hardware supports VC-1 Encode, you must:

C.1 On input:

C.1.1 Support NV12

C.1.2 If your hardware supports it:

C.1.2.1 Support IYUV and YUY2

C.1.3 Support input buffers of (and query in this order):

C.1.3.1 IMF2DBuffer2

C.1.3.2 IMF2DBuffer

C.1.3.3 IMFMediaBuffer

C.2 On output:

C.2.1 Support Simple profile

C.2.2 Support Main profile

C.2.3 Support Advanced profile

C.3 Your VC-1 encoder must expose the following interfaces:

C.3.1 IMFTransform

C.3.2 ICodecAPI

C.3.2.1 ICodecAPI requires the following functions to be implemented:

C.3.2.1.1 IsSupported

C.3.2.1.2 GetValue

C.3.2.1.3 SetValue

C.3.2.2 ICodecAPI requires the following properties to be supported:

C.3.2.2.1 Peak-constrained VBR mode

C.3.2.2.2 Quality-based VBR mode

C.3.2.2.3 CBR encoding

C.3.2.2.4 GOP distance

C.3.2.2.5 Frame-QP control

C.3.2.2.6 Adaptive bitrate control

C.3.2.2.7 Force-Key-frame control

C.3.2.2.8 QualityVsSpeed control

C.3.2.2.9 Low-latency encoding

C.3.2.3 It is recommended that you additionally implement the following functions:

C.3.2.3.1 GetDefaultValue

C.3.2.3.2 GetParameterRange

C.3.2.3.3 GetParameterValues

C.3.2.3.4 IsModifiable

C.3.2.3.5 SetAllDefaults

C.3.3 IMFAttributes

C.3.3.1 Via IMFTransform::GetAttributes

C.3.3.2 The two required attributes are:

C.3.3.2.1 MFT_ENUM_HARDWARE_URL_Attribute

C.3.3.2.2 MFT_ENUM_HARDWARE_VENDOR_ID_Attribute

C.3.4 It is recommended that you implement an asynchronous MFT

C.3.4.1 Asynchronous MFTs require the following additional interfaces:

C.3.4.1.1 IMFMediaEventGenerator

C.3.4.1.2 IMFShutdown

C.3.4.2 Asynchronous MFTs are encouraged to avoid creating threads and are recommended to use the MF Thread Pool

C.3.4.2.1 Registration with MMCS via IMFRealTimeCLientEx::SetWorkQueue is critical to meet performance goals

C.3.5 IMFRealTimeClientEx is recommended

C.4 Encoding Settings

C.4.1 Through input media type negotiation, the VC-1 Encoder must support:

C.4.1.1 MF_MT_INTERLACE_MODE

C.4.1.1.1 The encoder must preserve interlace from input to output or reject interlace

C.4.1.2 MF_MT_MINIMUM_DISPLAY_APERTURE

C.4.2 Through output media type negotiation, the VC-1 Encoder must support:

C.4.2.1 MF_MT_SUBTYPE

C.4.2.2 MF_MT_FRAME_SIZE

C.4.2.3 MF_MT_FRAME_RATE

C.4.2.4 MF_MT_AVG_BITRATE

C.4.2.5 MF_MT_PIXEL_ASPECT_RATIO

C.4.3 It is recommended that your VC-1 Encoder supports:

C.4.3.1. B frame encoding

C.5 Multiple Instances

C.5.1 It is required that your VC-1 encoder must support a minimum of 3 concurrent instances

C.5.1.1 These instances may be in the same process or in different processes

C.6 Merit Validation

C.6.1 It is required that your VC-1 encoder supports the trusted merit verification process

C.6.2 It is required that your VC-1 encoder be a separate binary, individually certified and signed

C.7 Additional Requirements

C.7.1 Your VC-1 encoder must work with the Windows ASF file sink

C.7.2 Your VC-1 encoder must implement proper order of encoding configuration

C.8 Installation

C.8.1 It is required that your VC-1 encoder is registered and unregistered along with the device driver used in the encoder

C.9 Performance

C.9.1 It is required that your VC-1 encoder must be capable of real-time encoding 1280x720x30fps up to 7 Mbps **on x86/x64 systems**

C.9.2 VC1 encoder must be capable of real-time encoding 720x480x30fps up to 5 Mbps on **ARM systems**

Exceptions: Not Specified

Business Justification:

HMFT is a feature introduced in Windows 7 to enable Independent Hardware Vendors (IHVs) to provide hardware media solutions.

Scenarios:

Hardware accelerated video encoding via Media Foundation for encoding and transcoding scenarios. On the encoding side, this includes Windows Live Movie Maker and webcam capture scenarios. On the transcoding side, this includes transcoding for Play To home media streaming, and synchronizing to the device.

Success Metric: Not Specified

Enforcement Date: Windows RC

Comments:

New

Device.Streaming.Tuner

Description:

These requirements allow a tuner to correctly stream video content on Windows

Related Requirements:

- Device.Streaming.Tuner.AnalogRequirements
- Device.Streaming.Tuner.AnalogSupportsAudioAndVideo
- Device.Streaming.Tuner.AutomatedCaptureGraph
- Device.Streaming.Tuner.AVStreamWDMAndInterfaceRequirements
- Device.Streaming.Tuner.ConditionalAccessSystems
- Device.Streaming.Tuner.ContinuousChannelChanges
- Device.Streaming.Tuner.ContinuousReboots
- Device.Streaming.Tuner.ContinuousStreaming
- Device.Streaming.Tuner.DetectsSignalsFromAllSources
- Device.Streaming.Tuner.DigitalTunerUsesBDA
- Device.Streaming.Tuner.FirstRun
- Device.Streaming.Tuner.HybridComboImplementation
- Device.Streaming.Tuner.IPFunctionality
- Device.Streaming.Tuner.ISDBTSupportBML
- Device.Streaming.Tuner.iSerialNumber
- Device.Streaming.Tuner.MPEGEncoding
- Device.Streaming.Tuner.MSVidCtl
- Device.Streaming.Tuner.MultipleClientAppSupport
- Device.Streaming.Tuner.MultipleStreams
- Device.Streaming.Tuner.PBDA
- Device.Streaming.Tuner.PBDACustomizedProductRequirements
- Device.Streaming.Tuner.PBDAExtensibilityPages
- Device.Streaming.Tuner.ProperlySignalsChannelChanges
- Device.Streaming.Tuner.ResourcesAcquiredAtDeviceAcquire
- Device.Streaming.Tuner.ScanForServices
- Device.Streaming.Tuner.ScheduledRecording
- Device.Streaming.Tuner.SignalQualityAndLock
- Device.Streaming.Tuner.SleepStates
- Device.Streaming.Tuner.SupportsAutomaticRendering
- Device.Streaming.Tuner.TimeShiftedPlayback
- Device.Streaming.Tuner.UVC
- Device.Streaming.Tuner.UVCDriver
- Device.Streaming.Tuner.WMCEncoding

Device.Streaming.Tuner.AnalogRequirements

Target Feature: Device.Streaming.Tuner

Title: Analog TV tuner or capture device requirements

Applicable OS Versions:

Windows 8 Client x64
Windows 8 Client x86
Windows 7 Client x64
Windows 7 Client x86

Description:

General Requirements

NTSC TV tuners and analog capture devices must be able to encode full-resolution composite signals at 720 samples per line with 8-bit luminance and chrominance sampling. If the device sources uncompressed component video, the video must be encoded in a 4:2:2 color format (either UYVY or YUY2).

If the device sources compressed video (such as MPEG2 or Windows Media Video), the compressed video must be encoded as 4:2:2, 4:1:1, or 4:2:0, as appropriate.

Audio Decoding Requirements

NTSC TV tuners must include stereo audio decoding. If SAP is available in the device's intended market, it must also be implemented.

Content Protection

NTSC TV tuners must comply with the following requirements:

Pass line 20 VBI data containing CGMS-A and APS bits as specified by IEC 61880 to Media Center.

Pass line 21 VBI data containing CGMS-A and APS bits as specified by EIA-608-B to Media Center.

VBI Requirements

Premium analog TV tuners must support raw VBI pass through of all lines listed below.

Lines 10 through 21 for NTSC and lines 7 through 23 for PAL/SECAM must be captured as part of the VBI stream.

In PAL/SECAM, line 23 can appear as a black video line for the visible video signal to prevent any changes to the video image area**.**

The tuner must output all captured VBI lines in raw form on the capture filter with a separate capture pin.

Raw VBI data must be captured at a true sampling rate of 27 or 28.63 MHz (8 x Fsc for NTSC). Artificial oversampling from a lesser sampling rate is not allowed. The resulting samples must comply with the timing requirements in the EIA-608-B and BT.653 specifications for closed captioning and

teletext, respectively. The captured data must cover the entire data line (including clock run-in) along with a suitable margin of extra samples on both sides.

All VBI samples must be supplied in undecoded, luma-only format (1400 to 2000 luma samples/VBI line) including all scan lines that may contain encoded data signals on each video field. To ensure accurate data reception, data transmitted on all lines of the VBI must be oversampled exactly 4.7 or 5 times the North American Basic Teletext (NABTS) data bit rate (or locale-specific data bit rate).

For example, if 288 bits of NABTS data on a scan line, approximately 1,354 one-byte samples, plus the necessary margin, must be captured per scan line if 4.7x oversampling is used. This would be 1,440 one-byte samples if 5x oversampling were used. This represents the number required for timing tolerances in the NABTS specification and also for timing uncertainties within the capture hardware. If the hardware cannot provide 4.7 or 5 times oversampled VBI data, the device-specific driver must compensate by resampling, so that 4.7 or 5 times oversampled data are presented to the operating system.

Exhaustive Scanning

A TV tuner designed for analog tuning in PAL and SECAM regions must implement the Scanning DDI by supporting the exhaustive scanning properties as defined by KSMedia.h. These new interfaces provide support for both hardware and software driven scanning, with hardware scanning being the preferred solution.

Design Notes:

For additional information, see "Video Rendering with 8-bit YUV Formats" at http://go.microsoft.com/fwlink/?LinkId=40511

Exceptions: Not Specified

Business Justification:

Required to work with WMC.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

STREAM-0023

Device.Streaming.Tuner.AnalogSupportsAudioAndVideo

Target Feature: Device.Streaming.Tuner

Title: Analog TV tuner supports both audio and video capture

Applicable OS Versions:

Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client x86
Windows 8 Client x64

Description:

Both audio and video capture must occur on the same physical device, with tightly synchronized A/V elementary streams (<100ms phase skew (which determines audio and video synchronization, failure of which causes lip-synch issues)), and low interrupt rates (<500ms/second). For audio and compressed video multiple small host capture buffers are required (a minimum of 64 buffers and a maximum of 32 KB per buffer).

Exceptions: Not Specified

Business Justification:

This provides an experience with analog tuners that is comparable to digital tuners.

Scenarios:

Analog tuners can provide similar experience as digital tuners.

Success Metric: Pass or Fail

Enforcement Date: 1/1/2010

Comments:

STREAM-0010

Device.Streaming.Tuner.AutomatedCaptureGraph

Target Feature: Device.Streaming.Tuner

Title: TV tuner that supports Media Center functionality implements automated capture graph building with Media Center video control

Applicable OS Versions:

Windows 8 Client x64
Windows 8 Client x86
Windows 7 Client x64
Windows 7 Client x86

Description:

A TV tuner capture card that supports Media Center functionality must support automated capture graph building with Media Center video control (VidControl).

This means that the Microsoft Video Control has to be able to:

Enumerate available tuners by finding the correct registry entry

For digital tuners, a capture graph is built connecting the Microsoft Network provider to the Tuner Source Filter and Tuner Capture Filter

For analog tuners, a segmented graph is built that includes the necessary crossbars, encoder, mux and demux

Where necessary, VBI pin connections have to be made for RAW and/or sliced VBI Transition the graph to run state and tune to a channel to view content

Exceptions: Not Specified

Business Justification:

Supports functionality of WMC.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

STREAM-0057

Device.Streaming.Tuner.AVStreamWDMAndInterfaceRequirements

Target Feature: Device.Streaming.Tuner

Title: Streaming media device driver is based on AVStream class and WDM and meets interface requirements

Applicable OS Versions:

Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client x86
Windows 8 Client x64

Description:

WDM Requirement

Device drivers for any streaming media device must use the AVStream class WDM device driver model as defined in the Windows Driver Kit. AVStream is the replacement technology for the older stream class driver model, which is outdated and will no longer be enhanced. Drivers for streaming media devices must also support all of the required pins, properties, and settings as defined in the Windows Driver Kit.

Note: PCI-E based video capture devices must use the AVStream class. USB based devices must be UVC compliant as defined in **Device.Streaming.Tuner.UVCDriver** and **Device.Streaming.Tuner.UVC**.

Error Conditions

Error conditions include (but are not limited to) forced invalid pin connections, invalid property sets, buffers with invalid data, null pointers, and error conditions from drivers above or below on the stack.

Interface Requirements - IVideoEncoderAPI / ICodecAPI

Streaming media devices that encode video streams must use the IVideoEncoderAPI and ICodecAPI. This enables the device driver to uniformly configure hardware and software encoders in the system.

To support the Media Center functionality Broadcast receivers must support encode by using the Video Encoder API. Support for this feature allows Media Center to explicitly set the video data rate for optimal compatibility with DVD burning.

Specifically, the device and driver must support **VariableBitratePeak** mode defined by **IVideoEncoderAPI** to allow for limiting peak video rate. The device and driver must support setting the ENCAPIPARAM_PEAK_BITRATE property correctly through **IVideoEncoderAPI** to a value of 8. The ENCAPIRARAM_BITRATE value may be set to any value below the value of ENCAPIPARAM_PEAK_BITRATE to enable VBR recordings.

The video compression encoder for the tuner hardware must:

- Generate DVD-compliant MPEG-2 elementary video streams when the bit rate is set at or below 9Mbps VBR. If the software application sets the bit rate above allowable DVD data rates, DVD compliance is not required.
- Support dynamic bit rate change during runtime. The encoder must be capable of dynamically changing the encoding quality bit rate up or down without the need for the DirectShow filter to be stopped and restarted by changing the ENCAPIPARAM_BITRATE value. The ENCAPIPARAM_PEAK_BITRATE value is not required to change dynamically.
- Support all compression bit rates at least up to a Max Bit Rate of 9 Mbps in VariableBitrateAverage mode defined by **IVideoEncoderAPI**. Higher peak rates are encouraged but are not required.
- Support setting the ENCAPIPARAM_BITRATE correctly through **IVideoEncoderAPI** to a value of 2, up to a value of 9. Higher average rates are encouraged but are not required.

To enable Media Center to detect whether a driver is capable of supporting dynamic bit rate change, the INF file must contain the GUID. In addition, the driver must place the GUID in the registry. The registry value needs to be a DWORD with a value of "1" meaning supported and "0" or "Not Present" meaning unsupported. Add the following in the INF file under HKR,Capabilities:

"{BB4FAA02-596C-4129-8FB3-74E75421FA02}", 0x00010001,1
[KEY]
"{BB4FAA02-596C-4129-8FB3-74E75421FA02}"=dword:1

Where [Key] is the HKEY returned by **IGetCapabilitiesKey::GetCapabilitiesKey()**

Design Notes:

See the Windows Driver Kit, "Streaming Devices (Video and Audio)," "AVStream Class minidrivers," and "Stream Class Minidrivers."

Exceptions: Not Specified

Business Justification:

This requirement standardizes some of the aspects of analog stream input.

Scenarios:

User can successfully stream video from various sources and render to a variety of display options.

Success Metric: Pass or Fail

Enforcement Date: 1/1/2010

Comments:

STREAM-0001

Device.Streaming.Tuner.ConditionalAccessSystems

Target Feature: Device.Streaming.Tuner

Title: If implemented, Digital TV Tuners support arrival and removal of Conditional Access Systems smartcards

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 7 Client x64
- Windows 7 Client x86

Description:

In specific regions, TV Tuners are required to support Conditional Access Systems (CAS) which use a smartcard for authentication. When the card is removed, the TV tuner driver will notify the system through a system event. The expectation is that the driver will not fail, and the system will provide the appropriate level of error handling and continue to operate.

Design Notes

Assumptions

1. The device under test has access to protected content.

2. The device under test has a valid conditional access smartcard.

3. The device under test is configured to the appropriate broadcast format & region.

Test Requirements

1. Tune to Live TV on a channel that contains protected content.

2. Perform a manual surprise removal of the smartcard.

3. Validate that the tuner responds with a DFN (Descramble Failure Notification).

4. Verify that the device under test continues to be operational.

Exceptions: Not Specified

Business Justification:

Verify that basic functionality is present for Windows Media Center (WMC).

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

STREAM-0067

Device.Streaming.Tuner.ContinuousChannelChanges

Target Feature: Device.Streaming.Tuner

Title: TV Tuners must support continuous channel changes.

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 7 Client x64
- Windows 7 Client x86

Description:

TV Tuners must support continuous channel changes.

Design Notes:

Assumptions

The device under test will be configured to the appropriate broadcast format.

Test Requirement

1. Tune to Live TV on lowest channel

2. Verify that Live TV is functional

3. Tune to next channel

4. Verify that Live TV is functional

5. Repeat steps 2-4 continuously

The specifically period of time for testing is defined in the Windows Logo Kit

Exceptions: Not Specified

Business Justification:

Verify that basic functionality is present for Windows Media Center (WMC).

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

STREAM-0066

Device.Streaming.Tuner.ContinuousReboots

Target Feature: Device.Streaming.Tuner

Title: TV Tuners function after continuous reboots

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 7 Client x64
- Windows 7 Client x86

Description:

TV Tuner devices must able to stream after system reboots

Design Notes:

Assumptions

The device under test is configured.

Test Requirement

1. Tune to Live TV

2. Verify Live TV

3. Reboot

4. Repeat steps 2 3 multiple (at least 5) times

Exceptions: Not Specified

Business Justification:

Verify that tuners can function after user reboots to provide basic user experience.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

STREAM-0073

Device.Streaming.Tuner.ContinuousStreaming

Target Feature: Device.Streaming.Tuner

Title: TV Tuners must support continuous streaming of Live TV

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 7 Client x64
- Windows 7 Client x86

Description:

TV Tuners must sustain a minimum period of continuous streaming of Live TV. Tuners that can operate without errors continuously represent a higher quality customer experience.

Design Notes:

Assumptions

The device under test will be configured to the appropriate broadcast format.

Test Requirement

1. Tune to Live TV

2. Sustain continuous streaming of Live TV streaming without error

Note that the specific amount of time required for the test is defined in the Windows Logo Kit.

Exceptions: Not Specified

Business Justification:

Ensure that tuners can sustain a signal for end users over a large period of time.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

STREAM-0065

Device.Streaming.Tuner.DetectsSignalsFromAllSources

Target Feature: Device.Streaming.Tuner

Title: TV tuner detects signals from all input sources

Applicable OS Versions:

Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client x86
Windows 8 Client x64

Description:

:

TV tuners must be able to detect signals from all implemented input sources (such as RF, composite, and S-Video). Sharing connections internally between S-Video and composite auxiliary inputs makes discrete signal detection between these two inputs impossible. For this reason Microsoft strongly discourages sharing connections internally between independent auxiliary connectors.

The driver must correctly expose the proper mediums and media types related to information off the crossbar pins to determine which S-Video or composite jack matches to which S-Video jack.

OpenCable Unidirectional Receivers (which can tune to both terrestrial and cable, analog and digital content) must be able to scan and detect signals on the various modulation types (such as NTSC, 8VSB, QAM64, and QAM256).

OpenCable Unidirectional Receivers (OCUR) must also comply with requirements outlined in DTOS for CableLabs certification of OCUR devices.

Design Notes:

Microsoft recommends that analog tuners meet video quality and RF certification requirements for TV tuners as specified by ISF Research Labs

Exceptions: Not Specified

Business Justification:

This is required for TV tuners to work with Windows Media Center (WMC).

Scenarios:

User can successfully stream video from various sources and render to a variety of display options.

Success Metric: Pass or Fail

Enforcement Date: 1/1/2010

Comments:

STREAM-0016

Device.Streaming.Tuner.DigitalTunerUsesBDA

Target Feature: Device.Streaming.Tuner

Title: Digital TV tuner uses the Broadcast Driver Architecture (BDA)

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 7 Client x64
- Windows 7 Client x86

Description:

Digital TV tuners must use the Broadcast Driver Architecture (BDA) as defined in "Broadcast Driver Architecture Drivers" in the Windows Driver Kit.

Devices and systems in regions with Digital TV standards that are not yet supported by Broadcast Driver Architecture (BDA) are exempt from this requirement. BDA does not yet support all Digital TV standards used in specific regions. In these regions support for BDA is not required. This exemption allows for proprietary Digital TV implementations and is valid until support for the regional Digital TV standard is provided by BDA.

Signal Diagnostics

Digital TV tuners must provide signal strength, signal quality, signal present, and signal lock status diagnostic information through the BDA interface KSPROPSETID_BdaSignalStats as defined in the Microsoft Windows SDK.

Exceptions: Not Specified

Business Justification:

Required to work with WMC.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

STREAM-0021

Device.Streaming.Tuner.FirstRun

Target Feature: Device.Streaming.Tuner

Title: WMC AQ: TV Tuners complete First Run and play Live TV within Windows Media Center

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 7 Client x64
- Windows 7 Client x86

Description:

The TV tuner device must complete First Run and play Live TV within Windows Media Center.

Design Notes:

First Run is defined as the first time a customer runs the tuner setup in Media Center.

Exceptions: Not Specified

Business Justification:

Verify the end user experience for end users in Windows Media Center.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

STREAM-0074

Device.Streaming.Tuner.HybridComboImplementation

Target Feature: Device.Streaming.Tuner

Title: Tuner- Additional Qualifications – Hybrid/Combo Implementation

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 7 Client x64
- Windows 7 Client x86

Description:

The tuner shall correctly declare the relationship (Combo vs. Hybrid) between all of its Tuner Instances. Hybrid Tuner instances must not be useable in parallel. Combo tuner instances must be useable in parallel. For multi-tuner hybrid devices, all of the independent tuner instances must be useable in parallel.

Exceptions: Not Specified

Business Justification:

This requirement ensures consistent reported capabilities for consumers when considering a Hybrid vs Combo tuner.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

STREAM-0091

Device.Streaming.Tuner.IPFunctionality

Target Feature: Device.Streaming.Tuner

Title: If implemented, Streaming devices that implement IP functionality support Windows Rally technologies.

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 7 Client x64
- Windows 7 Client x86

Description:

Streaming media devices that implement IP functionality must meet the minimum Windows Rally requirements as defined under Device Connectivity. These are requirements **Device.Connectivity.Network.LLTD**, **Device.Connectivity.Network.VerticalPairing.WCN**, **Device.Connectivity.Network.PnPX.PnPX** and **Device.Connectivity.WSD.Schema**. The Windows Rally requirements ensure that all network connected devices are simple to setup, configure and diagnose.

Exceptions: Not Specified

Business Justification:

Ensures that network connected devices work with the operating system in a manner seamless to the end user.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

STREAM-0059

Device.Streaming.Tuner.ISDBTSupportBML

Target Feature: Device.Streaming.Tuner

Title: WMC AQ: ISDB-T Digital TV Tuners must support BML

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 7 Client x64
- Windows 7 Client x86

Description:

Tuner Devices that operate in the ISDB-T broadcast region must support BML through Windows Media Center.

Design Notes:

Assumptions

The tuner is configured for ISDB-T successfully, and Live TV is verified. Furthermore, the signal contains a BML test stream. The customer will provide the test harness with information on which channel contains BML.

Test Requirements

The test must perform the following actions:

1. Tune to Live TV that contains a BML stream.

2. Verify that BML is operational

Exceptions: Not Specified

Business Justification:

Verifies that BML is supported in the ISDB-T markets.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

STREAM-0063

Device.Streaming.Tuner.iSerialNumber

Target Feature: Device.Streaming.Tuner

Title: USB TV tuner implements the iSerialNumber field to ensure unique serial numbers across devices of the same model

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 7 Client x64
- Windows 7 Client x86

Description:

To reuse the same PDO and to ensure that the device experience is unchanged whether the device is reinserted into the same port or a new port, hardware vendors must store a serial number on their device. The serial number must be unique for all devices that share the same device installation identifier.

USB TV tuners must implement USB 2.0 high-speed support.

USB TV tuners must implement the iSerialNumber field to ensure unique serial numbers across devices of the same model.

USB TV tuners must implement both **Device.Connectivity.UsbDevices.SerialNumbers** and **Device.Connectivity.UsbDevices.SerialNumbersUseValidCharacters** requirements.

Design Notes:

See USB 2.0 Specification, Table 9-8.

Exceptions: Not Specified

Business Justification:

Required to work with WMC.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

STREAM-0017

Device.Streaming.Tuner.MPEGEncoding

Target Feature: Device.Streaming.Tuner

Title: MPEG-2 video stream encoding consumes appropriate processor cycles

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 7 Client x64
- Windows 7 Client x86

Description:

MPEG-2 video stream encoding, implemented on a desktop system, must consume no more than 45 percent of total processor cycles during real-time encoding. MPEG-2 video streams are defined, at a minimum, to be 720x480at 30Hz or 720x240at 60Hz.

Exceptions: Not Specified

Business Justification:

Ensure baseline user experience.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

STREAM-0049

Device.Streaming.Tuner.MSVidCtl

Target Feature: Device.Streaming.Tuner

Title: Tuners support MSVidCtl to record programs

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 7 Client x64
- Windows 7 Client x86

Description:

Tuner devices must support the Broadcast Driver Architecture and support MSVidCtl for filter and graph management.

Exceptions: Not Specified

Business Justification:

Validate basic functionality of tuners.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

STREAM-0072

Device.Streaming.Tuner.MultipleClientAppSupport

Target Feature: Device.Streaming.Tuner

Title: Streaming media device supports multiple client applications & instances by using a single device

Applicable OS Versions:

- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows 8 Client x64

Description:

Application Sharing

The client applications must be able to open the device and then use the device simultaneously or sequentially, depending on the device capabilities. The device can either allow one application to actively use it while the other application is in a pause or stop state or the device can allow both applications to use it at simultaneously.

Multiple Instances

Streaming media devices must be able to control multiple instances of the device with multiple application usage. An example would be a PC with three identical USB webcams, each being used by a different application. A second example would be a PC with two TV receivers, each being independently used by two different applications.

Exceptions: Not Specified

Business Justification:

This allows for great device usage with Window Media Center Releases. This is a key user scenario for eHomes future Media Center releases.

Scenarios:

User can successfully stream video from various sources and render to a variety of display options. Key to eHome scenarios

Success Metric: Pass or Fail

Enforcement Date: 1/1/2010

Comments:

STREAM-0003

Device.Streaming.Tuner.MultipleStreams

Target Feature: Device.Streaming.Tuner

Title: WMC AQ: Devices with multiple TV Tuners must support simultaneous use of all the tuner streams (including Homogeneous & Heterogeneous Broadcast types)

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 7 Client x64
- Windows 7 Client x86

Description:

Devices with multiple TV Tuners must support simultaneous use of all the tuner streams (including Homogeneous & Heterogeneous Broadcast types)

Exceptions: Not Specified

Business Justification:

Verify TV tuners can support multiple streams properly.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

STREAM-0068

Device.Streaming.Tuner.PBDA

Target Feature: Device.Streaming.Tuner

Title: WMC AQ: Digital TV Tuners requiring protected streams will use Protected Broadcast Driver Architecture (PBDA)

Applicable OS Versions:

Windows 8 Client x64
Windows 8 Client x86
Windows 7 Client x64
Windows 7 Client x86

Description:

Digital tuners supporting protected content will support Protected Broadcast Driver Architecture (PBDA) to meet the Additional Qualifications (AQ) for Windows Media Center.

The Protected Broadcast Driver Architecture (PBDA) supports tuners that need to send protected content, custom setup metadata, custom guide data, and other formats not natively supported by Windows Media Center.

Design Notes:

PBDA is binding independent and allows content to be transmitted in several ways. Windows Media Center utilizes the PBDA platform.

Exceptions: Not Specified

Business Justification:

PBDA is the new platform as part of Windows to stream protected, high-definition content inside of windows media center.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

STREAM-0060

Device.Streaming.Tuner.PBDACustomizedProductRequirements

Target Feature: Device.Streaming.Tuner

Title: WMC AQ: PBDA tuner meets Windows Media Center Customized product requirements

Applicable OS Versions:

Windows 8 Client x64
Windows 8 Client x86
Windows 7 Client x64
Windows 7 Client x86

Description:

If a PBDA tuner includes functionality for extending Windows Media Center first run experience or includes an Interactive TV application, the extended first run experience end application should comply with the Windows Media Center Customized Product Requirements specification.

Exceptions: Not Specified

Business Justification:

Validate WMC extensions.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

STREAM-0089

Device.Streaming.Tuner.PBDAExtensibilityPages

Target Feature: Device.Streaming.Tuner

Title: WMC AQ: PBDA devices author extensibility pages per the Windows Media Center guidelines

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 7 Client x64
- Windows 7 Client x86

Description:

PBDA Tuners implementing extensibility pages or MMIs should follow the requirements in Windows Media Center Customized Product Requirements specification. This is the device equivalent of the System Fundamentals requirement, SYSFUND-0216. It is being reflected here to enable IHVs to find all relevant information at one location.

Design Notes:

The Windows Media Center Customized Product Requirements specification can be found at: http://go.microsoft.com/fwlink/?LinkId=141797

Exceptions: Not Specified

Business Justification:

This requirement validates compliance with SYSFUND-0216.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

STREAM-0090

Device.Streaming.Tuner.ProperlySignalsChannelChanges

Target Feature: Device.Streaming.Tuner

Title: TV tuner properly signals channel changes

Applicable OS Versions:

- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows 8 Client x64

Description:

Analog TV tuner drivers must send KSTV_TUNER_CHANGE_INFO packets of Tuning_Begin/Tune_End, to signal channel changes. Digital tuners must also demonstrate channel change capabilities. This capability is verified via visual inspection at this time.

Design Notes:

The Tuning_Begin packet should occur at the start of a tune, on one of the last good packet. A Tune_End should be sent at the end of the tune, on one the first good_packets. All data between these two events is considered to be invalid/snow.

Exceptions:

Analog tuners only

Business Justification:

This is required for TV tuners to work with Windows Media Center (WMC

Scenarios:

User can successfully stream video from various sources and render to a variety of display options.

Success Metric: Pass or Fail

Enforcement Date: 1/1/2010

Comments:

STREAM-0015

Device.Streaming.Tuner.ResourcesAcquiredAtDeviceAcquire

Target Feature: Device.Streaming.Tuner

Title: Tuner resources are not acquired until the Device Acquire state

Applicable OS Versions:

Windows 7 Client x86
Windows 7 Client x64
Windows 8 Client x86
Windows 8 Client x64

Description:

In DirectShow, TV tuner drivers must support automatic rendering between the multiplexer and encoder filters. If the capture filter and multiplexer are inserted into a DirectShow graph, a RenderPin() call on the capture pin must result in automatic filter insertion and connection.

Exceptions: Not Specified

Business Justification:

This is required for TV tuners to work with Windows Media Center (WMC).

Scenarios:

User can successfully stream video from various sources and render to a variety of display options.

Success Metric: Pass or Fail

Enforcement Date: 1/1/2010

Comments:

STREAM-0014

Device.Streaming.Tuner.ScanForServices

Target Feature: Device.Streaming.Tuner

Title: WMC AQ: TV Tuners must be able to scan for services multiple times

Applicable OS Versions:

Windows 8 Client x64
Windows 8 Client x86
Windows 7 Client x64
Windows 7 Client x86

Description:

Tuner Devices must be able to perform multiple scans without error in operation. The Tuner must perform scan for services through first and subsequent runs, and provide consistent results at the end of each scan.

Design Notes:

Assumption

The tuner is not yet configured.

Test Requirements

The test must perform the following actions:

1. Tune to Live TV

2. Re-start First Run initiating a new scan for services

3. Verify Live TV is functional

4. Repeat steps 2& 3 20-times

Exceptions: Not Specified

Business Justification:

End users expect the tuner to continue to operate.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

STREAM-0062

Device.Streaming.Tuner.ScheduledRecording

Target Feature: Device.Streaming.Tuner

Title: WMC AQ: Scheduled recording wakes up a Device from states S1-S4

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 7 Client x64
- Windows 7 Client x86

Description:

The tuner devices must operate after a future recording is scheduled and the system enters into states S1-S4

Design Notes:

Assumption

The tuner device is configured.

Test Requirement

1. Schedule a 5 minute recording for 5 minute into the future

2. Place the system into hibernation/suspend

3. Verify that system wakes and recording completed

Exceptions: Not Specified

Business Justification:

Verify that a tuner works after system resumes.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

STREAM-0076

Device.Streaming.Tuner.SignalQualityAndLock

Target Feature: Device.Streaming.Tuner

Title: Tuner- Additional Qualifications – Signal Quality and Signal Lock Test Suite

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 7 Client x64
- Windows 7 Client x86

Description:

All of the digital tuner instances on a physical tuner card shall be able to accurately calculate and report Signal Quality and Signal Lock to Media Center. The Signal Quality values shall be consistent and in 10% proximity to each other. Out of 100 reported Signal Quality values on the same TV signal, a maximum 10% of these values may vary more than 10% of the average value. Retrieving Signal Statistics from the

Exceptions: Not Specified

Business Justification:

This requirement ensures an end users experience is comparable to consumer electronics products.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

STREAM-0092

Device.Streaming.Tuner.SleepStates

Target Feature: Device.Streaming.Tuner

Title: TV Tuners resume from all system states (S1-S4)

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 7 Client x64
- Windows 7 Client x86

Description:

Tuner Devices must be able to sustain system state transitions multiple number of times. Microsoft acknowledges that this is a key test scenarios that all AQ tuners must support. The Tuner must complete a task after the system returns from hibernation (or sleep) multiple times.

Design Notes:

Assumptions

The Tuner is in a ready state, and has completed First Run.

Test Requirements

The test must perform the following actions:

1. Tune to Live TV

2. After 1 minute, enter into sleep/hibernation state

3. Return from sleep/hibernation state

4. Repeat steps 2 3 20 times.

5. Verify that Live TV continues to function

Exceptions: Not Specified

Business Justification:

Verifies works with WMC.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

STREAM-0061

Device.Streaming.Tuner.SupportsAutomaticRendering

Target Feature: Device.Streaming.Tuner

Title: TV tuner supports automatic rendering

Applicable OS Versions:

- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows 8 Client x64

Description:

In DirectShow, TV tuner drivers must support automatic rendering between the multiplexer and encoder filters. If the capture filter and multiplexer are inserted into a DirectShow graph, a RenderPin() call on the capture pin must result in automatic filter insertion and connection.

Exceptions:

This is only required for analog TV tuners

Business Justification:

This is required for analog tuners to work with Windows Media Center (WMC).

Scenarios:

User can successfully stream video from various sources and render to a variety of display options.

Success Metric: Pass or Fail

Enforcement Date: 1/1/2010

Comments:

STREAM-0014

Device.Streaming.Tuner.TimeShiftedPlayback

Target Feature: Device.Streaming.Tuner

Title: WMC AQ: TV Tuner Devices support time-shifted playback within Windows Media Center

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 7 Client x64
- Windows 7 Client x86

Description:

TV Tuner devices must be able to play Live TV, Pause, Rewind, Fast Forward, and return to Live TV within Windows Media Center (WMC).

Exceptions: Not Specified

Business Justification:

Validate that features within WMC are supported.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

STREAM-0071

Device.Streaming.Tuner.UVC

Target Feature: Device.Streaming.Tuner

Title: USB streaming video camera must comply with USB Video Class specifications

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 7 Client x64
- Windows 7 Client x86

Description:

USB streaming video cameras must comply with the USB Video Class specifications. At a minimum, all mandatory properties and commands must be implemented. All implemented commands must comply with the specifications. USB streaming video cameras that use MJPEG, YUY2, or MPEG-2 Transport Stream (TS) for capture, or Digital Video (DV) for capture or render, must also work with the Microsoft-provided USB Video Class driver.

Exceptions: Not Specified

Business Justification:

Security-enhanced, robust, high-performance class drivers enable IHVs to spend less time writing and debugging drivers and more creating better devices for their customers. Improves the overall quality of multimedia device drivers.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

STREAM-0053

Device.Streaming.Tuner.UVCDriver

Target Feature: Device.Streaming.Tuner

Title: UVC-capable device complies with USB Video Class specifications and works with Microsoft UVC driver

Applicable OS Versions:

- Windows 7 Client x86
- Windows 7 Client x64
- Windows 8 Client x86
- Windows 8 Client x64

Description:

Devices designed to comply with the USB Video Class specifications must work with the Microsoft USB Video Class driver. These devices also must comply with the requirements set in the USB Video Class specifications. At a minimum, all mandatory properties and commands must be implemented. All commands implemented must comply with the specifications.

Exceptions: Not Specified

Business Justification:

Secure, robust, high performance class drivers allow IHVs to spend less time writing and debugging drivers and more creating better devices for their customers. Improves overall multimedia device driver quality.

Scenarios:

User can successfully stream video from various sources and render to a variety of display options.

Success Metric: Pass or Fail

Enforcement Date: 1/1/2010

Comments:

STREAM-0008

Device.Streaming.Tuner.WMCEncoding

Target Feature: Device.Streaming.Tuner

Title: Encoder supports minimum requirements to enable Media Center functionality

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 7 Client x64
- Windows 7 Client x86

Description:

Encoders implemented with standard definition tuners must:

- Generate DVD-compliant MPEG2 elementary video streams when bit rate is set to less than 9Mbps average VBR.
- Support dynamic bit rate change. The encoder must be able to dynamically change the encoding quality bitrate up or down without the requirement to stop and restart.
- Support all compression bit rates at least up to a maximum bit rate of 9 MBPS in VBR mode.
- Respond correctly to peak bit rate and average bit rate settings pass through IVideoEncoderAPI.

For Media Center to detect whether a driver can support dynamic bit rate change, the INF file must contain the GUID. In addition, the driver must place the GUID in the registry. The registry value must be a DWORD with a value of1 if it is supported and 0 or not present if it is not supported.

The following must be added in the INF file under
HKR,Capabilities,"{BB4FAA02-596C-4129-8FB3-74E75421FA02}", 0x00010001,1

[KEY]
"{BB4FAA02-596C-4129-8FB3-74E75421FA02}"=dword:1
where [Key] is the HKEY returned by IGetCapabilitiesKey::GetCapabilitiesKey().

Exceptions: Not Specified

Business Justification:

Supports functionality of WMC.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

STREAM-0050

Device.Streaming.Webcam.Base

Description:

Webcam features

Related Requirements:

Device.Streaming.Webcam.Base.AVStreamWDMAndInterfaceRequirements
Device.Streaming.Webcam.Base.BasicPerf
Device.Streaming.Webcam.Base.DirectShowAndMediaFoundation
Device.Streaming.Webcam.Base.IPFunctionality
Device.Streaming.Webcam.Base.KSCategoryVideoCameraRegistration
Device.Streaming.Webcam.Base.MultipleClientAppSupport
Device.Streaming.Webcam.Base.SurpriseRemoval
Device.Streaming.Webcam.Base.UsageIndicator

Device.Streaming.Webcam.Base.AVStreamWDMAndInterfaceRequirements

Target Feature: Device.Streaming.Webcam.Base

Title: Streaming media device driver must be based on AVStream class and WDM, and must meet interface requirements

Applicable OS Versions:

Windows 8 Client x64
Windows 8 Client x86
Windows 7 Client x64
Windows 7 Client x86
Windows 8 Client ARM

Description:

WDM Requirement

Device drivers for any streaming media device must use the AVStream class and Windows Driver Model (WDM) as defined in the Windows Driver Kit. AVStream is the replacement technology for the older stream class driver model, which is outdated and will no longer be enhanced. Drivers for streaming media devices must also support all of the required pins, properties, and settings as defined in the Windows Driver Kit.

Note: Peripheral Component Interconnect Express (PCI-e)based video capture devices must use the AVStream class. USB based devices must be USB Video Class (UVC) compliant as defined in

Device.Streaming.Webcam.USBClassDriver.UVCDriver and **Device.Streaming.Webcam.USBClassDriver.UVC**.

Error Conditions

Error conditions include (but are not limited to) forced invalid pin connections, invalid property sets, buffers with invalid data, null pointers, and error conditions from drivers above or below on the stack.

Interface Requirements: IVideoEncoderAPI and ICodecAPI

Streaming media devices that encode video streams must use **IVideoEncoderAPI** and **ICodecAPI**. This enables the device driver to uniformly configure hardware and software encoders in the system.

To support the Media Center functionality, broadcast receivers must support encoding by using the Video Encoder application programming interface (API). Support for this feature enables Media Center to explicitly set the video data rate for optimal compatibility with DVD burning.

Specifically, the device and driver must support **VariableBitratePeak** mode defined by **IVideoEncoderAPI** to allow for limiting peak video rate. The device and driver must support setting the **ENCAPIPARAM_PEAK_BITRATE** property correctly through **IVideoEncoderAPI** to a value of 8. The **ENCAPIRARAM_BITRATE** value may be set to any value below the value of **ENCAPIPARAM_PEAK_BITRATE** to enable variable bit rate (VBR) recordings.

The video compression encoder for the tuner hardware must do the following:

- Generate DVD-compliant MPEG-2 elementary video streams when the bit rate is set at or below 9 megabits per second (Mbps) VBR. If the software application sets the bit rate above allowable DVD data rates, DVD compliance is not required **(Required on x86 and x64 architectures and operating systems only).**
- Support dynamic bit rate change during run time. The encoder must be capable of dynamically changing the encoding quality bit rate up or down without requiring the Microsoft DirectShow filter to be stopped and restarted by changing the **ENCAPIPARAM_BITRATE** value. The **ENCAPIPARAM_PEAK_BITRATE** value is not required to change dynamically.
- Support all compression bit rates at least up to a maximum bit rate of 9 Mbps in **VariableBitrateAverage** mode defined by **IVideoEncoderAPI**. Higher peak rates are encouraged but are not required.
- Support setting ENCAPIPARAM_BITRATE correctly through **IVideoEncoderAPI** to a value of 2, up to a value of 9. Higher average rates are encouraged but are not required.

To enable Media Center to detect whether a driver can support dynamic bit rate change, the .inf file must contain the GUID. In addition, the driver must place the GUID in the registry. The registry value must be a **DWORD** with a value of 1 (meaning supported) and 0 or Not Present (meaning unsupported). Add the following in the .inf file under HKR,Capabilities.

"{BB4FAA02-596C-4129-8FB3-74E75421FA02}", 0x00010001,1

[KEY]

"{BB4FAA02-596C-4129-8FB3-74E75421FA02}"=dword:1

where [KEY] is the HKEY returned by **IGetCapabilitiesKey::GetCapabilitiesKey()**

Design Notes:

For implementation details, see "Streaming Devices (Video and Audio)", "AVStream Class minidrivers", and "Stream Class Minidrivers" in the Windows Driver Kit.

Exceptions: Not Specified

Business Justification:

This requirement standardizes some of the aspects of analog stream input.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

STREAM-0001

Device.Streaming.Webcam.Base.BasicPerf

Target Feature: Device.Streaming.Webcam.Base

Title: Captured frame data must be provided within two frame periods

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 8 Client ARM

Description:

Video camera hardware must provide captured frame data to the driver within two frame periods of the initiation of capture from the sensor.

Exceptions: Not Specified

Business Justification:

Minimizing camera latency substantially improves the user experience in communication scenarios.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

STREAM-1158

Device.Streaming.Webcam.Base.DirectShowAndMediaFoundation

Target Feature: Device.Streaming.Webcam.Base

Title: Support for streaming media device must be based on DirectShow or Media Foundation architectures

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 7 Client x64
- Windows 7 Client x86
- Windows 8 Client ARM

Description:

Support for streaming media devices must be based on the Microsoft DirectShow or Microsoft Media Foundation. Substitute components may be used instead of DirectShow components that are provided with the operating system. Substitute components must include equivalent functionality based on the components provided with the operating system and must support at least the same inputs and outputs defined for DirectShow.

Design Notes

DirectShow support is available on x86 and x64 platforms only.

Exceptions: Not Specified

Business Justification:

Multimedia devices and drivers must be either Media Foundation or DirectShow compliant for full compatibility.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

STREAM-0006

Device.Streaming.Webcam.Base.IPFunctionality

Target Feature: Device.Streaming.Webcam.Base

Title: If implemented, Streaming devices that implement IP functionality must support Windows Rally technologies.

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 7 Client x64
- Windows 7 Client x86
- Windows 8 Client ARM

Description:

Streaming media devices that implement IP functionality must meet the minimum Windows Rally requirements as defined under Device Connectivity. These are requirements **Device.Connectivity.Network.VerticalPairing.WCN**, **Device.Connectivity.Network.PnPX.PnPX** and **Device.Connectivity.WSD.Schema**. The Windows Rally requirements ensure that all network connected devices are simple to setup, configure and diagnose.

Exceptions: Not Specified

Business Justification:

Ensures that network connected devices work with the operating system in a manner seamless to the end user.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

STREAM-0059

Device.Streaming.Webcam.Base.KSCategoryVideoCameraRegistration

Target Feature: Device.Streaming.Webcam.Base

Title: Non-Microsoft webcam driver must register under KSCategory_Video_Camera

Applicable OS Versions:

- Windows 8 Client x86
- Windows 8 Client x64
- Windows 8 Client ARM

Description:

All non-Microsoftwebcam drivers must register under the category for Media Foundations Capture Applications to detect the camera.

Exceptions: Not Specified

Business Justification:

This requirement is needed so that there is one category for webcams to be enumerated.

Scenarios: Not Specified

Success Metric: Pass/Fail

Enforcement Date: Windows RC

Comments:

New

Device.Streaming.Webcam.Base.MultipleClientAppSupport

Target Feature: Device.Streaming.Webcam.Base

Title: Streaming media device must support multiple client applications and instances by using a single device

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 7 Client x64
- Windows 7 Client x86
- Windows 8 Client ARM

Description:

Application Sharing

The client applications must be able to open the device and then use the device simultaneously or sequentially, depending on the device's capabilities. The device can allow one application to actively use it while the other application is in a pause or stop state, or the device can allow both applications to use it simultaneously.

Multiple Instances

Streaming media devices must be able to control multiple instances of the device with usage of multiple applications. An example is a computer that has three identical USB webcams, each being used by a different application. Another example is a computer that has two TV receivers, each being independently used by two different applications.

Exceptions: Not Specified

Business Justification:

This allows for great device usage with Window Media Center releases. This is a key user scenario for eHomes future Media Center releases.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

STREAM-0003

Device.Streaming.Webcam.Base.SurpriseRemoval

Target Feature: Device.Streaming.Webcam.Base

Title: Removable streaming media device must support surprise removal of that device

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 7 Client x64
- Windows 7 Client x86
- Windows 8 Client ARM

Description:

All hot-pluggable streaming media devices must support their surprise removal from the host bus.

Exceptions: Not Specified

Business Justification:

End users must have the ability to remove devices at will without consequence.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

STREAM-0069

Device.Streaming.Webcam.Base.UsageIndicator

Target Feature: Device.Streaming.Webcam.Base

Title: Video capture device must have a visual indicator other than the main display to indicate usage

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86

Windows 8 Client ARM

Description:

A video capture device must have a visual indicator other than the main display (for example an LED)that indicates when it is recording a user. The visual indicator should be on when device is capturing video and off when the device is not capturing video.

Exceptions: Not Specified

Business Justification:

Providing a visual indication that the capture device is in use gives the user feedback that he or she is being observed and that a process is using the capture device so that the user can troubleshoot during application failures.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

STREAM-8002

Device.Streaming.Webcam.H264

Description:

Webcam features

Related Requirements:

Device.Streaming.Webcam.H264.H264Support

Device.Streaming.Webcam.H264.H264Support

Target Feature: Device.Streaming.Webcam.H264

Title: If implemented, H.264 implementation must comply with USB Video Class driver

Applicable OS Versions:

Windows 8 Client x64
Windows 8 Client x86
Windows 8 Client ARM

Description:

If H.264 format is supported by the device, the implementation must be compliant with the Windows USB Video Class (UVC) driver. Windows driver follows the following spec: http://go.microsoft.com/fwlink/?LinkId=233063

At minimum following must be supported:

1. Pins and formats

    1.1 A preview pin on the same device advertising PINNAME_VIDEO_PREVIEW pin category must be supported with at least NV12 format

    1.2 H.264 format must be on a separate video capture pin advertising PINNAME_VIDEO_CAPTURE pin category.

2. Descriptor and controls

    2.1. H.264 Video Format Descriptor

    2.2. H.264 Video Frame Descriptor

    2.3. H.264 Probe/Commit Control

        2.3.1. Cconfiguring bUsage

        2.3.2. cConfiguring bmRateControlModes

        2.3.3. cConfiguring bmLayerOrViewLayout

    2.4. H.264 Encoding Units

        2.4.1. H.264 Encoding Unit Descriptor

        2.4.2. Select Layer Control

        2.4.3. Video Resolution Control

        2.4.4. Rate Control Parameter Control

        2.4.5. Synchronization and Long Term Reference Frame Control

Exceptions: Not Specified

Business Justification:

Provide proper support for the H.264 format with the class driver.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

STREAM-8003

Device.Streaming.Webcam.NonMSDriver

Description:

Webcam features

Related Requirements:

Device.Streaming.Webcam.NonMSDriver.VideoInfoHeader2

Device.Streaming.Webcam.NonMSDriver.VideoInfoHeader2

Target Feature: Device.Streaming.Webcam.NonMSDriver

Title: Video capture driver must implement VIDEOHEADER2

Applicable OS Versions:

Windows 8 Client x64
Windows 8 Client x86
Windows 7 Client x64
Windows 7 Client x86
Windows 8 Client ARM

Description:

Drivers for Windows Driver Model (WDM) streaming video capture devices must implement support for the **VIDEOINFOHEADER2** structure. This support indicates video source information such as interlace format, aspect ratio, and color space information (if the device supports capturing of interlaced video, pixel aspect ratios other than 1:1, or nonstandard YCbCr transfer matrix), gamma curve, chromaticity coordinates, or reference black and white levels. Note: This requirement applies to both USB-based and Peripheral Component Interconnect Express (PCI-e)-based video capture devices.

Design Notes:

For implementation details, see "Streaming Devices" and the AVStream sample capture driver in the Windows Driver Kit.

Exceptions: Not Specified

Business Justification:

Video capture must be compatible with Windows to be displayed correctly.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

STREAM-0055

Device.Streaming.Webcam.USBClassDriver

Description:

Webcam features

Related Requirements:

Device.Streaming.Webcam.USBClassDriver.UVC
Device.Streaming.Webcam.USBClassDriver.UVCDriver

Device.Streaming.Webcam.USBClassDriver.UVC

Target Feature: Device.Streaming.Webcam.USBClassDriver

Title: USB streaming video camera must comply with USB Video Class specifications

Applicable OS Versions:

Windows 8 Client x64
Windows 8 Client x86
Windows 7 Client x64
Windows 7 Client x86
Windows 8 Client ARM

Description:

USB streaming video cameras must comply with the USB Video Class specifications. At a minimum, all mandatory properties and commands must be implemented. All implemented commands must comply with the specifications. USB streaming video cameras that use MJPEG, orYUY2 for capture, or Digital Video (DV) for capture or render, must also work with the Microsoft-provided USB Video Class driver.

Exceptions: Not Specified

Business Justification:

Security-enhanced, robust, high-performance class drivers enable IHVs to spend less time writing and debugging drivers and more creating better devices for their customers. Improves the overall quality of multimedia device drivers.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

STREAM-0053

Device.Streaming.Webcam.USBClassDriver.UVCDriver

Target Feature: Device.Streaming.Webcam.USBClassDriver

Title: UVC-capable device must comply with USB Video Class specifications and work with the Microsoft UVC driver

Applicable OS Versions:

- Windows 8 Client x64
- Windows 8 Client x86
- Windows 7 Client x64
- Windows 7 Client x86
- Windows 8 Client ARM

Description:

Devices that are designed to comply with the USB Video Class (UVC) specifications must work with the Microsoft UVC driver. These devices also must comply with the requirements set in the UVC specifications. At a minimum, all mandatory properties and commands must be implemented. All implemented commands must comply with the specifications.

Exceptions: Not Specified

Business Justification:

Security-enhanced, robust, high-performance class drivers allow IHVs to spend less time writing and debugging drivers and more creating better devices for their customers. Improves the overall quality of multimedia device drivers.

Scenarios: Not Specified

Success Metric: Not Specified

Enforcement Date: Not Specified

Comments:

STREAM-0008